

The Siemens logo is displayed in a white rectangular box in the upper left corner. The word "SIEMENS" is written in a bold, teal, sans-serif font. The background of the entire slide is a low-angle photograph of a modern glass skyscraper reaching towards a blue sky with scattered white clouds.

SIEMENS

Teamcenter 11.3 Client Customization

PLM00075 - 11.3

Contents

Getting started with client customization

Introduction to client customization	1-1
Before you begin	1-1
Client interfaces	1-3
Rich client interface layout	1-3
Thin client interface layout	1-7
Basic concepts about client customization	1-8
The difference between client and server customization	1-8
Siemens PLM Software customization support	1-8
Syntax definitions	1-9
Basic tasks for client customization	1-10

Enterprise-wide configuration

Methods of enterprise-wide configuration	2-1
Using the Business Modeler IDE	2-1
Suppressing menu commands	2-1
Using style sheets	2-2
Introduction to style sheets	2-2
Types of style sheets	2-3
Search for style sheets	2-13
Create a custom style sheet based on an existing style sheet	2-15
Create a custom style sheet by importing an XML file	2-18
Registering style sheets	2-19
Sample customizations using style sheets	2-23
Localizing style sheets	2-36
Style sheet definition	2-37

Rich client customization

Introduction to rich client customization	3-1
Basic concepts about rich client customization	3-1
What are perspectives and views?	3-1
Understanding the Eclipse rich client platform framework	3-1
Adding menus and toolbars	3-2
Introduction to SWT	3-4
Basic tasks for rich client customizations	3-8
Process for creating rich client customizations	3-8
Enable rich client customization	3-9
Export plug-ins	3-11
Ensure your customizations appear	3-13
Distributing rich client customizations	3-14
Sample rich client customizations	3-27
Commonly used rich client customizations	3-27

Infrequently used rich client customizations	3-114
Advanced rich client customizations	3-200
Customizing Command Suppression	3-200
Displaying files in the viewer	3-204
Customizing tabs	3-205
Customizing the rich client to perform additional validations on a file	3-208
Creating pre-actions and post-actions in Resource Manager and Classification	3-211
Customize the Launch Pad	3-220
Tips for rich client customization	3-222
Localization of rich client customizations	3-222
Updating your rich client customizations from previous versions	3-223
Hide perspectives	3-223
Adding a third-party JAR file to your plug-in	3-223
Troubleshooting rich client customization	3-224
Common problems in rich client customization	3-224
Rich client debugging	3-225
Enabling client-side logging	3-228
Listener leaks	3-232
Rich client customization reference	3-233
Teamcenter extension points	3-233
Command line options for rich client startup	3-237
Coding standards	3-241
Property beans	3-244
Rich client Javadoc	3-262
Common Teamcenter command IDs	3-311
Plug-in locations of perspectives	3-312
Application Integration Framework (AIF)	3-314

Thin client customization

Introduction to thin client customization	4-1
Customizing navigation pane and menu entries	4-1
Modifying browser behavior	4-2
Generating a thin client page	4-3
Process for generating a thin client page	4-3
Choosing the menu command and displaying the dialog box	4-4
Submitting the data and submitting the Web request	4-6
Processing on the Teamcenter server	4-7
Receiving the response and displaying feedback	4-8
Basic thin client customization	4-8
Thin client customization recommendations	4-8
Top-level pages	4-9
Directory structure	4-10
Deploying thin client customization changes	4-11
Thin client preferences	4-12
Cascading style sheets (CSS)	4-13
Thin client menu system	4-15
Adding and modifying business object icons in the thin client	4-19
Configuration settings in the user interface	4-19

Customizing forms for the thin client	4-21
Methods of customizing forms for the thin client	4-21
Altering form content	4-24
Thin client custom form override mechanism	4-24
Thin client custom form override example	4-28
Customize property names in the user interface	4-30
Customizing the thin client with TcScript	4-31
Write TcScript	4-31
TcScript values	4-34
TcScript operators	4-37
Accessing Teamcenter data with TcScript	4-39
TcScript Statements	4-42
TcScript error handling	4-44
TcScript helper functions	4-46

Glossary A-1

Tables

3-8. My Teamcenter File→New menu commands	3-311
3-9. My Teamcenter Edit menu commands	3-312
3-10. Plug-in locations of perspectives	3-312

1. Getting started with client customization

Introduction to client customization

Teamcenter provides several interfaces, including a rich client interface, a web-based thin client interface, the Active Workspace client, and Teamcenter Client for Microsoft Office.

The following clients have supported customization methods:

- **Rich client**
- **Thin client**
- Active Workspace client

You can also perform server-side enterprise-wide customizations that affect all clients. Examples of enterprise-wide customizations are:

- **Business Modeler IDE**
- **XML Rendering Templates (XRT)**
- **Command suppression**

Before you begin

Prerequisites

- **Rich client**
You must have the following installed on your machine to customize the rich client:
 - The rich client itself
For more information, see the *Linux Client Installation* or the *Windows Client Installation*.
 - The Java Development Kit (JDK) 7 or later
You can download the JDK from this Web site:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

Set the **JRE_HOME** environment variable to point to the location of the JRE. Set the **JAVA_HOME** and **JDK_HOME** environment variables to point to the location of the JDK.

Caution:

You must install the 32-bit version of Java if you want to install and run the 32-bit version of the rich client, or you must install the 64-bit version of Java if you want to install and run the 64-bit version of the rich client. You cannot run in mixed modes.

- The Eclipse 3.8 integrated development environment (IDE)
For more information, see this Web site:

<http://archive.eclipse.org/eclipse/downloads/drops/R-3.8-201206081200/>

- Thin client
To successfully customize the thin client, you should be familiar with the following:

- Integration Toolkit (ITK) programming
- HTTP and Web applications in general
- XML, HTML, and the document object models (DOMs)
- JavaScript
- Asynchronous JavaScript and XML (AJAX)
- Cascading style sheets (CSS)
- Deploying in a Web application server environment

Enable client customization

- Rich client
To enable rich client customization, you must install Eclipse and configure it to run the rich client.
For more information, see *Process for enabling rich client customization*.

- Thin client
You do not have to do anything else to enable thin client customization.

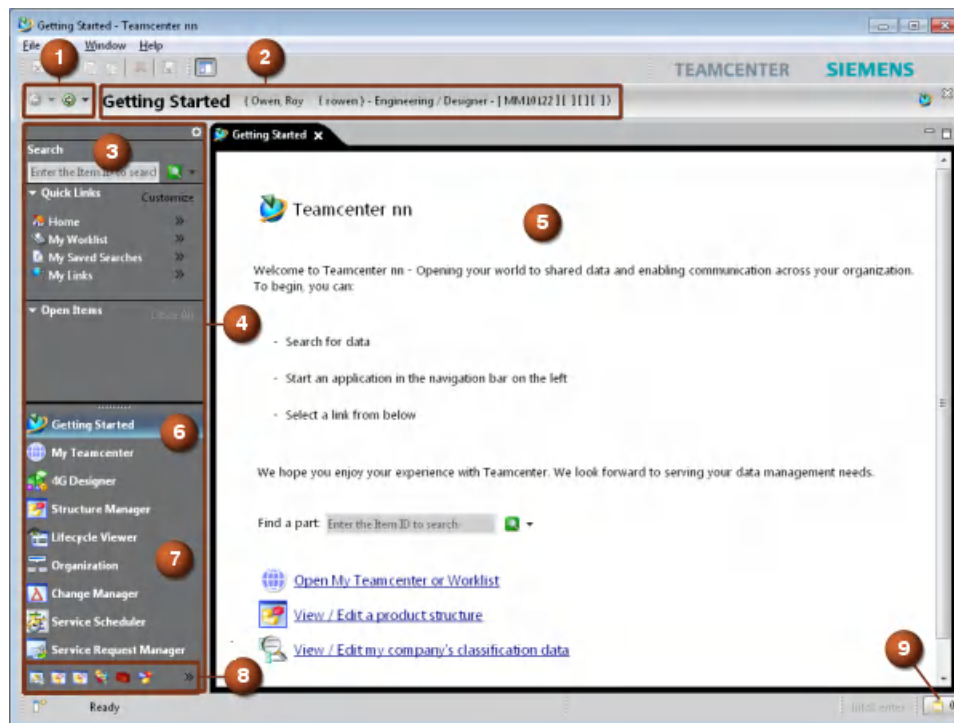
Configure client customization

- Rich client
Once you install the prerequisites and set up the Eclipse IDE, no additional steps are required to configure rich client customization.
- Thin client
No additional steps are required to configure thin client customization.

Client interfaces

Rich client interface layout

The rich client interface has a standard menu bar and toolbar with options that vary depending on the currently active application perspective. You can place the cursor over a rich client toolbar button to display a tooltip description.



- 1 **Back and Forward buttons** The **Back** and **Forward** buttons allow you to move between loaded Teamcenter applications. The small arrows next to the buttons let you select from the list of currently loaded applications.
- 2 **Application banner** The application banner shows the name of the active application and lists the current user and role. You can double-click the user and role to display the **User Settings** dialog box in which you can change your current role if multiple roles are available to your user.
- 3 **Search box** The **Search** box provides predefined quick searches using dataset, item ID, item name, keyword search, and advanced search features.
- 4 **Navigation pane** The navigation pane provides quick access to the data you use most. In addition to finding, organizing, and accessing your data, you can configure the display of the Teamcenter

perspective buttons in the navigation pane to display only those perspectives that you use regularly to perform your tasks.

Click the reorder button  above the **Search** box to display the **Navigation Section Ordering** dialog box that lets you hide sections or change the order of sections in navigation pane.

- | | | |
|---|------------------|---|
| 5 | Application pane | The application pane displays the application perspectives that are open in your Teamcenter session. By default, the Getting Started application perspective displays a single Getting Started view. |
|---|------------------|---|

Note:

Application perspectives are composed of views that can be moved elsewhere in the Teamcenter window or can be dragged to the desktop. Such detached views remain connected to Teamcenter and continue to function in concert with other views.

- | | | |
|---|--|--|
| 6 | Getting Started
application button | Provides access to the Getting Started application. |
| 7 | Application
buttons | Application buttons provide access to your most frequently used Teamcenter application perspectives. |
| 8 | Application button
bar | The application button bar provides access to Teamcenter application buttons that do not fit in the application button area of the navigation pane. |
| 9 | Clipboard button | The clipboard button displays the Clipboard Contents dialog box, which contains references to objects that have been cut or copied from your workspace. The total number of objects on the clipboard is displayed to the right of the symbol. |
-

Note:

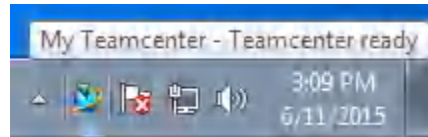
The status message area on the lower-left side of the Teamcenter window is available to any application to indicate whether the client is ready for input or is working, so the user interface may not be accepting input at that time.

The status messages on the lower-right side of the Teamcenter window indicate the status or activity of background threads for any potentially long operations.

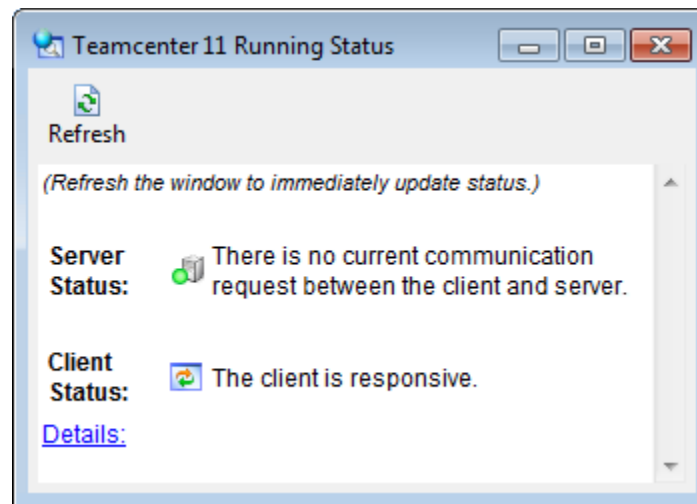
- The default message is **Working**, but other messages, such as **Loading children**, can be supplied by the application running the background thread.
- Information shown in this area can often be observed in more detail in the **Progress** view.
- In the **Progress** view, some operation messages have a **Cancel** button **X** that can be useful in cases where an operation was started but is not wanted at that time, such as when a user is loading thousands of nodes but determines that operation is not needed.

Note:

On Windows systems, operational status for the rich client interface and the Teamcenter server is provided by the Teamcenter icon  in the system tray.



To display the running status dialog box, click the Teamcenter icon in the system tray .



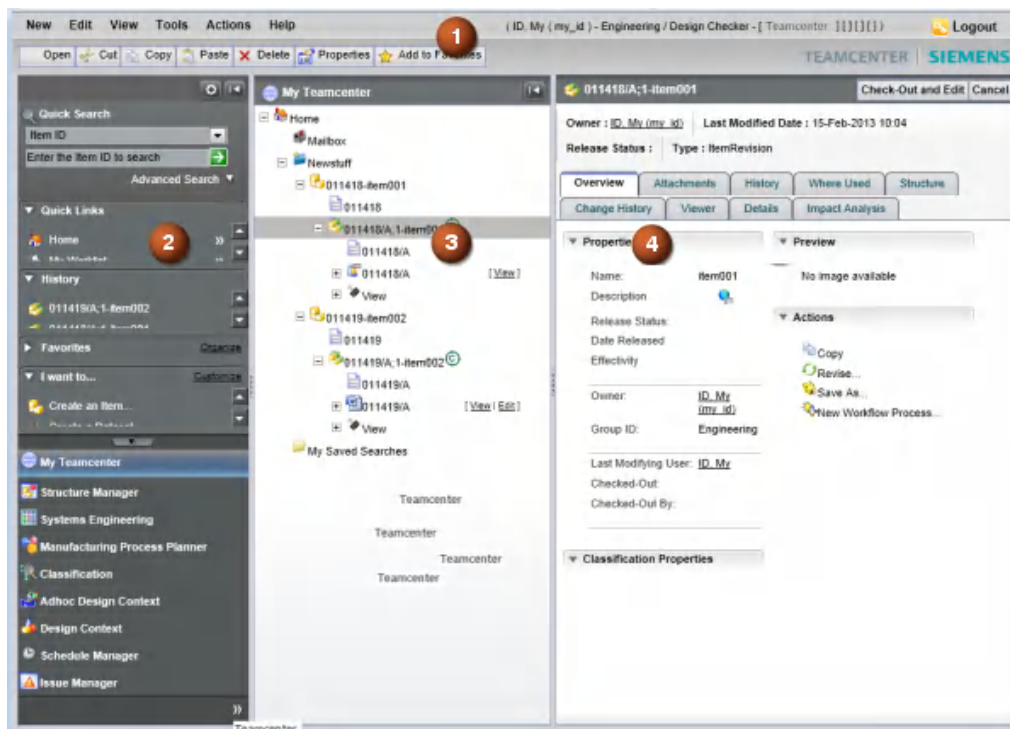
The server and user interface condition symbols show the current status of the rich client interface and the Teamcenter server.

- The server status indicates the state of the Teamcenter server:
 -  The server is ready, but there is no current communication between the client and the server.
 -  The server is busy.
 -  The server is idle.
 -  The server is disconnected.
- The client status indicates the condition of the rich client:
 -  The user interface is responsive.
 -  The user interface is unresponsive.

Note:

When used, the **Data Share Manager** icon is also displayed in the system tray. The Data Share Manager is a separate executable with its own user interface that lets you view large file uploads and downloads, and manage them by pausing, resuming, or canceling the processes.

Thin client interface layout



The thin client interface has a standard menu bar and toolbar with options that vary depending on the currently active application. You can place the cursor over a thin client toolbar button to display a tooltip description.

- | | | |
|---|-------------------------|--|
| 1 | Menu bar and button bar | Provides access to menu and button commands and lists the current user name, group, role, revision rule, and server. |
| 2 | Navigation pane | Provides search functionality, link groups, and application buttons.

You can expand and collapse the entire pane and pane groups, select what is displayed in primary and secondary areas, and configure the display of applications. |
| 3 | Component pane | Displays component hierarchies in a tree view. You can resize this pane to be wider or narrower. |
| 4 | Data pane | The data pane displays tabbed information. |

- Depending on the type of object selected and the configuration specified in your installation's XRT style sheet, you may see tabs such as **Overview**, **Details**, **Related Datasets**, **Attachments**, **History**, **Change History**, **Where Used**, **Structure**, **Impact Analysis**, **Available Revisions**, and **Viewer**.
- Groups of objects within a tab, called *object sets*, can be configured in the XRT style sheet. Each group of information can be configured to display in table, list, or thumbnail. Each group can also be supported by action command buttons such as **Cut** and **Copy**.
- You can configure the columns in the **Details** tab.

The default layout for thin client applications has a header bar containing menus, command buttons, and session information above three panes, arranged vertically side-by-side. Some thin client applications arrange the component pane and the data pane horizontally.

The layout configuration for a thin client application can only be changed by customization.

Note:

If instant messaging is configured, the data pane may also display the current Microsoft Office Communicator status of the owning and last modified users.

Basic concepts about client customization

The difference between client and server customization

Client customization means changing how the client user interface looks as well as how the client behaves. Teamcenter is based on a client-server architecture. Both the client and server layers can be customized. The *client* is the user interface (UI) layer and is built and customized using the Java language (rich client) or JavaScript (thin client) and other methods. The *server* layer can be customized using the Integration Toolkit (ITK) and the C++ programming language.

For information about customizing the server layer, see the [Server Customization](#).

Siemens PLM Software customization support

Siemens PLM Software is committed to maintaining compatibility between Teamcenter product releases. If you customize functions and methods using published APIs and documented extension points, be assured that the next successive release will honor these interfaces. On occasion, it may become necessary to make behaviors more usable or to provide better integrity. Our policy is to notify customers at the time of the release *prior* to the one that contains a published interface behavior change.

Siemens PLM Software does not support code extensions that use unpublished and undocumented APIs or extension points. All APIs and other extension points are unpublished unless documented in the official set of technical manuals and help files issued by Siemens PLM Software Technical Communications.

The Teamcenter license agreements prohibit reverse engineering, including: decompiling Teamcenter object code or bytecode to derive any form of the original source code; the inspection of header files; and the examination of configuration files, database tables, or other artifacts of implementation. Siemens PLM Software does not support code extensions made using source code created from such reverse engineering.

If you have a comment or would like to request additional extensibility, contact the Siemens PLM Software customer support representatives at GTAC for further assistance.

Syntax definitions

This manual uses a set of conventions to define the syntax of Teamcenter commands, functions, and properties. Following is a sample syntax format:

```
harvester_jt.pl [bookmark-file-name bookmark-file-name ...]  
                [directory-name directory-name ...]
```

The conventions are:

- Bold** Bold text represents words and symbols you must type exactly as shown.
In the preceding example, you type **harvester_jt.pl** exactly as shown.
- Italic* Italic text represents values that you supply.
In the preceding example, you supply values for *bookmark-file-name* and *directory-name*.
- text-text* A hyphen separates two words that describe a single value.
In the preceding example, *bookmark-file-name* is a single value.
- [] Brackets represent optional elements.
- ... An ellipsis indicates that you can repeat the preceding element.

Following are examples of correct syntax for the **harvester_jt.pl** command:

```
harvester_jt.pl  
harvester_jt.pl assembly123.bkm  
harvester_jt.pl assembly123.bkm assembly124.bkm assembly125.bkm  
harvester_jt.pl AssemblyBookmarks
```


Basic tasks for client customization

Following are some of the basic tasks you perform during client customizations:

- Change the layout of pages in both the rich client and thin client using style sheets.
- Set up your environment to perform rich client-only customizations.
- Override Teamcenter commands.
- Create custom forms on the rich client.
- Add a command to a menu on the rich client.

2. Enterprise-wide configuration

Methods of enterprise-wide configuration

You can use the following tools to apply enterprise-wide changes for all clients.

- Business Modeler IDE
- Style sheets
- Command suppression

Using the Business Modeler IDE

The Business Modeler IDE (Integrated Development Environment) is a tool for configuring and extending the data model of your Teamcenter installation. The data model objects define the objects and rules used in Teamcenter.

Administrators and business analysts use the IDE to:

- Create new data model objects such as:
 - Business objects (for example, items and datasets)
 - Properties
 - Lists of values (LOVs)
 - Rules
- Perform C++ customizations.

Suppressing menu commands

Carefully select which menu commands to suppress so that end users are not presented with unnecessary menu commands.

Note:

If all commands are removed from a menu, the menu itself is deleted. If commands are suppressed, redundant dividers are removed.

- Suppressing menu commands in both the thin client and rich client

To suppress menu commands in the thin client, Siemens PLM Software recommends that you use the Command Suppression application in the rich client. This hides the commands in both the thin client and rich client. The **WEB_menu_entry_suppressions** Web-specific preference must have no values to work correctly. Some commands are unique to the thin client and do not appear in the standard menu list in the Command Suppression application. These commands can be found under the **dhtml** heading.

- Suppressing menu commands in the thin client only

To suppress thin client menu commands independently from the rich client, set the **WEB_menu_entry_suppressions** preference, which ignores the command suppression set in the rich client. This preference is set at the site, group, or role level. The values are a list of menu keys found in the ***menu.xml** files located in the *staging_location\webapp_root\teamcenter\dhtml\common\intl\language* directory. For example:

```
WEB_menu_entry_suppressions=
whereClassifiedAction
openInUGAction
```

This example suppresses the **View→Where Classified** and **View→NX/Manager** menu commands and ignores the command suppressions set in the rich client.

Using style sheets

Introduction to style sheets

Style sheets are the easiest way to codelessly customize the rich client and thin client. You can customize the display by editing the style sheet. The advantages are:

- You can customize using configuration instead of coding.
- The customization affects both the rich client and thin client.

You can use style sheets to change the layout of pages such as forms, the **Properties** dialog box, the **Summary** view, and the creation wizard dialog boxes.

Style sheets are XML documents stored in **XMLRenderingStylesheet** datasets. This gives more control to sites regarding how dialog boxes are displayed. The XML code allows sites to define a subset of properties to display, the display order, the user interface rendering components to be used, and more.

Sites can use XML code to customize not only forms but also individual fields in the forms. To see all the available style sheets, search for all the **XMLRenderingStylesheet** datasets.

When a style sheet is registered for a specific object type or form, it defines the display of the object or form properties. Registration information is stored in the `<type_name>.RENDERING` and `<type_name>.FORMRENDERING` preferences.

Note:

If you change style sheets, clear the cache to see the style sheet changes in the clients. For example, to see the changes in the thin client, clear the Web browser cache. To see the changes in the rich client, exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

If using the rich client to edit or register XML rendering templates (stylesheets), you must have DBA privileges.

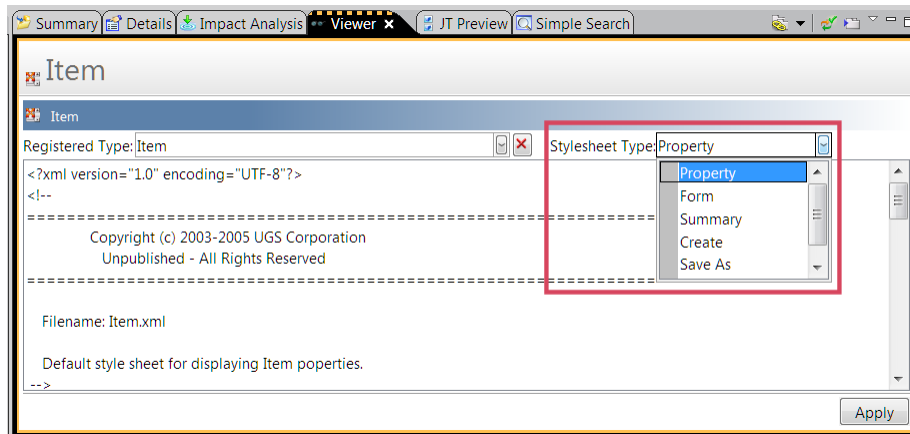
Types of style sheets

Overview of style sheet types

Following are the available types of style sheets:

- **Property**
Defines the layout of the **Properties** dialog box.
- **Form**
Defines the layout of forms, such as the **Item Master** form or the **Item RevisionMaster** form.
- **Summary**
Defines the layout of the **Summary** tab.
- **Create**
Defines the layout of dialog boxes used in the creation wizard when you choose **File→New→Other** and some portions of dialog boxes when you choose **File→New→object**.
- **Save As**
Defines the layout of **Save As** dialog boxes when you select an object and choose **File→Save As**.

These types are set on a style sheet using the **Stylesheet Type** box in the rich client.

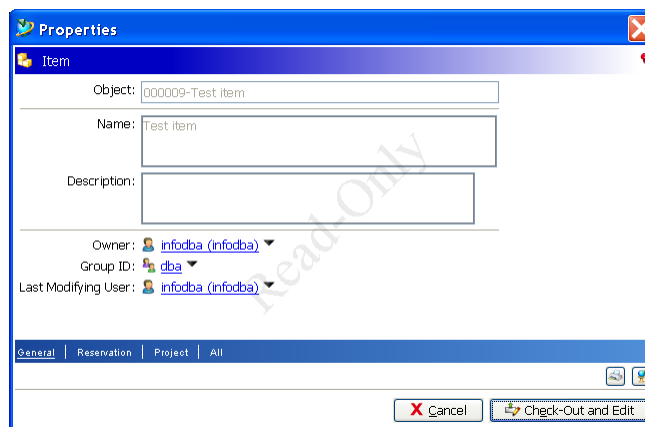


Types of available style sheets

Property style sheet

The **Property** style sheet type defines the layout of the **Properties** dialog box.

To view the **Properties** dialog box in the rich client, right-click an object and choose **View Properties**.



Properties dialog box in the rich client

To view the **Properties** dialog box in the thin client, click the arrow on an object and choose **Properties**.

Properties dialog box in the thin client

To set a style sheet to the **Property** type, in the rich client open the style sheet dataset in the **Viewer** tab and select **Property** in the **Stylesheet Type** box.

Property style sheet

Notice how this **Item** style sheet defines the layout of the **Properties** dialog box for the selected item.

Form style sheet

The **Form** style sheet defines the layout of forms, such as the **Item Master** form or the **Item RevisionMaster** form.

To view a form in the rich client, select an instance of a form and click the **Viewer** tab.

The screenshot shows a software window titled '000009' with a tabbed interface. The 'Viewer' tab is active, displaying the 'Item Master' form. The form contains several input fields: 'Project ID', 'Previous ID', 'Serial Number', 'Item Comment', 'User Data 1', 'User Data 2', and 'User Data 3'. A 'General' tab is visible at the bottom. At the bottom right of the window, there are two buttons: 'Check-Out and Edit' and 'Clear'. A large 'Read-Only' watermark is overlaid diagonally across the form fields.

Form in the rich client

To view a form in the thin client, click the arrow on a form object and choose **Open**.

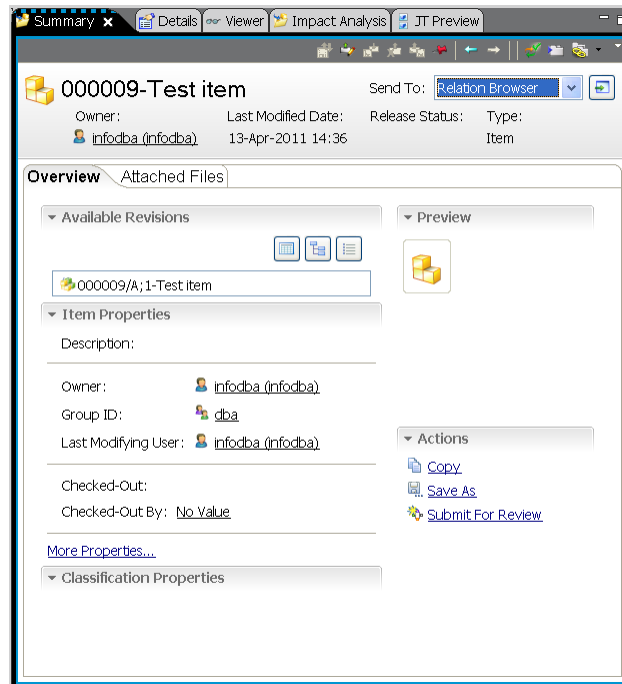
The screenshot shows a software window titled '000018'. It displays a form with the same fields as the rich client version: 'Project ID', 'Previous ID', 'Serial Number', 'Item Comment', 'User Data 1', 'User Data 2', and 'User Data 3'. The fields are presented as a list of labels followed by input lines. At the bottom left, there are two buttons: 'Check-Out and Edit' and 'Cancel'.

Form in the thin client

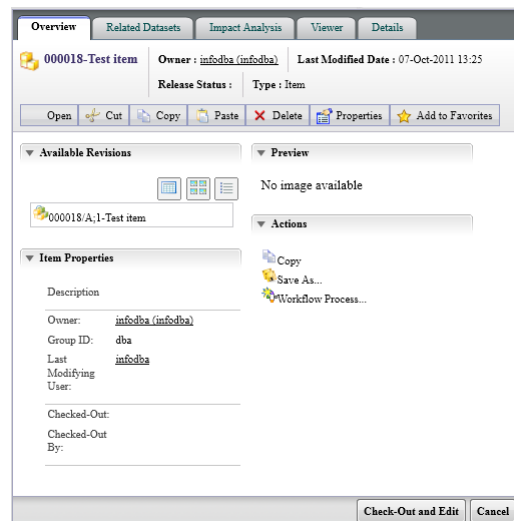
To set a style sheet to the **Form** type, in the rich client, open the style sheet dataset in the **Viewer** tab and select **Form** in the **Stylesheet Type** box.

Summary style sheet

The **Summary** style sheet type defines the layout of the **Summary** tab in the rich client and the **Overview** tab in the thin client.

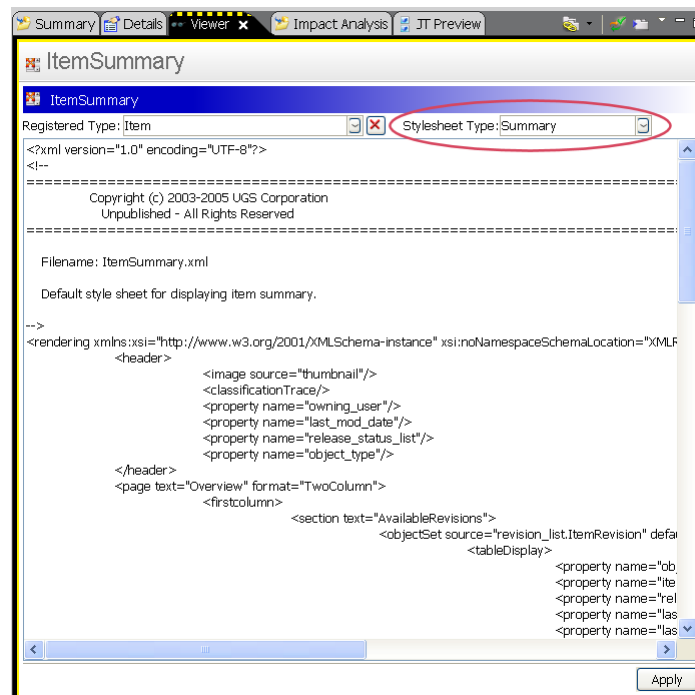


Summary tab in the rich client



Overview tab in the thin client

To set a style sheet to the **Summary** type, in the rich client, open the style sheet dataset in the **Viewer** tab and select **Summary** in the **Stylesheet Type** box.



Summary style sheet

Notice how this **ItemSummary** style sheet defines the layout of the **Summary** tab in the rich client and the **Overview** tab in the thin client.

Base summary style sheets

In Teamcenter 10, a set of revision summary style sheets are updated, including the style sheets for **ItemRevision** and **DocumentRevision** business objects. These updated style sheets are named with a **Base** prefix on the style sheet file names, for example, **BaseItemRevSummary**, to indicate they are the primary style sheets to use for those business object types.

The previous versions of the summary style sheets remain in the system so that you can still use them if you want. To change back to the previous version of the style sheet, change the registration of the business object type to point to the older style sheet using the `<type-name>.SUMMARYRENDERING=<dataset-name(dataset-uid)>` and `<dataset-name(dataset-uid)>.SUMMARY_REGISTEREDTO=<type-name>` preferences.

The following table shows the new and old summary style sheet file names.

Object viewed by end users	Business object type	New style sheet	Old style sheet
Change notice revision	ChangeNoticeRevision	BaseChangeNoticeRevisionSummary	ChangeNoticeRevisionSummary
Change request revision	ChangeRequestRevision	BaseChangeRequestRevisionSummary	ChangeRequestRevisionSummary
Commercial part revision	CommercialPart Revision	BaseCommercialPartRevSummary	ItemRevSummary (inherited from ItemRevision)
Deviation request revision	Cm0DevRqstRevision	BaseCm0DevRqstRevisionSummary	Cm0DevRqstRevisionSummary
Document revision	DocumentRevision	BaseDocumentRevSummary	ItemRevSummary (inherited from ItemRevision)
EDA revision	EDA Revision	BaseEDARevSummary	ItemRevSummary (inherited from ItemRevision)
EDA component revision	EDAComp Revision	BaseEDACompRevSummary	ItemRevSummary (inherited from ItemRevision)
EDA schematic revision	EDASchem Revision	BaseEDASchemRevSummary	ItemRevSummary (inherited from ItemRevision)
EPM workflow task	EPMTask	BaseEPMTaskSummary	EPMTaskSummary
Item revision	ItemRevision	BaseItemRevSummary	ItemRevSummary
Problem report revision	ProblemReportRevision	BaseProblemReportRevisionSummary	ProblemReportRevisionSummary
Requirement revision	Requirement Revision	BaseRequirementRevisionSummary	RequirementRevisionSummary
Vendor	Vendor	BaseVendorSummary	ItemRevSummary (inherited from ItemRevision)
Vendor revision	VendorRevision	BaseVendorRevisionSummary	ItemRevSummary (inherited from ItemRevision)
Vendor part	ManufacturerPart	BaseManufacturerPartSummary	ManufacturerPartSummary
Vendor part revision	ManufacturerPart Revision	BaseManufacturerPartRevSummary	ManufacturerPartRevSummary

Create style sheet

The **Create** style sheet type defines the layout of dialog boxes used in the creation wizard.

To view creation dialog boxes in the rich client, choose **File→New→Other**.

Note:

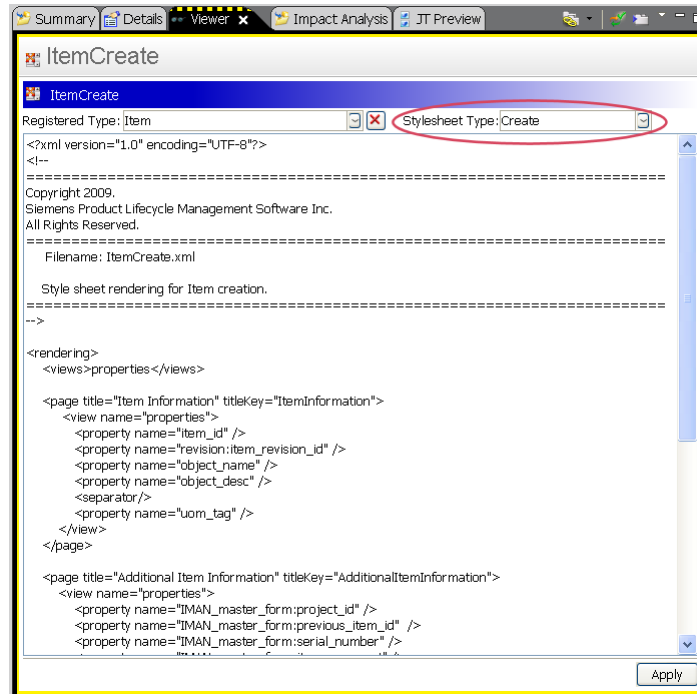
Only some portions of dialog boxes are defined with the create style sheet when you choose **File→New→object**.

Creation dialog box in the rich client

To view creation dialog boxes in the thin client, choose **New→object**.

Creation dialog box in the thin client

To set a style sheet to the **Create** type, in the rich client, open the style sheet dataset in the **Viewer** tab and select **Create** in the **Stylesheet Type** box.



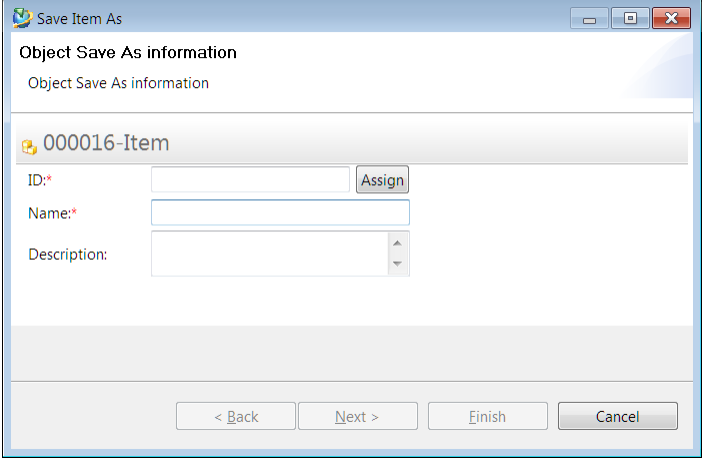
Create style sheet

Notice how this **ItemCreate** style sheet defines the layout of the creation dialog boxes for the **Item** business object.

Save As style sheet

The **Save As** style sheet type defines the layout of dialog boxes used in the Save As wizard.

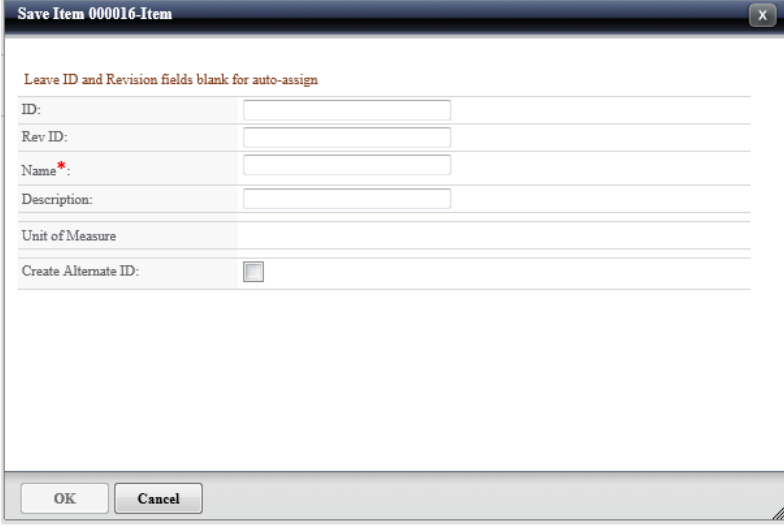
To view **Save As** dialog boxes in the rich client, select an object and choose **File→Save As**.



The image shows a 'Save Item As' dialog box in a rich client. The title bar says 'Save Item As'. Below the title bar, it says 'Object Save As information' and 'Object Save As information'. There is a section for '000016-Item' with an icon. Below this, there are three input fields: 'ID:*' with an 'Assign' button, 'Name:*', and 'Description:'. At the bottom, there are four buttons: '< Back', 'Next >', 'Finish', and 'Cancel'.

Save As dialog box in the rich client

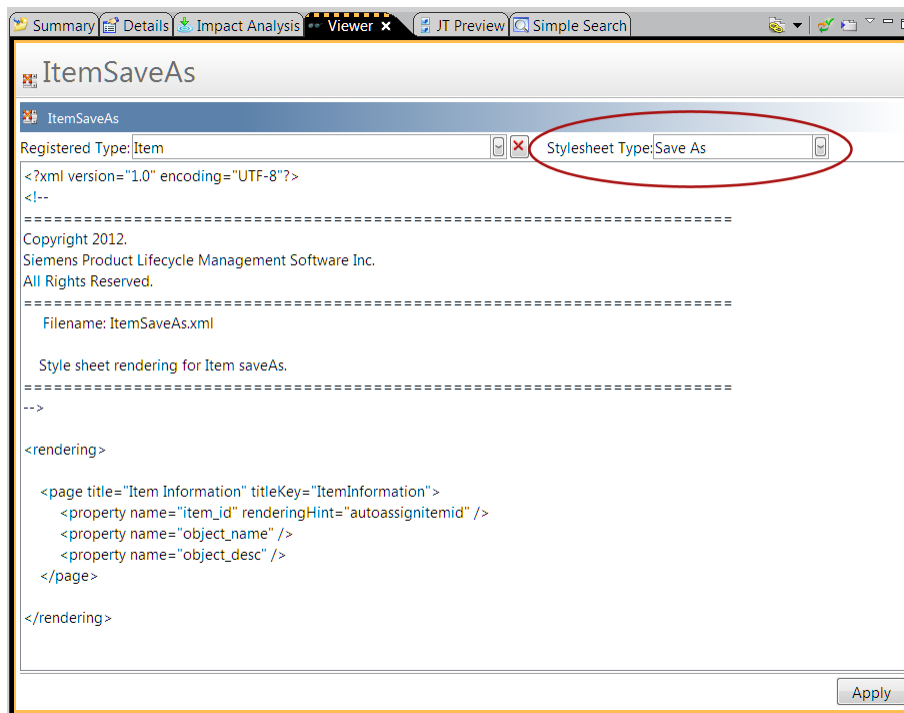
To view **Save As** dialog boxes in the thin client, select an object and choose **Edit**→**Save As**.



The image shows a 'Save Item 000016-Item' dialog box in a thin client. The title bar says 'Save Item 000016-Item'. Below the title bar, it says 'Leave ID and Revision fields blank for auto-assign'. There are several input fields: 'ID:', 'Rev ID:', 'Name *', 'Description:', 'Unit of Measure', and 'Create Alternate ID:' with a checkbox. At the bottom, there are two buttons: 'OK' and 'Cancel'.

Save As dialog box in the thin client

To set a style sheet to the **Save As** type, in the rich client, open the style sheet dataset in the **Viewer** tab and select **Save As** in the **Stylesheet Type** box.



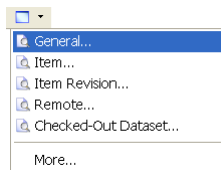
Save As style sheet

Note how this **ItemSaveAs** style sheet defines the layout of the **Save As** dialog boxes for the **Item** business object.

Search for style sheets

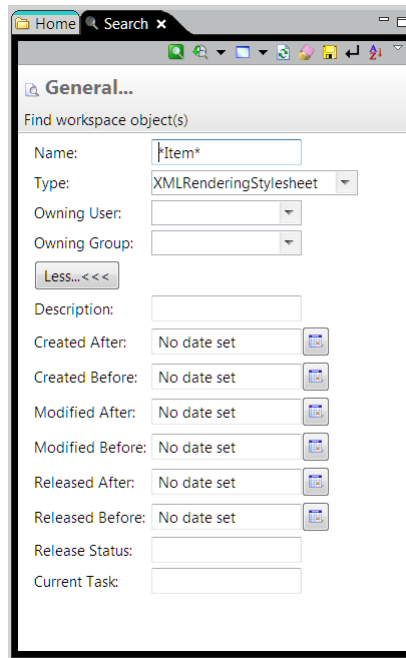
To find style sheets in the rich client, search for **XMLRenderingStylesheet** datasets.

1. Click the **Open Search View** button .
2. Click the arrow on the **Select a Search** button and choose **General**.



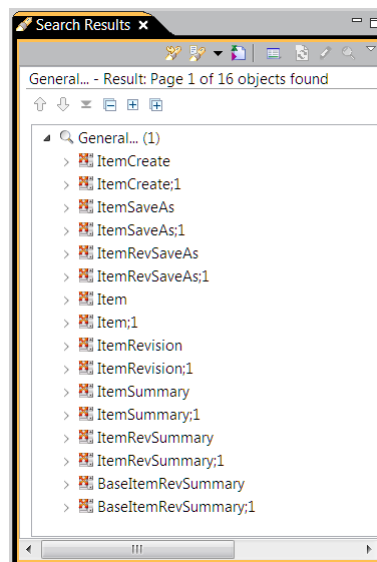
Starting the search for style sheets

3. In the **Type** box, type **XMLRenderingStylesheet**.
If you are looking for style sheets for a particular kind of object, enter a string in the **Name** box to look for those kinds of style sheets. For example, if you want to find all style sheets for items or item revisions, type ***Item*** in the **Name** box.



Searching for XMLRenderingStylesheet datasets

4. Press the Enter key or click the **Execute the Search** button . The results are displayed in the **Search Results** view.

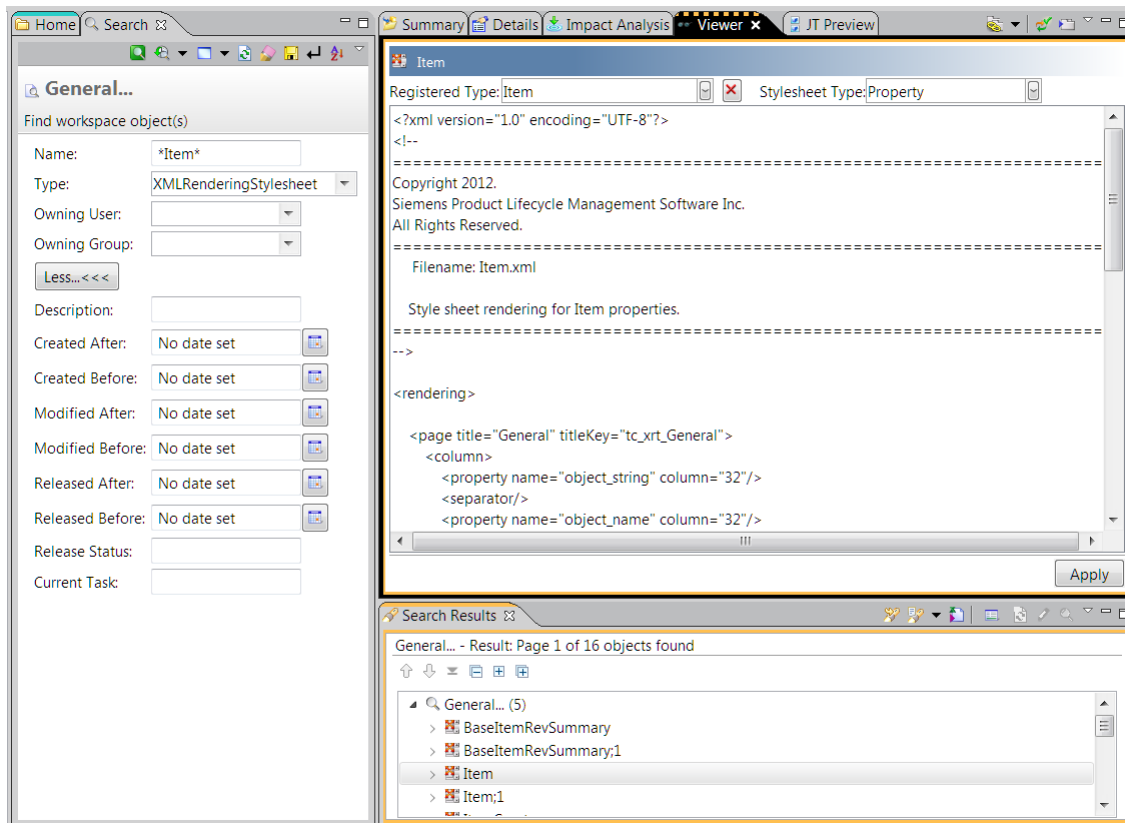


Viewing the search results for XMLRenderingStylesheet datasets

5. In the **Search Results** tab, select the style sheet you want to view. Click the **Viewer** tab to see the style sheet.

Note:

Do not double-click a style sheet (**XMLRenderingStylesheet** dataset file) in an attempt to open it. If you do, you receive an **Unable to open** error message. Instead, select the style sheet and view its contents in the **Viewer** view.



Viewing the style sheet contents in the rich client

Create a custom style sheet based on an existing style sheet

You can create your own custom style sheet.

For example, you create a custom business object in the Business Modeler IDE and install it to the rich client, and you want to create a unique style sheet to display the custom properties. To create the style sheet, in the rich client, search for **XMLRenderingStylesheet** datasets, save one as your own custom style sheet dataset, and then register the custom style sheet for use with the custom business object.

1. In the rich client, search for a style sheet you can base your new style sheet on.
2. In the **Search Results** view, select the style sheet you want to use, choose **File**→**Save As**, and rename it. For example, if you want to create a style sheet to be used with a custom **A5_MyItem** business object, you could name the style sheet **A5_MyItem**.

The new style sheet dataset is saved in your **Newstuff** folder in the **Home** view and is still displayed in the **Viewer** tab.

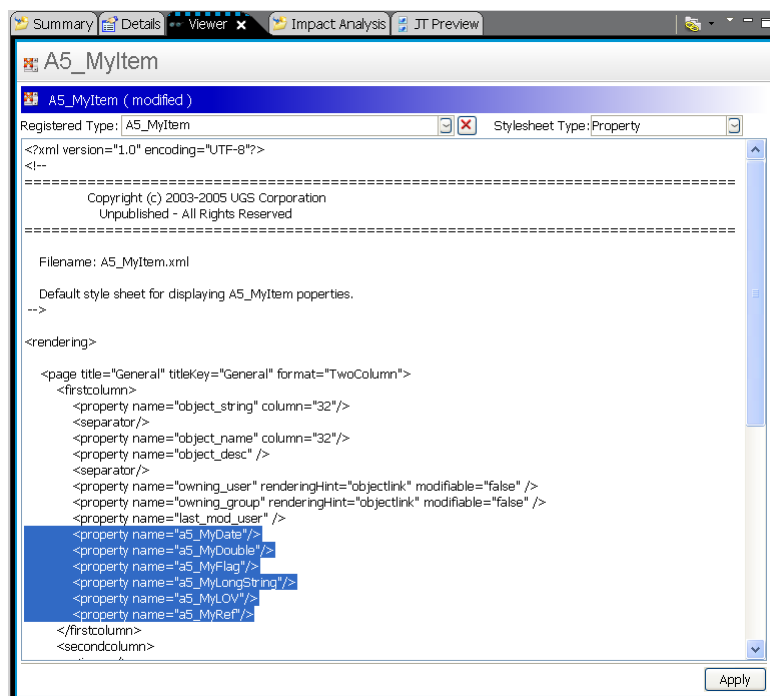
3. Edit the style sheet.
 - a. Change the named reference for the file by selecting the style sheet dataset and choosing **View→Named References**. In the **Name** column in the **Named References** dialog box, change the old name of the file to the new save as file name.
 - b. In the **Viewer** tab, click the arrow in the **Registered Type** box and select the business object type you want to register it to. For example, if you have a custom **A5_MyItem** business object added to your server, select **A5_MyItem** from the list.
 - c. Edit the style sheet in the **Viewer** tab to include the elements you want displayed in the layout.
For example, if you want to display custom properties, add them where you want them to appear on the page, like this:

```
<page title="General" titleKey="tc_xrt_General">
  <column>
    <property name="object_string" column="32"/>
    <separator/>
    <property name="object_name" column="32"/>
    <property name="object_desc" />
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false" />
    <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
    <property name="last_mod_user" />
    <property name="a5_MyDate"/>
    <property name="a5_MyDouble"/>
    <property name="a5_MyFlag"/>
    <property name="a5_MyLongString"/>
    <property name="a5_MyLOV"/>
    <property name="a5_MyRef"/>
  </column>
  <column>
    <image/>
  </column>
</page>
```

- d. To change the style sheet type, click the arrow in the **Stylesheet Type** box. You can choose one of the following types:

Property
Form
Summary
Create

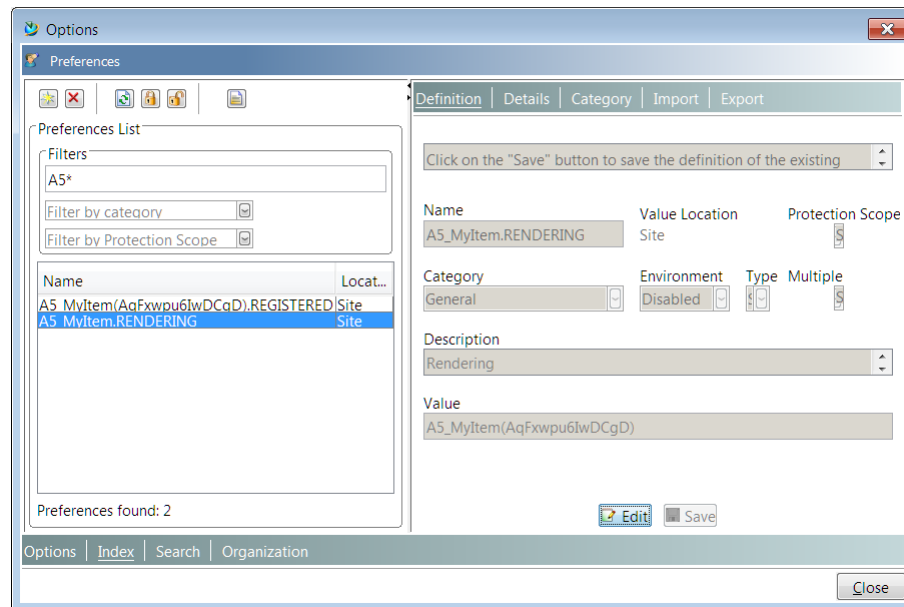
4. When you are done making changes, click the **Apply** button in the lower right corner of the view.



Create a custom style sheet

Because you used the **Registered Type** box on the **Viewer** tab to register the style sheet with a business object type, two new preferences are created (a **REGISTEREDTO** preference and a **RENDERING** preference). These preferences apply the style sheet to the business object so that the style sheet is displayed in the situation you set it for (for example, for display of the business object's property, summary, form, or create information).

- To see the two new preferences, choose **Edit**→**Options** and at the bottom of the dialog box, click **Search**.



Viewing the <dataset_name>.REGISTEREDTO and <type_name>.RENDERING preferences

Notice how in the **Current Values** box of the <type_name>.RENDERING preference there is a number in parentheses after the name of the business object. That is a GUID number that identifies that unique business object, and that GUID number is set in the <dataset_name(dataset-UID)>.REGISTEREDTO preference. This ensures that the style sheet is applied to the correct business object.

- To see the style sheet changes in the clients, clear the cache. For example, to see the changes in the thin client, clear the Web browser cache. To see the changes in the rich client, exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

To make the new style sheets available for quick loading to clients, run the **generate_client_meta_cache** utility to add the new style sheets to client cache, for example:

```
generate_client_meta_cache -u=infodba -p=infodba -g=dba update
stylesheets
```

Create a custom style sheet by importing an XML file

You can create a new style sheet by creating a new dataset and associating it with an XML file.

- In the rich client My Teamcenter application, create a new dataset of type **XMLRenderingStylesheet**.
- Import the XML file as a named reference to the dataset as follows:
 - Right-click the dataset in the navigation tree and choose **Named References**. Teamcenter displays the **Named References** dialog box.

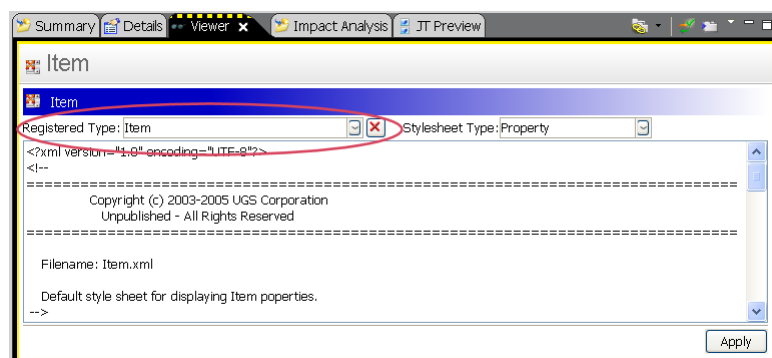
- b. Click the **Add** button.
Teamcenter displays the **Add File** dialog box.
 - c. Locate and select the XML file in your operating system directory and click **Add**.
Teamcenter displays the XML file in the **Named References** dialog box.
 - d. Click **Close**.
3. Register the style sheet by setting the **Form.REGISTEREDTO** preference.
4. To see the style sheet changes in the clients, clear the cache. For example, to see the changes in the thin client, clear the Web browser cache. To see the changes in the rich client, exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

Registering style sheets

Register a style sheet

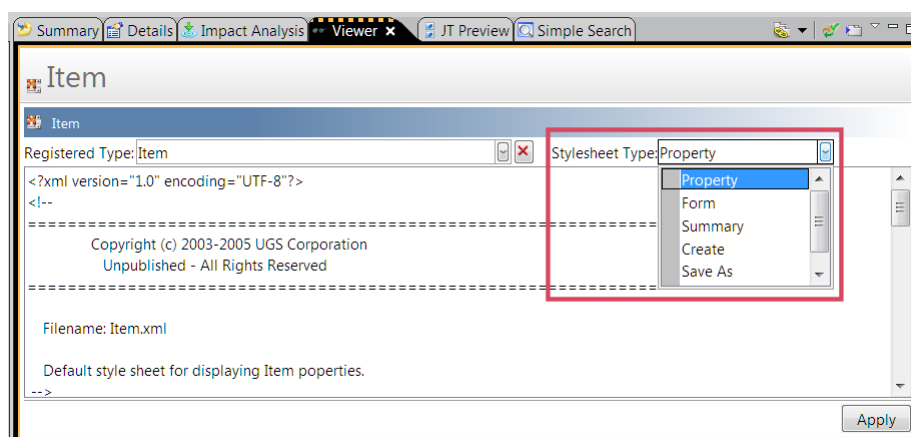
Each commonly used business object type (such as item, folder, and dataset) has style sheets that define the layout of its properties in the user interface.

1. To see the business object that a style sheet is registered to, in the rich client, first search for **XMLRenderingStylesheet** datasets.
2. Select the style sheet, and in the **Viewer** tab, see the business object listed in the **Registered Type** box on the style sheet



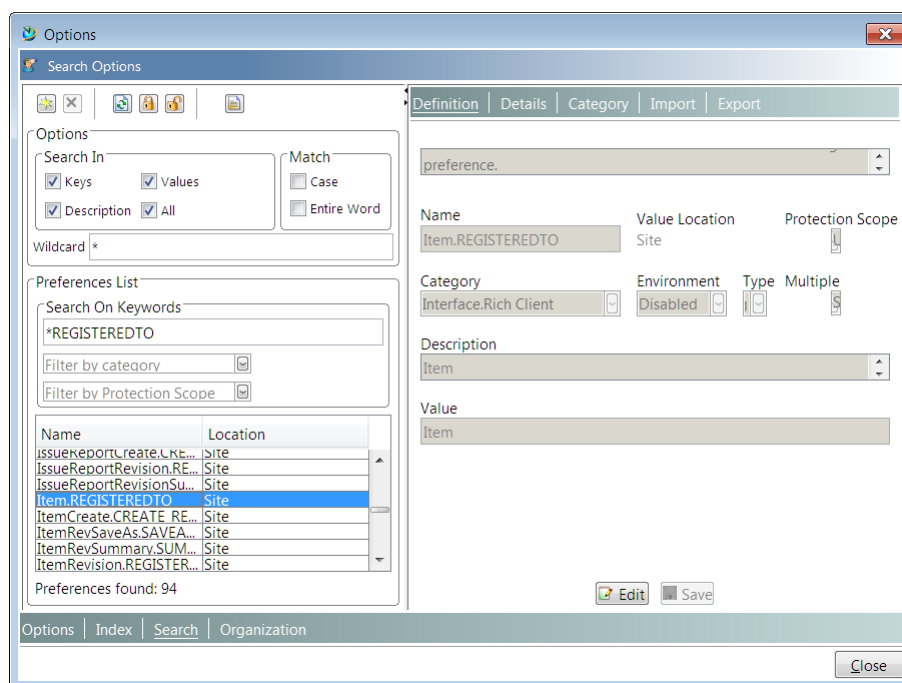
Viewing the business object type that the style sheet is registered to

3. To specify how the style sheet is to be used, select it in the **Stylesheet Type** box (for example, for properties display, form rendering, **Summary** view, or creation dialog boxes).



Viewing the style sheet type

4. When the **Registered Type** box in the **Viewer** tab is used to register a new property type style sheet with a business object type, a new **REGISTEREDTO** preference and a new **RENDERING** preference are created. These preferences apply the XML rendering style sheet dataset type to the business object type so that the style sheet is displayed in the situation you set it for (for example, for display of the business object's property, summary, form, or create information). To see preferences, choose **Edit**→**Options** and click **Search** at the bottom of the dialog box.



Viewing the REGISTEREDTO preferences

For example, the `<dataset_name(dataset-UID)>.REGISTEREDTO=<type-name>` preference indicates the business object type that an XML rendering dataset is registered to for use as a property type style sheet. Therefore, the `Item.REGISTEREDTO=Item` preference means that the

Item XML rendering style sheet dataset is registered as a property type style sheet to the **Item** business object type.

Similarly, the `<type_name>.RENDERING=<dataset_name(dataset-UID)>` preference indicates the XML rendering dataset used to render the properties shown on a particular business object type. Therefore, the **Item.RENDERING=Item** preference means that for **Item** business objects, the **Item** XML rendering style sheet dataset is used to render the properties in the user interface.

Note:

The UID (unique ID) is set in the preferences when custom XML rendering datasets or custom business objects are used. The UID ensures that the correct style sheet is applied to the correct business object.

When you register a style sheet, the preferences that are created depend on the type of style sheet that is registered:

- Property

```
<dataset_name(dataset-UID)>.REGISTEREDTO=<type-name>
<type_name>.RENDERING=<dataset_name(dataset-UID)>
```

- Form

```
<dataset-name(dataset-uid)>.FORM_REGISTEREDTO=<type-name>
<type-name>.FORMRENDERING=<dataset-name(dataset-uid)>
```

- Summary

```
<dataset-name(dataset-uid)>.SUMMARY_REGISTEREDTO=<type-name>
<type-name>.SUMMARYRENDERING=<dataset-name(dataset-uid)>
```

- Create

```
<dataset-name(dataset-uid)>.CREATE_REGISTEREDTO=<type-name>
<type-name>.CREATERENDERING=<dataset-name(dataset-uid)>
```

Note:

Use the **TC_CreateRenderingInheritTypeList** preference to define the children business object types to inherit the create rendering format.

- Save As

```
<dataset-name(dataset-uid)>.SAVEAS_REGISTEREDTO=<type-name>
<type-name>.SAVEASRENDERING=<dataset-name(dataset-uid)>
```

Note:

Use the **TC_SaveAsRenderingInheritTypeList** preference to define the children business object types to inherit the save as rendering format.

Using predefined style sheets

The Teamcenter installation provides several predefined style sheets that are registered to display the properties of the following objects:

EPM job
Folder
Form
Group
Group member
ImanFile
Item
Item revision
Reservation
Role
Site
Tool
User
Volume

The registration information is stored in the preference; each object type has two entries used to display regular properties, as follows:

- **<type_name>.RENDERING**
The value of this key is the dataset name or the dataset UID used to display properties of this type of object.
- **<dataset_name>(UID).REGISTEREDTO**
The value of this key is the type name for which this dataset is registered.

Therefore, the value for **<type_name>.RENDERING** is the dataset name/UID and the value for **<dataset_name>(UID).REGISTEREDTO** is the business object name.

The following keys are used to display form properties:

business-object-name.FORMRENDERING

dataset-name/uid.FORM_REGISTEREDTO

Verify the registration of forms in the rich client interface

1. In one of the rich client applications, choose **Options** from the **Edit** menu. Teamcenter displays the **Options** dialog box.
2. Click the **Search** link in the lower portion of the window. Teamcenter displays the **Preferences By Search** pane.
3. Type **form** in the **Search on Keywords** box. Teamcenter displays the preferences that begin with *form* in the **Preferences List**.
4. Click the **Form.REGISTEREDTO** entry in the **Preferences List**. Teamcenter displays the business objects to which the **Form** style sheet can be applied in the right pane of the window.

If no rendering registration is found for the currently selected type, the system searches the hierarchy to determine if a rendering is registered to any of the parent classes or parent types. If one is found, that rendering is honored.

Creating form preferences for new business objects

To create a form for a customized business object (in other words, one not provided by Teamcenter), you must create the following preferences:

- *new-business-object-name*.**RENDERING**
- *new-business-object-name*.**REGISTEREDTO**

The value of the preferences must be the name of the customized business object.

For example, to create a new business object called **MyItem**, you must create the following preferences and set the value of each to **MyItem**:

- **MyItem.RENDERING**
- **MyItem.REGISTEREDTO**

After you create the preferences, either modify a predefined style sheet or create a new style sheet.

Sample customizations using style sheets

Modify the Summary view using style sheets

You can modify what appears in the rich client **Summary** view and the thin client **Overview** panel by codelessly changing the rendering using Teamcenter style sheets.

The **Properties** dialog box and the item create panes in the New Business Object wizard also use style sheet rendering and can be similarly customized. You can also use this technique to create custom renderings for new custom business objects.

This example adds the **checked_out** property to the folder summary header rendering, and a **Contents** section to the folder summary page.

1. In the rich client, find all **XMLRenderingStylesheet** datasets using the search capability by removing all search criteria except for **Type** and setting it to **XMLRenderingStylesheet**.
2. Select the **FolderSummary** dataset and choose **File→Save As**. Type **MyFolderSummary** in the **Name** box.
3. Select **MyFolderSummary** and the **Viewer** pane.

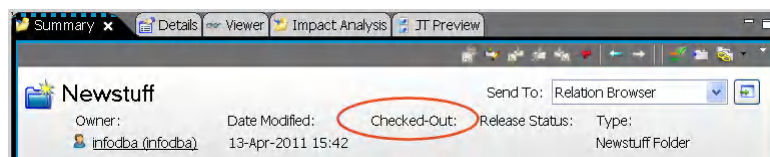
The XML style sheet is displayed in the viewer.

4. In the **Registered Type** box, select **Folder**, and in the **Stylesheet Type** box, select **Summary**.
5. Edit the **MyFolderSummary** style sheet to add the **checked_out** property to the header area. The following is an example of the modified header area:

```
<header>
  <image source="type"/>
  <property name="owning_user" />
  <property name="last_mod_date" />
  <property name="checked_out" />
  <property name="release_status_list" />
  <property name="object_type" />
</header>
```

6. Click the **Apply** button.
7. Choose **Edit→Options**, click the **Search** link at the bottom of the **Options** dialog box, locate the **Folder.SUMMARYRENDERING** preference, and verify that the preference value has been changed from **FolderSummary** to **MyFolderSummary**. If it has not been changed, click the **Edit** button and change the value to **MyFolderSummary**.
8. Select a folder and select the **Summary** pane.

You see the **checked_out** property displayed as **Checked-Out**.

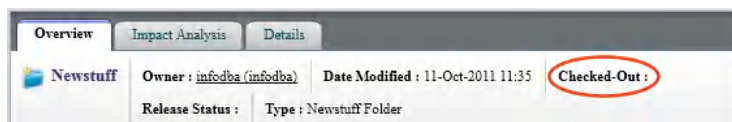


Adding a property to the rich client Summary pane

Note:

If the change doesn't appear in the rich client, you can exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

In the thin client, the change appears as follows.



Adding a property to the thin client Overview tab

9. You can configure the folder summary in many other ways. For example, to show folder contents using the **objectSet** tag, place the following code highlighted in **bold** into the **MyFolderSummary** style sheet after the properties section, and click the **Apply** button:

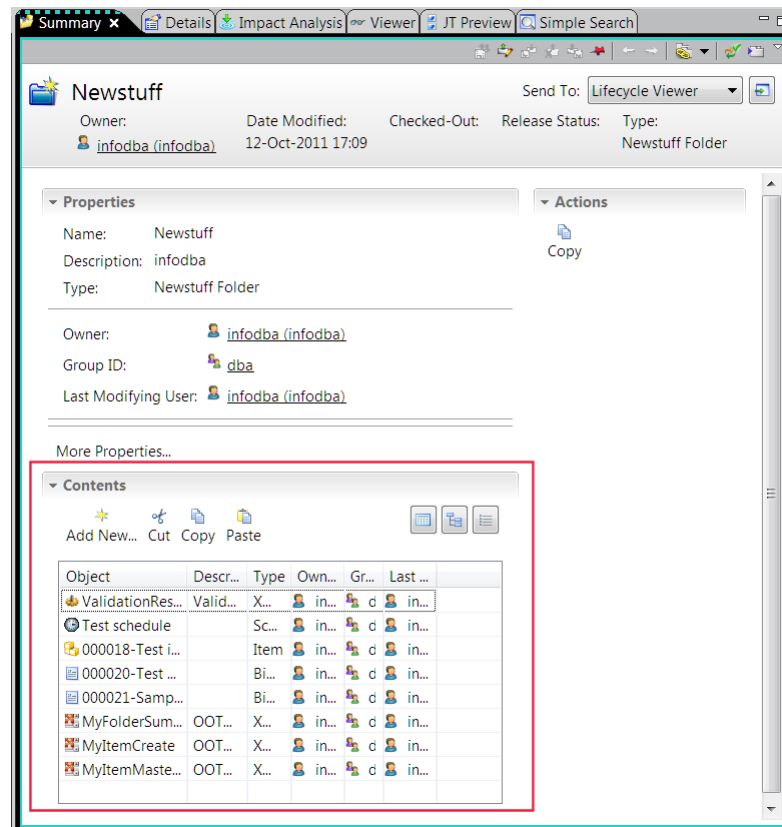
```
<section titleKey="tc_xrt_ItemProperties">
  <property name="object_name"/>
  <property name="object_desc"/>
  <property name="object_type"/>
  <separator/>
  <property name="owning_user" renderingHint="objectlink" modifiable="false"/>
  <property name="owning_group" renderingHint="objectlink" modifiable="false"/>
  <property name="last_mod_user"/>
  <separator/>
  <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
<section title="Contents">
  <objectSet source="contents.WorkspaceObject" defaultdisplay="thumbnailDisplay"
sortBy="object_name" sortdirection="ascending">
    <tableDisplay>
      <property name="object_string"/>
      <property name="object_desc"/>
      <property name="object_type"/>
      <property name="owning_user"/>
      <property name="owning_group"/>
      <property name="last_mod_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
      <property name="object_string"/>
      <property name="object_desc"/>
      <property name="object_type"/>
      <property name="owning_user"/>
      <property name="owning_group"/>
      <property name="last_mod_user"/>
    </treeDisplay>
    <listDisplay>
      <command actionKey="newBusinessObjectContextualAction"
commandId="com.teamcenter.rac.common.AddNew" renderingHint="commandbutton"/>
      <command actionKey="cutAction" commandId="org.eclipse.ui.edit.cut"
renderingHint="commandbutton">
        <parameter name="localSelection" value="true"/>
      </command>
    </listDisplay>
  </objectSet>
</section>
```

```

</command>
<command actionKey="copyAction" commandId="com.teamcenter.rac.copy"
renderingHint="commandbutton"/>
<command actionKey="pasteAction"
commandId="com.teamcenter.rac.viewer.pastewithContext" renderingHint="commandbutton"/>
</objectSet>
</section>

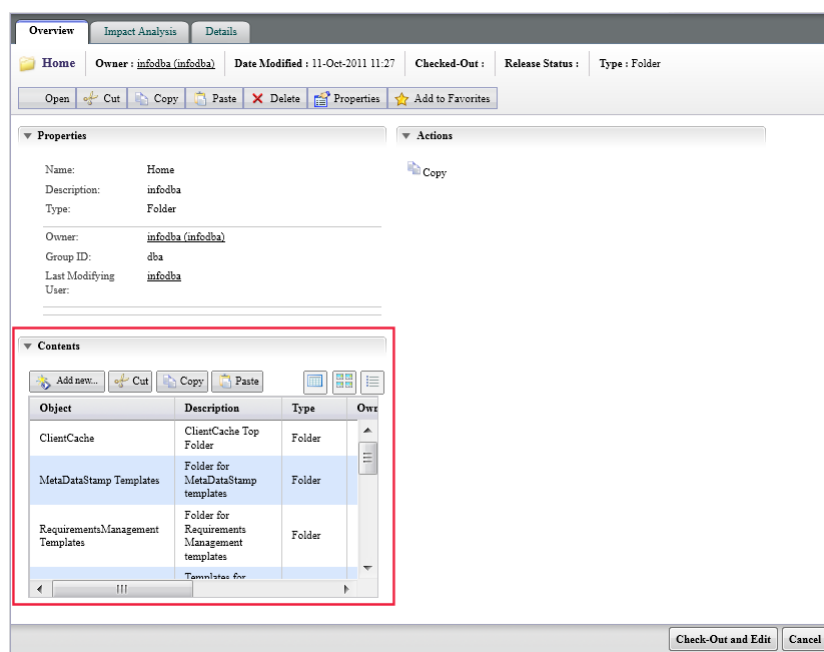
```

When you select a folder and click the **Summary** tab, the **Contents** section is displayed.



Adding a section to the rich client Summary pane using the objectSet tag

In the thin client, the change appears as follows.



Adding a section to the thin client Overview tab using the objectSet tag

Note:

Only three **<views>** elements are supported on the **Summary** view: **impactanalysis**, **viewer**, and **properties**.

- **impactanalysis** is for where-used/referenced information.
- **viewer** is for an image preview.
- **properties** is for any combination of properties.

The style sheet contains **<page>** elements, and each **<page>** can contain multiple views. If a user right-clicks the **Summary** view, a shortcut menu with the view list is displayed; the user can hide or display views. The **viewer** view can be used for items, item revisions, and datasets, but not folders or forms. The other two views can be used for any object.

Modify the Properties pane on the Summary view using style sheets

You can modify what appears in the **Properties** pane and its layout by codelessly changing the rendering using Teamcenter style sheets. This example adds the **protection** property to the folder general rendering.

1. In the rich client, find all **XMLRenderingStylesheet** datasets using the search capability by removing all search criteria except for **Type** and setting it to **XMLRenderingStylesheet**.
2. Select the **FolderSummary** dataset and choose **File**→**Save As**. Type **MyFolderSummary** in the **Name** box.

The **MyFolderSummary** style sheet is saved in the **Newstuff** folder.

3. Select **MyFolderSummary** and the **Viewer** pane.

The XML style sheet is displayed in the viewer.

4. In the **Registered Type** box, select **Folder**, and in the **Stylesheet Type** box, select **Summary**. Click the **Apply** button in the lower right corner.

Note:

This changes the value in the **Folder.SUMMARYRENDERING** preference to **MyFolderSummary**. To see the change, choose **Edit**→**Options**, click the **Search** link at the bottom of the **Options** dialog box, locate the **Folder.SUMMARYRENDERING** preference, and verify that the preference value has been changed from **FolderSummary** to **MyFolderSummary**. If it has not been changed, click the **Edit** button and change the value to **MyFolderSummary**.

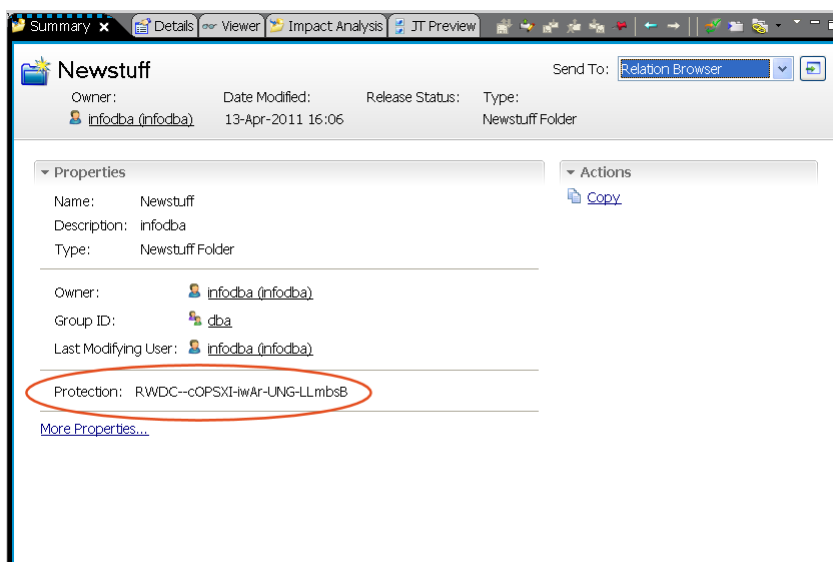
5. Edit the **MyFolderSummary** style sheet to add a separator and the **protection** property to the general area. The following is an example of the modified general area:

```
<section titleKey="tc_xrt_ItemProperties">
  <property name="object_desc"/>
  <separator/>
  <property name="owning_user" renderingHint="objectlink" modifiable="false"/>
  <property name="owning_group" renderingHint="objectlink" modifiable="false"/>
  <property name="last_mod_user"/>
  <separator/>
  <property name="checked_out"/>
  <property name="checked_out_user"/>
  <separator/>
  <property name="protection" />
</separator/>

  <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
```

6. Click the **Apply** button.
7. In the rich client, select a folder and click the **Summary** tab.

The **protection** property is displayed.



Adding a property to the Properties pane on the Summary tab in the rich client

Note:

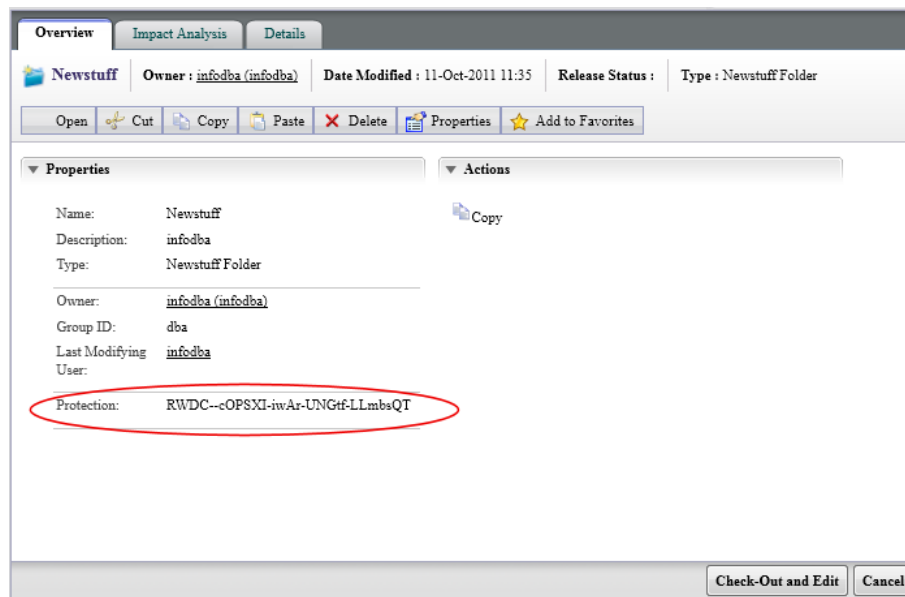
If your customization does not appear, exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

If your customizations still do not appear, clear cache by deleting the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

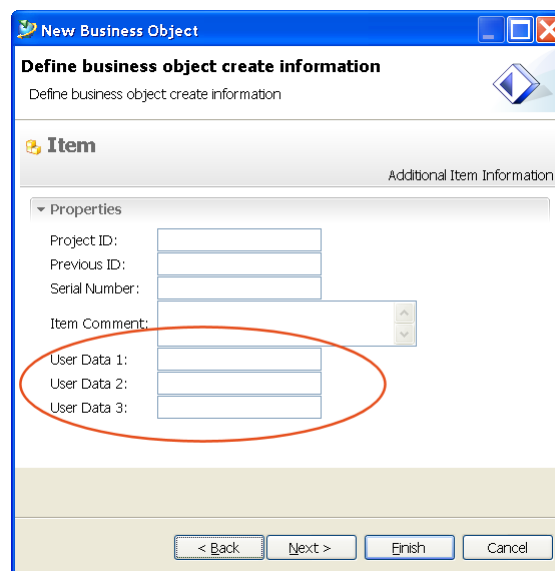
In the thin client, the change appears as follows.



Adding a property to the Properties pane on the Overview tab in the thin client

Modify the item create panels in the New Business Object wizard using style sheets

You can modify what appears in the New Business Object wizard in the rich client when you choose the **File→New→Other** menu command by codelessly changing the rendering using Teamcenter style sheets. This example modifies the item creation panes by removing the **user_data** boxes from the **Additional Item Information** dialog box (which uses the **Item Master** form) and the **Item Revision Information** dialog box (which uses the **ItemRevision Master** form).



User Data boxes on the Item Master form

1. In the rich client, find all **XMLRenderingStylesheet** datasets using the search capability by removing all search criteria except for **Type** and setting it to **XMLRenderingStylesheet**.

2. Select the **ItemCreate** dataset and choose **File→Save As**. Type **MylItemCreate** in the **Name** box.

The **MylItemCreate** style sheet is saved in the **Newstuff** folder.

3. Select **MylItemCreate** and the **Viewer** pane.

The XML style sheet is displayed in the viewer.

4. In the **Registered Type** box, select **Item**, and in the **Stylesheet Type** box, select **Create**. Click the **Apply** button in the lower right corner.

Note:

This changes the value in the **Item.CREATERENDERING** preference to **MylItemCreate**. To see the change, choose **Edit→Options**, click the **Search** link at the bottom of the **Options** dialog box, and locate the **Item.CREATERENDERING** preference. If the value has not been changed, click the **Edit** button and change the value from **ItemCreate** to **MylItemCreate**.

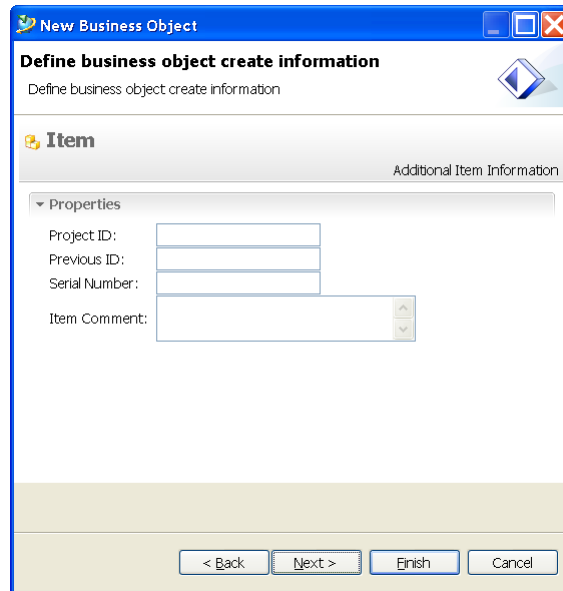
5. Edit the **MylItemCreate** style sheet to remove the **user_data** elements shown in bold in the following sample:

```
<section title="Additional Item Information" titleKey="tc_xrt_AdditionalItemInformation"
initialstate="collapsed">
  <property name="IMAN_master_form:project_id" />
  <property name="IMAN_master_form:previous_item_id" />
  <property name="IMAN_master_form:serial_number" />
  <property name="IMAN_master_form:item_comment" />
  <property name="IMAN_master_form:user_data_1" />
  <property name="IMAN_master_form:user_data_2" />
  <property name="IMAN_master_form:user_data_3" />
</section>

<section title="Item Revision Information" titleKey="tc_xrt_ItemRevisionInformation"
initialstate="collapsed">
  <property name="revision:IMAN_master_form_rev:project_id" />
  <property name="revision:IMAN_master_form_rev:previous_version_id" />
  <property name="revision:IMAN_master_form_rev:serial_number" />
  <property name="revision:IMAN_master_form_rev:item_comment" />
  <property name="revision:IMAN_master_form_rev:user_data_1" />
  <property name="revision:IMAN_master_form_rev:user_data_2" />
  <property name="revision:IMAN_master_form_rev:user_data_3" />
</section>
```

6. Click the **Apply** button.
7. To test the customization in the rich client, choose **File→New→Other** and select **Item**.

The **user_data** boxes are removed from the new item create panes.



Item Master Form with User Data boxes removed in the rich client

Note:

If your customizations do not appear, exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

If your customizations still do not appear, clear cache by deleting the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

To see the change in the thin client, choose **New→Item**.

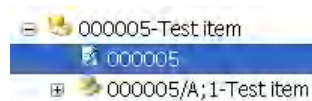
Item Master Form with User Data boxes removed in the thin client

Modify a form's rendering using style sheets

You can modify how a Teamcenter-provided form is rendered by codelessly changing the Teamcenter style sheets. This example modifies the item master form.

1. In the rich client, choose **File**→**New**→**Item** to create a new item.

To see the default item master form, open the form just under the item.



Selecting the item master form

The default item master form appears:

Default item master form

- Find all **XMLRenderingStylesheet** datasets using the rich client search capability by removing all search criteria except for **Type** and setting it to **XMLRenderingStylesheet**.
- Select the **FormXMLRenderingStylesheet** dataset and choose **File→Save As**. Type **MyItemMasterRenderer** in the **Name** box.

The **MyItemMasterRenderer** style sheet is saved in the **Newstuff** folder.

- Select **MyItemMasterRenderer** and the **Viewer** pane.

The XML style sheet is displayed in the viewer.

- In the **Registered Type** box, select **Item Master**, and in the **Stylesheet Type** box, select **Form**. Click the **Apply** button in the lower left corner.

Note:

This creates a new **Item Master.FORMRENDERING** preference. To see the preference, choose **Edit→Options**, click the **Search** link at the bottom of the **Options** dialog box, and locate the **Item Master.FORMRENDERING** preference. Note that its value is set to **MyItemMasterRenderer**.

- Replace the **rendering** tag contents of the **MyItemMasterRenderer** style sheet with the following sample code:

```
<rendering>
  <page title="General" titleKey="tc_xrt_General">
    <property name="project_id" />
    <property name="serial_number" renderingHint="textfield" column="25" />
    <property name="item_comment" renderingHint="textarea" column="10"
      row="5" />
    <property name="owning_user" />
  </page>
  <page title="Advanced">
```

```

<column>
  <property name="user_data_1" renderingHint="lovbutton"
    renderingStyle="titled" border="true" />
  <separator />
  <property name="user_data_2" renderingStyle="titled" />
</column>
<column>
  <property name="user_data_3" renderingStyle="titled"
    border="true" />
  <property name="previous_item_id" renderingStyle="titled" />
</column>
</page>
</rendering>

```

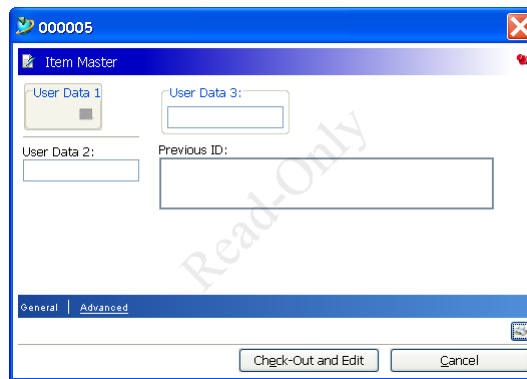
Note:

Notice how this code adds an **Advanced** link to the form.

7. Click the **Apply** button.
8. Open the item master form to see the new layout.

Customized layout of the form's General properties page in the rich client

Click the **Advanced** link at the bottom of the form to open the additional properties on the form.



Customized layout of the form's Advanced properties page in the rich client

Note:

To see the change in the rich client, you may need to exit the rich client and restart it using the **-clean** command argument to remove the old configuration from cache.

If your customizations still do not appear in the rich client, clear cache by deleting the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

Localizing style sheets

Localization is the translation of text into the local language. XML rendering templates (XRTs), also known as style sheets, have their localized text in the **tc_xrt_text_locale.xml** file on the server in the following location: **TC_ROOT\lang\textserver\en_US**. When you customize a style sheet with new text, you must enter the text to this file in the different languages to be displayed in the user interface. If you do not enter localized text strings, the unlocalized text appears in the user interface between exclamation marks (for example: **!MyText!**) except when in the **command** tag.

Style sheets use several tags:

- **titleKey**
Finds text values in the text server files.
- **textKey**
Finds text values in the text server files.

Values used in the **titleKey** style sheet attribute have a corresponding localized text string in the **tc_xrt_text_locale.xml** text server file. This text server file contains all the style sheet text key entries

for the rich client and the thin client. All keys are prefixed by **tc_xrt_**. (If the keys in the style sheet do not have this prefix, the system adds the prefix and then looks in the text server files for the corresponding values.)

Following is code from the **ItemRevSummary.xml** style sheet:

```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_ItemProperties">
```

Following are the corresponding keys in the *TC_ROOT\lang\textserver\en_US\tc_xrt_text_locale.xml* file:

```
<key id="tc_xrt_Overview">Overview</key>
...
<key id="tc_xrt_ItemProperties">Item Properties</key>
```

Note:

- There is no automated upgrade utility to move custom key/value pairs into the **tc_xrt_text_locale.xml** file. You must move them manually.
- Certain character sets cannot be used, for example, Chinese.

Style sheet definition

Example of XML style sheet definition and rendering

The following code shows an example of an XML definition for item properties found in the **Item.xml** XML rendering style sheet file:

```
<rendering>

  <page title="General" titleKey="tc_xrt_General">
    <column>
      <property name="object_string" column="32"/>
      <separator/>
      <property name="object_name" column="32"/>
      <property name="object_desc" />
      <separator/>
      <property name="owning_user" renderingHint="objectlink" modifiable="false" />
      <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
      <property name="last_mod_user" />
    </column>
    <column>
      <image/>
    </column>
  </page>

  <page title="Reservation" titleKey="tc_xrt_Reservation">
```

```

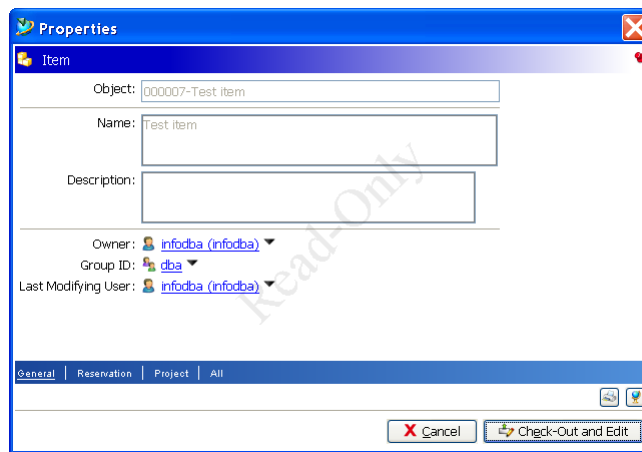
    <property name="checked_out" />
    <property name="checked_out_user" />
    <separator/>
    <property name="checked_out_date" />
    <property name="checked_out_change_id" />
    <separator/>
    <property name="reservation" />
  </page>

  <page title="Project" titleKey="tc_xrt_Project">
    <property name="proj_assign_mod_date" />
    <property name="project_ids" />
    <separator/>
    <property name="project_list" />
  </page>

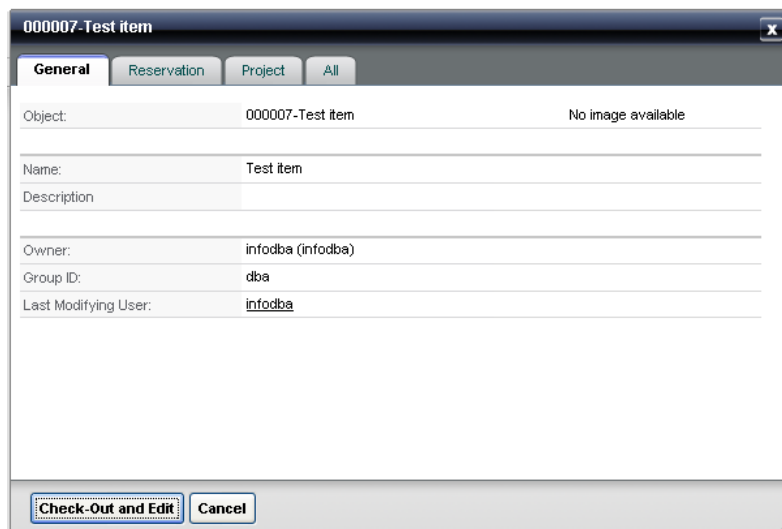
  <page title="All" titleKey="tc_xrt_All">
    <all type="property"/>
  </page>
</rendering>

```

The following figures show the resulting item properties dialog box in the rich client and thin client.



Item properties dialog box in the rich client



Item properties dialog box in the thin client

XML elements

all

Lists all properties of the defined object.

ATTRIBUTES

type

Indicates whether to list all the object properties or only form properties. The valid values for this attribute are **property** and **form**.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

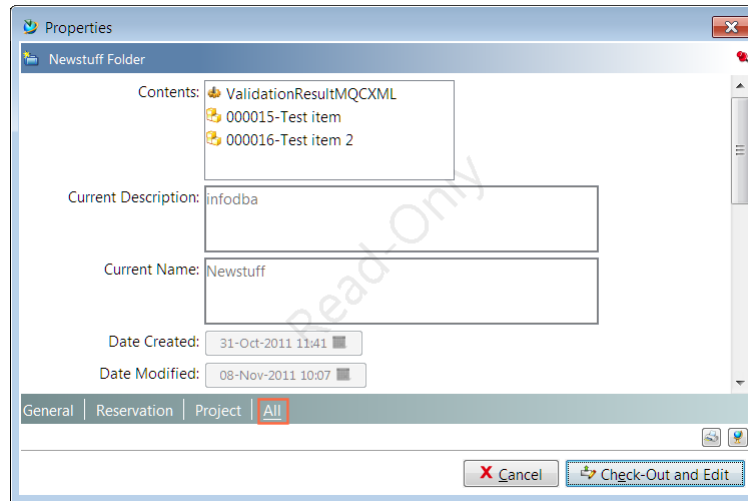
This tag can be used on the following types of style sheets (but not the New Business Object wizard):

Property
Summary
Form

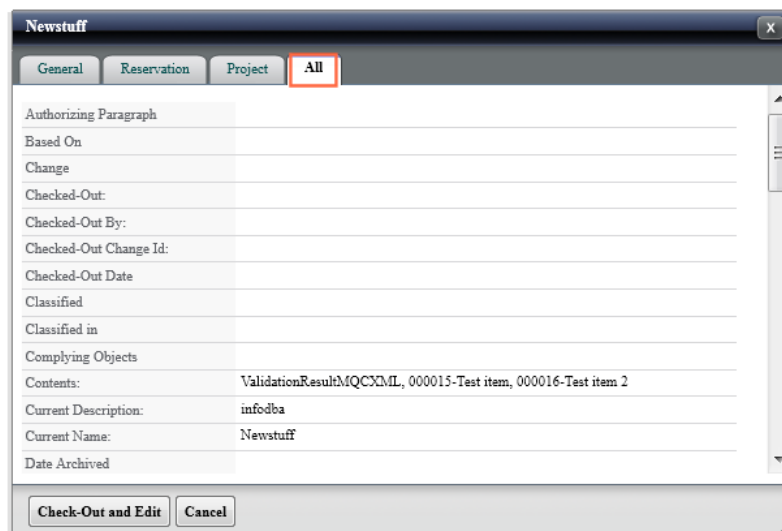
EXAMPLE

Following is sample code from the **Folder.xml** XML rendering style sheet showing the **all** element:

```
<page title="All" titleKey="tc_xrt_All">
  <all type="property"/>
</page>
```



The all element in the rich client



The all element in the thin client

Note:

The **<conditions>**, **<GoverningProperty>**, and **<Rules>** tags are not supported within the **<all>** tag.

attachments

Specifies which objects attached to an item revision should appear in the attachments list. The format for the element value is:

relation.type

For example:

```
<attachments>
IMAN_reference.MSExcel
</attachments>
```

The example displays all attachments to the item revision with a **MSExcel** type and an **IMAN_reference** relation. Separate multiple entries with commas.

Caution:

As of Teamcenter 10.1, the **attachments** tag is deprecated and is replaced by the **objectSet** tag.

For example:

```
<view name="attachments">
  <attachments>IMAN_specification.UGMASTER, IMAN_reference.MSExcel,
  IMAN_Rendering.DirectModel</attachments>
</view>
```

Is replaced by:

```
<section titleKey="tc_xrt_attachments" title="Attachments">
  <objectSet source="IMAN_specification.UGMASTER, IMAN_reference.MSExcel,
  IMAN_Rendering.DirectModel" defaultdisplay="linkDisplay" sortBy="object_string"
  sortdirection="ascending">
    <linkDisplay/>
  </objectSet>
</section>
```

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemRevSummary2007.xml** XML rendering style sheet showing the **attachments** element:

```
<view name="attachments">
  <attachments>IMAN_specification.UGMASTER, IMAN_reference.MSExcel,
  IMAN_Rendering.DirectModel, SimplifiedRendering.JtSimplification</attachments>
</view>
```

break

Inserts a break in the pane.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

Property
Summary
Form
Create

EXAMPLE

```
<page titleKey="tc_xrt_General" title="General">
  <property name="project_id" icon="images/group.gif"/>
  <property name="serial_number" renderingHint="textfield" renderingStyle="headed"/>
  <property name="item_comment" renderingHint="textarea" column="30" row="5"/>
  <property name="user_data_1" renderingHint="lovcombobox" renderingStyle="titled"
border="true" />
  <separator />
  <property name="user_data_1" />
  <property name="user_data_2" />
```

```
<break />
</page>
```

classificationProperties

Specifies that the **classification properties** of the current object should be displayed. Properties and their values are rendered as name/value pairs in static text.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

SUPPORTED STYLE SHEETS

This tag can be used only on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **classificationProperties** element:

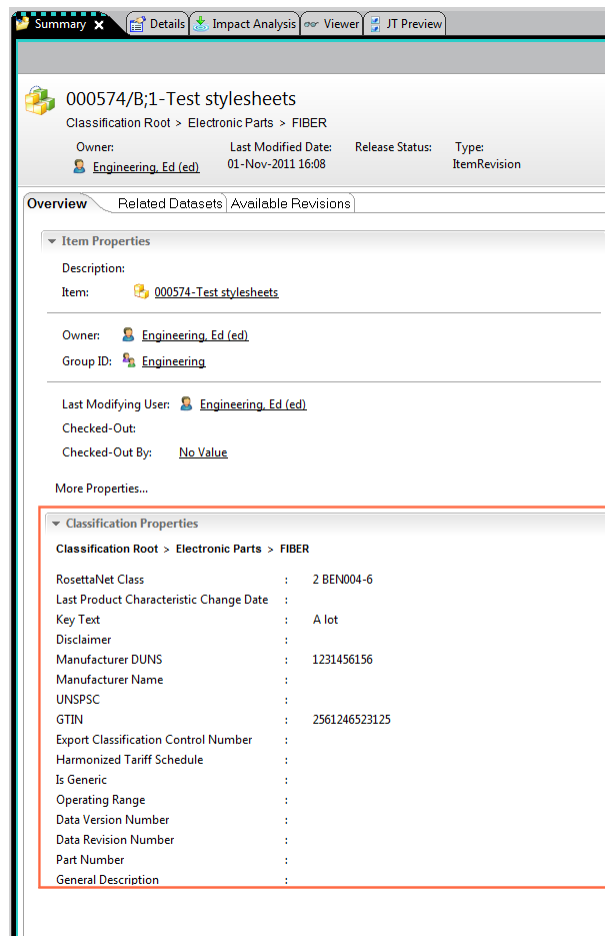
```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="listDisplay"
sortdirection="descending" sortBy="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section titleKey="tc_xrt_ItemProperties">
```

```

        <property name="object_desc"/>
        <separator/>
        <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
        <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
        <property name="last_mod_user"/>
        <separator/>
        <property name="checked_out"/>
        <property name="checked_out_user"/>
        <separator/>
        <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
    </section>
    <section titleKey="tc_xrt_ClassificationProperties">
<classificationProperties/>
</section>
</column>

```

Following is how classification properties are rendered in the rich client.



classificationProperties in the rich client

classificationTrace

Specifies that the **classification** traces of the item should be displayed, for example, **Home Care > Cleaners > Detergents**.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

SUPPORTED STYLE SHEETS

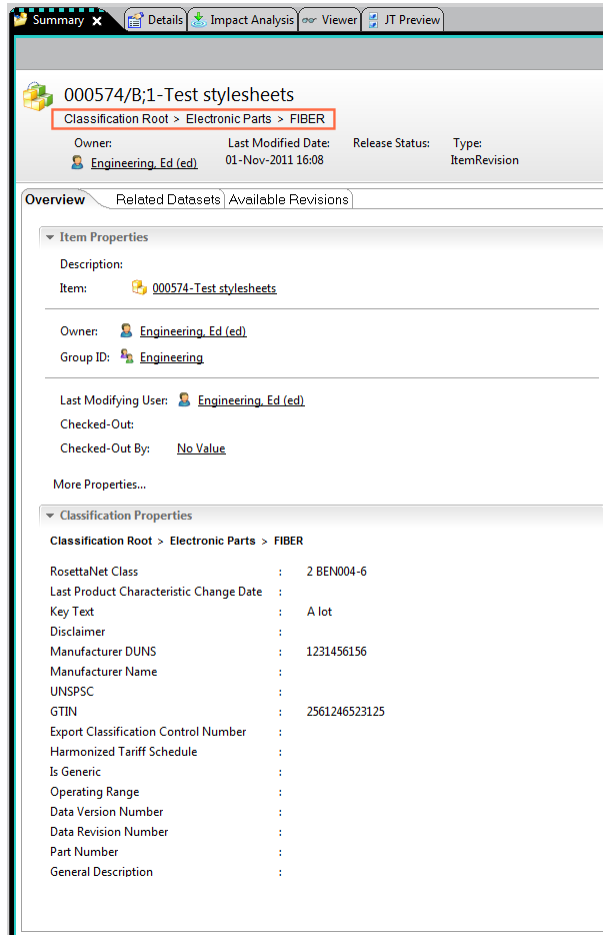
This tag can be used only on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **classificationTrace** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
  <property name="last_mod_date"/>
  <property name="release_status_list"/>
  <property name="object_type"/>
</header>
```

Following is how the classification trace is rendered in the rich client.



classificationTrace in the rich client

column

Defines the layout of the column defined on a page.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets (but not the New Business Object wizard):

Property Summary Form

EXAMPLE

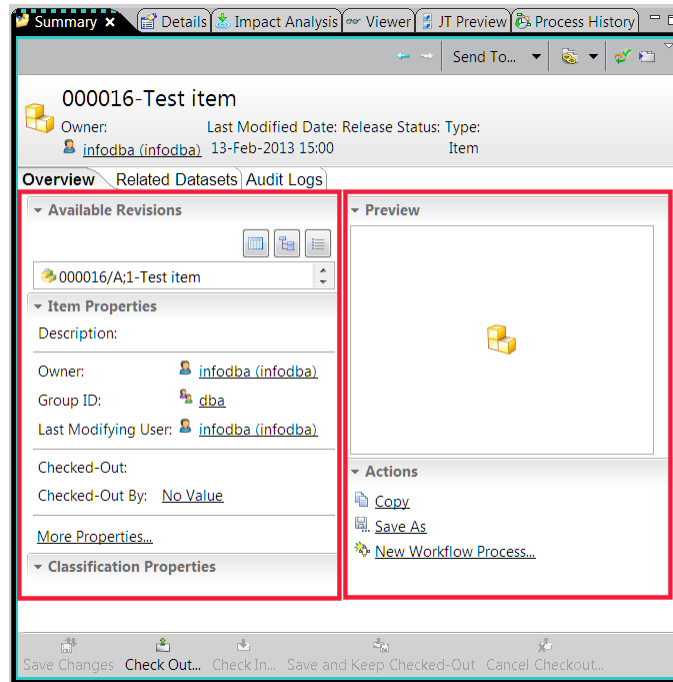
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **column** element:

```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaulttdisplay="listDisplay"
sortdirection="descending" sortby="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section titleKey="tc_xrt_ItemProperties">
      <property name="object_desc"/>
      <separator/>
      <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
      <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
      <property name="last_mod_user"/>
      <separator/>
      <property name="checked_out"/>
      <property name="checked_out_user"/>
      <separator/>
      <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
    </section>
    <section titleKey="tc_xrt_ClassificationProperties">
      <classificationProperties/>
    </section>
  </column>
  <column>
    <section titleKey="tc_xrt_Preview">
      <image source="preview"/>
    </section>
  </column>
</page>
```

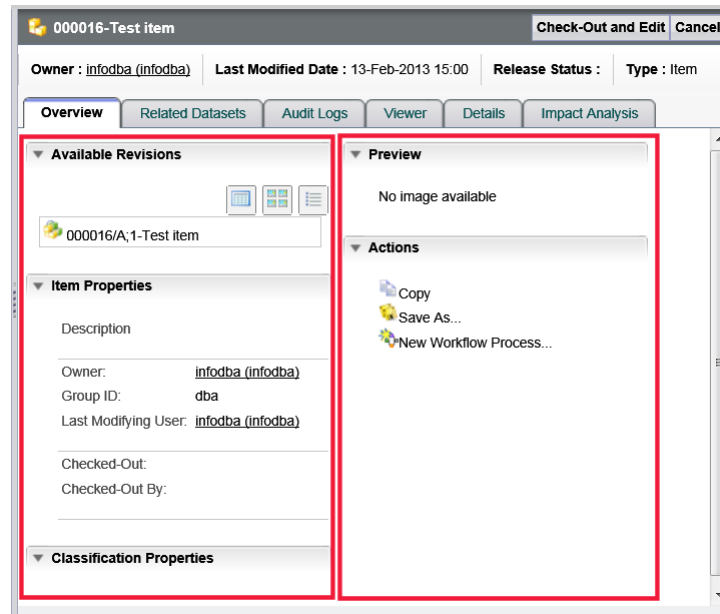
```

</section>
<section titleKey="tc_xrt_actions" commandLayout="vertical">
  <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
  <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
  <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess"
titleKey="tc_xrt_newProc" />
</section>
</column>
</page>

```



column element in the rich client



column element in the thin client

command

Specifies a command representation to be displayed.

ATTRIBUTES

actionKey

Specifies the action key that presents the command in the thin client. (If you do not want to present the command in the thin client, you can omit this attribute.)

commandId

Specifies the command to be executed. This attribute is used only by the rich client. The attribute value must be a key into a property file and must be a valid command ID defined in the rich client. This is a string attribute that is required.

defaultTitle

Specifies the string to be displayed if a key is not found in the property file. This is an optional attribute.

Caution:

As of Teamcenter 10.1, the **defaultTitle** attribute is deprecated and is replaced by the **title** and **titlekey** attributes.

icon

Specifies the icon to be displayed before the command label. The attribute value must be a key into a property file. This is an optional attribute.

renderingHint

Specifies whether the command is rendered as a hyperlink or as a button. Valid values are **hyperlink** and **commandbutton**. This attribute is optional. If the attribute is not specified, the command is rendered as a hyperlink.

text

Specifies the text to display. The attribute value must be a key into a property file. If the key is not found, the attribute value itself is displayed as static text. If neither the **text** or **icon** attributes are specified, the value of the **commandId** attribute is rendered. This attribute is optional.

Caution:

As of Teamcenter 10.1, the **text** attribute is deprecated and is replaced by the **title** and **titlekey** attributes.

title

Specifies the default string of the title for this user interface element. This attribute is used when the string in the **titleKey** attribute is not found in the locale file. This is an optional attribute.

titleKey

Specifies the key used to search for the title in the locale file. If it is not defined, the string defined by the **title** attribute is used. This is an optional attribute.

tooltip

Specifies the tooltip for the command. The attribute value must be a key into a property file. This attribute is optional but is required if the **icon** attribute is specified.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used only on the **Summary** style sheets.

Note:

The **command** tag is ignored in the **header** section. Commands cannot be added to the toolbar.

EXAMPLE

Following is sample code using the **command** element on an object set:

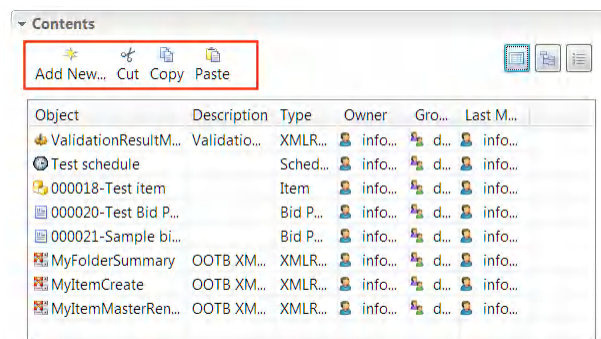
```
<objectSet source="contents.WorkspaceObject" defaultdisplay="thumbnailDisplay"
```

```

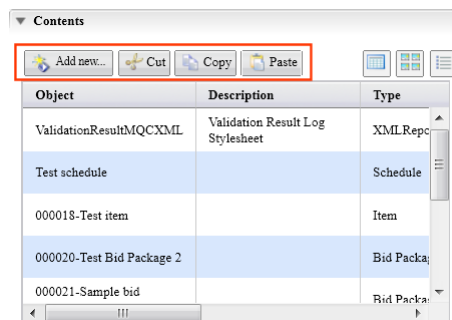
sortby="object_name" sortdirection="ascending">
  <tableDisplay>
    <property name="object_string"/>
    <property name="object_desc"/>
    <property name="object_type"/>
    <property name="owning_user"/>
    <property name="owning_group"/>
    <property name="last_mod_user"/>
  </tableDisplay>
  <thumbnailDisplay/>
  <treeDisplay>
    <property name="object_string"/>
    <property name="object_desc"/>
    <property name="object_type"/>
    <property name="owning_user"/>
    <property name="owning_group"/>
    <property name="last_mod_user"/>
  </treeDisplay>
  <listDisplay/>
    <command actionKey="newBusinessObjectContextualAction"
commandId="com.teamcenter.rac.common.AddNew" renderingHint="commandbutton"/>
    <command actionKey="cutAction" commandId="org.eclipse.ui.edit.cut"
renderingHint="commandbutton">
      <parameter name="localSelection" value="true"/>
    </command>
    <command actionKey="copyAction" commandId="com.teamcenter.rac.copy"
renderingHint="commandbutton"/>
    <command actionKey="pasteAction"
commandId="com.teamcenter.rac.viewer.pastewithContext" renderingHint="commandbutton"/>
  </objectSet>

```

In this example, the **command** element adds the **Add New**, **Cut**, **Copy**, and **Paste** buttons in the user interface.



command element in the rich client



command element in the thin client

For a customization example that uses the **command** element, see [Modify the Summary view using style sheets](#).

conditions

Indicates that rules are specified for a property.

The **GoverningProperty** and **Rule** elements are used within the **conditions** element. The **GoverningProperty** tag defines the property to apply the condition to, and the **Rule** tag defines the state of the property. If multiple properties within a single form are to be tracked, there are multiple entries of the **GoverningProperty** tag within the **conditions** tag. Combining entries for a different form types within a single stylesheet is not permitted.

Note:

The **<conditions>**, **<GoverningProperty>**, and **<Rules>** tags are not supported:

- in the **Viewer** view if the style sheet is registered for properties or form display.
- On the **All** page of the property page, or inside an **<all>** property tag.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

**Property
Form**

EXAMPLE

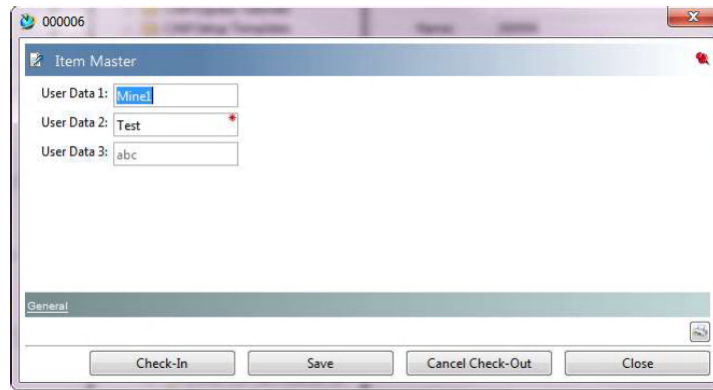
Following is sample style sheet code showing the **conditions** element:

```
<rendering>
  <property name = "user_data_1"/>
  <property name = "user_data_2"/>
  <property name = "user_data_3"/>
  <conditions>
    <GoverningProperty propertyname = "user_data_1" propertyvalue = "Mine1">
      <Rule propertyname = "user_data_2" state = "required"/>
      <Rule propertyname = "user_data_3" state = "disabled"/>
    </GoverningProperty>
    <GoverningProperty propertyname = "user_data_1" propertyvalue = "Do IM">
      <Rule propertyname = "user_data_2" state = "disabled"/>
      <Rule propertyname = "user_data_3" state = "required"/>
    </GoverningProperty>
  </conditions>
</rendering>
```

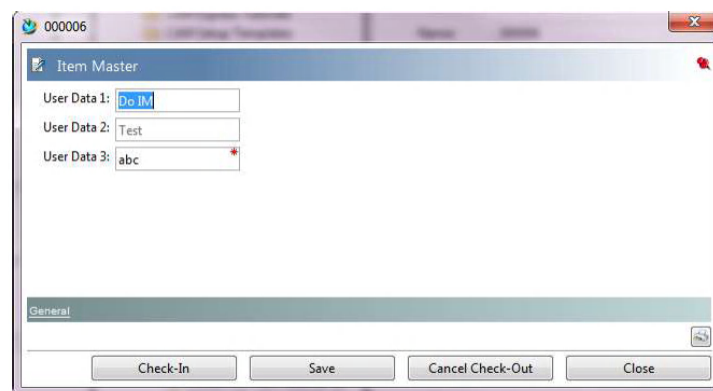
The first **GoverningProperty** node in the sample states that if the **User Data 1** box contains a value of **Mine1**, then the **User Data 2** box is required and the **User Data 3** box is disabled. The second **GoverningProperty** node in the sample states that if the **User Data 1** box contains a value of **Do IM**, then the **User Data 2** box is disabled and the **User Data 3** box is required.

Following is the resulting user interface.

No value match in the User Data 1 box



Value in the User Data 1 box matches the first GoverningProperty node



Value in the User Data 1 box matches the second GoverningProperty node

For another example, see [Set properties to be conditionally mandatory or disabled](#).

customPanel

Embeds a custom panel in a dialog box.

ATTRIBUTES

java

Specifies the fully qualified Java implementation class name responsible for building the custom user interface (for example, **com.teamcenter.rac.MyCustomPanel**). This is supported in the rich client only.

js

Specifies a JavaScript function that generates the HTML (for example, **<customPanel js="MyCustomCalendar()" />**).

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the **Create** style sheets.

EXAMPLE

Following is a custom style sheet that uses the **customPanel** element. (This example works in the rich client only.)

```
<rendering>
  <page title="General" titleKey="tc_xrt_General">
    <section title="Item Information" titleKey="tc_xrt_ItemInformation">
      <property name="item_id" />
      <property name="revision:item_revision_id" />
      <property name="object_name" />
      <property name="object_desc" />
      <separator/>
      <property name="uom_tag" />
      <separator/>
      <customPanel java="com.teamcenter.rac.ui.commands.newbo.mypanel.MyPanel" />

      <section title="Additional Item Information"
titleKey="tc_xrt_AdditionalItemInformation" initialstate="collapsed">
        <property name="IMAN_master_form:project_id" />
        <property name="IMAN_master_form:previous_item_id" />
        <property name="IMAN_master_form:serial_number" />
        <property name="IMAN_master_form:item_comment" />
        <property name="IMAN_master_form:user_data_1" />
        <property name="IMAN_master_form:user_data_2" />
        <property name="IMAN_master_form:user_data_3" />
      </section>

      <section title="Item Revision Information"
titleKey="tc_xrt_ItemRevisionInformation" initialstate="collapsed">
        <property name="revision:IMAN_master_form_rev:project_id" />
        <property name="revision:IMAN_master_form_rev:previous_version_id" />
        <property name="revision:IMAN_master_form_rev:serial_number" />
        <property name="revision:IMAN_master_form_rev:item_comment" />
        <property name="revision:IMAN_master_form_rev:user_data_1" />
        <property name="revision:IMAN_master_form_rev:user_data_2" />
        <property name="revision:IMAN_master_form_rev:user_data_3" />
      </section>
    </page>
  </rendering>
```

You must create your own custom panel to pass to the **customPanel** tag. Following is the **MyPanel.java** file that defines the custom panel:

```

package com.teamcenter.rac.ui.commands.newbo.mypanel;

import com.teamcenter.rac.ui.commands.create.bo.NewBOWizard;
import com.teamcenter.rac.util.AbstractCustomPanel;
import com.teamcenter.rac.util.IPageComplete;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Label;
import org.eclipse.swt.widgets.Text;
import org.eclipse.ui.forms.widgets.FormToolkit;
public class MyPanel extends AbstractCustomPanel implements IPageComplete
{
    private Composite composite;
    private Text text;

    public MyPanel()
    {
    }

    public MyPanel( Composite parent )
    {
        super( parent );

        @Override
        public void createPanel()
        {
            FormToolkit toolkit = new FormToolkit( parent.getDisplay() );
            composite = toolkit.createComposite( parent );

            GridLayout gl = new GridLayout( 2, false );
            composite.setLayout( gl );

            GridData gd = new GridData( GridData.FILL_HORIZONTAL );
            gd.grabExcessHorizontalSpace = true;

            composite.setLayoutData( gd );

            GridData labelGD = new GridData( GridData.HORIZONTAL_ALIGN_END );
            Label label = toolkit.createLabel( composite, "Object_Name: " );
            label.setLayoutData( labelGD );

            GridData typeTextGd = new GridData( GridData.FILL_HORIZONTAL );
            text = toolkit.createText( composite, "" );
            text.setText( "This is my own panel" );
            text.setLayoutData( typeTextGd );
        }

        public boolean isPageComplete()
        {
            String txt = text.getText();
            return txt.length() == 0 ? false : true;
        }

        @Override
        public Composite getComposite()
        {
            return composite;
        }
    }

```

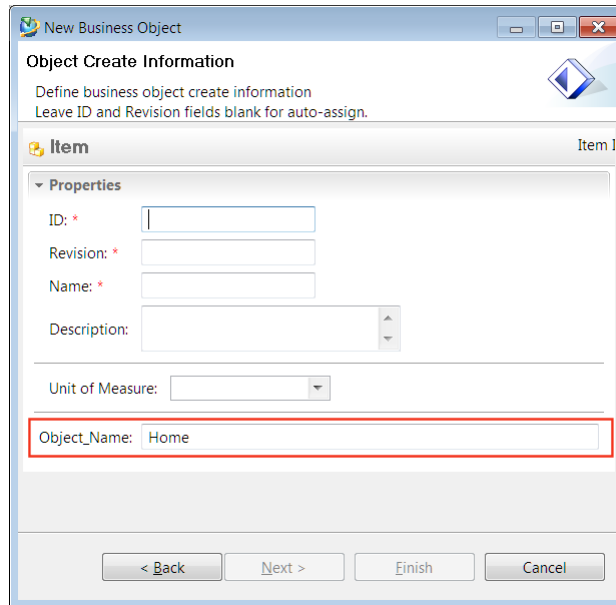
```

@Override
public void updatePanel()
{
    if( input != null )
    {
        NewBOWizard wizard = (NewBOWizard) input;
        String msg = "";
        if( wizard.model.getTargetArray() != null )
        {
            try
            {
                msg = wizard.model.getTargetArray()[0].getProperty(
                    "object_name" ).toString();
            }
            catch( Exception e )
            {
                e.printStackTrace();
            }
        }
        else
        {
            msg = "Nothing is selected";
        }
        text.setText( msg );
    }
}

@Override
public Object getUserInput()
{
    return null;
}
}

```

Following is the resulting custom panel added to the New Business Object wizard, which is run when you choose **File**→**New**→**Other** in the rich client.



customPanel example in the rich client

firstcolumn

Defines the layout of the first column defined on a page. The **secondcolumn** element defines the layout of the second column on a page.

This element applies only if the **TwoColumn** format is set in the **format** attribute on the **page** element.

Note:

When placing properties in **firstcolumn** and **secondcolumn** elements, you must place the properties elements inside the **section** tag. If you do not, the properties do not appear in the thin client user interface.

Caution:

As of Teamcenter 10.1, the **firstcolumn** tag is deprecated and is replaced by the **column** tag.

For example:

```
<page title="General" titleKey="tc_xrt_General" format="TwoColumn">
  <firstcolumn>
    <property name="object_string" column="32"/>
    <separator/>
    <property name="object_name" column="32"/>
    <property name="object_desc" />
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false" />
    <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
    <property name="last_mod_user" />
  </firstcolumn>
  <secondcolumn>
    <image/>
  </secondcolumn>
</page>
```

Is replaced by:

```
<page title="General" titleKey="tc_xrt_General">
  <column>
    <property name="object_string" column="32"/>
    <separator/>
    <property name="object_name" column="32"/>
    <property name="object_desc" />
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false" />
    <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
    <property name="last_mod_user" />
  </column>
  <column>
    <image/>
  </column>
</page>
```

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets (but not the New Business Object wizard):

**Property
Summary
Form**

EXAMPLE

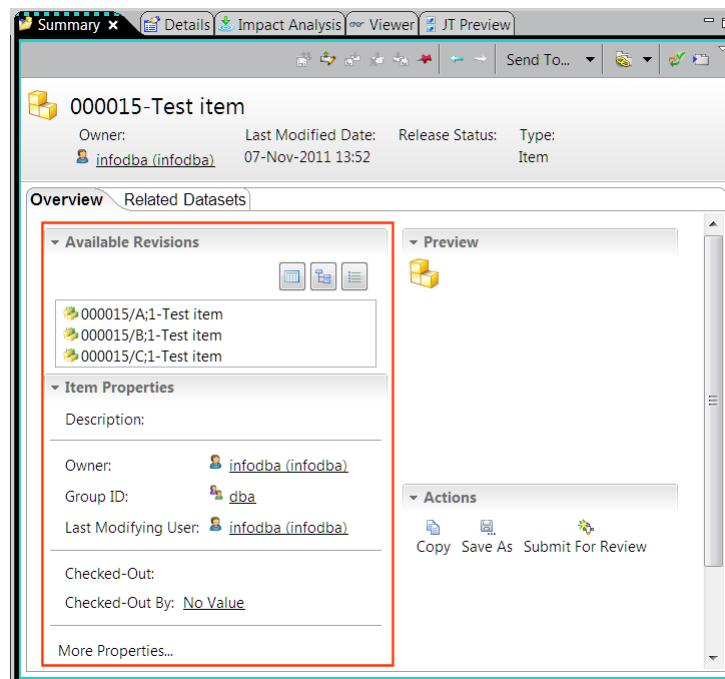
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **firstcolumn** element:

```
<page text="Overview" format="TwoColumn">
  <firstcolumn>
    <section text="AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="thumbnailDisplay" sortdirection="descending"
sortby="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section text="ItemProperties">
      <property name="object_desc"/>
      <separator/>
      <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
      <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
      <property name="last_mod_user"/>
      <separator/>
      <property name="checked_out"/>
      <property name="checked_out_user"/>
      <separator/>
      <command commandId="com.teamcenter.rac.properties"
text="moreProperties"/>
    </section>
    <section text="ClassificationProperties">
      <classificationProperties/>
    </section>
  </firstcolumn>
</page>
```

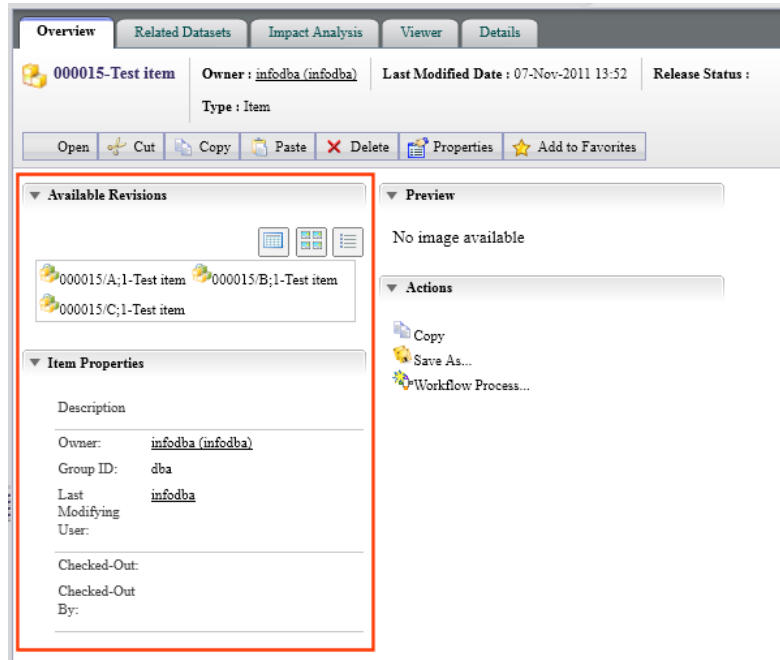
```

    </section>
  </firstcolumn>
  <secondcolumn>
    <view name="viewer" />
    <view name="actions">
      <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
      <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
      <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess" text="newProc" />
    </view>
  </secondcolumn>
</page>

```



firstcolumn element in the rich client



firstcolumn element in the thin client

GoverningProperty

Specifies the name and value of the field that initiates the **Rule** element. The **GoverningProperty** tag must be contained within a **condition** tag and used in conjunction with a **rule** tag.

Note:

The **conditions** and **GoverningProperty** tags do not work in the **Viewer** view if the style sheet is registered for properties or form display.

ATTRIBUTES

propertyname

Specifies the name of the field that triggers the rule if its value matches.

propertyvalue

Specifies the property value that triggers the rule.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

**Property
Form**

EXAMPLE

```
<rendering>
  <property name = "user_data_1"/>
  <property name = "user_data_2"/>
  <property name = "user_data_3"/>
  <conditions>
    <GoverningProperty propertyname = "user_data_1" propertyvalue = "Mine1">
      <Rule propertyname = "user_data_2" state = "required"/>
      <Rule propertyname = "user_data_3" state = "disabled"/>
    </GoverningProperty>
    <GoverningProperty propertyname = "user_data_1" propertyvalue = "Do IM">
      <Rule propertyname = "user_data_2" state = "disabled"/>
      <Rule propertyname = "user_data_3" state = "required"/>
    </GoverningProperty>
  </conditions>
</rendering>
```

For a user interface example, see [conditions](#).

For another example, see [Set properties to be conditionally mandatory or disabled](#).

header

Specifies that a header area must be displayed in the page.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

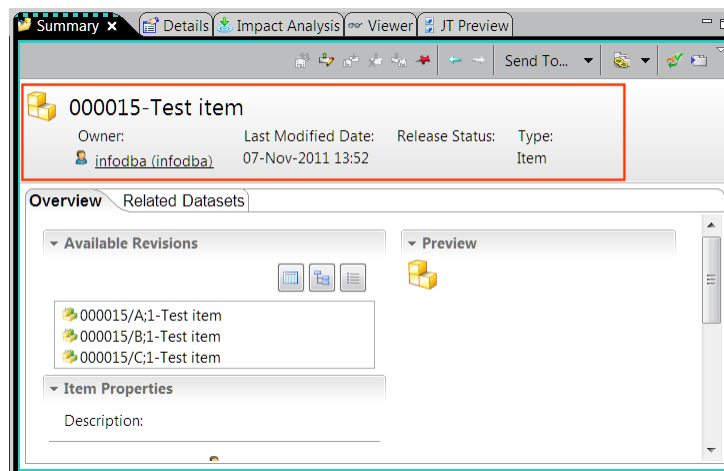
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **header** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
  <property name="last_mod_date"/>
  <property name="release_status_list"/>
  <property name="object_type"/>
</header>
```

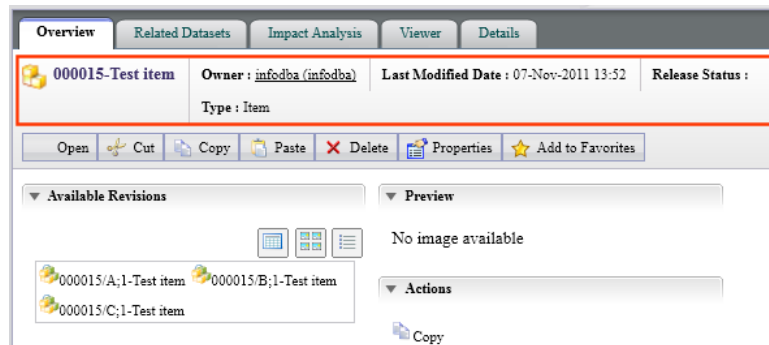
Note:

The valid XML elements for this element are:

- <classificationTrace>
- <image>
- <property>



header element in the rich client



header element in the thin client

image

Specifies that an image is to be rendered. Image dimensions are always kept proportional when being scaled or resized.

ATTRIBUTES

maxheight

Specifies the maximum height in pixels to which the image should be scaled. This is a string attribute that is optional.

maxwidth

Specifies the maximum width in pixels to which the image should be scaled. This is a string attribute that is optional.

source

Specifies the source of the image to display. The attribute value can be a **thumbnail**, **preview**, or **type** keyword. This is a string attribute that is optional.

tooltip

Specifies the tooltip associated with the image. The attribute value must be a key into a property file. This is a string attribute that is optional.

Note:

For backward compatibility, if no attributes are specified and the current object type is an **Item**, **ItemRevision**, or **Dataset** business object, an attempt is made to find and render any preview image that is associated with the object.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

**Property
Summary
Form**

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **image** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
  <property name="last_mod_date"/>
  <property name="release_status_list"/>
  <property name="object_type"/>
</header>
```

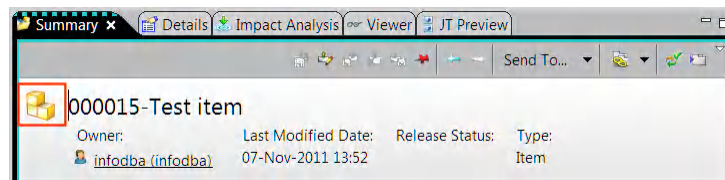


image element in the rich client

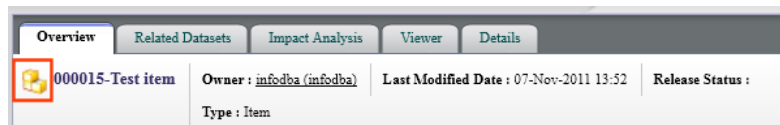


image element in the thin client

label

Specifies a label to be rendered.

ATTRIBUTES

class

Defines the cascading style sheet (CSS) class used to provide the style for the label text. The CSS class must be an existing thin client CSS.

```
label text="Sample text."
class="textCSSClass"
```

style

Controls font style for the label text, including font size, weight, name, and style (such as italic). The format follows the CSS guideline, for example:

```
style="font-size:14pt;font-style:plain;
font-family:Tahoma;font-weight:bold"
```

text

Specifies the text to use for the label. The attribute value must be a key into a property file. If the key is not found, the attribute value itself is displayed as static text. This attribute is required.

URL addresses in the **label text** and **property** tags are automatically rendered. (In the rich client, URL addresses are also automatically rendered for **textfield** and **textarea** rendering hints.)

Note:

The use of double-byte character sets, like Chinese, is not supported with the **text** attribute. They must be localised using **textKey**.

As of Teamcenter 10.1, the **text** tag is deprecated.

textKey

Support is provided for localized values. For example, the tagging `<label text="Hello World" />` displays text on the property page, and `<label textKey="k_version_name" />` displays the localized text in the provided property.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on all style sheet types.

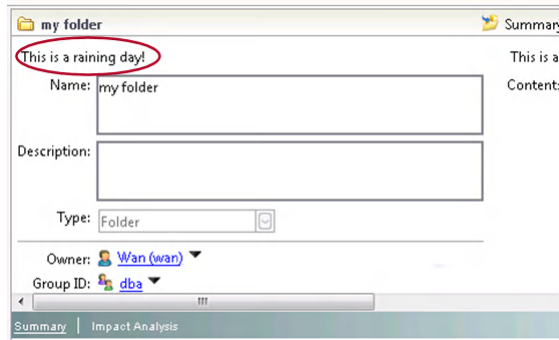
EXAMPLES

- Text

Following is sample code for the **label text** element:

```
<label text="This is a raining day!" />
```

The following figure shows the text on the page.



Label tag example

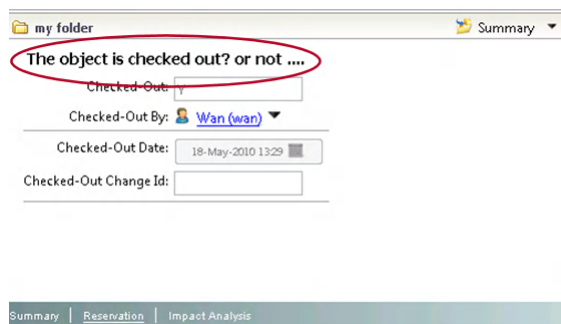
- **style** attribute

The property style sheet page tagging includes the **style** attribute within the **label text** and **property** tags to control font style. Support is provided for font size, weight, name, and style (such as italic). The format follows the CSS guideline, for example, **style="font-size:14pt;font-style:plain;font-family:Tahoma;font-weight:bold"**.

For example:

```
<page title="Reservation" titleKey="tc_xrt_Reservation"
visibleWhen="object_desc==Testing*">
<label text="The object is checked out? or not ...."
style="font-size:14pt; font-style:plain;font-family:Tahoma; font-weight:bold" />
...
</page>
```

This results in the font style shown in the following figure.



style tagging example

- **URL** rendering

URL addresses included in the **label** and **property** tag are automatically rendered.

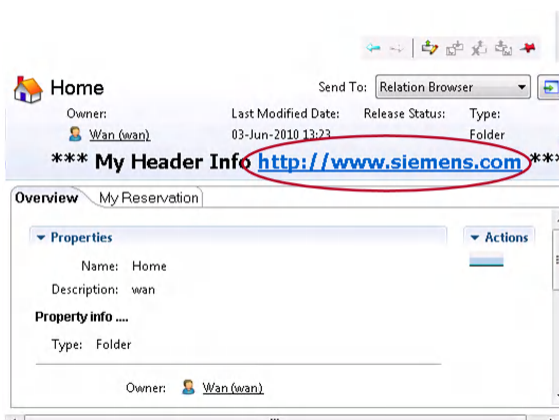
For example:

```
<label text="Press www.abcnews.com to view the latest headlines!"
style=" font-size:14pt;font-style:plain;font-family:Tahoma;font-weight:normal" />
```

Consider the following code:

```
<label text="***My Header Info http://www.siemens.com ***" />
```

Because a URL is included in the **label** tag, it is automatically rendered on the page, as shown in the following figure.



URL rendering example

listDisplay

Displays a set of objects in a list format.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on **Summary** style sheets.

EXAMPLE

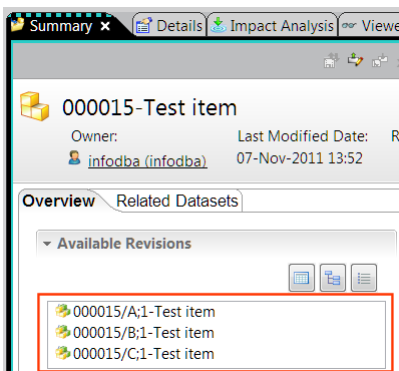
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **listDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
    sortdirection="descending" sortby="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
    </tableDisplay>
  </objectSet>
</section>
```

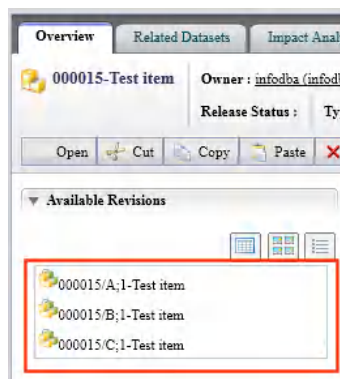
```

        <property name="release_status_list"/>
        <property name="last_mod_date"/>
        <property name="last_mod_user"/>
        <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
        <property name="object_string"/>
        <property name="item_revision_id"/>
        <property name="release_status_list"/>
        <property name="last_mod_date"/>
        <property name="last_mod_user"/>
        <property name="checked_out_user"/>
    </treeDisplay>
    <listDisplay/>
</objectSet>
</section>

```



listDisplay element in the rich client

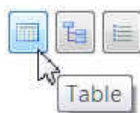


listDisplay element in the thin client

objectSet

Provides a set of display options for the selected object. This element is a replacement for the **attachments** element.

Click the appropriate button to view the object's characteristics in a table, list, or tree.



objectset buttons in the rich client

Note that in the thin client, the tree option is rendered as a thumbnail display.



objectset buttons in the thin client

ATTRIBUTES

defaultdisplay

Specifies the default format to use when displaying the set of objects. Valid values are **treeDisplay**, **tableDisplay**, **listDisplay**, or **thumbnailDisplay**. The default value is **listDisplay**. This is a string attribute that is optional.

maxColumnCharCount

Specifies the maximum number of characters for a column. The default is **80**. Set to **-1** for unlimited column size.

sortby

Specifies the object property to sort the set of objects by prior to rendering. The default value is **object_string**. This is a string attribute that is optional.

sortdirection

Specifies the direction in which the set of objects should be sorted. Valid values are **ascending** or **descending**. The default value is **ascending**. This is a string attribute that is optional.

source

Specifies the comma-delimited set of run-time properties or relations that return the desired set of objects. The format for the attribute value is *property.type*, where *property* is the name of a relation, run-time, or reference property, and *type* represents the type of objects to be included. Multiple *property.type* values can be specified as a comma-separated list, such as **contents.Item** or **contents.ItemRevision**. When using a relation property, you must explicitly specify the relation. Subtypes are not traversed. This is a string attribute that is required.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

Following is sample code using the **objectSet** tag:

```
<objectSet source="contents.WorkspaceObject" defaulttdisplay="thumbnailDisplay"
sortby="object_name" sortdirection="ascending">
  <tableDisplay>
    <property name="object_string"/>
    <property name="object_desc"/>
    <property name="object_type"/>
    <property name="owning_user"/>
    <property name="owning_group"/>
    <property name="last_mod_user"/>
  </tableDisplay>
  <thumbnailDisplay/>
  <treeDisplay>
    <property name="object_string"/>
    <property name="object_desc"/>
    <property name="object_type"/>
    <property name="owning_user"/>
    <property name="owning_group"/>
    <property name="last_mod_user"/>
  </treeDisplay>
  <listDisplay/>
  <command actionKey="newBusinessObjectContextualAction"
commandId="com.teamcenter.rac.common.AddNew" renderingHint="commandbutton"/>
  <command actionKey="cutAction" commandId="org.eclipse.ui.edit.cut"
renderingHint="commandbutton">
    <parameter name="localSelection" value="true"/>
  </command>
  <command actionKey="copyAction" commandId="com.teamcenter.rac.copy"
renderingHint="commandbutton"/>
  <command actionKey="pasteAction"
commandId="com.teamcenter.rac.viewer.pastewithContext" renderingHint="commandbutton"/>
</objectSet>
```

Note:

In an object set, you can use the **command** tag to add buttons for existing menu commands. For example, you can place cut, copy, and paste buttons on the object set. When you add command buttons, you must add the **actionKey** argument to make the buttons appear in the thin client. The action key value to use for each command can be found in the *staging-location\webapp_root\teamcenter\ldhtml\common\intl\language\wsomenu.xml* file.

Property tags used in an **objectSet** are able to display relation properties of the **source** objects. Use the *relationname.propertyname* format to access them. For example, to display the name of the relation type, use the following:

```
<property name = "TC_Attaches.type_string" />
```

You can use the **localSelection** parameter to specify the selected object. If the **localSelection** parameter is set to **true** for any command action, the action is performed on the object selected in the object set. For example, In case of the cut action, if the **localSelection** parameter is **true**, the cut action operates on the object selected in the object set list. If the **localSelection** parameter is **false** (or not set), the cut action operates on the object for which the summary is being shown and not on an object selected in the object set. Although the **localSelection** parameter works with all commands defined in the object set views, it only makes sense to use it with certain commands. For example, any commands such as **Revise**, **Cut**, and **Delete** that should operate on objects in the object set view should have the **localSelection** parameter set to **true**, and any commands such as **Add New** and **Paste** that do *not* operate on the object selected should not use this parameter.

In Systems Engineering, to connect objects with any Trace Link relation (**FND_TraceLink** and its subtypes), you can use the runtime properties of the Workspace object:

- **fnd0defining_objects** (for defining objects)
- **fnd0complying_objects** (for complying objects)

These properties can be displayed using a stylesheet with the same *property.type* format. However, when you use the **Add New** command to add a new object, these properties are not set or modified.

See the Systems Engineering guide for information about **configuring trace link features** and **configuring defining and complying object calculation**.

Following is how the sample code is rendered in the user interface. This object set shows the contents of a folder.

Object	Description	Type	Owner	Gro...	Last M...
ValidationResultM...	Validatio...	XMLR...	info...	d...	info...
Test schedule		Sched...	info...	d...	info...
000018-Test item		Item	info...	d...	info...
000020-Test Bid P...		Bid P...	info...	d...	info...
000021-Sample bi...		Bid P...	info...	d...	info...
MyFolderSummary	OOTB XM...	XMLR...	info...	d...	info...
MyItemCreate	OOTB XM...	XMLR...	info...	d...	info...
MyItemMasterRen...	OOTB XM...	XMLR...	info...	d...	info...

Object set in the rich client (with table button selected)

Following is the same object set rendered in the thin client.

Object	Description	Type
ValidationResultMQCXML	Validation Result Log Stylesheet	XMLRep
Test schedule		Schedule
000018-Test item		Item
000020-Test Bid Package 2		Bid Packa
000021-Sample bid		Bid Packa

Object set in the thin client (with table button selected)

For a customization example that includes an object set, see [Modify the Summary view using style sheets](#).

page

Presents a tab panel in a dialog box or view. If the **page** element is not defined in the XML file, a default page is created.

ATTRIBUTES

format

Specifies the format to be used for this page. This attribute can have one of these values: **OneColumn** or **TwoColumn**. The default value is **OneColumn**. This attribute is optional.

The **firstcolumn** tag defines the layout of the first column, and the **secondcolumn** tag defines the layout of the second column.

Caution:

As of Teamcenter 10.1, the **format** attribute is deprecated on the **page** tag. It is no longer needed because the **firstcolumn** and **secondcolumn** tags are deprecated.

text

Specifies the title to be displayed for the page. The attribute value must be a key into a property file. If the key is not found, the attribute value itself is displayed as static text. This is a required attribute.

Caution:

As of Teamcenter 10.1, the **text** attribute is deprecated and is replaced by the **title** and **titlekey** attributes.

title

Specifies the default string of the title for this tab. This attribute is used when the string in the **titleKey** attribute is not found in the locale file. This is an optional attribute.

titleKey

Specifies the key used to search for the title in the locale file. If it is not defined, the string defined by the **title** attribute is used. This is an optional attribute.

visibleWhen

Defines the conditional display of a tab based on one of three types of expressions comparing a property or preference to a value. The value can be **null** or a string, including a string containing the * wildcard character. Multiple values can be checked with an array property or preference. When checking an array value, use a comma as a delimiter for the values. The three types of expressions check the following:

- The value of a property on the selected object
- The value of a Teamcenter preference
- The value of a property on an object related to the selected object
- To check the value of a property on the selected object, use the real (database) name of the property in the expression.
If you want to show a **"Test"** page if the **object_desc** property begins with the word **Testing**, use the following:

```
<page title="Test" titleKey="tc_xrt_testpage"
      visibleWhen="object_desc==Testing*">
```

The following examples show **"My Page"** based on the value of a property called **myProp**. Display the page if **myProp** contains "test".

```
<page titleKey="My Page" visibleWhen="myProp==*test*">
```

Display the page if **myProp** does *not* contain "test".

```
<page titleKey="My Page" visibleWhen="myProp!=*test*">
```

Display the page if **myProp** contains no value (null).

```
<page titleKey="My Page" visibleWhen="myProp==null">
```

- To check the value of a Teamcenter preference, use **{pref:preference-name}** to differentiate it from a property-based expression. Following are some examples:
Display the page when the **TC_Enable_MyPref** preference has no value.

```
<page titleKey="My Page"
      visibleWhen="{pref:TC_Enable_MyPref}==null">
```

Display the page when the **Item_ColumnPreferences** preference contains **object_string** and **object_type**, as the first two values.

```
<page titleKey="My Page"
      visibleWhen="{pref:Item_ColumnPreferences}==object_string,object_type,*">
```

- To check a property on an object *related* to the selected object, you must include the reference or relation property name and the property name from the related object separated by a period.
Display the page when the owner of the selected object is **user1**.

```
<page title="Custom User Page"
      visibleWhen="owning_user.user_id==user1">
```

Display the page when the status of the selected object is **TCM Released**.

```
<page title="Custom Status Page"
      visibleWhen="release_status_list.name==TCM Released">
```

Display the page when two specific statuses are present on the object — both **TCM Released** and **Approved**.

```
<page title="Special Status Page"
      visibleWhen="release_status_list.name==TCM Released,Approved">
```

Display the page when there is a PDF dataset attached with a specification relation.

```
<page title="The PDF Page"
      visibleWhen="IMAN_specification.object_type==*PDF*">
```

Note:

If there is only one page, and the **visibleWhen** condition hides this page, the rich client ignores this condition and makes the page visible.

If you specify a reference property but you do not specify a property on the related object, the default value will be the secondary object's localized value - typically object_string.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

Property
Summary
Form
Create

EXAMPLES

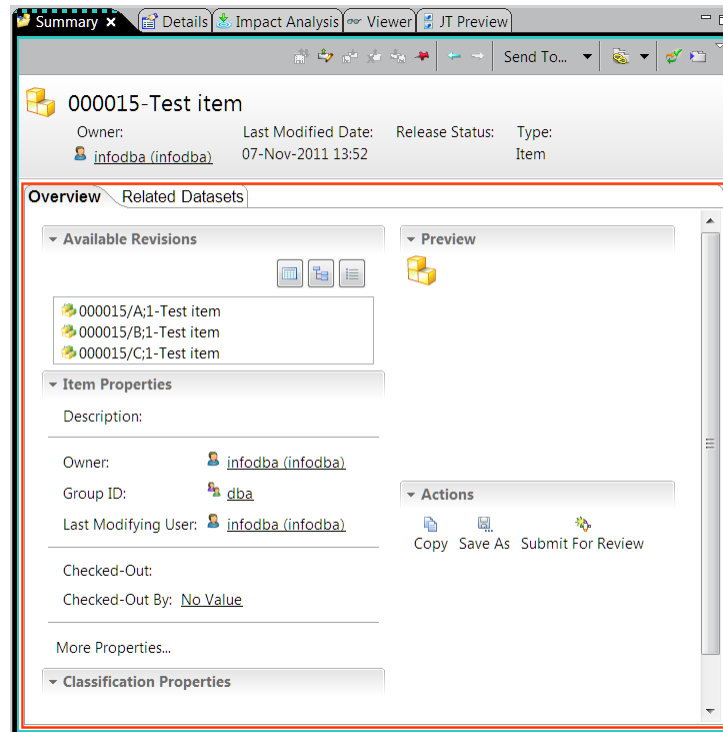
- **page** element
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **page** element:

```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaulttdisplay="listDisplay"
sortdirection="descending" sortBy="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
        </treeDisplay>
      </objectSet>
    </section>
  </column>
</page>
```

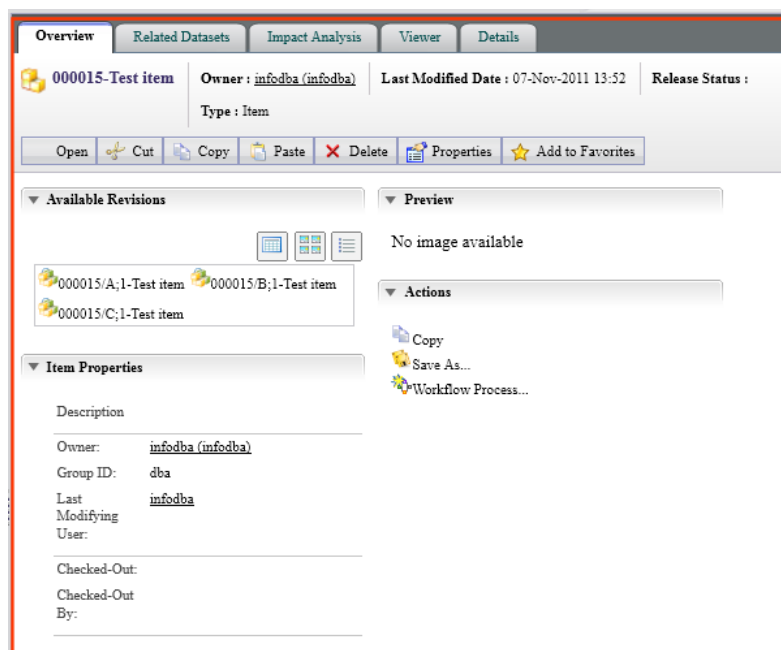
```

        <property name="last_mod_user"/>
        <property name="checked_out_user"/>
    </treeDisplay>
    <listDisplay/>
</objectSet>
</section>
<section titleKey="tc_xrt_ItemProperties">
    <property name="object_desc"/>
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
    <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
    <property name="last_mod_user"/>
    <separator/>
    <property name="checked_out"/>
    <property name="checked_out_user"/>
    <separator/>
    <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
<section titleKey="tc_xrt_ClassificationProperties">
    <classificationProperties/>
</section>
</column>
<column>
    <section titleKey="tc_xrt_Preview">
        <image source="preview"/>
    </section>
    <section titleKey="tc_xrt_actions" commandLayout="vertical">
        <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
        <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
        <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess" titleKey="tc_xrt_newProc" />
    </section>
</column>
</page>

```

page element in the rich client



page element in the thin client

- **visibleWhen** attribute
To specify a single conditional evaluation for the component property, include the **visibleWhen** parameter on the property style sheet page, for example, **visibleWhen="object_desc!=abc"**. Consider the following code:

```
<page title="Reservation" titleKey="tc_xrt_Reservation"
visibleWhen="object_desc==Testing*">
...
</Page>
```

If the word **Testing** is used in the **object_desc** property, the **Reservation** link appears. In the following example, the **object_desc** property is blank, and therefore the **Reservation** link does not appear.

Condition is not met for the visibleWhen parameter

Now change the description value to **Testing my code**; the **Reservation** link appears after you save your changes, as shown in the following figure.

Condition is valid for the visibleWhen parameter

parameter

Passes in the name/value parameters to the parent command. This is a child element of the **command** element. An example of a parameter is **localSelection**.

ATTRIBUTES

name

Specifies the parameter name, for example, **searchName**. This is a required attribute.

value

Specifies the parameter value, for example, **CustomSearch**. This is a required attribute.

SUPPORTED CLIENTS

Rich client

Thin Client

SUPPORTED STYLE SHEETS

This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **parameter** element:

```
<command actionKey="pasteAction" commandId="com.teamcenter.rac.viewer.pastewithContext"
renderingHint="commandbutton"/>
<command actionKey="cutAction" commandId="org.eclipse.ui.edit.cut"
renderingHint="commandbutton">
    <parameter name="localSelection" value="true"/>
</command>
```

Note:

When you define a custom command for the rich client that uses parameters, you must add the parameters with the **commandParameter** tag in the **plugin.xml** file of the custom plug-in, for example:

```
<extension
    id="com.myco.command.parameter.test.commands.category"
    name="Test Category"
    point="org.eclipse.ui.commands">
    <command
        name="Test command"
        categoryId="com.teamcenter.ddp.commands.category"
        id="com.myco.command.parameter.test.commands.testCommand">
        <commandParameter id="localStage1" name="localStage1"
optional="true" />
        <commandParameter id="localStage2" name="localStage2"
optional="true" />
        <commandParameter id="localStage3" name="localStage3"
optional="true" />
        <commandParameter id="localStage4" name="localStage4"
optional="true" />
        <commandParameter id="source" name="source" optional="true" />
    </command>
    </extension>
<extension
    point="org.eclipse.ui.handlers">
    <handler
        commandId="com.myco.command.parameter.test.commands.testCommand"
        class="com.myco.command.parameter.test.handlerTest">
    </handler>
</extension>
```

When you add the custom command in a stylesheet, the parameters defined in the stylesheet must match the **commandParameter** values defined in the plug-in, for example:

```
<command commandId="com.myco.command.parameter.test.commands.testCommand"
renderingHint="commandbutton">
    <parameter name="localStage1" value="1"/>
    <parameter name="localStage2" value="2"/>
    <parameter name="localStage3" value="3"/>
    <parameter name="localStage4" value="4"/>
</command>
```

If you add only the parameters in the style sheet and do not also add the parameters in the **plugin.xml** file using the **commandParameter** tag, when you execute the command in the rich client interface you receive an error that no parameters are found.

property

Specifies the real (database) name, not the **display name**, of the object property you want to display. You must include at least one property in the XML definition, otherwise, the system displays an empty panel.

Note:

You cannot add the same property multiple times in the same stylesheet.

ATTRIBUTES

border

Determines whether the border is displayed. Valid values are **true** and **false**. This works only with the titled style. This is supported in both the rich client and thin client.

column

Applies only to the **textfield** and **textarea** rendering hints. It sets the number of columns. This is supported in both the rich client and thin client.

modifiable

Specifies if the **owning_user** or **owning_group** property can be modified (**true** or **false**). For all other properties, use a property rule instead.

name

Specifies the database name of a property on the object. This is a required attribute. This is supported in both the rich client and thin client.

Note:

When using this attribute on a **create** style sheet, which is used in the New Business Object wizard, there is additional functionality. You can specify a property from another object that is related to the original object by the **revision**, **IMAN_master_form**, or **IMAN_master_form_rev** relations. To do this, specify the relation trail followed by the name of the property on the destination related object, separated by a colon.

For example, if a **create** style sheet is registered for an item,

- to display the revision ID, you would use

```
name=revision:item_revision_id
```

- to display the **project_id** property from the item's master form.

```
name=IMAN_master_form:project_id
```

- to display the **serial_number** property from the item revision master form, you need to traverse from the item to the revision, and then to the revision's master form.

```
name=revision:IMAN_master_form_rev:serial_number
```

renderingHint

Specifies the component used to **render** this property. This is an optional attribute. If not defined, the default renderer is used based on the property type.

renderingStyle

Defines the rendering style used in the rendering component. There are three styles: headed, headless, and titled.

- **Headed**

This is the default rendering style. The property name is displayed on the left followed by the property value renderer. This is supported in both the rich client and thin client.

- **Headless**

This style renders only the property value without displaying the property name in front of it. This is supported in both the rich client and thin client.

- **Titled**

The property name is displayed on the top of the property value renderer. This is supported in both the rich client and thin client.

row

Applies only to **textarea** elements. It sets the number of the rows for the element. This is supported in both the rich client and thin client.

style

Controls font style for the label text, including font size, weight, name, and style (such as italic). The format follows the CSS guideline, for example:

```
style="font-size:14pt;font-style:plain;
font-family:Tahoma;font-weight:bold"
```

visibleWhen

Defines the conditional display of a property based on one of two types of expressions comparing a property or preference to a value. The value can be **null** or a string, including a string containing wildcard characters. Multiple values can be checked with an array property or preference. When checking an array value, use a comma as a delimiter for the values. The two types of expressions check the following:

1. The value of a property on the selected object

```
<property name="p1" visibleWhen="<Property name>==<Some value>"/>
```

```
<property name="p1" visibleWhen="<Property name>!=<Some value>"/>
```

```
<property name="p1" visibleWhen="<Property name>==null"/>
```

```
<property name="p1" visibleWhen="<Property name>!=null"/>
```

2. The value of a Teamcenter preference

```
<property name="p1" visibleWhen="{pref:<Preference name>}==<Some value>"/>
```

```
<property name="p1" visibleWhen="{pref:<preference name>}!=<Some value>"/>
```

- To check the value of a property on the selected object, use the real (database) name of the property in the expression.
If you want to show a **"myprop"** property only if the **object_desc** property begins with the word **Testing**, use the following:

```
<property name="myprop" visibleWhen="object_desc==Testing*"/>
```

- To check the value of a Teamcenter preference, use **{pref:preference-name}** to differentiate it from a property-based expression. Following are some examples:
Display a property when the **Cust_Enable_MyProp** preference is set to **true**.

```
<property name="myprop" visibleWhen="{pref:Cust_Enable_MyProp}==true">
```

Note:

Using **visibleWhen** for properties is only for use in SWT-based style sheet views in the rich client. For example, **Summary** (when checked out), **Create**, and **SaveAs**.

It will only work with SWT-based property beans, and will not work with Swing-based property beans.

In the current implementation of SWT styles aheet rendering in the **Summary** view, a plain **Label** instead of a **LabelPropertyBean** is used to display property values in read-only mode (**RenderFlat**). Since the **visibleWhen** framework implementation is based on **AbstractPropertyBeans** and its children, the **visibleWhen** feature will not be available in the **Summary** view in read-only mode. It will only be available when the object is checked-out.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

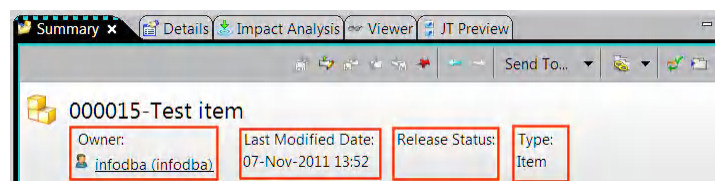
Property
Summary
Form
Create

EXAMPLES

- **property** element

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **property** element:

```
<header>
  <image source="thumbnail"/>
  <classificationTrace/>
  <property name="owning_user"/>
</property name="last_mod_date"/>
<property name="release_status_list"/>
<property name="object_type"/>
</header>
```



property elements in the rich client



property elements in the thin client

- URL rendering

URL addresses in the **label text** and **property** tags are automatically rendered. (In the rich client, URL addresses are also automatically rendered for **textfield** and **textarea** rendering hints.) For an example, see [label](#).

rendering

Root element

ATTRIBUTES

Version

Specifies the version of the XML schema. When an older version is detected, the program automatically converts the old scheme to the new one.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag is required on all types of style sheets:

Property
Summary
Form
Create

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **rendering** element:

```
<rendering xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="XMLRenderingStylesheet_Schema.xsd">
  <header>
    <image source="thumbnail"/>
    <classificationTrace/>
    <property name="owning_user"/>
    <property name="last_mod_date"/>
    <property name="release_status_list"/>
    <property name="object_type"/>
  </header>
  .
  .
  .
</rendering>
```

Rule

Applies the rule to the field if the **GoverningProperty** element matches its conditions. The **Rule** tag must be contained within a **conditions** tag and used in conjunction with a **GoverningProperty** tag.

ATTRIBUTES

propertyname

Specifies the name of the field to require or disable.

state

Indicates whether to make the field required or disabled. The valid values for this attribute are **required** and **disabled**.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets:

Property Form

EXAMPLE

```
<rendering>
  <property name = "user_data_1"/>
  <property name = "user_data_2"/>
  <property name = "user_data_3"/>
  <conditions>
    <GoverningProperty propertyname = "user_data_1" propertyvalue = "Minel">
      <Rule propertyname = "user_data_2" state = "required"/>
    <Rule propertyname = "user_data_3" state = "disabled"/>
    </GoverningProperty>
    <GoverningProperty propertyname = "user_data_1" propertyvalue = "Do IM">
      <Rule propertyname = "user_data_2" state = "disabled"/>
    <Rule propertyname = "user_data_3" state = "required"/>
    </GoverningProperty>
  </conditions>
</rendering>
```

For a user interface example, see [conditions](#).

For another example, see [Set properties to be conditionally mandatory or disabled](#).

secondColumn

Defines the layout of the second column defined on a page. The **firstcolumn** element defines the layout of the first column on a page.

This element applies only if the **TwoColumn** format has been set in the **format** attribute on the **page** element.

Note:

When placing properties in **firstcolumn** and **secondcolumn** elements, you must place the properties elements within the **section** tag. If you do not, the properties do not appear in the thin client user interface.

Caution:

As of Teamcenter 10.1, the **secondcolumn** tag is deprecated and is replaced by the **column** tag.

For example:

```
<page title="General" titleKey="tc_xrt_General" format="TwoColumn">
  <firstcolumn>
    <property name="object_string" column="32"/>
    <separator/>
    <property name="object_name" column="32"/>
    <property name="object_desc" />
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false" />
    <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
    <property name="last_mod_user" />
  </firstcolumn>
  <secondcolumn>
    <image/>
  </secondcolumn>
</page>
```

Is replaced by:

```
<page title="General" titleKey="tc_xrt_General">
  <column>
    <property name="object_string" column="32"/>
    <separator/>
    <property name="object_name" column="32"/>
    <property name="object_desc" />
    <separator/>
    <property name="owning_user" renderingHint="objectlink"
modifiable="false" />
    <property name="owning_group" renderingHint="objectlink"
modifiable="false" />
    <property name="last_mod_user" />
  </column>
  <column>
    <image/>
  </column>
</page>
```

For information about the **column** tag, see *column*.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets (but not the New Business Object wizard):

Property
Summary
Form

EXAMPLE

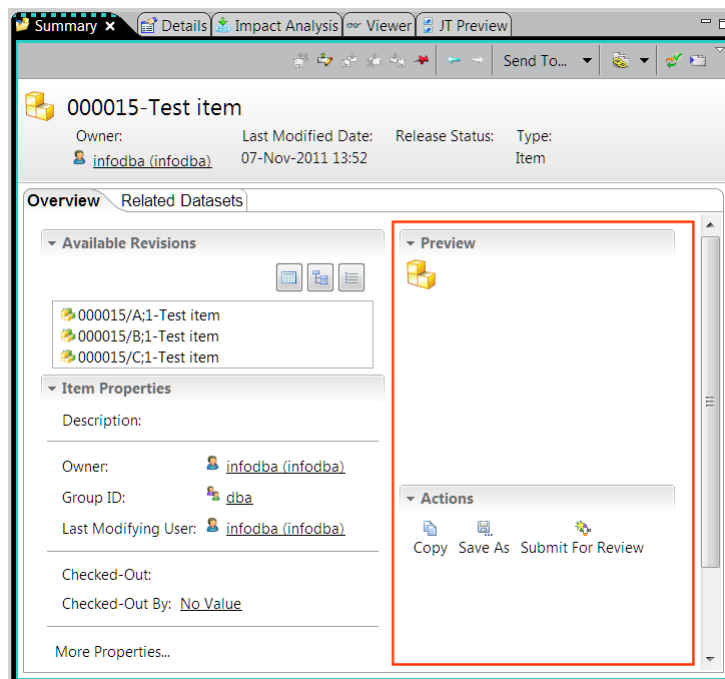
Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **secondcolumn** element:

```
<page text="Overview" format="TwoColumn">
  <firstcolumn>
    <section text="AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="thumbnailDisplay"
sortdirection="descending" sortby="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section text="ItemProperties">
      <property name="object_desc"/>
      <separator/>
      <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
      <property name="owning_group" renderingHint="objectlink"
modifiable="false"/>
      <property name="last_mod_user"/>
      <separator/>
      <property name="checked_out"/>
      <property name="checked_out_user"/>
      <separator/>
      <command commandId="com.teamcenter.rac.properties"
text="moreProperties"/>
    </section>
  </firstcolumn>
  <secondcolumn>
  </secondcolumn>
</page>
```

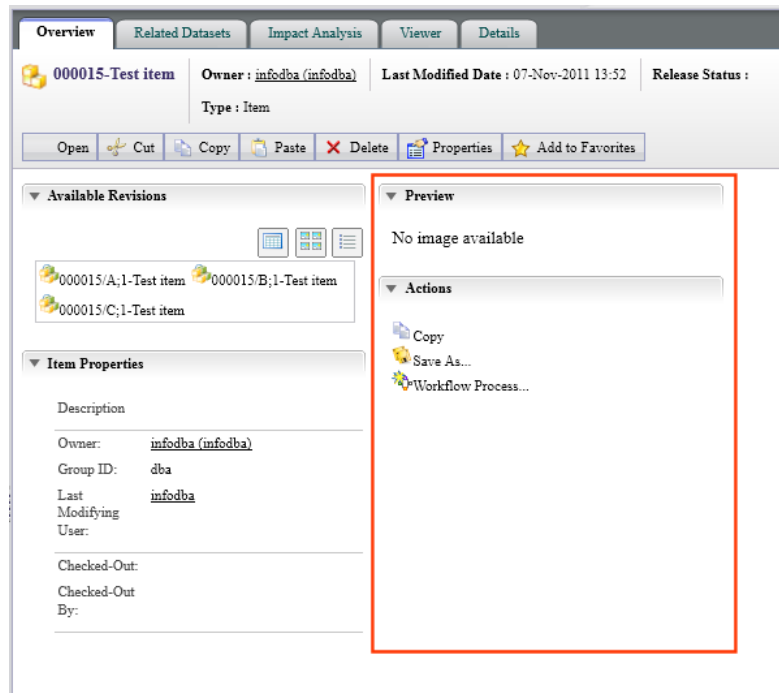
```

</section>
<section text="ClassificationProperties">
  <classificationProperties/>
</section>
</firstcolumn>
<secondcolumn>
  <view name="viewer"/>
  <view name="actions">
    <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
    <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
    <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess"
text="newProc" />
  </view>
</secondcolumn>
</page>

```



secondcolumn element in the rich client



secondcolumn element in the thin client

section

Defines a group of elements to be displayed together within a named section.

Note:

This element is a replacement for the **view** element.

ATTRIBUTES

commandLayout

Controls the layout of commands. Valid values are **horizontal** or **vertical**.

initialstate

Specifies whether the view or section should be expanded or collapsed on initial rendering. Valid values are **expanded** or **collapsed**. The default value is **expanded**. This attribute is optional.

text

Specifies the title to be displayed on the section header. The attribute value must be a key into a property file. If the key is not found, the attribute value itself is displayed as static text. This attribute is required.

Caution:

As of Teamcenter 10.1, the **text** attribute is deprecated and is replaced by the **title** and **titlekey** attributes.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

EXAMPLE

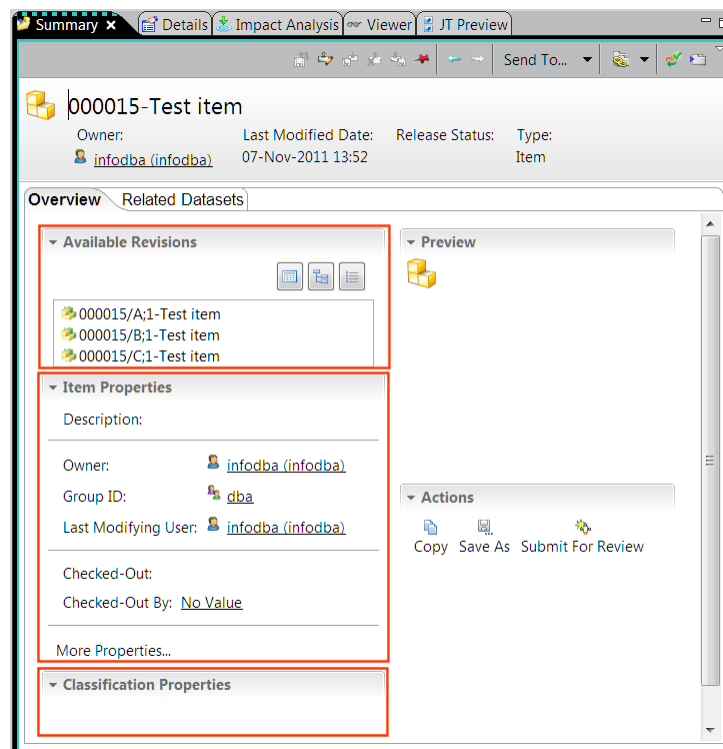
- Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **section** element:

```
<page titleKey="tc_xrt_Overview">
  <column>
    <section titleKey="tc_xrt_AvailableRevisions">
      <objectSet source="revision_list.ItemRevision"
defaultdisplay="listDisplay" sortdirection="descending"
sortby="item_revision_id">
        <tableDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </tableDisplay>
        <thumbnailDisplay/>
        <treeDisplay>
          <property name="object_string"/>
          <property name="item_revision_id"/>
          <property name="release_status_list"/>
          <property name="last_mod_date"/>
          <property name="last_mod_user"/>
          <property name="checked_out_user"/>
        </treeDisplay>
        <listDisplay/>
      </objectSet>
    </section>
    <section titleKey="tc_xrt_ItemProperties">
      <property name="object_desc"/>
      <separator/>
      <property name="owning_user" renderingHint="objectlink"
modifiable="false"/>
      <property name="owning_group" renderingHint="objectlink"
```

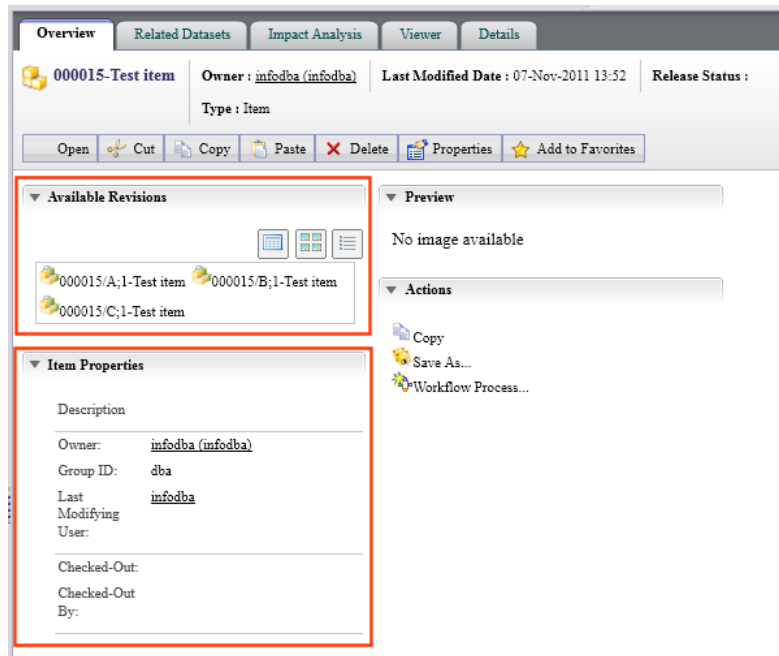
```

modifiable="false"/>
    <property name="last_mod_user"/>
    <separator/>
    <property name="checked_out"/>
    <property name="checked_out_user"/>
    <separator/>
    <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
<section titleKey="tc_xrt_ClassificationProperties">
    <classificationProperties/>
</section>
</column>
<column>
    <section titleKey="tc_xrt_Preview">
        <image source="preview"/>
    </section>
    <section titleKey="tc_xrt_actions" commandLayout="vertical">
        <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
        <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
        <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess" titleKey="tc_xrt_newProc" />
    </section>
</column>
</page>

```



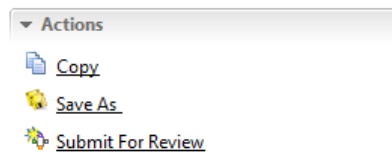
section elements in the rich client



section elements in the thin client

- The following example demonstrates the use of the **commandLayout** attribute. The following code defines the command layout as vertical:

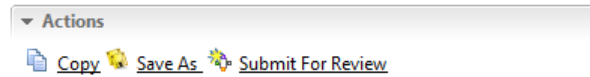
```
<section titleKey="tc_xrt_actions" commandLayout="vertical">
  <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
  <command actionKey="saveAsAction" commandId="org.eclipse.ui.file.saveAs" />
  <command actionKey="newProcessAction" commandId="com.teamcenter.rac.newProcess"
text="newProc" />
</section>
```



Vertical command layout

The following code defines the command layout as vertical:

```
<section titleKey="tc_xrt_actions" commandLayout="horizontal">
  <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
  <command actionKey="saveAsAction" commandId="org.eclipse.ui.file.saveAs" />
  <command actionKey="newProcessAction" commandId="com.teamcenter.rac.newProcess"
text="newProc" />
</section>
```



Horizontal command layout

separator

Adds a separator in the pane.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets and the New Business Object wizard:

Property
Summary
Form
Create

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **separator** element:

```
<section titleKey="tc_xrt_ItemProperties">
  <property name="object_desc"/>
  <separator/>
  <property name="owning_user" renderingHint="objectlink" modifiable="false"/>
  <property name="owning_group" renderingHint="objectlink" modifiable="false"/>
  <property name="last_mod_user"/>
  <separator/>
  <property name="checked_out"/>
  <property name="checked_out_user"/>
  <separator/>
  <command commandId="com.teamcenter.rac.properties"
titleKey="tc_xrt_moreProperties"/>
</section>
```

Item Properties

Description:

Owner:

Group ID:

Last Modifying User:

Checked-Out:

Checked-Out By:

[More Properties...](#)

separator elements in the rich client

Item Properties

Description:

Owner:

Group ID:

Last Modifying User:

Checked-Out:

Checked-Out By:

separator elements in the thin client

tableDisplay

Displays a set of objects in a table format.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

Thin client

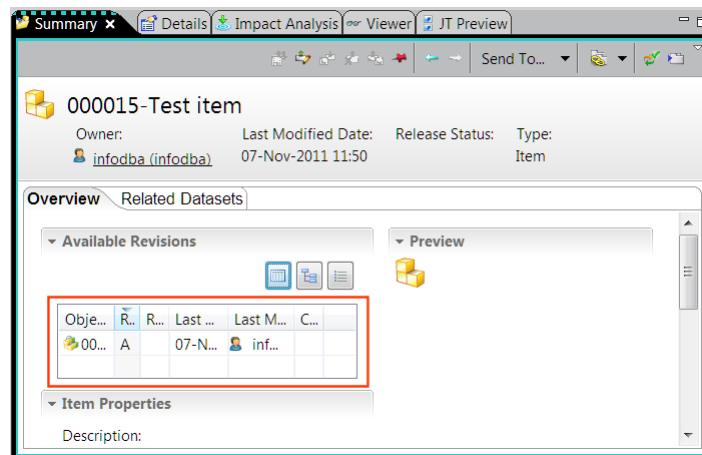
SUPPORTED STYLE SHEETS

This tag can be used on the **Summary** style sheets.

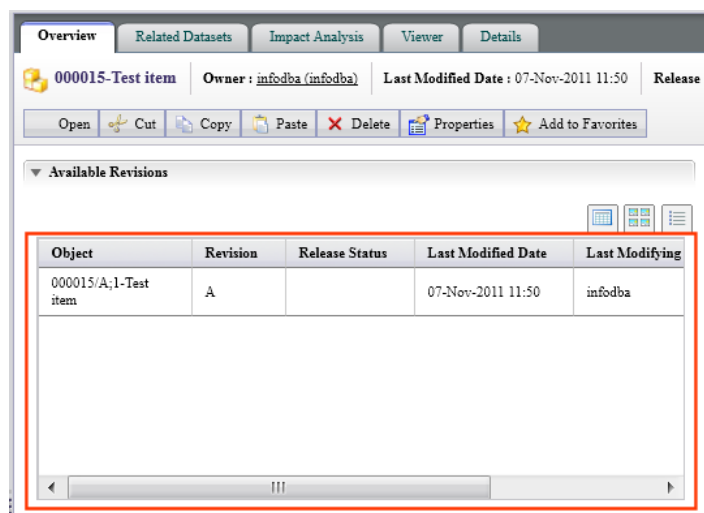
EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **tableDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
sortdirection="descending" sortby="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
    <treeDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </treeDisplay>
    <listDisplay/>
  </objectSet>
</section>
```



tableDisplay element in the rich client



tableDisplay element in the thin client

thumbnailDisplay

Displays a set of objects using thumbnails arranged in a grid.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

SUPPORTED STYLE SHEETS

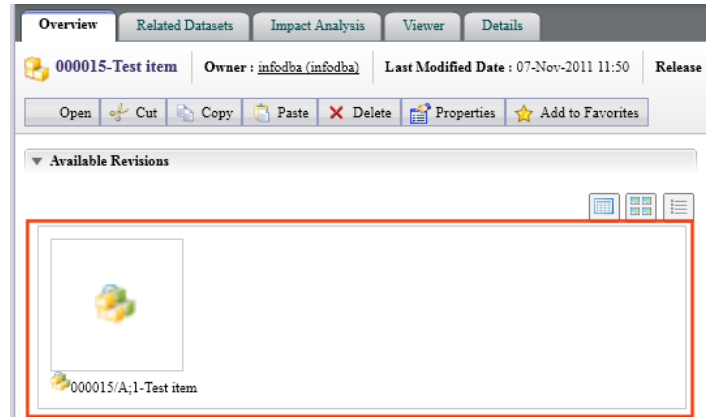
This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **thumbnailDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
    sortdirection="descending" sortBy="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
  </objectSet>
</section>
```

```
<thumbnailDisplay/>
<treeDisplay>
  <property name="object_string"/>
  <property name="item_revision_id"/>
  <property name="release_status_list"/>
  <property name="last_mod_date"/>
  <property name="last_mod_user"/>
  <property name="checked_out_user"/>
</treeDisplay>
<listDisplay/>
</objectSet>
</section>
```



thumbnailDisplay element in the thin client

The **thumbnailDisplay** element is not supported in the thin client. The **treeDisplay** element is rendered instead.

treeDisplay

Displays a set of objects in a tree format.

ATTRIBUTES

None.

SUPPORTED CLIENTS

Rich client

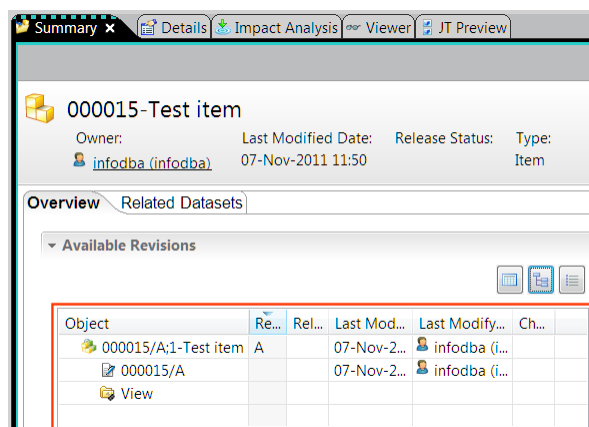
SUPPORTED STYLE SHEETS

This tag can be used on **Summary** style sheets.

EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **treeDisplay** element:

```
<section titleKey="tc_xrt_AvailableRevisions">
  <objectSet source="revision_list.ItemRevision" defaultdisplay="listDisplay"
    sortdirection="descending" sortBy="item_revision_id">
    <tableDisplay>
      <property name="object_string"/>
      <property name="item_revision_id"/>
      <property name="release_status_list"/>
      <property name="last_mod_date"/>
      <property name="last_mod_user"/>
      <property name="checked_out_user"/>
    </tableDisplay>
    <thumbnailDisplay/>
  <treeDisplay>
    <property name="object_string"/>
    <property name="item_revision_id"/>
    <property name="release_status_list"/>
    <property name="last_mod_date"/>
    <property name="last_mod_user"/>
    <property name="checked_out_user"/>
  </treeDisplay>
  <listDisplay/>
</objectSet>
</section>
```



treeDisplay element in the rich client

The **treeDisplay** element is not supported in the thin client. The **thumbnailDisplay** element is rendered instead.

view

Defines a group of properties to be displayed together. If there are **firstcolumn** and **secondcolumn** elements, this element must be within them.

Caution:

As of Teamcenter 10.1, the **view** tag is deprecated and is replaced by the **section** tag.

For example:

```
<view name="viewer"/>
  <view name="actions">
    <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
    <command actionKey="saveAsAction" commandId="org.eclipse.ui.file.saveAs" />
    <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess"
text="newProc"
  />
  </view>
```

Is replaced by:

```
<section titleKey="tc_xrt_viewer">
  <image source="preview"/>
</section>
<section titleKey="actions" commandLayout="vertical">
  <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
  <command actionKey="saveAsAction" commandId="org.eclipse.ui.file.saveAs" />
  <command actionKey="newProcessAction" commandId="com.teamcenter.rac.newProcess"
text="newProc" />
</section>
```

For information about the **section** tag, see *section*.

ATTRIBUTES

name

Specifies the display name of the view. This name can be localized in the **textserver** files. This is a required attribute. The following view names are valid: **properties**, **viewer**, **impactanalysis**, **actions**, and **attachments**.

SUPPORTED CLIENTS

Rich client

Thin client

SUPPORTED STYLE SHEETS

This tag can be used on the following types of style sheets and the New Business Object wizard:

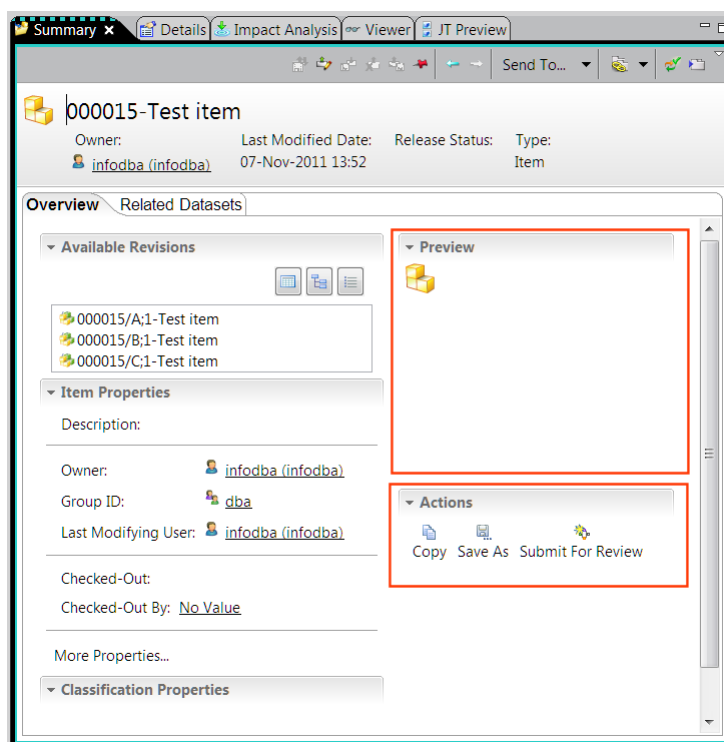
**Property
Summary
Form**

Create

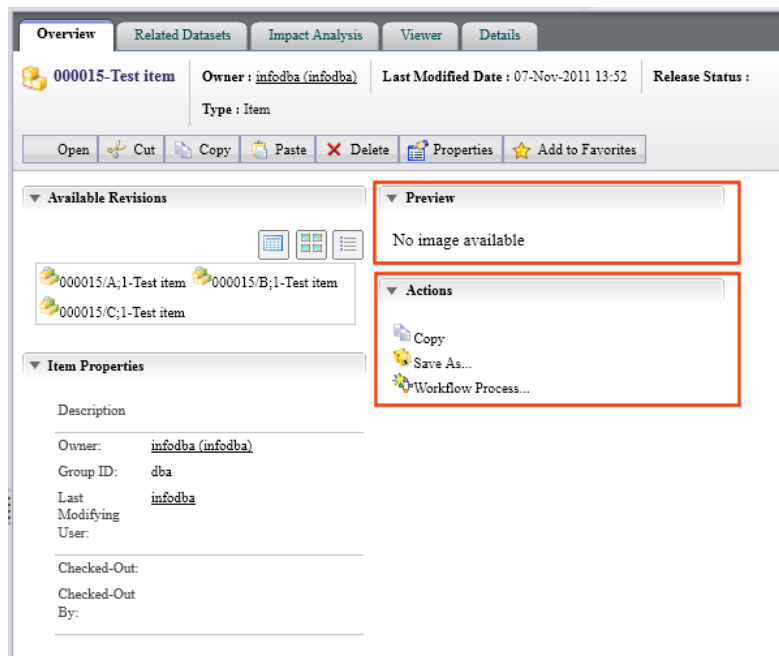
EXAMPLE

Following is sample code from the **ItemSummary.xml** XML rendering style sheet showing the **view** element:

```
<secondcolumn>
<view name="viewer"/>
<view name="actions">
  <command actionKey="copyAction" commandId="com.teamcenter.rac.copy" />
  <command actionKey="saveAsAction"
commandId="org.eclipse.ui.file.saveAs" />
  <command actionKey="newProcessAction"
commandId="com.teamcenter.rac.newProcess"
text="newProc" />
</view>
</secondcolumn>
```



view elements in the rich client



view elements in the thin client

Rendering hints

Standard rendering hints

The **renderingHint** is an attribute on a **<property>** tag that allows you to specify which user interface widget to use to present the corresponding property. It is an optional attribute, and if not defined, the default rendering is used based on the property type.

The JavaBeans used depend on the specified rendering hint and rendering style. The following table lists the JavaBeans used based on the hint and style definition. All rendering hints are supported in the rich client. If the rendering style is not defined, the default style is **headed**. There is no titled style for SWT.

For standard rendering hints, see the **com.teamcenter.rac.viewer/plugin.xml** file for the SWT version of rendering hints, and see the **com.teamcenter.rac.common/plugin.xml** file for the Swing version of rendering hints. Siemens PLM Software recommends you use SWT rather than Swing, because Swing is being phased out in favor of SWT.

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
array	Used for array only: character, date, double, float, integer, logical, short, string, note,	Yes – date, string (text area), string LOV, typed reference	Headed or headless SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.)

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
	typed reference, typed relation.	(UID) LOV are supported	- Swing: PropertyArray Titled (Swing only) TitledPropertyArray
checkbox	Logical.	Yes	Headed or headless - SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) - Swing: PropertyCheckbox Titled (Swing only) TitledPropertyCheckbox
checkboxoptionlov	Used with LOV only: character, date, double, float, integer, short, string, note, typed ref, typed relation.	Yes – Rendered using same component as string LOV array	Headed or headless - SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) - Swing: PropertyCheckboxOptionLov Titled (Swing only) TitledPropertyCheckboxOptionLov
datebutton	Date.	Yes – Rendered as calendar button .	Headed or headless - SWT: DateControlPropertyBean - Swing: PropertyDateButton Titled (Swing only)

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
label Note: As of Teamcenter 9.1, the button rendering hint and the PropertyButton bean and TitledPropertyButton bean are deprecated and are no longer supported. Use the label hint instead.	All types. Used for display only; you cannot change the value.	Yes	• TitledPropertyDateButton Headed or headless • SWT: LabelPropertyBean • Swing: PropertyLabel Titled (Swing only) • TitledPropertyLabel
localizablearray	String.	No	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyLocalizableArray Titled (Swing only) • Not applicable.
localizablelovuicomp As of Teamcenter 10.1, this rendering hint is deprecated and replaced by the lovuicomp and lovuicomp_titled rendering hints.	String.	No	Headed or headless • SWT: LocalizedLOVUICompPropertyBean • Swing: PropertyLocalizableLOVUIComponent Titled (Swing only) • TitledPropertyLOVUIComponent

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
localizabletextarea	String.	No	Headed or headless • SWT: LocalizedTextAreaPropertyBean • Swing: PropertyLocalizableTextArea Titled (Swing only) • Not applicable.
localizabletextfield	String.	No	Headed or headless • SWT: LocalizedTextfieldPropertyBean • Swing: PropertyLocalizableTextField Titled (Swing only) • Not applicable.
localizablelongtextpanel	String, note.	Yes – Rendered as text area	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyLocalizableLongTextPanel Titled (Swing only) • Not applicable.
logical	Logical.	No	Headed or headless • SWT: LogicalPropertyBean • Swing: PropertyLogicalPanel

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
			Titled (Swing only)
			• TitledPropertyLogicalPanel
longtext	Use only for string type when the maximum length is bigger than 2500.	Yes – Rendered as textarea .	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyLongText
			Titled (Swing only)
			• TitledPropertyLongText
lovbutton	Used with LOV only: character, date, double, float, integer, logical, short, string, note, typed ref, typed relation.	Yes – Rendered as regular LOV	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyLOVButton
As of Teamcenter 10.1, this rendering hint is deprecated and replaced by the lovuicomp and lovuicomp_titled rendering hints.			Titled (Swing only)
			• TitledPropertyLOVButton
lovcombobox	Used with LOV only: character, date, double, float, integer, logical, short, string, note, typed ref, typed relation.	Yes – Rendered as regular LOV	Headed or headless • SWT: LOVComboBoxPropertyBean • Swing: PropertyLOVCombobox
As of Teamcenter 10.1, this rendering hint is deprecated and replaced by the lovuicomp and lovuicomp_titled rendering hints.			Titled (Swing only)
	The column="name" argument does not apply for the		• TitledPropertyLOVCombobox

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
	lovcombobox rendering hint. You cannot use the lovcombobox rendering hint in cascading or hierarchical LOVs. You can only use the lovuicomp rendering hint.		
lovuicomp	Used with LOV only: character, date, double, float, integer, logical, short, string, note, typed ref, typed relation.	No The rich client only supports this rendering hint for LOV display.	Headed or headless • SWT: LOVUIComponentPropertyBean • Swing: PropertyLOVUIComponent Titled (Swing only) • TitledPropertyLOVUIComponent
objectlink	Typed ref, typed relation.	Yes – Rendered as a text field	Headed or headless • SWT: ObjectLinkPropertyBean • Swing: PropertyObjectLink Titled (Swing only) • TitledPropertyObjectLink
panel	Unused.	Yes – Rendered using separators	Headed or headless • SWT: Not applicable. • Swing: PropertyPanel

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
			Titled (Swing only)
			• TitledPropertyPanel
radiobutton	Character, date, double, float, integer, logic, short, string, note.	Yes	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyRadioButton
			Titled (Swing only)
			• TitledPropertyRadioButton
radiobuttonoptionlov	Used with LOV only: character, date, double, float, integer, short, string, note, typed ref, typed relation.	Yes – Rendered using same component as string LOV array	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyRadioButtonOptionLov
			Titled (Swing only)
			• TitledPropertyRadioButtonOptionLov
slider	SWT	No	Headed or headless • SWT: SliderPropertyBean • Swing: PropertySlider
			Titled (Swing only)
			• TitledPropetySlider

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
		T h e S W T b e a n d o e s n o t s u p p o r t o t h e r d a t a t y p e s .	
	Swing	C h a r a	

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
----------------	--------------------------	---------------------------------	-----------

		c t e r , d o u b l e , f l o a t , i n t e g e r , s h o r t , n o t e , s t r i n g .	
--	--	--	--

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
		For property types except in the generated standard short , this attribute is not recommended	

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
		n v e r t s t h e p r o p e r t y v a l u e t o a n i n t e g e r . T h e v a l u e d e f	

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
		a u l t s t o O i f a n y e r r o r s o c c u r .	
styledtextarea	Character, date, double, float, integer, logic, short, string, note.	Yes	Headed or headless - SWT: StyledTextAreaPropertyBean - Swing: PropertyStyledTextArea Titled (Swing only) TitledPropertyStyledTextArea
styledtextfield	Character, date, double, float, integer, logic, short, string, note.	No	Headed or headless - SWT: StyledTextfieldPropertyBean - Swing:

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
			PropertyStyledTextField
			Titled (Swing only)
			• TitledPropertyStyledTextField
textarea	Character, date, double, float, integer, logic, short, string, note.	Yes	Headed or headless • SWT: TextAreaPropertyBean • Swing: PropertyTextArea
			Titled (Swing only)
			• TitledPropertyTextArea
textfield	Character, date, double, float, integer, logic, short, string, note.	Yes	Headed or headless • SWT: TextfieldPropertyBean • Swing: PropertyTextField
			Titled (Swing only)
			• TitledPropertyTextField
togglebutton	Character, date, double, float, integer, logic, short, string, note.	No – Rendered as checkbox	Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyToggleButton
			Titled (Swing only)

Rendering hint	Use on property types	Supported in thin client?	JavaBeans
togglebuttonoptionlov	Used with LOV only: character, date, double, float, integer, short, string, note, typed ref, typed relation.	Yes – Rendered using same component as string LOV array	• TitledPropertyToggleButton Headed or headless • SWT: Not applicable. (Currently using LegacyPropertyBridgeBean.) • Swing: PropertyToggleButtonOptionLov Titled (Swing only) • TitledPropertyToggleButtonOptionLov

Add a custom rendering hint

The **renderingHint** attribute on the **property** tag allows you to specify which user interface widget to use to present the corresponding property. In addition to the supported standard rendering hints, you can also add your own custom rendering hints.

For the standard rendering hints, see the **com.teamcenter.rac.viewer/plugin.xml** file for the SWT version of rendering hints, and see the **com.teamcenter.rac.common/plugin.xml** file for the Swing version of rendering hints.

The method of adding a custom rendering hint is different depending on whether you are using the SWT or Swing property bean.

- **SWT**
The SWT property bean is used by the **Summary** view, the **New→Other** wizard, and the **Viewer** view.
1. Create a custom plug-in that uses the **com.teamcenter.rac.common.renderingHint** extension point.
 2. Create a Java class by extending from the **AbstractPropertyBean** bean. Following is a simple example of custom property bean:

```
<extension point="com.teamcenter.rac.common.renderingHint">
  <renderingHint
    id="mytextfield" priority="0">
    <propertyBean
      class="com.teamcenter.rac.viewer.stylesheet.beans.MyTextfieldPropertyBean">
```

```

        </propertyBean>
    </renderingHint>
</extension>

```

- Swing

The Swing property bean is used by the **Properties** dialog box and the **Viewer** view. There are two ways you can implement the Swing property bean:

- Add the new rendering hint definition to a custom **stylesheet_user.property** file. Each rendering hint has a key for the class path of the Java bean defined in the **com.teamcenter.rac.stylesheet.properties** file, found in the **com.teamcenter.rac.common** JAR file. You can plug in your own bean by overwriting the entry in the properties file to replace the default Java bean, or you can add new entries for custom Java beans. The key has the following format for headed or headless beans:

```
rendering-hint.DEFINITION
```

The following format is for titled beans.

```
rendering-hint_titled.DEFINITION
```

For example:

```

textfield.DEFINITION=com.teamcenter.rac.stylesheet.
    PropertyTextField

textfield_titled.DEFINITION=com.teamcenter.rac.stylesheet.
    TitledPropertyTextField

```

- Use the **com.teamcenter.rac.common.renderingHint** extension point.
 1. Create a custom plug-in that uses the **com.teamcenter.rac.common.renderingHint** extension point.
 2. Create a Java class by extending from the **InterfacePropertyComponent** component. Note the difference from the SWT version, because it is using the **legacyPropertyBean** element for the class in the extension definition:

```

<extension point="com.teamcenter.rac.common.renderingHint">
    <renderingHint id="mytextfield" priority="0">
        <legacyPropertyBean
            class="com.teamcenter.rac.stylesheet.MyPropertyTextField"/>
    </renderingHint>
</extension>

```

After you finish creating the Java code, add the new package or class to your custom plug-in **export-package** list in the **MANIFEST.MF** file, for example:

```
Export-Package: com.teamcenter.rac.viewer.customplugin.bean
```


Following is an example Java class that creates a custom SWT rendering hint:

```
package com.teamcenter.rac.viewer.customplugin.beans;
import com.teamcenter.rac.common.controls.LOVComboBox;
import com.teamcenter.rac.kernel.TCPropertyDescriptor;
import com.teamcenter.rac.viewer.stylesheet.beans.LOVComboBoxPropertyBean;
import java.util.Map;
import org.eclipse.core.runtime.CoreException;
import org.eclipse.core.runtime.IConfigurationElement;
import org.eclipse.core.runtime.IExecutableExtension;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.ui.forms.widgets.FormToolkit;
/**
 *
 * Constructor
 *
 * @param lovCombo
 */
public class MyLOVComboBoxPropertyBean
    extends LOVComboBoxPropertyBean implements IExecutableExtension
{
    /**
     *
     * Constructor
     *
     * @param toolkit FormToolkit to use to create UI widgets
     * @param parent composite
     * @param renderFlat Flag indicating it's read only or modifiable
     * @param paramTable Map of attributes which in stylesheet file.
     * @published
     */
    public MyLOVComboBoxPropertyBean( LOVComboBox lovCombo )
    {
        super( lovCombo );
    }
    /**
     * Constructor
     *
     * @param toolkit FormToolkit to use to create UI widgets
     * @param parent composite
     * @param renderFlat Flag indicating it's read only or modifiable
     * @param paramTable Map of attributes which in stylesheet file.
     * @published
     */
    public MyLOVComboBoxPropertyBean( FormToolkit toolkit, Composite parent,
        boolean renderFlat, Map paramTable )
    {
        super(toolkit, parent, renderFlat, paramTable);
    }
    @Override
    public void load( TCPropertyDescriptor desc )
        throws Exception
    {
        super.load( desc );
        System.out.println("This is mylovcombobox.");
        // force it to load the LOVs
        lovComboBox.initializeData(false);
    }
    @Override
    public void setUIFValue( final Object value )
    {
        lovComboBox.getDisplay().asyncExec( new Runnable()
        {
            public void run()
            {
                if ( value != null )
                {
                    lovComboBox.setSelectedItem( value );
                }
            }
        } );
    }
}
```

```

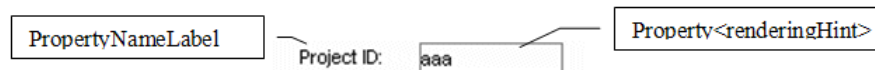
        }
        else
        {
            lovComboBox.setText(""); //$NON-NLS-1$
        }
    }
} );
}
@Override
public void setInitializationData( IConfigurationElement config,
    String propertyName, Object data )
    throws CoreException
{
    //
}
}

```

Rendering style

Each type of renderer supports three styles: headed, headless, and titled.

- **Headed**
Displays the property name on the left followed by the property value renderer. This is the default rendering style.
This style has two components, the **PropertyNameLabel** JavaBean for the property name and the **PropertyrenderingHint** JavaBean for the renderer (for example, **PropertyTextField** or **PropertyTextArea**).



Headed rendering style

- **Headless**
Renders only the property value without displaying the property name.
This style contains only one JavaBean, **PropertyrenderingHint**.
- **Titled**
Displays the property name above the property value renderer.
This style uses only the **TitledPropertyrenderingHint** JavaBean, for example **TitledPropertyTextField** or **TitledPropertyTextArea**.



Titled rendering style

Default renderers

The following table displays the default renderer for each type. If the rendering hint is not provided, the default renderer is used.

Type	Default renderer
string	textfield if size < 60 textarea if 60 < size < 2500 longtext if size >= 2500
char	textfield
double	textfield
float	textfield
int	textfield
short	textfield
long	textfield
date	datebutton
logical	logical
note	textfield if size < 60 textarea if size > 60
TypedReference/ UntypedReference	objectlink
All array types	array
Any property with an LOV attached	lovuicomp In turn, the lovuicomp renderer uses the lovcombox renderer if the number of list of values does not exceed 500, and if there are more, the lovpopupbutton renderer is used.

Set properties to be conditionally mandatory or disabled

You can use the **conditions** tag to configure rules on forms to make properties be conditionally mandatory or disabled on the rich client and thin client. Teamcenter clients implement specified rules locally using XML rendering style sheets without modifying the underlying server side metamodel.

Note:

Style sheets are inherited by form types even where they have not been set.

1. Create a new **XMLRenderingStylesheet** dataset using the rich client.

Note:

If you create a style sheet manually that uses **conditions** tags and attach it to the **XMLRenderingStylesheet** dataset as a named reference, the style sheet must use UTF-8 encoding. If you use the style sheet viewer to create your dataset, it automatically uses UTF-8.

2. Select the **Viewer** pane.

The XML style sheet is displayed in the viewer.

3. Edit the style sheet to add your conditions between the **rendering** tags. You must include rendering tags in the style sheet. If you use conditions, you cannot use default rendering.

The following example makes the **user_data_2** property required if the **user_data_1** property is set to **axle** and disabled if the **object_desc** property is set to **part**:

```
<conditions>
  <GoverningProperty propertyname = "user_data_1" propertyvalue = "axle">
    <Rule propertyname = "user_data_2" state = "required"/>
  </GoverningProperty>
  <GoverningProperty propertyname = "object_desc" propertyvalue = "part">
    <Rule propertyname = "user_data_2" state = "disabled"/>
  </GoverningProperty>
</conditions>
```

The following rules apply to the XML code:

- You can add more than one **Rule** tag to a **GoverningProperty** tag. If the **GoverningProperty** is true, all of its **Rule** tags are applied.

Caution:

The **GoverningProperty** tag is not supported in the create style sheet or the summary style sheet.

- If there is a data or format error in the XML code, no error message is displayed, the conditions are not applied, and the form uses default functionality.
- If you specify conflicting or circular rules, Teamcenter does not resolve the problem. The rules are applied, and no error message is displayed.
A circular rule specifies that field 1 makes field 2 disabled and field 2 makes field 1 disabled. A conflicting rule specifies that a field is both required and disabled.

4. Select **Form** in the **Stylesheet Type** list and select the appropriate object from the **Registered Type** list.
5. Click the **Apply** button.

6. Select the form and select the **Viewer** pane.

The conditions are applied to the form. If you set a property that triggers the rule, the rule is applied after you move out of the property field (in other words, click or tab outside the field).

3. Rich client customization

Introduction to rich client customization

The Teamcenter rich client is based on a client-server architecture. Both the client and server layers can be customized. The rich client is the user interface (UI) layer and is built and customized using the Java language. The server layer can be customized using the Integration Toolkit (ITK) and the C++ programming language.

Because the rich client is based on Eclipse, your customizations have access to all Eclipse extension points and OSGi services. They can also use Teamcenter-developed customization techniques. To customize the rich client, you can use:

- Base Eclipse extension points and services (for example, the **org.eclipse.ui.menus** extension point).
- Teamcenter extension points and services (for example, the application extension points).
- Teamcenter customization mechanisms (for example, style sheets).

The basic customization technique is to create a plug-in that contains the customizations and deploy the custom plug-in to the rich client install. To effectively customize the rich client, you must be comfortable working with Eclipse.

Basic concepts about rich client customization

What are perspectives and views?

Within the rich client user interface, application functionality is provided in *perspectives* and *views*. Some applications use perspectives and views to arrange how functionality is presented. Other applications use a single perspective and view to present information.

Note:

Your administrator can use the **HiddenPerspectives** preference to prevent the display of some Teamcenter perspectives in the rich client.

If your site has online help installed, you can access application and view help from the rich client **Help** menu or by pressing F1.

Understanding the Eclipse rich client platform framework

When the rich client is installed, Java archive (JAR) files are installed that are Eclipse plug-in files. They are located in the `TC_ROOT\portal\plugins` directory, and the file names start with **com.teamcenter**.

These files comprise rich client and the resources required to run it. When you add a new custom plug-in, you deploy it into this directory.

The Teamcenter rich client is hosted within the Eclipse rich client platform (RCP) framework. The RCP is a general purpose application framework which provides strong support for modular and extensible component-based development through the use of plug-ins.

You can find more information about the RCP's features, advantages, and use at the Eclipse Web site:

<http://www.eclipse.org/>

For more information about using Eclipse, see the *Platform Plug-in Developer Guide*, which can be found at the following location:

<http://help.eclipse.org/indigo/index.jsp>

To customize the RCP, you need to use the Eclipse IDE.

Adding menus and toolbars

To add menu bars, toolbars, and shortcut menus, use the declarative approach provided by Eclipse. The definition of the menu bar, toolbar, and context menus are provided in the individual plug-in's **plugin.xml** file. Menus and the resulting application logic they call can be placed in a Model-View-Controller (MVC) paradigm. The three parts to the MVC paradigm are:

- **Command**
Command has a globally unique ID and represents the abstract semantic concept of a behavior, such as copy, paste, and save. A command is *not* the implementation of that behavior nor is it the visual representation of that behavior.

```
<command id="com.teamcenter.rac.command"
        name="%com.teamcenter.rac.command.name">
</command>
```

- **Menu contributions**
Menu contributions represent a particular view or visual representation of a command. The menu contributions create the menu and toolbar structures and insert them into the correct Eclipse location. The location is specified as an Uniform Resource Identifier (URI) and can be any one of the following:

- Main menu
- Main toolbar
- View toolbar
- View menu

- Context (popup) menu
- Trim area

The menu contribution can define a menu label, mnemonic, or icon. It contains visual references to already defined commands. The visual representations of commands may include labels, icons, and mnemonics. Menu contributions also may include separators. Separators are only visible if there are visible commands before and after a separator. The menu contribution can define when it will be visible with a **visibleWhen** clause. The **visibleWhen** clause refers to all standard Eclipse expressions. This expression can be very simple or very complex and evaluates to either **true** or **false** which determines whether a menu is visible or not.

```
<menuContribution locationURI="menu:org.eclipse.ui.main.menu">
  <menu id="file" label="%menu.file.label" mnemonic="%menu.file.mnemonic">
    <command commandId="org.eclipse.ui.file.refresh"
      mnemonic="%command.refresh.mnemonic"
      style="push">
    </command>
    <separator name="sep1" visible="true"/>
    <command commandId="org.eclipse.ui.file.exit"
      mnemonic="%command.exit.mnemonic"
      style="push">
    </command>
  </menu>
</menuContribution>
```

Note:

You can define the icon for a menu toolbar item by specifying a URL. The URL is case sensitive (for example, **icon.PNG** is not the same as **icon.png**). If you give an incorrect icon URL, a warning is logged to the log file and the **Console** view, if displayed. The menu or toolbar item with the incorrect URL definition is not visible until it is corrected and the rich client is restarted.

If you run in development mode inside the IDE and pull the icon files from a source folder, Windows is not case sensitive and finds the icons. However, once you run the rich client from the command line where the icons are packaged in the plug-in's JAR file, the Java API does not find the icons since those APIs are case sensitive.

Always verify the case is correct for image icons.

A command can also be bound to a key sequence using the **org.eclipse.ui.bindings** extension point.

- Handler
A *handler* implements one particular behavior for a given command. For any given command, there can be zero or several handlers defined. However only none or one handler may be active for a given command. The active handler controls the command's enabled state. Handlers most commonly extend the **AbstractHandler** class. Handlers are provided an application context in their **execute(*)** method. If a command has no active handlers defined, any menu contributions defined for a command are not visible. A command can also define a default handler ensuring that a command always has an active handler. The handler can be declaratively activated via the **ActiveWhen** clause or programmatically activated. The handler also defines declaratively when a

command appears enabled in any menu contribution with the **enabledWhen** expression for the handler.

For more information and full examples about how to use the Eclipse declarative approach to menus and toolbars, see the following links:

- http://wiki.eclipse.org/Menu_Contributions
- http://wiki.eclipse.org/Platform_Command_Framework
- http://wiki.eclipse.org/Command_Core_Expressions
- <http://www.vogella.de/articles/EclipseCommands/article.html>

Introduction to SWT

The Standard Widget Toolkit (SWT) is a graphical toolkit for use with the Java platform, and is maintained by the Eclipse Foundation. It is an alternative to the Abstract Window Toolkit (AWT) and Swing Java toolkits.

For more information about SWT, see the following URL:

<http://www.eclipse.org/swt/>

Because the rich client already uses SWT, the necessary files are already loaded into your Eclipse environment from `TC_ROOT\portal\plugins` when you defined the target platform. Therefore, when you create a Java file, you can use the **import org.eclipse.swt.widgets** command to include SWT in your coding.

Note:

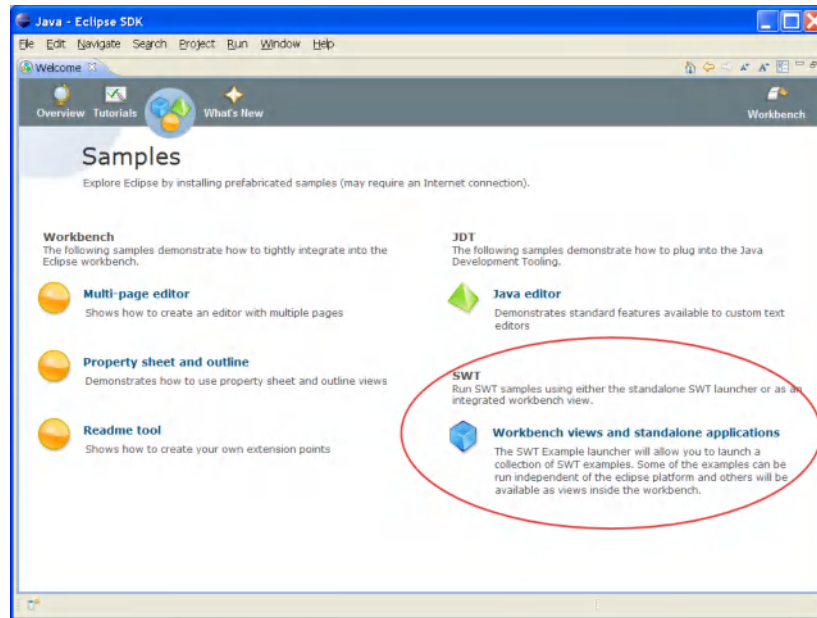
Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

To become familiar with SWT, you can download SWT samples and run the SWT tutorial in Eclipse:

1. Download the Eclipse 3.8 Software Development Kit (SDK) kit for your platform from the following Web site:

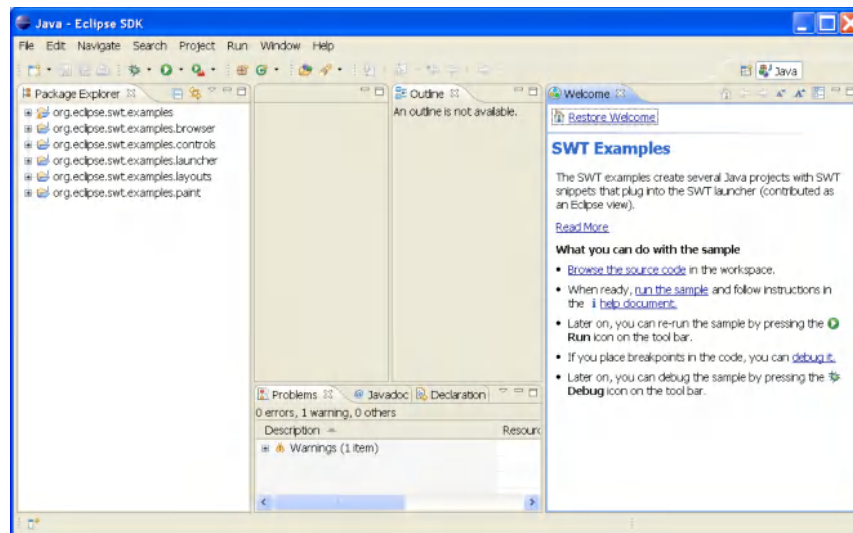
<http://archive.eclipse.org/eclipse/downloads/drops/R-3.8-201206081200/>

2. Run the SWT samples:
 - a. Click **Samples** on the **Welcome** page, and under **SWT** select **Workbench views and standalone applications**.



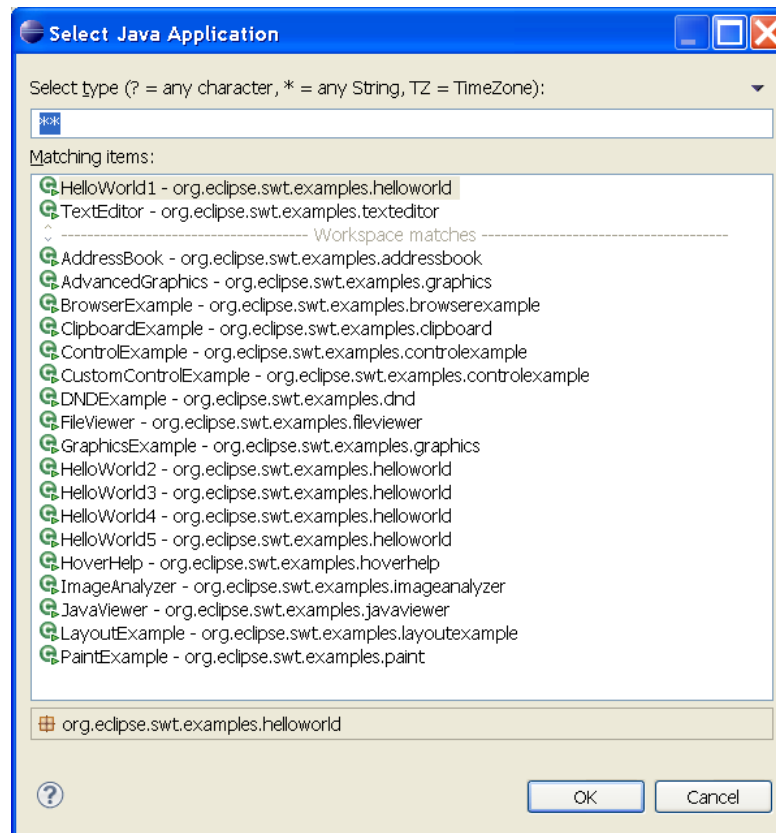
Launching the SWT samples

The SWT sample projects are installed in Eclipse.



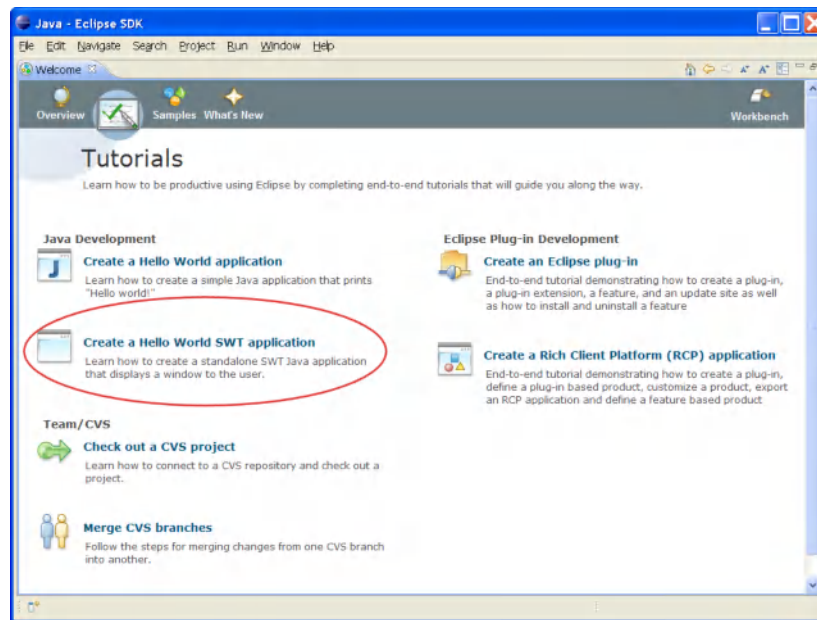
SWT sample projects in Eclipse

- b. To see the samples in action, right-click the **org.eclipse.swt.examples** project, choose **Run As→Java Application**, and select samples from the dialog box.



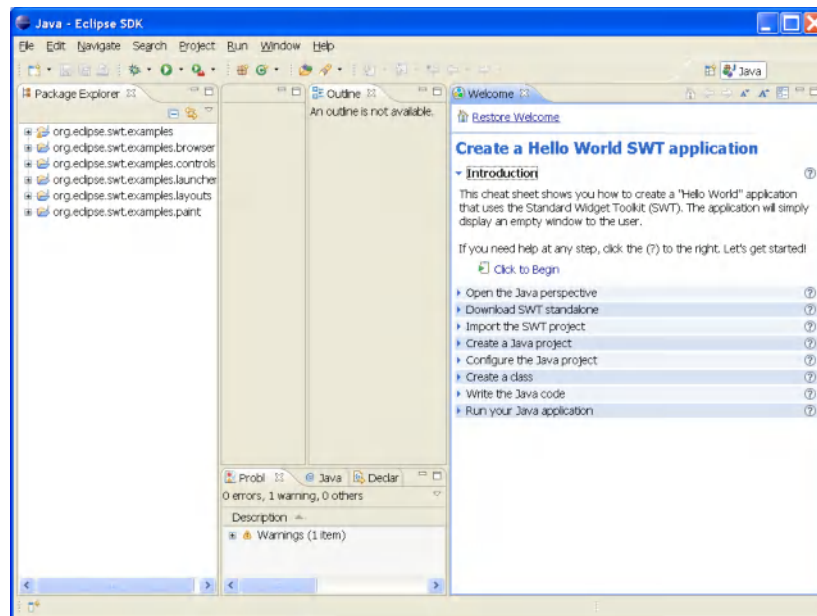
Available SWT samples

3. Run the SWT tutorial:
 - a. Choose **Help**→**Welcome**. On the **Welcome** page, click **Tutorials** and choose **Create a Hello World SWT application** tutorial.



Launching the SWT tutorial

The tutorial is displayed.



SWT tutorial

- b. Run the tutorial.

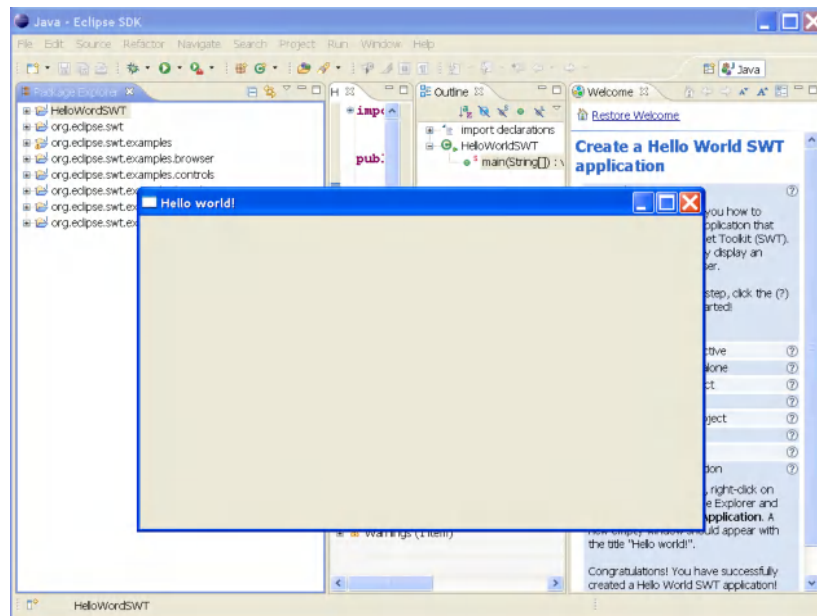
Note:

In the **Download SWT standalone** step of the tutorial, go to the following URL:

<http://download.eclipse.org/eclipse/downloads/>

Select the latest release build, and find the **SWT Binary and Source** download.

- c. To run your application, right-click the **HelloWorldSWT** project in the **Package Explorer** and choose **Run As→Java Application**.
A new empty window should appear with the title **Hello world!**.



HelloWorldSWT application

Basic tasks for rich client customizations

Process for creating rich client customizations

When you create a rich client customization, follow this general process to create, test, and distribute the customization:

1. Set up your Eclipse environment for rich client customization.
2. Create the desired customization.
3. If you create a custom rich client plug-in, export the plug-in to the **TC_ROOT\portal\plugins** directory or a shared directory.

4. Clear cache and register any new plug-in to ensure the customization appears in the rich client.
5. Run the rich client to verify the customization.
6. After testing is successful, distribute the customization to your organization.

Enable rich client customization

Process for enabling rich client customization

To enable rich client customization, you must install Eclipse and configure it to run the rich client.

Note:

The Teamcenter rich client must be installed on your machine first.

The following steps are required to customize the rich client with Eclipse for the first time:

1. Install Eclipse.
2. Set the project preferences.
3. Run the rich client from Eclipse.

Install Eclipse

1. If you have not already downloaded and installed the Java Development Kit (JDK) version 7 or later, do so before proceeding further. You can download the JDK from the following Web site:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

2. Download the Eclipse 3.8 Software Development Kit (SDK) kit for your platform from the following Web site:

<http://archive.eclipse.org/eclipse/downloads/drops/R-3.8-201206081200/>

3. After you download the kit, extract the ZIP file to a directory on your machine. The **eclipse** directory is automatically created below the installation directory.
4. Create a batch file that sets the environment, starts the server, and launches Eclipse using the JDK command line parameters. Use the following template to create your batch file:

```
set FMS_HOME=TC_ROOT\tccs
set JAVA_HOME=jre-install-directory
```

```

set JRE_HOME=jre-install-directory
set CLASSPATH=TC_ROOT\portal
set PATH=%FMS_HOME%\bin;%FMS_HOME%\lib;TC_ROOT\portal;%PATH%
start "TAO ImR" /min cmd /c "TC_ROOT\iiopservers\start_imr.bat"
Eclipse-install-directory\eclipse.exe -vm jdk-install-directory\bin\javaw

```

Note:

You can use the **portal.bat** file as template for this batch file.

5. Run the batch file you just created.
Eclipse displays the **Workspace Launcher** dialog box.
6. Use the default location for the workspace. This location is used by Eclipse to store the project related information. Click **OK**.
Eclipse displays the welcome window. You can close it.

Set the project preferences

1. In Eclipse, choose **Window**→**Preferences** to open the **Preferences** dialog box.
In the tree on the left, double-click the **Java** node, then select the **Installed JREs** node.
Verify that the correct Java Runtime Environment (JRE) version is listed and checked. If the correct version is not listed, follow these steps:
 - a. Next to the **Installed JREs** list, click the **Add** button.
 - b. In the **JRE Type** dialog box, select **Standard VM** and click **Next**.
 - c. In the **JRE Definition** dialog box, type the JRE directory in the **JRE home** box. It is the **jre** directory under the Java SDK installation directory.
 - d. Type the name of the JRE in the **JRE name** box.
 - e. Click **Finish** to save the new definition and close the dialog box.
 - f. In the **Preferences** dialog box, select the new JRE.
2. In the **Preferences** dialog box, double-click the **Plug-in Development** node, then select the **Target Platform** node.
3. In the **Target Platform** dialog box, click **Add**.
4. In the **Target Definition** box, ensure **Nothing** is selected and click **Next**.
5. In the **Target Content** dialog box, click **Add**.
6. In the **Add Content** dialog box, select **Directory** and click **Next**.

7. Navigate to the `TC_ROOT\portal` directory and click **Finish**.
The target platform is loaded.
8. Click the **Finish** button.
The **Target Platform** dialog box appears.
9. In the **Target Platform** dialog box, select the target you just set and click **OK**.

Run the rich client from Eclipse

1. In Eclipse, choose **Run**→**Debug Configurations**.
2. In the tree on the left of the **Create, manage, and run configurations** dialog box, double-click **Eclipse Application**, then select the **New_configuration** node.
3. In the **Name** box, type **RichClient**.
4. In the **Main** tab, perform the following:
 - a. Select the **Clear** check box to clear the workspace before running the application.
 - b. Ensure that **Run a product** is selected and that the product is **com.teamcenter.rac.aifrcp.product**.
 - c. Ensure the correct JRE appears in the **Runtime JRE** box.
5. Click the **Arguments** tab and type the following in the **VM arguments** box:


```
-Xms256m -Xmx1024m
```
6. Click the **Debug** button in the bottom right of the dialog box.
Ensure that the rich client logon dialog box appears. When it does, either click **Cancel** or log on the rich client.

Export plug-ins

Export your custom plug-in to the rich client

To test your custom plug-in, export it as a JAR file to the rich client `TC_ROOT\portal\plugins` directory.

1. In Eclipse, right-click the customization project and choose **Export**.
2. In the **Export** dialog box, choose **Plug-in Development**→**Deployable plug-ins and fragments**.
3. Click **Next**.

4. Click the **Browse** button to the right of the **Directory** dialog box and browse to the `TC_ROOT\portal` directory.
5. Click **Finish**.

The JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

Before running the rich client to verify the customization, run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client, and clear the rich client cache.

Export your custom plug-in to a shared directory

If you want to store custom plug-ins in a shared directory on an external machine to be referenced by rich client installations, you can export the custom plug-in to the shared directory instead of the `TC_ROOT\portal\plugins` directory. Use a **.link** file on the rich client installations to point to the shared directory:

1. Create the shared directory.
 - a. Create a directory (for example, **shared**) to hold the custom plug-ins, and share it so it can be accessed by outside machines as a mapped directory.
 - b. Place within the shared directory an **eclipse\features** subdirectory and a **eclipse\plugins** subdirectory, for example:


```
z:\shared\eclipse\features
z:\shared\eclipse\plugins
```
 - c. Export the custom plug-ins from Eclipse to the **eclipse\plugins** subdirectory on the shared directory.
2. Link to the shared directory from the rich client installations.
 - a. In each rich client installation that requires access to the shared directory, add a **links** folder inside the `TC_ROOT\portal\` directory.
 - b. In the **links** folder, create a plain text file with a **.link** file extension (for example, **custom.link**) that points to the shared directory, for example:

```
path=\\z:\\shared
```

Or:

```
path=/z:/shared
```

The slash is either one (/) or two (\\) depending on direction. Leading slashes (/ or \\) are required to tell Eclipse that this is an absolute path. Otherwise, it converts it to a relative path and does not support referencing another drive.

- c. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-ins in the shared directory with the rich client.

Ensure your customizations appear

You can run the rich client from Eclipse, and thereby test if your customizations appear in the rich client.

But running your customization from Eclipse is not a true test of whether your customizations will work in an end user's rich client installation. After you create a customization, you must perform the following steps to ensure it appears in the rich client when it is run outside the Eclipse IDE.

Note:

You must perform these steps whenever you create a new plug-in to add to the rich client, which should be in most cases of rich client customization (with the exception of style sheets). You must always wrap your rich client customizations in a plug-in to ensure that the out-of-the-box rich client is unchanged, thereby maintaining its integrity the next time you upgrade to a newer version of Teamcenter. If you customize the out-of-the-box rich client files, you can lose those customizations the next time you upgrade.

1. **Export your custom plug-in** from Eclipse to the rich client.
2. Run the `TC_ROOT\portal\registry\genregxml.bat` file to register new plug-ins and other changes with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

3. Ensure that the **-clean** and **-initialize** arguments are added to the **portal.bat** file (between the **start Teamcenter.exe** command and %*).

Run the **portal.bat** file to launch the rich client to verify that your customizations appear.

Note:

To confirm that the plug-in is registered in the rich client, choose **Help→About→Installation Details** and search the list to see the registered plug-ins.

4. If your customizations still do not appear, to clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

Distributing rich client customizations

Distributing customizations to four-tier rich clients

Process for distributing customizations to four-tier rich clients

If you used Over-the-Web Installer (OTW) to initially install a rich client, you can use OTW to distribute your customized files to it.

1. Package your customized files into a JAR file.
2. Create an installable component descriptor (ICD) file that defines your custom solution.

Note:

If you later modify the custom installable component descriptor (ICD) file, for the changes to take effect, you must delete the client's Over-the-Web Installer distribution directory and reinstall.

3. Run the Web Application Manager (**insweb**) to create a distribution instance for your custom solution.

Creating a solution file

The following code shows a sample solution file named **sample.icd**. This file deploys the **sample.jar** file to the **RC_ROOT\plugins** directory on the rich client machine.

```
#The name of the solution. It appears on the solution list in the Web
#Application Manager.
[NAME]
Sample Solution for Rich Client 4-Tier
```

```

#The description of the solution that appears in a tooltip when the user
#hovers the mouse over the solution name in the solution list. You can use
#HTML code to format the text.
[DESCRIPTION]
This installs the sample.jar file.
#Specify the jar file. If you have more than one jars, you need to repeat
#this section for each file like this:
#[COPYFILE]
#{
#    [FROM]
#    sample.jar
#
#    [TO]
#    plugins
#}
#[COPYFILE]
#{
#    [FROM]
#    sample2.jar
#
#    [TO]
#    plugins
#}
#[COPYFILE]
#{
#    [FROM]
#    sample3.jar
#
#    [TO]
#    plugins
#}
[COPYFILE]
{
    [FROM]
    sample.jar
    [TO]
    plugins
}
#The version of this Sample Solution.
[VERSION]
11
#Requires Teamcenter rich client install.
[PREREQUISITE_SOLUTIONS]
rac4t:11
#This file defines a Solution.
[SOLUTION]
Y
#It's a Distribution Instance Solution.
[SOLUTION_TYPE]
DS_INSTANCE
#Install information
[MANIFEST_INFO]
{
    #This is the RC_ROOT.
    [SUBDIR]
    rac
    [TARGETS]
    <target name="file_download_unix">
    </target>
    <target name="new_solution_unix">

```

```

</target>
<target name="file_download_win">
</target>
<target name="new_solution_win">
</target>
[DOWNLOAD_FILES]
{
    #This section tells the OTW installer to download the jar.
    #The location is subfolder to [SUBDIR] from above.
    #If you have more than one jar, you need to list them here like this:
    #[COMMON]
    #plugins/sample.jar:1.0
    #plugins/sample2.jar:1.0
    #plugins/sample3.jar:1.0
    [COMMON]
    plugins/sample.jar:1.0
    [SOLARIS]
    [HPUX]
    [AIX]
    [NT_INTEL]
}
[PACKAGED_FILES]
{
    [SOLARIS]
    [HPUX]
    [AIX]
    [NT_INTEL]
}
}

```

Distribute a solution file

1. Launch the Web Application Manager:

- a. Open the **WEB_ROOT** directory.
This is the directory in which you installed the Web Application Manager on your hard drive.
- b. In Windows, double-click the **insweb.bat** file. In UNIX, type the following command:

```
insweb
```

The Web Application Manager displays the **Teamcenter Web Application Manager** dialog box.

2. Click **Copy ICDs**.

The Web Application Manager displays the **Copy ICD Files** dialog box.

3. Enter the path to your solution file and click **OK**.

The Web Application Manager displays a **Progress** dialog box and copies the ICD file.

4. When copying is complete, click **OK**.

The Web Application Manager displays the **Teamcenter Web Application Manager** dialog box.

5. In the **Web Applications** list, select the rich client instance and click **Modify**.
The Web Application Manager displays the **Modify Web Application** dialog box.
6. Click **Modify Disk Locations**.
The Web Application Manager displays the **Modify Disk Locations** dialog box.
7. Ensure the path to your solution files is listed in the **Disk Locations for Install Images** box. If it is not, click **Add**.
8. Click **Add Solutions**.
The Web Application Manager displays the **Add Solutions** dialog box.
9. Select your solution and click **OK**.
The Web Application Manager begins installation of the solutions and displays a **Progress** dialog box.
10. When installation is complete, click **OK** to close the **Progress** dialog box.
The Web Application Manager displays the **Modify Web Application** dialog box.
11. Click **OK**.
12. Deploy the distribution instance as described in the appropriate Teamcenter server installation guide (either UNIX and Linux Server Installation or Windows Server Installation).
13. Launch the Over-the-Web-installed rich client or the local **otw.bat** file to download and use the **sample.jar** file. You can verify the JAR file was downloaded to the designated installation location by looking in the **RC_ROOT\plugins** directory.

Note:

Once you install a customization using a solution file, you cannot remove it using Over-the-Web Installer.

ICD tags

The following table lists the tags used in ICD files.

Optionally, you can add comments to an ICD file. Any line that begins with a pound sign (#) is ignored by the system.

Tag	Description
APPMAP	Signifies that the following lines (until the next tag or end of file) contain the file names of any application maps that must be added to the overall application map for this particular component. The order in which the application map files appear in the ICD file is the order they appear in the master application_map.xml file. The application map path name entries are relative to the WEB-INF directory.

Tag	Description
	Use of this tag is optional.
BOILERPLATE_XML	Signifies that the following tag block (grouping of the lines between the braces) contains boilerplate XML code that should be generated into the web.xml file as listed. There can be only one BOILERPLATE_XML tag block per ICD file. The available subtags for the BOILERPLATE_XML tag are derived from the Web application descriptor in Sun's J2EE Servlet Specification web_app_2.2.dtd (which can be found at http://java.sun.com/j2ee/dtds). Each valid subelement for the web-app element can be used as a separate subtag in the BOILERPLATE_XML tag block (except for display-name, description, and session-config, which are gathered from the Web Application Manager). The subtag should use the subelement name in all capital letters. For example, for the servlet-mapping subelement, the appropriate subtag for the BOILERPLATE_XML tag would be [SERVLET_MAPPING] The list of valid subtags is as follows: CONTEXT-PARAM DISTRIBUTABLE ENV-ENTRY EJB-REF ERROR-PAGE ICON MIME-MAPPING LOGIN-CONFIG RESOURCE-REF SECURITY-CONSTRAINT SECURITY-ROLE SERVLET SERVLET-MAPPING TAGLIB WELCOME-FILE-LIST Note: The CONTEXT-PARAM subtag is used for context parameters that are not editable from the Modify Context Parameters window in the Web Application Manager. Following each subtag provide a valid block of XML code that uses only that subelement. Use only one subtag for the entire block of entries for that particular subelement. For an example, see the teamcenter_wae.icd file.
COMPONENT_DEPENDENCIES	Signifies that the following lines (until the next tag or end of file) contains the name and version of other components upon which your component is dependent. For example, your component may be dependent on Global Services connector components. Direct dependencies come from Java references to interfaces and classes, JSP include statements, applets, HTML and .jpg and .gif files, text bundle textid references, application map references, system preferences references, and so on. If your customization refers to any entity defined in another component, you have a direct dependency on that other component, and you must list that component in the COMPONENT_DEPENDENCIES section of your ICD file. The format is: <i>component : major-version . minor-version</i>

Tag	Description
	Replace <i>component</i> with the name of the component on which your custom component is dependent (use the name of its ICD file, without the .icd extension). Replace <i>major-version</i> and <i>minor-version</i> with the version of <i>component</i> that is necessary for your custom component to function properly. For example, the following entry specifies that your component requires version 00500.000001 of the teamcenter_reports component: <pre>teamcenter_reports:00500.000001</pre> Caution: The value supplied for this tag must match the exact case of the ICD file of the component on which your component is dependent. For example, if your component is dependent on the Siemens PLM Software component that has an ICD file named teamcenter_soa.icd (Teamcenter Services), your custom component's ICD file must use the lowercase teamcenter_soa value in its COMPONENT_DEPENDENCIES section.
CONTEXT_PARAM	Signifies that the following tag block (grouping of the lines between the curly brackets) contains a definition for a context parameter that is generated into the web.xml file. Multiple context parameters can be defined in a single ICD file by using multiple CONTEXT_PARAM tag blocks. Subtags are used inside the tag block to provide information about the context parameter being defined. The values for context parameters defined in this manner can be changed in the Modify Context Parameters dialog window of the Web Application Manager. The following tag block is an example of a context parameter definition from the teamcenter_mwau.icd file: <pre>[CONTEXT_PARAM] { [PARAM_NAME] ApplicationInstance [DESCRIPTION] A name uniquely identifying the application instance within the application server. [INPUT_REQUIRED] Y [DEFAULT_VALUE] GlobalServicesInstance1 [TYPE] string }</pre> The CONTEXT_PARAM tag block can contain the following subtags, all of which can only be used as subelements of the CONTEXT_PARAM tag block: DEFAULT_VALUE Specifies the default value of the context parameter. If this tag is not used, then a blank default value is provided. DESCRIPTION Contains a description of the context parameter. This tag is required.

Tag	Description
	ENCRYPT_VALUE Specifies whether the Web Application Manager must encrypt the context parameter value. Valid values are Y (encrypted) and N (plain text). If this tag is not specified, the value is not encrypted. INPUT_REQUIRED Specifies whether the context parameter is a required parameter (by use of a Y or N entry). If a required context parameter has any blank values, the user is not allowed to continue the installation until the parameters are filled in. MAXIMUM Specifies the maximum integer value that is acceptable for this context parameter. This tag is only used when int, float, or double are specified in the TYPE tag. MINIMUM Specifies the minimum integer value that is acceptable for this context parameter. This tag is only used when int, float, or double are specified in the TYPE tag. PARAM_FILES Specifies a list of template files that contain tokens for this parameter that are replaced by the value of this parameter when the template is used to generate a configuration or other file used by a Teamcenter application. The substitution token in the template file is the name of the context parameter surrounded by @ signs. The following example shows the token for the JNDIName context parameter: @JNDIName@ The template file name must have the .template extension. For example, if the resulting file must be named TcEngConfig.xml, the template file must be named TcEngConfig.xml.template. If the template file is nested inside of a JAR file, you can list the JAR file name first, followed by a colon, and the template file name. For example, if you type ejb/core_ejbs.jar:meta-inf/TcEngConfig.xml.template, the Web Application Manager: • Extracts the ejb/core_ejbs.jar file (relative to either the webapp_root or earapp_root directory). • Creates a copy of the template renamed as TcEngConfig.xml. • Substitutes the parameter value for the tokens in the meta-inf/TcEngConfig.xml file. • Rebuilds the ejb/core_ejbs.jar file. PARAM_FUNCTION Specifies the name of the context parameter used in the web.xml file. This tag is required. PARAM_NAME Specifies the name of the context parameter used in the web.xml file. This tag is required. TYPE Specifies the data type of the context parameter value. Valid values are boolean, double, float, int, and string. If a TYPE tag is not provided for a context parameter, it is assumed the context parameter's value is a string. VALID_VALUES Specifies a list of values that can be used for this parameter. The Web Application Manager displays these values in a list box where the installer selects the desired value.

Tag	Description
DEFAULT_SOLUTION	Signifies when the solution being defined in this ICD file must be included by default when adding a new Web application in the Web Application Manager. The list of solutions included when adding a new Web application can be changed by the user prior to beginning installation. For example, the teamcenter_prestiersolution.icd ICD file, used for the Teamcenter – Web Tier Core Applications solution, contains the following: <pre>[DEFAULT_SOLUTION] Y</pre> This tag can only be used in an ICD file for a solution (as opposed to an ICD file for a component). Valid values are Y (required) or N (not required).
FILES	Signifies that the following line (until the next tag or end of file) contains the name of the installable image file that must be installed for this particular component. Because the Web Application Manager looks for .TZ files on UNIX and JAR files on Windows, entries in this tag section list only the file name, without the file extension. Use uppercase letters for the file name. For example, enter MYCO_MYFOUNDATION to specify that there is one installable image file for your component (its file name is MYCO_MYFOUNDATION.TZ on UNIX or MYCO_MYFOUNDATION.JAR on Windows).¹
INCOMPATIBLE_SOLUTIONS	Signifies that this solution cannot be installed in conjunction with other solutions. For example, the Teamcenter 11.2 Global Services – Application Directory solution (provides Global Services datastore files, database scripts, and samples for customization) cannot be installed with other solutions. The tagging from the ugs_globalservices_application_directory.icd file indicates that the solution cannot be installed with the Teamcenter solutions or other Global Services solutions: <pre>INCOMPATIBLE_SOLUTIONS] sdrc_foundation teamcenter_jetigateway teamcenter_legacyurlsupportsolution teamcenter_prestierbase teamcenter_prestierproxysolution ugs_globalservices_bpel_framework ugs_globalservices_enterprise_application ugs_globalservices_enterprise_application ugs_globalservices_omfg_connector ugs_globalservices_sap_connector ugs_globalservices_security_proxy ugs_globalservices_tceng_connector ugs_globalservices_tcent_connector ugs_globalservices_tcgsc_connector ugs_globalservices_teamcenter_connector ugs_globalservices_teamcenter_RAC_connector ugs_globalservices_teamcenter_SOA_connector ugs_globalservices_webservices_connector</pre>

¹ The **insweb** application takes the value it finds in the **FILES** tag and changes it to uppercase letters before looking for the actual image file. This means that although you can use lowercase letters to specify the file name in the ICD file, the actual file must be named using uppercase letters.

Tag	Description
INSTALLER	The values used must be the solution abbreviations and not their displayable names (for example, teamcenter_prestierbase is the solution abbreviation for the Teamcenter – Web Tier Infrastructure solution). Signifies that installers, such as InstallShield programs, are part of the component. This tag is used only in ICD files requiring separate installers, such as CAD connectors. There are different installers for different platforms (for example, UNIX, Windows, and so on). To support this, each installer has a different UID (unique identification) that points to each installer instance by platform. The INSTALLER tag block can contain the following subtags, all of which can only be used as subelements of the INSTALLER tag block: • DESCRIPTION Specifies the description of the installer as used on the installer HTML page. • DISPLAY_NAME Specifies the name of the installer as used on the installer HTML page. • GROUP Specifies the name of the group the installer appears in. • NAME Specifies the ID used to identify the installer. • REQUIRED Indicates that this is a required installer. All required installers must be installed before the client can launch. • UID Specifies the unique identification for the installer for each platform. • VERSION Specifies the current version of the installer. The value must be change_me_installer-name_installer_version, where <i>installer-name</i> is the name of the installer. The value for this tag is populated at build time.
NAME	Signifies that the next line in the file contains the full name of a solution. When a user eventually uses the Web Application Manager (insweb) to install your solution, the Web Application Manager uses the value you supply here when the Web Application Manager presents a list of the solutions available for installation. This tag is used only in ICD files for solutions (as opposed to ICD files for components). ICD files for solutions contain the SOLUTION tag. Solutions are a special type of component because they compose other components (for example, the Teamcenter 11.2 Global Services – Data Exchange solutions comprises the Global Services User Interface, BPEL Framework, SOA Connector, and Teamcenter Web Tier Infrastructure components).
NON_DEPLOYABLE	Signifies whether the solution is nondeployable and therefore does not contain APPMAP or SYSTEM_PREFERENCES tag values that must be included in the master application map (application_map.xml) or system preferences (system_preferences.xml) files. Documentation files are an example of a nondeployable solution (see the teamcenter_onlinedocs.icd file). Valid values are Y (nondeployable) or N (deployable). This tag can only be used in ICD files for solutions.
PREREQUISITE_SOLUTIONS	Specifies that the following lines (until the next tag or end of file) contain the abbreviations of any solutions that are prerequisites for this particular solution. The format is: <i>solution : major-version</i>

Tag	Description
	Replace <i>solution</i> with the name of the ICD file (without the icd extension) of the solution that your solution requires. Replace <i>major-version</i> with the minimum version of <i>solution</i> that is required to run your (custom) solution. This version information is related to the value specified for the VERSION tag in the ICD file of the solution that your solution requires. For example, if your solution requires major version 02005 of the teamcenter_prestierbase solution, enter the following value: <pre>tteamcenter_prestierbase:02005</pre> This tag is used only in ICD files for solutions (as opposed to ICD files for components). ICD files for solutions contain the SOLUTIONS tag. Solutions are a special type of component because they comprise other components.
SOLUTION	Signifies that the current ICD file defines a solution (as opposed to a component). For example, when the Teamcenter – Web Tier Core Applications solution is installed, its ICD file (teamcenter_prestiersolution.icd) contains the following: <pre>[SOLUTION] Y</pre> Valid values are Y (solution) or N (component).
SOLUTION_TYPE	Designates the type of solution the current ICD file defines, such as a thin client solution (THIN) or a distributable rich client 4-tier solution (DS_INSTANCE). For example, when the Teamcenter – Enterprise Tier Common Component solution is installed, its ICD file (teamcenter_common.icd) contains the following: <pre>[SOLUTION_TYPE] THIN</pre> If no value is supplied for this tag, then THIN is used as the default. This tag is used only in ICD files for solutions (as opposed to ICD files for components).
SYSTEM_PREFERENCES	Signifies that the following lines (until the next tag or end of file) contains the names of any system preferences files that this component adds to the system_preferences.xml file. The path name entries are relative to the WEB-INF directory. Use of this tag is optional.
VERSION	Signifies that the next line in the file contains the version identifier for a component. The format is as follows: <pre>major.minor.build</pre> For example: <pre>00300.000001.20021015</pre> When the Web Application Manager reads an ICD file, the value you supply for the <i>major</i> section of the version string must match the <i>major</i> value of all the other components.

Tag	Description
	Teamcenter imposes no limits to the number of digits in any section of the version number. However, the length in any section may be relevant to the Web Application Manager. For example, the version number 123.0456.999 is considered earlier than the version number 123.456.999 . This is because version checking is done by string comparison, and 0456 sorts before 456 .

Distributing customizations to two-tier rich clients

Process for distributing customizations to two-tier rich clients

The basic customization technique is to create a plug-in that contains the customizations and deploy the custom plug-in JAR file to the rich client installation's **TC_ROOT\portal\plugins** directory. This way, when you upgrade to a newer version of Teamcenter, you can simply install the custom plug-in without having to alter files in the Teamcenter installation.

Perform the following steps to distribute your rich client customizations created in Eclipse to two-tier rich clients

1. Package your custom files.
2. Create a feature file to install the custom files.

Package custom rich client files

When you create customizations to the rich client, typically they are comprised of JAR files exported from Eclipse that must be installed to the **TC_ROOT\portal\plugins** directory. Package these JAR files into a ZIP file so you can install them using Teamcenter Environment Manager (TEM).

1. To create a plug-in JAR file of your customization, in Eclipse right-click the project and choose **Export→Plug-in Development→Deployable plug-ins and fragments**.
2. Zip your custom JAR file into a **rac_feature-name.zip** file (for example, **rac_samplecust.zip**) with the root path for the JAR file set to **\plugins**.
The use of **rac_** in the name signifies that the ZIP file contains a rich client feature. For examples of these files, see the **portal\compressed_files** directory on the Teamcenter installation source.
3. Create a directory structure to hold the ZIP files, for example, **portal\compressed_files**.
You can use any directory structure you want. For convenience, this structure is the same used on the Teamcenter installation source.
4. Copy your ZIP files to the **portal\compressed_files** directory.
5. Create a feature file so you can install the custom files using TEM.

Create a feature file for rich client customizations

After you have packaged your rich client customization files into a ZIP file, you can create a feature file so that you can install them using Teamcenter Environment Manager (TEM).

1. Package your rich client customization files JAR files into a ZIP file.
2. In a text editor, create an XML feature file that points to the ZIP file, for example, **feature_rac_mycust.xml**.

Tip:

For examples of **feature_rac_feature-name.xml** files, see the **install\modules** directory on the installation source.

Following is an example of a feature file:

```
<?xml version="1.0" encoding="UTF-8"?>
<feature>
  <name value="Rich Client Customizations"/>
  <size value="21"/>
  <os value="windows" version="5.1"/>
  <os value="sunos" version="5.10"/>
  <os value="hp-ux" version="11.11"/>
  <root value="true"/>
  <group value="package"/>
  <singular value="true"/>
  <guid value="7560C67ED5B01B65C39B8DF7364066C8"/>
  <relation>
    <depends>
      <or value="FF18D25DA73019F880BCFFBC0029CA28"/>
      <or value="E2564104E1B964BB23D78078DBA34EEA"/>
      <or value="A2564824E1B434AC23D70178DBA34BCA"/>
    </depends>
  </relation>
  <files>
    <code>
      <unzip file="portal/compressed_files/rac_samplecust.zip"
        todir="portal"/>
    </code>
  </files>
</feature>
```

In the feature file, change the following lines:

- In the **size** tag, change the **value** text to the approximate size (in megabytes) of your customization files.
- Add or remove **os** tags to match the platforms where you use rich client.

- In the **guid** tag, change the **value** text to a unique value exactly 32 characters in length and composed of any combination of the numbers 0–9 and the letters A–F.
- In the **unzip** tag, change the **file** text to the name of the ZIP file that contains your customizations.
To ensure that the **todir="portal"** line correctly unzips to the **portal\plugins** directory, ensure that **plugins** is set as the location in the ZIP file.

Tip:

If your customizations include a packaged Business Modeler IDE template project, you can edit the packaged project's feature file to include the new rich client feature by adding a node similar to the following. This is not required; you can still have a separate feature file for the rich client customizations.

```
<feature>
  <name value="feature-display-name"/>
  <property name="feature_id" value="rac"/>
  <property name="bmode_optional" value="true"/>
  <relation>
    <depends>
      <or value="FF18D25DA73019F880BCFFBC0029CA28"/>
      <or value="E2564104E1B964BB23D78078DBA34EEA"/>
      <or value="A2564824E1B434AC23D70178DBA34BCA"/>
    </depends>
  </relation>
  <size value="0"/>
  <singular value="true"/>
  <guid value="85B3827CD5A0FA61220398F6C60C21AB"/>
  <files>
    <code>
      <unzip file="portal/compressed_files/rac_feature-name.zip"
        todir="portal"/>
    </code>
  </files>
</feature>
```

3. Copy the XML feature file to the **TC_ROOT\install\install\modules** directory on the installation source.
4. Open the **TC_ROOT\install** directory and run Teamcenter Environment Manager (TEM).
5. Click **Configuration Manager** and click **Next**.
6. Click **Perform maintenance on an existing configuration** and click **Next**.
7. Ensure the correct configuration is selected and click **Next**.
8. Ensure **Add/Remove Features** is selected and click **Next**.

9. Under the **Extensions** node, select the check box for your new files. The name of the box is the one you gave it in the feature file. Click **Next**.
10. In the **Confirm Selections** panel, click **Next** to start installing your customization files.

Sample rich client customizations

Commonly used rich client customizations

Adding menu commands to a menu, toolbar, and shortcut menu

Add a menu command to a menu

The rich client uses the Eclipse menus, commands, and handlers mechanism for menus. This example adds the **Sample Command** menu command to the end of the **Tools** menu in the **My Teamcenter** perspective only.

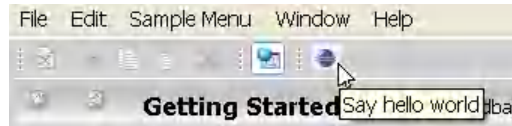
1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.addmenuitem** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
 - f. To verify the configuration so far, choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.

When you run the rich client from Eclipse, the new **Sample Menu** menu appears on the menu bar.



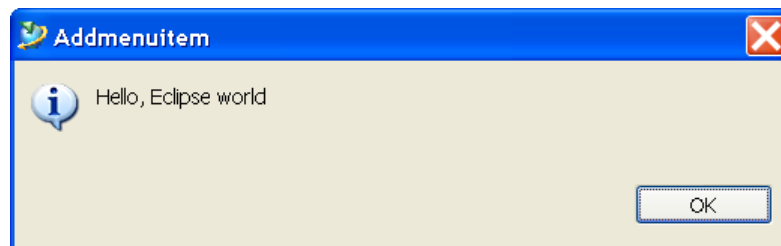
Custom menu command on the menu bar

At the far right of the toolbar, there is also a new button.



Custom button on the toolbar

Selecting either the new menu or new button opens the following dialog box.



Action launched from the custom menu command or button

2. Edit the **com.mycom.addmenuitem** plug-in's **plugin.xml** file to place the menu command at the bottom of the **Tools** menu. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension point="org.eclipse.ui.commands">
    <command
      name="Sample Command"
      id="com.mycom.addmenuitem.commands.sampleCommand">
    </command>
  </extension>
  <extension point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.addmenuitem.commands.sampleCommand"
      class="com.mycom.addmenuitem.handlers.SampleHandler">
    </handler>
  </extension>
  <extension point="org.eclipse.ui.menus">
    <menuContribution locationURI="menu:tools?after=additions">
      <command
        commandId="com.mycom.addmenuitem.commands.sampleCommand"
        mnemonic="S"
        icon="icons/sample.gif"
        id="com.mycom.addmenuitem.menus.sampleCommand">

```

```

        <visibleWhen>
            <reference
                definitionId="com.teamcenter.rac.ui.inMainPerspective">
            </reference>
        </visibleWhen>
    </command>
</menuContribution>
</extension>
</plugin>

```

The **visibleWhen** clause uses the **com.teamcenter.rac.ui.inMainPerspective** definition to control visibility. This definition is in the **com.teamcenter.rac.common** plug-in's **plugin.xml** file. Each Teamcenter perspective has an **inMainPerspective** definition, usually in the plug-in's **plugin.xml** file.

For example, the **inMainPerspective** definition for Structure Manager is in the **plugin.xml** file in the **com.teamcenter.rac.pse** plug-in. If you want the menu command to be visible in Structure Manager, change the **visibleWhen** clause to the following:

```

<visibleWhen>
    <reference
        definitionId="com.teamcenter.rac.pse.inMainPerspective">
    </reference>
</visibleWhen>

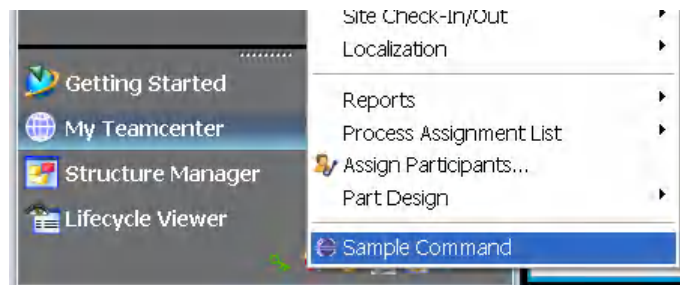
```

To add the menu to the menu bar instead of the **Tools** menu, change the **locationURI** attribute to the following:

```
locationURI="menu:org.eclipse.ui.main.menu?after=additions"
```

3. To verify the configuration, choose **Run→Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.

The menu command is inserted at the bottom of the **Tools** menu in the **My Teamcenter** perspective.



Custom menu command moved to the Tools menu

4. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the `.properties` files, or you add new plug-ins or change plug-in content, you must run the `genregxml` script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a `RegistryLoader` file for each locale in the `portal\Registry` directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Add a menu command to the shortcut menu

The rich client uses the Eclipse menus, commands, and handlers mechanism for the shortcut menu (the right-click menu). This example adds the **Sample Command** button to the shortcut menu in the **My Teamcenter** perspective only.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type `com.mycom.addshortcut` in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:

- A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
- e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. To verify the configuration so far, choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.

When you run the rich client from Eclipse, the new **Sample Menu** menu appears on the menu bar.

3. Edit the **com.mycom.addshortcut** plug-in's **plugin.xml** file to place the menu command on the shortcut menu. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension point="org.eclipse.ui.commands">
    <command
      name="Sample Command"
      id="com.mycom.addshortcut.commands.sampleCommand">
    </command>
  </extension>
  <extension point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.addshortcut.commands.sampleCommand"
      class="com.mycom.addshortcut.handlers.SampleHandler">
    </handler>
  </extension>
  <extension point="org.eclipse.ui.menus">
    <menuContribution
      locationURI="popup:org.eclipse.ui.popup.any?after=additions">
      <command
        commandId="com.mycom.addshortcut.commands.sampleCommand"
        mnemonic="S"
        icon="icons/sample.gif"
        id="com.mycom.addshortcut.menus.sampleCommand">
        <visibleWhen>
          <reference
            definitionId="com.teamcenter.rac.ui.inMainPerspective">
          </reference>
        </visibleWhen>
      </command>
    </menuContribution>
  </extension>
</plugin>
```

The **visibleWhen** clause uses the **com.teamcenter.rac.ui.inMainPerspective** definition to control visibility. This definition is in the **com.teamcenter.rac.common** plug-in's **plugin.xml** file. Each

Teamcenter perspective has an **inMainPerspective** definition, usually in the plug-in's **plugin.xml** file.

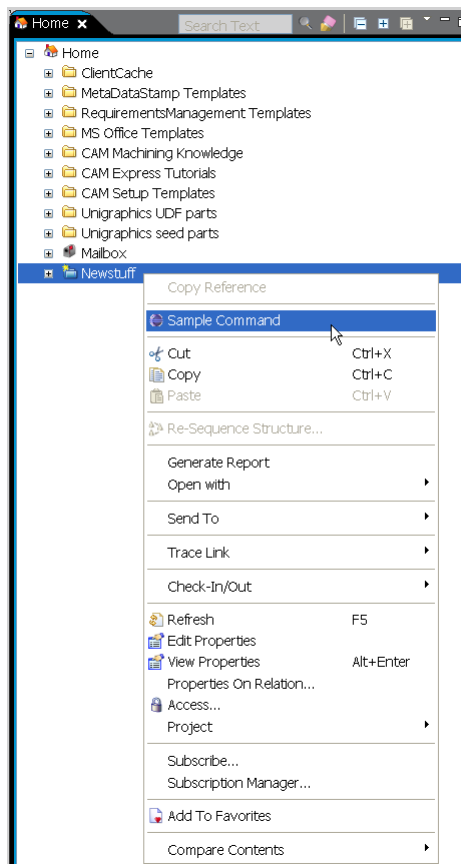
For example, the **inMainPerspective** definition for Structure Manager is in the **plugin.xml** file in the **com.teamcenter.rac.pse** plug-in. If you want the menu command to be visible in Structure Manager, change the **visibleWhen** clause to the following:

```
<visibleWhen>
  <reference
    definitionId="com.teamcenter.rac.pse.inMainPerspective">
  </reference>
</visibleWhen>
```

4. Verify the customization.

- Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
- Right-click on any object in the **My Teamcenter** perspective. For example, right-click a folder.

The **Sample Command** menu command appears on the shortcut menu.



Custom menu command added to the shortcut menu

- c. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- A. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- B. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- C. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Add a button to the toolbar

The rich client uses the Eclipse menus, commands, and handlers mechanism for the toolbar. This example adds the **Sample Command** button to the toolbar in the **My Teamcenter** perspective only.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type `com.mycom.addbutton` in the **Project name** box. Click **Next**.

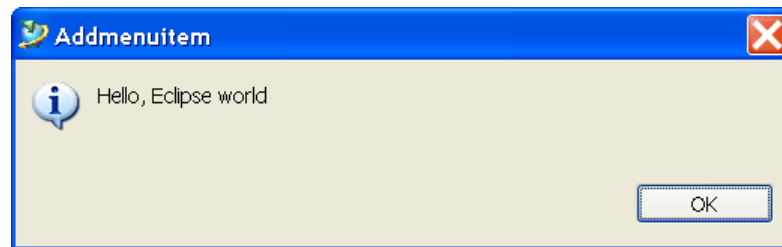
- d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. To verify the configuration so far, choose **Run→Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.

At the far right of the toolbar, there is a new button.



Custom button on the toolbar

Selecting the new button opens the following dialog box.



Action launched from the custom button

3. Edit the **com.mycom.addbutton** plug-in's **plugin.xml** file to place the button on the My Teamcenter toolbar. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension point="org.eclipse.ui.commands">
    <command
      name="Sample Command"
      id="com.mycom.addbutton.commands.sampleCommand">
    </command>
  </extension>
  <extension point="org.eclipse.ui.handlers">
```



```

<handler
  commandId="com.mycom.addbutton.commands.sampleCommand"
  class="com.mycom.addbutton.handlers.SampleHandler">
</handler>
</extension>
<extension point="org.eclipse.ui.menus">
  <menuContribution
    locationURI="toolbar:navigator_Toolbar?after=additions">
    <command
      commandId="com.mycom.addbutton.commands.sampleCommand"
      icon="icons/sample.gif"
      tooltip="Say hello world"
      id="com.mycom.addbutton.toolbars.sampleCommand">
    <visibleWhen>
      <reference
        definitionId="com.teamcenter.rac.ui.inMainPerspective">
      </reference>
    </visibleWhen>
    </command>
  </menuContribution>
</extension>
</plugin>

```

Each Teamcenter perspective has its own toolbar. For example, the My Teamcenter toolbar is **navigator_Toolbar**, and the Structure Manager toolbar is **pse_Toolbar**. The toolbar is usually defined in the **plugin.xml** file for the perspective. For example, the **pse_Toolbar** is defined in the **plugin.xml** file in the **com.teamcenter.rac.pse** plug-in.

4. To verify the configuration, choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.

The button is inserted in the middle of the toolbar in the **My Teamcenter** perspective.



Custom button location moved on the toolbar

5. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the **TC_ROOT\portal** directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.

- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Add a command to a menu or toolbar in a view

You can add a command to a toolbar or menu in a view. To illustrate the process, this example adds the exit command to My Teamcenter as a button on the **Details** view toolbar and as a command on the view menu.

1. Create the project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.exitcommand** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Update the project tabs.

Click the **Runtime** tab, click the **Add** button, and select the following:

com.mycom.exitcommand
com.mycom.exitcommand.handlers

3. Edit the project files.

- a. Edit the **com.mycom.exitcommand** plug-in's **plugin.xml** file to place the menu command on the **Details** view. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.teamcenter.rac.exit.commands.category">
    </category>
    <command
      name="Sample Command"
      categoryId="com.mycom.exitcommand.commands.category"
      id="com.mycom.exitcommand.commands.sampleCommand">
    </command>
    <command
      defaultHandler="com.mycom.exitcommand.handlers.SampleHandler"
      id="com.mycom.exitcommand.command1.Exit"
      name="Exit">
    </command>
  </extension>
  <extension
    point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.exitcommand.commands.sampleCommand"
      class="com.mycom.exitcommand.handlers.SampleHandler">
    </handler>
  </extension>
  <extension
    point="org.eclipse.ui.bindings">
    <key
      commandId="com.mycom.exitcommand.commands.sampleCommand"
      contextId="org.eclipse.ui.contexts.window"
      sequence="M1+6"
      schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
  </extension>
  <extension
    point="org.eclipse.ui.menus">
    <menuContribution
      locationURI="menu:com.teamcenter.rac.ui.views.DetailsView?after=group4">
    <menu
      id="closeMenu"
      label="Close">
    <command
      commandId="com.mycom.exitcommand.command1.Exit"
      label="Exit"
      style="push">
    </command>
    </menu>
  </menuContribution>
  </extension>
</plugin>
```

```

        tooltip="Exit the application">
    </command>
</menu>
</menuContribution>
<menuContribution
    locationURI="toolbar:com.teamcenter.rac.ui.views.DetailsView">
    <toolbar
        id="com.mycom.exitcommand.toolbar1">
        <command
            commandId="com.mycom.exitcommand.command1.Exit"
            icon="icons/sample.gif"
            label="Exit"
            style="push"
            tooltip="Exit the application">
        </command>
    </toolbar>
    <command
        commandId="com.mycom.exitcommand.command1.Exit"
        icon="icons/sample.gif"
        label="Exit"
        style="push"
        tooltip="Exit the application">
    </command>
</menuContribution>
</extension>
</plugin>

```

- b. Replace the code in the **SampleHandler.java** file with the following:

```

package com.mycom.exitcommand.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.ui.handlers.HandlerUtil;
/**
 * Our sample handler extends AbstractHandler, an IHandler base class.
 * @see org.eclipse.core.commands.IHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class SampleHandler extends AbstractHandler {
    /**
     * The constructor.
     */
    public SampleHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {
        HandlerUtil.getActiveWorkbenchWindow(event).close();
        return null;
    }
}

```

- c. Choose **File→Save All**.

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the `.properties` files, or you add new plug-ins or change plug-in content, you must run the `genregxml` script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a `RegistryLoader` file for each locale in the `portal\Registry` directory.

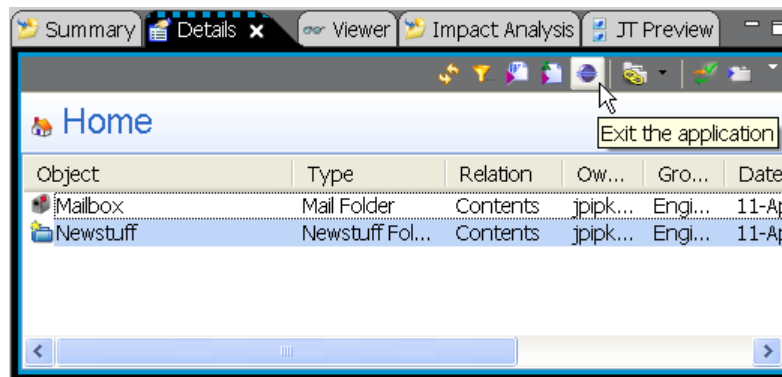
- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

5. Verify the customization.

- a. Launch My Teamcenter and click the **Details** view tab.

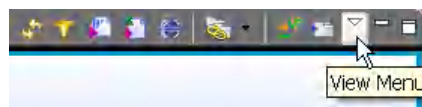
The exit command button is displayed on the toolbar of the view.



Exit command button on the view toolbar

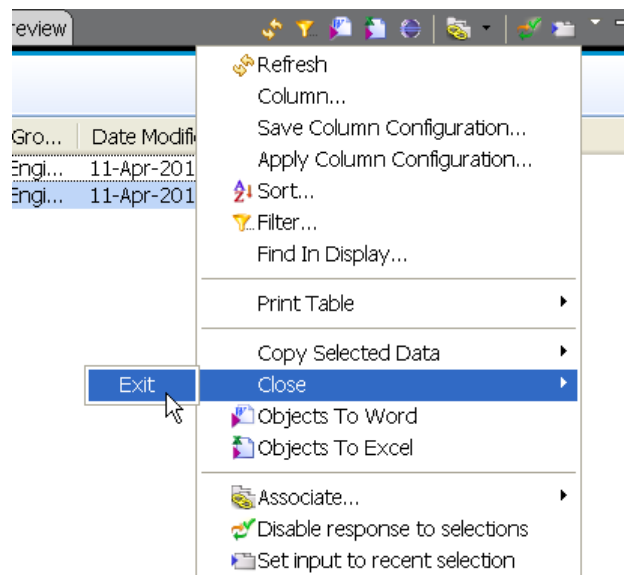
Do not click the button to exit Teamcenter. You must first verify the new exit command on the view menu.

- b. Click the menu button on the toolbar.



Menu button on the view toolbar

- c. Choose **Close**→**Exit** on the menu to exit Teamcenter.



Exit command on the view menu

Although this example shows how to add an exit command, you can use this process to add your own custom command to a view toolbar and menu.

Add a toggle menu item

You can create toggle menu items and tie them to state inside the rich client. To illustrate the process, this example adds an **Error Level** menu with toggled menu items. Select an object in My Teamcenter and select the toggle on the custom **Error Level** menu.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.toggle** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Dependencies** tab, click the **Add** button, and select the following:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
 - b. Click the **Runtime** tab, click the **Add** button, and select the following:

com.mycom.toggle
com.mycom.toggle.handlers
3. Create the project files.
 - a. Edit the **com.mycom.toggle** plug-in's **plugin.xml** file. Replace the entire contents of the **plugin.xml** file with the following code:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.teamcenter.rac.toggleexample.commands.category">
    </category>
    <command
      categoryId="com.mycom.toggle.commands.category"
      defaultHandler="com.mycom.toggle.handlers.ToggleInfoHandler"
      id="com.mycom.toggle.commands.infoCommand"
      name="Info">
      <state
        class="org.eclipse.ui.handlers.RegistryToggleState:true"
        id="org.eclipse.ui.commands.toggleState">
      </state>
    </command>
    <command
      categoryId="com.mycom.toggle.commands.category"
      defaultHandler="com.mycom.toggle.handlers.ToggleWarnHandler"
      id="com.mycom.toggle.commands.warnCommand"
      name="Warn">
      <state
        class="org.eclipse.ui.handlers.RegistryToggleState:false"
        id="org.eclipse.ui.commands.toggleState">
      </state>
    </command>
    <command
      categoryId="com.mycom.toggle.commands.category"
      defaultHandler="com.mycom.toggle.handlers.ToggleErrorHandler"
      id="com.mycom.toggle.commands.errorCommand"
      name="Error">
      <state
        class="org.eclipse.ui.handlers.RegistryToggleState:false"
        id="org.eclipse.ui.commands.toggleState">
      </state>
    </command>
  </extension>
  <extension
    point="org.eclipse.ui.menus">
    <menuContribution
      locationURI="menu:org.eclipse.ui.main.menu?after=additions">
      <menu
        id="com.mycom.toggle.menus.sampleMenu"
        label="Error Level"
        mnemonic="L">
        <command
          commandId="com.mycom.toggle.commands.infoCommand"
          label="Info"
          mnemonic="I"
          style="toggle">
          <visibleWhen>
            <reference
              definitionId="com.teamcenter.rac.ui.inMainPerspective">
            </reference>
          </visibleWhen>
        </command>
      </menu>
    </menuContribution>
  </extension>

```



```

        commandId="com.mycom.toggle.commands.warnCommand"
        label="Warn"
        mnemonic="W"
        style="toggle">
<visibleWhen>
    <reference
        definitionId="com.teamcenter.rac.ui.inMainPerspective">
    </reference>
</visibleWhen>
</command>
<command
    commandId="com.mycom.toggle.commands.errorCommand"
    label="Error"
    mnemonic="E"
    style="toggle">
<visibleWhen>
    <reference
        definitionId="com.teamcenter.rac.ui.inMainPerspective">
    </reference>
</visibleWhen>
</command>
</menu>
</menuContribution>
</extension>
</plugin>

```

- b. Open the **com.mycom.toggle.handlers** package, select the **SampleHandler.java** file, choose **File→Rename**, and rename the file to **ToggleInfoHandler.java**.

Replace the contents of the **ToggleInfoHandler.java** file with the following:

```

package com.mycom.toggle.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.Command;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import org.eclipse.jface.dialogs.MessageDialog;
import com.teamcenter.rac.util.Registry;
/**
 * Our sample handler extends AbstractHandler, an IHandler base class.
 * @see org.eclipse.core.commands.IHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class ToggleInfoHandler extends AbstractHandler {

    private Registry reg ;
    /**
     * The constructor.
     */
    public ToggleInfoHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {

```

```

        Command command = event.getCommand();
        boolean checked = HandlerUtil.toggleCommandState(command);
        IWorkbenchWindow window = HandlerUtil
            .getActiveWorkbenchWindowChecked(event);
        reg = Registry.getRegistry(this);
        if (checked) {
            MessageDialog.openInformation(window.getShell(),
                reg.getString("Info.TITLE"),
                reg.getString("Info.MESSAGE"));
        }
        return null;
    }
}

```

- c. Right-click the **com.mycom.toggle.handlers** package, choose **New→Class**, and add the following Java files:

ToggleErrorHandler.java
ToggleWarnHandler.java

- d. Replace the contents of the **ToggleErrorHandler.java** file with the following:

```

package com.mycom.toggle.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.Command;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import com.teamcenter.rac.util.Registry;
public class ToggleErrorHandler extends AbstractHandler {

    private Registry reg ;
    public ToggleErrorHandler() {
    }
    @Override
    public Object execute(ExecutionEvent event) throws ExecutionException {
        Command command = event.getCommand();
        boolean checked = HandlerUtil.toggleCommandState(command);
        IWorkbenchWindow window = HandlerUtil
            .getActiveWorkbenchWindowChecked(event);
        reg = Registry.getRegistry(this);
        if (checked) {
            MessageDialog.openError(window.getShell(),
                reg.getString("Error.TITLE"),
                reg.getString("Error.MESSAGE"));
        }
        return null;
    }
}

```

- e. Replace the contents of the **ToggleWarnHandler.java** file with the following:

```

package com.mycom.toggle.handlers;
import org.eclipse.core.commands.AbstractHandler;

```

```

import org.eclipse.core.commands.Command;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import com.teamcenter.rac.util.Registry;
public class ToggleWarnHandler extends AbstractHandler {
    private Registry reg ;

    public ToggleWarnHandler() {
    }
    @Override
    public Object execute(ExecutionEvent event) throws ExecutionException {

        Command command = event.getCommand();
        boolean checked = HandlerUtil.toggleCommandState(command);
        IWorkbenchWindow window = HandlerUtil
            .getActiveWorkbenchWindowChecked(event);
        reg = Registry.getRegistry(this);
        if (checked) {
            MessageDialog.openWarning(window.getShell(),
reg.getString("Warn.TITLE"),
            reg.getString("Warn.MESSAGE"));
        }
        return null;
    }
}

```

- f. Right-click the **com.mycom.toggle.handlers** package, choose **New→File**, and add a **handlers.properties** file.
- g. Replace the contents of the **handlers.properties** file with the following:

```

Info.TITLE=ToggleInfo
Info.MESSAGE=Info menu is toggled off
Error.TITLE=ToggleError
Error.MESSAGE=Error menu is toggled off
Warn.TITLE=ToggleWarn
Warn.MESSAGE=Warn menu is toggled off

```

- h. Choose **File→Save All**.
4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the **TC_ROOT\portal** directory.

- a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

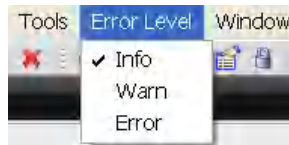
- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

5. Verify the customization.

- a. Launch My Teamcenter and choose **Error Level** on the menu bar.

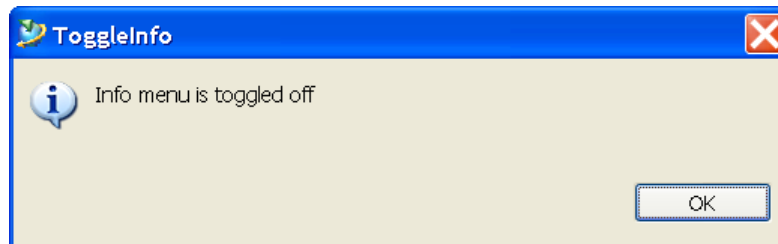
The error levels are displayed.



Error Level toggle menus

- b. Select toggles on the menu.

When you clear a toggle, a dialog box is displayed.



Toggle state dialog box

Adding views and applications

Add a view to the rich client

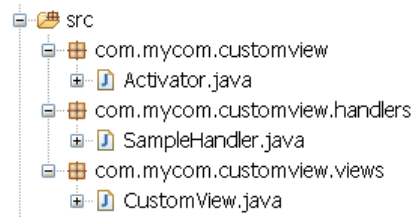
This view contains a menu item that is visible only when the view is active. It uses the following Eclipse extensions: commands, handlers, menus, expressions, and views.

Use the Eclipse plug-in create wizard to create the plug-in. Add packages and classes as needed and copy and paste the class and **plugin.xml** file content from the following steps. If you use the same names, you can cut and paste; otherwise, edit your Java files and **plugin.xml** file using the following steps as a guide.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.customview** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Click the **Dependencies** tab, click the **Add** button, and select the **com.teamcenter.rac.kernel** plug-in.
3. Create the project files.
 - a. Right-click the **com.mycom.customview** project and choose **New**→**Package**. In the **Name** box, type **com.mycom.customview.handlers**.
 - b. Right-click the **com.mycom.customview.handlers** package and choose **New**→**Class**. In the **Name** box, type **SampleHandler**.

- c. Create the **com.mycom.customview.views** package and the **CustomView** class in it.

The plug-in structure looks like this:



- d. Replace the code in the **Activator.java** file with the following:

```

package com.mycom.customview;
import org.osgi.framework.BundleContext;
import com.teamcenter.rac.kernel.AbstractRACPlugin;
import com.teamcenter.rac.services.IAspectService;
import com.teamcenter.rac.services.IAspectUIService;
/**
 * The activator class controls the plug-in life cycle
 */
public class Activator extends AbstractRACPlugin {
    // The plug-in ID
    public static final String PLUGIN_ID = "com.mycom.customview";
    // The shared instance
    private static Activator plugin;

    /**
     * The constructor
     */
    public Activator()
    {
        super();
        Activator.plugin = this;
    }
    /**
     * (non-Javadoc)
     * @see org.eclipse.core.runtime.Plugins#start
     * (org.osgi.framework.BundleContext)
     */
    public void start(BundleContext context) throws Exception {
        super.start(context);
        plugin = this;
    }
    /**
     * (non-Javadoc)
     * @see org.eclipse.core.runtime.Plugin#stop
     * (org.osgi.framework.BundleContext)
     */
    public void stop(BundleContext context) throws Exception {
        plugin = null;
        super.stop(context);
    }
    /**
     * Returns the shared instance
     */

```

```

        * @return the shared instance
        */
    public static Activator getDefault() {
        return plugin;
    }
    @Override
    public IAspectService getLogicService() {
        // TODO Auto-generated method stub
        return null;
    }
    @Override
    public IAspectUIService getUIService() {
        // TODO Auto-generated method stub
        return null;
    }
    protected void setupServices(BundleContext context) {
        // TODO Auto-generated method stub
    }
}

```

- e. Replace the code in the **SampleHandler.java** file with the following:

```

package com.mycom.customview.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.handlers.HandlerUtil;
import org.eclipse.jface.dialogs.MessageDialog;
/**
 * Our sample handler extends AbstractHandler, an IHandler base class.
 * @see org.eclipse.core.commands.IHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class SampleHandler extends AbstractHandler {
    /**
     * The constructor.
     */
    public SampleHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {
        IWorkbenchWindow window =
        HandlerUtil.getActiveWorkbenchWindowChecked(event);
        MessageDialog.openInformation(
            window.getShell(),
            "Customview",
            "Hello, from SampleHandler");
        return null;
    }
}

```

- f. Replace the code in the **CustomView.java** file with the following:

```

package com.mycom.customview.views;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Font;
import org.eclipse.swt.graphics.FontData;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Text;
import org.eclipse.ui.part.ViewPart;
public class CustomView extends ViewPart {

    /** ID corresponding to the RCP declaration */
    public static final String ID = CustomView.class.getName();

    public CustomView()
    {
        super();
    }
    @Override
    public void createPartControl(Composite parent) {
        // TODO Auto-generated method stub
        parent.setLayout( new FillLayout() );
        Text t = new Text( parent, SWT.BORDER );
        t.setBackground( parent.getDisplay().getSystemColor( SWT.COLOR_WHITE ) );
        t.setForeground( parent.getDisplay().getSystemColor( SWT.COLOR_BLUE ) );
        Font initialFont = t.getFont();
        FontData[] fontData = initialFont.getFontData();
        for (int i = 0; i < fontData.length; i++) {
            fontData[i].setHeight(18);
        }
        Font newFont = new Font(parent.getDisplay(), fontData);
        t.setFont( newFont );
        t.setText( " Hello from CustomView!! ");

    }
    @Override
    public void setFocus() {
        // TODO Auto-generated method stub

    }
}

```

- g. To create an empty **plugin.xml** file, click the **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

- h. Click the **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
    <extension point="org.eclipse.ui.commands">
        <command
            name="Sample Command"
            id="com.mycom.customview.commands.sampleCommand">
        </command>
    </extension>
    <extension point="org.eclipse.ui.handlers">

```



```

<handler
  commandId="com.mycom.customview.commands.sampleCommand"
  class="com.mycom.customview.handlers.SampleHandler">
</handler>
</extension>
<extension point="org.eclipse.ui.menus">
  <menuContribution
    locationURI="menu:org.eclipse.ui.main.menu?after=additions">
    <menu
      label="Sample Menu"
      mnemonic="M"
      id="com.mycom.customview.menus.sampleMenu">
      <command
        commandId="com.mycom.customview.commands.sampleCommand"
        mnemonic="S"
        id="com.mycom.customview.menus.sampleCommand">
        <visibleWhen>
          <reference
            definitionId="com.mycom.customview.inMainView" />
        </visibleWhen>
        </command>
      </menu>
    </menuContribution>
  </extension>
  <extension point="org.eclipse.ui.views">
    <view
      name="MyCom Custom View"
      class="com.mycom.customview.views.CustomView"
      id="com.mycom.customview.views.CustomView">
    </view>
  </extension>
  <extension point="org.eclipse.core.expressions.definitions">
    <definition id="com.mycom.customview.inMainView">
      <with variable="activePartId">
        <equals value="com.mycom.customview.views.CustomView" />
      </with>
    </definition>
  </extension>
</plugin>

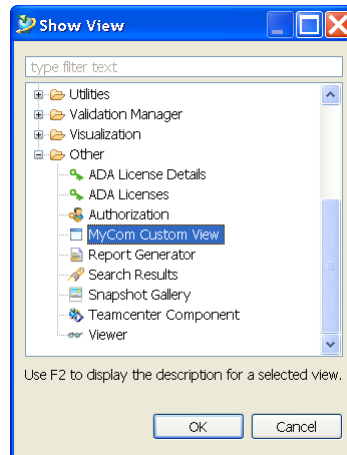
```

i. Choose **File**→**Save All**.

4. Verify the customization.

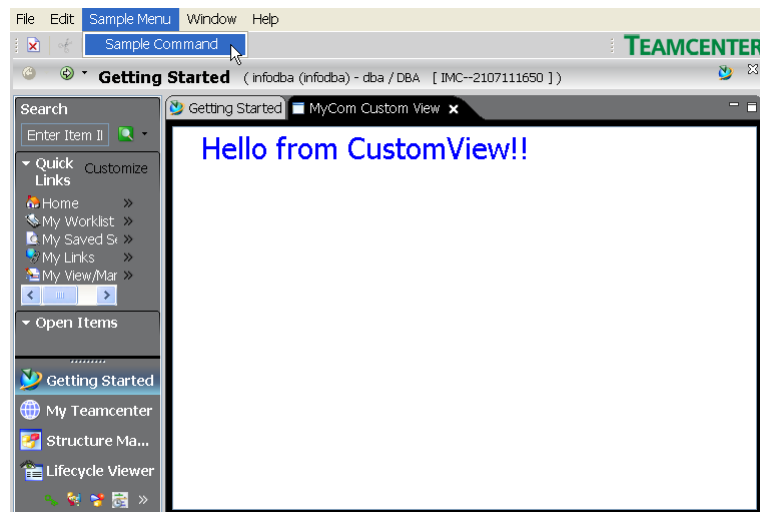
- a. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
- b. Choose **Window**→**Show View**→**Other**→**Other**→**MyCom Custom View**.

The custom view is displayed in the list.



Custom view in the list of available views

The **Sample Menu** menu command is visible when **MyCom Custom View** is the active view.



Custom menu command displayed when the custom view is open

5. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Create a view that uses the Selection Service

The following steps illustrate how to add a view that contains a JFace TableViewer to the rich client. The view is both a selection listener and selection provider and its behavior is similar to the **Details** view. This example also shows how to use Eclipse Job class and get Teamcenter services.

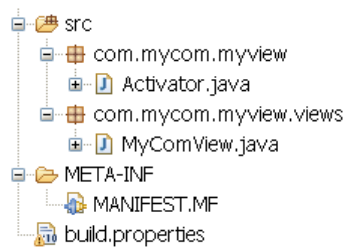
1. Create the project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.myview** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.

- a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
- b. Click the **Dependencies** tab, click the **Add** button, and select the following plug-ins:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util

3. Create the project files.
 - a. Right-click the **com.mycom.myview** project and choose **New→Package**. In the **Name** box, type **com.mycom.myview.views**.
 - b. Right-click the **com.mycom.myview.views** package and choose **New→Class**. In the **Name** box, type **MyComView**.

The plug-in structure looks like this:



- c. Replace the code in the **Activator.java** file with the following:

```

package com.mycom.myview;
import org.eclipse.jface.resource.ImageDescriptor;
import org.eclipse.ui.plugin.AbstractUIPlugin;
import org.osgi.framework.BundleContext;
/**
 * The activator class controls the plug-in life cycle
 */
public class Activator extends AbstractUIPlugin {
    // The plug-in ID
    public static final String PLUGIN_ID = "com.mycom.myview";
    // The shared instance
    private static Activator plugin;

    /**
     * The constructor
     */
    public Activator() {
    }
    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#start
     * (org.osgi.framework.BundleContext)

```

```

    */
    public void start(BundleContext context) throws Exception {
        super.start(context);
        plugin = this;
    }
    /*
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#stop
     * (org.osgi.framework.BundleContext)
     */
    public void stop(BundleContext context) throws Exception {
        plugin = null;
        super.stop(context);
    }
    /**
     * Returns the shared instance
     *
     * @return the shared instance
     */
    public static Activator getDefault() {
        return plugin;
    }
    /**
     * Returns an image descriptor for the image file at the given
     * plug-in relative path
     *
     * @param path the path
     * @return the image descriptor
     */
    public static ImageDescriptor getImageDescriptor(String path) {
        return imageDescriptorFromPlugin(PLUGIN_ID, path);
    }
}

```

- d. Replace the code in the **MyComView.java** file with the following:

```

package com.mycom.myview.views;
import org.eclipse.core.runtime.IProgressMonitor;
import org.eclipse.core.runtime.IStatus;
import org.eclipse.core.runtime.Status;
import org.eclipse.core.runtime.jobs.Job;
import org.eclipse.jface.viewers.ISelection;
import org.eclipse.jface.viewers.IStructuredContentProvider;
import org.eclipse.jface.viewers.ITableLabelProvider;
import org.eclipse.jface.viewers.LabelProvider;
import org.eclipse.jface.viewers.StructuredSelection;
import org.eclipse.jface.viewers.TableViewer;
import org.eclipse.jface.viewers.Viewer;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Image;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Display;
import org.eclipse.ui.ISelectionListener;
import org.eclipse.ui.IWorkbenchPart;
import org.eclipse.ui.part.*;
import com.mycom.myview.Activator;
import com.teamcenter.rac.aif.kernel.AIFComponentContext;
import com.teamcenter.rac.common.IContextInputService;

```

```

import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.util.OSGIUtil;
import com.teamcenter.rac.util.AdapterUtil;
// This view shows how to:
// be a selection listener
// be a selection provider
// use the Teamcenter OSGI IContextInputService
// use the Teamcenter selection to AIFComponentContext adapter
// use Eclipse Jobs (to jump off the main thread)
// ...see IC_SelectionListener()
public class MyComView extends ViewPart {
    public static final String ID = "com.mycom.myview.views.MyComView";
    private TableViewer m_viewer;
    private ISelection m_selection;
    private MyComView m_view;
    private IC_SelectionListener m_listener;
    private boolean m_firstTime = true;
    /**
     * The constructor.
     */
    public MyComView() {
        m_view = this;
    }
    @Override
    public void createPartControl(Composite parent) {
        parent.setLayout( new FillLayout() );
        // setup the viewer
        m_viewer = new TableViewer(parent, SWT.MULTI | SWT.H_SCROLL |
SWT.V_SCROLL);
        m_viewer.setContentProvider(new ViewContentProvider());
        m_viewer.setLabelProvider(new ViewLabelProvider());
        m_listener = new IC_SelectionListener();
        // register the selection listener

    getSite().getWorkbenchWindow().getSelectionService().addSelectionListener
        ( m_listener );
        // viewer is a selection provider
        getViewSite().setSelectionProvider( m_viewer );
    }
    @Override
    public void dispose() {
        // cleanup
        getViewSite().setSelectionProvider( null );

    getSite().getWorkbenchWindow().getSelectionService().removeSelectionListener
        (m_listener);
        super.dispose();
    }
    /**
     * Passing the focus request to the viewer's control.
     */
    @Override
    public void setFocus() {
        //do nothing
    }
    // this will be executed in a job...it will get the children in the job
    // and set the viewer input in the main thread
    void setViewerInput() {
        if( !(m_selection instanceof StructuredSelection) )

```

```

        {
            return;
        }
        // use the Teamcenter adapter
        AIFComponentContext ctxt = (AIFComponentContext) AdapterUtil.getAdapter(
            ( (StructuredSelection) m_selection ).getFirstElement(),
            AIFComponentContext.class, true );
        AIFComponentContext[] ctxts = null;
        if (ctxt != null)
        {
            TCComponent comp = (TCComponent) ctxt.getComponent();
            if (comp!=null)
            {
                try {
                    ctxts = comp.getChildren();
                } catch (TCEException e) {
                    // TODO Auto-generated catch block
                    e.printStackTrace();
                }
            }
        }
        final AIFComponentContext[] fctxts = ctxts;
        Display.getDefault().asyncExec(new Runnable() {
            public void run() {
                if (m_viewer.getContentProvider() != null)
                {
                    m_viewer.setInput(fctxts == null ? new
AIFComponentContext[0]
                                :
                                fctxts);
                }
            }
        });
    }
    // this is the viewer content provider
    class ViewContentProvider implements IStructuredContentProvider {
        public void inputChanged(Viewer v, Object oldInput, Object newInput) {
        }
        public void dispose() {
        }
        public Object[] getElements(Object parent) {
            return (AIFComponentContext[])parent;
        }
    }
    // this is the viewer label provider
    class ViewLabelProvider extends LabelProvider implements
    ITableLabelProvider {
        public String getColumnText(Object obj, int index) {
            AIFComponentContext ctxt = (AIFComponentContext)obj;
            TCComponent tc = (TCComponent)ctxt.getComponent();
            return tc.toString();
        }
        public Image getColumnImage(Object obj, int index) {
            return null;
        }
    }

    // this is the selection listener
    private class IC_SelectionListener implements ISelectionListener
    {

```

```

        public void selectionChanged( IWorkbenchPart part, ISelection
selection )
        {
            m_selection = selection;
            if( selection == null || selection.isEmpty() )
            {
                IContextInputService contextInputService =
(IContextInputService)

                                OSGIUtil.getService(
                                Activator.getDefault(), IContextInputService.class );
                if( contextInputService != null )
                {
                    m_selection = contextInputService.getInput();
                }
            }
            // ignore selections from this view
            // need m_firstTime to load the viewer if we open this view in a
            // perspective where it is not showing
            if( (m_firstTime) || (part != m_view) )
            {
                m_firstTime = false;
                if (m_selection != null)
                {
                    // use a job...don't want to tie up the main thread
                    Job j = new Job( "" )
                    {
                        @Override
                        protected IStatus run( IProgressMonitor monitor )
                        {
                            setViewerInput();
                            return Status.OK_STATUS;
                        }
                    };
                    j.schedule();
                }
            }
        }
    }
}

```

- e. To create an empty **plugin.xml** file, click the **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

- f. Click the **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
    <extension point="org.eclipse.ui.views">
        <view
            name="MyCom View"
            class="com.mycom.myview.views.MyComView"
            id="com.mycom.myview.views.MyComView">
        </view>
    </extension>
</plugin>

```


- g. Choose **File**→**Save All**.
- h. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- A. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- B. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

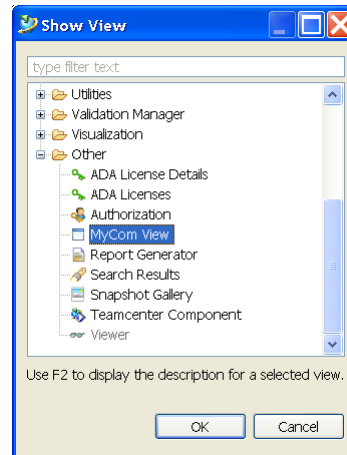
If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- C. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

- i. Verify the customization.
 - A. Launch the rich client and choose **Window**→**Show View**→**Other**→**Other**→**MyCom View**.

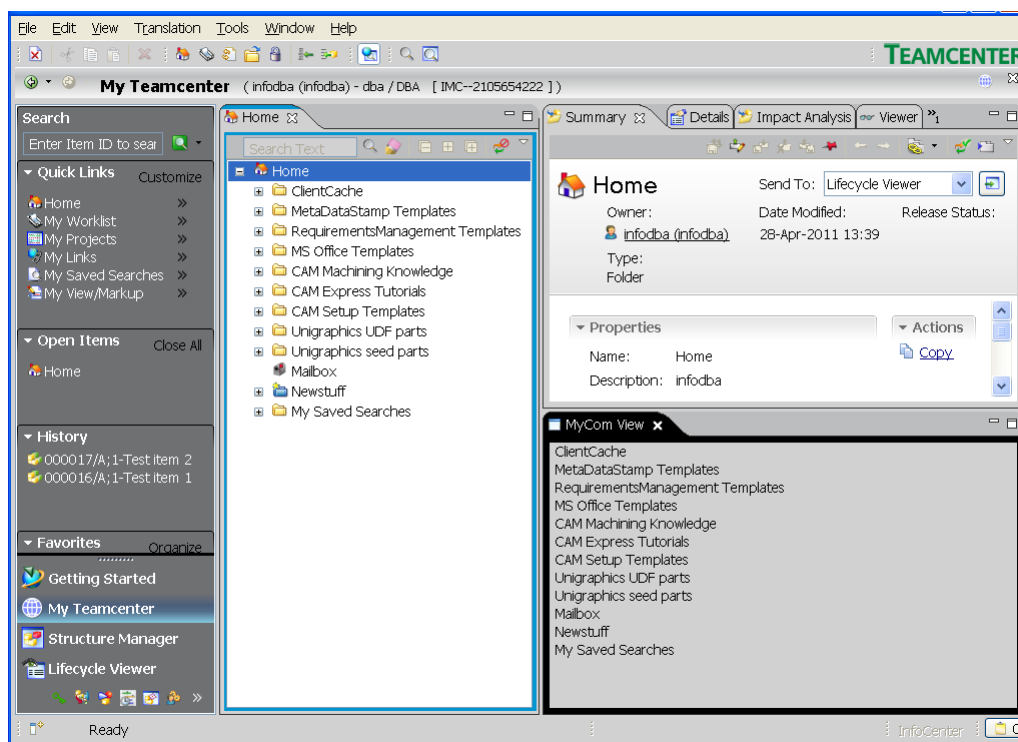
The custom view is displayed in the list.



Custom view in the list of available views

- B. Select the view.
- C. Select an object.

The **MyCom View** view shows the contents of the selected object.



Custom view displaying the contents of the selected object

Create a custom Viewer view

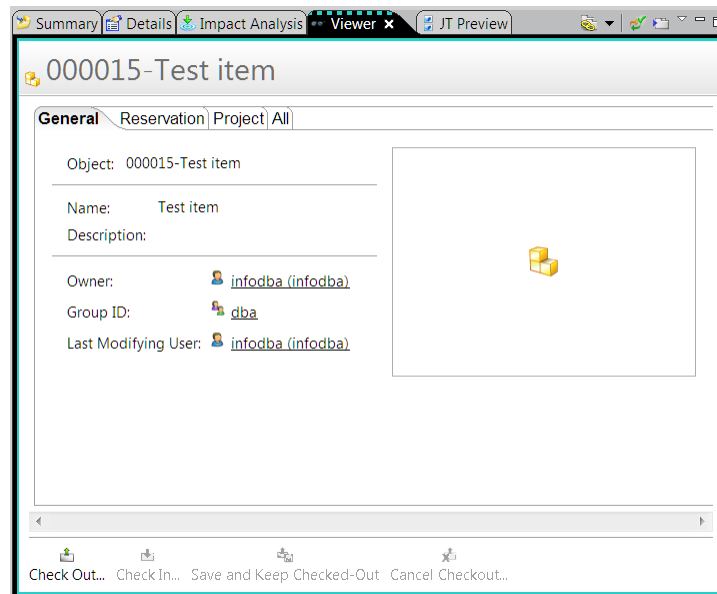
The **Viewer** view in My Teamcenter displays information about the selected object. As of Teamcenter 9.1, the **Viewer** view is converted from Swing to SWT. In this customization example using SWT, you

create two custom **Viewer** view definitions: a **HelloWorldViewer** definition that displays only the text **Hello***selected-object-name*! for the selected object, and a **MyPropertyViewer** definition that displays properties and checkin/checkout buttons at the bottom of the view.

This SWT customization example extends the **com.teamcenter.rac.viewer.ViewerViewRegistry** extension point to override the standard **TextViewer** viewer and replace it with a new viewers. In the example Java files, you implement **IViewerFactory** and **ISubViewer** components, and extend the **ViewerViewRegistry** extension point to register the new **SubView** definition.

After creating the view, you must update the **defaultViewerConfig.VIEWERCONFIG** preference to include this new view for the data type you want. For example, if you set the custom view for the text dataset type, the custom view appears when you select that dataset type.

Following is an example of a default **Viewer** view.



Viewer view

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.customviewerview** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:

- A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
- e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
- 2. Update the project tabs.
 - a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Click the **Dependencies** tab, click the **Add** button, and select the following plug-ins:
 - org.eclipse.ui**
 - org.eclipse.core.runtime**
 - com.teamcenter.rac.aifrcp**
 - com.teamcenter.rac.common**
 - com.teamcenter.rac.kernel**
 - com.teamcenter.rac.util**
 - com.teamcenter.rac.viewer**
- 3. Right-click the **com.mycom.customviewerview** project, choose **New→Package**, and create the following packages:
 - com.mycom.customviewerview.factory**
 - com.mycom.customviewerview.viewer**
- 4. Create the **HelloWorldViewer** definition.
 - a. Create the project files.
 - A. Right-click the **com.mycom.customviewerview.factory** package, choose **New→Class**, and create the **HelloWorldViewerFactory** class.
 - B. Right-click the **com.mycom.customviewerview.viewer** package, choose **New→Class**, and create the following classes:
 - HelloWorldViewer**
 - HelloWorldViewerContentProvider**
 - C. Replace the code in the **HelloWorldViewerFactory.java** file with the following:

```
package com.mycom.customviewerview.factory;

import org.eclipse.swt.widgets.Display;
```

```

import com.mycom.customviewerview.viewer.HelloWorldViewer;
import com.mycom.customviewerview.viewer.HelloWorldViewerContentProvider;
import com.teamcenter.rac.common.tcvviewer.factory.AbstractSWTViewerFactory;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.SubViewerListener;
import com.teamcenter.rac.util.viewer.ViewerEvent;

/**
 * This factory builds HelloWorldViewer.
 */
public class HelloWorldViewerFactory extends AbstractSWTViewerFactory
{
    @Override
    public void loadViewer( final SubViewerListener listener )
    {
        Runnable runnable = new Runnable()
        {
            public void run()
            {
                try
                {
                    //HelloWorldViewer viewer = new HelloWorldViewer( m_parent );
                    HelloWorldViewer viewer = new
HelloWorldViewer( getParent() );
                    viewer.setContentProvider( new
HelloWorldViewerContentProvider());
                    listener.done( viewer );
                }
                catch( Exception e)
                {
                    ViewerEvent viewerError = new ViewerEvent( this,
IViewerEvent.NOVIEWDATAFOUND );
                    listener.error( viewerError );
                }
                finally
                {
                }
            }
        };
        Display.getDefault().asyncExec( runnable );
    }
}

```

D. Replace the code in the **HelloWorldViewer.java** file with the following:

```

package com.mycom.customviewerview.viewer;

import org.eclipse.swt.SWT;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.swt.widgets.Label;
import com.teamcenter.rac.common.tcvviewer.TCComponentViewerInput;
import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.util.AdapterUtil;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.ViewerEvent;

```

```

import com.teamcenter.rac.viewer.view.AbstractSwtSubViewer;

/**
 * A simple "Hello World" example viewer.
 */
public class HelloWorldViewer
    extends AbstractSwtSubViewer //implements IAdaptable
{
    private Composite m_composite = null;
    private Label m_label = null;
    private TCCComponent m_comp = null;

    public HelloWorldViewer( Composite parent )
    {
        m_composite = new Composite( parent , SWT.NONE );
        m_composite.setLayout(new FillLayout());

        m_label = new Label( m_composite, SWT.None );
        m_label.setText( "" );
        m_label.setVisible(true);
        m_label.setEnabled(true);
    }

    /**
     * Currently this is just getting the TCCComponent object
     * from the viewer input.
     */
    @Override
    public void setInput( Object viewerInput )
    {
        TCCComponentViewerInput input = (TCCComponentViewerInput)
AdapterUtil.getAdapter( viewerInput,
        TCCComponentViewerInput.class );
        m_comp = (TCCComponent)input.getViewableObj();
        super.setInput( m_comp );
    }

    @Override
    public void inputChanged( Object input, Object oldInput )
    {
        HelloWorldViewerContentProvider cp =
(HelloWorldViewerContentProvider)
getContentProvider();
        String text = cp.getText( input );
        m_label.setText( text );

        //Notify the host viewer to reload.
        ViewerEvent viewerEvent = new ViewerEvent( this,
IVIEWER_EVENT.RELOADVIEW );
        viewerEvent.queueEvent();

        //Viewer SHOW_HEADER event will make this customized view generate
viewer header.
        ViewerEvent viewerEvent2 = new ViewerEvent( this,
IVIEWER_EVENT.SHOW_HEADER );
        viewerEvent2.queueEvent();
    }

    @Override

```

```

    public Control getControl() {
        return m_composite;
    }

    @Override
    public void refresh() {
        m_composite.layout();
    }

```

- E. Replace the code in the **HelloWorldViewerContentProvider.java** file with the following:

```

package com.mycom.customviewerview.viewer;

import org.eclipse.jface.viewers.IContentProvider;
import org.eclipse.jface.viewers.Viewer;

/**
 * ContentProvider for the HelloWorldViewer
 */
public class HelloWorldViewerContentProvider implements IContentProvider
{
    @Override
    public void dispose()
    {
        //Do nothing.
    }

    @Override
    public void inputChanged( Viewer viewer, Object oldInput, Object
newInput )
    {
        //Do nothing.
    }

    public String getText( Object input )
    {
        return "Hello " + input.toString() + "!";
    }
}

```

- F. To create an empty **plugin.xml** file, click the **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

- G. Click the **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
    <extension
        point="com.teamcenter.rac.viewer.ViewerViewRegistry">
        <viewer
            autoCheckout="false"
            factoryClassName="com.mycom.customviewerview.factory.
HelloWorldViewerFactory"
            id="HelloWorldViewer"

```

```

        isSwing="false"
        priority="100">
        <enableWhen
            checkEnabled="false">
        </enableWhen>
    </viewer>
    <viewer
        autoCheckout="false"
        factoryClassName="com.teamcenter.rac.common.tcvviewer.factory.
        SwingViewerFactory"
        id="SwingPropertyViewer"
        isSwing="true"
        priority="100"
        viewPanel="com.teamcenter.rac.common.tcvviewer.GenericViewer">
        <enableWhen
            checkEnabled="false">
        </enableWhen>
    </viewer>
</extension>

</plugin>

```

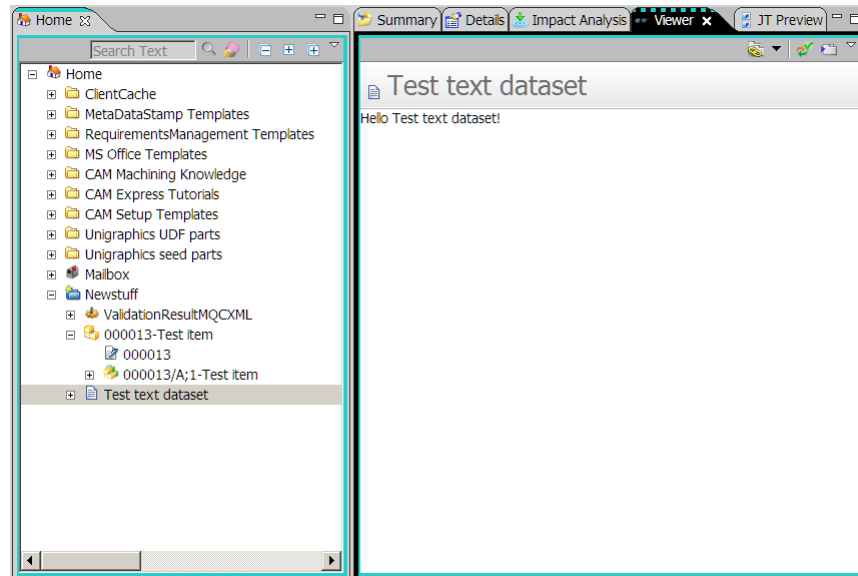
Note:

Copying and pasting the **plugin.xml** file content is a quick way to provide you with extensions for this example. However, when you create your own extensions, instead of placing content directly in the **plugin.xml** file, you work in the project's **Extensions** tab. Therefore, review the **Extensions** tab to see how you can create extensions yourself.

- H. Choose **File**→**Save All**.
- b. Verify the customization.
 - A. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
 - B. Edit the **defaultViewerConfig.VIEWERCONFIG** preference to point to the new **HelloWorldViewer** definition when you select a text dataset.

In My Teamcenter, choose **Edit**→**Options**→**Search**, open the preference, and change the **Text.TextViewer=Text** value to **Text.HelloWorldViewer**.
 - C. In My Teamcenter, create a text dataset by choosing **File**→**New**→**Dataset**, selecting **More** on the left side of the **New Dataset** dialog box, and selecting **Text** from the list of available dataset types.
 - D. Select the text dataset and click the **Viewer** view.

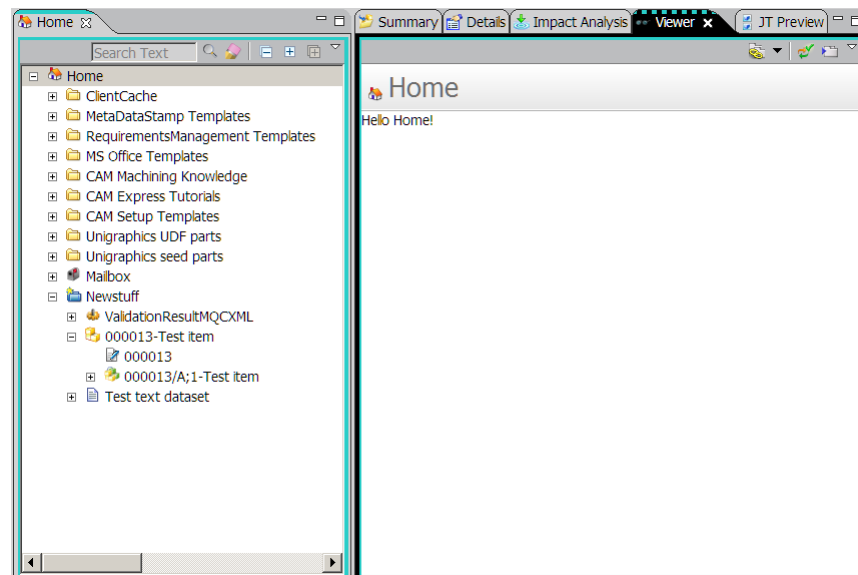
The message **Hellodataset-name!** is displayed in the **Viewer** view.



Viewer view using the HelloWorldViewer definition

- E. If you edit the **Viewer** view preferences to use the new **HelloWorldViewer** definition for another object type, the viewer is presented.

For example, if you edit the **defaultViewerConfig.VIEWERCONFIG** preference to include the **Folder.HelloWorldViewer** value, when you select a **Folder** object such as the **Home** folder, the **Viewer** view displays the following message: Hello, Home !



HelloWorldViewer view specified for another object type

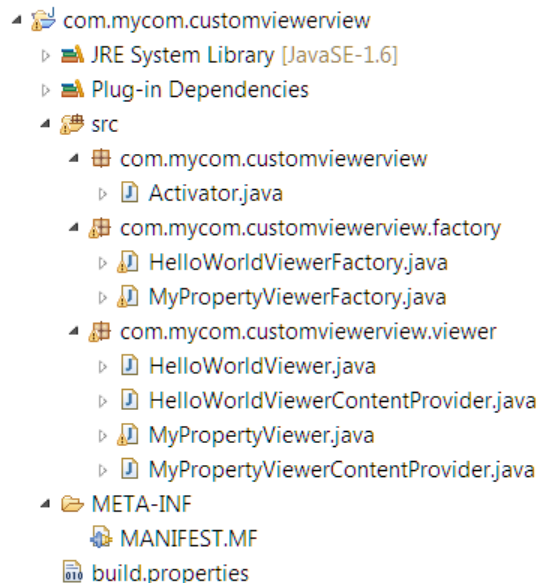
5. Create the **MyPropertyViewer** definition.

This example displays custom properties named **w2str**, **w2integer**, and **w2date** in a custom **Viewer** view. To see the properties, you must create them on a **Text** dataset, or create your own custom properties and modify the sample code.

- a. Create the project files.
 - A. Right-click the **com.mycom.customviewerview.factory** package, choose **New→Class**, and create the **MyPropertyViewerFactory** class.
 - B. Right-click the **com.mycom.customviewerview.viewer** package, choose **New→Class**, and create the following classes:

MyPropertyViewer
MyPropertyViewerContentProvider

The plug-in structure looks like this:



- C. Replace the code in the **MyPropertyViewerFactory.java** file with the following:

```
package com.mycom.customviewerview.factory;

import org.eclipse.swt.widgets.Display;

import com.mycom.customviewerview.viewer.MyPropertyViewer;
import com.mycom.customviewerview.viewer.MyPropertyViewerContentProvider;
import com.teamcenter.rac.common.tcviewer.factory.AbstractSWTViewerFactory;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.SubViewerListener;
import com.teamcenter.rac.util.viewer.ViewerEvent;

/**
 */
```

```

public class MyPropertyViewerFactory extends AbstractSWTViewerFactory
{
    @Override
    public void loadViewer( final SubViewerListener listener )
    {
        Runnable runnable = new Runnable()
        {
            public void run()
            {
                try
                {
                    MyPropertyViewer viewer = new
MyPropertyViewer( getParent() );
                    viewer.setContentProvider( new
MyPropertyViewerContentProvider() );
                    listener.done( viewer );
                }
                catch( Exception e)
                {
                    e.printStackTrace();
                    ViewerEvent viewerError = new ViewerEvent( this,
IViewerEvent.NOVIEWDATAFOUND );
                    listener.error( viewerError );
                } finally {
                }
            }
        };
        Display.getDefault().asyncExec( runnable );
    }
}

```

D. Replace the code in the **MyPropertyViewer.java** file with the following:

```

// @<COPYRIGHT>@
// =====
// Copyright 2011.
// Siemens Product Lifecycle Management Software Inc.
// All Rights Reserved.
// =====
// @<COPYRIGHT>@

package com.mycom.customviewerview.viewer;

import java.util.Arrays;
import java.util.Date;

import org.eclipse.core.runtime.IProgressMonitor;
import org.eclipse.swt.SWT;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.swt.widgets.Label;
import org.eclipse.ui.ISaveablePart;

import com.teamcenter.rac.aif.kernel.AIFComponentChangeEvent;

```

```

import com.teamcenter.rac.aif.kernel.AIFComponentEvent;
import com.teamcenter.rac.aif.kernel.AIFComponentPropertyChangeEvent;
import com.teamcenter.rac.aif.kernel.InterfaceAIFComponent;
import com.teamcenter.rac.aif.kernel.InterfaceAIFComponentEventListener;
import com.teamcenter.rac.aifrcp.AIFUtility;
import com.teamcenter.rac.common.SoaPropertyHelper;
//import com.teamcenter.rac.common.controls.LOVComboBox;
import com.teamcenter.rac.common.tcviewer.TCComponentViewerInput;
import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.util.AdapterUtil;
import com.teamcenter.rac.util.MessageBox;
import com.teamcenter.rac.util.SWTUIUtilities;
import com.teamcenter.rac.util.controls.DateControl;
import com.teamcenter.rac.util.controls.TextControl;
import com.teamcenter.rac.util.event.ClientEventDispatcher;
import com.teamcenter.rac.util.event.IClientEvent;
import com.teamcenter.rac.util.viewer.IViewerEvent;
import com.teamcenter.rac.util.viewer.ViewerEvent;
import com.teamcenter.rac.viewer.stylesheet.StylesheetRenderingConstants;
import com.teamcenter.rac.viewer.utils.CheckInOutComposite;
import com.teamcenter.rac.viewer.view.AbstractSwtSubViewer;

/**
 * Example Viewer which shows how to display a list of properties. This example
 * contains textfield, textarea, date, integer. If you have a LOV attached, you
 * can un-comment the lines for lov related code. (The example uses a string LOV.)
 * To use this example, you need to add extension
 * "com.teamcenter.rac.viewer.ViewerViewRegistry" and
 * modify preference "defaultViewerConfig.VIEWERCONFIG". For more detail, please refer to
 * RAC Customization Guide.
 */
public class MyPropertyViewer
    extends AbstractSwtSubViewer implements ISaveablePart, InterfaceAIFComponentEventListener
{
    private Composite m_composite = null;
    private TCComponent m_comp = null;

    private Label m_w2StrLabel = null;
    private TextControl m_w2StrText;
    private TCProperty m_w2StrProp;

    private Label m_w2IntegerLabel = null;
    private TextControl m_w2IntegerText;
    private TCProperty m_w2IntProp;

    private Label m_w2DateLabel = null;
    private DateControl m_w2DateButton;
    private TCProperty m_w2DateProp;

    private Label m_w2ObjDescLabel = null;
    private TextControl m_w2ObjDescText;
    private TCProperty m_w2ObjDescProp;

    // private Label m_lovLabel;
    // private LOVComboBox m_lovComboBox;
    // private TCProperty m_lovProp;

    public static String STR_PROP = "aos0str";
    public static String INT_PROP = "aos0integer";

```

```

public static String DATE_PROP = "aos0date";

public MyPropertyViewer( Composite parent )
{
    GridLayout parentLayout = SWTUIUtilities.tightGridLayout( 1 );

    Composite mainComposite =new Composite( parent , SWT.NONE );
    mainComposite.setLayout(parentLayout);

    m_composite = new Composite( mainComposite , SWT.NONE );
    m_composite.setLayout(new GridLayout(2, false) );
    m_composite.setLayoutData( new GridData( SWT.FILL, SWT.FILL, true, true ) );

    m_w2StrLabel = new Label( m_composite, SWT.None );
    m_w2StrText = new TextControl( m_composite, SWT.SINGLE | SWT.FLAT | SWT.BORDER );

    GridData m_w2StrTextGd = new GridData();
    m_w2StrTextGd.horizontalAlignment = SWT.FILL;
    m_w2StrTextGd.verticalAlignment = SWT.CENTER;
    m_w2StrTextGd.widthHint = 30 *
StylesheetRenderingConstants.DEFAULT_TEXTFIELD_COL_WIDTH;
    m_w2StrText.getTextWidget().setLayoutData( m_w2StrTextGd );

    m_w2IntegerLabel = new Label( m_composite, SWT.None );
    m_w2IntegerText = new TextControl( m_composite, SWT.SINGLE | SWT.FLAT | SWT.BORDER );
    m_w2IntegerText.getTextWidget().setLayoutData( m_w2StrTextGd );

    m_w2DateLabel = new Label( m_composite, SWT.None );
    m_w2DateButton = new DateControl( m_composite );

    m_w2ObjDescLabel = new Label( m_composite, SWT.None );
    m_w2ObjDescText = new TextControl( m_composite, SWT.WRAP | SWT.MULTI | SWT.V_SCROLL
| SWT.BORDER );

    GridData taGd = new GridData();
    taGd.horizontalAlignment = SWT.FILL;
    taGd.verticalAlignment = SWT.CENTER;
    // gd.grabExcessHorizontalSpace = true;
    taGd.widthHint = 30 * StylesheetRenderingConstants.DEFAULT_TEXTAREA_COL_WIDTH;
    taGd.heightHint = 5 * StylesheetRenderingConstants.DEFAULT_TEXTAREA_ROW_HEIGHT;
    m_w2ObjDescText.getTextWidget().setLayoutData( taGd );

    //      m_lovLabel = new Label( m_composite, SWT.None );
    //      m_lovComboBox = new LOVComboBox(m_composite, SWT.None);

    CheckInOutComposite m_cicoComposite = new CheckInOutComposite( mainComposite );
    m_cicoComposite.panelLoaded();

    AIFUtility.getDefaultSession().addAIFComponentEventListener( this );
}

/*
 * Currently this is just getting the TCComponent object
 * from the viewer input.
 */
@Override
public void setInput( Object viewerInput )
{
    TCComponentViewerInput input = (TCComponentViewerInput)

```

```

AdapterUtil.getAdapter( viewerInput,
    TCComponentViewerInput.class );
m_comp = (TCComponent)input.getViewableObj();
super.setInput( m_comp );
}

@Override
public void inputChanged( Object input, Object oldInput )
{
    MyPropertyViewerContentProvider cp = (MyPropertyViewerContentProvider)
getContentProvider();

    m_w2StrLabel.setText( cp.getPropertyDisplayName(STR_PROP) );
    m_w2StrProp = cp.getTCProperty(STR_PROP);
    m_w2StrText.getTextWidget().setText(m_w2StrProp.getStringValue());

    m_w2IntegerLabel.setText(cp.getPropertyDisplayName(INT_PROP));
    m_w2IntProp = cp.getTCProperty(INT_PROP);
    int ii = m_w2IntProp.getIntValue();
    if( !m_w2IntProp.getNullVerdict() )
    {
        m_w2IntegerText.getTextWidget().setText(Integer.toString(ii));
    }

    m_w2DateLabel.setText(cp.getPropertyDisplayName(DATE_PROP));
    m_w2DateProp = cp.getTCProperty(DATE_PROP);
    m_w2DateButton.setDate(m_w2DateProp.getDateValue());

    m_w2ObjDescLabel.setText(cp.getPropertyDisplayName("object_desc"));
    m_w2ObjDescProp = cp.getTCProperty("object_desc");
    m_w2ObjDescText.getTextWidget().setText(m_w2ObjDescProp.getStringValue());

//    m_lovLabel.setText(cp.getPropertyDisplayName("w2Str_e"));
//    m_lovProp = cp.getTCProperty("w2Str_e");
//    m_lovComboBox.setLOVComponent(m_lovProp.getLOV());
//    m_lovComboBox.setSelectedItem( m_lovProp.getPropertyData() );

    //Notify the host viewer to reload.
    ViewerEvent viewerEvent = new ViewerEvent( this, IViewerEvent.RELOADVIEW );
    viewerEvent.queueEvent();
}

@Override
public Control getControl()
{
    return m_composite;
}

@Override
public void refresh()
{
    if( m_composite == null || m_composite.isDisposed() )
    {
        return;
    }
    m_composite.layout();

    if( m_comp.isCheckedOut() )
    {
        // Make panel writable

```

```

        m_w2StrText.getTextWidget().setEditable(true);
        m_w2IntegerText.getTextWidget().setEditable(true);
        m_w2DateButton.setEnabled(true);
        m_w2ObjDescText.getTextWidget().setEditable(true);
    }
    else
    {
        // make panel read-only or just show values in label like OOTB behavior.
        m_w2StrText.getTextWidget().setEditable(false);
        m_w2IntegerText.getTextWidget().setEditable(false);
        m_w2DateButton.setEnabled(false);
        m_w2ObjDescText.getTextWidget().setEditable(false);
    }
}

// @Override
// public Object getAdapter( Class adapter )
// {
//     if( adapter.equals( IContributionItem[].class ) )
//     {
//         return getCommandContributions();
//     }
//     return null;
// }

// private IContributionItem[] getCommandContributions()
// {
//     List<IContributionItem> list = new ArrayList<IContributionItem>();
//     CommandContributionItemParameter contributionParameters = new
CommandContributionItemParameter(
//         PlatformUI.getWorkbench(), "", "com.teamcenter.rac.checkOut",
CommandContributionItem.STYLE_PUSH );
//     contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//     list.add( new CommandContributionItem( contributionParameters ) );
//     contributionParameters = new
CommandContributionItemParameter( PlatformUI.getWorkbench(), "",
//         "com.teamcenter.rac.checkIn", CommandContributionItem.STYLE_PUSH );
//     contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//     list.add( new CommandContributionItem( contributionParameters ) );
//     contributionParameters = new
CommandContributionItemParameter( PlatformUI.getWorkbench(), "",
//         "com.teamcenter.rac.saveCheckOut", CommandContributionItem.STYLE_PUSH );
//     contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//     list.add( new CommandContributionItem( contributionParameters ) );
//     contributionParameters = new
CommandContributionItemParameter( PlatformUI.getWorkbench(), "",
//         "com.teamcenter.rac.cancelCheckOut", CommandContributionItem.STYLE_PUSH );
//     contributionParameters.mode = CommandContributionItem.MODE_FORCE_TEXT;
//     list.add( new CommandContributionItem( contributionParameters ) );
//     return list.toArray( new IContributionItem[list.size()] );
// }

// Implementation of ISaveablePart
@Override

```

```

public boolean isDirty()
{
    // Check if the value has been changed
    String txt = m_w2StrText.getTextWidget().getText();
    if( !txt.equals(m_w2StrProp.getStringValue()) )
    {
        return true;
    }

    txt = m_w2IntegerText.getTextWidget().getText();
    if( !txt.isEmpty() && m_w2IntProp.getNullVerdict() ||
        txt.isEmpty() && !m_w2IntProp.getNullVerdict() )
    {
        return true;
    }
    if(!txt.isEmpty() && !txt.equals(Integer.toString(m_w2IntProp.getIntValue())))
    {
        return true;
    }

    Date date = m_w2DateButton.getDate();
    if( (date == null && !m_w2DateProp.getNullVerdict()) ||
        (date != null && m_w2DateProp.getNullVerdict()) )
    {
        return true;
    }
    if( date != null && !date.equals(m_w2DateProp.getDateValue()) )
    {
        return true;
    }

    txt = m_w2ObjDescText.getTextWidget().getText();
    if( !txt.equals(m_w2ObjDescProp.getStringValue()) )
    {
        return true;
    }

    // Object obj = m_lovComboBox.getSelectedObject();
    // if( obj == null && !m_lovProp.getNullVerdict() ||
    //     obj != null && m_lovProp.getNullVerdict() )
    // {
    //     return true;
    // }
    // if( obj != null && !obj.equals(m_lovProp.getPropertyData()) )
    // {
    //     return true;
    // }
    return false;
}

// Implementation of ISaveablePart
@Override
public boolean isSaveAsAllowed()
{
    return false;
}

// Implementation of ISaveablePart
@Override
public boolean isSaveOnCloseNeeded()

```



```

{
    return true;
}

// Implementation of ISaveablePart
@Override
public void doSaveAs()
{
    //not supported
}

// Implementation of ISaveablePart
@Override
public void doSave( final IProgressMonitor monitor )
{
    Exception ex2 = null;
    try
    {
        // If you want to setBusy(true), make sure it's reset to false after operation
        // Otherwise, after you modify the object, then select another object, the
        // confirmation dialog
        // about saving changes will not be displayed as the code considers the Viewer
        // is still busy.
        //setBusy( true );

        // Save any changes in the viewer
        TCProperty[] props = new TCProperty[] {m_w2StrProp, m_w2IntProp, m_w2DateProp,
m_w2ObjDescProp};
        //TCProperty[] props = new TCProperty[] {m_w2ObjDescProp, m_lovProp};

        String txt = m_w2StrText.getTextWidget().getText();
        m_w2StrProp.setStringValueData(txt);

        txt = m_w2IntegerText.getTextWidget().getText();
        if( txt.isEmpty() )
        {
            m_w2IntProp.setNullVerdict(true);
        }
        else
        {
            int ii = Integer.parseInt(txt);
            m_w2IntProp.setIntValueData(ii);
        }

        Date date = m_w2DateButton.getDate();
        m_w2DateProp.setDateValueData(date);

        txt = m_w2ObjDescText.getTextWidget().getText();
        m_w2ObjDescProp.setStringValueData(txt);

        //      Object obj = m_lovComboBox.getSelectedObject();
        //      if( obj == null )
        //      {
        //          m_lovProp.setNullVerdict(true);
        //      }
        //      else
        //      {
        //          m_lovProp.setStringValueData(obj.toString());
        //      }
    }
    catch (Exception ex)
    {
        ex2 = ex;
    }
    if (ex2 != null)
    {
        monitor.reportException(ex2);
    }
}

```

```

        SoaPropertyHelper.setPropertiesService( m_comp, props );
    }
    catch( Exception e )
    {
        ex2 = e;
        MessageBox.post(e);
    }
    finally
    {
        ClientEventDispatcher.fireEventLater( MyPropertyViewer.this,
            IClientEvent.SS_VIEWER_SAVE_COMPLETE, TCCComponent.class, m_comp,
Exception.class,
            ex2 );
    }
}

// Implement InterfaceAIFComponentEventListener
@Override
public void processComponentEvents( AIFComponentEvent[] events )
{
    if( getInput() == null )
    {
        // nothing to be done.
        return;
    }

    boolean requireUpdate = false;
    Arrays.sort( events );
    for( AIFComponentEvent event : events )
    {
        InterfaceAIFComponent targetComponent = event.getComponent();

        if( targetComponent == getInput() ) // changed.
        {
            if( event instanceof AIFComponentPropertyChangeEvent )
            {
                // For property change event, don't need to refresh the whole panel
            }
            else if( event instanceof AIFComponentChangeEvent )
            {
                requireUpdate = true;
            }
        }
    }

    if( requireUpdate )
    {
        SWTUIUtilities.asyncExec( new Runnable()
        {
            // update the check in/out UI widgets only
            @Override
            public void run()
            {
                refresh();
            }
        } );
    }
}
}

```

For the **Check-in**, **Check-out**, and **Cancel Check-out** buttons, you do not need to implement the **IAdaptable** or **ISaveablePart** methods. Instead, create the **CheckInOutComposite** component and add it to your viewer, for example:

```
CheckInOutComposite m_cicoComposite = new
CheckInOutComposite( mainComposite );
m_cicoComposite.panelLoaded();
```

In the **doSave()** method, run the save complete event so that checkin can be launched after saving for the **Check-in** button:

```
ClientEventDispatcher.fireEventLater( MyPropertyViewer.this,
    IClientEvent.SS_VIEWER_SAVE_COMPLETE, TCComponent.class, m_comp,
    Exception.class,
    ex2 );
```

For **Form** objects, the form display in the **Viewer** view is now in SWT tagging. To use the legacy Swing **FormViewer** object in the **Viewer** view for all forms, modify the **defaultViewerConfig.VIEWERCONFIG** preference and add the following entry: **Form.FORMViewer**. To use the legacy Swing **FormViewer** object for a certain form type, add an entry to the **defaultViewerConfig.VIEWERCONFIG** preference for the form type that must be displayed in the legacy view, for example, *form-type-name.FORMViewer*.

- E. Replace the code in the **MyPropertyViewerContentProvider.java** file with the following:

```
package com.mycom.customviewerview.viewer;

import org.eclipse.jface.viewers.IContentProvider;
import org.eclipse.jface.viewers.Viewer;

import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.kernel.TCPropertyDescriptor;
import com.teamcenter.rac.util.AdapterUtil;

/**
 * ContentProvider for the MyPropertyViewer
 */
public class MyPropertyViewerContentProvider implements IContentProvider
{
    private TCComponent m_comp;

    @Override
    public void dispose()
    {
    }

    @Override
    public void inputChanged( Viewer viewer, Object oldInput, Object
newInput )
    {
        String[] propNames = new String[] { "w2str", "w2integer", "w2date",
"object_desc" };
    }
}
```

```

        m_comp = (TCComponent)AdapterUtil.getAdapter(newInput,
TCComponent.class);
        if( m_comp == null )
        {
            return;
        }

        try
        {
            // Pre-load properties in one call to reduce the network calls.
            m_comp.getTCProperties(propNames);
        }
        catch (TCEException e)
        {
            e.printStackTrace();
        }
    }

    /**
     * Return the TCProperty
     * @param propName The property name
     * @return TCProperty
     */
    public TCProperty getTCProperty(String propName)
    {
        try
        {
            return m_comp.getTCProperty(propName);
        }
        catch (TCEException e)
        {
            e.printStackTrace();
        }
        return null;
    }

    /**
     * Return the property's display name
     * @param propName The property name
     * @return String
     */
    public String getPropertyDisplayName( String propName )
    {
        try
        {
            TCPropertyDescriptor propDesc = m_comp.getTypeComponent()
.getPropertyDescriptor(propName);
            return propDesc.getDisplayName();
        }
        catch (TCEException e)
        {
            e.printStackTrace();
        }
        return propName;
    }

    /**
     * Return the property value
     * @param propName The property name
     * @return Object

```

```

    */
    public Object getPropertyValue(String propName)
    {
        try
        {
            TCProperty prop = m_comp.getTCProperty(propName);
            if( prop != null )
            {
                return prop.getPropertyData();
            }
        }
        catch (Exception e)
        {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        return null;
    }
}

```

F. Click the **plugin.xml** tab and add the code in **bold** to the **plugin.xml** file:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
    <extension
        point="com.teamcenter.rac.viewer.ViewerViewRegistry">
        <viewer
            autoCheckout="false"
            factoryClassName="com.mycom.customviewerview.factory.
HelloWorldViewerFactory"
            id="HelloWorldViewer"
            isSwing="false"
            priority="100">
            <enableWhen
                checkEnabled="false">
            </enableWhen>
        </viewer>
        <viewer
            autoCheckout="false"
            factoryClassName="com.mycom.customviewerview.factory.
MyPropertyViewerFactory"
            id="MyPropertyViewer"
            isSwing="false"
            priority="100">
            <enableWhen
                checkEnabled="false">
            </enableWhen>
        </viewer>
        <viewer
            autoCheckout="false"
            factoryClassName="com.teamcenter.rac.common.tcvviewer.factory.
SwingViewerFactory"
            id="SwingPropertyViewer"
            isSwing="true"
            priority="100"
            viewPanel="com.teamcenter.rac.common.tcvviewer.GenericViewer">
            <enableWhen
                checkEnabled="false">

```

```
        </enableWhen>
      </viewer>
    </extension>

</plugin>
```

Note:

Copying and pasting content into the **plugin.xml** file is a quick way to provide you with extensions for this example. However, when you create your own extensions, instead of placing content directly in the **plugin.xml** file, you work in the project's **Extensions** tab. Therefore, review the **Extensions** tab to see how you can create extensions yourself.

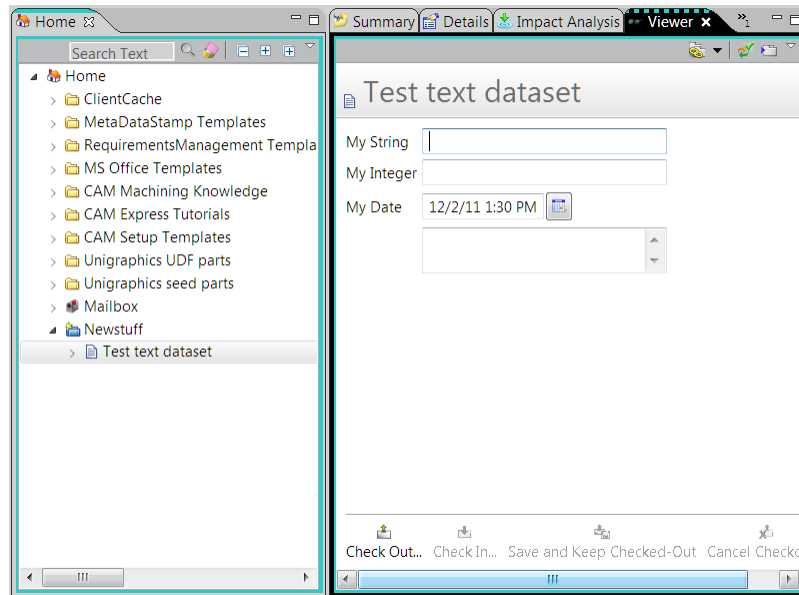
- G. Choose **File**→**Save All**.
- b. Verify the customization.
 - A. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
 - B. Edit the **defaultViewerConfig.VIEWERCONFIG** preference to point to the new **MyPropertyViewer** viewer when you select a text dataset.

In My Teamcenter, choose **Edit**→**Options**→**Search**, open the preference, and change the **Text.HelloWorldViewer** value to **Text.MyPropertyViewer** to load the other viewer when you select a text dataset.

- C. Select the text dataset and click the **Viewer** view.

The **Viewer** view displays the dataset information using the **MyPropertyViewer** definition.

Notice the checkin and checkout button at the bottom of the view.



Viewer view using the MyPropertyViewer definition

6. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the `portal\Registry` directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory.
 On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Add a new rich client application

This code creates a new application in the rich client that contains a single perspective and a single view. It is displayed in the navigation pane. The project that contains the application uses the following Eclipse extensions: `org.eclipse.ui.perspectives`, `org.eclipse.core.expressions.definitions`, and `org.eclipse.ui.views`. It also uses the Teamcenter `com.teamcenter.rac.aifrcp.application` extension.

Use the Eclipse plug-in create wizard to create the plug-in. Add packages and classes as needed and copy and paste the class and `plugin.xml` file content from the following steps. If you use the same names, you can cut and paste; otherwise, edit your Java files and `plugin.xml` file using the following steps as a guide.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type `com.mycom.customapp` in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Click the **Dependencies** tab, click the **Add** button, and select the `com.teamcenter.rac.kernel` plug-in.
3. Create the project files.
 - a. Create the following:

- The **com.mycom.customapp.perspectives** package and the **CustomPerspective** class in it.

Note:

To create a package, right-click the project and choose **New→Package**. To create a class, right-click the package and choose **New→Class**.

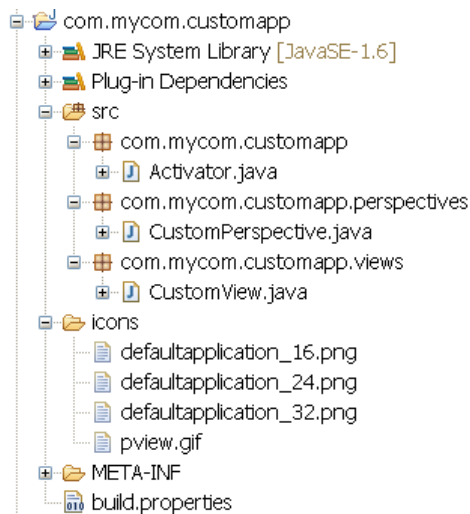
- The **com.mycom.customapp.views** package and the **CustomView** class in it.
- An **icons** directory.

Note:

To create a directory, right-click the project and choose **New→Folder**.

- Open the **com.teamcenter.rac.aifrcp_version.jar** in the **TC_ROOT\portal\plugins** directory, extract all the **defaultapplication_size.png** files and the **pview.gif** file, and copy the files to the **icons** directory you just created.

The plug-in structure looks like this:



- Replace the code in the **Activator.java** file with the following:

```
package com.mycom.customapp;
import org.osgi.framework.BundleContext;
import com.teamcenter.rac.kernel.AbstractRACPlugin;
import com.teamcenter.rac.services.IAspectService;
import com.teamcenter.rac.services.IAspectUIService;
/**
 * The activator class controls the plug-in life cycle
 */
```

```

public class Activator extends AbstractRACPlugin {
    // The plug-in ID
    public static final String PLUGIN_ID = "com.mycom.customapp";
    // The shared instance
    private static Activator plugin;

    /**
     * The constructor
     */
    public Activator() {
    }
    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#start
     * (org.osgi.framework.BundleContext)
     */
    @Override
    public void start(BundleContext context) throws Exception {
        super.start(context);
        plugin = this;
    }
    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#stop
     * (org.osgi.framework.BundleContext)
     */
    @Override
    public void stop(BundleContext context) throws Exception {
        plugin = null;
        super.stop(context);
    }
    /**
     * Returns the shared instance
     *
     * @return the shared instance
     */
    public static Activator getDefault() {
        return plugin;
    }
    @Override
    public IAspectService getLogicService() {
        // TODO Auto-generated method stub
        return null;
    }
    @Override
    public IAspectUIService getUIService() {
        // TODO Auto-generated method stub
        return null;
    }
    protected void setupServices(BundleContext context) {
        // TODO Auto-generated method stub
    }
}

```

- d. Replace the code in the **CustomPerspective.java** file with the following:

```

package com.mycom.customapp.perspectives;
import org.eclipse.ui.IFolderLayout;

```

```

import org.eclipse.ui.IPageLayout;
import org.eclipse.ui.IPerspectiveFactory;
public class CustomPerspective implements IPerspectiveFactory {

    /** The perspective ID */
    public static final String ID =
        "com.mycom.customapp.perspectives.CustomPerspective";
    @Override
    public void createInitialLayout(IPageLayout layout) {
        // TODO Auto-generated method stub
        layout.setEditorAreaVisible(false);
        String editorArea = layout.getEditorArea();
        IFolderLayout top = layout.createFolder("top", IPageLayout.TOP, -2f,
            editorArea);
        top.addView( "com.mycom.customapp.views.CustomView" );
    }
}

```

- e. Replace the code in the **CustomView.java** file with the following:

```

package com.mycom.customapp.views;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Font;
import org.eclipse.swt.graphics.FontData;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Text;
import org.eclipse.ui.part.ViewPart;
public class CustomView extends ViewPart {

    public CustomView() {
        super();
    }
    @Override
    public void createPartControl(Composite parent) {
        // TODO Auto-generated method stub
        parent.setLayout( new FillLayout() );
        Text t = new Text( parent, SWT.BORDER );
        t.setBackground( parent.getDisplay().getSystemColor( SWT.COLOR_WHITE ) );
        t.setForeground( parent.getDisplay().getSystemColor( SWT.COLOR_BLUE ) );
        Font initialFont = t.getFont();
        FontData[] fontData = initialFont.getFontData();
        for (int i = 0; i < fontData.length; i++) {
            fontData[i].setHeight(18);
        }

        Font newFont = new Font(parent.getDisplay(), fontData);
        t.setFont( newFont );
        t.setText( " Welcome to Custom Application !! ");
    }
    @Override
    public void setFocus() {
        // TODO Auto-generated method stub
    }
}

```

- f. To create an empty **plugin.xml** file, click the project's **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

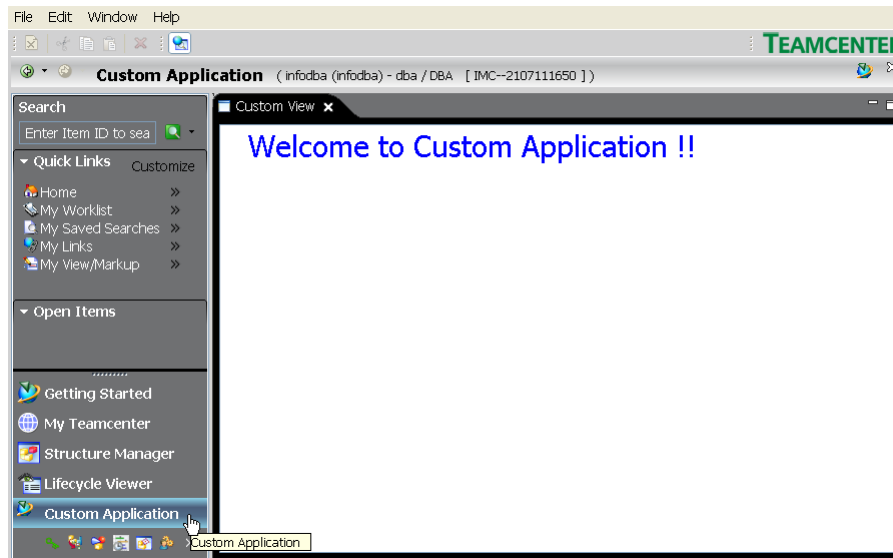
- g. Click the project's **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension point="com.teamcenter.rac.aifrcp.application">
    <aif_app_item
      displayMode="Primary"
      groupName="Mycompany"
      icon="icons/defaultapplication_32.png"
      id="com.mycom.customapp"
      name="Custom Application"
      ordinality="200"
      perspective_id="com.mycom.customapp.perspectives.CustomPerspective"
      session="com.teamcenter.rac.kernel.TCSession"
      tooltip="Custom Application"/>
  </extension>
  <extension point="org.eclipse.ui.perspectives">
    <perspective
      class="com.mycom.customapp.perspectives.CustomPerspective"
      icon="icons/defaultapplication_16.png"
      id="com.mycom.customapp.perspectives.CustomPerspective"
      name="Custom Application"/>
  </extension>
  <extension point="org.eclipse.ui.views">
    <view
      allowMultiple="false"
      class="com.mycom.customapp.views.CustomView"
      icon="icons/pview.gif"
      id="com.mycom.customapp.views.CustomView"
      name="Custom View"/>
  </extension>
  <extension point="org.eclipse.core.expressions.definitions">
    <definition id="com.mycom.customapp.inMainView">
      <or>
        <with variable="activePartId">
          <equals value="com.mycom.customapp.views.CustomView" />
        </with>
        <with variable="arc_property.ACTIVE_APPLICATION">
          <equals value="com.mycom.customapp" />
        </with>
      </or>
    </definition>
  </extension>
</plugin>
```

This example does not require the definitions extension; you can remove it. It is included for reference to show how you can determine if your application is active.

4. Choose **File**→**Save All**.
5. Verify the customization.

- a. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
- b. Verify that the new **Custom Application** button is shown in the left-hand navigation pane in the rich client.
- c. Click the **Custom Application** button to launch the new application.



Launching the custom application

6. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

Add an application to the Teamcenter Send To menu

This code allows users in a different application to select an appropriate object, choose an application from the **Send To** shortcut menu, and send the selected object to the custom application.

This code creates a new application that uses a **CumstomHandler** handler. Use the Eclipse plug-in create wizard to create the plug-in. Add packages and classes as needed and copy and paste the class and the **plugin.xml** file content from the following steps. If you use the same names, you can cut and paste; otherwise, edit your Java files and the **plugin.xml** file using the following steps as a guide.

Note:

To hide the **Send To** menu in the rich client, create a custom plug-in with code similar to the following in the **plugin.xml** file:

```
<extension
    point="org.eclipse.ui.activities">
    <activity
        description="This will hide the Send To submenu"
        id="com.mycom.sendtohidden"
        name="Sendto Activity">
    </activity>
    <activityPatternBinding
        activityId="com.mycom.sendtohidden"
        pattern="com.teamcenter.rac.common/com.teamcenter.rac.sendto">
    </activityPatternBinding>
</extension>
```

1. Create the project.
 - a. In Eclipse, choose **File→New→Project**.

- b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.sendtoapp** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.
 - a. Select the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Select the **Dependencies** tab, click the **Add** button, and select the following plug-ins:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel

3. Create the project files.
 - a. Create the following:
 - The **com.mycom.sendtoapp.perspectives** package and the **CustomPerspective** class in it.

Note:

To create a package, right-click the project and choose **New→Package**. To create a class, right-click the package and choose **New→Class**.

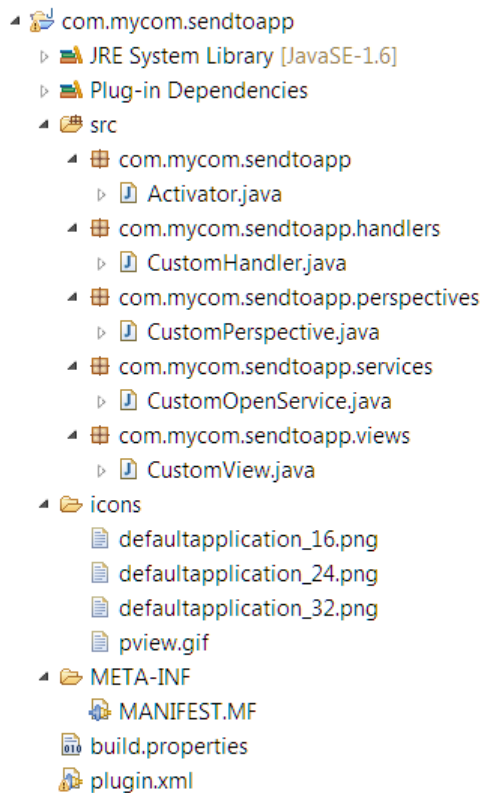
- The **com.mycom.sendtoapp.services** package and the **CustomOpenService** class in it.
- The **com.mycom.sendtoapp.views** package and the **CustomView** class in it.
- The **com.mycom.sendtoapp.handlers** package and the **CustomHandler** class in it.
- An **icons** directory.

Note:

To create a directory, right-click the project and choose **New→Folder**.

- b. Open the **com.teamcenter.rac.aifrcp_version.jar** in the **TC_ROOT\portal\plugins** directory, extract all the **defaultapplication_size.png** files and the **pview.gif** file, and copy the files to the **icons** directory you just created.

The plug-in structure looks like this:



- c. Replace the code in the **Activator.java** file with the following:

```

package com.mycom.sendtoapp;

import java.util.Dictionary;

import org.osgi.framework.BundleContext;

import com.mycom.sendtoapp.services.CustomOpenService;
import com.teamcenter.rac.kernel.AbstractRACPlugin;
import com.teamcenter.rac.services.IAspectService;
import com.teamcenter.rac.services.IAspectUIService;
import com.teamcenter.rac.services.IOpenService;
import com.teamcenter.rac.services.IServiceConstants;

/**

```



```

* The activator class controls the plug-in life cycle
*/
public class Activator extends AbstractRACPlugin {

    // The plug-in ID
    public static final String PLUGIN_ID = "com.mycom.sendtoapp";

    // The shared instance
    private static Activator plugin;

    /**
     * The constructor
     */
    public Activator() {
    }

    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#start(org.osgi.framework.
BundleContext)
     */
    @Override
    public void start(BundleContext context) throws Exception {
        super.start(context);
        plugin = this;
    }

    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#stop(org.osgi.framework.
BundleContext)
     */
    @Override
    public void stop(BundleContext context) throws Exception {
        plugin = null;
        super.stop(context);
    }

    /**
     * Returns the shared instance
     *
     * @return the shared instance
     */
    public static Activator getDefault() {
        return plugin;
    }

    @Override
    public IAspectService getLogicService() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public IAspectUIService getUIService() {
        // TODO Auto-generated method stub
        return null;
    }

    protected void setupServices(BundleContext context) {

```

```

        // TODO Auto-generated method stub
        Dictionary<String, String> properties = new java.util.Hashtable<String,
String>();
        // Register the open service
        properties.put( IServiceConstants.PERSPECTIVE,
            "com.mycom.sendtoapp.perspectives.CustomPerspective" );
        CustomOpenService customOpenService = new CustomOpenService();
        context.registerService( IOpenService.class.getName(),
            customOpenService, properties );
    }
}

```

- d. Replace the code in the **CustomPerspective.java** file with the following:

```

package com.mycom.sendtoapp.perspectives;
import org.eclipse.ui.IFolderLayout;
import org.eclipse.ui.IPageLayout;
import org.eclipse.ui.IPerspectiveFactory;
public class CustomPerspective implements IPerspectiveFactory {

    /** The perspective ID */
    public static final String ID =
        "com.mycom.sendtoapp.perspectives.CustomPerspective";
    @Override
    public void createInitialLayout(IPageLayout layout) {
        // TODO Auto-generated method stub
        layout.setEditorAreaVisible(false);
        String editorArea = layout.getEditorArea();
        IFolderLayout top = layout.createFolder("top", IPageLayout.TOP, -2f,
            editorArea);
        top.addView( "com.mycom.sendtoapp.views.CustomView" );
    }
}

```

- e. Replace the code in the **CustomOpenService.java** file with the following:

```

package com.mycom.sendtoapp.services;
import org.eclipse.jface.dialogs.MessageDialog;
import org.eclipse.swt.widgets.Display;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.PlatformUI;
import com.teamcenter.rac.aif.kernel.InterfaceAIFComponent;
import com.teamcenter.rac.services.IOpenService;
public class CustomOpenService implements IOpenService {
    @Override
    public boolean close() throws Exception {
        // TODO Auto-generated method stub
        return false;
    }
    @Override
    public boolean open(final InterfaceAIFComponent cmp) {
        Display.getDefault().asyncExec( new Runnable() {
            public void run() {
                IWorkbenchWindow window = PlatformUI.getWorkbench()
                    .getActiveWorkbenchWindow();
                MessageDialog
                    .openInformation(window.getShell(),

```

```

        "CustomOpenService",
        "You sent this component to the SendTo
Application: "
                                + cmp.toString());
    }
    });
    return false;
}
@Override
public boolean open(InterfaceAIFComponent[] cmps) {
    // TODO Auto-generated method stub
    return false;
}
}

```

- f. Replace the code in the **CustomView.java** file with the following:

```

package com.mycom.sendtoapp.views;
import org.eclipse.swt.SWT;
import org.eclipse.swt.graphics.Font;
import org.eclipse.swt.graphics.FontData;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Text;
import org.eclipse.ui.part.ViewPart;
public class CustomView extends ViewPart {

    public CustomView() {
        super();
    }
    @Override
    public void createPartControl(Composite parent) {
        // TODO Auto-generated method stub
        parent.setLayout( new FillLayout() );
        Text t = new Text( parent, SWT.BORDER );
        t.setBackground( parent.getDisplay().getSystemColor( SWT.COLOR_WHITE ) );
        t.setForeground( parent.getDisplay().getSystemColor( SWT.COLOR_BLUE ) );
        Font initialFont = t.getFont();
        FontData[] fontData = initialFont.getFontData();
        for (int i = 0; i < fontData.length; i++) {
            fontData[i].setHeight(18);
        }

        Font newFont = new Font(parent.getDisplay(), fontData);
        t.setFont( newFont );
        t.setText( " Welcome to the SendTo Application !! ");
    }
    @Override
    public void setFocus() {
        // TODO Auto-generated method stub
    }
}

```

- g. Replace the code in the **CustomHandler.java** file with the following:

```

package com.mycom.sendtoapp.handlers;

import org.eclipse.core.commands.AbstractHandler;

```

```

import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
import org.eclipse.ui.IWorkbench;
import org.eclipse.ui.IWorkbenchWindow;
import org.eclipse.ui.PlatformUI;
import org.eclipse.ui.WorkbenchException;

/**/
public class CustomHandler
    extends AbstractHandler
{
    @Override
    public Object execute( ExecutionEvent event )
        throws ExecutionException
    {
        IWorkbench workbench = PlatformUI.getWorkbench();
        IWorkbenchWindow window = workbench.getActiveWorkbenchWindow();
        try {
            workbench.showPerspective(
                "com.mycom.sendtoapp.perspectives.CustomPerspective", window, null );
        } catch (WorkbenchException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        // tell your code to open the components

        return null;
    }
}

```

Note:

Notice the commented text **// tell your code to open the components**. You must ensure that the component selected to send to the application can be opened in the application.

- h. To create an empty **plugin.xml** file, click the project's **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

- i. Click the project's **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
    <extension point="com.teamcenter.rac.aifrcp.application">
        <aif_app_item
            displayMode="Primary"
            groupName="Mycompany"
            icon="icons/defaultapplication_32.png"
            id="com.mycom.sendtoapp"
            name="SendTo Application"
            ordinality="200"

```

```

        perspective_id="com.mycom.sendtoapp.perspectives.CustomPerspective"
        session="com.teamcenter.rac.kernel.TCSession"
        tooltip="Send To Application"/>
    </extension>
    <extension point="org.eclipse.ui.perspectives">
        <perspective
            class="com.mycom.sendtoapp.perspectives.CustomPerspective"
            icon="icons/defaultapplication_16.png"
            id="com.mycom.sendtoapp.perspectives.CustomPerspective"
            name="SendTo Application"/>
    </extension>
    <extension point="org.eclipse.ui.perspectiveExtensions">
        <perspectiveExtension targetID="*">
            <perspectiveShortcut
                id="com.mycom.sendtoapp.perspectives.CustomPerspective" />
        </perspectiveExtension>
    </extension>
    <extension point="org.eclipse.ui.views">
        <view
            allowMultiple="false"
            class="com.mycom.sendtoapp.views.CustomView"
            icon="icons/pview.gif"
            id="com.mycom.sendtoapp.views.CustomView"
            name="Custom View"/>
    </extension>
    <extension point="org.eclipse.ui.commands">
        <command
            name="SendTo Application"
            id="com.mycom.sendtoapp.sendto">
        </command>
    </extension>
    <extension point="org.eclipse.ui.commandImages">
        <command
            icon="icons/defaultapplication_16.png"
            id="com.mycom.sendtoapp.sendto">
        </command>
    </extension>
    <extension point="org.eclipse.ui.handlers">
        <handler
            commandId="com.mycom.sendtoapp.sendto"
            class="com.mycom.sendtoapp.handlers.CustomHandler">
            <enabledWhen>
                <iterate ifEmpty="false">
                    <reference definitionId="com.mycom.sendtoapp.sendToActive"/>
                </iterate>
            </enabledWhen>
        </handler>
    </extension>
    <extension point="org.eclipse.ui.menus">
        <menuContribution
            locationURI="popup:com.teamcenter.rac.sendto?after=additions">
            <command
                commandId="com.mycom.sendtoapp.sendto">
                <visibleWhen>
                    <iterate ifEmpty="false">
                        <reference
                            definitionId="com.mycom.sendtoapp.sendToActive"/>
                    </iterate>
                </visibleWhen>
            </command>
        </menuContribution>
    </extension>

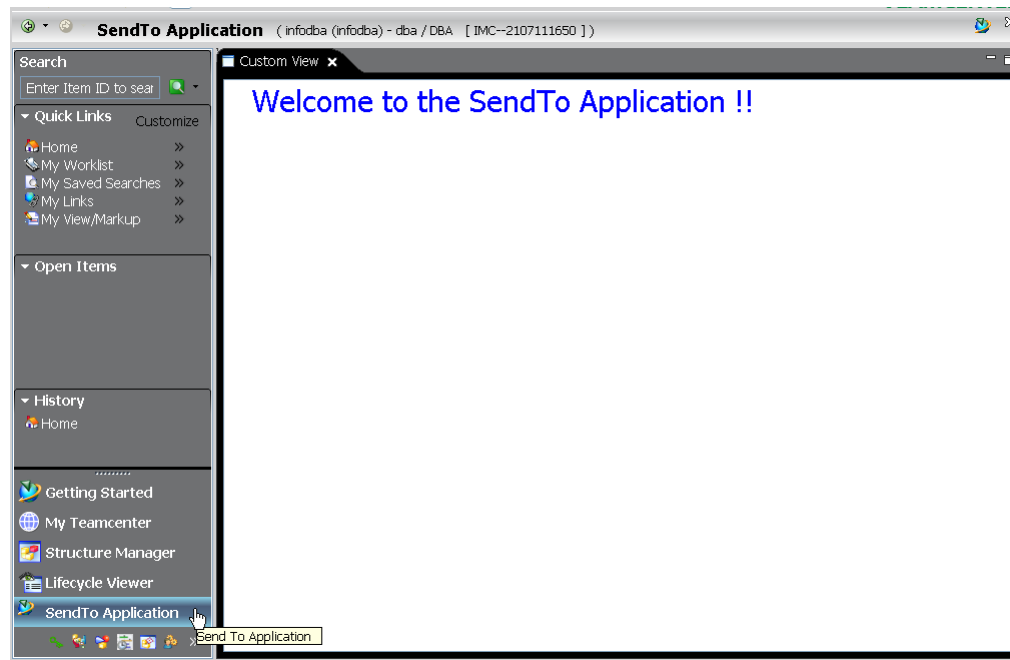
```

```

        </menuContribution>
    </extension>
    <extension point="org.eclipse.core.expressions.definitions">
        <definition id="com.mycom.sendtoapp.sendToActive">
            <and>
                <adapt type="com.teamcenter.rac.kernel.TCComponent">
                    <not>
                        <adapt type="com.teamcenter.rac.kernel.TCComponent">
                            <test
                                property=
                                    "com.teamcenter.rac.kernel.TCComponent.typeClass"
                                value="AllocationLine">
                            </test>
                        </adapt>
                    </not>
                </adapt>
            </and>
        </definition>
        <definition id="com.mycom.sendtoapp.inMainView">
            <or>
                <with variable="activePartId">
                    <equals value="com.mycom.sendtoapp.views.CustomView" />
                </with>
                <with variable="arc_property.ACTIVE_APPLICATION">
                    <equals value="com.mycom.sendtoapp" />
                </with>
            </or>
        </definition>
    </extension>
</plugin>

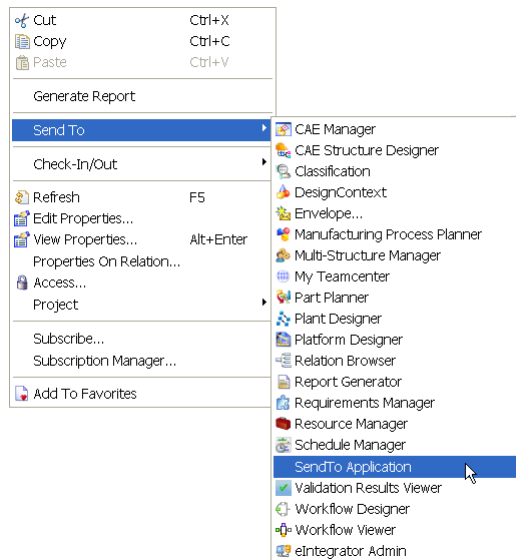
```

4. Choose **File**→**Save All**.
5. Verify the customization.
 - a. Choose **Run**→**Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
 - b. Verify that the new **SendTo Application** button is shown in the left-hand navigation pane in the rich client. Click the **SendTo Application** button.



New SendTo application in the navigation pane

Verify that when you right-click an object, the new application is displayed on the **Send To** menu.



New SendTo application added to the Send To menu

In addition to creating an application, this solution shows how to register a service in the **Activator.setupServices()** method. It also shows how to use a **CustomHandler** handler to take the perspective ID to open as an argument.

```
workbench.showPerspective( "com.mycom.sendtoapp.perspectives.CustomPerspective",
window, null );
```

6. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the `.properties` files, or you add new plug-ins or change plug-in content, you must run the `genregxml` script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the `portal\Registry` directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Hide a view

Using command suppression, you can hide views based upon user context. However, in order to hide a view from all users at all times, you must modify the `plugin.xml` file of an Eclipse rich client plug-in project. The visibility is controlled using the `org.eclipse.ui.activities` extension point. By binding an empty activity to the view, the rich client hides it completely from the UI regardless of user context.

In the following example, an empty activity, **Hide Custom List View**, is created, and then attached to a **Custom List View** view.

```
<extension point="org.eclipse.ui.activities">
  <activity
    id="com.teamcenter.rac.customization.examples.hidecustomlistview"
```



```

        name="Hide Custom List View">
    </activity>
    <activityPatternBinding
        activityId="com.teamcenter.rac.customization.examples.hidecustomlistview"
        isEqualityPattern="true"
        pattern="com.teamcenter.rac.customization.examples/
            com.teamcenter.rac.customization.examples.view.CustomListView">
    </activityPatternBinding>
</extension>

```

Note:

The actual **name** and **id** used for the empty activity are not relevant, however it is recommended to choose something descriptive.

Create a new type wizard

New type wizard overview

If you want more control over which business object types your users can create, and what the dialog boxes display, then you can create a custom New Business Object Wizard .

Using an Eclipse plug-in project, you can then create a menu contribution for your new wizard in the **File→New** menu.

You can create several new type wizards, each one allowing only a few or even just a single type of business object to be created. For complete control, limit user access to the **File→New→Object** menu. You can use Command Suppression to limit by group or role, or a plug-in project to remove it completely.

An Eclipse plug-in project is required for all the following steps.

1. Add a **menuContribution** for your wizard.
2. Utilize the **extWizard** extension point to create your wizard.
3. Utilize the **boTypesLoader** extension point to filter the types.
4. Utilize the **extWizardRef** extension point to add pages.
5. Utilize the **operation** extension point to create the object.

Add a menuContribution for your wizard

You must create some way for the user to start the wizard. Creating a **menuContribution** is the most common way to do this. The new UI selection will link to a new **command**, which will be associated with a new **handler** (Java class).

In this example, you will create a **Document** menu selection on the **File→New** menu, then create a corresponding **command** and **handler**.

Add the following menu contribution to the **menus** extension point in your project's **plugin.xml** file.

```
<extension point="org.eclipse.ui.menus">
...
  <menuContribution locationURI="menu:file.new?after=group1">
    <command
      commandId="com.mycom.newwizard.genericcustomitem"
      mnemonic="D"
      id="com.mycom.newwizard.genericcustomitem">
    </command>
  </menuContribution>
...
</extension>
```

Add the following to the **commands** extension point in your project's **plugin.xml** file.

```
<extension point="org.eclipse.ui.commands">
...
  <command
    id="com.mycom.newwizard.genericcustomitem"
    name="Document">
  </command>
...
</extension>
```

Add the following to the **handlers** extension point in your project's **plugin.xml** file.

```
<extension point="org.eclipse.ui.handlers">
...
  <handler
    class="com.mycom.newwizard.handlers.CustomWizardHandler"
    commandId="com.mycom.newwizard.genericcustomitem">
  </handler>
...
</extension>
```

Create the **Java** class for your **handler**.

```
package com.mycom.newwizard.handlers;

import org.eclipse.core.commands.ExecutionEvent;
```

```
import org.eclipse.core.commands.ExecutionException;

import com.teamcenter.rac.ui.commands.handlers.NewBOHandler;

public class CustomWizardHandler extends NewBOHandler {

    public Object execute( ExecutionEvent event )
        throws ExecutionException
    {
        wizardId = "com.mycom.newwizard.CustomObjectWizard";
        return super.execute( event );
    }

    public String getWizardTitle()
    {
        return "Document";
    }

}
```

Utilize the extWizard extension point to create your wizard

To create a custom type wizard you must use the **extWizard** extension point.

In this example, you create a new **extWizard** extension, with a **CustomObjectWizard** identification, which uses the existing **NewBOWizard** class.

Add the following extension to your project's **plugin.xml** file.

```
<extension
    id="extWizard"
    point="com.teamcenter.rac.util.extWizard">
    <extWizard
        class="com.teamcenter.rac.ui.commands.create.bo.NewBOWizard"
        id="com.mycom.newwizard.CustomObjectWizard">
    </extWizard>
</extension>
```

Utilize the boTypesLoader extension point to filter the types

You must provide a list of available business object types to the wizard. The user will be presented this list on the **BOTypePage**. The easiest way to do this is using the **boTypesLoader** extension point.

In this example, you use the **Document** business object type for your wizard.

Add the following to the **boTypesLoader** extension point in your project's **plugin.xml** file.

```

<extension point="com.teamcenter.rac.ui.commands.boTypesLoader">
...
    <BOTypesLoader base_classes="Document"
        class="com.teamcenter.rac.ui.commands.create.bo.NewBOTypesLoader"
        enableMRU="true"
        id="com.teamcenter.rac.ui.commands.create.bo.DocumentTypeLoader"
        parentId="com.mycom.newwizard.CustomObjectWizard">
    </BOTypesLoader>
...
</extension>

```

Note:

The **boTypesLoader** will find not only the business object type specified as the **base_class**, but also any instantiable children of that type.

In this example, **Document** happens to be the parent of several other instantiable business object types, including Email, Marketing Brief, and others.

To remove the children from the list, you must also use the **excludeTypes** tag to specify which child types you do not want included. Multiple type names can be entered, separated by a comma.

In this example, the Email and Marketing Brief types are removed from the list.

```

    <BOTypesLoader base_classes="Document"
        class="com.teamcenter.rac.ui.commands.create.bo.NewBOTypesLoader"
        enableMRU="true"
        id="com.teamcenter.rac.ui.commands.create.bo.DocumentTypeLoader"
        parentId="com.mycom.newwizard.CustomObjectWizard">
    <excludeTypes base_class="Document"
        exclude_types="Email,CPMarketingBrief"
        typesLoaderId="com.teamcenter.rac.ui.commands.create.bo.DocumentTypeLoader"
    </excludeTypes>
    </BOTypesLoader>

```

Utilize the extWizardRef extension point to add pages

Once you have defined the new wizard, you must add pages to it. Use the **extWizardRef** extension point to add pages. The two main pages required are **BOTypePage** and **CreatePropsPage**.

The order of the pages is controlled by the **ordinal** attribute; the pages are presented to the user in numerical order. The **BOTypePage** page must be presented before (have a lower **ordinal** number than) the **CreatePropsPage** page, since the required and available properties are determined by the business object type selected.

```

<extension
    id="itemWizardRef"
    point="com.teamcenter.rac.util.extWizardRef">
    <extWizardRef
        ordinal="100"
        pageId="com.teamcenter.rac.ui.commands.create.bo.BOTypePage"
        wizardId="com.mycom.newwizard.CustomObjectWizard">
    </extWizardRef>

```

```

<extWizardRef
    ordinal="200"

    pageId="com.teamcenter.rac.ui.commands.create.bo.CreatePropsPage"
    wizardId="com.mycom.newwizard.CustomObjectWizard">
</extWizardRef>
</extension>

```

The **CreatePropsPage** can be controled using an **XMLRenderingStylesheet**.

Following is an XML rendering stylesheet that could be used to display properties from both the Document and the DocumentRevision.

- **Registered Type: Document**
- **Stylesheet Type: Create**

DocumentCreate.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<rendering>
    <page title="Document Information"
        titleKey="tc_xrt_DocumentInformation">
        <section titleKey="tc_xrt_properties">
            <property name="object_name" />
            <property name="object_desc" />
            <separator/>
            <property name="revision:DocumentSubject" />
        </section>
        <section title="Additional Document Information"
            titleKey="tc_xrt_properties"
            initialState="collapsed">
            <property name="revision:DocumentTitle" />
            <property name="revision:DocumentAuthor" />
        </section>
    </page>
</rendering>

```

Utilize the operation extension point to create the object

Once all the properties are gathered, the business object can be created. The **NewBOOperation** command creates the new object and poasts it into the destination.

In this example, use the **operation** extension to call **NewBOOperation**.

Add the following to the **operation** extension point in your project's **plugin.xml** file.

```

<extension point="com.teamcenter.rac.ui.commands.operation">
...
    <operation
class="com.teamcenter.rac.ui.commands.create.bo.NewBOOperation"
        id="com.teamcenter.rac.ui.commands.NewItemOperation"
        wizardId="com.mycom.newwizard.CustomObjectWizard">
    </operation>
...
</extension>

```

Localize your customizations

You can localize (present in different languages) the rich client by either modifying the **plugin.xml** file or the *application_locale.properties* file. Modifying the properties file was used prior to Teamcenter 2007 and is still supported. However, if you are creating a localization of a new custom project, modify the Eclipse **plug-in.xml** file.

1. Create the project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.l10n** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Create the project files.
 - a. Add the following line to the end of the **MANIFEST.MF** file:

```
Bundle-Localization: plugin
```

The **MANIFEST.MF** file should appear as follows:

```

Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: L10n
Bundle-SymbolicName: com.mycom.l10n; singleton:=true
Bundle-Version: 1.0.0.qualifier
Bundle-Activator: com.mycom.l10n.Activator
Bundle-Vendor: MYCOM
Require-Bundle: org.eclipse.ui,
    org.eclipse.core.runtime
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Bundle-ActivationPolicy: lazy
Bundle-Localization: plugin

```

- b. Create a **plugin.properties** file in the **com.mycom.l10n** project by selecting the project and choosing **File→New→File**.

The empty **plugin.properties** file appears directly under the **com.mycom.l10n** project at the same level as the **plugin.xml** file.

- c. Add the following name/value pairs to the **plugin.properties** file:

```

myMenuItem.label=My Hello world
myTooltip.tip=My Say hello world
myCommand.name=My Command

```

- d. Edit the **plugin.xml** file replacing hard coded values with **%keyname** for the name/value pairs you typed in the **plugin.properties** file. For example:

```
label="%myMenuItem.label"
```

The **plugin.xml** file should appear as follows:

```

<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
    <extension point="org.eclipse.ui.commands">
        <category
            name="Sample Category"
            id="com.mycom.l10n.commands.category">
        </category>
        <command
            name="%myCommand.name"
            categoryId="com.mycom.l10n.commands.category"
            id="com.mycom.l10n.commands.sampleCommand">
        </command>
    </extension>
    <extension point="org.eclipse.ui.handlers">
        <handler
            commandId="com.mycom.l10n.commands.sampleCommand"
            class="com.mycom.l10n.handlers.SampleHandler">
        </handler>
    </extension>
    <extension point="org.eclipse.ui.bindings">
        <key
            commandId="com.mycom.l10n.commands.sampleCommand"

```

```

        contextId="org.eclipse.ui.contexts.window"
        sequence="M1+6"
        schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
</extension>
<extension point="org.eclipse.ui.menus">
    <menuContribution
        locationURI="menu:org.eclipse.ui.main.menu?after=additions">
        <menu
            label="%myMenuItem.label"
            mnemonic="M"
            id="com.mycom.110n.menus.sampleMenu">
            <command
                commandId="com.mycom.110n.commands.sampleCommand"
                mnemonic="S"
                icon="icons/sample.gif"
                id="com.mycom.110n.menus.sampleCommand">
            </command>
        </menu>
    </menuContribution>
    <menuContribution
        locationURI="toolbar:org.eclipse.ui.main.toolbar?after=additions">
        <toolbar
            id="com.mycom.110n.toolbars.sampleToolbar">
            <command
                commandId="com.mycom.110n.commands.sampleCommand"
                icon="icons/sample.gif"
                tooltip="%myTooltip.tip"
                id="com.mycom.110n.toolbars.sampleCommand">
            </command>
        </toolbar>
    </menuContribution>
</extension>
</plugin>

```

- e. Add a Spanish localization by creating a **plugin_es.properties** file at the same level as the **plugin.properties** file. The keys are identical in the **plugin_es.properties** file but have Spanish values.

```

myMenuItem.label=Mi Hola mundo
myTooltip.tip=Mi para Decir hola mundo
myCommand.name=Mi Orden

```

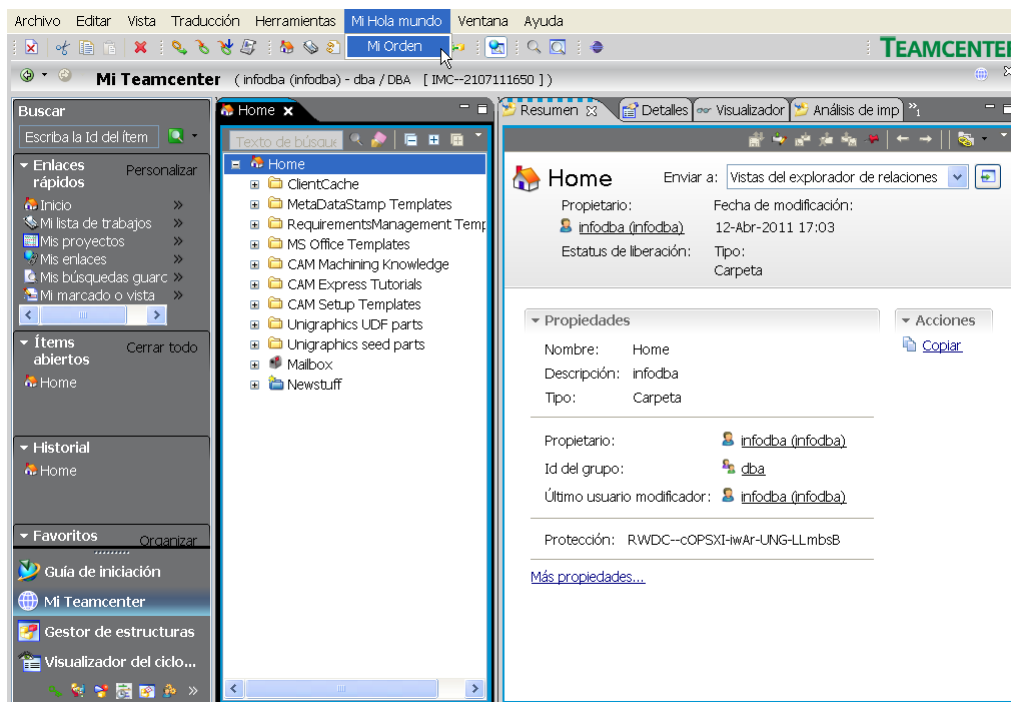
3. Verify the customization thus far.

- a. Create a new configuration for the Spanish locale by choosing **Run→Run Configurations** (or **Run→Debug Configurations**) and specifying **-nl es** instead of **-nl \${target.nl}** on the **Arguments** tab. For example:

```
-os ${target.os} -ws ${target.ws} -arch ${target.arch} -nl es
```

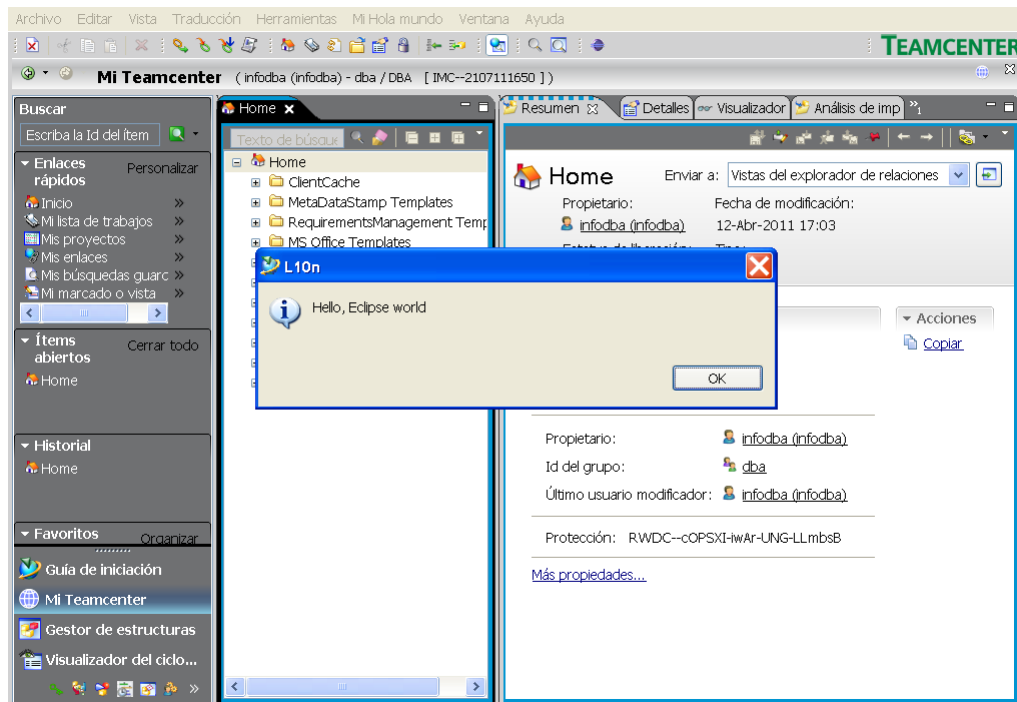
- b. To verify the configuration so far, choose **Run→Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.

- c. Choose **Mi Hola Mundo**→**Mi Orden** to run the custom command.



Choosing the custom menu command in the Spanish user interface

However, the message displayed by the custom command is not localized (translated).



Untranslated custom message box

The message should also be localized.

4. Localize the message.
 - a. In Eclipse, choose **Source**→**Externalize Strings**.

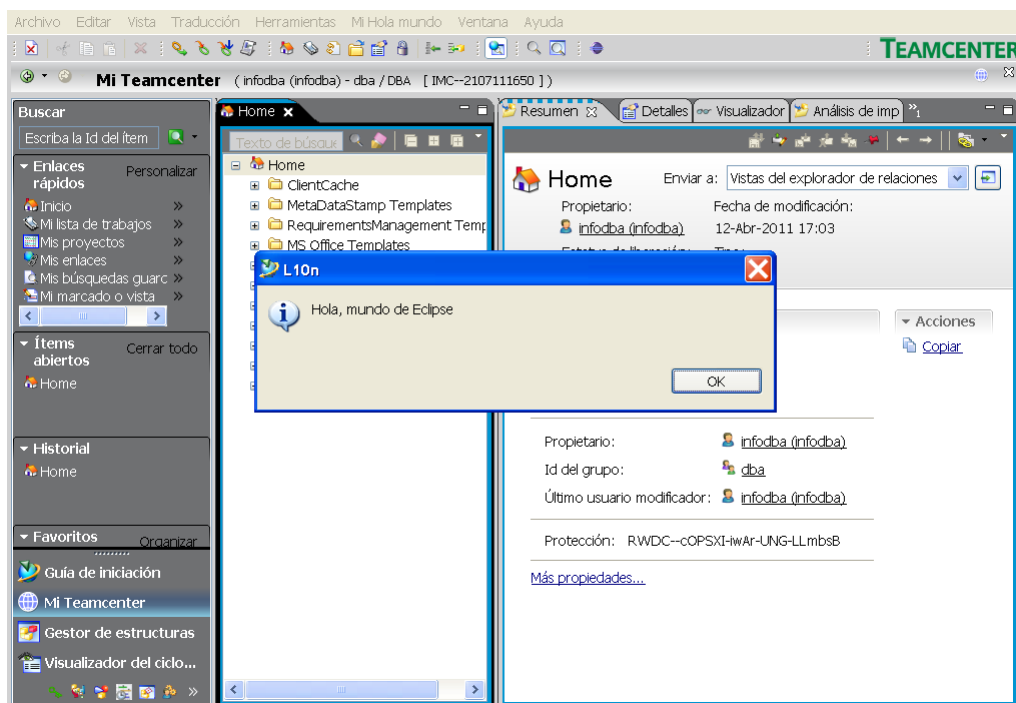
This creates the **Messages.java** and **messages.properties** files in the **com.mycom.l10n.handlers** package.

- b. Create a **messages_es.properties** file at the same level as the **messages.properties** file, and add the following content to the new file:

```
SampleHandler_0=L10n
SampleHandler_1=Hola, mundo de Eclipse
```

5. Verify the customization.
 - a. Choose **Run**→**Run Configurations** and choose the Spanish locale to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
 - b. Choose **Mi Hola Mundo**→**Mi Orden** to run the custom command.

The localized message is displayed.



Translated custom message box

For more information about localizing Eclipse plug-ins, see the Eclipse documentation. Also, you can unzip any of the Teamcenter plug-ins to see how the rich client uses localization. The **plugin.properties** and **plugin.xml** files in the **aifrcp** plug-in are good examples.

6. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the **TC_ROOT\portal** directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.

- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client.

When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

Customize the rich client properties files

You can customize the appearance of applications in the rich client by creating user properties files that override the default properties files. The default properties files, with a **.properties** extension, are located in the various **com.teamcenter.rac.component.jar** files in the **TC_ROOT\portal\plugins** directory. You override the default file by creating a **_user.properties** file and wrapping it in an Eclipse plug-in. For example, if you want to override a property in Manufacturing Process Planner, you must override the **mpp.properties** file located in the **com.teamcenter.rac.cme.legacy.jar** file with the **mpp_user.properties** file.

1. Create the Java project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **Project name** box, type the project name, which should be in the form of **com.mycom.project-name**. For example, type **com.mycom.propertiesfile**. Click **Next**.
 - d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Modify the properties file.
 - a. Navigate to the **TC_ROOT\portal\plugins** directory, and open the JAR file that contains the properties file you want to override. The file name has the form **property.properties**. For example, to override the **mpp.properties** file, open the **com.teamcenter.rac.cme.legacy.jar** file.

- b. Extract the properties file you want to override from the JAR file. For example, extract the **mpp.properties** file.

Note:

A JAR file uses ZIP compression and can be opened using standard unjar or unzip tools.

- c. In a text editor, open the properties file to determine the syntax of the property you want to modify.
 - d. Create a new *property_user.properties* in a text editor. For example, if you open the **mpp.properties** file, create the **mpp_user.properties** file.
 - e. Type the entries you want to use to override the defaults.
 - f. Save and exit the file.
3. Update the project tabs.
 - a. In Eclipse, click your project tab and click its **Dependencies** tab.
 - b. Under **Required Plug-ins**, click the **Add** button.
 - c. Select the following plug-ins from the list by holding down the Ctrl key while you click them:
 - **com.teamcenter.rac.aifrcp**
 - **com.teamcenter.rac.common**
 - **com.teamcenter.rac.external**
 - **com.teamcenter.rac.kernel**
 - **com.teamcenter.rac.neva**
 - **com.teamcenter.rac.tcapps**
 - **com.teamcenter.rac.util**
 - d. Click **OK**.
 - e. Click your project's **Runtime** tab.
 - f. Under **Exported Packages**, click the **Add** button.
 - g. Select your project and click **OK**.

- h. Click your project's **Extensions** tab.
- i. Click the **Add** button.
- j. Select the **com.teamcenter.rac.util.tc_properties** extension point.
- k. Click **Finish**.
- l. Click the **plugin.xml** tab. Type text in the file so it looks like the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension
    point="com.teamcenter.rac.util.tc_properties">
  </extension>
</plugin>
```

If the text is different, edit it in the tab to resemble the example.

- m. Save the project by choosing **File→Save All**.
4. Create a package.
- a. In the **Package Explorer** view, right-click your project and choose **New→Package**.
 - b. In the **Name** box, type the path name where the properties file was originally. For example, if you originally extracted the **mpp.properties** file, the package name should be **com.teamcenter.rac.cme.mpp**.
 - c. Click **Finish**.
 - d. From Windows Explorer, drag your modified *property_user.properties* file and drop it on the package you created in **Package Explorer**.
 - e. In **Package Explorer**, select **plugin.xml**.
 - f. Click the **MANIFEST.MF** tab and type your new package at the end of the **Export-Package** line. For example, if your project name is **com.mycom.propertiesfile** and the new package is called **com.teamcenter.rac.cme.mpp**, the line should read:

```
Export-Package: com.mycom.propertiesfile, com.teamcenter.rac.cme.mpp
```

The **MANIFEST.MF** file should look like the following:

```
Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: Propertiesfile
Bundle-SymbolicName: com.mycom.propertiesfile;singleton:=true
```

```

Bundle-Version: 1.0.0.qualifier
Bundle-Activator: com.mycom.propertiesfile.Activator
Bundle-Vendor: MYCOM
Require-Bundle: org.eclipse.ui,
    org.eclipse.core.runtime,
    com.teamcenter.rac.aifrcp;bundle-version="8000.2.0",
    com.teamcenter.rac.common;bundle-version="8000.2.0",
    com.teamcenter.rac.external;bundle-version="8000.2.0",
    com.teamcenter.rac.kernel;bundle-version="8000.2.0",
    com.teamcenter.rac.neva;bundle-version="1.0.0",
    com.teamcenter.rac.tcapps;bundle-version="8000.2.0",
    com.teamcenter.rac.util;bundle-version="8000.2.0"
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Bundle-ActivationPolicy: lazy
Export-Package: com.mycom.propertiesfile, com.teamcenter.rac.cme.mpp

```

- g. Click your project's **Runtime** tab. If one or more of the items listed in the **Export-Package** line in the previous step are missing, add the missing ones by clicking the **Add** button, selecting the missing packages, and clicking **OK**.

5. Save and export your changes.

6. Debug in Eclipse.

- a. In Eclipse, choose **Run**→**Debug Configurations**.
- b. In the **Debug** dialog box, under **Java Application** on the left-hand side, select the configuration you want to debug.
- c. At the bottom of the dialog box, click the **Debug** button.
This launches the application in debug mode. To change the perspective to the debug perspective, choose **Window**→**Open Perspective**→**Debug**.

You can then debug your application.

7. Run the **genregxml** script.

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded when the rich client starts. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory. The **RegistryLoader** file is added during rich client startup.

- a. Delete the old registry loader file:

```
TC_ROOT\portal\registry\RegistryLoader.xml.gz
```

- b. Run the following script:

```
TC_ROOT\portal\registry\genregxml
```

When the script is finished, a new **RegistryLoader.xml.gz** file is created.

8. Verify the customization.

Infrequently used rich client customizations

Add a table viewer

You can create a table in its own dialog window and add a menu command to launch it.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.tableviewer** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Dependencies** tab, click the **Add** button, and select the following:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
 - b. Click the **Runtime** tab, click the **Add** button, and select the following:

com.mycom.tableviewer
com.mycom.tableviewer.handlers
3. Create the project files.

- a. Edit the **com.mycom.tableviewer** plug-in's **plugin.xml** file. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.mycom.tableviewer.commands.category">
    </category>
    <command
      name="Sample Command"
      categoryId="com.mycom.tableviewer.commands.category"
      id="com.mycom.tableviewer.commands.sampleCommand">
    </command>
  </extension>
  <extension
    point="org.eclipse.ui.handlers">
    <handler
      commandId="com.mycom.tableviewer.commands.sampleCommand"
      class="com.mycom.tableviewer.handlers.TableViewerHandler">
    </handler>
  </extension>
  <extension
    point="org.eclipse.ui.bindings">
    <key
      commandId="com.mycom.tableviewer.commands.sampleCommand"
      contextId="org.eclipse.ui.contexts.window"
      sequence="M1+6"
      schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
  </extension>
  <extension
    point="org.eclipse.ui.menus">
    <menuContribution
      locationURI="menu:org.eclipse.ui.main.menu?after=additions">
    <menu
      label="Viewer"
      mnemonic="V"
      id="com.mycom.tableviewer.menus.sampleMenu">
    <command
      commandId="com.mycom.tableviewer.commands.sampleCommand"
      id="com.mycom.tableviewer.menus.sampleCommand"
      label="TableViewer"
      mnemonic="T">
    <visibleWhen>
      <reference
        definitionId="com.teamcenter.rac.ui.inMainPerspective">
      </reference>
    </visibleWhen>
    </command>
    </menu>
  </menuContribution>
  <menuContribution
    locationURI="toolbar:org.eclipse.ui.main.toolbar?after=additions">
    <toolbar
      id="com.mycom.tableviewer.toolbars.sampleToolbar">
```

```

        <command
            commandId="com.mycom.tableviewer.commands.sampleCommand"
            icon="icons/sample.gif"
            id="com.mycom.tableviewer.toolbars.sampleCommand"
            label="TableViewer"
            tooltip="Explore Tableviewer">
        </command>
    </toolbar>
</menuContribution>
</extension>
</plugin>

```

- b. Open the **com.mycom.tableviewer.handlers** package, select the **SampleHandler.java** file, choose **File→Rename**, and rename the file to **TableViewerHandler.java**.

Replace the contents of the **TableViewerHandler.java** file with the following:

```

package com.mycom.tableviewer.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
/**
 * Our TableViewerHandler extends AbstractHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class TableViewerHandler extends AbstractHandler {
    /**
     * The constructor.
     */
    public TableViewerHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {
        return new TableViewerExample();
    }
}

```

- c. Right-click the **com.mycom.tableviewer.handlers** package, choose **New→Class**, and add the following Java files:

Row.java
TableContentProvider.java
TableLabelProvider.java
TableViewerExample.java

- d. Replace the contents of the **Row.java** file with the following:

```

package com.mycom.tableviewer.handlers;
public class Row {
    private String key;
    private String value;
    public Row(String key, String value) {

```

```

        setKey(key);
        setValue(value);
    }
    public void setKey(String key) {
        this.key = key;
    }
    public String getKey() {
        return key;
    }
    public void setValue(String value) {
        this.value = value;
    }
    public String getValue() {
        return value;
    }
}

```

- e. Replace the contents of the **TableContentProvider.java** file with the following:

```

package com.mycom.tableviewer.handlers;
import java.util.ArrayList;
import java.util.List;
import org.eclipse.jface.viewers.IStructuredContentProvider;
import org.eclipse.jface.viewers.Viewer;
public class TableContentProvider implements IStructuredContentProvider {
    public Object[] getElements(Object parent) {
        List results = new ArrayList();
        if (parent instanceof ArrayList) {
            results = (ArrayList) parent;
        }
        return results.toArray();
    }
    public void dispose() {
    }
    public void inputChanged(Viewer viewer, Object oldInput, Object newInput) {
    }
}

```

- f. Replace the contents of the **TableLabelProvider.java** file with the following:

```

package com.mycom.tableviewer.handlers;
import org.eclipse.jface.viewers.ITableLabelProvider;
import org.eclipse.jface.viewers.LabelProvider;
import org.eclipse.swt.graphics.Image;
public class TableLabelProvider extends LabelProvider implements
    ITableLabelProvider {
    public Image getColumnImage(Object element, int columnIndex) {
        return null;
    }
    public String getColumnText(Object element, int columnIndex) {
        Row row = (Row) element;
        switch (columnIndex) {
            case 0:
                return row.getKey();
            case 1:
                return row.getValue();
        }
        return null;
    }
}

```

```

    }
}

```

- g. Replace the contents of the **TableViewerExample.java** file with the following:

```

package com.mycom.tableviewer.handlers;
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
import org.eclipse.jface.viewers.CellEditor;
import org.eclipse.jface.viewers.ColumnWeightData;
import org.eclipse.jface.viewers.ICellModifier;
import org.eclipse.jface.viewers.ISelection;
import org.eclipse.jface.viewers.IStructuredSelection;
import org.eclipse.jface.viewers.TableLayout;
import org.eclipse.jface.viewers.TableViewer;
import org.eclipse.jface.viewers.TextCellEditor;
import org.eclipse.swt.SWT;
import org.eclipse.swt.events.SelectionAdapter;
import org.eclipse.swt.events.SelectionEvent;
import org.eclipse.swt.layout.FillLayout;
import org.eclipse.swt.layout.GridData;
import org.eclipse.swt.layout.GridLayout;
import org.eclipse.swt.widgets.Button;
import org.eclipse.swt.widgets.Composite;
import org.eclipse.swt.widgets.Control;
import org.eclipse.swt.widgets.Display;
import org.eclipse.swt.widgets.Shell;
import org.eclipse.swt.widgets.Table;
import org.eclipse.swt.widgets.TableColumn;
import org.eclipse.swt.widgets.TableItem;
import com.teamcenter.rac.aif.AbstractAIFDialog;
import com.teamcenter.rac.util.Registry;
/**
 * This is an example program showing how to use a TableViewer
 */
public class TableViewerExample {
    private Table table;
    private TableViewer tableViewer;

    /** class registry */
    private final static Registry reg =
Registry.getRegistry( TableViewerExample.class );
    private final static String[] COLUMN_HEADINGS =
{ reg.getString( "Object"),reg.getString( "Type" ) };
    public TableViewerExample() {
        createTableViewerExample();
    }
    private void createTableViewerExample() {
        Shell shell = new Shell(SWT.SHELL_TRIM);
        shell.setImage( Registry.getRegistry(
AbstractAIFDialog.class ).getImage(
"aifDesktop.ICON" ) );
        shell.setText(reg.getString("details"));
        shell.setLayout(new FillLayout());
        createContents(shell);
        shell.setSize(400, 400);
        shell.open();
        while (!shell.isDisposed()) {

```

```

        if (!Display.getDefault().readAndDispatch())
            Display.getDefault().sleep();
    }
}

protected Control createContents(Composite parent) {
    final Composite composite = new Composite(parent, SWT.NONE);
    GridLayout layout = new GridLayout(1, false);
    layout.verticalSpacing = 10;
    composite.setLayout(layout);
    GridData data = new GridData(GridData.FILL_BOTH);
    data.grabExcessHorizontalSpace = true;
    composite.setLayoutData(data);
    table = new Table(composite, SWT.BORDER | SWT.V_SCROLL | SWT.MULTI
        | SWT.FULL_SELECTION);
    table.setLinesVisible(true);
    table.setHeaderVisible(true);
    data = new GridData(SWT.FILL, SWT.FILL, true, false);
    data.heightHint = 300;
    table.setLayoutData(data);
    TableLayout tableLayout = new TableLayout();
    table.setLayout(tableLayout);
    tableLayout.addColumnData(new ColumnWeightData(10, 50, true));
    TableColumn column = new TableColumn(table, SWT.NONE);
    column.setText(COLUMN_HEADINGS[0]);
    column.setAlignment(SWT.LEFT);
    tableLayout.addColumnData(new ColumnWeightData(15, 200, true));
    column = new TableColumn(table, SWT.NONE);
    column.setText(COLUMN_HEADINGS[1]);
    column.setAlignment(SWT.LEFT);
    tableViewer = new TableViewer(table);
    tableViewer.setColumnProperties(COLUMN_HEADINGS);
    tableViewer.setContentProvider(new TableContentProvider());
    tableViewer.setLabelProvider(new TableLabelProvider());
    CellEditor[] editors = new CellEditor[2];
    editors[0] = new TextCellEditor(table);
    editors[1] = new TextCellEditor(table);
    tableViewer.setCellEditors(editors);
    tableViewer.setCellModifier(new TableCellModifier());
    Composite buttonComposite = new Composite(composite, SWT.NONE);
    FillLayout fillLayout = new FillLayout(SWT.HORIZONTAL);
    fillLayout.spacing = 10;
    buttonComposite.setLayout(fillLayout);
    Button addButton = new Button(buttonComposite, SWT.PUSH);
    addButton.setText(reg.getString("Add"));
    addButton.addSelectionListener(new SelectionAdapter() {
        public void widgetSelected(SelectionEvent e) {
            Row row = new Row("", "");
            tableViewer.add(row);
            table.setTopIndex(table.getItemCount());
            table.select(table.getItemCount() - 1);
            tableViewer.editElement(row, 0);
        }
    });
    Button deleteButton = new Button(buttonComposite, SWT.PUSH);
    deleteButton.setText(reg.getString("Delete"));
    deleteButton.addSelectionListener(new SelectionAdapter() {
        public void widgetSelected(SelectionEvent e) {
            ISelection selection = tableViewer.getSelection();
            if (selection instanceof IStructuredSelection) {
                Iterator iterator = ((IStructuredSelection) selection)

```

```

        .iterator();
        while (iterator.hasNext()) {
            Object obj = iterator.next();
            tableViewer.remove(obj);
        }
    }
}

});

Button closeButton = new Button(buttonComposite, SWT.PUSH);
closeButton.setText(reg.getString("Close"));
closeButton.addSelectionListener(new SelectionAdapter() {
    public void widgetSelected(SelectionEvent e) {
        composite.getParent().dispose();
    }
});
return composite;
}

class TableCellModifier implements ICellModifier {
    public boolean canModify(Object element, String property) {
        return true;
    }
    public Object getValue(Object element, String property) {
        Object result = null;
        Row row = (Row) element;
        List<String> list = Arrays.asList(COLUMN_HEADINGS);
        int columnIndex = list.indexOf(property);
        switch (columnIndex) {
            case 0:
                result = row.getKey();
                break;
            case 1:
                result = row.getValue();
                break;
        }
        return result;
    }
    public void modify(Object element, String property, Object value) {
        List<String> list = Arrays.asList(COLUMN_HEADINGS);
        int columnIndex = list.indexOf(property);
        TableItem tableItem = (TableItem) element;
        Row row = (Row) tableItem.getData();
        switch (columnIndex) {
            case 0:
                String key = (String) value;
                if (key.length() > 0) {
                    row.setKey(key);
                }
                break;
            case 1:
                String v = (String) value;
                if (v.length() > 0) {
                    row.setValue(v);
                }
                break;
        }
        tableViewer.update(row, null);
    }
}

```

```
}
}
```

- h. Right-click the **com.mycom.tableviewer.handlers** package, choose **New→File**, and add a **handlers.properties** file.
- i. Replace the contents of the **handlers.properties** file with the following:

```
Object=Object
Type=Type
details=Details
Add=Add
Delete=Delete
Close=Close
```

- j. Choose **File→Save All**.

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

5. Verify the customization.

- a. Launch My Teamcenter and choose **Viewer**→**TableViewer** on the menu bar, or click the **Explore TableViewer** button on the toolbar.

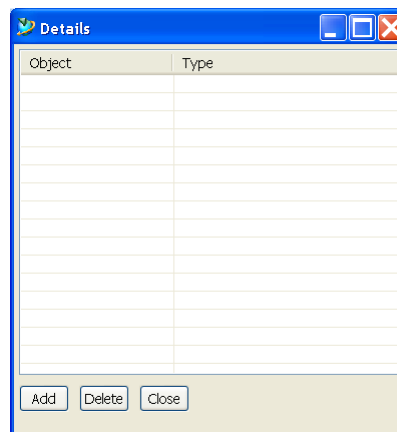


TableViewer menu command



TableViewer button

- b. In the **Details** dialog box, click the **Add** button to add a line to the table, or click the **Delete** button to remove a line.



TableViewer dialog box

Add a tree viewer

You can create a tree in its own dialog window and add a menu command to launch it. This example displays a file explorer tree that reads the contents of the hard disk.

By default, this example reads the **C:** drive on Windows systems. To read the **/user** drive on UNIX systems, change code in the **Explorer.java** file.

1. Create the project.

- a. In Eclipse, choose **File**→**New**→**Project**.
- b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.

- c. In the **New Plug-in Project** dialog box **Plug-in Project** pane, type **com.mycom.treeviewer** in the **Project name** box. Click **Next**.
 - d. In the **New Plug-in Project** dialog box **Content** pane, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Ensure the **Create a plug-in using one of these templates** check box is selected and select **Hello, World Command** in the list. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Dependencies** tab, click the **Add** button, and select the following:


```
com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.kernel
com.teamcenter.rac.util
```
 - b. Click the **Runtime** tab, click the **Add** button, and select the following:


```
com.mycom.treeviewer
com.mycom.treeviewer.handlers
```
 3. Create the project files.
 - a. Edit the **com.mycom.treeviewer** plug-in's **plugin.xml** file. Replace the entire contents of the **plugin.xml** file with the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    point="org.eclipse.ui.commands">
    <category
      name="Sample Category"
      id="com.mycom.treeviewer.commands.category">
    </category>
    <command
      name="Sample Command"
      categoryId="com.mycom.treeviewer.commands.category"
      id="com.mycom.treeviewer.commands.sampleCommand">
    </command>
  </extension>
</extension>
```

```

        point="org.eclipse.ui.handlers">
    <handler
        commandId="com.mycom.treeviewer.commands.sampleCommand"
        class="com.mycom.treeviewer.handlers.TreeViewerHandler">
    </handler>
</extension>
<extension
    point="org.eclipse.ui.bindings">
    <key
        commandId="com.mycom.treeviewer.commands.sampleCommand"
        contextId="org.eclipse.ui.contexts.window"
        sequence="M1+6"
        schemeId="org.eclipse.ui.defaultAcceleratorConfiguration">
    </key>
</extension>
<extension
    point="org.eclipse.ui.menus">
    <menuContribution
        locationURI="menu:org.eclipse.ui.main.menu?after=additions">
    <menu
        label="Viewer"
        mnemonic="V"
        id="com.mycom.treeviewer.menus.sampleMenu">
    <command
        commandId="com.mycom.treeviewer.commands.sampleCommand"
        id="com.mycom.treeviewer.menus.sampleCommand"
        label="TreeViewer"
        mnemonic="T"
        tooltip="Explore Treeviewer">
    <visibleWhen>
    <reference
        definitionId="com.teamcenter.rac.ui.inMainPerspective">
    </reference>
    </visibleWhen>
    </command>
    </menu>
</menuContribution>
<menuContribution
    locationURI="toolbar:org.eclipse.ui.main.toolbar?after=additions">
    <toolbar
        id="com.mycom.treeviewer.toolbars.sampleToolbar">
    <command
        commandId="com.mycom.treeviewer.commands.sampleCommand"
        icon="icons/sample.gif"
        id="com.mycom.treeviewer.toolbars.sampleCommand"
        label="TreeViewer"
        tooltip="Explore Treeviewer">
    </command>
    </toolbar>
    </menuContribution>
</extension>
</plugin>

```

- b. Open the **com.mycom.treeviewer.handlers** package, select the **SampleHandler.java** file, choose **File→Rename**, and rename the file to **TreeViewerHandler.java**.

Replace the contents of the **TreeViewerHandler.java** file with the following:

```

package com.mycom.treeviewer.handlers;
import org.eclipse.core.commands.AbstractHandler;
import org.eclipse.core.commands.ExecutionEvent;
import org.eclipse.core.commands.ExecutionException;
/**
 * TreeViewerHandler extends AbstractHandler, an IHandler base class.
 * @see org.eclipse.core.commands.IHandler
 * @see org.eclipse.core.commands.AbstractHandler
 */
public class TreeViewerHandler extends AbstractHandler {
    /**
     * The constructor.
     */
    public TreeViewerHandler() {
    }
    /**
     * the command has been executed, so extract extract the needed information
     * from the application context.
     */
    public Object execute(ExecutionEvent event) throws ExecutionException {
        Explorer explorer = new Explorer();
        explorer.setBlockOnOpen(true);
        explorer.open();
        return null;
    }
}

```

- c. Right-click the **com.mycom.treeviewer.handlers** package, choose **New→Class**, and add the following Java files:

Explorer.java
FileTreeExplorerContentProvider.java
FileTreeExplorerLabelProvider.java

- d. Replace the contents of the **Explorer.java** file with the following:

```

package com.mycom.treeviewer.handlers;
import java.io.File;
import org.eclipse.jface.viewers.TreeViewer;
import org.eclipse.swt.widgets.*;
import org.eclipse.jface.window.ApplicationWindow;
public class Explorer extends ApplicationWindow {
    public Explorer() {
        super(null);
    }
    protected Control createContents(Composite parent) {
        TreeViewer tv = new TreeViewer(parent);
        tv.setContentProvider(new FileTreeExplorerContentProvider());
        tv.setLabelProvider(new FileTreeExplorerLabelProvider());
        // if for Linux/Unix platforms
        // tv.setInput(new File("/usr/"));
        tv.setInput(new File("C:\\"));
        return tv.getTree();
    }
}

```

Note:

By default, this example reads the **C:** drive on Windows systems. To read the **/user** drive on UNIX systems, change code in the **Explorer.java** file.

- e. Replace the contents of the **FileTreeExplorerContentProvider.java** file with the following:

```
package com.mycom.treeviewer.handlers;
import java.io.File;
import org.eclipse.jface.viewers.ITreeContentProvider;
import org.eclipse.jface.viewers.Viewer;
public class FileTreeExplorerContentProvider implements ITreeContentProvider
{
    public Object[] getChildren(Object element)
    {
        Object[] children = ((File) element).listFiles();
        return children == null ? new Object[0] : children;
    }
    public Object[] getElements(Object element)
    {
        return getChildren(element);
    }
    public boolean hasChildren(Object element)
    {
        return getChildren(element).length > 0;
    }
    public Object getParent(Object element)
    {
        return ((File)element).getParent();
    }

    public void dispose()
    {
        //No implementation
    }
    public void inputChanged(Viewer viewer, Object old_input, Object new_input)
    {
        //No implementation
    }
}
```

- f. Replace the contents of the **FileTreeExplorerLabelProvider.java** file with the following:

```
package com.mycom.treeviewer.handlers;
import java.io.File;
import org.eclipse.jface.viewers.LabelProvider;
public class FileTreeExplorerLabelProvider extends LabelProvider
{
    public String getText(Object element)
    {
        return ((File) element).getName();
    }
}
```

- g. Choose **File→Save All**.

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

5. Verify the customization.

- a. Launch My Teamcenter and choose **Viewer**→**TreeViewer** on the menu bar, or click the **Explore TreeViewer** button on the toolbar.

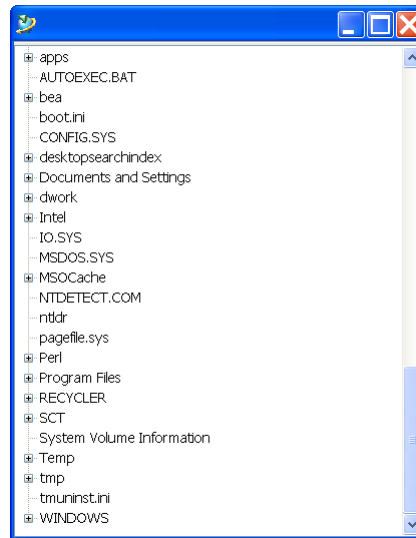


TreeViewer menu command



TreeViewer button

- b. Browse the tree structure.



Tree viewer

Add a quick search item

You can add an existing query to the quick search box at the top of the navigation pane. You can add any query from Query Builder, including queries you create yourself. In the following example, add the **Item Revision...** query to the quick search list.

1. Add values to quick search preferences.
 - a. Open the preferences by choosing **Edit→Options→Search**.
 - b. Open the **Quick_Access_Queries** preference and add the name of the query to the list (for example, add **Item Revision**).
 - c. Open the **Quick_Access_Queries_Attribute** preference and add the search attribute to use for the query (for example, **Name**):

```
Item Revision..._SearchAttribute=Name
```

Note:

The attribute to use for the search (for example, **Name**) must exist on the query. Look at the query in Query Builder to see the attributes on the query.

In this case, the attribute *must* have a key defined in the `qry_user_entry_names_locale.xml` file under the `TC_ROOT\lang\textserver` directory in order to be used in quick search. This is normally optional.

```
<textsrv filename="qry_user_entry_names_locale.xml">
  ...
  <key id="c9MyCustProp">My Custom Property</key>
  ...
</textsrv>
```

2. Clear cache.

- Thin client

Shut down Web services and delete temporary **tc_text.xml.mem** files, for example:

```
%TEMP%\V9000.0.1.20011number\number\TextSrv\en_US\tc_text.xml.mem.
```

Restart services.

- Rich client

To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

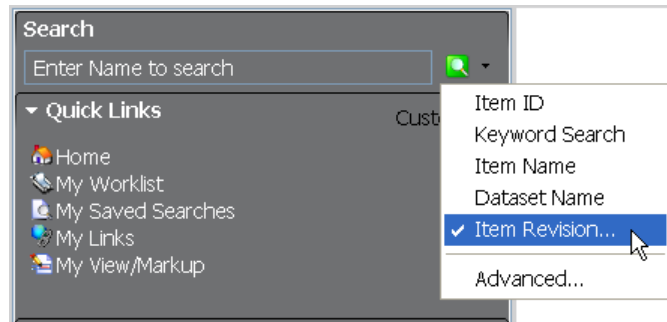
On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory.

On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

Also delete the directory containing temporary server cache (***.mem**) files for the Teamcenter server. On a Windows client, it is typically the `C:\temp\PTeamcenter-version` directory.

3. Test the new quick search item by logging on to Teamcenter and clicking the arrow to the right of the quick search.



Adding a new search type to the quick search list

Change the display color of read-only properties

Caution:

This functionality is part of the legacy swing bean rendering, and is not supported in SWT or Active Workspace. Read-only properties are simply displayed as labels.

You can change the foreground and background colors of read-only properties specified in style sheets. This affects all elements that use style sheets to set the layout, such as the **Summary** view and the **Properties** dialog box.

Override the default values for display. The format of the properties file is the following:

```
# Color for read only properties display
# Value can be a RGB value or color name.
PropertyBeanReadOnly.Background=255,255,128
PropertyBeanReadOnly.Foreground=gray
```

Note:

Background and foreground color customizations are possible for style sheet rendering in the **Summary** tab of My Teamcenter only when the selected object is in a checked-out state. This is because of *flat rendering*, when text or empty display areas do not have widgets (such as a text box in the background for a checked-in object).

Use the Teamcenter registry, which looks for the color setting in the following file order:

- customer.properties**
- properties_user.properties*
- properties.properties*

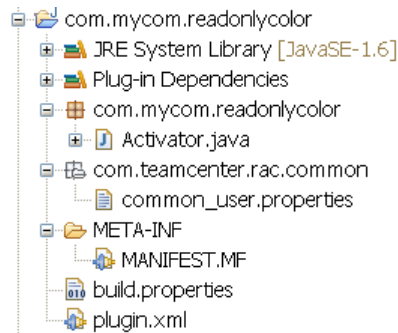
By changing the color settings in the *properties_user.properties* file, the default settings in the *application-name.properties* file are overridden.

Create a custom plug-in that extends the **com.teamcenter.rac.util.tc_properties** extension point and deploy it to the **plugins** directory.

1. Create the project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Then click **Next**.
 - c. In the **Project name** box, type **com.mycom.readonlycolor**. Click **Next**.
 - d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.
 - a. In Eclipse, click your project tab and click its **Dependencies** tab.
 - b. Under **Required Plug-ins**, click the **Add** button, select the **com.teamcenter.rac.util** plug-in, and click **OK**.
 - c. Click your project's **Extensions** tab, click the **Add** button, and select the **com.teamcenter.rac.util.tc_properties** extension point. Click **Finish**.
 - d. Click your project's **Overview** tab and select the **This plug-in is a singleton** check box.
 - e. Save the project by choosing **File→Save All**.
3. Create the project files.
 - a. In **Package Explorer**, right-click the **com.mycom.readonlycolor** project and choose **New→Package**.
 - b. In the **Name** box, type **com.teamcenter.rac.common**.
 - c. Click **Finish**.
 - d. Right-click the **com.teamcenter.rac.common** package and choose **New→File**.
 - e. In the **Name** box, type **common_user.properties**.
 - f. Click **Finish**.
 - g. Add the following to the **common_user.properties** file:

```
PropertyBeanReadOnly.Background=lightGray
PropertyBeanReadOnly.Foreground=blue
```

The plug-in structure looks like this:



- h. Click the **plugin.xml** tab. The text in the tab should look like the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.6"?>
<plugin>
  <extension
    point="com.teamcenter.rac.util.tc_properties">
  </extension>
</plugin>
```

If the text is different, edit it in the tab to resemble the example.

- i. Click your project's **MANIFEST.MF** tab. It should look like the following:

```
Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: Readonlycolor
Bundle-SymbolicName: com.mycom.readonlycolor;singleton:=true
Bundle-Version: 1.0.0.qualifier
Bundle-Activator: com.mycom.readonlycolor.Activator
Bundle-Vendor: MYCOM
Require-Bundle: org.eclipse.core.runtime,
  com.teamcenter.rac.util;bundle-version="8000.3.0"
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Bundle-ActivationPolicy: lazy
```

4. Package the project.

To package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.

- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client.

When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

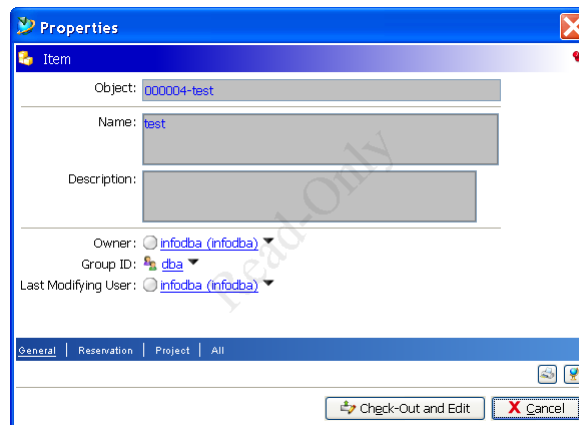
- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

5. Verify the customization.

- a. In the rich client, select an item and click **More Properties** in the **Summary** tab.

Read-only properties are gray with blue characters.



Changed color for read-only properties

Add a column to view occurrence notes

By default, you cannot view occurrence notes on intermediate data capture (IDC) objects in the Multi-Structure Manager. However, you can view the occurrence notes if you can add a column to display

them. To accomplish this, you must define the alias for the occurrence note type in the *TC_DATA\structure_alias.xml* file, and then add the column for the occurrence note type in the *IDCStructureColumnsShownPref* preference.

1. Create the occurrence note on the IDC object.
 - a. Create a structure in Multi-Structure Manager.
 - b. In the data pane to the right, select an occurrence (item revision) in the structure.
 - c. Right-click a column heading and add a column for an occurrence note type (for example, **UG NAME**).
 - d. For the selected occurrence, double-click in the empty cell in that column and type in your note (for example, type **This is my occurrence note**).
 - e. In the left pane, select that occurrence in the structure (the item revision) and choose **Tools→Intermediate Data Capture**. For our example, choose the **Transfer Mode Name** of **tcm_export** because it transfers the **UG NAME** type of occurrence note. When the IDC appears on the tab, there is no content in the data panel, and there is no occurrence note column (for example, no **UG NAME** column). That's because the occurrence note type column must be added to this IDC structure view using a customization.
 - f. Exit the rich client.
2. Open the *TC_DATA\structure_alias.xml* file and add code.

- a. Add the following under the **Alias name="BOM.Property"** node:

```
<Alias name="UG NAME" default=" " type="string" size="0"/>
```

- b. Add the following under the **AliasAlternate name="BOM.Property"** node:

```
<Alias name="UG NAME">
  <Path depth="0..0">*:*:@instancedRef</Path>
  <Xpath>/UserData/UserValue[@title='UG NAME']/@value</Xpath>
</Alias>
```

3. Regenerate the caches and restart the rich client.

Note:

```
generate_metadata_cache -force

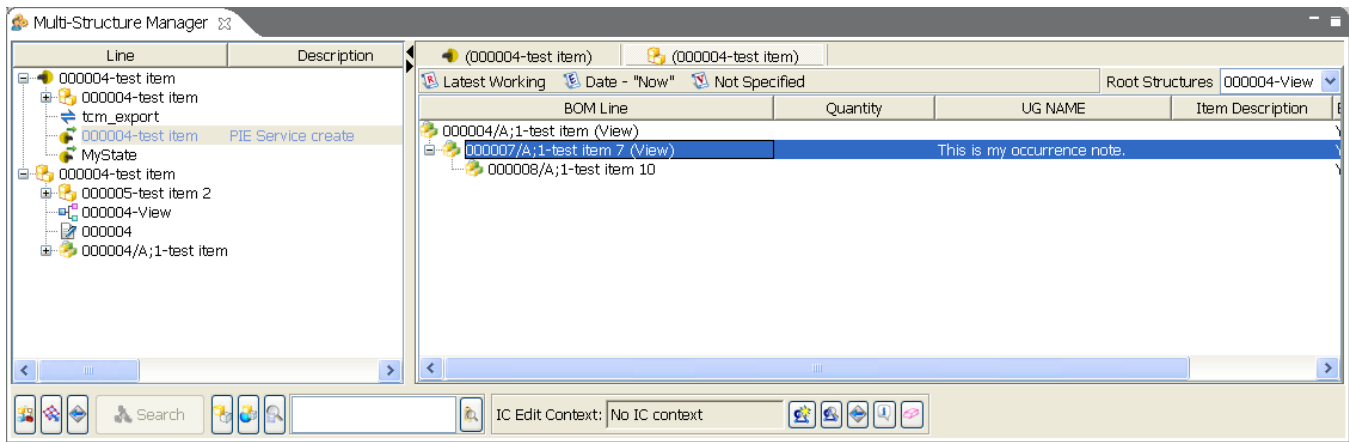
generate_client_meta_cache -log update all
```

4. Add the **UG NAME** value to the **IDCStructureColumnsShownPref** preference, and add a value for the new column's width to the **IDCStructureShownColumnWidthsPref** preference.

Note:

Since these are **interdependent preferences**, you cannot use the Options dialog box to make changes.

5. Select the item structure in My Teamcenter and send it to Multi-Structure Manager. The occurrence note displays on the IDC in the right data panel under the new column (for example, the **UG NAME** column).



New column for occurrence notes

Add perspectives to Manufacturing Process Planner

This example adds new perspectives to the Manufacturing Process Planner application. Creating new perspectives in Manufacturing Process Planner is similar to creating a new perspective in any other application in Teamcenter, but with some modifications. The project that contains the perspectives uses the Eclipse **org.eclipse.ui.perspectives** extension and the Teamcenter **com.teamcenter.rac.aifrcp.perspectiveDefs** extension.

Use the Eclipse plug-in create wizard to create the plug-in. Add packages and classes as needed and copy and paste the class and **plugin.xml** file content from the following steps. If you use the same names, you can cut and paste; otherwise, edit your Java files and **plugin.xml** file using the following steps as a guide.

1. Create the project.
 - a. In Eclipse, choose **File**→**New**→**Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.

- c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.mpp.perspectives** in the **Project name** box and clear the **Source folder** box. Click **Next**.
 - d. In the **New Plug-in Project** wizard **Content** dialog box, do the following:
 - A. Under **Options**, ensure the **This plug-in will make contributions to the UI** check box is selected.
 - B. Click the **No** button next to **Would you like to create a rich client application?**
 - C. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
2. Update the project tabs.
 - a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Click the **Dependencies** tab, click the **Add** button, and select the following:
 - org.eclipse.core.runtime**
 - org.eclipse.ui**
 - com.teamcenter.rac.cme.mpp**
 - com.teamcenter.rac.aifrcp**
 - c. Click the **Runtime** tab, click the **Add** button, and select **com.mycom.mpp.perspectives**.
 3. Create the project files.
 - a. Create project icons.
 - A. Right-click the project, choose **New**→**Package**, and create an **icons** package.
 - B. Click the **Runtime** tab, click the **Add** button, select the **Show non-Java packages** check box, and select **icons**.
 - C. Add the following icons to the package:

Icon	File name	Use
	consumptionPerspective_16.png	Small icon used in the Manufacturing - Process Consumption perspective
	consumptionPerspective_24.png	Medium icon used in the Manufacturing - Process Consumption perspective

Icon	File name	Use
	consumptionPerspective_32.png	Large icon used in the Manufacturing - Process Consumption perspective
	mbom_pbop_16.png	Small icon used in the Manufacturing - MBOM to Product BOP perspective
	mbom_pbop_24.png	Medium icon used in the Manufacturing - MBOM to Product BOP perspective
	mbom_pbop_32.png	Large icon used in the Manufacturing - MBOM to Product BOP perspective
	threeD_16.png	Small icon used in the Manufacturing - Product 3D perspective
	threeD_24.png	Medium icon used in the Manufacturing - Product 3D perspective
	threeD_32.png	Large icon used in the Manufacturing - Product 3D perspective

b. Create the **plugin.properties** file.

- A. Create a **plugin.properties** file in the **com.mycom.mpp.perspectives** project by selecting the project and choosing **File→New→File**.
- B. Add the following name/value pairs to the **plugin.properties** file:

```
#Properties file for com.mycom.mpp.perspectives
Bundle-Name = Manufacturing Incubator Plug-in
consumption.perspective.name = Manufacturing - Process Consumption
product-3d.perspective.name = Manufacturing - Product 3D
mbom-pbop.perspective.name = Manufacturing - MBOM to Product BOP
ProcessConsumption.tooltip = Process Consumption
Product3D.tooltip = Product 3D
MBOM2ProductBOP.tooltip = MBOM to Product BOP
```

c. Create the **plugin.xml** file.

- A. To create an empty **plugin.xml** file, click the project's **Extensions** tab and click the **Add** button in the **Extensions** view, and click the **Cancel** button in the **New Extension** dialog box.

A **plugin.xml** file is added to the project.

- B. Click the project's **plugin.xml** tab and replace the code in the **plugin.xml** file with the following:

```

<plugin>
  <extension
    point="org.eclipse.ui.perspectives">
    <perspective
      class="com.teamcenter.rac.aifrcp.perspective.GenericRACPerspective"
      icon="icons/consumptionPerspective_16.png"
      id="com.teamcenter.rac.cme.mpp.Consumption"
      name="%consumption.perspective.name">
    </perspective>
  </extension>
  <extension
    point="com.teamcenter.rac.aifrcp.perspectiveDefs">
    <perspective
      displayMode="Tertiary"
      icon16="icons/consumptionPerspective_16.png"
      icon24="icons/consumptionPerspective_24.png"
      icon32="icons/consumptionPerspective_32.png"
      id="com.teamcenter.rac.cme.mpp.Consumption"
      label="%consumption.perspective.name"
      legacyAppClass="com.teamcenter.rac.cme.application.
MFGLegacyApplication"
      legacyAppId="com.teamcenter.rac.cme.mpp.MPPApplication"
      ordinality="0"
      taskpaneID="TaskPane"
      tooltip="%consumption.perspective.name">
    <contextRef id="com.teamcenter.rac.cme.mpp.ConsumptionContext"/>
    <viewRef
      allocateSecondaryId="false"
      folderIdOverride="RAC_Folder_UL"
      id="com.teamcenter.rac.cme.processView"
      secondaryId="MPPApplication_1"
      placeholderOnly="false">
    </viewRef>
    <viewRef
      allocateSecondaryId="false"
      folderIdOverride="RAC_Folder_UR"
      id="com.teamcenter.rac.cme.productView"
      secondaryId="MPPApplication_1"
      placeholderOnly="false">
    </viewRef>
    <viewRef
      allocateSecondaryId="false"
      folderIdOverride="RAC_Folder_LR"
      id="com.teamcenter.rac.cme.plantView"
      secondaryId="MPPApplication_1"
      placeholderOnly="false">
    </viewRef>
    <folderLayout
      id="RAC_Folder_UR"
      ratio="0.5">
    </folderLayout>
    <viewRef
      folderIdOverride="RAC_Folder_UL"
      id="com.teamcenter.rac.cme.processView"
      placeholderOnly="true">
    </viewRef>
    <viewRef
      folderIdOverride="RAC_Folder_UR"
      id="com.teamcenter.rac.cme.productView"

```



```

        placeholderOnly="true">
    </viewRef>
    <viewRef
        folderIdOverride="RAC_Folder_LR"
        id="com.teamcenter.rac.cme.plantView"
        placeholderOnly="true">
    </viewRef>
</perspective>
</extension>
<extension point="org.eclipse.ui.contexts">
    <context id="com.teamcenter.rac.cme.mpp.ConsumptionContext"
        description="%consumption.perspective.name"
        name="%consumption.perspective.name"
        parentId="com.teamcenter.rac.cme.mpp.MPPApplication.
applicationContext"/>
</extension>
<extension
    point="org.eclipse.ui.perspectives">
    <perspective
        class="com.teamcenter.rac.aifrcp.perspective.
GenericRACPerspective"
        icon="icons/threeD_16.png"
        id="com.teamcenter.rac.cme.mpp.product3D"
        name="%product-3d.perspective.name">
    </perspective>
</extension>
<extension
    point="com.teamcenter.rac.aifrcp.perspectiveDefs">
    <perspective
        displayMode="Tertiary"
        icon16="icons/threeD_16.png"
        icon24="icons/threeD_24.png"
        icon32="icons/threeD_32.png"
        id="com.teamcenter.rac.cme.mpp.product3D"
        isLegacyAppDefault="false"
        label="%product-3d.perspective.name"
        legacyAppId="com.teamcenter.rac.cme.mpp.MPPApplication"
        ordinality="0"
        taskpaneID="TaskPane"
        tooltip="%product-3d.perspective.name">
    <contextRef id="com.teamcenter.rac.cme.mpp.MPPApplication.
applicationContext"/>
    <viewRef
        allocateSecondaryId="false"
        folderIdOverride="RAC_Folder_UL"
        id="com.teamcenter.rac.cme.productView"
        secondaryId="MPPApplication_1"
        networkId="3D_productView_001"
        placeholderOnly="false">
    </viewRef>
    <viewRef
        allocateSecondaryId="false"
        folderIdOverride="RAC_Folder_UR"
        id="com.teamcenter.rac.cme.graphics.3d"

secondaryId="com.teamcenter.rac.cme.productView_MPPApplication_1"
        networkRef="3D_productView_001"
        placeholderOnly="false">
    </viewRef>
    </perspective>

```

```

</extension>
<extension
    point="org.eclipse.ui.perspectives">
    <perspective
class="com.teamcenter.rac.aifrcp.perspective.GenericRACPerspective"
        icon="icons/mbom_pbop_16.png"
        id="com.teamcenter.rac.cme.mpp.mbom-pbop"
        name="%mbom-pbop.perspective.name">
    </perspective>
</extension>
<extension
    point="com.teamcenter.rac.aifrcp.perspectiveDefs">
    <perspective
        displayMode="Tertiary"
        icon16="icons/mbom_pbop_16.png"
        icon24="icons/mbom_pbop_24.png"
        icon32="icons/mbom_pbop_32.png"
        id="com.teamcenter.rac.cme.mpp.mbom-pbop"
        label="%mbom-pbop.perspective.name"
        legacyAppId="com.teamcenter.rac.cme.mpp.MPPApplication"
        ordinality="100"
        taskpaneID="DefaultTaskPane"
        tooltip="%mbom-pbop.perspective.name">
        <contextRef id="com.teamcenter.rac.cme.mpp.MPPApplication.
applicationContext"/>
        <viewRef
            allocateSecondaryId="false"
            folderIdOverride="RAC_Folder_UL"
            id="com.teamcenter.rac.cme.productBopView"
            secondaryId="MPPApplication_1"
            networkId="mbom-pbop.pbop1"
            placeholderOnly="false">
        </viewRef>
        <viewRef
            allocateSecondaryId="false"
            folderIdOverride="RAC_Folder_LL"
            id="com.teamcenter.rac.cme.productView"
            secondaryId="MPPApplication_1"
            placeholderOnly="false">
        </viewRef>
        <viewRef
            allocateSecondaryId="true"
            id="com.teamcenter.rac.cme.mpp.ProcessRelationView"
            networkRef="mbom-pbop.pbop1"
            placeholderOnly="false"
            secondaryId="mbom-pbom_001"
            standalone="false">
        </viewRef>
        <folderLayout
            id="RAC_Folder_LL"
            ratio="0.5">
        </folderLayout>
        <folderLayout
            id="RAC_Folder_UR"
            ratio="0.5">
        </folderLayout>
    </perspective>
</extension>
</plugin>

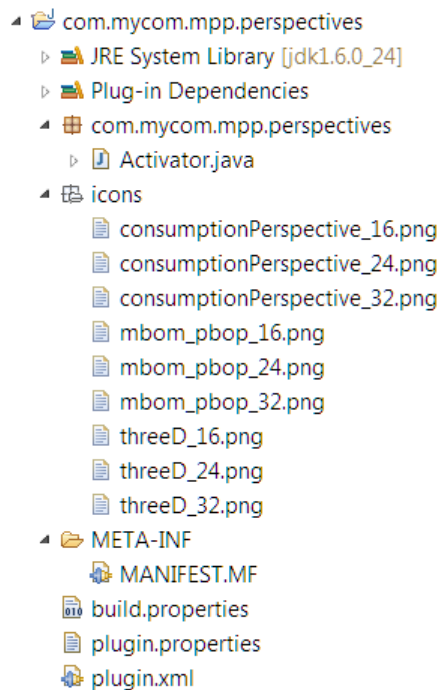
```

- d. Replace the contents of the **MANIFEST.MF** file with the following:

```
Manifest-Version: 1.0
Bundle-ManifestVersion: 2
Bundle-Name: %Bundle-Name
Bundle-SymbolicName: com.mycom.mpp.perspectives;singleton:=true
Bundle-Version: 1.0.0
Bundle-Activator: com.mycom.mpp.perspectives.Activator
Bundle-Vendor: %providerName
Bundle-Localization: plugin
Bundle-RequiredExecutionEnvironment: JavaSE-1.6
Bundle-ActivationPolicy: lazy
Require-Bundle: org.eclipse.core.runtime,
    org.eclipse.ui,
    com.teamcenter.rac.cme.mpp,
    com.teamcenter.rac.aifrcp
Export-Package: com.mycom.mpp.perspectives;x-internal:=true,
    icons;x-internal:=true
```

- e. Choose **File**→**Save All**.

The project files look like the following:

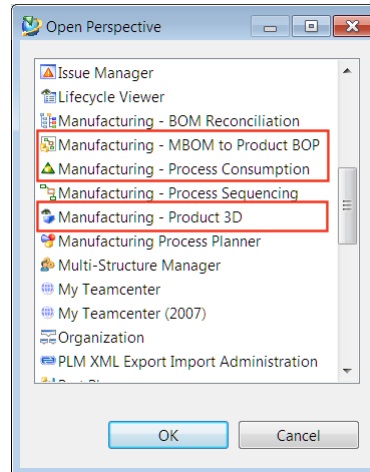


Project files for the com.mycom.mpp.perspectives plug-in

4. Verify the customization.

- a. To verify the configuration so far, choose **Run→Run Configurations** to start the rich client from your Eclipse environment. Clear the workspace by selecting the **Clear** check box in the launch configuration dialog box.
- b. Choose **Window→Open Perspective→Other**.

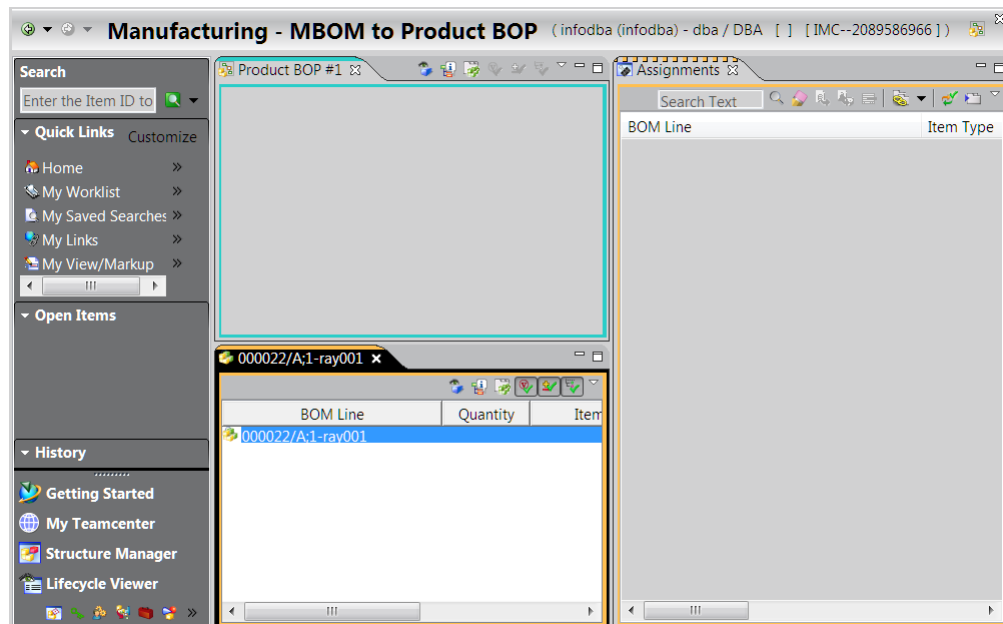
The following new perspectives appear in the list.



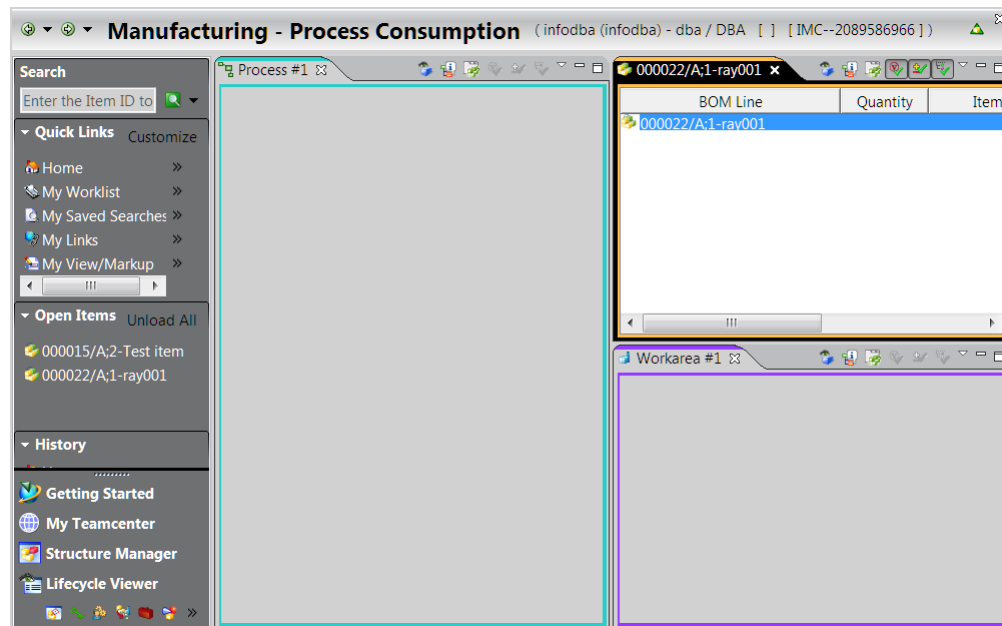
New Manufacturing Process Planner perspectives

- c. Choose the perspectives one-by-one.

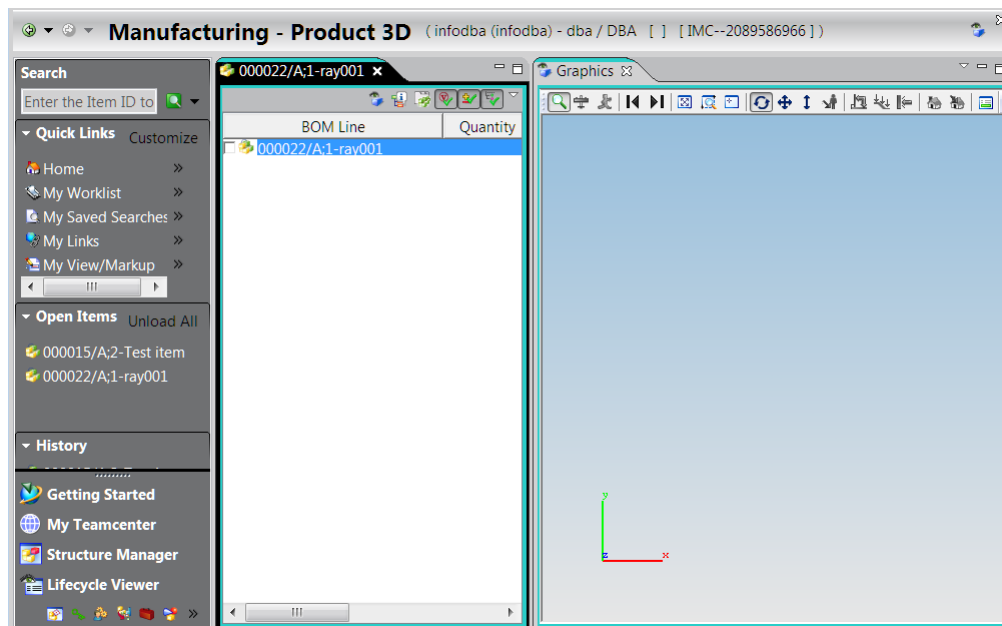
Following are the new perspectives.



Manufacturing - MBOM to Product BOP perspective



Manufacturing - Process Consumption perspective



Manufacturing - Product 3D perspective

5. Package the project.

If you want to package the project for distribution, export it to a JAR file and place it in the `TC_ROOT\portal` directory.

- Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.

- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client.

When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

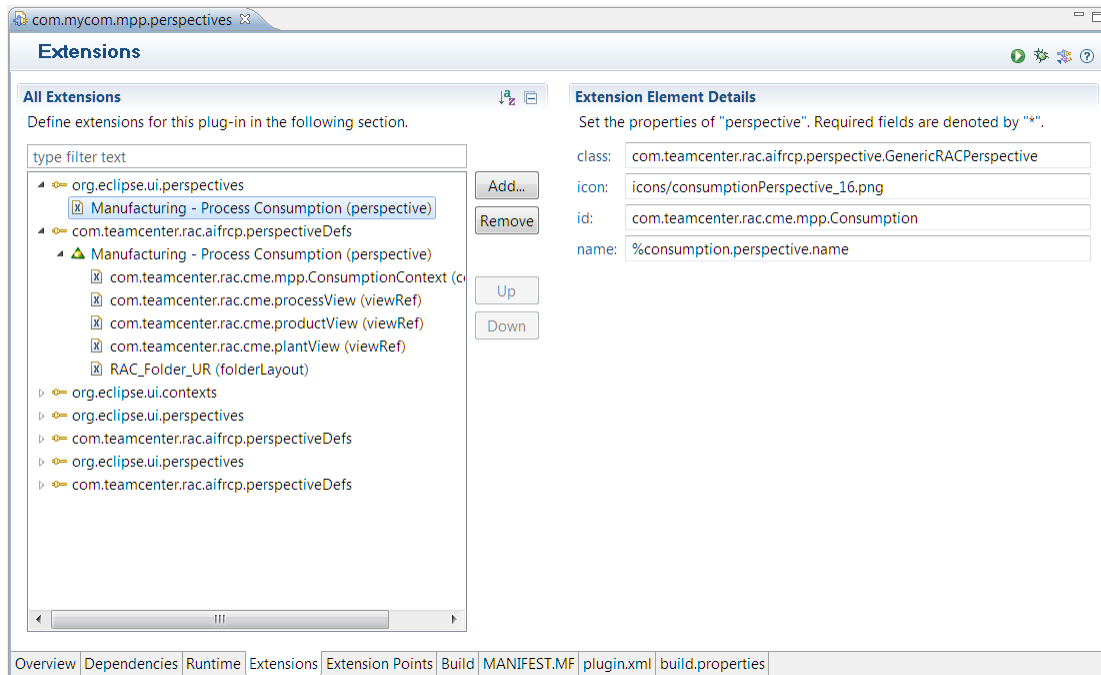
6. Examine the project's **Extensions** tab to review the extensions created in the project.

Copying and pasting the **plugin.xml** file content is a quick way to provide you with extensions for this example. However, when you create your own extensions, instead of placing content directly in the **plugin.xml** file, you work in the project's **Extensions** tab. Therefore, you should review the **Extensions** tab to see how you can create extensions yourself.

To make a perspective, a **org.eclipse.ui.perspectives** extension and a **com.teamcenter.rac.aifrcp.perspectiveDefs** extension must be created for each perspective.

- a. Examine the **org.eclipse.ui.perspectives** extension.

When the **org.eclipse.ui.perspectives** extension is created, the perspective gets a unique ID, name, and an icon, and the **com.teamcenter.rac.aifrcp.perspective.GenericRACPerspective** class is used.

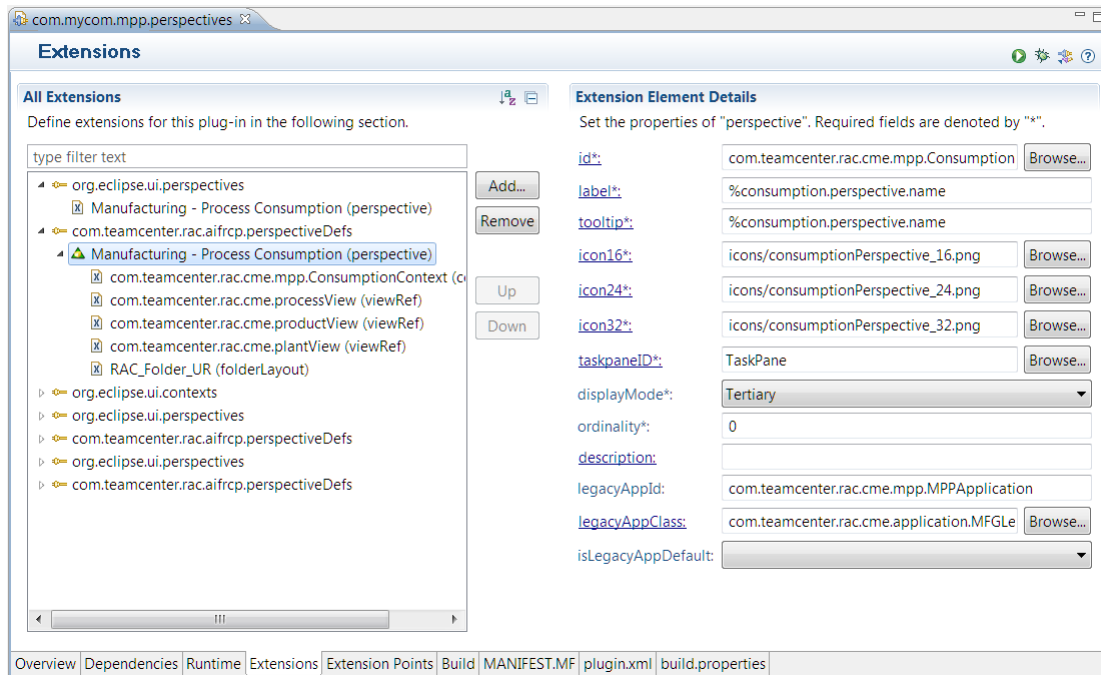


org.eclipse.ui.perspectives extension

- b. Examine the **com.teamcenter.rac.aifrcp.perspectiveDefs** extension.

When the **com.teamcenter.rac.aifrcp.perspectiveDefs** extension is created, use the ID previously given for the perspective and do the following:

- A. Type **TaskPane** in the **taskpaneID** box.
- B. Type values to the **displayMode** and **ordinality** boxes to define the location of the new perspective in the left hand navigation.
- C. Type **com.teamcenter.rac.cme.mpp.MPPApplication** in the **legacyAppID** box.
- D. Leave the **legacyAppClass** box empty.
- E. Choose **false** in the **isLegacyAppDefault** box.



com.teamcenter.rac.aifrcp.perspectiveDefs extension

- c. Under each **com.teamcenter.rac.aifrcp.perspectiveDefs** extension, do the following:
- Add a **contextRef** element and choose **com.teamcenter.rac.cme.mpp.MPPApplication.applicationContext** for the ID.
 - Add a **viewRef** element for each view you want to appear in the perspective.

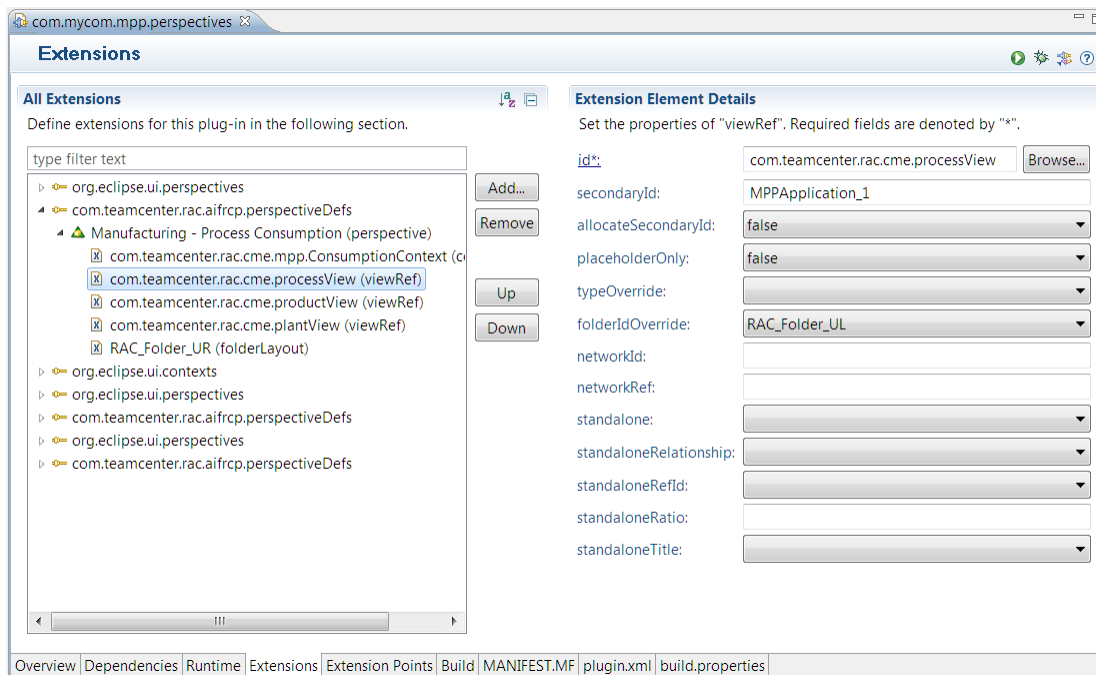
Following are the common primary views that are supported in Manufacturing Process Planner:

- **com.teamcenter.rac.cme.productView**
- **com.teamcenter.rac.cme.processView**
- **com.teamcenter.rac.cme.plantView**
- **com.teamcenter.rac.cme.plantBopView**
- **com.teamcenter.rac.cme.workinstructions.win32.STXLibraryView** (for STX)
- **com.teamcenter.rac.cme.applInterfaceView** (for IDC)
- **com.teamcenter.rac.cme.plantBopView** (for EBOP)
- **com.teamcenter.rac.cme.productBopView** (for EBOP)
- **com.teamcenter.rac.cme.genericBopView** (for EBOP)

For primary views:

- The **id** box shows the ID of the view you want to use.

- The **secondaryId** box value should be composed of the short application ID, (for example, **MPPApplication** for Manufacturing Process Planner), an underscore (**_**), and a view number between 1 and 10, for example, **MPPApplication_1**. Ensure this is the same number in any perspective you define so that this view represent the same view in all perspectives.
- In the **allocateSecondaryId** box, select **false**.
- In the **placeholderOnly** box, select **false**. It should be true only if you do not want this view to appear in the perspective by default.
- In the **folderIdOverride** box, select the folder you want the view to appear in.
- If you want to define a secondary view for this primary view in this perspective, type it in the **networkId** box. It does not matter what the ID is as long as it is unique in this perspective.



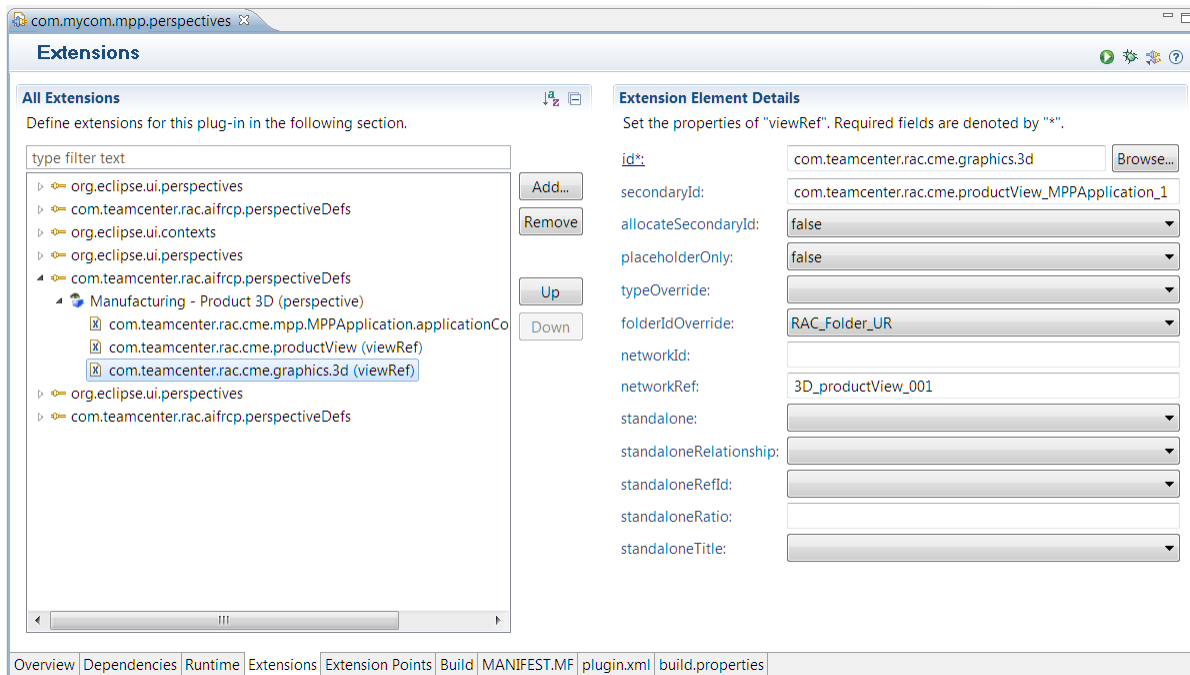
viewRef element (primary view)

- C. If you want to create a view to display graphics, add a **viewRef** element for the view and use the **com.teamcenter.rac.cme.graphics.3d** view type ID.

Because this is a secondary view, note the following:

- The value in the **secondaryId** box is composed of the ID of the primary view that you want to associate this graphics view to, an underscore (**_**), and the **secondaryId** value of the primary view, for example, **com.teamcenter.rac.cme.productView_MPPApplication_1**.

- In the **allocateSecondaryId** box, select **false**.
- In the **placeholderOnly** box, select **false**. It should be true only if you do not want this view to appear in the perspective by default.
- In the **folderIdOverride** box, select the folder you want the view to appear in.
- In the **networkId** box, type the **networkId** value you provided in the primary view.



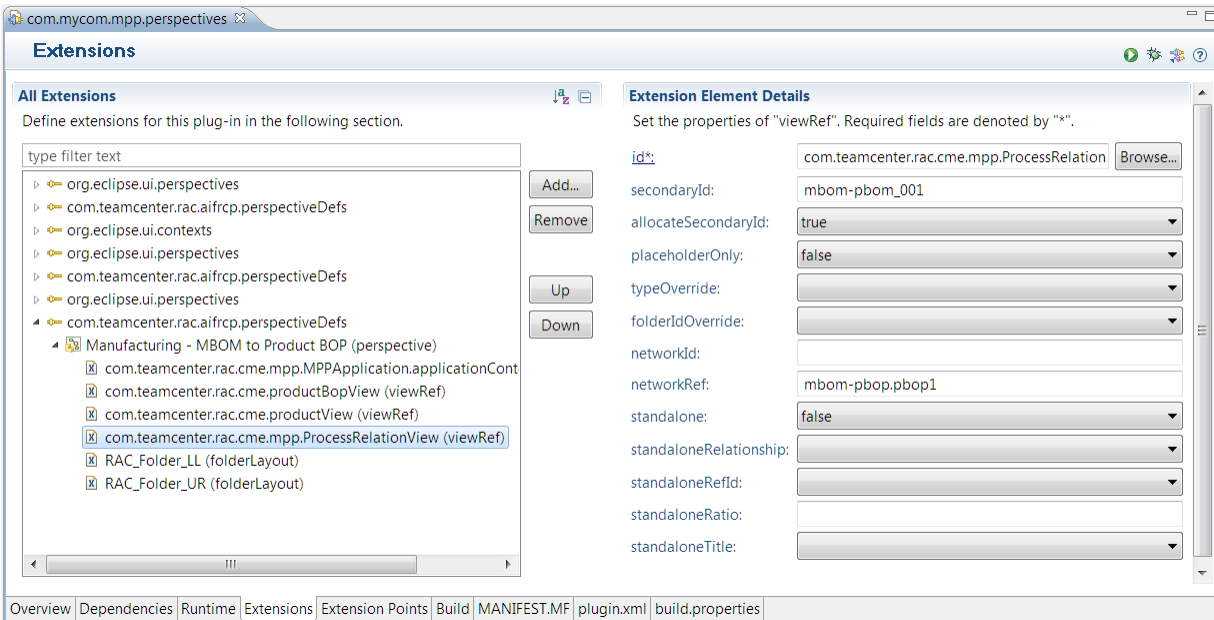
viewRef element for graphics (secondary view)

- D. If you want to create a secondary view that is not a graphics view, add a **viewRef** element for the view and set the **allocateSecondaryId** box to **true**.

Take note of the following for secondary views:

- In the **id** box, browse to the view you want to use for the secondary view.
- In the **secondaryId** box, type a unique ID for the view.
- In the **allocateSecondaryId** box, select **true**.
- In the **placeholderOnly** box, select **false**. It should be **true** only if you do not want this view to appear in the perspective by default.
- In the **folderIdOverride** box, select the folder you want the view to appear in.

- In the **networkRef** box, type the **networkId** value you provided in the primary view. (If the **networkRef** value is set, then the **secondaryId** value also must be set.)



viewRef element (secondary view)

Customize the workflow signoff pane

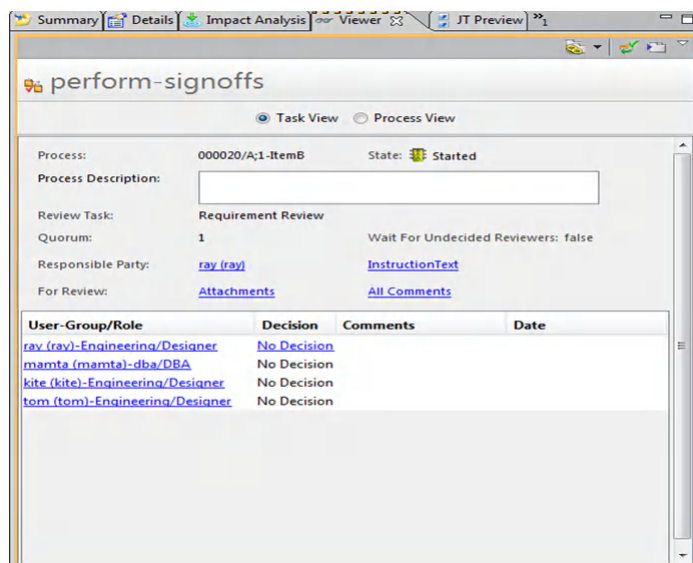
You can customize the workflow signoff pane by customizing the components displayed in the pane rather than writing a new class.

Note:

This example is for informational purposes only and does not represent a recommended customization. Rather, it is an illustration of the customization hooks available for the workflow signoff. Create a customization that fits your business needs.

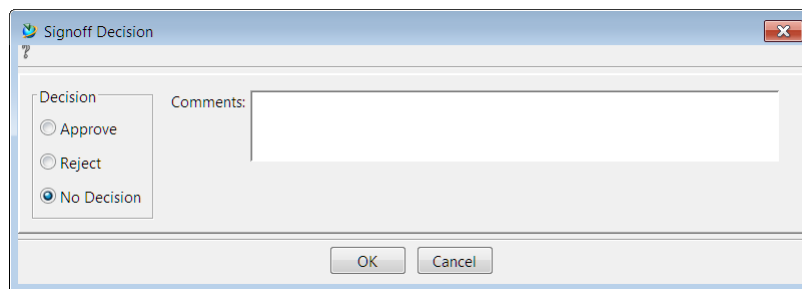
1. Before viewing the sample customization, perform the following steps to view the default signoff pane:
 - a. Log on to Teamcenter with a user whose default group and role is not **dba**.
 - b. Select an item revision and choose **File**→**New**→**Workflow Process**.
 - c. In the **Process Template** box, select the **Requirement Signoff** template and click the **Assign All Tasks** tab to assign the task to the current user and other reviewers.
 - d. Click **OK**.

- e. In the left pane, select **My Worklist**, open the **Tasks to Perform** folder, and select the review item in the folder.
- f. Click the **Viewer** tab to the right.
The default signoff pane is displayed.



Default signoff pane

- g. Click the **No Decision** link in the **Decision** column of the table.
The **Signoff Decision** dialog box is displayed.



Default Signoff Decision dialog box

The customization adds components to both the signoff pane and the **Signoff Decision** dialog box.

Before performing the customization, keep in mind the following:

- The `com\teamcenter\rac\workflow\commands\newperformsignoff\newperformsignoff_user.properties` file shown in the customization example contains the new classes that are overridden from existing ones. Default imports should be kept intact in the properties file, for example:

```
DecisionDialog=com.teamcenter.rac.workflow.commands.newperformsignoff.
CustomDecisionDialog
NewPerformSignoffTaskPanel=com.teamcenter.rac.workflow.commands.newperformsignoff.
CustomNewPerformSignoffTaskPanel
NewSignoffDecisionPanel=com.teamcenter.rac.workflow.commands.newperformsignoff.
CustomNewSignoffDecisionPanel
```

- In the **com\teamcenter\rac\common\tcviewer\tcviewer_user.properties** file shown in the customization example, the **EPMPPerformSignoffTask.VIEWPANEL** entry points to the custom **NewPerformSignoffTaskPanel** to be able to view the same dialog box in the My Teamcenter **Viewer** tab. Default imports should be kept intact in the properties file, for example:

```
EPMPPerformSignoffTask.VIEWPANEL=com.teamcenter.rac.workflow.commands.newperformsig
noff.
CustomNewPerformSignoffTaskPanel
```

- In the **Signoff Decision** dialog box, if the display name for decisions has to be customized, they can be controlled from the **newperformsignoff_user.properties** file, for example:

```
approve=Approve Decision reject=Reject Decision
```

2. Perform the following steps to customize the workflow signoff:

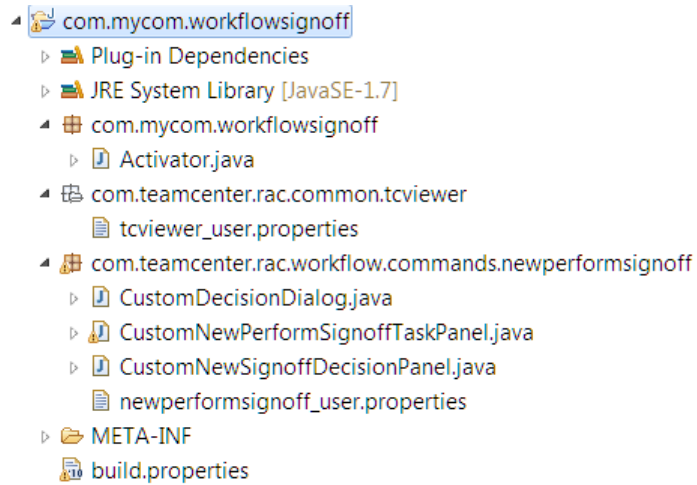
1. Create rejection codes in the Business Modeler IDE.
 - a. Add a new **p3_decision code** property to the **Signoff** business object to store the rejection decision codes in the signoff object.
 - b. Create a new **P3_Rejectcode** list of values containing a list of rejection codes. For example, the codes could be **RJ101**, **RJ102**, **RJ103**, and **RJ104**.
 - c. Attach the **P3_Rejectcode** list of values to the new **p3_decision code** property.
 - d. Deploy the changes to the server.
2. Create the project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
 - c. In the **New Plug-in Project** wizard **Plug-in Project** dialog box, type **com.mycom.workflowsignoff** in the **Project name** box. Click **Next**.
 - d. Do not change the default settings on the **Content** dialog box. Click **Finish**.
3. Update the project tabs.

- a. Click the **Overview** tab and select the **This plug-in is a singleton** check box.
 - b. Click the **Dependencies** tab, click the **Add** button, and select the following plug-ins:
 - org.eclipse.ui**
 - org.eclipse.core.runtime**
 - org.eclipse.osgi.services**
 - org.eclipse.ui**
 - com.teamcenter.rac.aifrcp**
 - com.teamcenter.rac.common**
 - com.teamcenter.rac.external**
 - com.teamcenter.rac.kernel**
 - com.teamcenter.rac.neva**
 - com.teamcenter.rac.util**
 - com.teamcenter.rac.tcapps**
 - c. Choose **File**→**Save All**.
4. Create the project files.
 - a. Right-click the **com.mycom.workflowsignoff** project and choose **New**→**Package**. In the **Name** box, type **com.teamcenter.rac.common.tcviewer**.
 - b. Right-click the **com.teamcenter.rac.common.tcviewer** package and choose **New**→**File**. In the **Name** box, type **tcviewer_user.properties**.
 - c. Right-click the **com.mycom.workflowsignoff** project and choose **New**→**Package**. In the **Name** box, type **com.teamcenter.rac.workflow.commands.newperformsignoff**.
 - d. Right-click the **com.teamcenter.rac.workflow.commands.newperformsignoff** package, choose **New**→**Class**, and create the following Java files:

CustomDesignDialog.java
CustomNewPerformSignoffTaskPanel.java
CustomNewSignoffDecisionPanel

Also right-click the **com.teamcenter.rac.workflow.commands.newperformsignoff** package, choose **New**→**File**, and create a **newperformsignoff_user.properties** file.

The plug-in structure looks like this:



e. Replace the code in the **Activator.java** file with the following:

```
package com.mycom.workflowsignoff;

import org.eclipse.ui.plugin.AbstractUIPlugin;
import org.osgi.framework.BundleContext;

/**
 * The activator class controls the plug-in life cycle
 */
public class Activator extends AbstractUIPlugin {

    // The plug-in ID
    public static final String PLUGIN_ID = "com.teamcenter.rac.custom"; //$NON-NLS-1$

    // The shared instance
    private static Activator plugin;

    /**
     * The constructor
     */
    public Activator() {
    }

    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#start(org.osgi.framework.BundleContext)
     */
    public void start(BundleContext context) throws Exception {
        super.start(context);
        plugin = this;
    }

    /**
     * (non-Javadoc)
     * @see org.eclipse.ui.plugin.AbstractUIPlugin#stop(org.osgi.framework.BundleContext)
     */
    public void stop(BundleContext context) throws Exception {
        plugin = null;
    }
}
```

```

        super.stop(context);
    }

    /**
     * Returns the shared instance
     *
     * @return the shared instance
     */
    public static Activator getDefault() {
        return plugin;
    }
}

```

- f. Replace the code in the **tcviewer_user.properties** file with the following:

```

import=com.teamcenter.rac.common.common,com.teamcenter.rac.util.viewer.view
er,
com.teamcenter.rac.productvision.productvision,
com.teamcenter.rac.scm.common.tcviewer.tcviewer

#The custom dialog to be visible in My Teamcenter Viewer Tab
EPMPPerformSignoffTask.VIEWPANEL=com.teamcenter.rac.workflow.commands.
newperformsignoff.CustomNewPerformSignoffTaskPanel

```

- g. Replace the code in the **CustomDecisionDialog.java** file with the following:

```

// @<COPYRIGHT>@
// =====
// Copyright 2012.
// Siemens Product Lifecycle Management Software Inc.
// All Rights Reserved.
// =====
// @<COPYRIGHT>@

package com.teamcenter.rac.workflow.commands.newperformsignoff;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;

import javax.swing.ButtonGroup;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JScrollPane;

import com.teamcenter.rac.aif.AIFDesktop;
import com.teamcenter.rac.kernel.TCCRDecision;
import com.teamcenter.rac.kernel.TCComponentSignoff;
import com.teamcenter.rac.kernel.TCComponentTask;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.util.iTextArea;

/**
 * The DecisionDialog constructs the Decision Panel.
 */

```



```

* This sample java class creates a new custom Decision Panel that
* adds the following to the existing panel:
*   - two rejects button corresponding to two different reject codes
*   - a sample custom text area "Notes"
*
* To override the existing performsignoff dialog, locate or create the file:
*   - com.teamcenter.rac.workflow.commands.newperformsignoff/
* newperformsignoff_user.properties
* and add/replace the "DecisionDialog" entry with this custom class name
*
* Ex:
*   DecisionDialog=
*   com.teamcenter.rac.workflow.commands.newperformsignoff.CustomDecisionDialog
*
*/
public class CustomDecisionDialog extends DecisionDialog
{
    /**
     *
     */
    private static final long serialVersionUID = 1L;

    protected JPanel rejectCodePanel;

    String rejectCode = null;

    boolean showRejectCode = false;

    public CustomDecisionDialog(AIFDesktop arg0, TCComponentTask arg1,
        TCComponentSignoff arg2) {
        super(arg0, arg1, arg2);
    }

    public void setDecision( TCCRDecision currentDecision)
    {
        decision = currentDecision;
    }

    @Override
    public void constructDialog()
    {
        super.constructDialog();

        // To reflect codes when you reopen the dialog after saving your Reject decision
        if (decision.equals(TCCRDecision.REJECT_DECISION))
        {
            getRejectCode();
            addRejectCode();
        }

        //You can also have approve codes similar to reject
        rbApprove.addItemListener(new ItemListener()
        {
            @Override
            public void itemStateChanged(ItemEvent e)
            {
                setRejectCode("");
                setDecision(TCCRDecision.APPROVE_DECISION);
            }
        } );
    }
}

```

```

rbReject.addItemListener(new ItemListener()
{
    @Override
    public void itemStateChanged(ItemEvent e)
    {
        addRejectCode();
        setDecision(TCCRDecision.REJECT_DECISION);
    }
} );

rbNoDecision.addItemListener(new ItemListener()
{
    @Override
    public void itemStateChanged(ItemEvent e)
    {
        setRejectCode("");
        setDecision(TCCRDecision.NO_DECISION);
    }
} );

okB.addActionListener( new ActionListener() {
    @Override
    public void actionPerformed( ActionEvent e )
    {
        commit();
    }
} );

//Adding components below comments field
JLabel notes = new JLabel("Notes");
iTextArea notesArea = new iTextArea(2,30);

//If it is a secure task we have an additional password field already existing
if( is_secure_task )
{
    rightPanel.add( "3.1.left.top.preferred.preferred", notes );
    rightPanel.add( "3.2.left.top.preferred.preferred", new JScrollPane( notesArea ) );
}
else
{
    rightPanel.add( "2.1.left.top.preferred.preferred", notes );
    rightPanel.add( "2.2.left.top.preferred.preferred",
new JScrollPane( notesArea ) );
}

validate();
repaint();
}

private void commit()
{
    try {
        signoffObj.setProperty("p3_decisioncode", rejectCode);
    } catch (TCEException ex) {
        ex.printStackTrace();
    }
    commitDecision();

    okB.setEnabled( false );
}

```

```

}

private void addRejectCode()
{
    if(!showRejectCode)
    {
        showRejectCode = true;
        //Show the Reject code LOV
        TCComponentListOfValues codeLov =
        TCComponentListOfValuesType.findLOVByName("P3_Rejectcode");
        try {
            codeLov.getListOfValues().getLOVDisplayValues();
        } catch (TCEException e1) {
            e1.printStackTrace();
        }

        cbCode = new LOVComboBox(codeLov);

        if(rejectCode=="")
        {
            cbCode.setSelectedIndex(0);
            rejectCode = cbCode.getSelectedString();
        }
        else
            cbCode.setSelectedString(rejectCode);

        cbCode.addActionListener(new ActionListener()
        {
            @Override
            public void actionPerformed(ActionEvent arg0) {
                //perform required updates
                setRejectCode(cbCode.getSelectedString());
            }
        } );

        rejectCodePanel = new JPanel();
        rejectCodePanel.add(cbCode);

        //Add the new Reject code Panel to existing DecisionP
        decisionP.add( "top.bind.left.center",rejectCodePanel);

        decisionP.validate();
        masterPanel.validate();

        validate();
        repaint();
    }
    else
    {
        showRejectCode = false;
        rejectCode = "";
        decisionP.remove(rejectCodePanel);

        decisionP.validate();
        masterPanel.validate();

        validate();
        repaint();
    }
}

```

```

private void setRejectCode(String code)
{
    rejectCode = code;
}

private void getRejectCode()
{
    try {
        rejectCode = signoffObj.getProperty("p3_decisioncode");
    } catch (TCEException e) {
        e.printStackTrace();
    }
}
}

```

- h. Replace the code in the **CustomNewPerformSignoffTaskPanel.java** file with the following:

```

// @<COPYRIGHT>@
// =====
// Copyright 2012.
// Siemens Product Lifecycle Management Software Inc.
// All Rights Reserved.
// =====
// @<COPYRIGHT>@

package com.teamcenter.rac.workflow.commands.newperformsignoff;

import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.Insets;

import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;

import com.teamcenter.rac.aif.AIFDesktop;
import com.teamcenter.rac.kernel.TCComponentTask;
import com.teamcenter.rac.util.iTextArea;

/**
 * The PerformSignoffTaskPanel constructs the Perform signoff Panel.
 *
 * This sample java class creates a new custom Perform Signoff Panel that adds
 * the following to the existing panel:
 *
 * - a sample custom button "CustomBtn1"
 * - a sample custom button "CustomBtn2" that is visible only in Workflow
 *   Viewer application
 * - a sample custom text area "Notes"
 * - replaces an existing text label "responsible party" with a custom label
 *   "CustomRespParty"
 *
 * To override the existing performsignoff dialog, locate or create the file:
 * - com.teamcenter.rac.workflow.commands.newperformsignoff/
 *   newperformsignoff_user.properties
 * and add/replace the "NewPerformSignoffTaskPanel" entry with this custom class name

```

```

*
* Ex:
* NewPerformSignoffTaskPanel=
* com.teamcenter.rac.workflow.commands.newperformsignoff.CustomNewPerformSignoffTaskPanel
*
*/

public class CustomNewPerformSignoffTaskPanel extends NewPerformSignoffTaskPanel
{
    //This constructor is called when we create dialog for My Teamcenter Viewer Tab
    public CustomNewPerformSignoffTaskPanel(AIFDesktop theDesktop, JPanel anAppPanel,
TCComponentTask aTask)
    {
        super(theDesktop,anAppPanel, aTask);
    }

    //This constructor gets called when we create dialog in Workflow Viewer
    public CustomNewPerformSignoffTaskPanel(AIFDesktop theDesktop,
NewPerformSignoffDialog aDialog, TCComponentTask aTask)
    {
        super(theDesktop,aDialog, aTask);
    }

    /**
    * InitPanel creates the main display panel and populates the fields with data
    */
    public void initPanel()
    {
        super.initPanel();

        //Adding extra button to the dialog
        JButton custBtn1 = new JButton("CustomBtn1");
        custBtn1.setEnabled(true);
        custBtn1.setVisible(true);
        buttonPanel.add(custBtn1);

        //Button visible in Workflow viewer but not in My Teamcenter Viewer Tab
        if (parentDialog != null)
        {
            JButton custBtn2 = new JButton("CustomBtn2");
            custBtn2.setEnabled(true);
            custBtn2.setVisible(true);
            buttonPanel.add(custBtn2);
        }

        GridBagConstraints cn = new GridBagConstraints();
        cn.anchor = GridBagConstraints.WEST;
        cn.insets = new Insets(2,16,2,16);

        JLabel notesLabel = new JLabel("Notes:");
        iTextArea notesArea = new iTextArea(2,30);

        //Keep adding components by incrementing gridy value
        cn.gridx = 0; cn.gridy = 9; cn.gridwidth = 1; cn.gridheight = 1;
cn.fill = GridBagConstraints.BOTH;
        propertiesPanel.add( notesLabel, cn );
        cn.gridx = 1; cn.gridy = 9; cn.gridwidth = 3; cn.gridheight = 2;
        propertiesPanel.add( new JScrollPane( notesArea ), cn );

        //Get existing Font from labels and reuse it for custom fields

```

```

Font font = lbProcess.getFont();
Font newFont1 = new Font(font.getName(), font.getStyle(), font.getSize()-1);
notesLabel.setFont(newFont1);

//Hide Label which exists on grid
lbRespParty.setVisible(false);

//Replace it with a new Label, calculate the row on grid using gridy
JLabel customRPLabel = new JLabel("CustomRespParty:");
cn.gridx = 0; cn.gridy = 5; cn.gridwidth = 1; cn.gridheight = 1; cn.fill =
GridBagConstraints.BOTH;
propertiesPanel.add( customRPLabel, cn );

repaint();
}
}

```

- i. Replace the code in the **CustomNewSignoffDecisionPanel.java** file with the following:

```

// @<COPYRIGHT>@
// =====
// Copyright 2012.
// Siemens Product Lifecycle Management Software Inc.
// All Rights Reserved.
// =====
// @<COPYRIGHT>@

package com.teamcenter.rac.workflow.commands.newperformsignoff;

import com.teamcenter.rac.aif.AIFDesktop;
import com.teamcenter.rac.kernel.TCComponentSignoff;
import com.teamcenter.rac.kernel.TCComponentTask;
import com.teamcenter.rac.kernel.TCException;

/**
 * The NewSignoffDecisionPanel constructs the Signoff Decision Panel.
 *
 * This sample java class creates a new custom Decision Panel that makes the
 * following changes to the existing panel:
 * - add decision code column
 * - change column widths
 *
 * To override the existing signoff decision dialog, locate or create the file:
 * - com.teamcenter.rac.workflow.commands.newperformsignoff/
 *   newperformsignoff_user.properties
 * and add/replace the "NewSignoffDecisionPanel" entry with this custom class name
 *
 * Ex:
 * NewSignoffDecisionPanel=
 * com.teamcenter.rac.workflow.commands.newperformsignoff.CustomNewSignoffDecisionPanel
 */
public class CustomNewSignoffDecisionPanel extends NewSignoffDecisionPanel
{
    /**
     *
     */
    private static final long serialVersionUID = 1L;

```

```

public CustomNewSignoffDecisionPanel(AIFDesktop arg0, TCComponentTask arg1,
    TCComponentSignoff[] arg2, NewPerformSignoffTaskPanel arg3) {
    super(arg0, arg1, arg2, arg3);
}

public void initPanel()
{
    //Needs to be called explicitly here
    super.initPanel();
}

//Do not modify the sequence of existing columns
//Just add to the array your new custom column
public String[] getColumnNames()
{
    String[] cols = super.getColumnNames();
    String[] newCols = new String[cols.length + 1];
    int i = 0;
    for(i=0; i<cols.length; i++)
    {
        newCols[i] = cols[i];
    }
    newCols[i] = "Decision Code";

    return newCols;
}

//You can modify the complete array to fit your needs for Column widths
//Or can just add width width for your custom column
public String[] getShownColumnWidths()
{
    String[] colWidth = super.getShownColumnWidths();
    String[] newColWidths = new String[colWidth.length + 1];
    int i = 0;
    for(i=0; i<colWidth.length; i++)
    {
        newColWidths[i] = colWidth[i];
    }
    newColWidths[i] = "100";

    return newColWidths;
}

//Here the "p3_decisioncode" property has been added on the "SignOff" business
//Object through BMIDE
//Then this custom property can be used to keep your reject code values persistent
//with SignOff Object.
private String getDecisionCode(TCComponentSignoff newSo)
{
    String decisionCode = null;
    try {
        decisionCode = newSo.getProperty("p3_decisioncode");
    } catch (TCEException e) {
        e.printStackTrace();
    }
    return decisionCode;
}

//The default values for first four rows should not be changed.

```

```
//Just add new value for new column(s)
public void updateRow(int row, TCComponentSignoff newSo)
{
    //Needs to be called explicitly here
    super.updateRow(row, newSo);

    Object obj = null;
    int rowCount = signoffTable.getRowCount();
    if (rowCount > 0 )
    {
        if (row < rowCount)
        {
            {obj = getDecisionCode(newSo);
              signoffTable.setValueAt(obj, row, 4);
            }
        }
    }
    validate();
    repaint();
}
}
```

- j. Replace the code in the **newperformsignoff_user.properties** file with the following:

```
import=com.teamcenter.rac.common.common,com.teamcenter.rac.workflow.common.common

#Override the existing classes with custom classes
DecisionDialog=com.teamcenter.rac.workflow.commands.newperformsignoff.
CustomDecisionDialog
NewPerformSignoffTaskPanel=com.teamcenter.rac.workflow.commands.newperformsignoff.
CustomNewPerformSignoffTaskPanel
NewSignoffDecisionPanel=com.teamcenter.rac.workflow.commands.newperformsignoff.
CustomNewSignoffDecisionPanel

#Change the visible text for decisions in Decision Dialog
approve=Approve Decision
reject=Reject Decision
```

- k. Choose **File→Save All**.
5. Choose the newly created package to be included in the export.
 - a. From the **META-INF** folder, open the **MANIFEST.MF** file.
 - b. From the **runtime** tab, add the
`com.teamcenter.rac.workflow.commands.newperformsignoff` package.
 - c. Save your changes.
 6. Package the project.

To package the project for distribution, export it to a JAR file and place it in the **TC_ROOT\portal** directory.

- a. Right-click the project, choose **Export**→**Plug-in Development**→**Deployable plug-ins and fragments**, select the `TC_ROOT\portal` directory as the destination, and click **Finish**.

The `com.mycom.project-name` JAR file is automatically generated into the `TC_ROOT\portal\plugins` directory.

- b. Run the `TC_ROOT\portal\registry\genregxml` file to register the plug-in with the rich client.

When the script is finished, a new `TC_ROOT\portal\registry\RegistryLoader.xml.gz` file is created.

Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

For more information, see *Ensure your customizations appear*.

7. Verify the customization.

- a. Log on to Teamcenter as the user with the workflow task.
- b. In the left pane, select **My Worklist**, open the **Tasks to Perform** folder, and select the review item in the folder.
- c. Click the **Viewer** tab to the right.

The default signoff pane is displayed. Note that the **Responsible Party** label is changed, a **Decision Code** column is added to the table, and a **CustomBtn1** is added.

User-Group/Role	Decision	Comments	Date	Decision Code
ray (ray)-Engineering/Designer	Reject Decision	Update required.	6-Sep-2012 1...	RJ102
mamta (mamta)-dba/DBA	No Decision			
tom (tom)-Engineering/Designer	No Decision			
kite (kite)-Engineering/Designer	No Decision			

Default signoff pane after decision update

- d. Click the **No Decision** link in the **Decision** column of the table.

The **Signoff Decision** dialog box is displayed. Note that the labels on the **Decision** buttons are changed, a box for the rejection code is added, and a **Notes** box is added.

New Signoff Decision dialog box

8. In the Business Modeler IDE, update the audit log with the new rejection codes.
 - a. Add the **p3_decision code** property to the **Fnd0WorkflowAudit** business object to store the decision code.
 - b. In the **Extensions** tab, open the **Code Generation**→**Libraries** folders, right-click the **Libraries** folder and create a new custom library named **P3_custom**.
 - c. In **Extensions** tab, in **Rules**→**Extensions**, create a new extension for the new library.
 - A. In the **Name** box, type **P3_CustomAuditLog**.

- B. In the **Language** box, select **CPlusplus**.
- C. In the **Library** box, select **P3_custom**.
- D. Click the **Add** button to the right of the **Availability** box to launch the **Extension availability** dialog box.

In the **Business Object Name** box, select **Fnd0AuditDefinition**; in the **Operation Name** box, select **fnd0writeAuditLog()**; and in the **Extension Point** box, select **PostAction**.

- d. In the **Extensions** tab, open the **Audit Manager**→**Audit Definitions** folders and open the **EPMTask:_Reject:isTrue** audit definition.

Click the **Add** button to the right of the **Audit Extensions** box and add your new **Extension P3_CustomAuditLog** extension.

- e. On the toolbar, choose **BMIDE**→**Save Data Model**.
- f. Right-click the newly created extension and choose **Generate extension code**.
- g. Click the **Navigator** tab, right-click the template, and select **Refresh**.

A new **src** folder appears in the template directory, which contains the generated **.cxx** and **.hxx** header files.

The generated files include the **liblibrary-name_undef.h** file and the **output\server\gensrc\library-name\liblibrary-name_exports.h** file. In this case, the files are **libP3_custom_undef.h** and **libP3_custom_exports.h**.

- h. Copy these four files and update the **.cxx** and **.hxx** files with your customization. The input parameters generated with the new function in **.cxx** and **.hxx** files should be replaced with new parameters as shown in the sample code. Also include the required header files in the **.hxx** file. The other two header files should not be changed. Build these files together to create a **dll** file with your library name, in this case, **libP3_custom**.

Following is the sample code for the extension:

```
int P3_CustomAuditLog( tag_t   targetObjTag,
                      int      /*secondaryObjectCount*/,
                      tag_t*    /*secondaryObjectTags*/,
                      char**    /*secondaryQualifiers*/,
                      tag_t     /*eventType*/,
                      int       paramCount,
                      char**    paramNames,
                      char**    paramValues,
                      int /*errorCode*/,
```

```

        const char* /*errorMessage*/,
        tag_t primaryAuditBOTag )
{
    int ifail = ITK_ok;

    //tag_t      job_tag      = NULLTAG;
    char*        signoff      = NULL;
    tag_t        signoffTag   = NULLTAG;
    tag_t        auditClassId = NULLTAG;
    tag_t        signoffClassId = NULLTAG;
    char*        audit_class_name = NULL;
    char*        signoff_class_name = NULL;
    int          signoffDecision = 0;
    logical      isProp        = false;
    char*        propVal       = 0;

    if ( paramCount ==0 )
    {
        return ITK_ok;
    }
    for ( int i=0; i<paramCount; i++ )
    {
        if(strcmp(paramNames[i],"signoff") == 0)
        {
            signoff=paramValues[i];
            break;
        }
    }
    if (signoff == 0)
    {
        return ITK_ok;
    }

    ifail = POM_class_of_instance( primaryAuditBOTag, &auditClassId );
    ifail = POM_name_of_class (auditClassId, &audit_class_name );
    //ifail = EPM_ask_job(targetObjTag, &job_tag);
    ifail = POM_string_to_tag(signoff, &signoffTag);

    ifail = POM_class_of_instance( signoffTag, &signoffClassId );
    ifail = POM_name_of_class (signoffClassId, &signoff_class_name );

    isProp = false;
    ifail = POM_attr_exists("p3_decisioncode", signoff_class_name,
&isProp);
    if (isProp)
    {
        ifail = AOM_ask_value_string ( signoffTag, "p3_decisioncode"

```

```

,&propVal);
    isProp=false;
    ifail = POM_attr_exists("p3_decisioncode", audit_class_name,
&isProp);
    if (isProp)
    {
        ifail = AOM_set_value_string(primaryAuditBOTag,
"p3_decisioncode", propVal);
    }
}

if (audit_class_name)
{
    MEM_free(audit_class_name);
}
if (signoff_class_name)
{
    MEM_free(signoff_class_name);
}
if(propVal)
{
    MEM_free(propVal);
}

return ITK_ok;

}

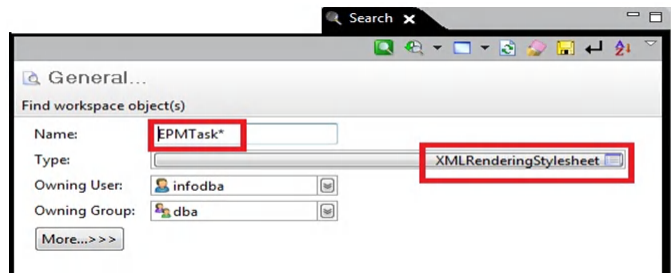
```

- i. In Teamcenter, update the **TC_customisation_libraries** preference to include the name of the new **libP3_custom** library.
- j. Copy the newly generated **dll** file to the **TCROOT\bin** directory.
- k. In Teamcenter, choose **Windows→Show View**, and select the **Process History** view.

A new view appears parallel to the **Summary** tab. In the new view, click the button to the far right of the view toolbar. Choose the **column...** command and a new window displays all the attributes of the audit. Select the **p3_decision** property. The value of this property is updated in the extension code using the signoff object.

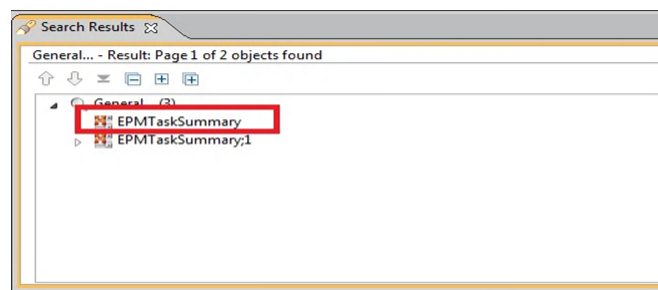
- l. Restart Teamcenter to see the updates. You see audit information created for this new property only for workflows that are rejected after this change is made. Existing audit records are not modified with this update.
9. See the results in the rich client.
 - a. To configure audit logs in the rich client, update the respective style sheet.

Search with the **General...** query for the **EPMTask*** name and select **XMLRenderingStylesheet** as the type.



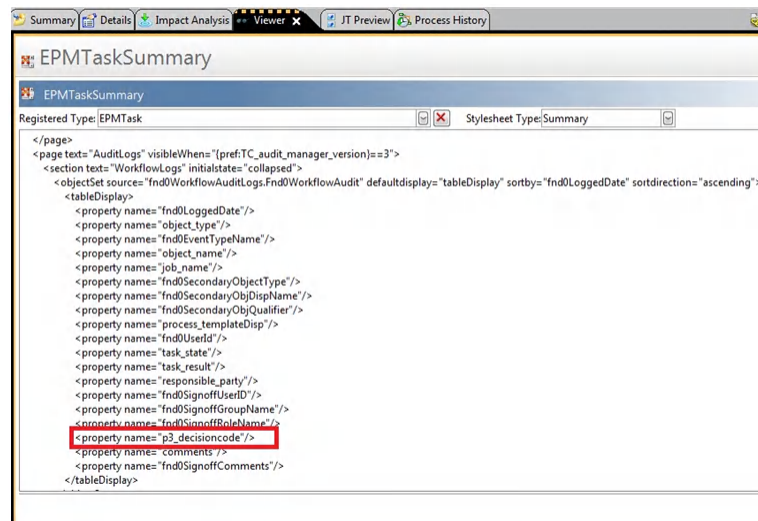
Searching for EPMTask

The following is displayed:



Viewing search results

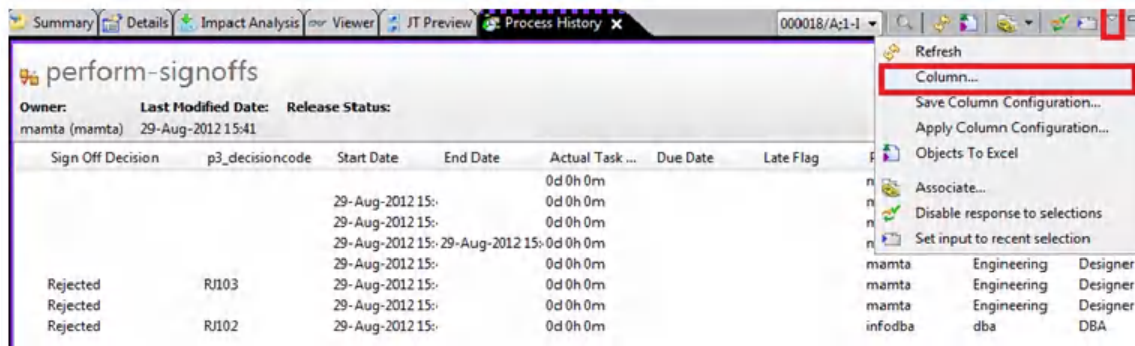
- b. Select the **EPM Task Summary** style sheet and edit it in the **Viewer** tab. Ensure the **Registered Type** and **Stylesheet Type** values are correctly set to **EPMTask** and **Summary**, respectively. Add the new **p3_decisioncode** property for the **Fnd0WorkflowAudit** business object and click **Apply** to save the changes.



Adding a property to the style sheet

- c. On the toolbar, choose **Windows**→**Show view**→**Process History**. A new view is opened parallel to the **Summary** view.

Add a new column using the right-most menu on the view using the custom property added to the **Fnd0WorkflowAudit** business object.

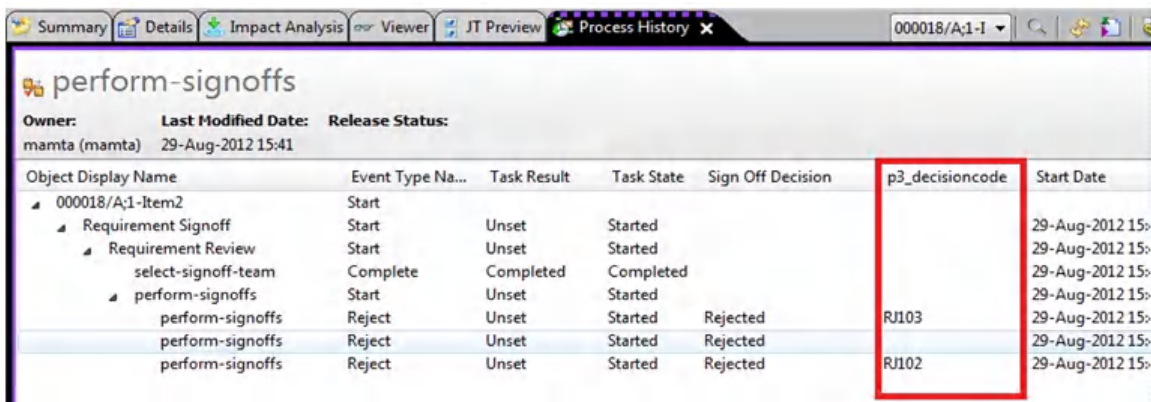


Adding a column

- d. Open the **Worklist**, select the task, and view its update in the **Process History** and the **Audit Logs** tabs.

Note:

If the **Process History** or **Audit Logs** tabs are not displayed, ensure that the **TC_audit_manager** preference is set to **ON** and **TC_audit_manger_version** preference is set to **3**.



Viewing the new column

Adding menus and toolbars

To add menu bars, toolbars, and shortcut menus, use the declarative approach provided by Eclipse. The definition of the menu bar, toolbar, and context menus are provided in the individual plug-in's **plugin.xml** file. Menus and the resulting application logic they call can be placed in a Model-View-Controller (MVC) paradigm. The three parts to the MVC paradigm are:

- Command

Command has a globally unique ID and represents the abstract semantic concept of a behavior, such as copy, paste, and save. A command is *not* the implementation of that behavior nor is it the visual representation of that behavior.

```
<command id="com.teamcenter.rac.command"
  name="%com.teamcenter.rac.command.name">
</command>
```

- Menu contributions

Menu contributions represent a particular view or visual representation of a command. The menu contributions create the menu and toolbar structures and insert them into the correct Eclipse location. The location is specified as an Uniform Resource Identifier (URI) and can be any one of the following:

- Main menu
- Main toolbar
- View toolbar
- View menu
- Context (popup) menu
- Trim area

The menu contribution can define a menu label, mnemonic, or icon. It contains visual references to already defined commands. The visual representations of commands may include labels, icons, and mnemonics. Menu contributions also may include separators. Separators are only visible if there are visible commands before and after a separator. The menu contribution can define when it will be visible with a **visibleWhen** clause. The **visibleWhen** clause refers to all standard Eclipse expressions. This expression can be very simple or very complex and evaluates to either **true** or **false** which determines whether a menu is visible or not.

```
<menuContribution locationURI="menu:org.eclipse.ui.main.menu">
  <menu id="file" label="%menu.file.label" mnemonic="%menu.file.mnemonic">
    <command commandId="org.eclipse.ui.file.refresh"
      mnemonic="%command.refresh.mnemonic"
      style="push">
    </command>
    <separator name="sep1" visible="true"/>
    <command commandId="org.eclipse.ui.file.exit"
      mnemonic="%command.exit.mnemonic"
      style="push">
    </command>
  </menu>
</menuContribution>
```


Note:

You can define the icon for a menu toolbar item by specifying a URL. The URL is case sensitive (for example, **icon.PNG** is not the same as **icon.png**). If you give an incorrect icon URL, a warning is logged to the log file and the **Console** view, if displayed. The menu or toolbar item with the incorrect URL definition is not visible until it is corrected and the rich client is restarted.

If you run in development mode inside the IDE and pull the icon files from a source folder, Windows is not case sensitive and finds the icons. However, once you run the rich client from the command line where the icons are packaged in the plug-in's JAR file, the Java API does not find the icons since those APIs are case sensitive.

Always verify the case is correct for image icons.

A command can also be bound to a key sequence using the **org.eclipse.ui.bindings** extension point.

- **Handler**

A *handler* implements one particular behavior for a given command. For any given command, there can be zero or several handlers defined. However only none or one handler may be active for a given command. The active handler controls the command's enabled state.

Handlers most commonly extend the **AbstractHandler** class. Handlers are provided an application context in their **execute(*)** method. If a command has no active handlers defined, any menu contributions defined for a command are not visible. A command can also define a default handler ensuring that a command always has an active handler. The handler can be declaratively activated via the **ActiveWhen** clause or programmatically activated. The handler also defines declaratively when a command appears enabled in any menu contribution with the **enabledWhen** expression for the handler.

For more information and full examples about how to use the Eclipse declarative approach to menus and toolbars, see the following links:

- http://wiki.eclipse.org/Menu_Contributions
- http://wiki.eclipse.org/Platform_Command_Framework
- http://wiki.eclipse.org/Command_Core_Expressions
- <http://www.vogella.de/articles/EclipseCommands/article.html>

Command

Command has a globally unique ID and represents the abstract semantic concept of a behavior, such as copy, paste, and save. A command is *not* the implementation of that behavior nor is it the visual representation of that behavior.

```
<command id="com.teamcenter.rac.command"
  name="%com.teamcenter.rac.command.name">
</command>
```

Menu contributions

Menu contributions represent a particular view or visual representation of a command. The menu contributions create the menu and toolbar structures and insert them into the correct Eclipse location. The location is specified as an Uniform Resource Identifier (URI) and can be any one of the following:

- Main menu
- Main toolbar
- View toolbar
- View menu
- Context (popup) menu
- Trim area

The menu contribution can define a menu label, mnemonic, or icon. It contains visual references to already defined commands. The visual representations of commands may include labels, icons, and mnemonics. Menu contributions also may include separators. Separators are only visible if there are visible commands before and after a separator. The menu contribution can define when it will be visible with a **visibleWhen** clause. The **visibleWhen** clause refers to all standard Eclipse expressions. This expression can be very simple or very complex and evaluates to either **true** or **false** which determines whether a menu is visible or not.

```
<menuContribution locationURI="menu:org.eclipse.ui.main.menu">
  <menu id="file" label="%menu.file.label" mnemonic="%menu.file.mnemonic">
    <command commandId="org.eclipse.ui.file.refresh"
      mnemonic="%command.refresh.mnemonic"
      style="push">
    </command>
    <separator name="sep1" visible="true"/>
    <command commandId="org.eclipse.ui.file.exit"
      mnemonic="%command.exit.mnemonic"
      style="push">
    </command>
  </menu>
</menuContribution>
```

Note:

You can define the icon for a menu toolbar item by specifying a URL. The URL is case sensitive (for example, **icon.PNG** is not the same as **icon.png**). If you give an incorrect icon URL, a warning is logged to the log file and the **Console** view, if displayed. The menu or toolbar item with the incorrect URL definition is not visible until it is corrected and the rich client is restarted.

If you run in development mode inside the IDE and pull the icon files from a source folder, Windows is not case sensitive and finds the icons. However, once you run the rich client from the command line where the icons are packaged in the plug-in's JAR file, the Java API does not find the icons since those APIs are case sensitive.

Always verify the case is correct for image icons.

A command can also be bound to a key sequence using the **org.eclipse.ui.bindings** extension point.

Handler

A *handler* implements one particular behavior for a given command. For any given command, there can be zero or several handlers defined. However only none or one handler may be active for a given command. The active handler controls the command's enabled state.

Handlers most commonly extend the **AbstractHandler** class. Handlers are provided an application context in their **execute(*)** method. If a command has no active handlers defined, any menu contributions defined for a command are not visible. A command can also define a default handler ensuring that a command always has an active handler. The handler can be declaratively activated via the **ActiveWhen** clause or programmatically activated. The handler also defines declaratively when a command appears enabled in any menu contribution with the **enabledWhen** expression for the handler.

Context menu suppression**Overview of context menu suppression**

The Teamcenter rich client uses Eclipse's platform menu contribution mechanism for displaying menus and toolbars. By default, all available commands are shown on a menu or toolbar unless the Teamcenter Command Suppression application has been configured by the administrator. Due to the nature of the platform, Command Suppression is unable to control the contents of a context menu.

Note:

A context menu is also referred to as a shortcut menu, right-click menu, or popup menu.

To reduce clutter, context menu suppression allows control over the contents of a context menu using an XML file. The elements in this file control which commands are suppressed, and in which view, application context, or with which types of objects.

Implementation

This XML file is stored in the Teamcenter database as a **Context Menu Suppression (Fnd0ContextMenuSuppRuleXML)** dataset and is registered using the **TC_ContextMenuSuppression** preference. The value of the preference is the name of the dataset.

Note:

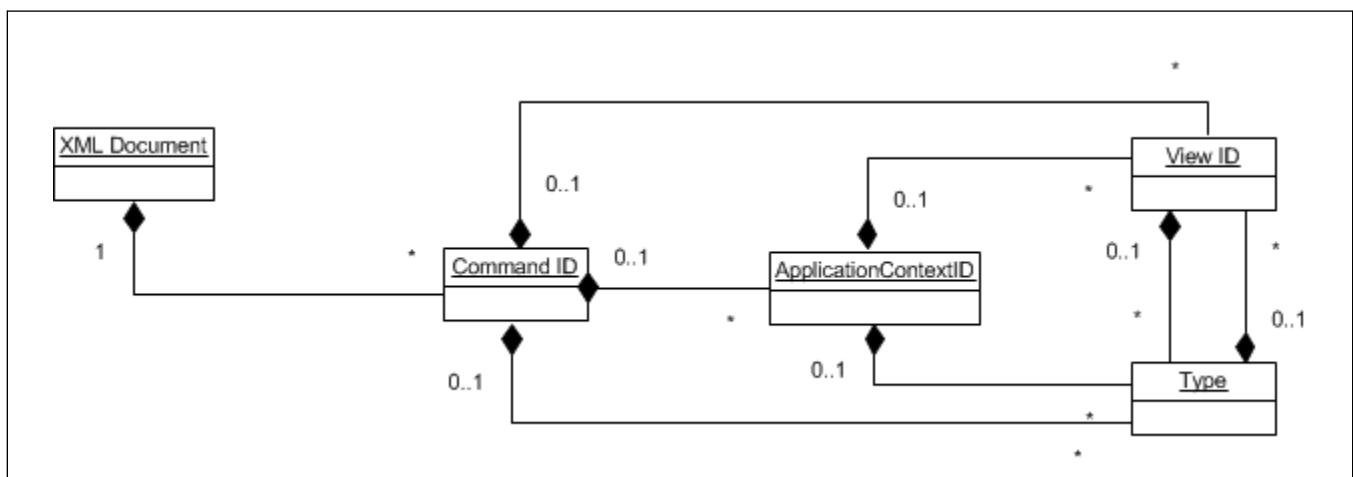
Ensure that **Context Menu Suppression** datasets maintain unique names; however, this is not required by Teamcenter.

The preference can have a site, group, role, or user protection scope, so suppression rules can be implemented based on the user's current credentials. If there are two datasets and two preferences, one pair for each of two groups, a user may see one set of commands on the context menu while in one group, and then, after switching groups, see a different set of commands.

XML file elements

The context menu suppression XML rules consist of the following elements.

- **Command ID**
- **View ID**
- **Type Name**
- **ApplicationContext ID**



XML schema definition

Command ID **Command ID** is a root node for each context menu suppression rule. The **Command ID** node can have one or more **View ID**, **Type Name**, or **ApplicationContext ID** nodes.

View ID Teamcenter applications consist of one or more views, each of which has an associated **View ID**. A rule with a **View ID** node restricts the suppression definition to a specific view. **View ID** elements can hold one or many **Type Name** elements. In such cases, commands can be suppressed for all types in a given view.

Type Name Specifies the database name of the object type for which a command must be suppressed when the selection is made in the rich client interface. This object type can be abstract, persistent, or dynamic.

- A rule defined for an abstract type is inherited by all of its child types.
- A rule defined for a persistent type is applied to the instance selected.
- A rule defined for a dynamic type is applied to the type it represents.

Caution:

Do not write suppression rules for dynamic types like **BOMLine**.

When a **BOMLine** is selected from Structure Manager, if the underlying component is an item revision for example, the suppression rules for item revision are applied instead.

ApplicationContext ID The **ApplicationContext ID** filter can be defined under the root element. This filter allows the administrator to write a suppression rule with respect to a specific application like Structure Manager, My Teamcenter, or Systems Engineering, and so forth. The **ApplicationContext ID** element can hold view or type elements.

When elements are nested, it is considered to be an **AND** grouping, while elements at the same level are considered to be an **OR** grouping.

Example 1

```
<command id="com.teamcenter.rac.expand">
  <view id="com.teamcenter.rac.ui.views.DetailsView"/>
  <view id="com.teamcenter.rac.ui.views.SummaryView"/>
  <type name="Folder"/>
  <type name="Dataset"/>
</applicationContext>
</command>
```

In this example, the **expand** command is suppressed when the current view is the **Details** view or the **Summary** view or if the object type is **Folder** or any **dataset**.

Example 2

```
<command id="org.eclipse.ui.edit.cut">
  <applicationContext id
="com.teamcenter.rac.ui.perspectives.navigatorContext">
    <view id="com.teamcenter.rac.ui.views.TCComponentView">
      <type name="Item"/>
    </view>
    <view id="com.teamcenter.rac.ui.views.SummaryView">
      <type name="Folder"/>
    </view>
    <type name="BOMLine">
      <view id="com.teamcenter.rac.pse.PSEView"/>
    </type>
  </applicationContext>
</command>
```

In this example, the **cut** command is suppressed for the My Teamcenter application when:

```
(type=Item AND view=Teamcenter Component)
OR
(type=Folder AND view=Summary)
OR
(type=BOMLine AND view=Structure Manager)
```

Getting the internal names

Because the XML file requires Teamcenter's *internal* names for objects, commands, views, and application contexts, it is important to know how to find them. Using the display name does not work.

Here are a few common examples.

Display name	Internal name
My Teamcenter (application context)	com.teamcenter.rac.ui.perspectives.navigatorContext
Summary (view)	com.teamcenter.rac.ui.views.SummaryView
Document Revision (object type)	DocumentRevision

To find complete listings of Teamcenter's *internal* names, refer to the following sources:

- Objects** The Business Modeler IDE is the best place to find the internal names for all objects.
- Commands, views, and** The **DumpCMSConfigInfo** utility generates a listing of these names.

application contexts

XML prolog

A full context menu suppression file must have a proper XML prolog, and all suppression elements must be enclosed within **contextMenuSuppression** tags.

```
<?xml version='1.0' encoding='UTF-8' ?>
<contextMenuSuppression>
  <command id ="org.eclipse.ui.edit.cut">
    <applicationContext id
="com.teamcenter.rac.ui.perspectives.navigatorContext">
      <type name="Item"/>
    </applicationContext>
  </command>
  <command ...>
    ...
  </command>
</contextMenuSuppression>
```

Caveats

- This is not a hierarchical or inheritance system. Only a single dataset is evaluated at any given time. If multiple preferences are defined, for example, a group and a role preference, standard Teamcenter preference evaluation precedence determines which preference is valid, and therefore which dataset is used. If the user changes group or role, the preferences are reevaluated, which may result in a different dataset being used.
- The **Summary** view's **Send To...** menu is built using the same rules as the context menu's **Send To...** command. Suppressing one of the **Send To...** commands from the context menu also suppresses it from the **Summary** view's **Send To...** menu.
- There is no upgrade mechanism provided for the context menu suppression rule XML. Any of the contents related to **Command ID**, **View ID**, and **ApplicationContext ID** are subject to change.

Customizing form and properties display

Introduction to customizing forms

A form is a logical storage mechanism that supports captured, informative, and derived data. *Captured data* are the boxes within a form in which users type data; these are generally required boxes. Business rules often dictate that certain boxes be populated before the form is created. *Informative data* are read-only boxes that appear within the form and cannot be modified by the user. *Derived data* are the boxes within a form that are the sum or combination of other boxes, or PDM data that is composed and displayed within the form. Derived data typically cannot be modified.

Forms capture information that users reference and view. These forms store data, such as work order, ECO, or ECN information. Information contained in forms can be used for queries. Companies typically use forms to:

- Capture and store information for work orders, ECOs, or ECNs. This is the most common use of forms.
- Maintain processing information to support other features. For example, a form can be developed to maintain the next available number when automatically generating numbers. This type of form is used by administrators.

There are two perspectives in creating forms. The first perspective is the actual creation of forms within the rich client. It takes very little work and requires no programming ability to create and display a form. The second perspective is that of the programmer, which focuses on the techniques used to develop sophisticated solutions.

There are several processes used to create forms in the Teamcenter rich client.

Note:

- Character (**char**) and character array (**char []**) data types are not supported in forms. Use a string (**string**) or a string array (**string []**) data type with a length of 1.
- The default string value for **False** is an empty string. Therefore, end users must click the **Show empty properties** link on the rendering page to see this logic attribute if its value is set to **False**.

Methods of form customization

The forms feature allows companies to create electronic versions of forms, representing logically organized data fields. Forms persist to a POM class object. The display is determined by the form properties in the POM class definition.

Teamcenter supports the following methods of form customization:

- XML style sheet
Allows you to define a set of properties to display, including the display order and user interface rendering component. The XML style sheet method is supported for both the rich client and thin client.
- Automatic forms
Allows you to display forms that have no associated interface definition. The interface is created automatically as the form is displayed, based on the storage fields identified within the form POM class.
- JavaBean

Allows you to define forms using JavaBeans and an IDE (such as Eclipse) to present form properties. Each bean knows how to display and save a specific property type. This method is less complex than the abstract rendering method but still requires some programming knowledge.

- **Abstract rendering**

Allows you to write the form display by extending the **AbstractRendering** class. This is the most flexible method of form customization. It is also the most complex method and requires coding.

Note:

The following class attributes have been lengthened from 32 to 128 bytes:

- **item_id** in the **Item** class
- **object_name** in the **WorkspaceObject** class
- **name** in the **AuditLog** class

If you used these attributes in customized code, ensure they still display correctly.

Developing automatic forms

You can create and implement automatic forms. All work is performed on the server; no work is required on the client. This form solution works with existing Teamcenter implementations. Once the rich client is installed, existing forms appear automatically because they are considered to be automatic forms and can display without customization. The rich client assumes that forms are automatic if they are not registered on the client. If an error occurs during form loading, the system throws an exception. All attempts are made to display the data.

1. Create a form type on the server side.
The form type can be created in Teamcenter using the Business Modeler IDE application. All methods require you to enter the form type name: **POM storage**.
2. Create the POM storage in the Teamcenter schema.

Developing forms using JavaBeans

Process for developing forms using JavaBeans

1. Launch your IDE and create a new **JPanel**.
2. Add components to the new **JPanel**. To select a library to add components, select the one that includes the **com.teamcenter.rac** plug-in files. If the library containing the plug-in files is not listed, add it.
3. Locate and select a property bean class.

Note:

The property JavaBeans are located in the **com\teamcenter\rac\stylesheets** directory.

4. Repeat the previous steps to add additional JavaBeans.
The property JavaBeans are now ready to use.

Create a sample custom form using JavaBeans

You can create a custom Java panel (**JPanel**) assigned to a custom business object and display the properties using JavaBeans.

In this example, when a user selects a custom **ItemRevision** business object and chooses the **Viewer** view or **View→Properties**, the custom panel is displayed. The custom properties on the business object are set on the Java panel using property beans (JavaBeans).

Note:

Panel rendering is registered to the business object by placing the following command into a custom **stylesheet_user.properties** file:

```
custom-business-object-name.JAVARENDERING.  
package-name.custom-java-panel-name
```

1. Create a custom item business object.
 - a. In the Business Modeler IDE, create a custom item business object as a child of the **Item** business object, for example, **A5_MyItem**.

Note:

The **A5_** portion of the name is an example of a naming prefix. When you create a Business Modeler IDE project, you are required to define a unique prefix that will be affixed to the name of all custom data model items you create to show that they belong to your custom data model. You can use your own prefix, but remember that the following coding examples use this example prefix.

- b. In the **Business Objects** view, open the item revision business object (for example, **A5_MyItemRevision**), click the **Properties** tab, and create the custom persistent properties you want to display in the panel, for example:
 - **a5_MyDate**
Select the **Date** attribute type.

Tip:

Type a display name for each of these new properties, for example, **Test Date**.

- **a5_MyDouble**
Select the **Double** attribute type.
- **a5_MyFlag**
Select the **Boolean** attribute type.
- **a5_MyLongString**
Select the **LongString** attribute type.
- **a5_MyLOV**
Select the **String** attribute type. Attach an LOV to this property, for example, **BillCodes**.
- **a5_MyRef**
Select the **TypedReference** attribute type. Choose a reference business object, for example, **Item**.

- c. Set the **Enabled** property constant to **True** for each of the new properties. This means the property is enabled for display in the user interface.
- d. Deploy the custom template from the Business Modeler IDE to your Teamcenter server. If you use the deployment wizard, select the **Generate Server Cache?** check box to generate shared server cache that contains the new data model.
- e. After deployment, test your new business object in the Teamcenter rich client by creating an instance of it.
For example, in the My Teamcenter application, choose **File→New→Item**.
Your new business object appears in the **New Item** dialog box. Choose your new business object and launch the **New Item** wizard.

2. Create the Eclipse project.

- a. In Eclipse, choose **File→New→Project**.
- b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.
- c. In the **Project name** box, type **com.mycom.custompanel**. Click **Next**.
- d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
- e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.

3. Update the project tabs.
 - a. Update the **Dependencies** tab.
 - A. In Eclipse, click your project tab and click its **Dependencies** tab.
 - B. Under **Required Plug-ins**, click the **Add** button.
 - C. Select the following plug-ins from the list by holding down the Ctrl key while you click them:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.external
com.teamcenter.rac.kernel
com.teamcenter.rac.neva
com.teamcenter.rac.tcapps
com.teamcenter.rac.util

- D. Click **OK**.
 - b. Update the **Runtime** tab.
 - A. Click your project's **Runtime** tab.
 - B. Under **Exported Packages**, click the **Add** button.
 - C. Select your project and click **OK**.
 - c. Update the **Extensions** tab.
 - A. Click your project's **Extensions** tab.
 - B. Click the **Add** button.
 - C. Select the **com.teamcenter.rac.util.tc_properties** extension point.
 - D. Click **Finish**.
 - d. Save the project by choosing **File**→**Save All**.
4. Create the project files.
 - a. Create the custom viewer panel.
 - A. In **Package Explorer**, right-click your project and choose **New**→**Package**.

- B. In the **Name** box, type **com.teamcenter.rac.stylesheet**. This is the path name where rich client style sheet files are located.
- C. Click **Finish**.
- D. Right-click the **com.teamcenter.rac.stylesheet** package and choose **New→File**.
- E. Open the project's **Runtime** tab, click the **Add** button, and add the **com.teamcenter.rac.stylesheet** package.
Note that the listed exported packages are **com.mycom.custompanel** and **com.teamcenter.rac.stylesheet**. This ensures that these packages are listed as exported packages in the **MANIFEST.MF** file.
- F. In the **File Name** box, type **CustomSamplePanel.java** and click **Finish**.
- G. Open the **CustomSamplePanel.java** file and enter the following code:

```
package com.teamcenter.rac.stylesheet;
import org.eclipse.swt.SWT;
import org.eclipse.swt.widgets.Display;
import org.eclipse.swt.widgets.Shell;
import com.teamcenter.rac.util.PropertyLayout;
import java.awt.BorderLayout;
import javax.swing.JLabel;
import javax.swing.JPanel;
/**
 * This example shows how to use the property java beans to display properties/form.
 * To use this example, please create an object with following attributes (or change the
 * property names used in this example to match the data model you have):
 * a5_MyDouble: double type
 * a5_MyDate:    date type
 * a5_MyFlag:    logical type
 * a5_MyLongString: long string type
 * a5_MyRef:     typed reference type
 * a5_MyLOV: a string type attached with LOV
 *
 * To register the usage of this panel, user needs to add a line to the
stylesheet_user.properties file,
 * with the format: <type_name>.JAVARENDERING=<package name>.CustomSamplePanel
 *
 * For example, I have this panel registered to display my custom item revision properties:
 *   A5_MyItemRevision.JAVARENDERING=com.teamcenter.rac.stylesheet.CustomSamplePanel
 *
 * When user selects A5_MyItemRevision and launches the properties dialog, this
CustomSamplePanel
 * should be displayed.
 */
public class CustomSamplePanel
    extends JPanel
{
    public CustomSamplePanel()
    {
        try
```

```

        {
            init();
        }
        catch( Exception e )
        {
            e.printStackTrace();
        }
    }
    private void init()
        throws Exception
    {
        this.setLayout( new BorderLayout() );
        this.setOpaque( false );
        JPanel jPanel1 = new JPanel();
        jPanel1.setLayout( new PropertyLayout() );
        jPanel1.setOpaque( false );
        JLabel jLabel1 = new JLabel("Test double");
        PropertyTextField doubleTextField = new PropertyTextField();
        doubleTextField.setProperty( "a5_MyDouble" );
        JLabel jLabel2 = new JLabel("Test date" );
        PropertyDateButton dateButton = new PropertyDateButton();
        //dateButton.setDate( (String) null );
        //dateButton.setDisplayFormat( "d-MMM-yyyy HH:mm:ss" );
        dateButton.setProperty( "a5_MyDate" );
        dateButton.setMandatory( true );
        JLabel jLabel3 = new JLabel("Test boolean");
        PropertyLogicalPanel logicalPanel = new PropertyLogicalPanel();
        logicalPanel.setProperty( "a5_MyFlag" );
        JLabel jLabel4 = new JLabel("Test longstring");
        PropertyLongTextPanel longTextPanel = new PropertyLongTextPanel();
        longTextPanel.setProperty( "a5_MyLongString" );
        JLabel jLabel5 = new JLabel("Test ref");
        PropertyObjectLink refLink = new PropertyObjectLink();
        refLink.setProperty( "a5_MyRef" );
        JLabel jLabel6 = new JLabel("Test Lov");
        PropertyLOVUIComponent lovUIComp = new PropertyLOVUIComponent();
        lovUIComp.setProperty( "a5_MyLOV" );

        this.add( jPanel1, BorderLayout.WEST );
        jPanel1.add( "1.1", jLabel1 );
        jPanel1.add( "1.2", doubleTextField );
        jPanel1.add( "2.1", jLabel2 );
        jPanel1.add( "2.2", dateButton );
        jPanel1.add( "3.1", jLabel3 );
        jPanel1.add( "3.2", logicalPanel );
        jPanel1.add( "4.1", jLabel4 );
        jPanel1.add( "4.2", longTextPanel );
        jPanel1.add( "5.1", jLabel5 );
        jPanel1.add( "5.2", refLink );
        jPanel1.add( "6.1", jLabel6 );
        jPanel1.add( "6.2", lovUIComp );
    }
}

```

Note:

If you want to set your panel's background color, you cannot use the **setOpaque(false)** tag. For more detail on the usage of this Java API, go to the following URL:

<http://download.oracle.com/javase/6/docs/api/javax/swing/JComponent.html#setOpaque%28boolean%29>

- b. Create a style sheet property file to render the new panel.
 - A. Right-click the **com.teamcenter.rac.stylesheet** package and choose **New→File**.
 - B. In the **File Name** box, type **stylesheet_user.properties** and click **Finish**. This file adds content to the **stylesheet.properties** file. The **_user** name in the file indicates that this is a custom file provided by a user.
 - C. Open the **stylesheet_user.properties** file and enter the following line:

```
A5_MyItemRevision.JAVARENDERING=
com.teamcenter.rac.stylesheet.CustomSamplePanel
```

- D. In the rich client, create a preference for this rendering. Choose **Edit→Options→Filters** and click the **Create a new preference definition** button in the upper left corner of the dialog box. In the **Name** box, type **A5_MyItemRevision.JAVARENDERING** and in the **Values** box, type **com.teamcenter.rac.stylesheet.CustomSamplePanel**.

- c. Save the new files by choosing **File→Save All**.

5. Package the project.

- a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**. The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.
- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client. When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

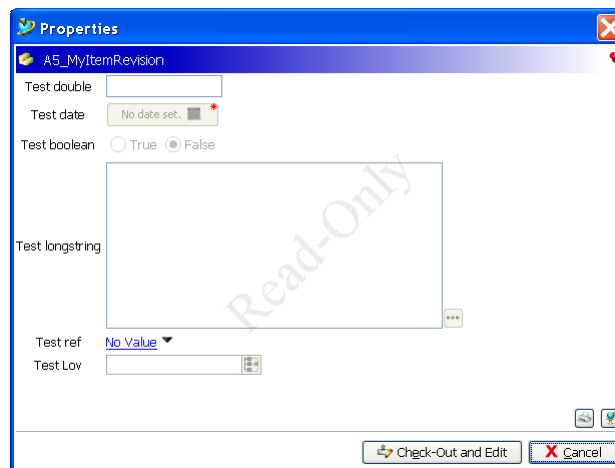
Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.
On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory.
On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

6. Verify the customization.

- a. In the rich client, choose **File→New→Item** and select the new custom item type (for example, **A5_MyItem**), and create an instance of it.
- b. Select the item revision and click the **Viewer** tab or choose **View→Properties** to see your new panel.



Custom form using JavaBeans

Using the Teamcenter property beans

You can create a **JPanel** component in which to contain the property beans, place them in the panel, and connect them directly to the form properties by entering in the property for the box. Once this is done, compile the form and register it with the rich client.

Registration is accomplished by adding an entry in the form of *Form-Type-Name*.**FORMJAVARENDERING=Your-Form-Class** to the

com.teamcenter.rac.stylesheet.stylesheet_user.properties file. Once registered, the rich client uses the custom form when the form type is opened.

For example, if the form type is **MyFormType** and the form display class is named **com.mycompany.forms.MyForm**, the entry is added to the properties file, as follows:

```
My\ Form\ Type.FORMJAVARENDERING=com.mycompany.forms.MyForm
```

Often, a form must obtain the reference to the **TCComponentForm** component with which it is rendering. Each property bean has knowledge of the **TCComponentForm** class; however, for the **JPanel** component to recognize the form, you must include a constructor that includes the **TCComponentForm** component. When you use your IDE to create the **JPanel** component, you may be provided a default constructor, for example:

```
public MyPanel()
{
}
```

To reference the **TCComponentForm** component, include a constructor requesting a reference, as shown in the following example:

```
// IDE given
public MyPanel()
{
}
// User written
public MyPanel ( TCComponentForm f )
{
}
```

When the form loads, the system looks first for the constructor with a reference to the **TCComponentForm** component. If one is found, it is used. If not, the default constructor is used.

Developing custom property beans

Each property bean fundamentally knows how to load and save itself. When the form is loaded and ready to be displayed, each property bean is notified to load itself and its data. The property bean uses the defined property and obtains the value from the Teamcenter database. When the user clicks the **Save** button, each property bean is notified to save. When the form is loaded, the top-level container (the **JPanel** component) that is registered for the form type is recursively cycled to look for the **InterfacePropertyComponent** interface. When found, the beans are instructed to either load or save. The property beans can be nested as deeply as desired within containers and still be selected by the system.

To implement property beans, decide which user interface class to subclass. Then, identify a Teamcenter property bean and implement the **InterfacePropertyComponent** interface, which contains two methods:

```
public void load ( TCComponent ) throws Exception
```

```
public void save ( TCComponent f ) throws Exception
```

After you create the component, all other JavaBean rules apply. For example, you can attach icons for reference within an Integrated Development Environment (IDE), such as Eclipse, and attach property rendering rules.

To perform checks prior to loading the property beans, override the **checkObject** method as follows:

```
public void checkObject() throws Exception
{
    //required checks
}
```

Note:

All IDEs that support JavaBeans work with the property beans.

To increase the efficiency of property beans, Siemens PLM Software recommends that you also implement the **InterfaceBufferedPropertyComponent** class. This requires a method that has the following signature:

```
public TCFormProperty saveProperty ( TCComponent f )
throws Exception
```

This method should only use the **setValue<type>Data** calls to the form property and return it. Therefore, all properties in the property bean system are collected and saved in one call. This increases the efficiency of the property bean.

Siemens PLM Software provides both **save()** and **saveProperty()** methods to allow for flexibility in form storage. All property beans delivered with Teamcenter use the **saveProperty()** method. If you choose to override any of the base property beans, Siemens PLM Software recommends that you override the **saveProperty()** method.

Developing forms by extending the abstract class

Process for developing forms by extending the abstract class

Perform the following steps to create a user interface form:

1. Create a form type on the server side.
The form type can be created in Teamcenter using the Business Modeler IDE application. All methods require you to enter the form type name: POM storage.
2. **Extend the schema.**
3. Create the form panel.
Create a Java object that extends from the **AbstractRendering** class, which requires the implementation of two methods: **loadRendering()** and **saveRendering()**. The **loadRendering()**

method is invoked when the form is constructed and reads the values from the **TCComponent** class to populate the form interface. The **saveRendering()** method writes the values from the user interface components to the **TCComponent** class. Override the **checkObject()** method to enable performance of checks prior to loading the form.

4. Register the form panel with the rich client.
Once a form class is written on the client, it must be registered in the **stylesheet_user.properties** registry file to be recognized within Teamcenter. The key within the **stylesheet_user.properties** file points to the implemented **AbstractRendering** class definition and is comprised of the form type name appended with **.FORMJAVARENDERING**. The associated client-side definition file for the Item Revision Master form type is shown in the following code example.

```
ItemRevision\ Master.FORMJAVARENDERING=com.teamcenter.rac.form.ItemRevisionMaster
```

The backslash character and a space (\) in the string create a space. If the backslash character is not used, the space is misinterpreted and the form is displayed using the automatic form display. Java interprets the key as **item** and does not parse past the space, considering it the delimiter for the key/value combination.

Create a sample custom form by extending the abstract class

You can extend the abstract class (**AbstractRendering**) to display properties for a custom business object.

In this example, you create a custom Java form assigned to the **ItemRevisionMaster** form. When a user chooses **File→New→Item** to launch the **New Item** wizard for a custom **Item** business object, a new form is displayed on the **Define additional item revision information** page of the wizard.

This example extends the **AbstractRendering** component. The sample code uses the **getRenderingModified** and **isRenderingModified** methods. These methods ensure the values are copied from the **New Item** wizard to the form.

Note:

The **isRenderingModified()** method is required to use this customized form in the **Viewer** view.

Note:

Rendering of the form is tied directly to the business object by placing the following command into a custom **stylesheet_user.properties** file:

```
custom-business-object-name.FORMJAVARENDERING.  
package-name.custom-java-form-name
```

1. Create a custom item business object.

- a. In the Business Modeler IDE, **create a custom item business object** as a child of the **Item** business object, for example, **A5_MyItem**.

Note:

The **A5_** portion of the name is an example of a naming prefix. When you create a Business Modeler IDE project, you are required to define a unique prefix that will be affixed to the name of all custom data model items you create to show that they belong to your custom data model. You can use your own prefix, but remember that the following coding examples use this example prefix.

- b. Deploy the new custom item business object from the Business Modeler IDE to your Teamcenter server.
- c. After deployment, test your new business object in the Teamcenter rich client by creating an instance of it.
For example, in the My Teamcenter application, choose **File→New→Item**.
Your new business object appears in the **New Item** dialog box. Choose your new business object and launch the **New Item** wizard.
Observe the boxes on the **Define additional item revision information** page. These are provided by the item revision master form. In the following steps, you are going to create your own custom form to replace it and to provide different boxes on this page.

Default form in the item creation wizard

2. Create the Eclipse project.
 - a. In Eclipse, choose **File→New→Project**.
 - b. In the **New Project** dialog box, select **Plug-in Project**. Click **Next**.

- c. In the **Project name** box, type **com.mycom.masterform**. Click **Next**.
 - d. Under **Options**, ensure the **Generate an activator** and **This plug-in will make contributions to the UI** check boxes are selected. Click **Next**.
 - e. Clear the **Create a plug-in using one of these templates** check box. Click **Finish**.
3. Update the project tabs.
 - a. Update the **Dependencies** tab.
 - A. In Eclipse, click your project tab and click its **Dependencies** tab.
 - B. Under **Required Plug-ins**, click the **Add** button.
 - C. Select the following plug-ins from the list by holding down the Ctrl key while you click them:

com.teamcenter.rac.aifrcp
com.teamcenter.rac.common
com.teamcenter.rac.external
com.teamcenter.rac.kernel
com.teamcenter.rac.neva
com.teamcenter.rac.tcapps
com.teamcenter.rac.util
 - D. Click **OK**.
 - b. Update the **Runtime** tab.
 - A. Click your project's **Runtime** tab.
 - B. Under **Exported Packages**, click the **Add** button.
 - C. Select the **com.mycom.custompanel** and **com.teamcenter.rac.stylesheet** packages and click **OK**.
 - c. Update the **Extensions** tab.
 - A. Click your project's **Extensions** tab.
 - B. Click the **Add** button.
 - C. Select the **com.teamcenter.rac.util.tc_properties** extension point.
 - D. Click **Finish**.

- d. Click the **plugin.xml** tab and replace the contents of the file with the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.2"?>
<plugin>
  <extension point="com.teamcenter.rac.util.tc_properties">
    <plugin_properties pluginName="com.mycom.masterform"/>
  </extension>
</plugin>
```

- e. Save the project by choosing **File→Save All**.

4. Create the project files.

- a. Create the custom item revision master form.

A. Right-click the **com.mycom.masterform** package and choose **New→File**.

B. In the **File Name** box, type **A5_MyItemMaster.java** and click **Finish**.

C. Open the **A5_MyItemMaster.java** file and enter the following code:

```
package com.mycom.masterform;

import com.teamcenter.rac.stylesheet.AbstractRendering;
import com.teamcenter.rac.kernel.TCComponent;
import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.kernel.TCProperty;
import com.teamcenter.rac.util.MessageBox;
import com.teamcenter.rac.util.PropertyLayout;
import com.teamcenter.rac.util.VerticalLayout;

import java.util.HashMap;
import java.util.Map;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class A5_MyItemMaster extends AbstractRendering {

  /**
   *
   */
  private static final long serialVersionUID = 1L;
  private JTextField data1_jtextfield;
  private JTextField data2_jtextfield;
  private TCProperty data1_tcproperty;
  private TCProperty data2_tcproperty;
  private TCComponent comp; // TcEng legacy used TCComponentForm, use TCComponent now

  public A5_MyItemMaster( TCComponent c)throws Exception
  {
    super ( c );
  }
}
```

```

        comp = c;
        loadRendering();
    }

    public void loadRendering() throws TCException
    {
        initializeUI();
        data1_tcproperty = comp.getTCProperty("user_data_1");
        data2_tcproperty = comp.getTCProperty("user_data_2");
        data1_jtextfield.setText(data1_tcproperty.getStringValue());
        data2_jtextfield.setText(data2_tcproperty.getStringValue());
    }

    public void saveRendering()
    {
        try
        {
            data1_tcproperty.setStringValueData(data1_jtextfield.getText() );
            data2_tcproperty.setStringValueData(data2_jtextfield.getText() );
            TCProperty[] ps = new TCProperty[2];
            ps[0] = data1_tcproperty;
            ps[1] = data2_tcproperty;

            // TcEng legacy used setTCProperties or setFormTCProperties,
            // use compnent.setTCProperties(ps);

            component.setTCProperties( ps );
        }
        catch ( Exception ex )
        {
            MessageBox.post(ex.getMessage(), null, MessageBox.ERROR);
        }
    }

    public boolean isRenderingModified()
    {
        if( data1_tcproperty != null && !
data1_jtextfield.getText().equals( data1_tcproperty.getStringValue() ) )
        {
            return true;
        }
        if( data2_tcproperty != null && !
data2_jtextfield.getText().equals( data2_tcproperty.getStringValue() ) )
        {
            return true;
        }
        else
        {
            return false;
        }
    }

    @Override
    public Map getRenderingModified ()
    {
        Map modifiedRendering = new HashMap<String, Object> ();
        if( data1_tcproperty != null && !
data1_jtextfield.getText().equals( data1_tcproperty.getStringValue() ) )

```

```

        {
            data1_tcproperty.setStringValueData(data1_jtextfield.getText() );
            modifiedRendering.put( "user_data_1", data1_tcproperty );
        }
        if( data2_tcproperty != null && !
data2_jtextfield.getText().equals( data2_tcproperty.getStringValue() ) )
        {
            data2_tcproperty.setStringValueData( data2_jtextfield.getText() );
            modifiedRendering.put( "user_data_2", data2_tcproperty );
        }
        return modifiedRendering;
    }

private void initializeUI()
{
    setLayout ( new VerticalLayout() );
    JPanel mainPanel = new JPanel( new PropertyLayout());
    mainPanel.setOpaque(false);
    mainPanel.setEnabled(false);
    // Create all the text fields
    data1_jtextfield = new JTextField(15);
    data2_jtextfield = new JTextField(15);
    //Add components to Panel
    mainPanel.add("1.1.right.center",new JLabel("User Data One"));
    mainPanel.add("1.2.left.center", data1_jtextfield);
    mainPanel.add("2.1.right.center",new JLabel("User Data Two"));
    mainPanel.add("2.2.left.center", data2_jtextfield);
    add("unbound.bind", mainPanel);
}
/**
 *The following code is optional depending on whether you want to pre-load the
 *property data from the original (selected) item revision during a Revise.
 * Without it the values are reset/left blank.
 */
@Override
public void setValues(String[] props, Object[] values)
{
    if ( props != null && props.length > 1 )
    {
        for( int i=0; i<props.length; i++ )
        {
            if( props[i].equals( "user_data_1" ) )
            {
                data1_jtextfield.setText(values[i].toString());
            }
            else if( props[i].equals( "user_data_2" ) )
            {
                data2_jtextfield.setText(values[i].toString());
            }
        }
    }
}
}

```


Note:

Note how this file uses the **TCCComponent** method to set properties on components. This method uses the underlying Teamcenter Services API. Prior to Teamcenter 9, the **SOAPPropertyHelper** method, which was a temporary wrapper on top of legacy code, could be used to set properties on components even though it was unpublished.

- b. Create a style sheet property file to render the new form.
 - A. In **Package Explorer**, right-click your project and choose **New→Package**.
 - B. In the **Name** box, type **com.teamcenter.rac.stylesheet**. This is the path name where rich client style sheet files are located.
 - C. Click **Finish**.
 - D. Right-click the **com.teamcenter.rac.stylesheet** package and choose **New→File**.
 - E. In the **File Name** box, type **stylesheet_user.properties** and click **Finish**. This file adds content to the **stylesheet.properties** file. The **_user** name in the file indicates that this is a custom file provided by a user.
 - F. Open the **stylesheet_user.properties** file and enter the following line:

```
A5_MyItemRevisionMaster.FORMJAVARENDERING=
com.mycom.masterform.A5_MyItemMaster
```

- c. Save the new files by choosing **File→Save All**.

5. Package the project.

- a. Right-click the project, choose **Export→Plug-in Development→Deployable plug-ins and fragments**, select the **TC_ROOT\portal** directory as the destination, and click **Finish**. The **com.mycom.project-name** JAR file is automatically generated into the **TC_ROOT\portal\plugins** directory.
- b. Run the **TC_ROOT\portal\registry\genregxml** file to register the plug-in with the rich client. When the script is finished, a new **TC_ROOT\portal\registry\RegistryLoader.xml.gz** file is created.

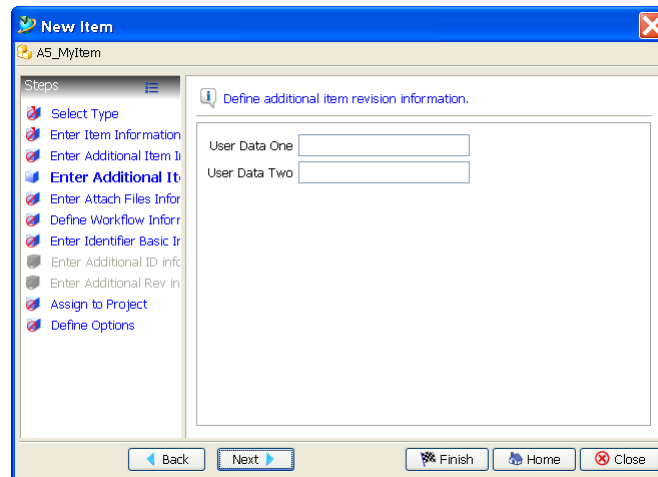
Note:

If you make changes to any of the **.properties** files, or you add new plug-ins or change plug-in content, you must run the **genregxml** script to ensure your changes are included when the rich client starts. This enhances performance because it caches the properties so they can be loaded at startup. The script takes no arguments and generates a **RegistryLoader** file for each locale in the **portal\Registry** directory.

- c. To clear cache, delete the **Teamcenter** subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client. This directory usually contains **RAC** and **TAO** subdirectories.
On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory.
On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

6. Verify the customization.

- a. In the rich client, choose **File→New→Item** and select the new custom item type (for example, **A5_MyItem**), and click **Next**.
- b. Assign the new item an ID and name and click **Next**.
- c. Click **Next** on the **Define additional item information** page.
- d. On the **Define additional item revision information** page, you should see your new item revision master form.



Custom form in the item creation wizard

General comments

The form user interface is not limited to creation using an integrated development environment (IDE) or to the use of any Java component. Third-party Java components can be used within the form.

The Eclipse IDE can be used to generate the contents of the form. Once created, you need only add the code required to read the property values from Teamcenter and set the property values within the associated component on the panel. Other components, such as **LOVButton**, make it easier to render the form properties.

If you upgrade from a previous version of Teamcenter that used the note attribute in the form storage class, instances created from that class will continue to use the note attribute. If you create a new storage class, it uses the string attribute, and instances created from that class use the string attribute.

Starting with Engineering Process Management 2005, you should use the new style sheet package (**com.teamcenter.rac.stylesheet**) instead of the old form package (**com.teamcenter.rac.form**). The old form package is deprecated. If you want to still use the old custom forms in the new package, move the entries you added to the **form_user.properties** file to the **stylesheet_user.properties** file.

Form performance issues

System performance when forms are saved depends largely on the way the API is used. All API components for saving and loading forms are found in the **TCComponentForm** component. The object retrieves form values and can represent all types of data found in Teamcenter. It is patterned after the **TCProperty** object. The **TCComponentForm** class is responsible for handling the retrieval and storage of properties to a form. The **TCFormProperty** object handles the data representation of the value (or property).

It is more efficient to save all form properties in one call, because this minimizes network traffic and limits the save action to one commit to the database. If each property is saved individually, each requires a network call and a commit to the database, which degrades system performance.

For example, a form containing 20 properties takes approximately two seconds to save when all properties are saved in one call and approximately 25 seconds to save when each property is saved individually. There are legitimate cases for saving properties both individually and collectively. Therefore, take care when deciding which API to use to achieve a desired result. The following examples illustrate the different usages of the form API and assume a component **f** of type **TCComponentForm**:

- **Obtaining one form property**

```
TCFormProperty p = f.getFormTCProperty("my_prop_name");
```

- **Obtaining all form properties**

```
TCFormProperty[] props = f.getAllFormProperties();
```

- **Saving one form property**

```
TCFormProperty p = f.getFormTCProperty("my_prop_name");
// Get the property to set
p.setStringValue ( "abc" ); // Set it.
At this point it is saved to the db.
```

- Saving several form properties

```
TCFormProperty[] ps = f.getAllFormProperties();
// Get the property to set
ps[0].setStringValueData ( "abc" );
// Sets the value but is not saved.
ps[1].setStringValueData ( "def" );
// Sets the value but is not saved.
f.setTCProperties(ps);    // Now is saved to the db.
```

In summary, performance gains depend on how form data is saved. Whenever possible, obtain an array of properties and set their values by using methods such as **setStringValueData()** as opposed to the **setStringValue()** method. The **setStringValue()** method sets the value and performs the save immediately. The **setStringValueData()** method sets the value but relies on a subsequent call to perform the commit. Finally, for an array of properties, make a call such as **setTCProperties()** to increase efficiency.

Form events

When a form is displayed, the size is governed by the standard of preferred size for a dialog box. However, it may be necessary to control the dialog box size prior to displaying the form. In this event, implement the **InterfaceRendererEvent** interface within the form display class. This interface forces the implementation of two methods: **dialogDisplayed (OpenFormDialog dlg, boolean state)**. The method is called before the dialog box is displayed. It is the place where the **setBounds()** method for the dialog box can be called.

When a form is displayed, the **okToModify()** method is invoked. If the form is modifiable, it is constructed and displayed as designed. However, if the form is not modifiable, logic is executed to determine what should or should not be editable. When a read-only form is displayed, the components shown in the following table are modified.

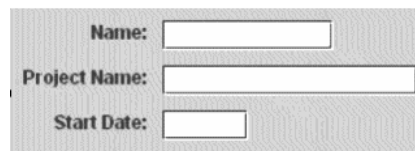
Component	Read-only effect
JTabbedPane	None
JSplitPane	None
JPanel	None
JLabel	None
JScrollPane	None
JProgressBar	None
JTextComponent	Edit ability set to False
All others	Disabled

When a form is not modifiable, all **Container** objects, such as **JTabbedPane**, are ignored and the remaining components are disabled. This is because **Container** objects allow users to traverse and work

through them to view the data. You may want to control the read-only ability of a component within a form, in which case you must override the read-only logic by implementing the **InterfaceRendererEvent** interface and providing body to the **setReadOnly()** method.

Form user interface display components

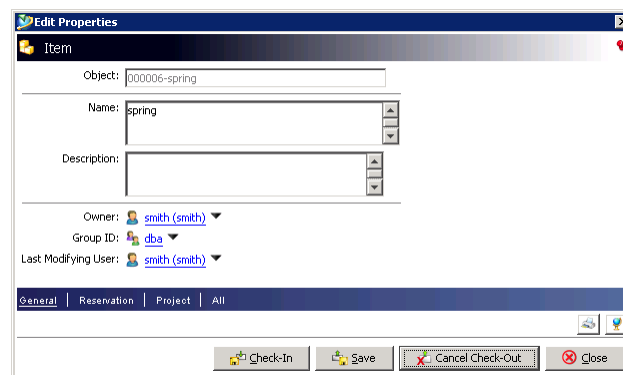
The rich client has changed the strategy of forms in Teamcenter. Forms within the rich client are user interface representations only. Teamcenter stores the information for the form; therefore, when the form is loaded and displayed in the rich client, all data is obtained from the server and placed into user interface components. The base component for a rich client form is a **JPanel** component that provides flexibility for the placement of forms within different parent containers. The following figure shows a form panel.



Form user interface display panel

Displaying a form

When a form is displayed, the user interface definition for the form is constructed, populated, and placed within a container. In the rich client, a form can be displayed under two paradigms: dialog box and viewer. The dialog box displays when the form is opened using the **Open** menu or the form is double-clicked.



Form displayed in a dialog box

The viewer paradigm is used when the form is selected and the **Viewer** tab is active (the following figure). The contents of nonwritable form are displayed as read-only. If the user has write access, the form can be edited.

The screenshot shows a software window titled '000006-spring'. It has a tabbed interface with 'Summary', 'Details', 'Viewer', 'Impact Analysis', and 'JT Preview'. The 'Details' tab is active, displaying a form with the following fields: 'Object' (000006-spring), 'Name' (spring), and 'Description' (Spring for assembly). Below these are three dropdown menus: 'Owner' (smith (smith)), 'Group ID' (dba), and 'Last Modifying User' (smith (smith)). At the bottom, there are four buttons: 'Check-In', 'Save', 'Cancel Check-Out', and 'Clear'. The 'General' tab is selected in the bottom navigation bar.

Form displayed in the rich client viewer

A reusable Java component is used for form panel construction. Given the component reference to the form, call the **RenderingLoader.load(TCComponent)** method, which constructs and returns the form panel. The form definition within the rich client is flexible and supports a wide variety of applications. When a form panel is loaded, the **RenderingLoader** looks to see if there is a registered form panel for the component. If so, the registered form panel is returned. If not, an automatic form is constructed and returned.

Advanced rich client customizations

Customizing Command Suppression

Introduction to customizing Command Suppression

You can suppress menu commands using the Command Suppression application in Teamcenter. If you want to make your custom plug-ins conform to the Command Suppression application, you must add the proper coding to the plug-ins.

The rich client uses Eclipse declarative commands, menu contributions, and handlers to define the vast majority of menu bar and toolbar items. Eclipse uses a Boolean expression-based syntax to allow control over visibility of any specific command in a menu using the **visibleWhen** expression. Just like Eclipse provides some sources like **activeContexts**, **activePartId**, and so on, a rich client **rac_command_suppression** source is defined.

Every command contribution in a custom **plugin.xml** file must have a **visibleWhen** expression consuming the **rac_command_suppression** source using the **with** expression. (That is, those command contributions using the **org.eclipse.ui.menus** extension point to contribute a command using the **menuContribution** source.) Using the **rac_command_suppression** source ensures that the command is visible only when it is not suppressed.

The **rac_command_suppression** source gets called whenever the workbench state changes, for example, when new commands are suppressed, when switching between perspectives, and so on.

The **with** expression (henceforth referred to as the *Command Suppression expression*) consuming the **rac_command_suppression** source must be specified in the **visibleWhen** expression of every command contribution. A template of this Command Suppression expression follows:

```
<with variable="rac_command_suppression">
  <not>
    <iterate operator="or">
      <equals value="command_ID_of_the_command_contribution"/>
    </iterate>
  </not>
</with>
```

Replace *command_ID_of_the_command_contribution* with the command ID of the respective command contribution.

Using the Command Suppression expression in the plugin.xml file

Using the Command Suppression expression in a rich client **plugin.xml** file varies based on the following scenarios:

- The command contribution does not contain a **visibleWhen** expression.
Example before changes:

```
<menuContribution locationURI="menu:edit?after=cut.ext">
  <command commandId="org.eclipse.ui.edit.cut" id="org.eclipse.ui.edit.cut"
    label="%cutAction.NAME">
  </command>
</menuContribution>
```

Example after changes:

```
<menuContribution locationURI="menu:edit?after=cut.ext">
  <command commandId="org.eclipse.ui.edit.cut" id="org.eclipse.ui.edit.cut"
    label="%cutAction.NAME">
    <visibleWhen>
      <with variable="rac_command_suppression">
        <not>
          <iterate operator="or">
            <equals value="org.eclipse.ui.edit.cut"/>
          </iterate>
        </not>
      </with>
    </visibleWhen>
  </command>
</menuContribution>
```

- The command contribution contains a **visibleWhen** expression with a nested **and** expression.
Example before changes:

```
<menuContribution locationURI="popup:org.eclipse.ui.popup.any
?after=org.eclipse.ui.edit.paste">
  <command commandId="com.teamcenter.rac.pasteDuplicate">
    <visibleWhen>
```

```

        <and>
            <reference definitionId="com.teamcenter.rac.cme.mpp.inMainView"/>
            <reference definitionId="com.teamcenter.rac.tcapps.
isPasteDuplicateAllowed"/>
        </and>
    </visibleWhen>
</command>
</menuContribution>

```

Example after changes:

```

<menuContribution locationURI="popup:org.eclipse.ui.popup.any
?after=org.eclipse.ui.edit.paste">
    <command commandId="com.teamcenter.rac.pasteDuplicate">
        <visibleWhen>
            <and>
                <reference definitionId="com.teamcenter.rac.cme.mpp.inMainView"/>
                <reference definitionId="com.teamcenter.rac.tcapps.
isPasteDuplicateAllowed"/>
                <with variable="rac_command_suppression">
                    <not>
                        <iterate operator="or">
                            <equals value="com.teamcenter.rac.pasteDuplicate"/>
                        </iterate>
                    </not>
                </with>
            </and>
        </visibleWhen>
    </command>
</menuContribution>

```

- The command contribution contains a **visibleWhen** expression with nested expressions (not the **and** expression).

Example before changes:

```

<menuContribution locationURI="menu:file?after=save.ext">
    <command commandId="com.teamcenter.rac.importAMRuleTree"
icon="icons/importamruletree_16.png" mnemonic="%importAMRuleTreeAction.MNEMONIC">
        <visibleWhen>
            <reference definitionId="com.teamcenter.rac.accessmanager.inMainView"/>
        </visibleWhen>
    </command>
</menuContribution>

```

Example after changes:

```

<menuContribution locationURI="menu:file?after=save.ext">
    <command commandId="com.teamcenter.rac.importAMRuleTree"
icon="icons/importamruletree_16.png" mnemonic="%importAMRuleTreeAction.MNEMONIC">
        <visibleWhen>
            <and>
                <reference definitionId="com.teamcenter.rac.accessmanager.inMainView"/>
                <with variable="rac_command_suppression">
                    <not>
                        <iterate operator="or">
                            <equals value="com.teamcenter.rac.importAMRuleTree"/>
                        </iterate>
                    </not>
                </with>
            </and>
        </visibleWhen>
    </command>
</menuContribution>

```



```

        </not>
      </with>
    </and>
  </visibleWhen>
</command>
</menuContribution>

```

Command Suppression constraints

Following are constraints on Command Suppression customization:

- The Command Suppression application cannot suppress commands that are dynamic contributions using the **<dynamic>** **</dynamic>** tag in the **menuContribution** section of the **org.eclipse.ui.menus** extension point.
- Any contributions that are done statically in the code (for example, the **Window** menu) cannot be suppressed.
- Any contributions that are done using Eclipse actions cannot be suppressed.

Naming convention for extensions and Command Suppression

Every command contribution in a plug-in has the **visibleWhen** expression containing the **rac_command_suppression** source provider. The naming convention of the extensions in a plug-in should always start with the bundle symbolic name to clearly indicate which plug-in is providing the contribution.

Assume that the **com.mycom.myapp** plug-in has the **com.mycom.myapp** bundle symbolic name in the **META-INF/MANIFEST.MF**. Assume that this plug-in creates the **Sample Perspective** perspective and makes contributions to the global menu bar and toolbar that are visible in the perspective. The perspective ID starts with the bundle symbolic name, for example:

```
com.mycom.myapp.perspectives.samplePerspectiveId
```

Because the command contributions from the **com.mycom.myapp** plug-in should be visible only in the **Sample Perspective** perspective, a reference definition can be defined and associated on all command contributions, for example:

```

<definition id="com.mycom.myapp.inMainPerspective">
  <with variable="activeContexts">
    <iterate operator="or">
      <or>
        <equals value="com.mycom.myapp.perspectives.samplePerspectiveId"/>
      </or>
    </iterate>
  </with>
</definition>

```

Assume that the plug-in contributes a sample command. This means that the plug-in must define a command ID using the **org.eclipse.ui.commands** extension point adhering to the naming convention, for example:

```
com.mycom.myapp.sampleCommand
```

To make this sample command visible only in the **Sample Perspective** perspective, and if it is not suppressed from the **Command Suppression** perspective, use a combination of this reference definition and the command suppression source provider, for example:

```
<command commandId="com.mycom.myapp.sampleCommand" tooltip="%sampleCommand.TIP">
  <visibleWhen>
    <and>
      <reference definitionId="com.mycom.myapp.inMainPerspective"/>
      <with variable="rac_command_suppression">
        <not>
          <iterate operator="or">
            <equals value="com.mycom.myapp.sampleCommand"/>
          </iterate>
        </not>
      </with>
    </and>
  </visibleWhen>
</command>
```

Displaying files in the viewer

On Windows platforms, you can display Microsoft Word, PowerPoint, Excel, Portable Document Format (PDF), and text files in the **Viewer** panel. On UNIX platforms, the properties of the file are displayed instead.

Microsoft Word, Excel, PowerPoint, PDF, and text files are displayed automatically when they are named references of corresponding datasets. No additional configuration required.

To display QAF files in the viewer if Teamcenter lifecycle visualization is not already installed:

1. In the **com.teamcenter.rac.common** plug-in, create an empty **com\teamcenter\rac\common\tcviewer\tcviewer_user.properties** file.
2. In the **com.teamcenter.rac.common** plug-in, open the **tcviewer\tcviewer.properties** file, copy the entire **DatasetViewer.VIEWSEARCHORDER** line, and paste it into the new **tcviewer_user.properties** file.
3. In the **tcviewer_user.properties** file, add **UG-QuickAccess-Binary** to the end of the **DatasetViewer.VIEWSEARCHORDER** line.
4. Add the following line to the **tcviewer_user.properties** file:

```
UGMASTER.VIEWPANEL=com.teamcenter.rac.common.tcviewer.DatasetViewer
```

5. In the **com.teamcenter.rac.util** plug-in, create a **com\teamcenter\rac\util\viewer\viewer_user.properties** file, and add the following line:

```
Imager.EXTENSION=gif,jpg,qaf
```

Note:

To disable the toolbar in the viewer, go to **com/teamcenter/rac/common/tcviewer**, create the **tcviewer_user.properties** file in a text editor, and add the following entry:

```
useNevaIEViewerToolBar=false
```

Customizing tabs

Customizing the data tabs display

If you have administrator privileges, you can customize how data tabs appear in Multi-Structure Manager and other applications. When a user runs one of these applications and selects the data panel, several tabs appear in the panel. You can change the tabs that appear by editing the application properties file.

There are two types of tabs in each application:

- Tabs that always appear for a particular parent panel regardless of whether anything is selected.
- Tabs that appear only when particular components are selected in the parent panel.

You can customize how the second, selection-specific group of tabs is displayed.

To determine the tabs to display, the system checks four criteria:

- The class type of the selected display component, for example:

```
BOMLine
CfgAttachmentLine
TcItemBOPLine
```

- The subtype of the selected display component, which is generally the same as the class type. However, for BOM lines, it is the occurrence type and for attachment lines it is the relation to the parent.
- The class type of the underlying component, such as **ItemRevision**.
- The subtype of the underlying component (the component type name).

For each selection, the system checks for six properties and adds all the tabs found. You can edit these properties to change the tabs that are presented to the user:

```
Display-component-classtype.TABS
Display-component-subtype.TABS
Display-component-classtype.underlying component classtype.TABS
Display-component-classtype.underlying component subtype.TABS
Display-component-subtype.underlying component classtype.TABS
Display-component-subtype.underlying component subtype.TABS
```

For example, in the Multi-Structure Manager application, the default properties are:

```
BOMLine.TABS=Referencers, Variant, Attachments, InClassAtt, CMEViewer, Report,
IncrementalChangeInfo
TcItemBOPLine.TABS=Referencers, Variant, Attachments, InClassAtt, CMEViewer,
Report, IncrementalChangeInfo
AppGroupBOPLine.TABS=Referencers, Attachments, CMEViewer, IncrementalChangeInfo
GDELine.TABS=Referencers, InClassAtt, CMEViewer, Report, IncrementalChangeInfo
GDELinkLine.TABS=Referencers, InClassAtt, CMEViewer, Report,
IncrementalChangeInfo
MEAppearanceLine.TABS=Referencers, Attachments, CMEViewer, IncrementalChangeInfo
CfgAttachmentLine.TABS=Referencers, CMEViewer, IncrementalChangeInfo, Report
BOMLine.ItemRevision.TABS=ProductAppearance
TcItemBOPLine.ItemRevision.TABS=ProductAppearance
CfgAttachmentLine.ItemRevision.TABS=InClassAtt
```

You can add or delete the names of tabs that are displayed for each data panel in this file.

Edit a custom properties file to display tabs

To display tabs, you can place the changes in a **_user.properties** file to override the default **.properties** file.

1. Extract the **application.properties** file from the appropriate **com.teamcenter.rac.component.jar** file using the **jar xf** command. For example, if you want to extract the **mpp.properties** file for Part Planner, the command looks like this:

```
jar xf com.teamcenter.rac.cme.legacy.jar com/teamcenter/rac/cme/mpp/mpp.properties
```

Note:

For more information about the **jar** command, see Sun's Java documentation.

2. Save the extracted file as the **application_user.properties** file.
3. In the custom properties file, edit the **.TABS** line to include the tab you want.
4. Insert the custom properties file into your own custom plug-in.

Sample tab customization

The following example is for creating new tabs for Manufacturing Process Planner.

1. In the Business Modeler IDE, create a new type as a child of the **MEProcess** business object. When the object is deployed to the rich client and displayed in Manufacturing Process Planner, the tabs in the data pane are different than the tabs for the **MEProcess** business object. You must customize the tabs for the new business object so that they match tabs for the parent.
2. Open the *installation-location\portal\plugins\com.teamcenter.rac.cme.legacy* JAR file and find the **mpp.properties** file in the following directory:

com\teamcenter\rac\cme\mpp\mpp.properties

3. Make a copy of the **mpp.properties** file and rename it into a customer-specific *application_user.properties* file.
4. In the custom properties file, create **.TABS** entries for your custom business object. Manufacturing Process Planner accepts the following definitions in the properties files:

```
line-type.TABS= tab-1, tab-2, tab-n
line-subtype.TABS=tab-1, tab-2, tab-n
line-type.underlying-type.TABS= tab-1, tab-2, tab-n
line-type.underlying-subtype.TABS= tab-1, tab-2, tab-n
line-subtype.underlying-type.TABS= tab-1, tab-2, tab-n
line-subtype.underlying-subtype.TABS= tab-1, tab-2, tab-n
```

line-type is the type of the BOM line, for example, **BOMLine**, **ImanItemBOPLine**, or **MfgOBvrProcess**. *line-subtype* is the subtype of a line and it can be an occurrence type or a relation type, for example, **MEConsumed** (in some cases, it is equal the line type). *underlying-type* is the type of the underlying component; a **BOPLine** can have the underlying **MEOperationRevision** type, **MEProcessRevision** type, or other types. *underlying-subtype* is the subtype of the underlying component; like the line subtype, the underlying subtype can also be the same as the underlying type.

If a **BOMLine** type matches more than one definition, the result is the sum of tabs from all matched definitions. For example, an item name **I1** is assigned to an operation as **MEConsumed** type. The following tab lines are defined:

```
ImanItemBOPLine.TABS= Variant
ImanItemBOPLine.ItemRevision.TABS= CMEViewer
BOMLine.TABS= Referencers
BOMLine.ItemRevision.TABS= ProductAppearance
```

Selecting the **I1** item in the process structure (below the operation) matches **ImanItemBOPLine.TABS** and **ImanItemBOPLine.ItemRevision.TABS**, and as a result, the system shows the **Variant** and **CMEViewer** tabs. But selecting **I1** in the BOM structure matches **BOMLine.TABS** and **BOMLine.ItemRevision.TABS**; therefore, the system shows the **References** and **ProductAppearance** tabs.

Note:

The tabs are also defined in the **mpp.properties** file:

```
tab.CLASS
tab.ICON
```

The tab label and tooltip are defined in the **mpp_locale.properties** file:

```
tab.TABLABEL
tab.TOOLTIP
```

5. Insert the custom properties file into your own custom plug-in.

Customizing the rich client to perform additional validations on a file

You can customize the rich client to allow users to perform additional validations on a file when creating a dataset or importing named references. To do this, you must implement the **filesSelector** extension point in the **com.teamcenter.rac.common** plug-in.

You must specify the following attributes in the extension point:

- **class**
Specifies the name of the custom Java class that implements the extension point. This customer-supplied custom class must implement the **IFilesSelector** interface.
This Java-type attribute is required.
- **priority**
Specifies the priority of the extension using a valid integer value. If multiple extensions are available for the **filesSelector** extension point, the rich client refers to this attribute to choose the extension with the highest priority.
This string-type attribute is required.

Following are the methods in the **IFilesSelector** interface:

- **initialize**
Custom code must implement this method to allow a user to interactively select the file by popping up the appropriate dialog box.
Siemens PLM Software recommends that you use the **ImportFilesFileChooser** dialog box defined in the **com.teamcenter.rac.commands.namedreferences** package. This is a standard Teamcenter dialog box that gets information about the selected file from the user. Apart from selecting the file, this dialog box has additional widgets that allow a user to select the file type and reference type.
- **getSelectedFiles**
Custom code must implement this method to return information about the file selected by the user. This method must not perform any GUI interaction with the end user.

This method encapsulates the file object, file type, and reference type information in a **TCFileDescriptor** object and return it to the caller. The **TCFileDescriptor** constructor is designed to perform validation on the file type and reference type, and throw exceptions if the validation fails. Custom code is expected to handle these exceptions.

- **getFormattedText**

Custom code must implement this method to return the path of the selected file. This method should not attempt any GUI interaction with end users.

- **resetSelectedFiles**

Custom code must implement this method to clear the list of selected files.

The following example code demonstrates how a simple validation can be performed using the **filesSelector** extension point. This sample code validates if the file being imported is of length 0 bytes or if the file object selected by the user is a directory:

```
package sample;

import com.teamcenter.rac.commands.namedreferences.ImportFilesFileChooser;
import com.teamcenter.rac.commands.newdataset.IFilesSelector;
import com.teamcenter.rac.commands.newdataset.TCFileDescriptor;
import com.teamcenter.rac.kernel.TCComponentDatasetDefinition;
import com.teamcenter.rac.kernel.TCComponentDatasetDefinition.TCInvalidFileTypeException;
import com.teamcenter.rac.kernel.TCComponentDatasetDefinition.TCInvalidRefTypeException;
import com.teamcenter.rac.kernel.TCException;
import java.io.File;
import javax.swing.JFileChooser;
import java.util.List;
import java.util.ArrayList;
import javax.swing.JOptionPane;

public class SampleFileSelector implements IFilesSelector {

    private List<TCFileDescriptor> listOfFileDesc = null;

    public void initialize(TCComponentDatasetDefinition dsDef,
        String[] extensions, String refType)
    {
        ImportFilesFileChooser fc= new ImportFilesFileChooser ( dsDef, null, extensions,
refType );
        int ret = fc.showDialog ( null, null );
        if ( ret == JFileChooser.APPROVE_OPTION )
        {
            File selectedFile= fc.getSelectedFile();
            String fileType = fc.getType();
            String fileRefType = fc.getReferenceType();
            listOfFileDesc = new ArrayList<TCFileDescriptor>(1);
            try
            {
                TCFileDescriptor currDesc=null;
                //Any additional validations which customer wants to implement
                isSelectedFileValid(selectedFile);
                //Create TCFileDescriptor object
                currDesc = new TCFileDescriptor(dsDef,selectedFile,fileType,fileRefType);
                listOfFileDesc.add(currDesc);
            }
        }
    }
}
```

```

        }
        catch (TCInvalidRefTypeException ex)
        {
            //Do appropriate exception handling
        }
        catch (TCInvalidFileTypeException ex)
        {
            //Do appropriate exception handling
        }
        catch(TCException ex)
        {
            //Do appropriate exception handling
        }
        catch(Exception ex)
        {
            JOptionPane.showMessageDialog(null, ex);
            resetSelectedFiles();
            return;
        }
    }
}

public List<TCFileDescriptor> getSelectedFiles()
{
    return listOfFileDesc;
}

public String getFormattedText()
{
    if ((listOfFileDesc != null) && (listOfFileDesc.size()>0))
    {
        String displayText=listOfFileDesc.get(0).getFile().getPath();
        for (int i=1;i<listOfFileDesc.size();++i)
        {
            displayText = displayText + ", " +
listOfFileDesc.get(i).getFile().getPath();
        }
        return displayText;
    }
    return null;
}

//Additional user-validation methods, We have chosen some simple validation examples
//Customer can similar validation in preferred fashion
public String isFileADirectory(File fileObj)
{
    if ( fileObj.isDirectory())
    {
        return "You have selected directory (" + fileObj.getName() + "),
please select a valid file";
    }
    return null;
}

public String isFileZeroLength(File fileObj)
{
    if ( fileObj.length()==0)
    {
        return "You have selected file (" + fileObj.getName() + " with length 0,

```



```

    please select a valid file";
    }
    return null;
}

public void resetSelectedFiles()
{
    listOfFileDesc = null;
}

public boolean isSelectedFileValid(File fileObj) throws Exception
{
    String validationMessage=null;
    validationMessage = isFileADirectory(fileObj);
    if (validationMessage!=null)
    {
        throw new Exception(validationMessage);
    }
    validationMessage = isFileZeroLength(fileObj);
    if (validationMessage!=null)
    {
        throw new Exception(validationMessage);
    }
    return true;
}
}

```

Creating pre-actions and post-actions in Resource Manager and Classification

Customizing Resource Manager

You can customize Resource Manager using the following packages, found in the **com.teamcenter.rac.cme.legacy** plug-in:

- **Actions**
com.teamcenter.rac.cme.actions
- **Commands**
com.teamcenter.rac.cme.resource
- **Operations**
com.teamcenter.rac.cme.operations
- **Dialogs**
com.teamcenter.rac.cme.dialogs

The classes that belong to the Resource Manager menu bar and toolbar tokens are specified in the **mrm.properties** properties file, located in the **com.teamcenter.rac.cme.mrm** package. To override the settings in the **mrm.properties** file, create the **mrm_user.properties** file and make your changes there.

The class for the action token is specified like this:

```
newMRMAction=com.teamcenter.rac.cme.actions.MRMNewItemAction
```

The action command has its own registry token with the following syntax:

```
action-registry—token.COMMAND=command-registry—token
```

For example:

```
newMRMAction.COMMAND=newMRMCommand
```

The class for the command token is specified as follows:

```
newMRMCommand=com.teamcenter.rac.cme.resource.MRMNewItemCommand
```

The following code shows an example of the Resource Manager action registry entries:

```
# File->New->Resource...
# -----
newMRMAction=com.teamcenter.rac.cme.actions.MRMNewItemAction
newMRMAction.ICON=/com/teamcenter/rac/cme/images/mrmnew_16.png
newMRMAction.COMMAND=newMRMCommand
newMRMCommand=com.teamcenter.rac.cme.resource.MRMNewItemCommand
newMRMDialog=com.teamcenter.rac.cme.dialogs.MRMNewItemDialog
newMRMOperation=com.teamcenter.rac.cme.operations.MRMNewItemOperation
```

Customizing Classification

You can customize Classification using the following packages:

- **Actions**
com.teamcenter.rac.classification.common.actions
- **Commands**
com.teamcenter.rac.classification.common.commands
- **Operations**
com.teamcenter.rac.classification.common.operations

The classes that belong to the Classification toolbar tokens are specified in the **common.properties** properties file, located in the **com.teamcenter.rac.classification.common** package. To override the settings in the **common.properties** file, create the **common_user.properties** file and make your changes there.

The class for the action token is specified as follows:

```
g4mSave.ACTION=com.teamcenter.rac.classification.common.actions.G4MSaveAction
```

The class for the command is specified as follows:

```
g4mSave.COMMAND=com.teamcenter.rac.classification.common.actions.G4MSaveCommand
```

The following code shows an example of the Classification action registry entries:

```
g4mSave.ICON=images/save_16.png
g4mSave.SHOWLABEL=false
g4mSave.ENABLED=false
g4mSave.ACTION=com.teamcenter.rac.classification.common.actions.G4MSaveAction
g4mSave.COMMAND=com.teamcenter.rac.classification.common.actions
                    .G4MSaveCommand
g4mSave.ACTIVE=edit,new
```

Note:

All customizations made in the Classification **common.properties** file are visible in Resource Manager as well. If you want to customize something in Classification only and keep the standard behavior in Resource Manager, you must add the original action/command entries from the Classification **common.properties** file to the Resource Manager **mrm.properties** file.

Develop Java pre-code and post-code

Note:

The action collects only the values from the user interface and passes them into the command. Therefore, do *not* customize the action, instead customize the command.

1. To customize a command, subclass the original command.
2. In the constructor of your new class, call the constructor of the original command.
3. Override the **executeCommand()** method.
With this method you can execute your pre-code. Then call the implementation of the original command with the **super.executeCommand()** method. If you want to execute this part only if some conditions are fulfilled, you can put this method in an **if** statement. After that, you can execute your post-code.

The following code shows an example of a customized command:

```
package com.teamcenter.rac.cme.resource;
import javax.swing.JOptionPane;
import com.teamcenter.rac.aif.AbstractAIFApplication;
public class MySaveCommand extends MRMSaveCommand
{
    public MySaveCommand(AbstractAIFApplication application)
    {
        super (application);
    }
}
```

```

}
protected void executeCommand() throws Exception
{
    int option;

    // Put your Pre-Save-Action code here...
    option = JOptionPane.showConfirmDialog(null,
                                           "This is my Pre-Save-Action!!!",
                                           "MySave",
                                           JOptionPane.OK_OPTION);

    // Execute the original command
    if (allValuesAreOK())
    {
        super.executeCommand();
    }

    // Put your Post-Save-Action code here...
    option = JOptionPane.showConfirmDialog(null,
                                           "This is my Post-Save-Action!!!",
                                           "MySave",
                                           JOptionPane.OK_OPTION);
}
} // End of class MySaveCommand

```

Customizing complex commands

Customizing complex commands for Resource Manager and Classification

Most commands are simple. It is enough to customize before and after the original command using the *Develop Java pre-code and post-code* procedure. For more complex commands, you can customize not only before and after, but also within the original command. To customize within a command, modify the **executeCommand()** method, using the following code:

```

protected void executeCommand() throws Exception
{
    int option;

    // Put your Pre-Save-Action code here...
    option = JOptionPane.showConfirmDialog(null,
                                           "This is my Pre-Save-Action!!!",
                                           "MySave",
                                           JOptionPane.OK_OPTION);

    // Execute the first step of the original command
    super.executeCommandStep1();

    // Put your intermediate customization code here...
    // Execute the second step of the original command
    super.executeCommandStep2();

    // Put your intermediate customization code here...
    // Execute the third step of the original command
    super.executeCommandStep3();

    // Put your Post-Save-Action code here...
    option = JOptionPane.showConfirmDialog(null,

```

```

        "This is my Post-Save-Action!!!",
        "MySave",
        JOptionPane.OK_OPTION);
    }

```

The following are the exact names of the different command step methods for the complex commands:

MRMNewItemCommand
MRMSaveCommand
MRMEditCommand
MRMCancelCommand
MRMDeleteCommand
MRMCreatePFMemberCommand
G4SaveCommand

Resource Manager – Create

MRMNewItemCommand runs the Resource Manager **New Resource** dialog box (based on the **newMRMDialog** registry key). This dialog box allows you to define the item ID, revision ID, item name, description, and item type. Those values are passed into the **MRMNewItemOperation** operation. This creates the item, classifies it, and opens it in the Resource Manager assembly tree.

Symbol:	
Menu:	File→New→Resource
Toolbar:	Create a new resource
Action:	MRMGenericAction (newMRMAction)
Command:	MRMNewItemCommand (newMRMCommand)
Dialog box:	MRMNewItemDialog (newMRMDialog)

Resource Manager – Save

MRMSaveCommand checks if the context is in **EDIT** or **NEW** mode; otherwise, no save is allowed. If the root item revision is classified or if this is an ICO without a workspace object, the Classification **Save Action**, based on the **g4mSave.ACTION** registry key, gets called. If this item revision is not classified, the context is set back to **SHOW** mode.

Afterward, if this resource is an assembly, it checks if the resource has multiple propagation start points defined. In this case a warning is displayed. Then the precision for the BOM is set to **Precise**, and the

Resource Manager BOM **Save Action**, based on the **saveMRMBOMSaveAction** registry key, is called to save the BOM changes. Last, the root and selected labels are updated.

Symbol:	
Menu:	Edit→Save Resource
Toolbar:	Save current resource
Action:	MRMGenericAction (saveMRMAAction)
Command:	MRMSaveCommand (saveMRMCommand)
BOM Save:	MPPsaveAction (saveMRMBOMSaveAction)

Resource Manager – Edit

MRMEditCommand checks if the context is in **EDIT** or **NEW** mode; otherwise, no save is allowed. If the root item revision is classified or if this is an ICO without workspace object, the Classification **Save Action**, based on the **g4mSave.ACTION** registry key, gets called. If this item revision is not classified, the context is set back to **SHOW** mode.

Afterward, if this resource is an assembly, this checks if the resource has multiple propagation start points defined. In this case a warning is displayed. Then the precision for the BOM is set to **Precise**, and the Resource Manager **BOM Save Action**, based on the **saveMRMBOMSaveAction** registry key, is called to save the BOM changes. Last, the root and selected labels are updated.

Symbol:	
Menu:	Edit→Edit Resource
Toolbar:	Edit current resource
Action:	MRMGenericAction (editMRMAAction)
Command:	MRMEditCommand (editMRMCommand)

Resource Manager – Cancel

The **MRMCancelCommand** command creates the Classification **Cancel Action**, based on the **g4mCancel.ACTION** registry key and runs this action on the root, selected, and display contexts of the Resource Manager application.

Symbol:	
Menu:	Edit→Cancel Edit
Toolbar:	Cancel Edit
Action:	MRMGenericAction (cancelMRMAAction)
Command:	MRMCancelCommand (cancelMRMCommand)

Resource Manager – Delete

This method creates the Classification **Delete Action**, based on the **g4mDelete.ACTION** registry key, and runs this action on the root context of the Resource Manager application.

Symbol:	
Menu:	Edit→Delete Resource
Toolbar:	Delete current resource
Action:	MRMGenericAction (deleteMRMAAction)
Command:	MRMDeleteCommand (deleteMRMCommand)

Resource Manager – Create Graphics

The **MRMCreatePFMemberCommand** command creates the Classification **Create Part Family Action**, based on the **g4mcreatePFMember.ACTION** registry key, and runs this action on the root context of the Resource Manager application.

Symbol:	
Menu:	Tools→Create Graphics

Toolbar:	Create Graphics based on Part Family Template or TCL Macro
Action:	MRMGenericAction (createPFMember)
Command:	MRMCreatePFMemberCommand (createPFMemberCommand)

Classification – Save

The **G4MSaveCommand** saves the current instance to the database.

Symbol:	
Menu:	Not applicable.
Toolbar:	Save current instance
Action:	G4MSaveAction (g4mSave)
Command:	G4MSaveCommand (g4mSave.COMMAND)

You can define a pre-hook and/or post-hook for the **G4MSaveCommand** by defining a command for the following property keys:

Pre-hook:	g4mSave.customPreHook.COMMAND
Post-hook:	g4mSave.customPostHook.COMMAND

The **com.teamcenter.rac.classification.common.commands** package contains the **G4MSaveCustomPreHook.java** and **G4MSaveCustomPostHook.java** examples. The custom hook must satisfy the following two requirements:

- The hook must extend from the **AbstractAIFCommand** command.
- The constructor has a **AbstractG4MContext** parameter.

This is an example of a pre-hook:

```
package com.teamcenter.rac.classification.common.commands;

import com.teamcenter.rac.aif.AbstractAIFCommand;
import com.teamcenter.rac.classification.common.AbstractG4MContext;
import com.teamcenter.rac.kernel.TCComponent;
```



```

import com.teamcenter.rac.kernel.TCException;
import com.teamcenter.rac.kernel.ics.ICSAApplicationObject;
import com.teamcenter.rac.kernel.ics.ICSPROPERTY;
import com.teamcenter.rac.util.MessageBox;
import com.teamcenter.rac.util.log.Debug;

public class G4MSaveCustomPreHook extends AbstractAIFCommand
{
    private AbstractG4MContext m_context;

    //=====

    public G4MSaveCustomPreHook( AbstractG4MContext theContext )
    {
        m_context = theContext;
    }

    //=====

    @Override
    public void executeCommand()
    {
        Debug.println( "G4M", "G4MSaveCustomPreHook start ..." );

        try
        {
            // the icsApp contains the Data of the classification instance
            ICSApplicationObject icsApp = m_context.getICSApplicationObject();

            // get the classified object if required
            TCComponent tcComp = m_context.getClassifiedComponent();

            // name of the class the instance is classified in
            System.out.println( "class ID = " + icsApp.getClassId() );

            // to set a value we currently need to get all values
            // replace the value of the attribute we want to change
            ICSPROPERTY properties[];

            properties = icsApp.getProperties();
            for( int i = 0; i < properties.length; i++ )
            {
                if( properties[i].getId() == -2345 && properties[i].getValue() == null )
                {
                    // value is not set
                    properties[i].setValue( "new Value" );
                }
            }

            // set the Values
            icsApp.setProperties( properties );
        }
        catch( TCException iex )
        {
            // Post Message Box if something went wrong
            MessageBox.post(

m_context.getRegistry().getString( "g4mSaveUnexpectedTCException.MSG",

```

```

"unexpected TCEException" ),
    "" + iex,
    m_context.getRegistry().getString( "g4mSaveCustomPreHook.TITLE",
"G4MSaveCustomPreHook" ),
    MessageBox.ERROR );
}
catch( Exception ex )
{
    // Post Message Box if something went wrong
    MessageBox.post(

m_context.getRegistry().getString( "icaXmlExport.UnexpectedException.MSG",
"unexpected Exception" ),
    "" + ex,
m_context.getRegistry().getString( "g4mSaveCustomPreHook.TITLE",
"G4MSaveCustomPreHook" ),
    MessageBox.ERROR );
}

    Debug.println( "G4M", "G4MSaveCustomPreHook done." );
}
}

```

Customize the Launch Pad

The Launch Pad is an application that provides a desktop space to place items and notes for quick reference.

Perform the following steps to create your own Launch Pad configuration:

1. Expose the **Launch Pad** perspective by removing the Launch Pad from the list of hidden perspectives in the **HiddenPerspectives** preference.
2. Open the `TC_ROOTportal\plugins\com.teamcenter.rac.launchpad` JAR file and extract the **l1_launchpad.xml** file. This is the default Launch Pad configuration file. You can use this as the basis for your own Launch Pad configuration file. The file's contents are as follows:

```

<?xml version="1.0" encoding="UTF-8"?>
<launchpad name="Getting Started"
xmlns="http://com.teamcenter.rac.launchpad">

    <launchnode type="URLLaunch" title="Web Links"
title_image="url_16.png" contents_image="links.png">
        <attrib name="url" value="www.google.com/calendar"/>
        <attrib name="url" value="www.npr.org"/>
    </launchnode>

    <launchnode type="MyTasksLaunch" title="My Tasks"
title_image="task_16.png" contents_image="mytasks.png"/>

</launchpad>

```

3. Rename the file (for example, to **MyDefault_Launchpad.xml**) and change it as you like. Also create icons to use for your Launch Pad configuration.
For example, you could create a file containing the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<launchpad name="Default" xmlns="http://
com.teamcenter.rac.launchpad">
    <launchnode type="MyTasksLaunch" title_image="icon.png"/>
    <launchnode type="ScheduleLaunch"/>
    <launchnode type="RequirementsLaunch"/>
    <launchnode type="ApplicationLaunch" title="NX"
title_image="application_16.png"
contents_image="VehicleArchitecture.png">
        <attrib name="target" value="NX"/>
    </launchnode>
</launchpad>
```

Tip:

You can look at the **schema** directory in the *TC_ROOTportal\plugins\com.teamcenter.rac.launchpad* plug-in to see the definitions for file elements you can use in the Launch Pad XML file.

4. Create a Launch Pad Rendering dataset and import your XML file:
 - a. In My Teamcenter, select the folder where you want to place your new dataset and choose **File→New→Dataset**.
 - b. In the **New Dataset** dialog box, choose the **Launch Pad Rendering** type, type the name of the dataset (for example, **MyDefault_Launchpad**) in the **Name** box, select **TextEditor** in the **Tool Used** box, and click the **Import** button.
 - c. In the **Import File** dialog box, select **Fnd0XMLRendering** in the **Reference** box, select your Launch Pad configuration XML file (for example, **MyDefault_Launchpad.xml**), and click **Import**.
 - d. Click **OK**.
The dataset is created.
5. Add icons referenced in the XML file to the named references for the dataset:
 - a. Right-click the new **Launch Pad Rendering** dataset and choose **Named References**.
 - b. In the **Named References** dialog box, click the **Add** button and select the icons referenced in your Launch Pad XML file.
 - c. When you are finished, click **Close**.

6. Now that you have created the **Launch Pad Rendering** dataset, create a **DEFAULT_LAUNCHRENDERING** preference whose value is the Launch Pad Rendering dataset name (for example, **MyDefault_Launchpad**).
Now when you click the **Launch Pad** button in the left navigation bar, your custom Launch Pad configuration is displayed. (You may have to close the Launch Pad and open it again before the new configuration is displayed.)
7. You can pin objects to the Launch Pad such as items, item revisions, folders, and so on, by right-clicking the objects and choosing **Pin to Launch Pad**. Create additional Launch Pad configurations for these objects so that when you double-click the object pinned on the Launch Pad, the configuration for that object type is launched.
 - a. Create an XML configuration file for a specific object type by placing that type into the **launchpad name** node of the XML configuration file. For example, to create a configuration for **Folder** types, use **launchpad name="Folder"**, or to create a configuration for **Item** types, use **launchpad name="Item"**. Then create the Launch Pad dataset, import the XML file, and add the necessary icons to the dataset's named references.
 - b. Create a *business-object-name_LAUNCHRENDERING* preference for that business object type (for example, **Folder_LAUNCHRENDERING** or **Item_LAUNCHRENDERING**).
 - c. When you double-click an that business object type (for example, an item or folder) that is pinned to the Launch Pad, the new configuration is displayed. (To return to the default configuration, click the **Getting Started** button on the Launch Pad toolbar.)

Tips for rich client customization

Localization of rich client customizations

There are two ways to localize the rich client user interface:

- **Properties files**

Siemens PLM Software recommends that you place your custom localizations into a custom plug-in. However, some aspects of style sheets still require that you use the following properties files:

- **messages_locale.properties**

Used by the SWT version of style sheet parsing for the **page** and **section** tags on the **Summary** view, the **Viewer** view, and the **Properties** dialog box.

This file is located in the **com.teamcenter.rac.viewer** plug-in in the **com/teamcenter/rac/viewer/stylesheet/viewer/** package.

- **xmlstylesheet_locale.properties**

Used by the SWT version of style sheet parsing for the **command** and **label** tags on the **Summary** view, the **Viewer** view, and the **Properties** dialog box.

This file is located in the **com.teamcenter.rac.viewer** plug-in in the **com/teamcenter/rac/viewer/stylesheet/xmlstylesheet/** package.

- **stylesheet_locale.properties** (Optional)

Used by the Swing version of style sheet parsing for the **Properties** dialog box. Use of this file is not required for localization, but you can override it in your custom plug-in.

This file is located in the **com.teamcenter.rac.common** plug-in in the **com/teamcenter/rac/stylesheet/** package.

- **Plug-in**

The registry checks the localization plug-in to see if there is a localized property defined in the **plugin.properties** file in the **com.teamcenter.rac.common_version.jar** file. If the string needs to be localized, use % and the key name in the value area to define the key value in the **plugin.properties** file.

For more information, see the following Web sites:

<http://www.eclipse.org/articles/Article-Internationalization/how2I18n.html>

<http://www.eclipse.org/articles/Article-Speak-The-Local-Language/article.html>

Updating your rich client customizations from previous versions

If you customized the rich client in previous versions of Teamcenter or Teamcenter engineering process management, you may have to update your customizations to work with the current version. Some of the significant changes are:

- Customizations should be placed into Eclipse plug-ins if they are not already.
- Some Java API classes, methods, and constructors are deprecated or obsolete and should be replaced. For more information about deprecated and obsolete API, see the *Teamcenter 11.3 Release Bulletin*.
- Some applications have their own plug-in. Open the **TC_ROOT\portal\plugins** directory to see which applications have their own plug-in.

Hide perspectives

1. Log on to Teamcenter as a user that has **dba** permissions.
2. Choose **Edit**→**Options**→**Search** and search for the **HiddenPerspectives** preference.
3. Modify the preference to include the perspective you want to hide.
See the description of the **HiddenPerspectives** preference for more information about perspectives that can be hidden.

Adding a third-party JAR file to your plug-in

Sometimes, you may need to add a third-party JAR file to your plug-in project. To accomplish this, add the JAR file to a project library.

1. To create a folder to hold the JAR file, right-click your Eclipse project, choose **New→Folder**, and create a **lib** folder. This stands for *libraries* and can contain all the third-party JAR files required by the project.
2. To import the JAR file, right-click the **lib** folder, choose **Import→General→File System**, select the directory that contains the JAR file on your system, and select the JAR file to import. The JAR file is imported into the **lib** folder.
3. To add the **lib** folder to the classpath, on the **Runtime** tab, click the **Add** button and select the **lib** folder
4. To ensure the new JAR file is built with the project, click the **Build** tab, and in the **Binary Build** pane, select the check the box next to the **lib** folder.
5. Click the **Save** button.
6. To update the class path, right-click the project and choose **PDE Tools→Update classpath**. The JAR file is added to the **Referenced Libraries** container under the project.
7. To add the JAR to the project Java build path properties, perform the following steps:
 - a. Right-click the project, choose **Properties**, and select **Java Build Path** in the left pane.
 - b. In the **Java Build Path** dialog box, click the **Add Library** button, select **User Library**, click **Next**, select the **lib** folder, and click **Finish**.
 - c. In the **Java Build Path** dialog box, select the **lib** library, click the **Add JARs** button, select the JAR file under the **lib** folder, and click **OK**.

Troubleshooting rich client customization

Common problems in rich client customization

Eclipse startup error

If you get the error shown in the following figure during Eclipse's startup, either the Java Runtime Environment is not installed or the **PATH** statement does not contain the *JDK-installation-directory\bin* directory.



Customizations from a new plug-in do not appear

Teamcenter has a base set of Eclipse plug-ins for the rich client. They are located in the `TC_ROOT\portal\plugins` directory, and the file names start with **com.teamcenter**. When you add a new plug-in, you deploy it into this directory.

If you add or remove a plug-in when customizing the rich client and your changes do not appear, delete the **Teamcenter** subdirectory in the user's home directory on the client. This clears the cache.

On a Windows client, it is typically the `%HOMEDRIVE%%HOMEPATH%\Teamcenter` directory. On a Linux client, it is typically the `$HOME/Teamcenter/` directory.

Unable to load application error

When you start the rich client, you may get the following error:

```
Unable to Load Application: application-name
Details java.lang.reflect.InvocationTargetException
```

This error can occur when you expose custom classes using Eclipse but do not include the class information in the **MANIFEST.MF** file. To correct this error, ensure that you have added your plug-in on the **Runtime** tab in the **Exported Packages** pane in Eclipse. This adds class information to the **MANIFEST.MF** file.

Previous to Teamcenter 2007.2, the classpath alone was sufficient to handle custom class information. In later Teamcenter versions, you must add your plug-in to the **Exported Packages** pane on the project **Runtime** tab to ensure that the **MANIFEST.MF** file is correctly updated.

Rich client debugging

Rich client debugging tools

You can debug your customizations to the rich client by using the following tools:

- **Print Object** view
Displays the selected object's internal attribute values. Use it to determine if there is an incorrect value for an attribute.

- **Communication Monitor** view
Shows you the calls between the rich client and server. Use it to determine if your data is being correctly exchanged between the server and client.
- **Performance Monitor** tool
Tracks the calls between the server and the database. Use it to determine how fast the server and the database are communicating.
- **DB Walker** view
Examines the database. This view is for Siemens PLM Software internal use only.

There are also standard Eclipse views that can help you debug your customization.

Debug using the Print Object view

1. In the rich client, choose **Window**→**Show View**→**Print Object**.
2. Select an object, such as an item or dataset. You can also copy an object's tag from a syslog and paste it in the **UID** box.
The object's attributes and their values appear in the **Print Object** pane at the bottom of the rich client.

Note:

You cannot edit the values in the **Print Object** pane.

3. (Optional) Limit what is displayed in the pane by changing **All attrs** in the list to one of the following:
 - **Refs only**
Shows only those attributes whose value is a tag.
 - **Strings only**
Shows only those attributes whose value is a string.
 - **Refs & Strings**
Shows only those attributes whose value is a tag or a string.
4. (Optional) Expand what is displayed in the pane by changing **Hide OM attrs** in the list to **Show OM attrs**.
This shows the object manager attributes in addition to the attributes already displayed.
5. If you want to save the attributes and their values to a file, click the **SAVE** button in the pane.
6. To see attributes of another object, leave the pane open and select the other object. If you want to see attributes of an object you selected earlier, select it from the list at the top of the pane.

Debug using the Communication Monitor view

1. In the rich client, choose **Window→Show View→Communication Monitor**.
The values appear in the **Communication Monitor** pane at the bottom of the rich client.
2. To choose what you want to see in the monitor, click the **Menu** button  in the **Communication Monitor** pane and choose one or more of the following:
 - **Show Server Calls**
Displays an entry for each call to the server.
 - **Show Stack Trace**
Displays the call stack trace for each call to the server.
 - **Show Request**
Displays the XML request sent to the server.
 - **Show Response**
Displays the XML response returned by the server.
 - **Show Timing**
Displays the length of the server call in seconds.
3. If you want to clear the data, click the **Clear** button  in the pane.
4. If you want to save the data to a file, click the **Save As** button  in the pane.

Debug using the Performance Monitor tool

1. Set the **TC_PERFORMANCE_MONITOR** environment variable to **0**.
The tool appears the next time you open the rich client.
2. To see the data from the server and client, click the **Report** button. The data is also logged along with the text in the **Log comment** box. It also resets the counters.
 - The SQL and server CPU statistics are retrieved from the server.
 - **Wallclock time since reset** is the time since the last reset.
Times are shown in milliseconds. If the top of the Performance Monitor states that the Hi-Res timer is in use, times are accurate to 1 millisecond. Otherwise the standard operating system clock is in use; Microsoft Windows uses a 60 Hertz clock, so times on Windows are accurate to about 16 milliseconds.
 - **Client CPU time** is available only with the Hi-Res timer.

- **Server calls made** is a count of all calls made to the server, not including the call to get the SQL statistics.

Note:

If you select the **Reset on first server call** check box, the Performance Monitor is reset after the next server call after you click the **Report** button.

3. To clear the data and reset all counters, click the **Reset** button.

Note:

If you select the **Reset on first server call** check box, the Performance Monitor is reset after the next server call after you click the **Reset** button.

4. To save the data to a file, click the **Save** button.

Debug using Eclipse views

Some Eclipse views may be helpful when debugging your customizations:

- **Console view**
Shows the standard output, standard error, and standard input for your program. Used to debug problems with customizations and to monitor rich client activity.
For more information, see the following URL in the Eclipse help:
http://help.eclipse.org/indigo/index.jsp?topic=/org.eclipse.jdt.doc.user/reference/views/console/ref-console_view.htm
- **Progress view**
Shows the progress of background jobs. You can connect this view to your customizations if you want to see the progress when your customizations run.
For more information, see the following URL in the Eclipse help:

http://help.eclipse.org/indigo/index.jsp?topic=/org.eclipse.platform.doc.isv/guide/runtime_jobs_progress.htm

Enabling client-side logging

Changing the logging level and location

Teamcenter uses the **log4j** mechanism for logging. You can change logging parameters in the **TcLogger.properties** file, located in the *TC_ROOT\portal\plugins\configuration_version* directory, to specify the level and scope of logging, as well as changing the log file location.

You can change the logging level for the entire rich client, a package, or a single class, as well as the log location:

- Rich client debug-level logging

To enable debug-level logging for the entire rich client, change the **log4j.rootLogger** line in the **TcLogger.properties** file to **DEBUG**. The line should look like this:

```
log4j.rootLogger=DEBUG, TcLoggerConsoleAppender, TcLoggerFileAppender,
TcLogContext
```

- Package debug-level logging

To enable debug-level logging for a package, add a line in the **TcLogger.properties** file for it. For example, if you want to enable debug-level logging for Structure Manager, the line looks like this:

```
log4j.logger.com.teamcenter.rac.pse=DEBUG
```

- Single class debug-level logging

To enable debug-level logging for a single class, add a line in the **TcLogger.properties** file for it. For example, if you want to enable logging for the **TCSession** class, the line looks like this:

```
log4j.logger.com.teamcenter.rac.kernel.TCSession=DEBUG
```

- Log location

By default, the rich client log is the *operating-system-user-name_TcRAC.log* file in your operating system's temporary directory. You can change the location by changing the **log4j.appender.TcLoggerFileAppender.file** entry in the **TcLogger.properties** file. This is the default location:

```
log4j.appender.TcLoggerFileAppender.file=${osgi.instance.area}/
${user.name}_TcRAC_${timestamp}.log
```

Adding appenders

You can easily add or remove an appender to a logger. The content of all appender output is identical unless you add a filter to the content. Each appender supports a pattern layout that determines the format of the output. By default, the rich client has two kinds of appenders:

- A console appender which outputs to the console. To see console output in the rich client outside of Eclipse, you must use the **-consolelog** flag on the command line when you run the rich client.
- A file appender which outputs to a file.

Pattern layouts

Use a pattern layout to include more information in the console or log file. Each appender has a pattern layout, which is a substitution string for the output.

- **c**

Display the logger (category) name.

- **C**

Display the fully qualified class name of the caller issuing the logging request.

- **d**

Display the date of the logging event.

```
%d{HH:mm:ss,SSS}
```

- **F**

Display the file name where the logging request was issued. This slows execution.

- **l**

Display the location information of the caller that generated the logging event. This slows execution.

- **L**

Display the line number from where the logging request was issued. This slows execution.

- **m**

Display the message.

- **M**

Display the method name where the logging request was issued. This slows execution.

- **n**

Insert a new line.

- **p**

Display the priority of the logging event (**DEBUG**, **WARN**, **INFO**).

- **r**

Display the number of milliseconds elapsed since the start of the application until the creation of the logging event.

- **t**

Display the name of the thread that generated the logging event.

- **x**

Display the nested diagnostic context (NDC) associated with the thread that generated the logging event.

For example:

- **%-5p: %m%n**

This layout produces the following log message:

```
ERROR: There is something wrong here!
```

- **%n%-5p: %d{HH:mm:ss,SSS} %x - %C{1}.%M:%L%n%m%n**

This layout produces the following log message:

```
INFO : 11:51:54,871 - Class.method:90
Session logging via TcLoggerFileAppender into file
C:\Windows\temp\smithj_TcRAC.log
```

Add logging to your code

1. Import the Apache logger:

```
import org.apache.log4j.Logger;
```

2. Add a static logger to the file at the top of the class:

```
private final static Logger logger = Logger.getLogger( MyClass.class );
```

3. Specify the logger output you want:

- **Error condition**

```
logger.error( String [,Throwable] );
```

- **Warning condition**

```
logger.warn( String [,Throwable] );
```

- **Information condition**

```
logger.info( String [,Throwable] );
```

- **Debug condition**

```
logger.debug( String [,Throwable] );
```

4. (Optional) Add debug control flags to the **TcLogContext** appender defined in the **TcLogger.properties** file:

- a. Add the boolean flag to the **TcLogger.properties** file.
- b. Add the getter and setting for the flag to the **com.teamcenter.rac.util.log.TcLogContext** line.
- c. Use the getter to control your debug code:

```
if( TcLogger.getLogContext().getMyFlag() ) doSomething();
```

Listener leaks

Events in Java are fired by means of listeners. An object registers interest with a target object, so that when an event occurs the listening object is notified. For this relationship to be maintained, the target object must maintain a reference to the listening object. The Java memory management facilities look only to delete objects from virtual memory when the objects are no longer referenced by any other Java object. The problem at hand is the removal of listeners.

Rich client does not currently have a system in place to facilitate the removal of listeners. This creates two issues:

- It causes a memory leak.
The memory leak issue exists because references are maintained to Java objects that are no longer needed. This creates the situation in which the garbage collector runs but is unable to remove the old objects because they are still tied as listeners. Under this condition, the virtual memory of the Java VM eventually runs out.
- It begins to impact performance of the UI, because old components are being needlessly updated.
The performance issue is more prevalent than the memory leak. System performance begins to deteriorate quickly under certain UI conditions. The use of the viewer illustrates this, because as new viewers are displayed, they add their components to the session, attached as listeners. The UI appears sluggish and eventually becomes unusable.

The **InterfaceSignalOnClose** and **SignalOnClose** classes are used to remedy listener leaks:

- **InterfaceSignalOnClose**

The **InterfaceSignalOnClose** class requires the implementation of the **closeSignaled()** method. This interface is designed to signify the desire to be notified when closure is to commence. The **closeSignaled()** method is designed to remove any listeners that were created during the life of the object.

This interface signifies that the implementing class registers interest to be known when closure occurs. The implementing class is required to implement the **closeSignaled()** method. The **closeSignaled()** method is invoked when closure is commencing (as shown in the following code):

```
public interface InterfaceSignalOnClose
{
    public void closeSignaled();
}
```

Example: An implementation of the **closeSignaled()** method can look like the following:

```

public void closeSignaled()
{
    TCSession session = (TCSession) application.getSession();
    if (session != null)
    {
        session.removeAIFComponentEventListener( this );
    }
}

```

- **SignalOnClose**

The **SignalOnClose** class is designed to signal the processing of the components to detach themselves from listeners and prepare to be closed. This class contains a single method, **close()**, which is designed to be passed a reference to a **Container** object. The **Container** object is the start of a recursive walk down the component tree to look for instances of **InterfaceSignalOnClose** classes. If instances are found, the classes are notified that closure is commencing. At this point, it is the responsibility of the implementing class to take appropriate action.

Rich client customization reference

Teamcenter extension points

To see the available extension points, open a project in Eclipse and click the **Add** button in the project's **Extensions** tab.

Some of the documented **rich client customization examples** show detailed use of Teamcenter extension points.

The following Teamcenter extension points are available for your use.

Extension point	Description
Application	Adds an application to the rich client. All new applications are plugged in from this extension point.
ApplicationTaskPane	Adds an application task pane to an application. Each application task pane can be comprised of one or more application task pane sections. An application task pane is associated with: • ID Unique ID of the application task pane. • title Application task pane title. • class Application task pane implementation class.

Extension point	Description
ApplicationTaskPaneSection	• ApplicationTaskPaneSectionID Sequence of section component IDs that contribute to the application task pane. Each of the ApplicationTaskPaneSectionID defined should correspond to the ID attribute on the extensions defined for the ApplicationTaskPaneSection extension point. Adds an application task pane section to an application task pane. Each ApplicationTaskPaneSection can be composed of zero or more section components. An application task pane section is associated with: • ID Unique ID of the application task pane section. • title Application task pane section title. • iconBundleName Location of the icons if they are located in a different bundle. • icon Application task pane section icon. • class Application task pane section implementation class. • secondaryTitle Application task pane section secondary title. • secondaryActionClass Secondary action implementation class.
SectionComponent	Adds a section component to an application task pane section. A section component is associated with: • ID Unique ID of the section component.

Extension point	Description
	• title Section component title. • iconBundleName Location of the icons if they are located in a different bundle. • icon Section component icon. • class Section component implementation class. • secondaryTitle Section component secondary title. • secondaryActionClass Secondary action implementation class. • SectionComponentID Sequence of section component IDs that contribute to the application task pane section. Each of the SectionComponentID defined should correspond to the ID attribute on the extensions defined for the SectionComponent extension point.
boTypesLoader	Contributes extensions used to load types. The types loader class decides which base type and its subtypes to load, the types to be excluded, and if the most recently used list (MRU) is enabled.
boTypesPage	Contributes extensions that add custom panels to the types selection page in New Other dialog box.
buttonProvider	Registers the button providers with the Clipboard composite area.
extWizard	Registers wizards.
extWizardPage	Registers pages with the specified wizard.
extWizardRef	Registers pages with the specified wizard.
filesSelector	Defines a custom implementation for the file selection dialog box displayed when the user creates a dataset or imports files to an existing dataset and performs any additional validations on the imported files.

Extension point	Description
health	Registers the IHealthCheckControl Factories for the health check status bar area.
Kernel_Components	Controls the mapping of the server side schema type definitions to the client. For an example of implementation, see the plugin.xml file in the com.teamcenter.rac.kernel plug-in JAR file. For example: <pre> <extension-point id="Kernel_Components" name="components" schema="schema/components.exsd"/> <extension id="com.teamcenter.rac.kernel" point="com.teamcenter.rac.kernel.Kernel_Components"> <entry id="ACTLine" name="com.teamcenter.rac.kernel.TCComponentACTLine" type="com.teamcenter.rac.kernel.TCComponentBOMLineType" icct="com.teamcenter.tc.kernel.icctstubs.ICCTBOMLine" /> ... </pre>
Kernel_Services	Defines the ICCT-based services used by the kernel. ICCT is deprecated so you should not use this extension. Use service-oriented architecture (SOA) for client-server communication instead of ICCT. For an example of implementation, see the plugin.xml file in the com.teamcenter.rac.kernel plug-in JAR file. For example: <pre> <extension-point id="Kernel_Services" name="services" schema="schema/services.exsd" /> <extension id="com.teamcenter.rac.kernel" point="com.teamcenter.rac.kernel.Kernel_Services"> <entry id="BMF_SERVICE" name="com.teamcenter.rac.kernel.BMFService" /> ... </pre>
navigatorRoots	Adds root node objects to the Navigator view.
openConfiguration	Contributes extensions used when the File→Open menu is chosen. • typeName Teamcenter type name • perspectiveId

Extension point	Description
	Perspective ID that should be the same as the ID in the org.eclipse.ui.perspectives extension point or in the perspective_id attribute of the com.teamcenter.rac.aifrcp.application extension point. • applicationId The id attribute on the aif_app_item element under the com.teamcenter.rac.aifrcp.application extension point.
openWithConfiguration	Matches editors with Teamcenter types in the Open With menu. • typeName Teamcenter type name. • editorId Editor ID that is registered using the org.eclipse.ui.editors extension point. • perspectiveId If specified, the perspective is posted prior to opening the editor in the perspective. If the specified perspective does not have an editor area visible, then a default perspective (metadata-editing perspective) is posted prior to opening the editor.
operation	Contributes extensions that are used to perform an operation from a wizard or dialog box.
ProjectSections	Supports associating sections within a view.
tcOpenConfiguration	Allows the extenders to specify a perspective ID to open with in conjunction with the Eclipse core expressions. When the core expression evaluates to true , the perspective ID registered against it is used to open the selected object.
tc_properties	Defines the entry point where the customer can plug in their override properties file.

Command line options for rich client startup

You can specify the following command line options during rich client startup.

Note:

- You can specify these same command line options in Eclipse when you test your customization. Choose **Run→Debug Configurations** or **Run→Run Configurations**, select the application in the left pane that you want to test, click the **Arguments** tab, and enter the command line options in the **Program arguments** box.
- You can also use some of these command line arguments (such as **-attach**) when constructing a URL to launch the rich client in a four-tier environment. To obtain a URL in the four-tier rich client, right-click an object such as an item and choose **Copy**, and then paste the resulting URL into the address bar of a Web browser. This allows you to launch the rich client and automatically open the copied object. For example:

```
http://svi6w101:7001/tc/launchapp?-attach=true&-s=226TCSession&
-o=QNG11_93oEfenBAAAAAAAAAAAAA
```

In this example, **launchapp?** launches the rich client, and **-attach** launches within an already-running rich client session if one is available. (While not rich client command line options, the **-s** argument designates the session, and the **-o** argument designates the object to open.)

If you have multiple rich client installations on your system, the constructed URL only opens the most recently installed rich client because the registry is updated at each installation.

- **-arch architecture**

Defines the processor architecture on which the Eclipse platform is running. The Eclipse platform ordinarily computes the optimal setting using the prevailing value of Java **os.arch** property. If specified here, this is the value that the Eclipse platform uses. The value specified here is available to plug-ins as **BootLoader.getOSArch()**. Example values: **x86**, **sparc**, **PA-RISC**, **ppc**.

- **-attach**

Attaches the new client to an existing session.

For example, a user launches CATIA from the rich client and works on a structure. The same structure is also loaded in Structure Manager. Selecting a part in CATIA, the user wants to synchronize the same selections in Structure Manager. This is where the **-attach** argument is useful to indicate that the synchronization of the selections should happen in an existing rich client session.

- **-application applicationId**

Specifies the application to run. Applications are declared by plug-ins supplying extensions to the **org.eclipse.core.runtime.applications** extension point. This argument is typically not required. If specified, the value overrides the value supplied by the configuration. If not specified, the Eclipse Workbench is run. For example, to launch My Teamcenter, use the following command:

```
portal.bat
-application=com.teamcenter.rac.ui.perspectives.navigatorPerspective
```

- **-clean**

Cleans cached data used by the OSGi framework and Eclipse run time. This is useful if you have new plug-ins you have added to your environment. Try to run Eclipse once with this argument if you observe startup errors after installation, update, or using a shared configuration.

Note:

When you start the rich client, if your customization changes still do not appear in the user interface after running the **-clean** argument, delete the Teamcenter subdirectory in the user's home directory on the client. This directory is automatically created again when the user starts the rich client.

On a Windows client, it is typically the **%HOMEDRIVE%%HOMEPATH%\Teamcenter** directory. On a Linux client, it is typically the **\$HOME/Teamcenter/** directory.

If you delete this directory, the last state of the rich client is lost, and the user interface appears as it does at initial startup.

- **-configuration** *configurationFileURL*
The location for the Eclipse platform configuration file, expressed as a URL. The configuration file determines the location of the Eclipse platform, the set of available plug-ins, and the primary feature. Note that relative URLs are not allowed. The configuration file is written to this location when the Eclipse platform is installed or updated.
- **-consolelog**
Mirrors the Eclipse platform's error log to the console used to run Eclipse. It is effective when combined with **-debug**.
- **-data** *workspacePath*
The path of the workspace on which to run the Eclipse platform. The workspace location is also the default location for projects. Relative paths are interpreted relative to the directory that Eclipse was started from.
- **-detach**
Detaches the client from an existing session.
Starts clients as separate sessions. This is the default behavior, even if the **-detach** option is not specified.
- **-debug** [*optionsFile*]
Puts the platform in debug mode and loads the debug options from the file at the given location, if specified. This file indicates which debug points are available for a plug-in and whether or not they are enabled. If a file location is not given, the platform looks in the directory that eclipse was started from for a file called **.options**. Both URLs and file system paths are allowed as file locations.
- **-dev** [*classpathEntries*]
Puts the platform in development mode. The optional classpath entries (a comma separated list) are added to the run-time classpath of each plug-in. For example, when the workspace contains plug-ins being developed, specifying **-dev bin** adds a classpath entry for each plug-in project's directory named **bin**, allowing freshly generated class files to be found there. Redundant or nonexistent classpath entries are eliminated.

- **-DskipRegReload** *args*

If you are starting the rich client inside the Eclipse IDE, this option prevents the registry database from loading if it already exists, which can save up to a minute or more during startup depending on your system. You can add this argument to your run/debug configuration. However, if you use this argument, any changes you make to the registry property files are not used. If you are making changes to the registry database, do not use this argument.

- **-initialize**

Initializes the configuration being run. All run-time related data structures and caches are refreshed. This is useful with shared installs; running Eclipse once with this option from an account with write privileges improves startup performance.

- **-nl** *locale*

Defines the name of the locale on which the Eclipse platform is running. The Eclipse platform ordinarily computes the optimal setting automatically. If specified here, this is the value that the Eclipse platform uses. The value specified here is available to plug-ins as **BootLoader.getNL()**. For example, you can use the following: **en_US** or **fr_FR_EURO**.

- **-nosplash**

Runs the platform without putting up the splash screen.

- **-os** *operatingSystem*

Defines the operating system on which the Eclipse platform is running. The Eclipse platform ordinarily computes the optimal setting using the prevailing value of Java **os.name** property. If specified here, this is the value that the Eclipse platform uses. The value specified here is available to plug-ins as **BootLoader.getOS()** and used to resolve occurrences of the **\$os\$** variable in paths mentioned in the plug-in manifest file. For example, you can use one of the following: **win32**, **linux**, **hpux**, **solaris**, **aix**.

- **-perspective** *perspectiveId*

The perspective to open in the active workbench window on startup. If this parameter is not specified, the perspective that was active on shutdown will be opened.

- **-plugincustomization** *propertiesFile*

The location of a properties file containing default settings for plug-in preferences. These default settings override default settings specified in the primary feature. Relative paths are interpreted relative to the directory that Eclipse was started from.

- **-product** *productId*

The ID of the product to run. The product gives the launched instance of Eclipse its personality, and determines the product customization information used. This replaces **-feature**, which is still supported for compatibility.

- **-refresh**

Option for performing a global refresh of the workspace on startup. This reconciles any changes that were made in the file system since the platform was last run.

- **-showlocation** [*workspaceName*]

Option for displaying the location of the workspace in the window title bar. The optional workspace name argument displays the provided name in the window title bar instead of the location of the workspace.

- **-vm *vmPath***
The location of Java Runtime Environment (JRE) to use to run the Eclipse platform. If not specified, the JRE is at **jre**, sibling of the Eclipse executable. Relative paths are interpreted relative to the directory that Eclipse was started from.
- **-vmargs *args***
When passed to Eclipse, this option customizes the operation of the Java Virtual Machine (VM) used to run Eclipse. If specified, this option must come at the end of the command line. The given arguments are dependant on the VM that is being run.

Coding standards

File organization

The following file and directory structure standards should be used when developing the rich client customization code:

- All package names must be lowercase. Do not use space characters in package names.
- The general package registry should have the same name as the last package name. For example, for the **com.mycompany.rac.explorer** package, the file for the **ResourceBundle** object that contains the registry information is **explorer.properties**.
- Image files associated with a particular package must be located within the images directory below the package.
- Image file names must consist of all lowercase letters.

Naming conventions

The following table describes the recommended naming convention for the various Java types.

Java type	Rule	Example	Comment
Interface	Interface [<i>name</i>], I [<i>name</i>]	InterfaceAIFOperationListener, ISelectionService	Teamcenter standard
Abstract class	Abstract [<i>name</i>]	AbstactAIFApplication	Teamcenter/Java standard

Java type	Rule	Example	Comment
Exception class	[name]Exception	SomethingHappenedException	Java standard
Variable naming	Lowercase first word, uppercase first letter of other words.	factoryName	Java standard
Accessor methods	Getting: Use getXXX , except for Booleans, where XXX is allowed. Setting: use setXXX .	getFactoryName(), setFactoryName(), isReady()	Java standard
Source files	Same as Class name (including case).	ISelectionService.java	Java requirement
Class names	Uppercase first character of each word.	ComponentManager	Java standard

Property conventions

Property files can be categorized in three ways:

- Core development
- Localization
- User properties

The reason for this distinction is that customers modify the user property files while maintaining the links to the core development property files. The following table describes the properties files using **explorer** as an example.

File	Description
explorer.properties	Base property file
explorer_locale.properties	Property file for the purpose of localization
explorer_user.properties	Customer created property file

Source code conventions

The source conventions follow the Sun Java source code standards that match the industry norms for Java development.

Dialog box standards

The following standards should be used when customizing the rich client dialog boxes:

- Dialog boxes must always be modal unless the situation requires that they be nonmodal, such as when the user selects additional information when the dialog box is visible.
- Mnemonics should be used for common dialog box buttons, such as **OK**, **Apply**, and **Cancel**.
- The initial location of the dialog box must be screen centered, and the sizing of the dialog box must be adjusted with a sizing factor.
- Dialog boxes should include a default focus when displayed.
- **Text field/area policies:**
 - Allow only the maximum number of characters that the property can accept.
 - Send an audible beep when the maximum is reached.
 - Set all text area components to the initial size of 3 rows by 30 columns.
 - Select the text when the focus is gained inside the text field.
 - Set word wrapping to **true** for text area components.
- **Color policies**

Whenever possible, use the default color provided by the base component. Allow the current look and feel to determine the color.

 - If it is not possible to use the default color, use the **SystemColor** class.
 - If neither the default color nor the **SystemColor** class suffice, define the color in the property files so users can change it.
- **Font policies**

Whenever possible, use the default font provided by the base component. If the default font is not sufficient, try one of the following options:

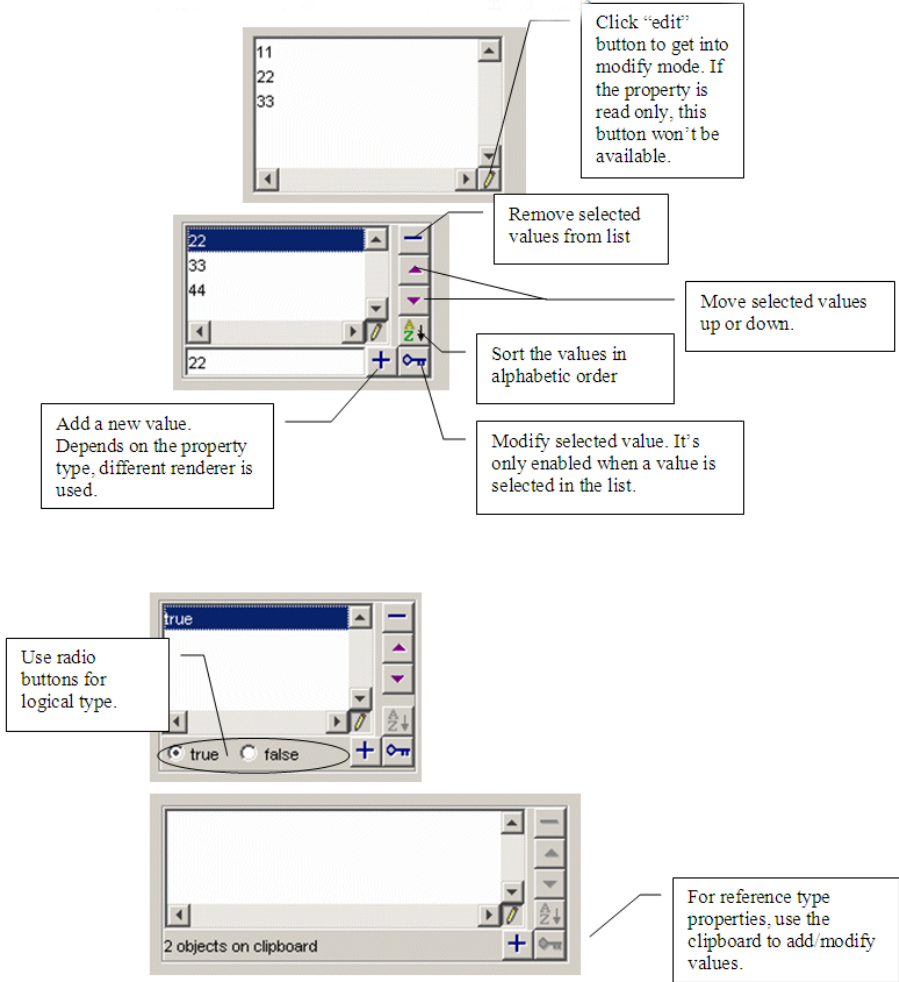
 1. Offset the font based on the size of the current font. Do not hard code the font name, because the font may not be available on all platforms.
 2. Mention the font in the property file.

Property beans

The following table lists JavaBeans that you can use to customize the properties display.

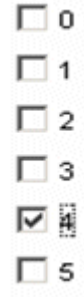
Note:

To display a red asterisk in the upper-right corner of a UI widget to indicate a mandatory property, use the Business Modeler IDE to set the **Required** property constant to **true**.

JavaBean	Description
PropertyArray	Renders all array type properties. It is composed with a list box and buttons to access the values in the list box (as shown in the following figure). Based on the type of the property, a different renderer is used for modifying or adding values. If the property is read only, users cannot enter edit mode.  The diagram shows three examples of the PropertyArray bean UI: Integer Array: A list box containing values 11, 22, and 33. Callouts indicate: "Click 'edit' button to get into modify mode. If the property is read only, this button won't be available." "Remove selected values from list" "Move selected values up or down." "Sort the values in alphabetic order" Logical Type: A list box containing the value "true". Below it are radio buttons for "true" and "false". A callout states: "Use radio buttons for logical type." Reference Type: A list box containing the text "2 objects on clipboard". A callout states: "For reference type properties, use the clipboard to add/modify values." Additional callouts for the array UI include: "Add a new value. Depends on the property type, different renderer is used." "Modify selected value. It's only enabled when a value is selected in the list."

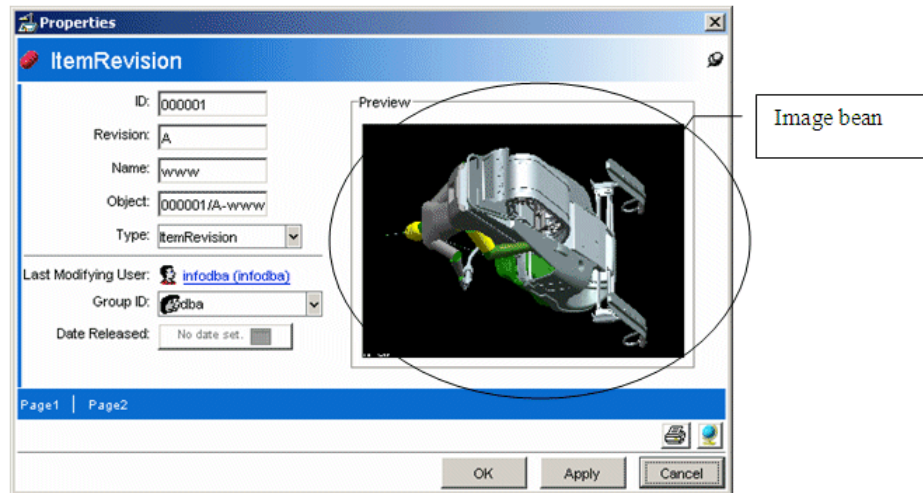
PropertyArray

JavaBean	Description
PropertyCheckbox	property
	The property name presented by the bean.
	modifiable
PropertyCheckbox	Indicates whether the property is modifiable. If not, the edit button is not available and the user cannot enter edit mode.
	Renders any nonarray type property, except reference/relation type, using JCheckBox. A flag indicates whether the value is saved only when the check box is selected or whether it is always saved. There is a variable for the value to use on saving for a selected button or for the deselected button. If the flag is set to save regardless of whether the button is selected or deselected, both the selected value to save and the deselected value to save must be set. The following figure illustrates an implementation of the PropertyCheckbox bean.
	
PropertyCheckbox	PropertyCheckbox
	Properties:
	property
PropertyCheckbox	The property name presented by this bean. When a component is provided, the check box is selected if the property value is same as the selected value.
	modifiable
	Indicates if the property is modifiable. If not, the check box is disabled.
PropertyCheckbox	selectedValue
	Specifies the value used for saving when the check box is selected.
	deselectedValue
PropertyCheckbox	Specifies the value to use for saving when the check box is deselected.
	saveOnlyWhenSelected
	Indicates to save only when check box is selected or to always save.

JavaBean	Description
PropertyCheckboxOptionLov	Presents each value in the LOV as a check box (as shown in the following figure). This bean is designed for any type property that has a LOV attached. If the property is not an array, the check boxes are added to a button group; otherwise, they are not added to the button group and multiple selections are allowed.  PropertyCheckboxOptionLov Properties: property The property name presented by the bean. modifiable Indicates whether the property is modifiable. If not, the check boxes are disabled. lovName Specifies the name of the LOV that the bean will use. If undefined, the LOV information is retrieved from the property descriptor.
PropertyImage	This bean is limited to items, item revision, datasets, and BOM view revisions. It walks down these objects to find a dataset with a displayable image and displays that image. It uses the same logic as Engineering Process Management Visualization to find the image dataset and read the search order defined in the tcviewer.properties file. For example, the following figure shows an image attached to the selected item revision.

JavaBean

Description



PropertyImage

PropertyLabel

Renders any nonarray type property and displays the value of the property as the label text. Users cannot change the label text; therefore, save does not apply to this bean.

Properties:

property

The property name presented by the bean. When a component is provided, the property value is displayed as the label text.

PropertyLogicalPanel

This bean is used to present a logical type property. It uses two buttons, one for value **true** and another for **false**.

property

The property name presented by the bean.

modifiable

Indicates whether the property is modifiable. If not, the buttons are disabled.

Note:

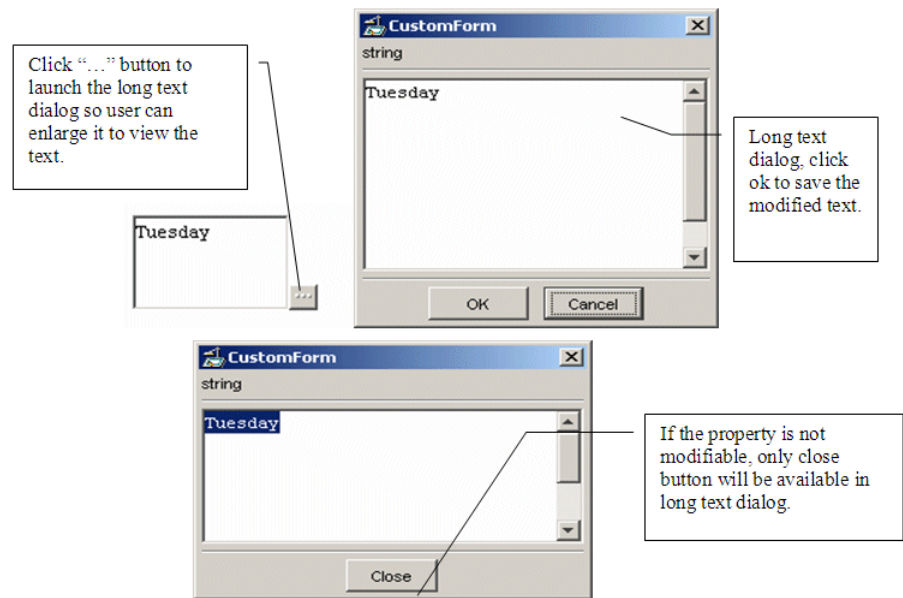
For Boolean properties, if neither true or false are selected, the value of the property is NULL.

PropertyLongText

This bean can be used with any string or note type properties, but is generally used to render text with lengths over 2500 characters. It contains a text area and a button. The long text dialog box is displayed by clicking the button, making it easier to browse the text (as shown in the following figure).

JavaBean

Description

**PropertyLongText**

Properties:

property

The property name presented by the bean.

modifiable

Indicates whether the property is modifiable. If not, the text area cannot be edited and only the close button is available in the long text dialog box.

PropertyLOVButton

Caution:

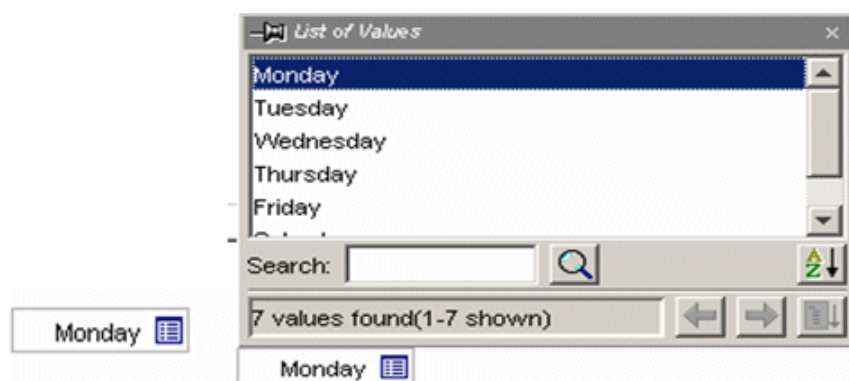
As of Teamcenter 10.1, this bean is deprecated and is replaced by the **PropertyLOVDisplayer** bean.

Used for any property that has an LOV attached to it except logical type.

It uses **LOVPopupButton** to present this property (as shown in the following figure).

JavaBean

Description



LOVPopupButton

Properties:

property

The property name presented by the bean. When a component is provided, the button text is set to the property value.

modifiable

Indicates whether the property is modifiable. If not, the button is disabled.

lovName

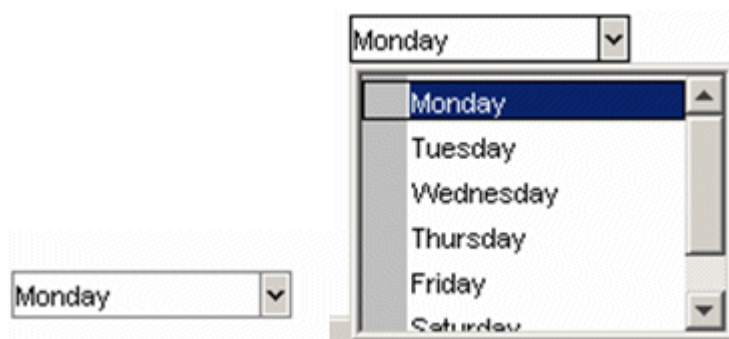
Specifies the name of the LOV that the bean uses. If undefined, the LOV information is retrieved from the property descriptor.

**PropertyLOV
Combobox**

Caution:

As of Teamcenter 10.1, this bean is deprecated and is replaced by the **PropertyLOVDisplayer** bean.

This bean is similar to the **PropertyLOVPopupButton** bean; however, it uses **LOVComboBox** rather than **LOVPopupButton** to present the property (as shown in the following figure).



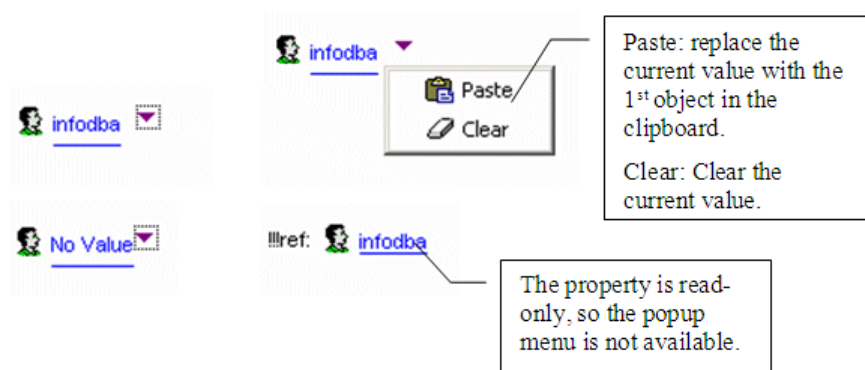
PropertyLOVPopupButton

Properties:

JavaBean	Description
	property The property name presented by the bean. modifiable Indicates whether the property is modifiable. If not, the combo box is disabled. lovName Specifies the name of the LOV that the bean will use. If undefined, the LOV information is retrieved from the property descriptor.
PropertyNameLabel	Renders the name of a property (as shown in the following figure). By specifying the name property, it shows either the displayable name or real name of the property according to the setting. This bean can be used along with other beans that display the property value. PropertyNameLabel Properties: property Specifies the property name presented by the bean. displayName Indicates whether the displayable name or real name is used. colon Indicates whether or not a colon is displayed after the name.
PropertyObjectLink	This bean renders reference type properties. A shortcut menu is provided for users to modify the value (as shown in the following figure). If the value is not modifiable, the shortcut menu is not available. When the link is clicked, the system displays a dialog box with properties of this reference component.

JavaBean

Description



PropertyObjectLink

Properties:

property

The property name presented by the bean.

modifiable

Indicates whether the property is modifiable. If not, the shortcut menu is not available.

PropertyPanel

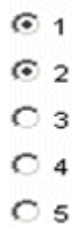
This bean is a generic container that allows the integrator to control how property values are stored and displayed. You must override the load and save methods to implement this standard behavior. You can combine one or more UI components within **JPanel** to provide custom behavior.

For example, if a property in a form called **status** is an integer and is either **1** or **2** meaning **Running** or **Stopped**, you may want two toggle buttons with radio behavior to represent this property. To accomplish this, use the **PropertyPanel** bean with two **JToggleButton**s contained within.

You must override the load and save methods of the **PropertyPanel** to determine the selection state from the two buttons. The **PropertyPanel** is designed for special behavior beyond the scope of the other property beans described in this manual.

PropertyRadioButton

The usage of this bean is same as that of the **PropertyCheckbox** bean; however, rather than using **JCheckBox**, **JRadioButton** is used. The following figure illustrates an implementation of the **PropertyRadioButton** bean.

JavaBean	Description
	 PropertyRadioButton Properties: property The property name presented by the bean. When a component is provided, the button is selected if the property value is the same as the selected value. modifiable Indicates whether the property is modifiable. If not, the button is disabled. selectedValue Specifies the value used for saving when the button is selected. deselectedVaue Specifies the value to use for saving when the button is deselected. saveOnlyWhenSelected Indicates to save only when the button is selected or to always save.
PropertyRadioButtonOptionLov	Same as the PropertyCheckboxOptionLov bean, except it uses buttons (as shown in the following figure).  PropertyRadioButtonOptionLov
PropertySlider	Renders any numeric type property. For double or float types, the value is cast to an integer. For string or note types, the value is converted to an integer if possible. Upon save, the value set on the slider is converted to the corresponding property type and saved. The following figure illustrates an implementation of the PropertySlider bean.

JavaBean

Description

**PropertySlider**

Properties:

property

The property name presented by the bean. When a component is provided, the slider value is set according to the property value.

modifiable

Indicates whether the property is modifiable. If not, the slider is disabled.

PropertyTextArea

Renders any nonarray type property except reference/relation type. This bean wraps the rendering of the given POM property into the **JTextArea**, and is generally used for longer text (as shown in the following figure). Upon saving, it attempts to convert the string to the corresponding type. If it cannot convert the string to the required type, an exception is thrown.

**PropertyTextArea**

Properties:

property

Specifies the property name presented by the bean. When a component is provided, the property value is displayed inside the text area.

modifiable

Indicates whether the property is modifiable. If not modifiable, the text area cannot be edited.

PropertyTextField

Renders any nonarray type property, except reference/relation type properties (as shown in the following figure). This bean wraps the rendering of the given POM property into **JTextField**. Upon saving,

JavaBean	Description
	the bean attempts to convert the string to the corresponding property type. If it cannot convert the string to the required type, an exception is thrown.  PropertyTextField Properties: property Specifies the property name presented by the bean. When a component is provided, the property value is displayed inside the box. modifiable Indicates whether the property is modifiable. If not modifiable, the text box cannot be edited.
PropertyToggleButton	The usage of this bean is the same as the PropertyCheckbox bean; however, this bean uses JToggleButton rather than JCheckBox. The following figure illustrates an implementation of the PropertyToggleButton bean.  PropertyToggleButton Properties: property The property name presented by the bean. When a component is provided, the button is selected if the property value is the same as the selected value. modifiable Indicates whether the property is modifiable. If not, the button is disabled. selectedValue Specifies the value used for saving when the button is selected. deselectedVaue

JavaBean

Description

Specifies the value to use for saving when the button is deselected.

saveOnlyWhenSelected

Indicates to save only when the button is selected or to always save.

PropertyToggleButtonOptionLov

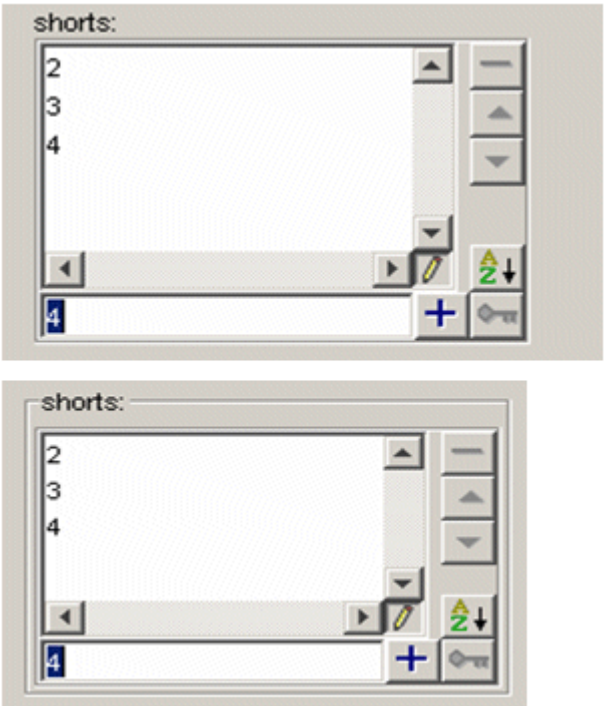
Same as **PropertyCheckboxOptionLov**, except it uses buttons (as shown in the following figure).



PropertyToggleButtonOptionLov

TitledPropertyArray

Displays the property name above the property array (as shown in the following figure) and is similar to the **PropertyArray** bean.



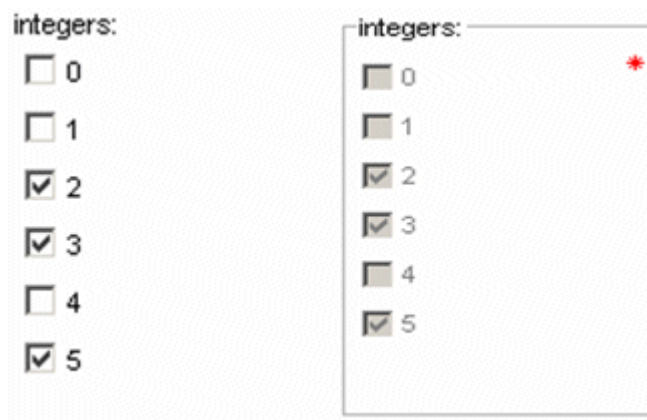
TitledPropertyArray

JavaBean	Description
	bordered Specifies whether a border is drawn around the panel. In addition, you can apply the properties of the PropertyArray bean.
TitledPropertyCheckbox	Displays the property name above the check box (as shown in the following figure). This bean is similar to the PropertyCheckbox bean and actually contains two beans: PropertyNameLabel and PropertyCheckbox.



TitledPropertyCheckbox

	bordered Indicates whether a border is drawn around the check box. In addition, you can apply the properties of the PropertyCheckbox bean.
TitledPropertyCheckboxOptionLov	Displays the property name above the check boxes (as shown in the following figure). This bean is similar to the PropertyCheckboxOptionLov bean.



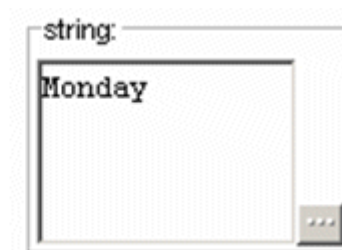
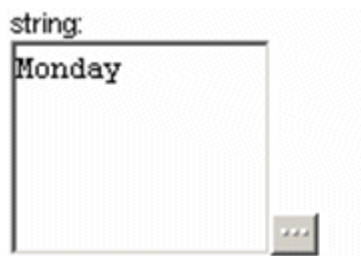
TitledPropertyCheckboxOptionLov

bordered	Indicates whether a border is drawn around the check boxes.
-----------------	---

JavaBean	Description
	In addition, you can apply the properties of the PropertyCheckboxOptionLov bean.
TitledPropertyLabel	Displays the property name above the label (as shown in the following figure). This bean is similar to the PropertyLabel bean, and it actually contains two beans: PropertyNameLabel and PropertyLabel.  The figure shows two UI components. On the left is a PropertyLabel bean, which consists of the text 'string:' followed by 'Monday' on the same line. On the right is a TitledPropertyLabel bean, which consists of a rectangular box containing 'string:' on the top line and 'Monday' on the bottom line. TitledPropertyLabel Properties: bordered Indicates whether a border is drawn around the label. In addition, you can apply the properties of the PropertyLabel bean.
TitledPropertyLogicalPanel	Displays the property name above the box (as shown in the following figure). This bean is similar to the PropertyLogicalPanel bean.  The figure shows two UI components. On the left is a PropertyLogicalPanel bean, which consists of the text 'logical:' followed by two radio buttons labeled 'true' and 'false' on the same line. On the right is a TitledPropertyLogicalPanel bean, which consists of a rectangular box containing 'logical:' on the top line and two radio buttons labeled 'true' and 'false' on the bottom line. TitledPropertyLogicalPanel bordered Indicates whether a border is drawn around the text area. In addition, you can apply the properties of the PropertyLongText bean. Note: For Boolean properties, if neither true or false are selected, the value of the property is NULL.

JavaBean**Description****TitledProperty
LongText**

Displays the property name above the long text panel (as shown in the following figure). This bean is similar to the **PropertyLongText** bean.

**TitledPropertyLongText****bordered**

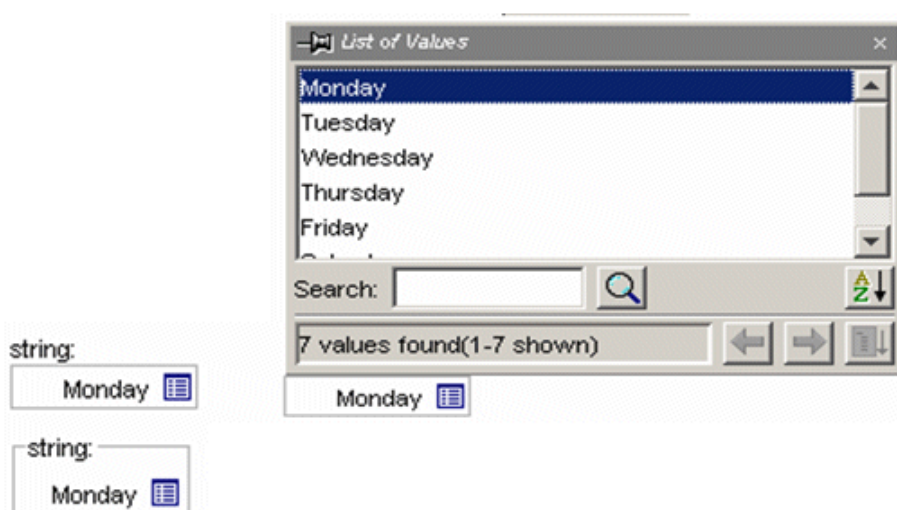
Indicates whether a border is drawn around the text area.

In addition, you can apply the properties of the **PropertyLongText** bean.

**TitledProperty
LOVButton****Caution:**

As of Teamcenter 10.1, this bean is deprecated and is replaced by the **TitledPropertyLOVDisplayer** bean.

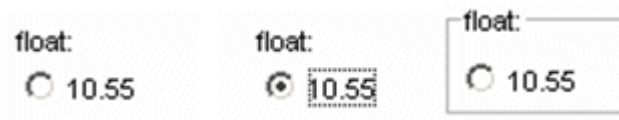
Displays the property name above the LOV button (as shown in the following figure). This bean is similar to the **PropertyLOVButton** bean and actually contains two beans: **PropertyNameLabel** and **PropertyLOVButton**.

**TitledPropertyLOVButton****bordered**

JavaBean	Description
	Indicates whether a border is drawn around the LOV popup button. In addition, you can apply the properties of the PropertyLOVButton bean.
TitledPropertyLOVCombobox	Displays the property name above the LOV combo box (as shown in the following figure). This bean is similar to the PropertyLOVCombobox bean. Caution: As of Teamcenter 10.1, this bean is deprecated and is replaced by the TitledPropertyLOVDisplayer bean.  The figure shows two UI elements side-by-side. The left element is a simple text label 'string:' above a dropdown menu containing 'Monday'. The right element is the same but with a rectangular border around the dropdown menu. TitledPropertyLOVCombobox
	bordered Indicates whether a border is drawn around the LOV combo box. In addition, you can apply the properties of the PropertyLOVCombobox bean.
TitledPropertyObjectLink	Displays the property name above the link (as shown in the following figure). This bean is similar to the PropertyObjectLink bean.  The figure shows two UI elements side-by-side. The left element is a text label 'ref:' above a link 'infodba' with a small icon to its left and a downward arrow to its right. The right element is the same but with a rectangular border around the link and arrow. TitledPropertyObjectLink
	bordered Indicates whether a border is drawn around the link. In addition, you can apply the properties of the PropertyObjectLink bean.
TitledPropertyPanel	Displays the property name above the panel. This is the same as the PropertyPanel bean.
TitledPropertyRadioButton	Displays the property name above the button (as shown in the following figure). This bean is similar to the PropertyRadioButton bean.

JavaBean

Description



TitledPropertyRadioButton

bordered

Indicates whether a border is drawn around the button.

In addition, you can apply the properties of the **PropertyRadioButton** bean.

**TitledPropertyRadio
ButtonOptionLov**

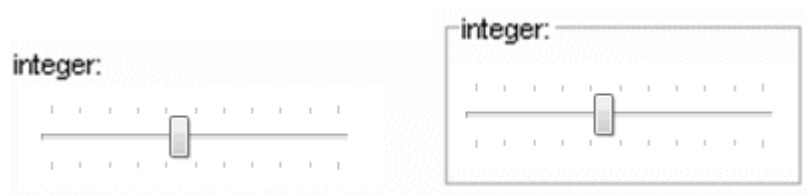
Displays the property name above the buttons (as shown in the following figure). This bean is similar to the **PropertyRadioButtonOptionLov** bean.



TitledPropertyRadioButtonOptionLov

TitledPropertySlider

Displays the property name above the slider (as shown in the following figure). This bean is similar to the **PropertySlider** bean, and it actually contains two beans: **PropertyNameLabel** and **PropertySlider**.

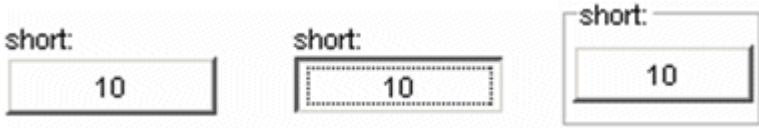
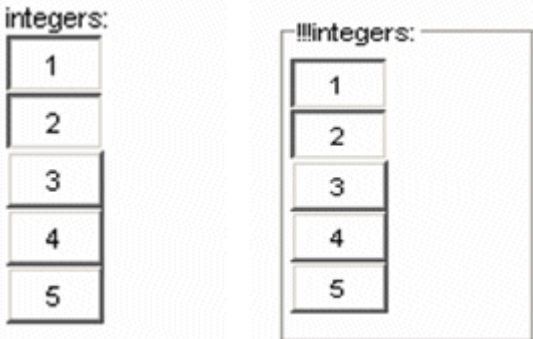


TitledPropertySlider

bordered

Indicates whether a border is drawn around the slider.

JavaBean	Description
TitledPropertyTextArea	In addition, you can apply the properties of the PropertySlider bean. Displays the property name above the text area (as shown in the following figure). This bean is similar to the PropertyTextArea bean and actually contains two beans: PropertyNameLabel and PropertyTextArea.  TitledPropertyTextArea Properties: bordered Indicates whether a border is drawn around the text area. In addition, you can apply the properties of the PropertyTextArea bean.
TitledPropertyTextField	Displays the property name above the text box (as shown in the following figure). This bean is similar to the PropertyTextField bean and actually contains two beans: PropertyNameLabel and PropertyTextField.  TitledPropertyTextField Properties: bordered Indicates whether a border is drawn around the text box. In addition, you can apply the properties of the PropertyTextField bean.
TitledPropertyToggleButton	Displays the property name above the button. This bean is similar to the PropertyToggleButton bean. The following figure illustrates an implementation of the TitledPropertyToggleButton bean.

JavaBean	Description
	 TitledPropertyToggleButton bordered Indicates whether a border is drawn around the button. In addition, you can apply the properties of the PropertyToggleButton bean.
TitledPropertyToggleButtonOptionLov	Displays the property name above the buttons (as shown in the following figure). This bean is similar to the PropertyToggleButtonOptionLov bean.  TitledPropertyToggleButtonOptionLov

Rich client Javadoc

User interface components documented in Javadoc

Teamcenter provides user interface components that you can use in your rich client customizations. The Javadoc documentation for these rich client components are in the **JavaDoc.zip** file within the *Teamcenter-version_pub.zip* file provided with the Teamcenter installation source. After unzipping the **JavaDoc.zip** file, look at the Javadoc found in the following packages:

- **javadoc\com.teamcenter.rac.common\index.html**
Contains user interface components that you can use in your rich client customizations. For a listing of APIs, see *User interface components in the com.teamcenter.rac.common package*. (The list contains a sample of the components available. For a complete list, see the Javadoc.)
- **javadoc\com.teamcenter.rac.util\index.html**
Contains generic components, layout managers, and JavaBeans.

For a listing of APIs, see *User interface components in the com.teamcenter.rac.util package*. (The list contains a sample of the components available. For a complete list, see the Javadoc.)

Use SWT instead of Swing

Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

Following are Swing classes. Siemens PLM Software discourages their use and encourages the use of SWT/JFace:

- AbstractUINode
- AIFTree
- ButtonLayout
- GenericTableModel
- GraphPanel
- HorizontalLayout
- InterfaceUINodeLayout
- LOVComboBox
- LOVDialog
- LOVPanel
- LOVPopupButton
- PropertyLayout
- ReferencerUINode
- SplitPane
- TCComponentUINode
- TCTable
- TCTableCellRenderer
- TCTableLine
- TCTableModel
- TCTableSelectionAdapter
- TCTree
- TCTreeCellRenderer
- TCTreeNode
- TCTreeOpenEvent
- TCTreeOpenListener
- TCTypeRenderer
- VerticalLayout

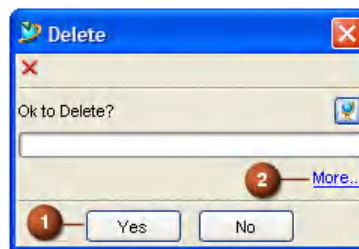
User interface components in the com.teamcenter.rac.common package

AbstractProgressDialog

This class contains the definition for the **AbstractProgressDialog** class, which displays the progress of multiple components individually, as they are processed. The **AbstractProgressDialog** class has the following features:

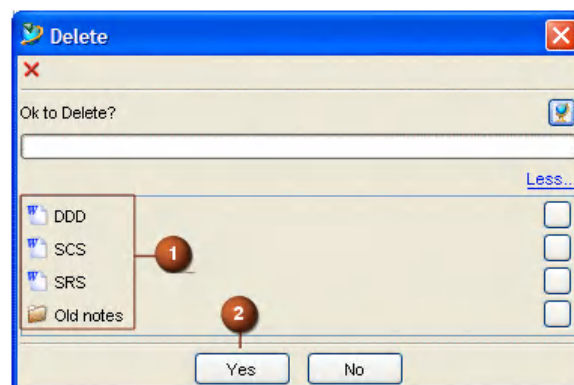
- The status of the operation, such as in-progress, successful completion, or failure of completion, is indicated for each component on the dialog box in the following figure.
- If an operation fails, the tooltip on the error symbol describes the error.
- If the error symbol is clicked, the system displays a detailed error message.
- If confirmation is required before the operation can proceed, the **AbstractProgressDialog** component can display the confirmation flag.
- If confirmation is not required, the operation processes the first component as soon as the dialog box is displayed.
- When an operation is in progress, a **Stop** button displays on the dialog box and can be used to stop the operation. When the delete operation is aborted, the operation in progress on a component cannot be stopped; however, the operation is stopped before the next component is deleted.

The following figures show the behavior of the delete operation.



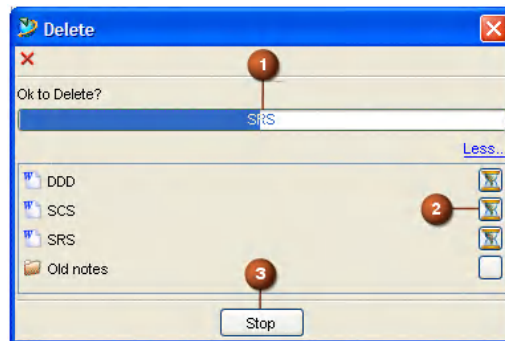
Delete dialog

- 1 Click **Yes** to initiate **Delete** operation.
- 2 Click **More** to see components to be deleted.



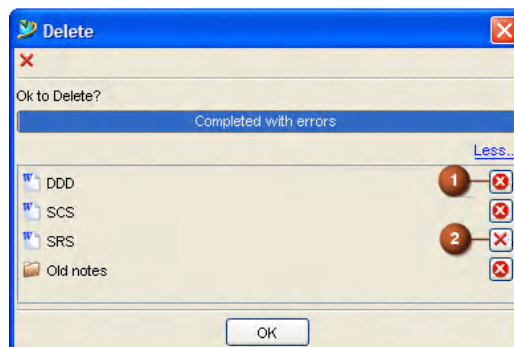
Expanded Delete dialog

- 1 Components to be deleted.
 - 2 Click **Yes** to initiate **Delete** operation.
-



Progress indicators

- 1 Overall progress indicator.
 - 2 Hourglasses indicate components in the midst of being deleted.
 - 3 If **Stop** is clicked, the components being processed are deleted and the operation is stopped.
-



Completion indicators

- 1 Indicates unsuccessful completion of the operation.
 - 2 Indicates successful completion of the operation.
-

The following Teamcenter commands subclass from the **AbstractProgressDialog** class:

- **Cut**
- **Copy**
- **Paste**

- Delete
- Check-In
- Check-Out
- Transfer Check-Out
- Publish
- Unpublish

The following code shows how the **AbstractProgressDialog** class is used to display the components and execute the delete operation. The dialog box calls the **initializeDialog** method in its constructor. The **initializeDialog** method sets the display parameters for the components.

```
public class DeleteDialog extends AbstractProgressDialog
{
    //The Constructor calls the initializeDialog method
    initializeDialog(targets);

    //The initialize Dialog sets the methods for the display in the
    AbstractProgressDialog
    public void initializeDialog (AIFComponentContext[] targets)
    {
        try
        {
            // set the title for the dialog
            setDialogTitle(r.getString("command.TITLE"));
            //display the components that are successfully processed
            setDisplaySuccessComponents(true);

            //Set the icon to be displayed on the dialog
            setCommandIcon(r.getImageIcon("delete.ICON"));

            //Set the icon that needs to be set if the components is successfully processed
            setSuccessIcon(r.getImageIcon("delete.ICON"));
            //set the confirmation to true. The user has to press the Yes button
            //to initiate the operation.
            setConfirmationText(r.getString("confirmationText"));

            // for delete, don't need to show parent
            setShowParentFlag(false);
            // display objects that needs to be cut and their parents on dialog
            setTCComponents(targets);
        }
        catch ( Exception ex )
        {
            //Show the messageBox and return
        }
    }
    // ask user's confirmation before starting operation
    setConfirmationFlag(true);
}
//The abstract method to be implemented to execute the selected operation
```



```

protected void getOperations ( AIFComponentContext compContext )
{
    // In here the developer has to use addOperation method to add the
    // operation.
    addOperation ( new DeleteOperation( compContext ) );
}
// There's also an overloaded getOperations() method which can be used by the developers.
protected void getOperations ( TCCComponent parentComp, AIFComponentContext[] ctxts )
{
    // Create whatever operation you want to based on the parent component
    // and the contexts.
    // Then use the addOperation() method to add the operation.
    addOperation ( createdOperation );
}

```

The **startOperation()** method is called when you click the **Yes** button. This method builds the **DeleteOperation** class, which in turn calls the **startProcess()** method from the **AbstractProgressDialog** class. The **startProcess()** method sets the focus on the first displayed component, sets the in-progress icon against the component, and calls the **getOperations()** method, which is an abstract method that the subclass must implement. The **getOperations()** method in the **DeleteDialog** class calls the **DeleteOperation** class on the first component.

The methods implemented by the subclass are shown in the following code:

```

setDialogTitle(r.getString("command.TITLE"));
setDisplaySuccessComponents(true);
setCommandIcon(r.getImageIcon("delete.ICON"));
setSuccessIcon(r.getImageIcon("delete.ICON"));
setConfirmationText(r.getString("confirmationText"));
setShowParentFlag(false);
setTCCComponents(targets);
setConfirmationFlag(true);
public void run()
public void execute( AIFComponentContext compContext ) throws Exception
(This method builds the appropriate operation on the component)
public void startOperation()
(This method calls the startProcess() method from the AbstractProgressDialog

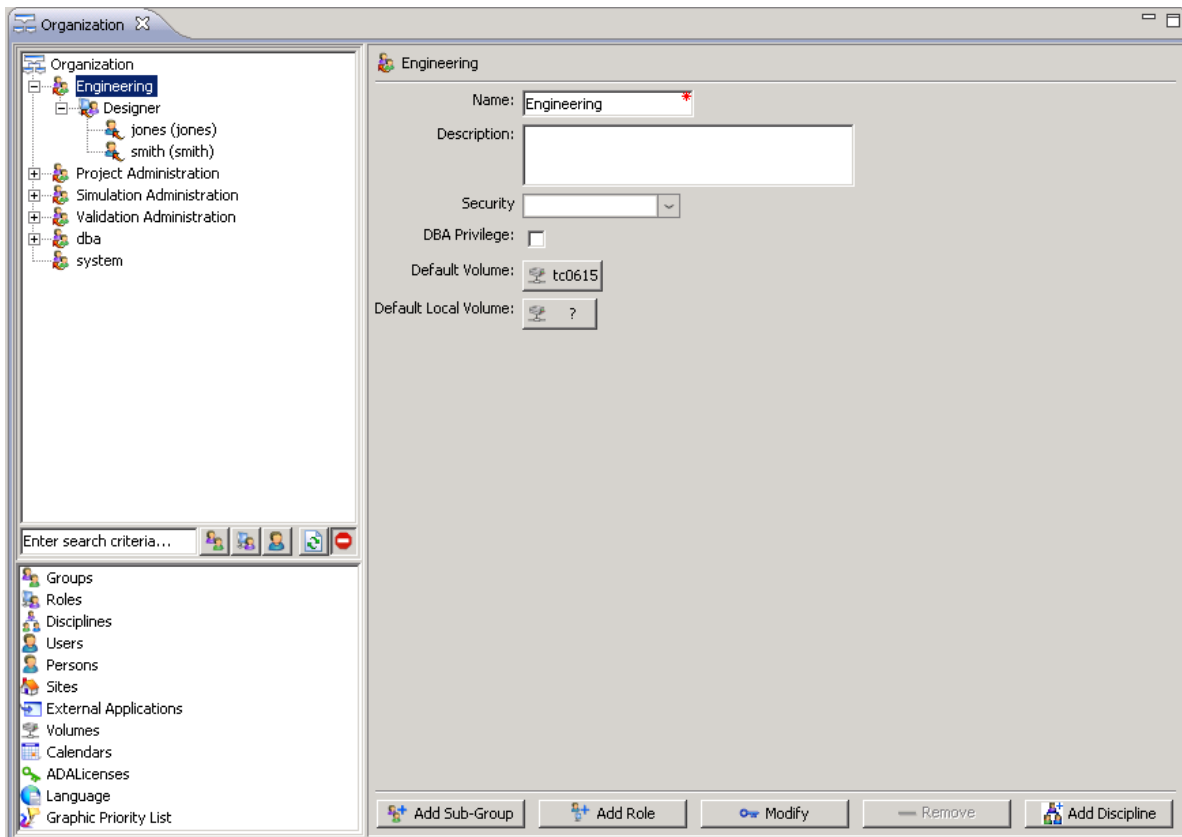
```

ExpansionRule

The **ExpansionRule** component gets the specific components of related types that are attached to a component. Users can set the relations for specific types of components and use the **getChildren()** method to get the required children. The **ExpansionRule** component also provides the ability to filter out unwanted children.

GroupPanel

The **GroupPanel** reusable component extends the **JPanel** component and can be used by any application to display group information (as shown in the following figure).



Group panel in the Organization Selection dialog box

The **GroupPanel** component can be constructed and added to the dialog box for display (as shown in the following code example):

```
GroupPanel groupPanel = new GroupPanel (grpComponent);;
mainSplitPane.setRightComponent (groupPanel);
```

Lists of values (LOVs)

Lists of values (LOVs) are lists in property boxes in the user interface. In the end-user client applications, the format of the LOV box appears as a table rather than a simple list (except for hierarchical and range LOVs). This allows for the display of columns that show the value's description and other properties. The **LOVDisplayer** Java class is the primary class used to define the list of values display.

Safety Code: 1

Value	Description
1	Meets safety standards
2	Does not meet safety standards
35	Under review for safety
40	Safety approval pending

List of values display

The list of values table format allows users to:

- Sort the values in the columns.
- Scroll the list of values using a scroll bar.
- Resize the box containing the list of values.
- Type in the LOV box to narrow the choices when there are many values to choose from. You can even use an asterisk (*) for wildcard searching.

Note:

If end users type text when looking for a value in the format of an integer, double, or date, they must type the exact value.

Following are the list of values Java classes:

```
com.teamcenter.rac.common.lov.common.ILOVConstants
com.teamcenter.rac.common.lov.common.ILOVDataService
com.teamcenter.rac.common.lov.common.LOVColumnInfo
com.teamcenter.rac.common.lov.common.LOVPropertyKey
com.teamcenter.rac.common.lov.common.LOVPropertyValue
com.teamcenter.rac.common.lov.common.LOVRow
com.teamcenter.rac.common.lov.view.components.LOVDataDisplayView
com.teamcenter.rac.common.lov.view.components.LOVDisplayer
com.teamcenter.rac.common.lov.view.components.LOVSelectionDisplayView
com.teamcenter.rac.common.lov.view.components.LOVTreeTable
com.teamcenter.rac.common.lov.view.components.LOVWindowResizer
com.teamcenter.rac.common.lov.view.controls.LOVDataDisplayView
com.teamcenter.rac.common.lov.view.controls.LOVDisplayer
com.teamcenter.rac.common.lov.view.controls.LOVSelectionDisplayView
com.teamcenter.rac.common.lov.viewmodel.LOVDataDisplayProperties
com.teamcenter.rac.common.lov.viewmodel.LOVDataServiceImpl
com.teamcenter.rac.common.lov.viewmodel.LOVSelectionDisplayProperties
com.teamcenter.rac.common.lov.viewmodel.LOVViewModel
```

Note:

As of Temcenter 10.1, the following list of values Java classes are deprecated:

```
com.teamcenter.rac.common.lov.InterDependentLOVPanel
com.teamcenter.rac.common.lov.LOVUIComponent
com.teamcenter.rac.common.lov.LOVPanel
com.teamcenter.rac.common.lov.LOVPopupButton
com.teamcenter.rac.common.lov.LOVDialog
com.teamcenter.rac.common.lov.LOVComboBox
com.teamcenter.rac.common.lov.HierarchicalLOVComponent
com.teamcenter.rac.common.lov.LOVHierarchyPanel
com.teamcenter.rac.common.controls.SWTLovPopupButton
com.teamcenter.rac.common.controls.SWTLovPopupDialog
com.teamcenter.rac.common.controls.LOVUIComponent
com.teamcenter.rac.common.controls.LOVComboBox
```

Note:

In rich client forms, the Tab key moves focus from field to field, except for lists of values. To move focus between lists of values, users can press Ctrl+Tab.

MRUButton

The **MRUButton** component is used by applications to maintain a list of previously referenced **InterfaceAIFComponent** objects. It is generic and works with any AIF application within the AIF framework. When one of the objects is clicked, the **open** method for the application is invoked with the component.

The **MRUButton** component maintains a list of components supplied by the application. The application must populate the components to the **MRUButton** component and determine the number of objects to place inside the component. Different behaviors can be associated with the use of the **MRUButton** component in different applications. A purge limit is commonly associated with the **MRU** button; however, the **MRUButton** component shown in the following figure does not employ a purge limit.

The code in the following example constructs the **MRUButton** component:

```
mruButton = new MRUButton ( explorerApp );
mruButton.setSuggestedVerticalAlignment (MRUButton.TOP);
mruButton.loadMRUEntries();
```

OpenByNameButton

The **OpenByNameButton** component allows an application to open new objects that can be manipulated. It also allows users to issue queries and view properties of the objects located by the query.

The objects that are found by the query are loaded into the table. Instead of loading all objects, only the first set is loaded. To load the next set, click the **Load Next** button. To load all objects, click the **Load All** button. The **Load Previous** button loads the previous set and appends it to the current selection. Once the object is located, double-click the object to open it in the application.

Objects can be selected in the table and copied to the clipboard. The **Copy** button is active only when an item has been selected.

The **OpenByNameButton** component enforces the limitation that a query can only be performed on a single object type, for example, item revisions or folders. This is usually acceptable, as most applications can only support one root object type. Examples of this are My Teamcenter, which only supports folders as root objects, and Structure Manager, which only supports occurrences as root objects.

The **OpenByNameButton** component is subclassed from the **AbstractPopupButton** class, which allows it to be inserted into the user interface where it is treated like any other Java button.

This composition allows you to add things to uniquely identify the button and its purpose, such as tooltips and icons.

The following code shows how to construct the **OpenByNameButton** component for the My Teamcenter application, which only searches on folders:

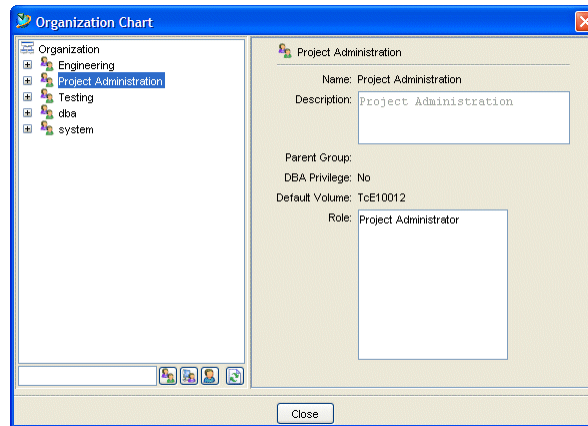
```
OpenByNameButton openByNameButton = new OpenByNameButton(explorerApp, "Folder");
```

OrgSelectionDialog

The **OrgSelectionDialog** reusable component displays the organization chart in tree form. The root of the chart is a root group, and nodes in the tree represent groups, roles, and users. The tree displays hierarchies within the organization.

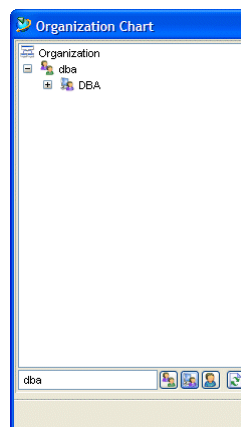
This dialog box consists of a vertical split pane. The left pane displays the tree, and the right pane displays information about the selected role, group, or user node.

The dialog box shown in the following figure displays information about a fictional organization. The first-level nodes in the tree represent groups. Groups can be expanded to display the hierarchical groups or roles contained within the group. Roles can be expanded to display the users assigned to the role.



OrgSelectionDialog component

In addition, the dialog box provides the ability to search for a specific group, role, or user within the organization (as shown in the following figure). If you click the **Reload** button, the tree displays all top-level groups in the organization. Both figures show the features of the **OrgSelectionDialog** component.



Organization dialog box search feature

The **Access** dialog box presents the **OrgSelectionDialog** component as a separate dialog box. The **OrgSelectionDialog** component handles its own events and display; therefore, dialog boxes that use this dialog component must invoke the **OrgSelectionDialog** component. The code in the following example shows the **OrgSelectionDialog** component constructed from the **AccessDialog** component:

```

JButton selectUserButton = new JButton(appReg.getImageIcon("selectUser.ICON"));
selectUserButton.addActionListener ( new ActionListener()
{
    public void actionPerformed (ActionEvent e)
    {
        //Create the OrgSelectionDialog
        orgSelectionDialog = new OrgSelectionDialog(parent, target);
        orgSelectionDialog.addPropertyChangeListener(AccessDialog.this);
        orgSelectionDialog.setVisible(true);
    }
});

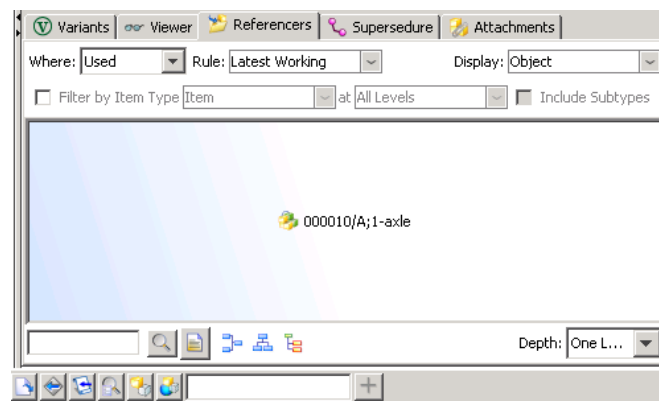
```

The **OrgSelectionDialog** component is constructed and added as a **propertyChangeListener** to the **AccessDialog** component, and the visibility of the dialog component is set to **true**. The Organization Selection displays when you click the **Select User** button in the **Access** dialog box.

ReferencersPanel

The **ReferencersPanel** component displays where-used and where-referenced diagrams. An object can be sent to the **Referencers** panel, where the user can double-click it to search for where the object is used or referenced.

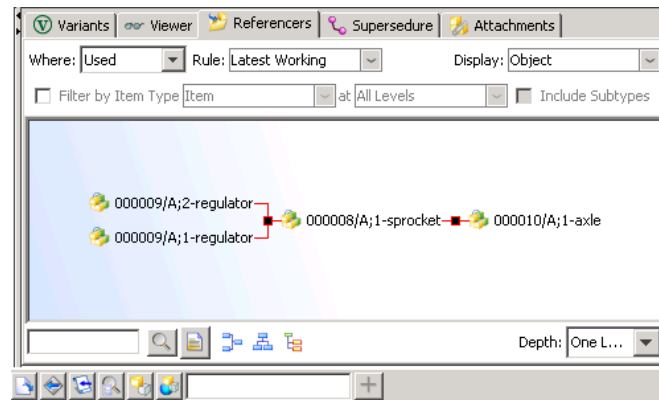
When a component is sent to the **Referencers** panel, whether it defaults to the where-referenced or where-used display depends on the component type. Users can define which components to display in the panel.



Referencers panel

The **Referencers** panel can display the nodes in reverse horizontal style, vertical style, or tree-look style. Three layout managers are used to accomplish this:

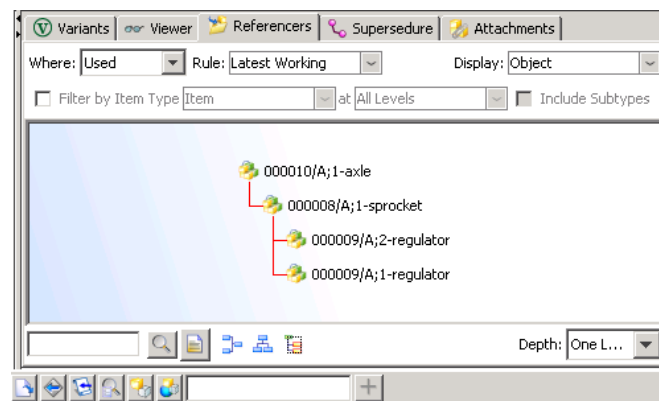
- **ReferencersReverseHorizontalNodeLayout**
Displays the nodes in reverse horizontal order where the structure is expanded from right to left horizontally and can display where-referenced and where-used information in different colors. The **ReferencersReverseHorizontalNodeLayout** layout manager extends from the **ReverseHorizontalNodeLayout** component.



Referencers reverse horizontal node layout

- **ReferencersTreeLookNodeLayout**

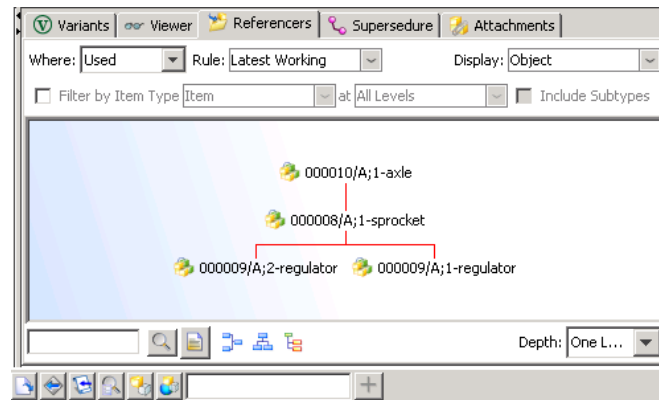
Displays the nodes in a **JTree** manner and can display where-referenced and where-used information in different colors. The **ReferencersTreeLookNodeLayout** layout manager extends from the **TreeLookNodeLayout** component.



Referencers tree look node layout

- **ReferencersVerticalNodeLayout**

Displays the nodes in vertical order expanded from top to bottom and can display different where-referenced and where-used information in different colors. The **ReferencersVerticalNodeLayout** layout manager extends from the **VerticalNodeLayout** component.



Referencers vertical node layout

ReferencerUINode

The **ReferencerUINode** component is an extension of the **TCComponentUINode** component. It adds an attribute that tells if a where-used or where-referenced expansion is associated with the node.

Note:

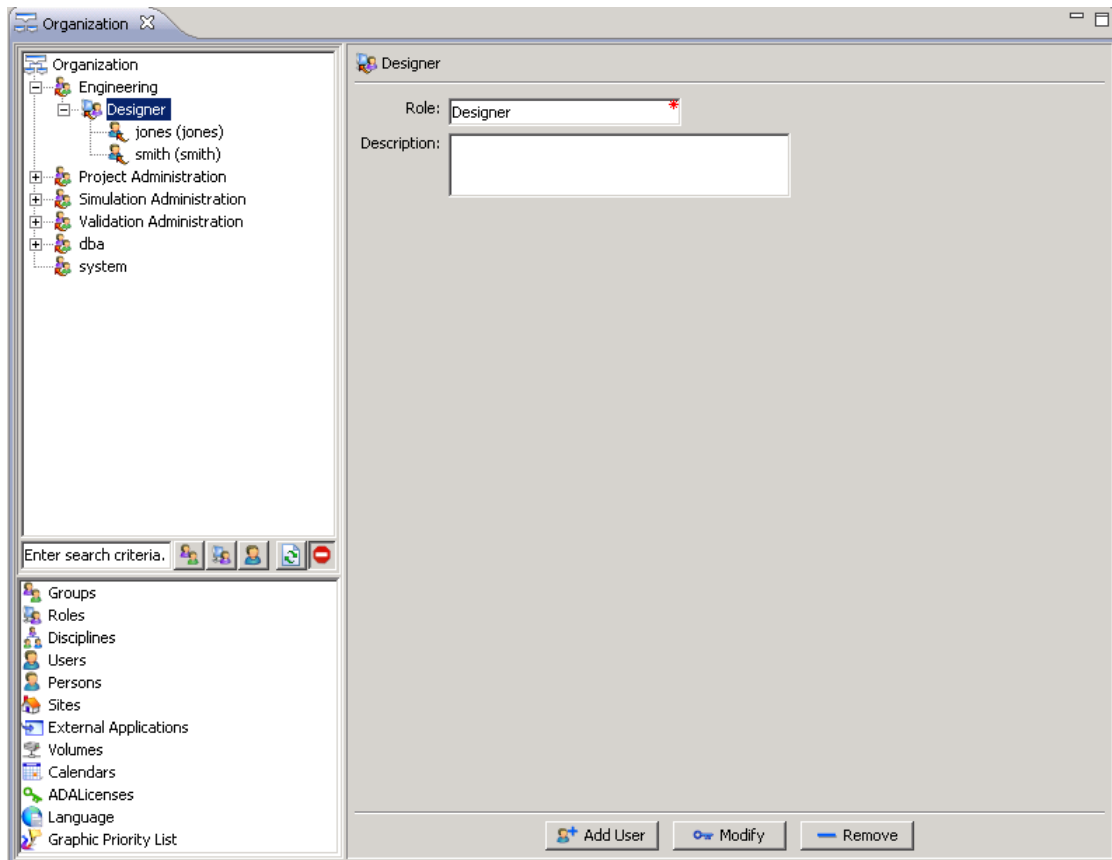
This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

This example shows how to construct a **ReferencersPanel** component and set a component in it:

```
referencersPanel = new ReferencersPanel(explorerApp, false, false, false, false);
referencersPanel.setComponent ( c );
```

RolePanel

The **RolePanel** reusable component extends the **JPanel** component and can be used by any application to display role information (as shown in the following figure).



Role panel in the Organization Selection dialog box

The following code shows how the **RolePanel** component is constructed and added to the dialog box.

```
RolePanel rolePanel = new RolePanel (roleComponent);
mainSplitPane.setRightComponent (rolePanel);
```

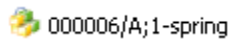
TCComponentUINode

The **TCComponentUINode** component is an extension of the **AbstractUINode** class, and creates a UI node based on an **TCComponent** object.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

The UI node is presented with the **TCComponent** object name and rendered icon (as shown in the following figure).



Item revision UI component

The following code shows the code used to create an **TCComponentUINode** component as the root in a **GraphPane** panel.

```
// myFolder is an TCComponent
TCComponentUINode node = new TCComponentUINode ( myFolder );
GraphPane panel = new GraphPane ( new VerticalNodeLayout() );
// add the UI node to the panel and set it as the root
panel.setRoot ( node );
```

TCConstants

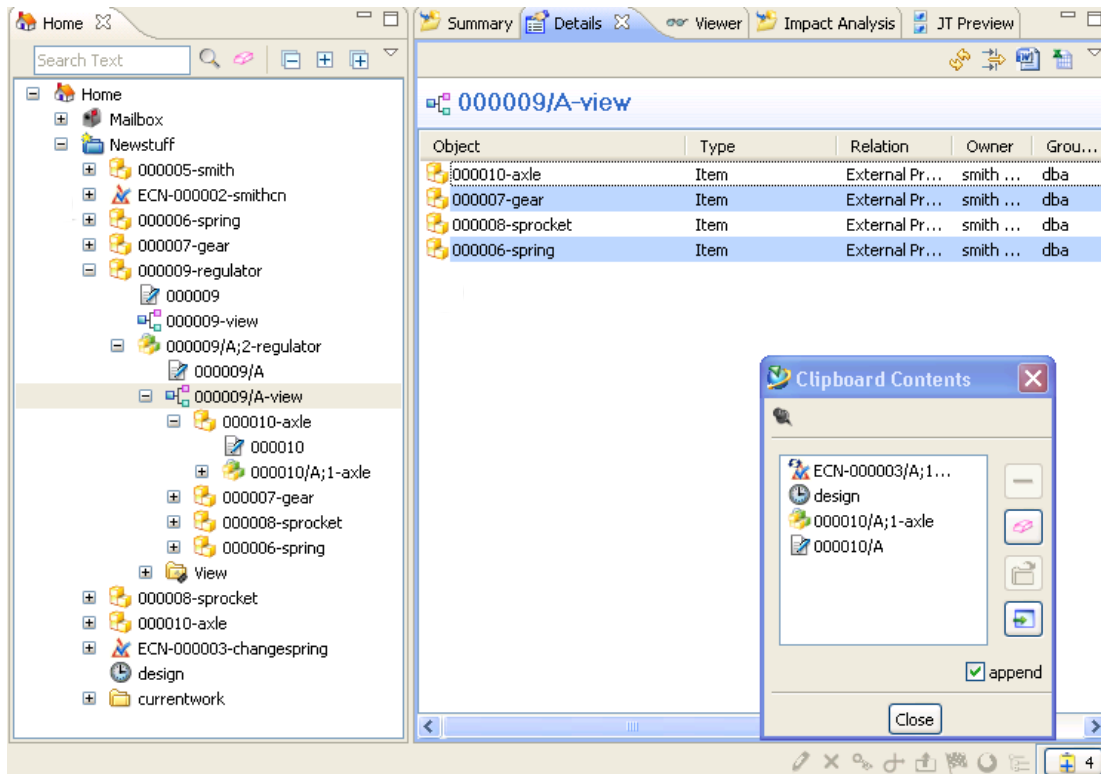
The **TCConstants** class contains constants used across Teamcenter and its related packages. Use the static variables defined in this class rather than hard-coding strings.

TCTypeRenderer

The **TCTypeRenderer** class is an implementation of a renderer that returns an icon based on either the Teamcenter object type or an object property. Many Teamcenter components use this render class.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.



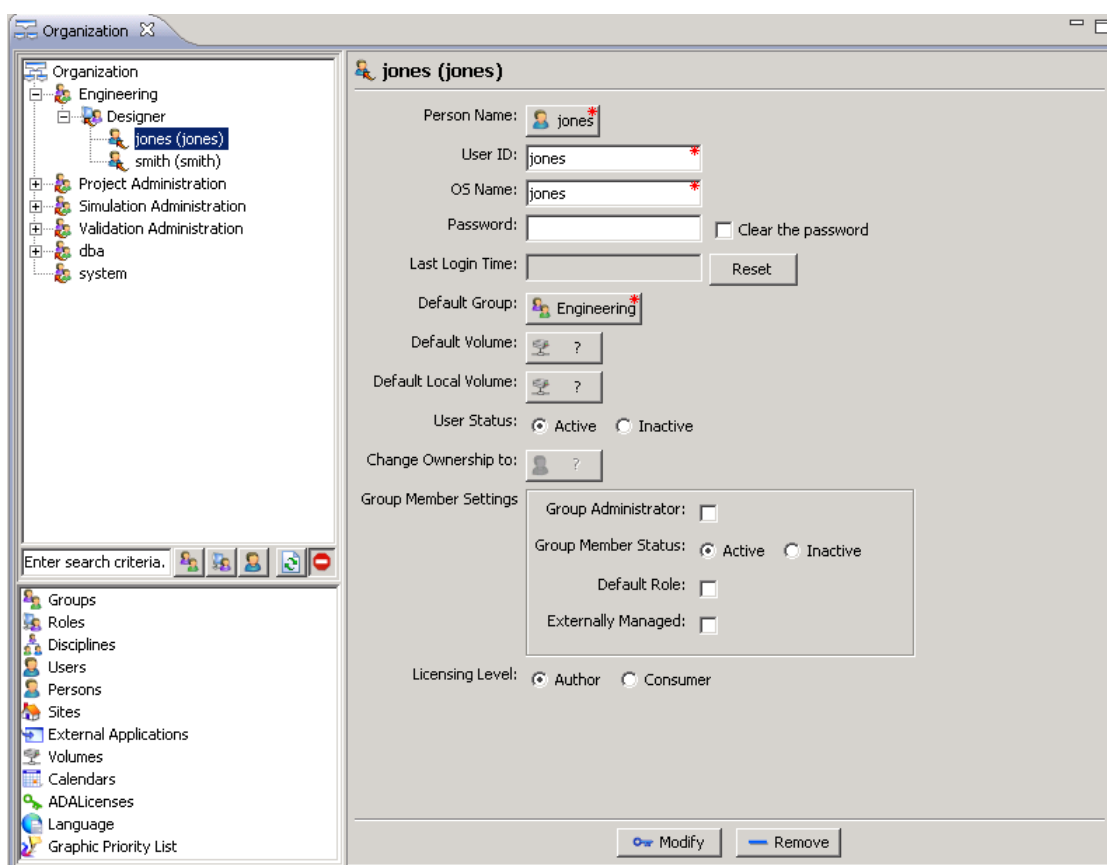
Usage of the TCTypeRenderer class

The following code example shows how to obtain the icon for an **TCComponent** object using the **TCTypeRenderer** class:

```
String text;
if ( comp instanceof TCComponent )
{
  TCComponent ic = (TCComponent)comp;
  text = ic.toString();
  JButton bt = new JButton();
  bt.setText ( text );
  bt.setIcon ( TCTypeRenderer.getIcon ( ic, false ) );
}
```

UserPanel

The **UserPanel** reusable component extends the **JPanel** component and can be used by any application to display group information (as shown in the following figure).



User panel in the Organization Selection dialog box

The **UserPanel** component can be constructed and added to the dialog box for display (as shown in the following code example).

```
UserPanel userPanel = new UserPanel (roleComponent);
mainSplitPane.setRightComponent (userPanel);
```

User interface components in the com.teamcenter.rac.util package

AbstractDialog

The **AbstractDialog** component enhances the functionality of the **JDialog** component, allowing you to instantiate the dialog box from a nonvisible application and keep it modal. The **AbstractDialog** component also provides **centerToScreen** methods.

The **AbstractDialog** class must be inherited by another class in order to work. For example, the **StringViewerDialog** component extends the **AbstractDialog** class, as follows:

```
public class StringViewerDialog extends AbstractDialog
{
    // Class implementation here.
};
```

AbstractPopupButton

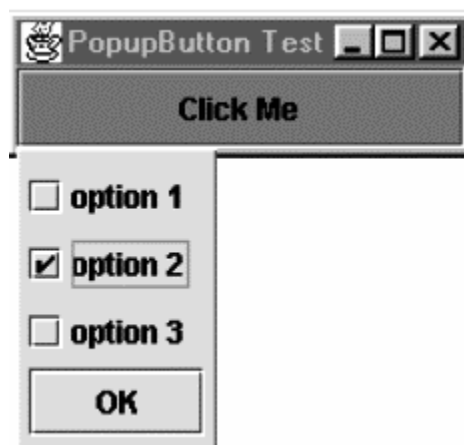
The **AbstractPopupButton** class creates a custom popup window from a button. This allows a small UI component to display a larger window that functions like a dialog box.

The **AbstractPopupButton** component works like a **JComboBox** component. It initially displays as a button (as shown in the following figure).



Initial state of an AbstractPopupButton component

It displays a popup window when the button is clicked (as shown in the following figure). Classes that extend from the **AbstractPopupButton** class must implement the UI for the popup window.



AbstractPopupButton component popup window

The following example adds three check boxes and a button to the popup window. When you click the **OK** button, the popup window closes.

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import javax.swing.*;
import com.teamcenter.rac.util.*;
public class PopupButtonTest extends AbstractPopupButton
{
    PopupButtonTest (String txt)
    {
        super(txt);
    }
    public void initPopupWindow()
```

```

{
    // get the panel for popup window
    JPanel popupWin = getPanel();
    JPanel main = new JPanel ( true );
    main.setLayout ( new VerticalLayout(2,4,4,4,4) );

    JCheckBox op1 = new JCheckBox ( "option 1" );
    JCheckBox op2 = new JCheckBox ( "option 2" );
    JCheckBox op3 = new JCheckBox ( "option 3" );
    JButton okButton = new JButton ( "OK" );
    okButton.addActionListener( new ActionListener()
    {
        public void actionPerformed ( ActionEvent e )
        {
            // post down the popup window
            postDown();
        }
    });
    main.add ( "top", op1 );
    main.add ( "top", op2 );
    main.add ( "top", op3 );
    main.add ( "top", okButton );
    // add the created panel to the popup window
    popupWin.add ( "Center", main );
}
public static void main(String s[])
{
    WindowListener l = new WindowAdapter()
    {
        public void windowClosing(WindowEvent e) {System.exit(0);}
    };
    Frame f = new Frame("PopupButton Test");
    f.addWindowListener(l);
    f.add("Center", new PopupButtonTest ("Click Me"));
    f.pack();
    f.setSize(new Dimension(350,350));
    f.show();
}
}

```

GenericTableModel

The **GenericTableModel** class is an implementation of a custom **TableModel** component that stores and caches the strings it manages.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

The following example shows code used to create a **GenericTableModel** component:

```

//Create the Table to show the history
columnNames.addElement(appReg.getString("date"));

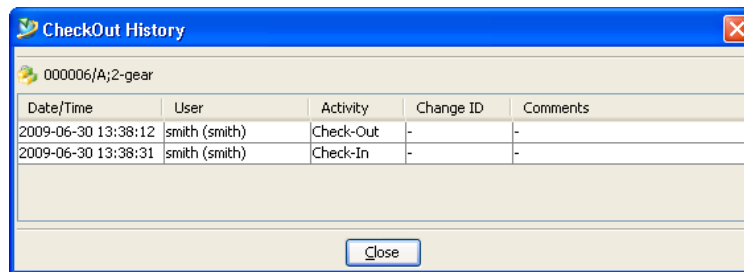
```

```

columnNames.addElement(appReg.getString("user"));
columnNames.addElement(appReg.getString("activity"));
columnNames.addElement(appReg.getString("changeId"));
columnNames.addElement(appReg.getString("comments"));
        dataModel = new GenericTableModel(columnNames, 0);
dataModel.markColumnEditable(0, true);
JTable historyTable = new JTable (dataModel);
.....

```

The following figure shows a table created with the **GenericTableModel** component.



Date/Time	User	Activity	Change ID	Comments
2009-06-30 13:38:12	smith (smith)	Check-Out	-	-
2009-06-30 13:38:31	smith (smith)	Check-In	-	-

Table created using GenericTableModel component

iTextArea

The **iTextArea** component is a subclass of the **JTextArea** class and displays all the same behaviors. Use this component to achieve a consistent look and feel across the system. The goal is to use this component across the entire commercial Teamcenter interface, so that when standards change or are enhanced the changes can be placed in this class and be inherited by all implementing components.

Using the **iTextArea** subclass provides the following benefits over using the **JTextArea** class:

- **Field selection when focus is gained**

Data is automatically selected when the focus is gained within the text area.

- **Improved focus traversal**

The up arrow and down arrow keys provide next and previous focus traversal abilities.

- **Require signaling**

You no longer must override the paint method to implement the paint, as required. Instead, invoke the **setRequired(Boolean state)** method.

iTextField

The **iTextField** component is a subclass of the **JTextField** class and displays all the same behaviors. Use this component to achieve a consistent look and feel across the system. The goal is to use this component across the entire commercial Teamcenter interface, so that when standards change or are enhanced the changes can be placed in this class and be inherited by all implementing components.

Using the **iTextField** subclass provides the following benefits over using the **JTextField** class:

- **Field selection when focus is gained**

Data is automatically selected when the focus is gained within the text field.

- **Improved focus traversal**

The up arrow and down arrow keys provide next and previous focus traversal abilities.

- **Require signaling**

You no longer must override the paint method to implement the paint, as required. Instead, invoke the **setRequired(Boolean state)** method.

Layout manager

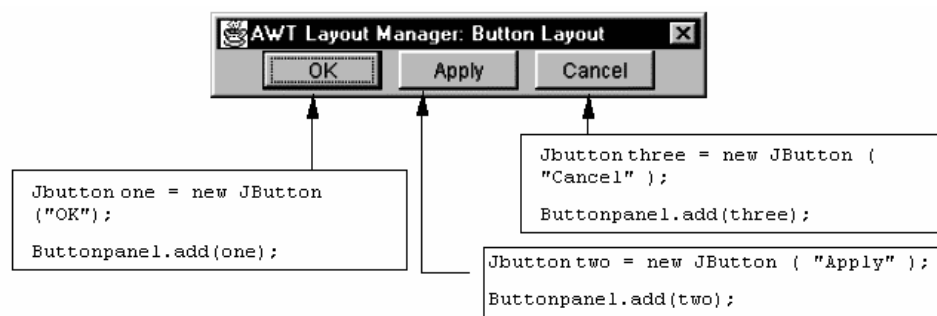
ButtonLayout

Horizontal ButtonLayout layout manager examples

The following figure shows the **ButtonLayout** layout manager with horizontal orientation and center alignment.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.



Horizontal button layout with center alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.HORIZONTAL,
ButtonLayout.CENTER)
```

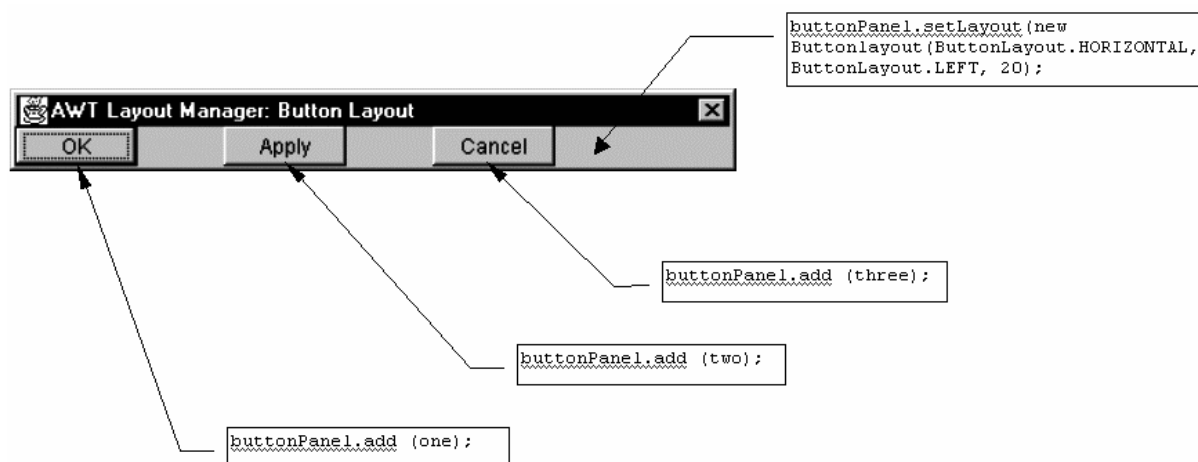
Buttons are added in the order in which they are displayed. In the previous figure, the sequence is **OK**, **Apply** and **Cancel**.

The following figure shows the dialog box when it is resized. The components maintain their orientation and alignment.



Results of resizing the dialog box

The following figure shows the **ButtonLayout** layout manager with horizontal orientation, left alignment, and a 20-unit gap between the buttons.

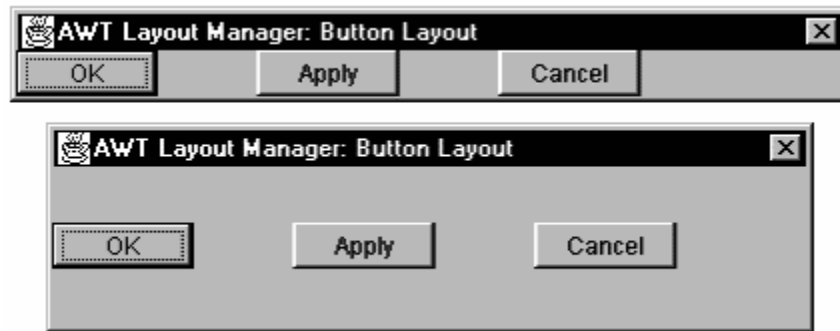


Horizontal button layout with left alignment and a 20-unit gap

The constructor is called, as follows:

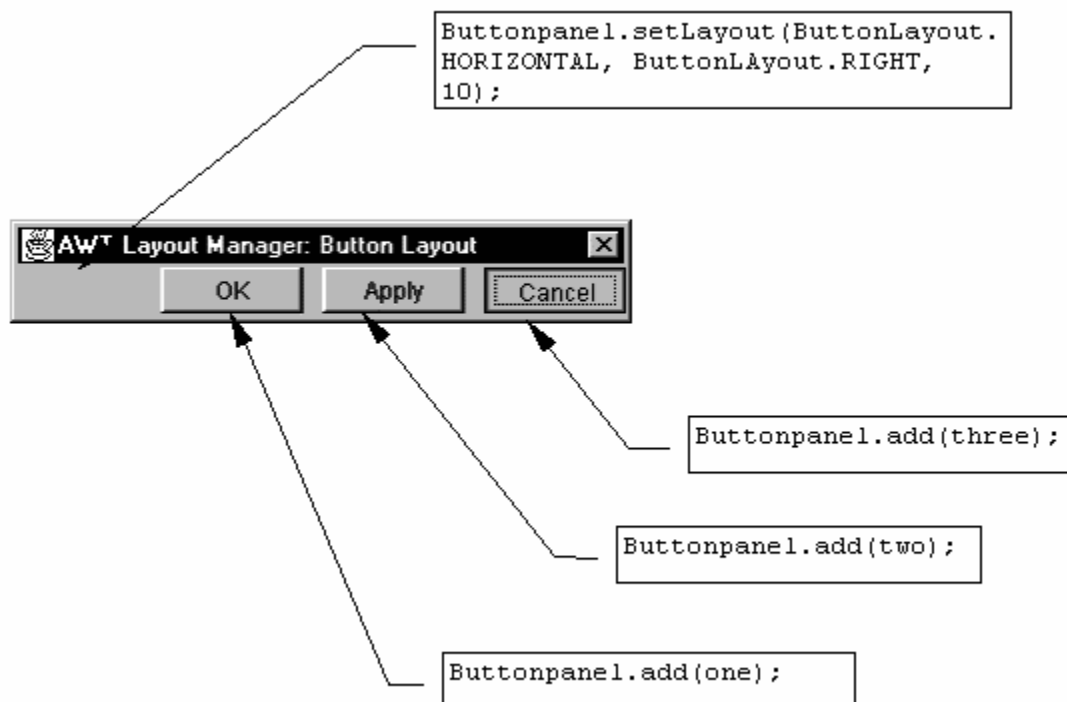
```
buttonPanel.setLayout (new ButtonLayout(ButtonLayout.HORIZONTAL,
ButtonLayout.LEFT, 20);
```

Since the alignment is set to **LEFT**, the first button added is aligned to the left corner of the dialog box and the remaining buttons are placed to the right of the first, with a 20-unit gap between buttons. The following figure shows the dialog box when it is resized.



Results of resizing the dialog box

The following figure shows the **ButtonLayout** layout manager with horizontal orientation, right alignment, and a 20-unit gap between the buttons.

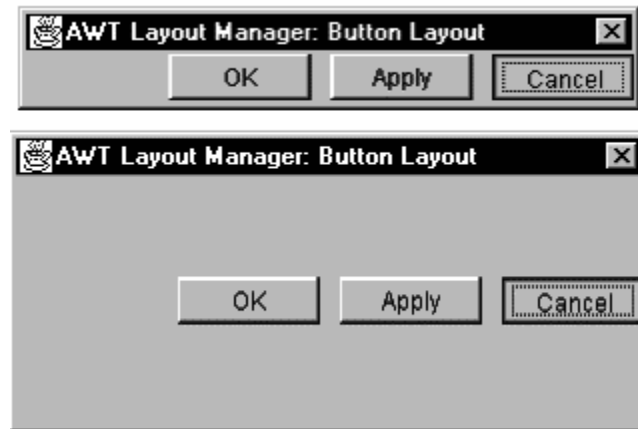


Horizontal button layout with right alignment and a 20-unit gap

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.HORIZONTAL,
ButtonLayout.RIGHT, 20);
```

Since the alignment is set to **RIGHT**, the first button added is aligned to the right corner of the dialog box and the remaining buttons are placed to the left of the first, with a 20-unit gap between buttons. The following figure shows the dialog box when it is resized.



Results of resizing the dialog box

The following example shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout ( Frame parent, String title )
    {
        super ( parent, title, false );
        // Create a new panel with a ButtonLayout Manager
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new ButtonLayout (ButtonLayout.HORIZONTAL));
        // Create three buttons
        one = new JButton ( "OK" );
        two = new JButton ( "Apply" );
        three = new JButton ( "Cancel" );
        // Add the buttons to the Panel created
        buttonPanel.add (one );
        buttonPanel.add (two);
        buttonPanel.add (three);
        this.add (buttonPanel);
        this.pack ();
    }
}
```

The **buttonPanel** panel is created. The panel uses the **ButtonLayout** layout manager and assumes the default values for its parameters. Three buttons are created and added to the panel. The buttons are positioned horizontally, in the center of the panel, with a 10-unit gap between buttons. The buttons are

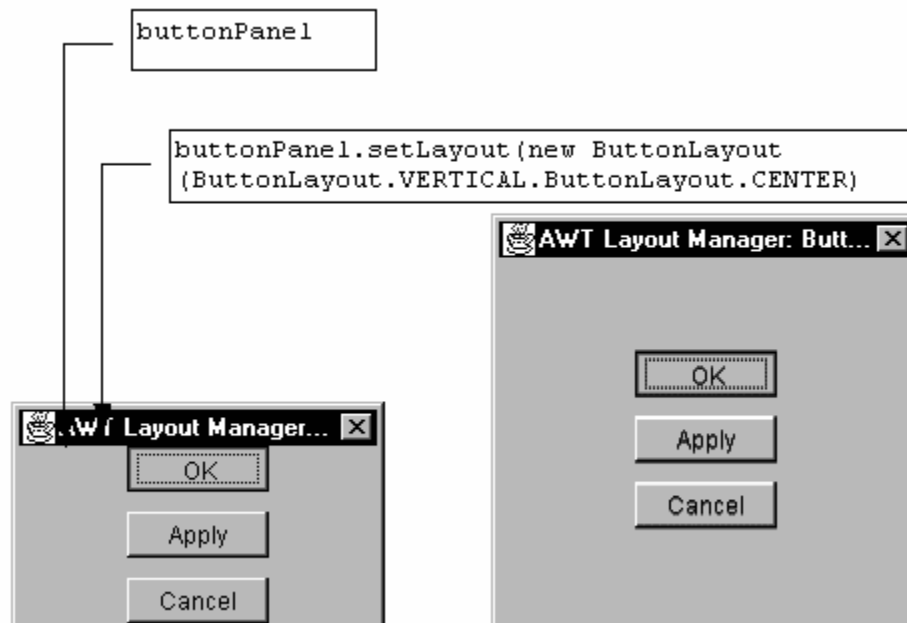
not resized when the panel is resized. The position of the buttons in relation to the edges of the panel also remains unchanged.

Vertical BorderLayout layout manager examples

The following figure shows the **ButtonLayout** layout manager with vertical orientation and center alignment.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.



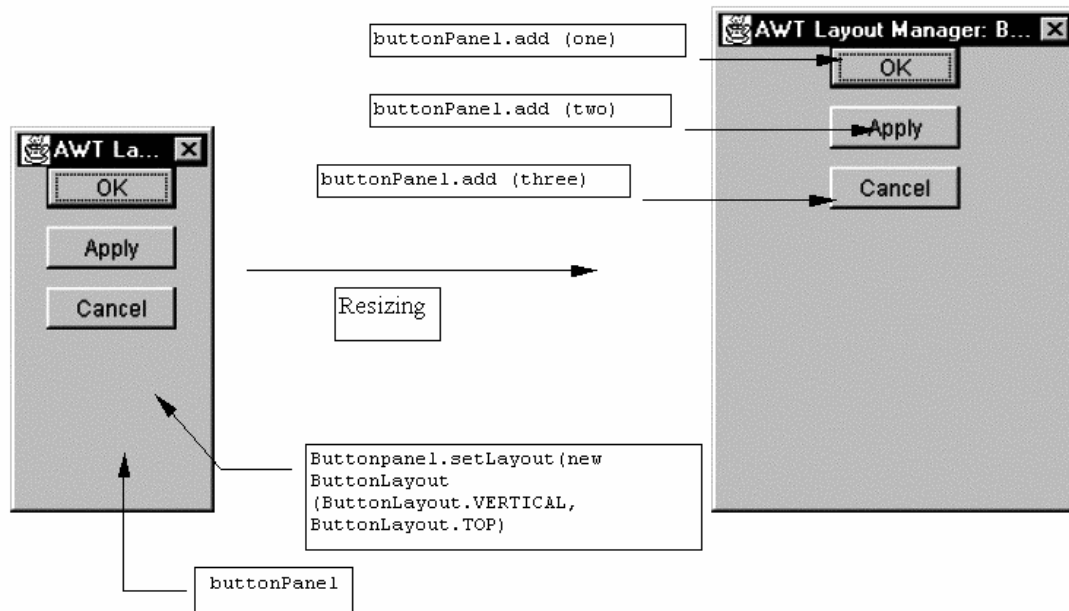
Vertical button layout with center alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.VERTICAL,
ButtonLayout.CENTER) ;
```

When the panel is resized, the buttons maintain their size and alignment in relation to the dialog box. The buttons are placed in the sequence in which they are added to the panel.

The following figure shows the **ButtonLayout** layout manager with vertical orientation and top alignment.

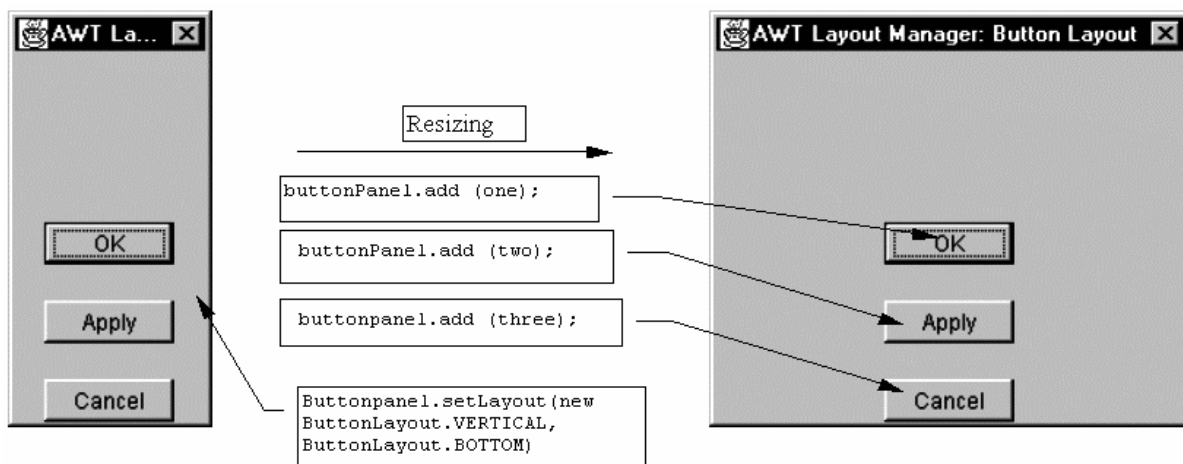


Vertical button layout with top alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.VERTICAL,
ButtonLayout.TOP) ;
```

The following figure shows the **ButtonLayout** layout manager with vertical orientation and bottom alignment.



Vertical button layout with bottom alignment

The constructor is called, as follows:

```
buttonPanel.setLayout (new ButtonLayout (ButtonLayout.VERTICAL,
ButtonLayout.BOTTOM);
```

The following figure shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout ( Frame parent, String title )
    {
        super ( parent, title, false );
        // Create a new panel with a ButtonLayout Manager
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new ButtonLayout(ButtonLayout.HORIZONTAL));
        // Create three buttons
        one = new JButton ( "OK" );
        two = new JButton ( "Apply" );
        three = new JButton ( "Cancel" );
        // Add the buttons to the Panel created
        buttonPanel.add (one );
        buttonPanel.add (two);
        buttonPanel.add (three);
        this.add (buttonPanel);
        this.pack ();
    }
}
```

The **buttonPanel** panel is created. The panel uses the **ButtonLayout** layout manager and assumes default values for its parameters. Three buttons are created and added to the panel. The buttons are positioned horizontally in the center of the panel with a 10-unit gap between buttons. Resizing a **ButtonLayout** panel does not resize the buttons. The position of the buttons relative to the four edges of the panel also remains unchanged.

HorizontalLayout

The **HorizontalLayout** layout manager positions children and attachments horizontally in the **container** object.

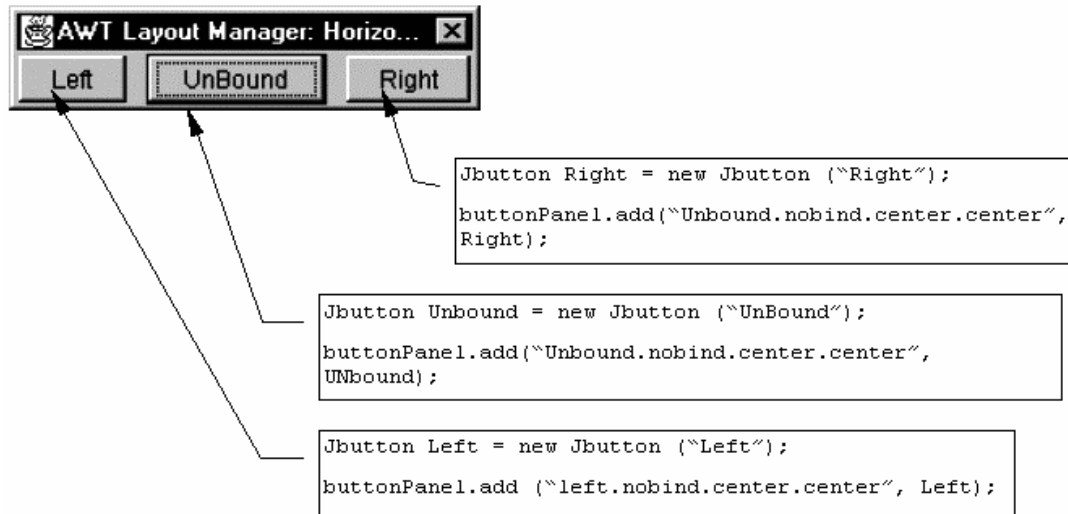
Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

When a component is added to the **container** object, the position of the component is determined by the parameters passed in the **name** field. These parameters are **{Attachment, bind, HorizontalAlignment, VerticalAlignment}**. The default positioning for a component added without correct formatting in the **name** field is **(left.bind.center.center)**. **bind** and **nobind** indicate that the component will be resized or maintained based on the space available. Top components are positioned

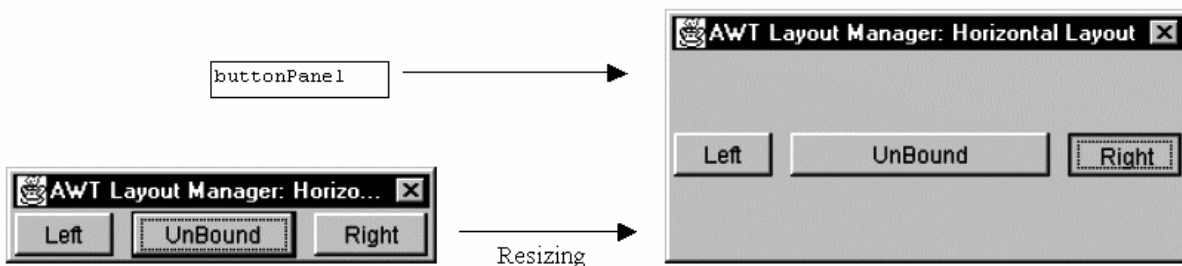
first, followed by bottom components and unbound components. If you do not call the **add** function with a name string in the argument, an exception occurs.

The following figure shows the use of the **HorizontalLayout** layout manager.



Horizontal layout with center alignment

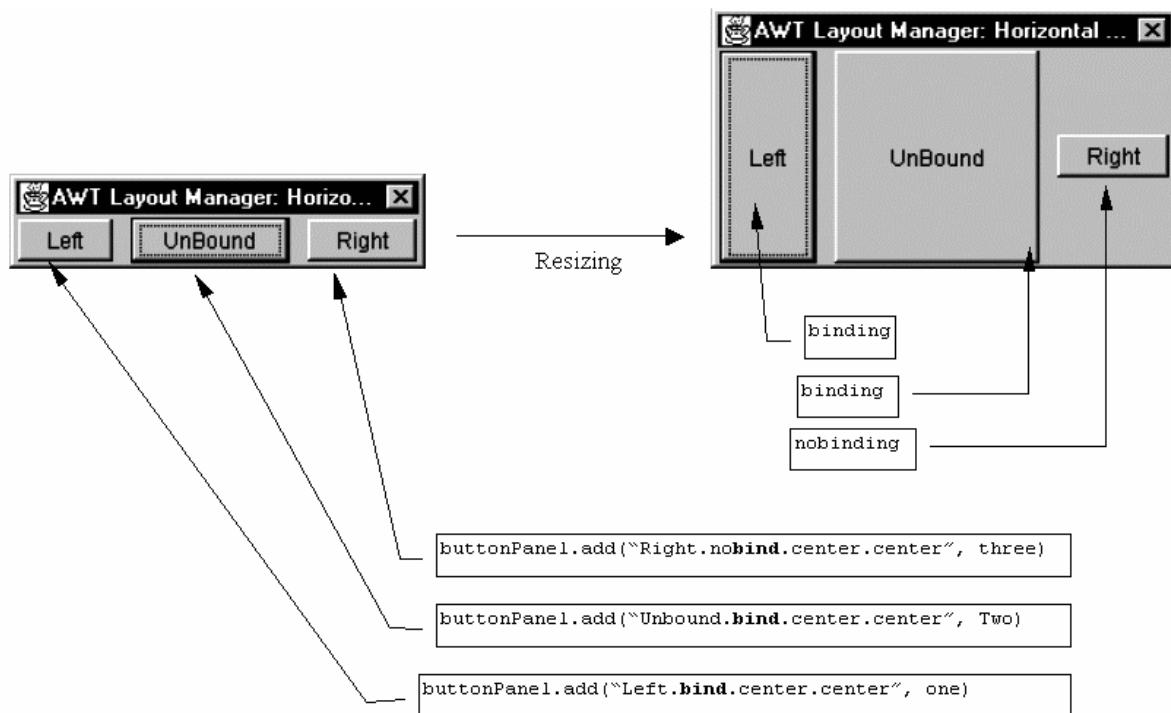
The following figure shows the results when the dialog box is resized.



Results of resizing the dialog box

When the dialog box is resized, the **Left** and **Right** buttons maintain their shape, size, and alignment in relation to the edges of the dialog box because they are added with a **nobind** parameter. The **Unbound** button is resized when the dialog box is resized because it is added to the dialog box using a **bind** parameter.

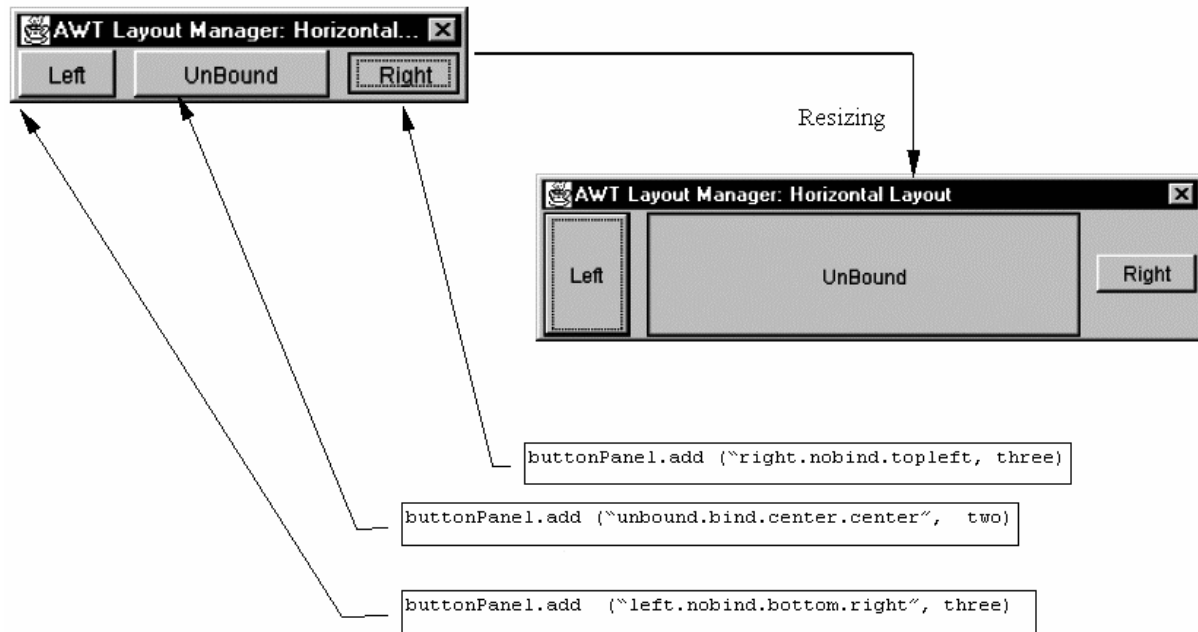
The following figure shows the dialog box when the placement of the components is changed according to the parameters indicated in the figure. The behavior of the dialog boxes when resized is also shown.



Horizontal layout with components added

When you resize dialog boxes, components added with the **bind** parameter are resized.

In the following figure, the **Left** and **Unbound** buttons are added with a **bind** parameter and the **Right** button is added with a **nobind** parameter. When the dialog box is resized, the **Left** and **Unbound** buttons are resized, but the **Right** button maintains its size and alignment.



Horizontal layout with components added

In addition to controlling the placement of the components within the dialog box, the **LayoutManager** component controls the spacing of the components relative to the edges of the dialog box by passing the parameters into the constructor for the **LayoutManager** component. The order in which the parameters are passed into the constructor is important. The constructor is as follows:

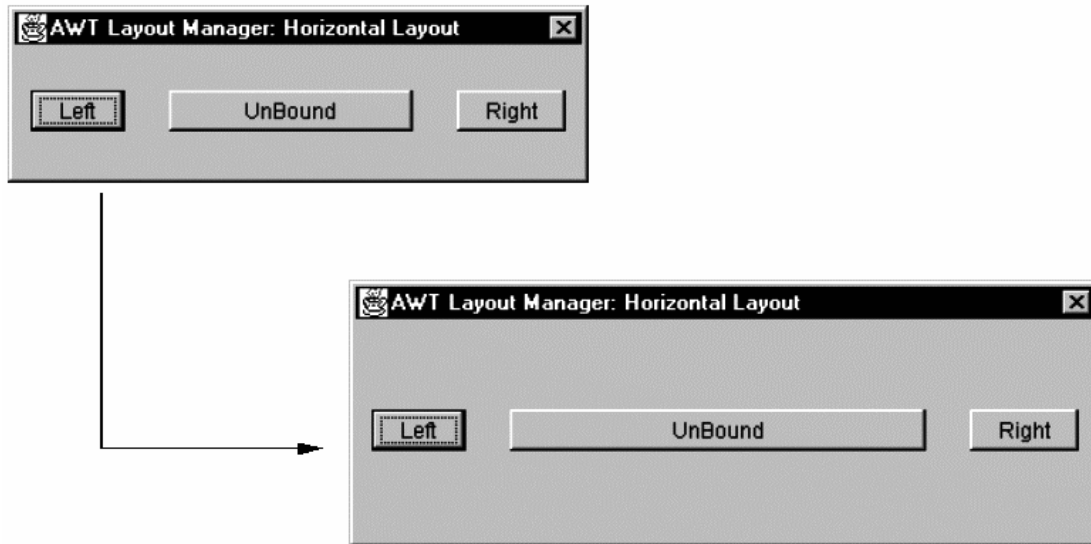
```
new HorizontalLayout ( vgap, lm, rm, tm, bm);
```

vgap indicates the distance between the components in the dialog box. **lm** indicates the left margin, **rm** indicates the right margin, **tm** indicates the top margin, and **bm** indicates the bottom margin. All parameters are expressed in integers and measured in pixels.

The following figure shows the dialog box that results when the following parameters are passed to the constructor:

```
new HorizontalLayout (25, 10, 10, 25, 25);
```

The following figure also shows the behavior when the dialog box is resized. The margins and spacing between components are maintained.



Horizontal layout with parameters

The following example shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;

    public testlayout( Frame parent, String title )
    {
        super ( parent, title, false );
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new HorizontalLayout(10,2,2,2,0));
        one = new JButton ( "Left" );
        two = new JButton ( "Right" );
        three = new JButton ( "UnBound" );
        buttonPanel.add ( "left.nobind.center.center", one );
        buttonPanel.add ( "unbound.nobind.center.center", three );
        buttonPanel.add ( "right.nobind.center.center", two );
        this.add (buttonPanel);
        this.pack ();
    }

    public static main void ( String [] args){
    {
        Frame f = new Frame ( "Horizontal Layout Test" );
        f.resize ( 100,100 );
        f.show();
        testlayout d = new testlayout ( f, "AWT Layout Manager: Horizontal Layout" );
        d.show();
    }
    }
}
```

The **buttonPanel** panel is created. The **HorizontalLayout** layout manager is used and three buttons are created and added to the panel. The placement of the buttons in each of the examples is determined by the parameters passed in the **name** field.

PropertyLayout

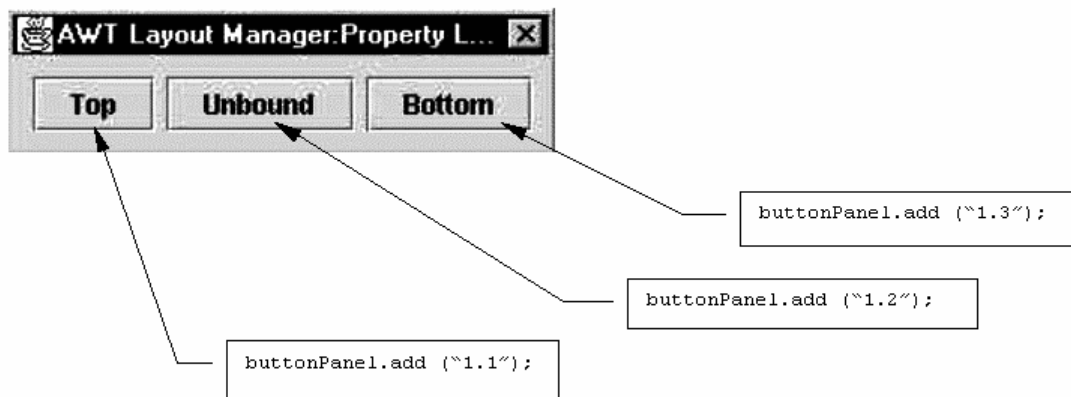
The **PropertyLayout** layout manager positions children and attachments vertically within the **container** object.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

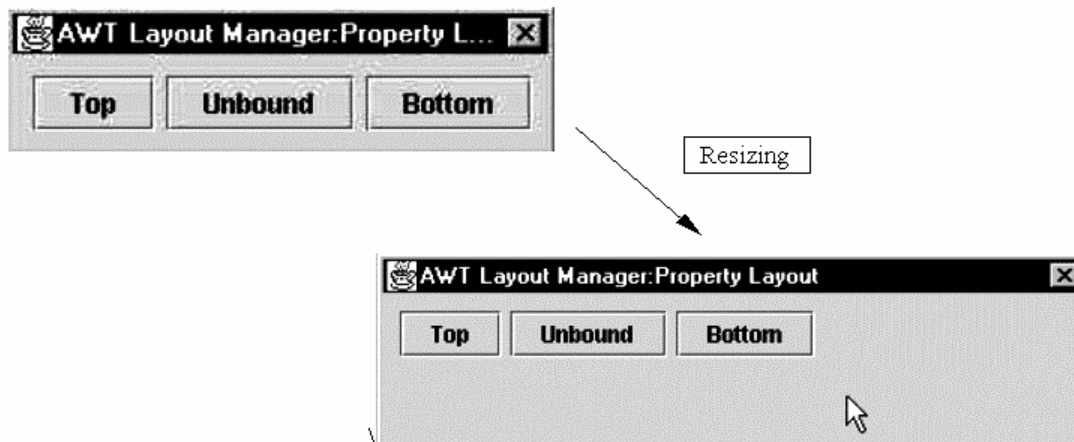
When a component is added to the **container** object, its position is determined by a mask passed through the **name** field that describes how to position the component and whether resizing of the component is desired. The parameters are **Row**, **Column**, **HorizontalAlignment**, **VerticalAlignment**, **HorizontalAttachment**, and **VerticalAttachment**. The default positioning for a component added without correct formatting in the **name** field is **1.1.center.center.preferred.preferred**.

The following figure shows the use of the **PropertyLayout** layout manager with default parameters for each of the components.



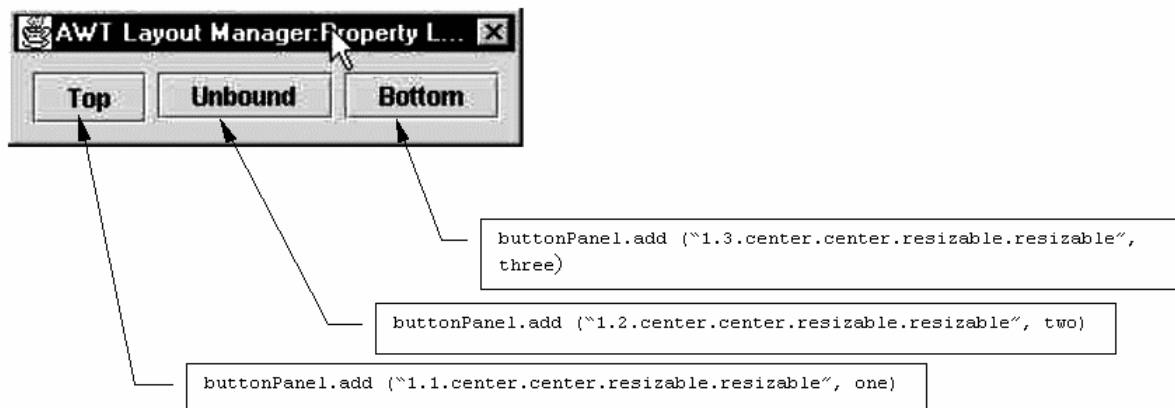
Property layout with components added

The code creates a **buttonPanel** panel using the **PropertyLayout** layout manager. Three buttons are created and added to the panel in three separate columns. When resized, the placement and size of the components remain unchanged (as shown in the following figure). The default positioning of the components in the dialog box is **1.1.center.center.preferred.preferred**. Because the horizontal alignment and vertical alignment parameters are set to **preferred**, the components are not resized along with the dialog box.



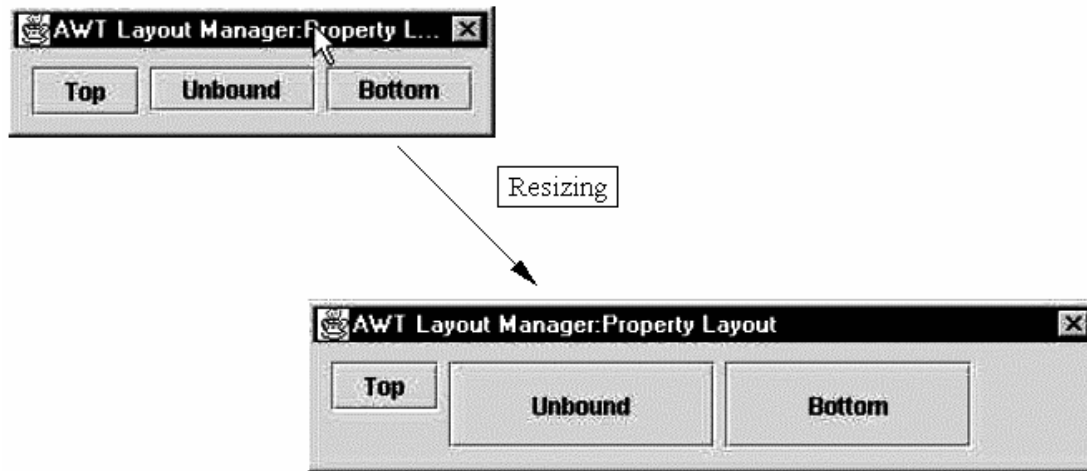
Results of resizing the dialog box

The following figure shows the layout of a dialog box using the **PropertyLayout** layout manager. The components are placed according to the parameters indicated in the figure. Note that the **HorizontalAttachment** and **VerticalAttachment** parameters are set to **resizable**.



Property layout with components added

The following figure shows the behavior when the dialog box is resized. All components are resized except the component placed in the column 1, row 1 position. The other components in the dialog box are resized both horizontally and vertically.



Results of resizing the dialog box

In addition to controlling the placement of components in the dialog box, the **LayoutManager** component controls the spacing of the components relative to the edges of the dialog box. This is achieved by the parameters passed into the constructor. The order in which the parameters are passed into the constructor is important. The constructor is as follows:

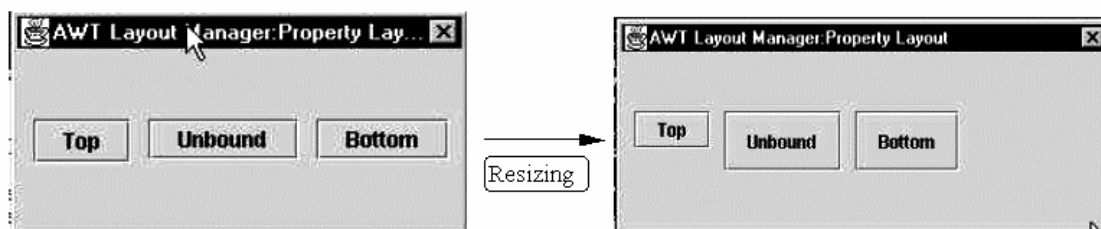
```
new PropertyLayout ( hgap, vgap, lm, rm, tm, bm);
```

hgap indicates the horizontal distance between components in the dialog box. **vgap** indicates the vertical distance between components in the dialog box. **lm** indicates the left margin, **rm** indicates the right margin, **tm** indicates the top margin, and **bm** indicates the bottom margin. All parameters are expressed in integers and measured in pixels. The components are added to the dialog box horizontally, that is, they are placed on the dialog box in the same row but in different columns.

The following figure shows the dialog box that results from passing the following parameters to the constructor:

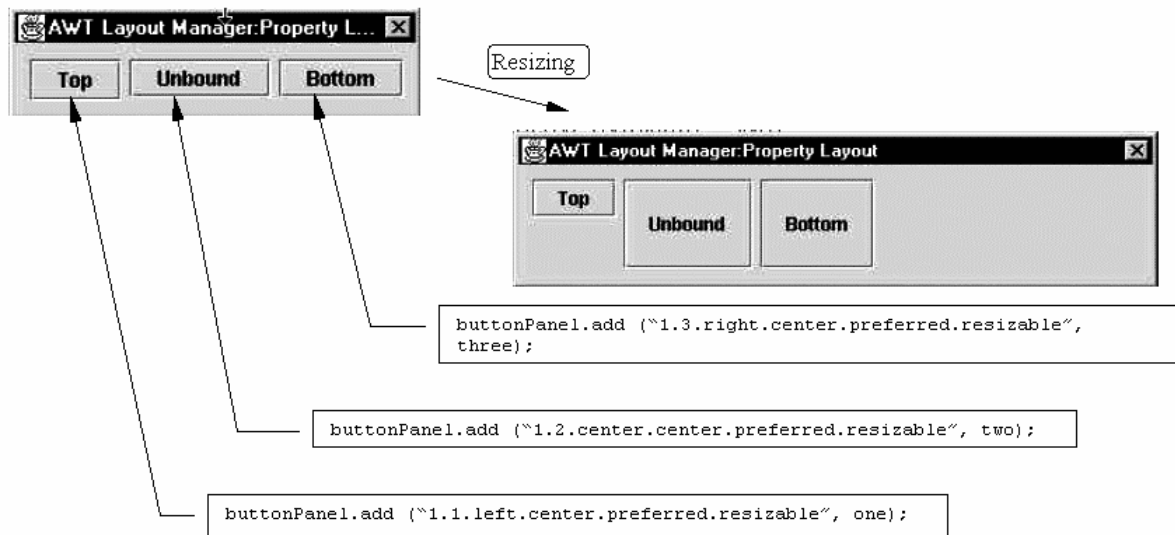
```
new PropertyLayout (10, 20, 10, 10, 40, 40);
```

The following figure also shows the behavior when the dialog box is resized. Only the left and top margins and spacing between components are maintained when the dialog box is resized.



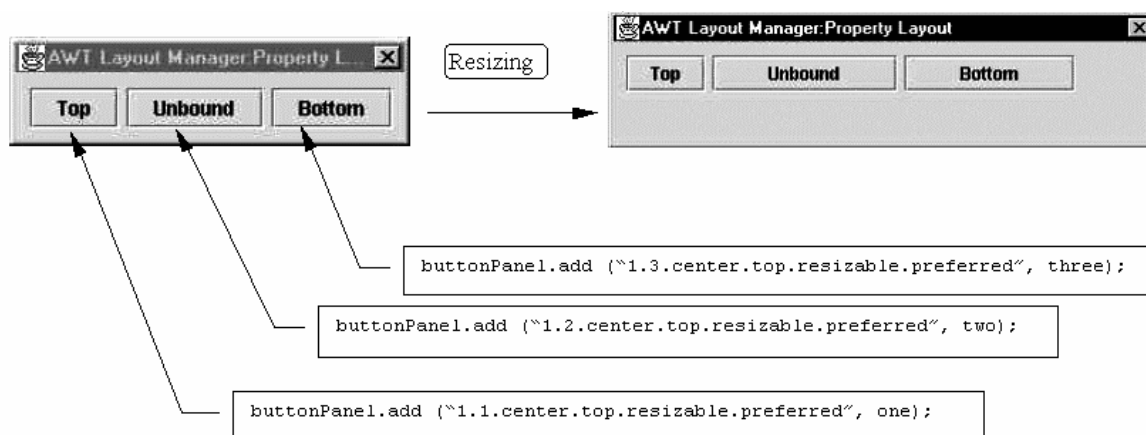
PropertyLayout Manager with margin setup

The following figure shows a dialog box in which the components are only resizable vertically. The **HorizontalAttachment** parameter is set to **preferred**. The buttons in the figure do not seem aligned due to the positions selected for their placement.



Results of resizing the dialog box

The following figure shows a dialog box in which the components are only resizable vertically. The **HorizontalAttachment** parameter is set to **preferred**.



Results of resizing the dialog box

The following example shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout( Frame parent, String title )
```

```

{
    super ( parent, title, false );
    buttonPanel = new JPanel();
    buttonPanel.setLayout ( new PropertyLayout());
    one = new JButton ( "Top" );
    two = new JButton ( "Bottom" );
    three = new JButton ( "UnBound" );
    buttonPanel.add ( "1.1 ", one );
    buttonPanel.add ( "1.2", two );
    buttonPanel.add ( "1.3", three );
    this.getContentPane().add (buttonPanel);
    this.pack ();
}
public static void main ( String[] args ){
{
    Frame f = new Frame ( "Vertical Layout Test" );
    f.show();
    testlayout d = new testlayout ( f, "AWT Layout Manager: Vertical Layout" );
    d.show();
}
}
}

```

VerticalLayout

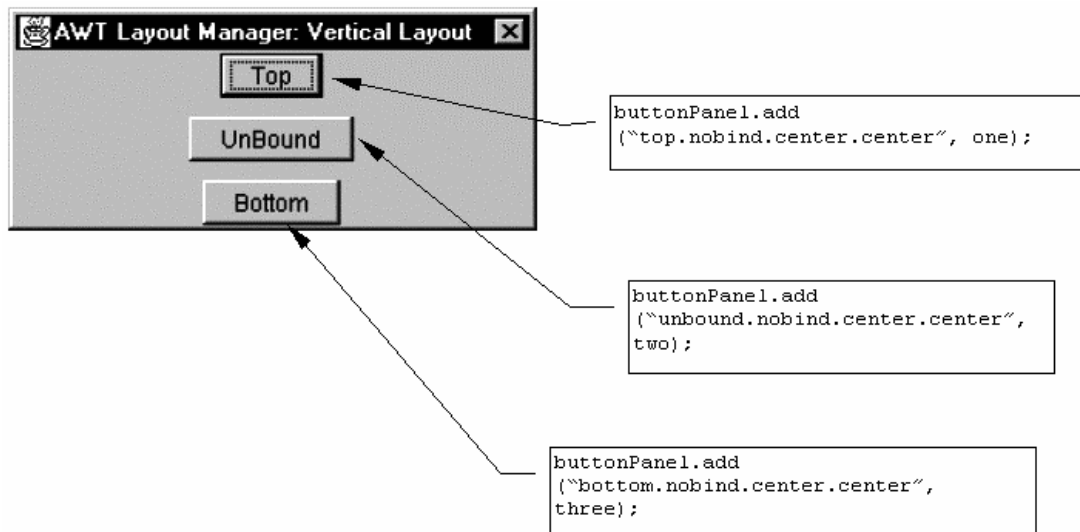
The **VerticalLayout** layout manager positions children and attachments vertically in the **container** object.

Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

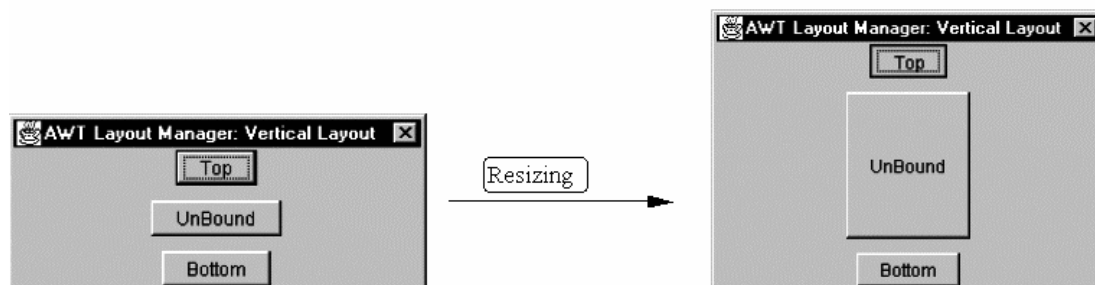
When a component is added to the **container** object, the position of the component is determined by the parameters passed in the **name** field. These parameters are **{Attachment.Binding.HorizontalAlignment.VerticalAlignment}**. The default positioning for a component added without correct formatting in the **name** field is **(top.center.center.bind)**. Top components are positioned first, followed by bottom components and unbound components. If you do not call the **add** function with a name string in the argument, an exception occurs.

The following figure shows the use of a **VerticalLayout** layout manager. The placement of the components within the dialog box is determined by the parameters passed into the **name** field.



Vertical layout with components added

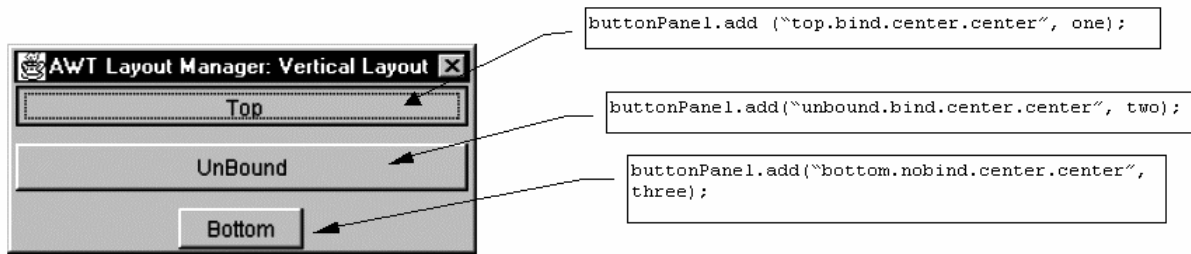
The following figure shows the results of resizing the dialog box.



Results of resizing the dialog box

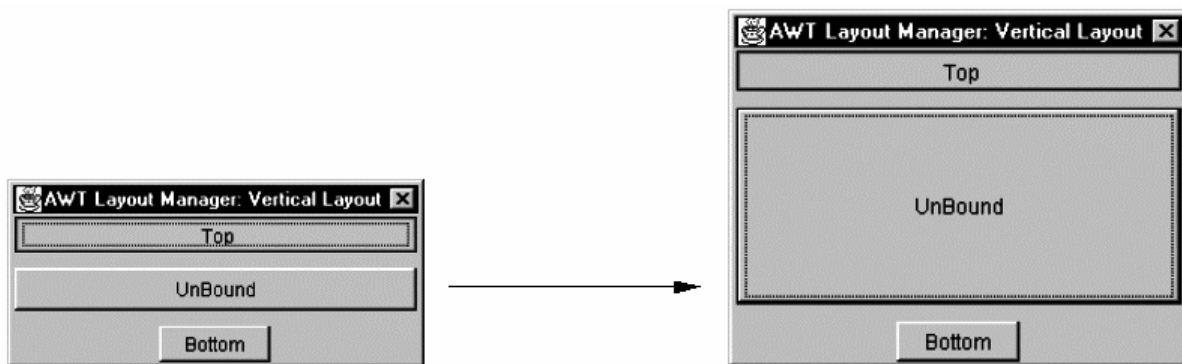
The code creates a new **buttonPanel** panel. The layout in the panel is set to **VerticalLayout** with default parameters. Three buttons are created and added to the panel. The top components are positioned first, followed by the bottom components and unbound components. All the buttons are added to the panel with a **nobind** parameter and placed in the center of the dialog box. When the dialog box is resized, the **Unbound** button is resized, but the **Top** and **Bottom** components are not.

The following figure shows the layout of the components added with a **bind** parameter. The **Top** and **Unbound** components are added with a **bind** parameter. The **Bottom** component is added with a **nobind** parameter.



Horizontal layout with components added

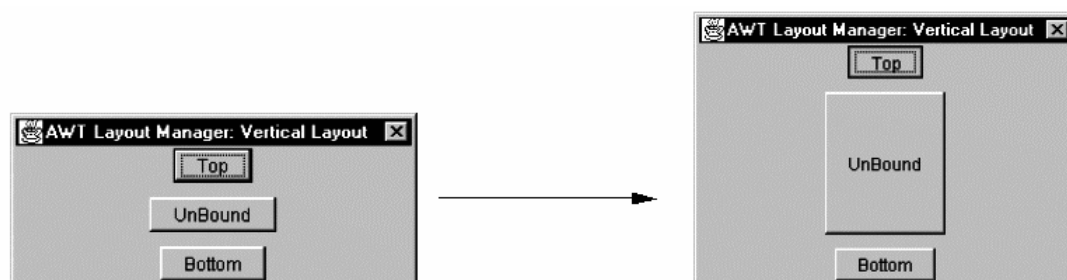
The following figure shows the resizing behavior of the components added with a **bind** parameter.



Results of resizing the dialog box

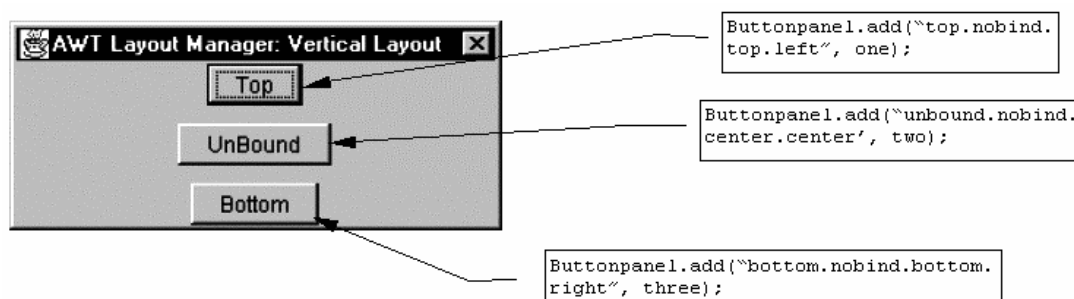
Upon resizing the dialog box, the **unbound** component is resized. The **top** component, which is added with a **bind** parameter, remains attached to the edges of the dialog box. The **bottom** component, added with a **nobind** parameter, does not change.

The following figure shows the behavior of the dialog box when resized.



Results of resizing the dialog box

The following figure shows the layout of the dialog boxes when the components are added with the parameters indicated in the figure.



Vertical layout with components added

In addition to controlling the placement of components in the dialog box, the **LayoutManager** component also controls the spacing of the components relative to the edges of the dialog box. This is achieved by the parameters passed into the **LayoutManager** constructor. The order in which the parameters are passed into the constructor is important. The constructor is:

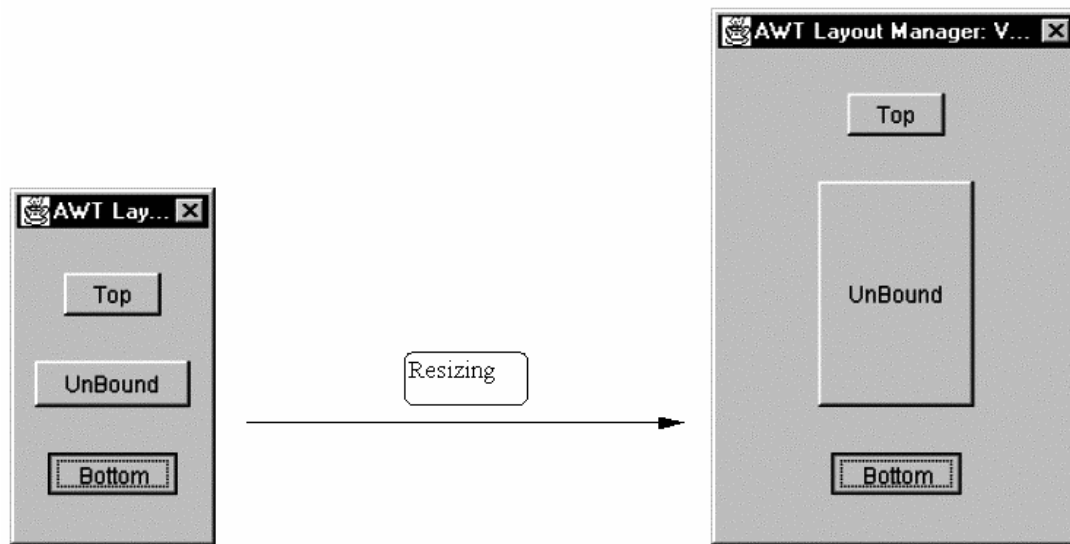
```
new VerticalLayout ( vgap, lm, rm, tm, bm);
```

vgap indicates the distance between the components in the dialog box. **lm** indicates the left margin, **rm** indicates the right margin, **tm** indicates the top margin, and **bm** indicates the bottom margin. All parameters are expressed in integers and measured in pixels.

The following figure shows the dialog box layout when the following parameters are passed to the constructor:

```
new VerticalLayout (25, 10, 10, 25, 25);
```

The behavior of the dialog box when resized is also shown in the following figure. The dialog box maintains the margins and the spacing between the components when resized.



Vertical layout with margin setup

The following figure shows the code used to create the previous examples:

```
public class testlayout extends JDialog
{
    protected JButton one, two, three;
    protected JPanel buttonPanel;
    public testlayout( Frame parent, String title )
    {
        super ( parent, title, false );
        buttonPanel = new JPanel();
        buttonPanel.setLayout ( new VerticalLayout(10,2,2,2,0));
        one = new JButton ( "Top" );
        two = new JButton ( "UnBound" );
        three = new JButton ( "Bottom" );
        buttonPanel.add ( "top.nobind.center.center", one );
        buttonPanel.add ( "unbound.nobind.center.center", two );
        buttonPanel.add ( "bottom.nobind.center.center", three);
        this.add (buttonPanel);
        this.pack ();
    }
    public static void main ( String[] args ){
        {
            Frame f = new Frame ( "Vertical Layout Test" );
            f.resize ( 100,100 );
            f.show();
            testlayout d = new testlayout ( f, "AWT Layout Manager: Vertical Layout" );
            d.show();
        }
    }
}
```

MessageBox

The **MessageBox** class communicates informational, warning, working, and error messages to the user (as shown in the following figure).



MessageBox

The **MessageBox** class is a specialized **JDialog** class that provides the ability to create and display a wide variety of message boxes to the user. Examples of the information that appears in message boxes include help, detailed messages, and general messages with icons that are set based on the type of **MessageBox** component.

The following example shows code used to create a **MessageBox** component:

```
JFrame frm = new JFrame();
JPanel displayPanel = new JPanel(new BorderLayout());
JButton invokeButton = new JButton("Invoke MessageBox");
final MessageBox msgBox = new MessageBox(frm, "Some Message", "Title", MessageBox.ERROR);

displayPanel.add("Center", invokeButton);

frm.getContentPane().add(displayPanel);
invokeButton.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        msgBox.setVisible(true);
    }
});

frm.addWindowListener(new WindowAdapter()
{
    public void windowClosing(WindowEvent e)
    {
        System.exit(0);
    }
});
frm.pack();
frm.validate();

frm.setVisible(true);
```

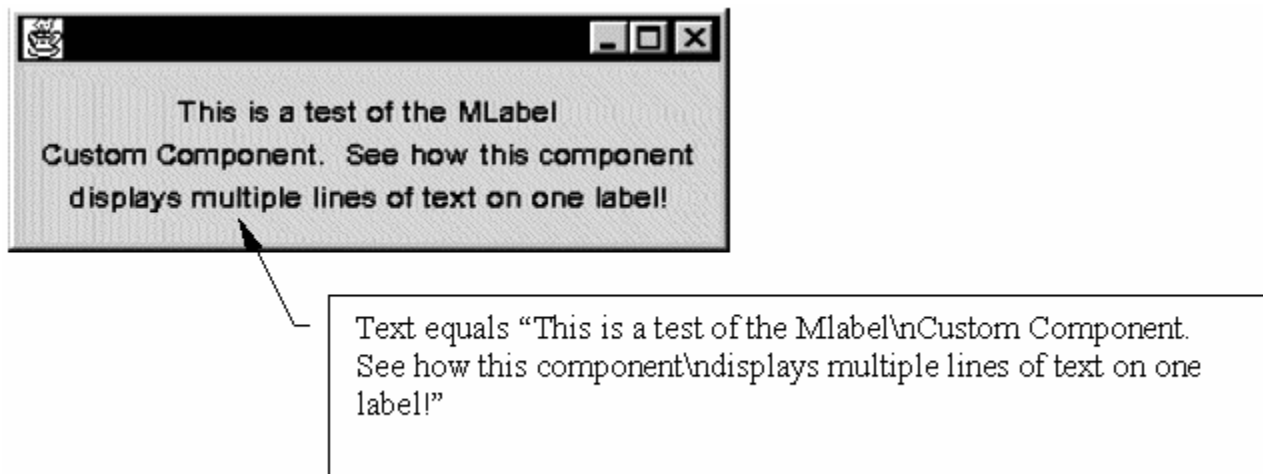
The following figure shows the message box produced by the code.



MessageBox produced from sample code

MLabel

The **MLabel** component displays text on multiple lines (as shown in the following figure). The current AWT label and **JLabel** components are only capable of displaying one line of text. The backslash and *n* character (*\n*) delimit the lines of text.

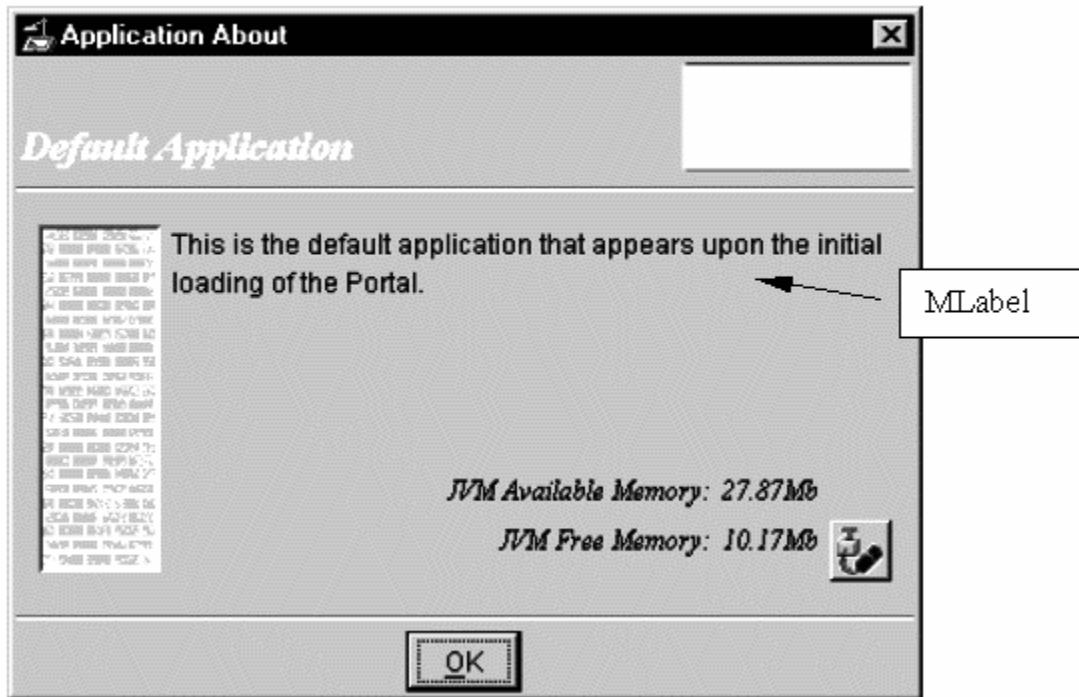


MLabel component

The code in the following example constructs an **MLabel** component:

```
int fontSize = getFont().getSize();
Font fontText = new Font("TimesRoman", Font.PLAIN, fontSize+4);
Font f2 = new Font("TimesRoman", Font.BOLD, fontSize+32);
MLabel labelBanner = new MLabel("TC Portal\nDesktop");
labelBanner.setFont(f2);
labelBanner.setTextAlignment(MLabel.CENTER);
MLabel labelVersion = new MLabel("Version 6.0\nUpdate version: 0");
labelVersion.setFont(fontText);
labelVersion.setTextAlignment(MLabel.CENTER);
```

The following figure shows the results of the code.



MLabel component produced from sample code

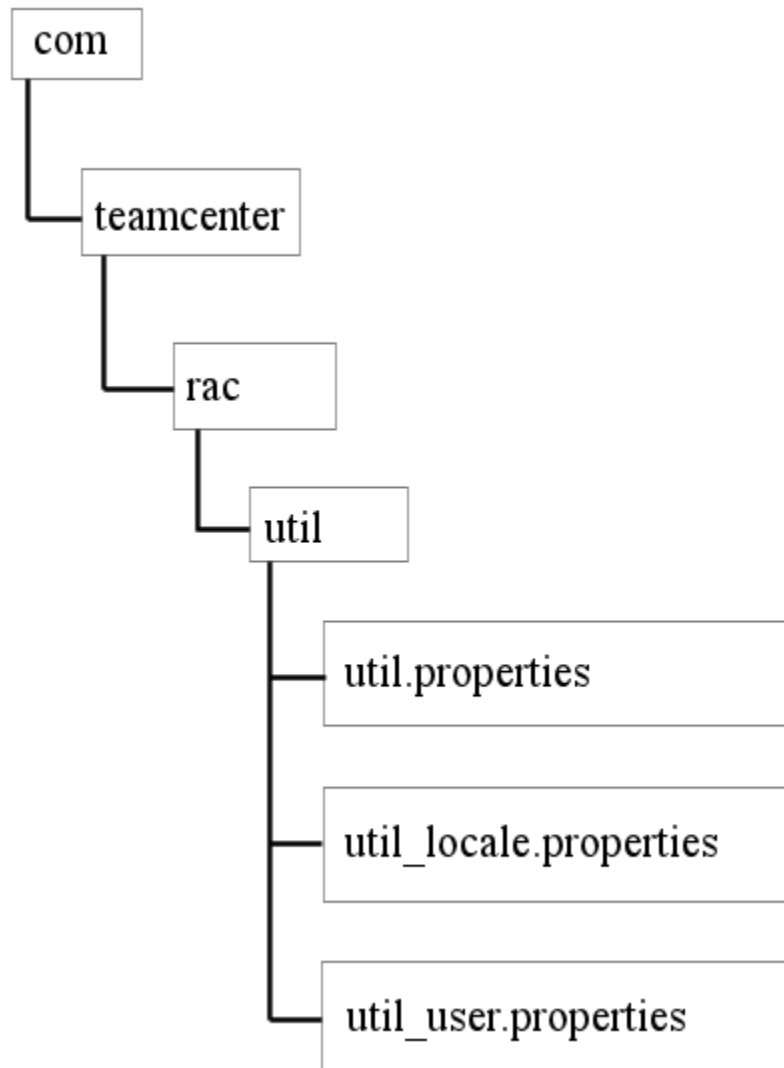
Registry

The **Registry** class contains registry information and provides a means of obtaining information stored in a resource bundle, extending the functionality of the resource bundle by way of encapsulation. This class provides the ability to native data types and instance classes via key registry entries. The registry files must match the package name.

The following code shows an example of a registry file.

```
# comments
import=com.teamcenter.rac.util
myLabel=My Label:
myIcon=images\myIcon.gif
ok=OK
ok.MNEMONIC=O
cancel=Cancel
cancel.MNEMONIC=C
```

The import statement in the preceding figure imports another registry file. It can also import multiple files using a comma delimiter. The following figure shows the hierarchy of the registry files in the **com.teamcenter.rac.util** package.



Hierarchy of the `com.teamcenter.rac.util` package

The keys and values inside the **util_user.properties** file override those defined in the **util_locale.properties** and **util.properties** files. The locale version property files are provided for localization and the user property files are provided so that users can define their own values. If no user version is defined, the default values in the **util.properties** file are used.

References to **Registry** objects are obtained by the **getRegistry()** static methods provided in this class (as shown in the following code example):

```

Registry reg = Registry.getRegistry ( this );
String label = reg.getString ( "myLabel" );
ImageIcon myIcon = reg.getImageIcon ( "myIcon" );
    The following is the corresponding registry file read by the above code:
# comments
import=com.teamcenter.rac.util
myLabel=My Label:
myIcon=images\myIcon.gif
  
```



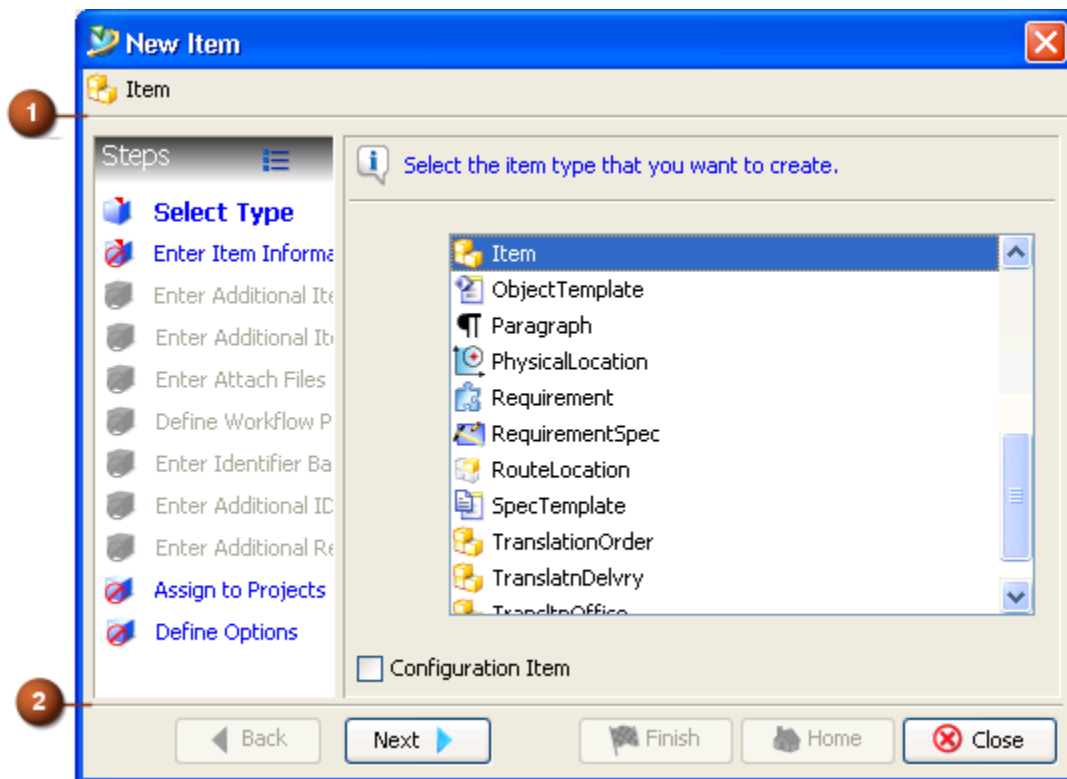
```

ok=OK
ok.MNEMONIC=O
cancel=Cancel
cancel.MNEMONIC=C

```

Separator

The **Separator** component visually separates user interface components (as shown in the following figure). Separators can be oriented either horizontally or vertically.



Separator in New Item dialog box

- 1 **Separator** used to separate dialog box header from the body
- 2 **Separator** used to separate dialog box footer from the body

The code in the following example uses the **Separator** component:

```

import java.awt.*;
import java.awt.event.*;
import java.util.*;
import javax.swing.*;
import com.teamcenter.rac.util.*;
public class SeparatorTest extends JPanel
{
    SeparatorTest ()
    {

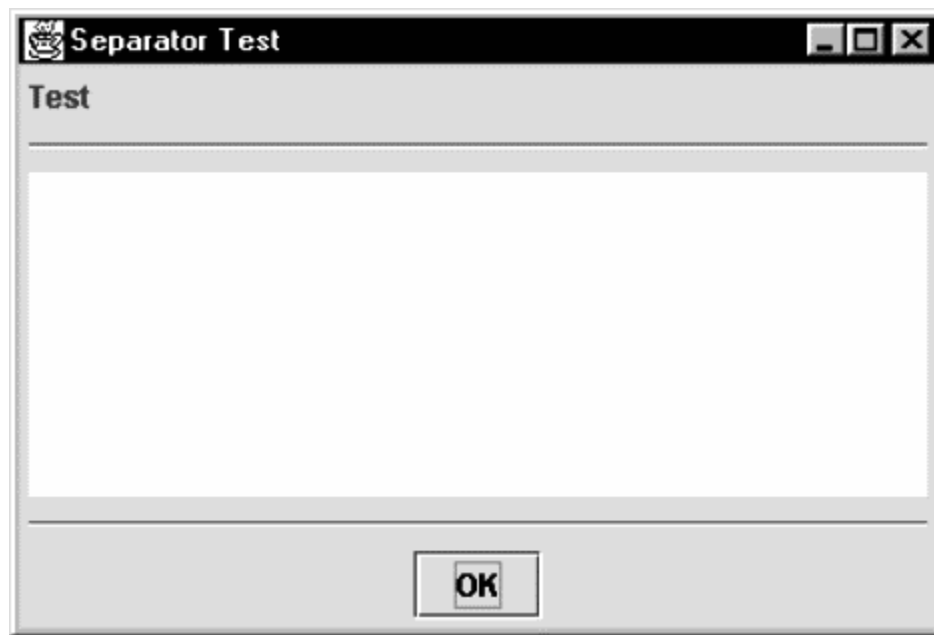
```

```

        this.setLayout ( new VerticalLayout(10,4,4,4,4) );
        JLabel label = new JLabel ( "Test" );
        JTextArea text = new JTextArea ( 10, 3 );
        JButton okButton = new JButton ( "OK" );
        this.add ("top.nobind.left.top", label);
        this.add ("top.bind", new Separator());
        this.add ("bottom.nobind.center.center", okButton);
        this.add ("bottom.bind", new Separator());
        this.add ("unbound.bind", text);
    }
    public static void main(String s[])
    {
        WindowListener l = new WindowAdapter()
        {
            public void windowClosing(WindowEvent e) {System.exit(0);}
        };
        Frame f = new Frame("Separator Test");
        f.addWindowListener(l);
        f.add("Center", new Separatortest());
        f.pack();
        f.setSize(new Dimension(350,350));
        f.show();
    }
}

```

The following figure shows the results of the code.



Separator produced from sample code

SplitPane

The **SplitPane** component is similar to the **JSplitPane** component. It extends from the **JPanel** component and creates a split pane with two panels: either left and right, or top and bottom.

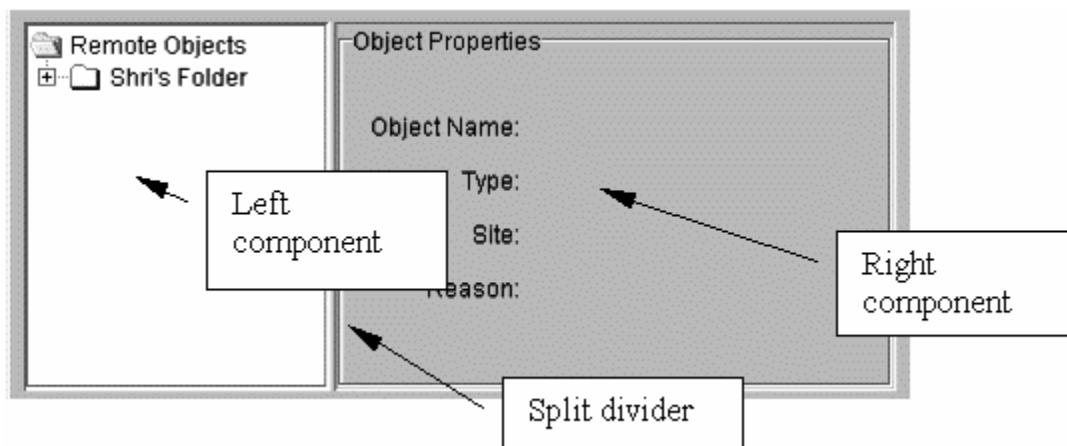
Note:

This component is a Swing class. Teamcenter is moving toward SWT/JFace as the user interface toolkit and moving away from AWT and Swing. Siemens PLM Software encourages you to customize Teamcenter using SWT/JFace components. Siemens PLM Software will discontinue Swing/AWT support in a future version.

The code in the following example creates the **SplitPane** component. It adds two components, one for the left pane and one for right. The divider is set at 45 percent of the total size of the **SplitPane** component.

```
SplitPane splitPane = new SplitPane ( SplitPane.HORIZONTAL_SPLIT );
JPanel leftPanel = new JPanel ();
JPanel rightPanel = new JPanel ();
// add components to splitPane
splitPane.setLeftComponent ( leftPanel );
splitPane.setRightComponent ( rightPanel );
splitPane.setDividerLocation(0.45);
splitPane.setDividerSize(2);
```

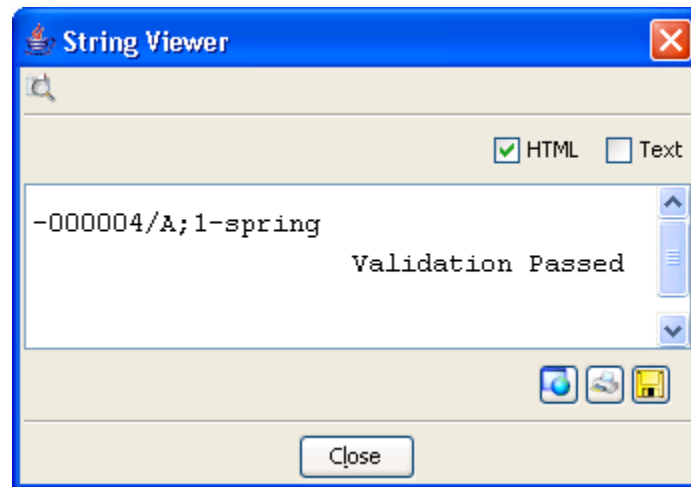
The following figure shows the pane produced by the code.



SplitPane component used in a dialog box

StringViewerDialog

The **StringViewerDialog** class loads a string array and displays text in a scrollable format. This class extends from the **AbstractDialog** class. The dialog box allows you to save and print the string array text and can also act as a simple text editor. In the following figure, the entire panel is a **StringViewerDialog** component.



StringViewerDialog component

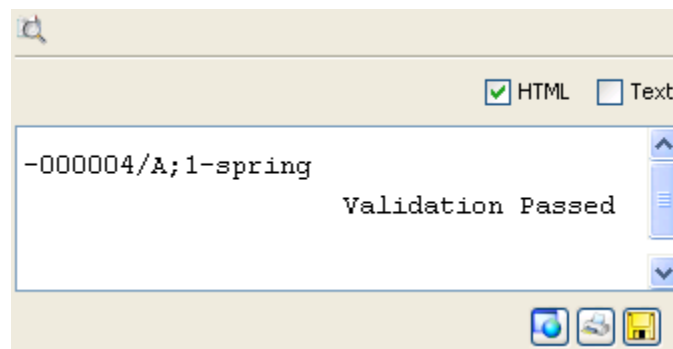
The code in the following example creates a **StringViewerDialog** component.

```
StringViewerDialog dlg = new StringViewerDialog (file);
dlg.setVisible(true);
```

StringViewerPanel

The **StringViewerPanel** component displays a text file or string array using a **JTextArea** component to store and render the text. A scroll pane ID handles text scrolling. The **StringViewerPanel** component serves as both a simple text editor and a file viewer and supports print and saving files.

The following figure shows the Validation report, constructed using the **StringViewerPanel** component.



Validation report

The code in the following example creates a **StringViewerPanel** component:

```
private StringViewerPanel stringViewerPanel = null;
stringViewerPanel = new StringViewerPanel(file);
parentPanel.add ( "unbound.bind.center.top", stringViewerPanel );
```

Common Teamcenter command IDs

Teamcenter command IDs have many uses. The most common being to add a command to a rich client menu or toolbar, or adding a command to a page or objectSet using an XML style sheet.

Note:

- The preferred method to locate commandIDs is to use the rich client command-line utility **DumpCMSConfigInfo**. It will generate a **CSV** file containing an accurate listing of command IDs for your specific rich client configuration, as well as one for your view IDs, and one for your application context IDs as well.

```
teamcenter -application com.teamcenter.rac.util.DumpCMSConfigInfo
```

- When a command to create a new object is added within the **objectSet** tag, only commands for **New Item**, **New Dataset**, **New Part**, and **New Other** actions paste the new object with the relation specified in the **<objectSet source="relation.business-object">** tag. All other new object types use the default paste relation.

The following code example shows creating a new object with the **com.teamcenter.rac.newDesign** command ID. However, the default paste relation is used instead of the **IMAN_specification** relation in the **objectSet source** tag:

```
<objectSet source = "IMAN_specification.Design Revision "
defaultdisplay = "tableDisplay" sortBy = "object_string" sortdirection =
"ascending">
.
.
.
<command actionKey = "newDesignAction" commandId = "com.teamcenter.rac.NewDesign"
renderingHint = "commandbutton"/>
```

The following tables show a few common examples of a menu command, and their associated command IDs.

My Teamcenter File→New menu commands

Menu command	Command ID
File→New→Item	com.teamcenter.rac.newItem
File→New→Folder	com.teamcenter.rac.newFolder

My Teamcenter Edit menu commands

Menu command	Command ID
Edit→Cut	org.eclipse.ui.edit.cut
Edit→Copy	org.eclipse.ui.edit.copy
Edit→Paste	org.eclipse.ui.edit.paste

Plug-in locations of perspectives

The following table lists plug-ins that define perspectives in the rich client. If you want to customize rich client perspectives, look for these plug-ins in the `TC_ROOT\portal\plugins` directory. The perspectives are declared in the plug-ins with the `org.eclipse.ui.perspectiveExtensions` extension point in the `plugin.xml` file.

To hide perspectives, you can use the `HiddenPerspectives` preference.

Plug-in locations of perspectives

Perspective	Plug-in
Access Manager	com.teamcenter.rac.accessmanager
ADA License	com.teamcenter.rac.adalicense
Appearance Configuration	com.teamcenter.rac.appearanceconfiguration
As-Built Manager	com.teamcenter.rac.asbuiltmanager
Audit Manager	com.teamcenter.rac.auditmanager
Authorization	com.teamcenter.rac.authorization
CAE Manager	com.teamcenter.rac.caese com.teamcenter.rac.smb
Change Manager	com.teamcenter.rac.cm
Classification	com.teamcenter.rac.classification.icm
Classification Admin	com.teamcenter.rac.classification.icadmin
CM Viewer (Classic)	com.teamcenter.rac.ecmanagement
Command Suppression	com.teamcenter.rac.commandsupporession

Perspective	Plug-in
Database Utilities	com.teamcenter.rac.databaseutilities
Design Context	com.teamcenter.rac.designcontext
DPV Measurements	com.teamcenter.rac.dpv
Getting Started	com.teamcenter.rac.aifrcp
Issue Manager	com.teamcenter.rac.issuemanager
Launch Pad	com.teamcenter.rac.launchpad
Lifecycle Viewer	com.teamcenter.rac.vis
Manufacturing Process Planner	com.teamcenter.rac.cme.mpp
Multi-Structure Manager	com.teamcenter.rac.cme.collaborationcontext
My Teamcenter	com.teamcenter.rac.ui
Organization	com.teamcenter.rac.organization
Part Planner	com.teamcenter.rac.cme.pmp
Plant Designer	com.teamcenter.rac.cme.fse
Platform Designer	com.teamcenter.rac.architecturemodeler
PLM XML/TC XML Export Import Administration	com.teamcenter.rac.plmxmlimportadministration
Project	com.teamcenter.rac.project
Query Builder	com.teamcenter.rac.querybuilder
Registry Editor	com.teamcenter.rac.aif.registryeditor
Relation Browser	com.teamcenter.rac.tcgrb
Report Builder	com.teamcenter.rac.crf
Report Generator	com.teamcenter.rac.cme.cmereport
Systems Engineering	com.teamcenter.rac.requirementsmanager.win.embeddedword
Resource Manager	com.teamcenter.rac.cme.mrm
Schedule Manager	com.teamcenter.rac.schedule
Service Manager	com.teamcenter.rac.servicemanager
Service Planner	com.teamcenter.rac.serviceplanner
Setup Wizard	com.teamcenter.rac.setupwizard

Perspective	Plug-in
Structure Manager	<code>com.teamcenter.rac.pse</code>
Subscription Administration	<code>com.teamcenter.rac.subscriptionmonitor</code>
Systems Engineering	<code>com.teamcenter.rac.se.ui</code>
Validation Manager	<code>com.teamcenter.rac.validation</code>
Volume Management	<code>com.teamcenter.rac.vm</code>
Web Browser	<code>com.teamcenter.rac.aifrcp</code>
Workflow Designer	<code>com.teamcenter.rac.workflow.processdesigner</code>
Workflow Viewer	<code>com.teamcenter.rac.workflow.processviewer</code>

Application Integration Framework (AIF)

AIF customization and development

In versions prior to Teamcenter 2007, the rich client ran on its own Application Integration Framework (AIF). The AIF and rich client were both written using Java. The rich client used Swing for its user interface and all customization mechanisms were Teamcenter-developed.

In versions prior to Teamcenter 2007, the Application Integration Framework (AIF) was an integration framework that enabled developers and customers to build custom interfaces to applications for Teamcenter, NX, and personal use. This framework provided the foundation through which applications could be launched and executed in a standard manner. It also provided the basic design for applications, the base classes and methods, and a methodology for creating and handling events generated by the user interface. In addition, the AIF provided tools to handle the registration of components, represented by JavaBeans, as well as a mechanism for locating and passing messages to those components.

This framework allowed products to be built using a standard interface and methodology. It was modular and dynamic with regard to how applications were registered and launched, allowing applications to be written in a plug-and-play manner and to be released independently of the framework. Applications could be independent of both Teamcenter and NX, but the framework provided simple mechanisms to use Teamcenter as the primary data management system. The **AIFDesktop** component provided the main window users interacted with during the course of their session.

In Teamcenter 2007, the rich client was hosted within the Eclipse rich client platform (RCP) framework. The RCP is a general purpose application framework that provides strong support for modular and extensible component-based development through the use of plug-ins. The rich client took limited advantage of the Eclipse framework. There was a single Teamcenter perspective that all Teamcenter applications used. The user interface was mostly Swing running with the aid of the SWT_AWT bridge.

Starting with Teamcenter 8, the rich client took advantage of many Eclipse features and introduced many SWT-based controls. Some of the rich client changes include:

- Each application became its own perspective.
- Menus were declarative.
- Eclipse OSGi services were used.
- Many Teamcenter extension points, services, and SWT controls were added.

Note:

Teamcenter still supports the AIF in the Eclipse RCP. However, you should make every effort to migrate your customization to use the Eclipse RCP menu, toolbar, and status bar functionality.

Integrating the Application Integration Framework (AIF) desktop with the Eclipse workbench

If you have older AIF desktop customizations, you can integrate them with the newer Eclipse RCP. However, although Teamcenter still supports the AIF in the Eclipse RCP, you should make every effort to migrate your customization to use the Eclipse RCP menu, toolbar, and status bar functionality.

In the rich client, the main integration point is an *application*. The active application defines what is showing and its layout in the main pane area. The current application controls the contents of the main menu bar and toolbar. The navigation pane is always on and helps you to select the current application. In the rich client, there is a main banner that identifies the current application and how to switch between active applications. The main mechanism for defining the set of applications has been the **portal.properties** file. This file is still supported and is augmented by the **aif_application** extension point. The extension point supports three different types of scenarios:

- Adding new traditional or legacy-based applications without modifying the **portal.properties** file.
- Adding new pure SWT-based applications.
For more information about how pure SWT user interface components work together in a workbench, see the following Web site:

<http://www.eclipse.org/articles/Article-UI-Workbench/workbench.html>

- Adding hybrid applications or defining existing legacy applications to be hybrid to allow Swing-based user interface components from within an Eclipse perspective that can also manage pure SWT-based user interface components simultaneously. Though these components can co-exist, there is no additional infrastructure for these different user interface components to collaborate.

The rich client desktop is built on top of the Eclipse RCP workbench, including the menubar, toolbar, and status bar. The RCP workbench is augmented with additional shell trim that defines the main application switcher/banner bar and the navigation pane. The remaining area is the current active Eclipse

perspective. By default, there is a simple Teamcenter perspective that simply holds a tabbed stack of views. Each rich client application is forced to be associated with an Eclipse perspective. Whenever a rich client application is activated, the associated Eclipse perspective is made active. By default, an application is associated with the Teamcenter perspective so legacy applications are not forced to define a perspective to be associated with. If any application also contains a non-null main Swing JPanel, that panel is wrapped in an AWT_SWT bridge view and placed in the stack of views. Since each rich client application is associated with an Eclipse perspective, you might want to read more about what an Eclipse perspective is.

For more information, see the following Web site:

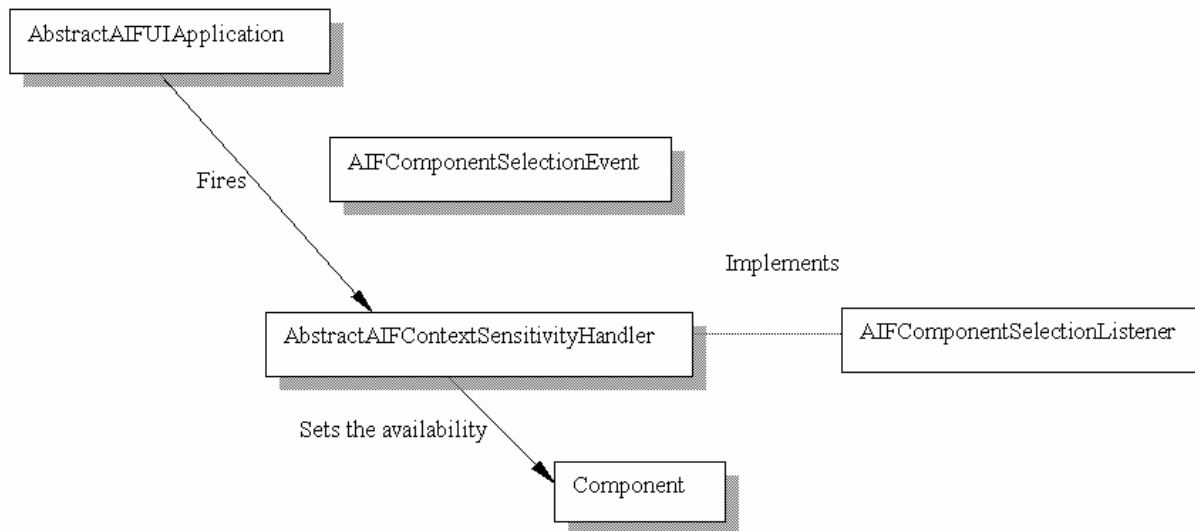
<http://www.eclipse.org/articles/using-perspectives/PerspectiveArticle.html>

AIF context sensitivity

Introduction to AIF context sensitivity

The Application Integration Framework (AIF) supports the concept of context sensitivity for all UI components. Context sensitivity is controlling the availability of UI components when certain conditions or states exist. In some cases, context sensitivity can be confusing to the user because he or she may not know what to do to activate an option. One of the techniques currently used to solve this is to change the tooltip text.

The model for the context sensitivity system is tied to the application where the action appears. There is a selection listener associated with the application such that when something is selected within the application, the application notifies all listeners that a selection is changed. The idea behind this model is that the application fires the event to all listeners and each UI component has an associated *handler* that knows the UI component it is working for. The handler contains the logic that determines the validity of the UI component with which it is associated. In basic terms, the handler typically sets a component to disabled (**setEnabled(false)**). However, the handler is flexible and is designed to allow any action to be taken upon the associated UI component. For example, instead of disabling a component you may want to write a handler that sets a UI component visibility based upon a certain state. The handler interrogates the application for the state, and based on the result invokes the UI components **setVisible()** method. The handler simply implements one method. That method is responsible for making the verdict and applying the appropriate action to the UI component.



Context Sensitivity object model

AIF context sensitivity registration

The registration process is where the application, handler, and UI component are bound together based on the context sensitivity object model. Each application maintains a list of handlers (listeners) that are notified whenever a selection is made within the application.

To register a handler (listener) with an application, invoke the following:

```
// c is an instance of a Component
MyHandler h = new MyHandler ( c );
app.addSelectionListener ( h );
```

This is typically done within the menu bar and toolbar construction. This can also be done within the application panel construction for components located there.

There is a special implementation for Teamcenter applications that utilizes the action system for the menu bar and toolbar. Register your handler within the **actions.properties** file, and it is automatically used with the associated command. Therefore, to register the simple **RequiredSelectionHandler** handler, which is prepackaged for use, add it to your action definition within the **actions.properties** file, as follows:

```
<yourCommand>.SELECTION_HANDLER=
com.teamcenter.rac.aif.common.contextsensitivity.RequiredSelectionHandler
```

No other registration is required for the command. When the selections change within the application, your command is available only when something is selected. If nothing is selected, it is not available.

If you want to keep components within the toolbar and menu bar synchronized like a state selection, you can use this mechanism by manually triggering the selection event and having your handlers refer to a class variable within the application for the state. The system is written generically so that it can handle a variety of cases.

Write a handler for AIF context sensitivity

Base AIF provides the framework for context sensitivity as well as one implemented handler, **RequiredSelectionHandler**. The **RequiredSelectionHandler** handler looks at the given application, and if one or more components are selected, the associated components are enabled. Otherwise, the component is disabled. This is enabled for many actions that currently appear within the menu bar and toolbar, more notably the cut, copy, and paste actions.

Use the following steps to write your own handler:

1. Write a class that subclasses the **AbstractAIFContextSensitivityHandler** class.
2. Provide the implementation method within the class public void **componentSelected** (**SelectionEvent e**). The implementation of the **componentSelected()** class interrogates the application to get the desired data and takes the appropriate action, such as checking the number of components selected within the application and setting the state of the component. The following code shows the source for the **RequiredSelectionHandler** handler:

```
public class RequiredSelectionHandler extends
AbstractAIFContextSensitivityHandler
implements AIFComponentSelectionListener
{
    public RequiredSelectionHandler ( Component cmp )
    {
        super ( cmp );
    }
    public void componentSelected ( AIFComponentSelectionEvent e )
    {
        AbstractAIFUIApplication a = e.getApplication();
        Component c = getComponent();
        if ( a.getTargetContexts() != null )
        {
            c.setEnabled ( true );
        }
        else
        {
            c.setEnabled ( false );
        }
    }
}
```

3. Register your handler with the action by either adding to the registry or manually registering it within the UI parent in which it is contained (menu bar, toolbar, panel).

To manually initiate the firing of a selection event, invoke the **fireSelectionEvent()** method within the application (**a.fireSelectionEvent()**).

4. Thin client customization

Introduction to thin client customization

The Teamcenter thin client is based on a client-server architecture. Both the client and server layers can be customized. The client is the user interface (UI) layer and can be customized using JavaScript and other methods. The server layer can be customized using many methods, including TcScript, the Integration Toolkit (ITK), and the C++ programming language.

To fully understand the inner workings of the thin client for large-scale customizations, you should explore the code. The standard, mainstream methods and paradigms employed by the thin client are covered in this guide, but there are many exception cases where one or more standard practices have been substituted with an alternate approach to achieve an alternate result. If the standard interfaces documented here do not cover your special case, your knowledge of the existing user interface is invaluable. Try identifying a similar special case in the user interface and tracing how it works.

Customizing navigation pane and menu entries

The thin client presents functionality via a navigation pane and menus that can change dynamically according to the application in use. Three basic types of actions are performed within this interface:

- Navigate to a new URL, replacing the current page, for example, navigating from your worklist to your home folder.
- Open a dialog box within the current page, for example, opening the **New Item** dialog box using the **New→Item** command.
- Perform an action within the current page, for example, cut or copy an object from or to another object.

A menu command that requires an object to be selected before the command can be performed is called a *context-sensitive* menu command. A menu command that is not dependent on object selection is called a *context-insensitive* menu command. Determining placement of menu names and commands is often decided based upon desired placement of context-sensitive and context-insensitive commands.

Menu names, commands, and behavior can be configured using XML files stored in the following location inside the **tc.war** file:

teamcenter/dhtml/common/intl/en_US

This directory defines all US English menu entries, left-hand navigation (LHN) panels, and tools. If your site uses localization for other languages, add additional directories using standard Java locales. For example, if your site uses German, use the **teamcenter/dhtml/common/intl/de_DE** directory. For French, use the **teamcenter/dhtml/common/intl/fr_FR** directory, and so on.

toolbar.xml

Defines all menu entries and tools.

All context-sensitive commands are placed in the menu bar, and all context-insensitive commands in the navigation pane. This ensures that all navigation commands behave consistently regardless of the object selected.

Files like changemanagermenu.xml and datasetmenu.xml

Define the OOTB menu configurations for different application pages.

Files like changemanagermenu_custom.xml and datasetmenu_custom.xml

Define your custom menu configurations for different application pages.

Use the configurations defined by the XML files in the language directories to optimize the thin client configuration at your site. Menu and navigation commands can be added or removed site-wide.

User-based, group-based, or role-based customization of the XML menu files is not supported using this method. Use menu entry suppression instead.

For information about configuring menu and toolbar commands, see the [Application Administration](#).

Modifying browser behavior

You can control whether a menu command launches a new browser or updates a specific browser. For example, the **toolbar.xml** file configures online help to open in a new browser by default. When users click on **Web Collection** in the navigation pane, the online help launches in a second window. Click **Web Collection** again and a third window opens.

To prevent this behavior, modify the toolbar subentry, defining the **target** value with a specific browser to be used. For example:

```
tool title="Help" image="/tc/images/lhn_help.gif"
subentry title="Web Collection" url="/cgi-bin/tc?TC_file=
redirs/tcwebhelp" target="tchelp"/
subentry title="General Collection"
url="/cgi-bin/tc?TC_file=redirs/tcwebhelp&core=y" target="tchelp"/
subentry title="About Teamcenter..."
url="javascript:popupAboutDialog();" /
/tool
```

In this example, the **target** value is set to **tchelp**.

The **target** attribute is valid only on the **menuentry**, **subentry**, and **tool** entries. The attribute only takes effect when its respective menu entry is listed in the **WEB_menu_entry_new_window** preference; otherwise, it is ignored.

Generating a thin client page

Process for generating a thin client page

This detailed process shows the end-to-end flow of a typical request/response transaction in the Teamcenter thin client:

1. The user chooses a menu command.
For more information, see *Choosing the menu command and displaying the dialog box*.
2. A dialog box is instantiated for user input.
3. The user enters data in the dialog box and submits it by clicking either the **OK** or **Apply** button.
For more information, see *Submitting the data and submitting the Web request*.
4. A server-side request is initiated.
For more information, see *Processing on the Teamcenter server*.
5. The selected action is performed.
6. The results are displayed as feedback to the user.
For more information, see *Receiving the response and displaying feedback*.

For example, if the user creates a new folder, the process is:

1. The user chooses **New→Folder** in My Teamcenter.
2. The **New Folder** dialog box appears.
3. The user types a name and a description for the folder, selects a type from the list, and clicks **OK**.
4. A request to the server is generated that includes the data from the dialog box and the appropriate insertion point.
5. The new folder is created on the server and attached using ITK.
6. An XML response is returned from the server with either the data to insert the new folder in the user interface or an error message.

If you review the flow of this typical example, it helps you understand how the thin client works. You can follow this example in detail to see how the infrastructure components work together to implement this behavior.

Choosing the menu command and displaying the dialog box

To see the association between the menu entry and the JavaScript, see the following example:

```
<menuentry title="Folder..." key="newFolderAction"
url="javascript:popupNewFolderDialog();"
image="teamcenter/dhtml/icons/folder.png">
</menuentry>
```

Note that menu commands can execute arbitrary JavaScript. The menu XML is delivered to the client and rendered as HTML with the JavaScript action attached. In this case, the **popupNewFolderDialog()** function is called when the user chooses the **New→Folder** menu command.

Note:

Most menu entries are defined in the XML files, but the **New** menu entries are the exception. The **New** menu items are read from the **WEB_displayed_new_menu_objects** preference, as shown by the following code in the line in the *staging_location\webapp_root\teamcenter\dhtml\common\intl\language\wsomenu.xml* file:

```
<menu type="new" title="New">
  <menu type="NewmenuPlaceHolder" desc="Placeholder for menu entries of
WEB_displayed_new_menu_objects preference"></menu>
```

The menu XML is delivered to the client and rendered as HTML with a JavaScript action attached. In this case, the **popupNewFolderDialog()** function is called when the user chooses the **New→Folder** menu command.

The **popupNewFolderDialog()** function is based on a standard dialog box invocation template function that is used in many places. It maintains a single dialog box instance; it is not useful to have more than one open **New Folder** dialog box at a time. In the *staging_location\webapp_root\teamcenter\dhtml\apps\dialogs\new.js* file (shown in the following code sample), a global variable tracks the existence of the **New Folder** dialog box.

```
// new folder dialog is a singleton
var globalNewFolderDialog = null;

function popupNewFolderDialog( context )
{
    var selcon = checkSelCon( context );
    var form;
    var browserName=getBrowserName();

    if( checkDialogReset( selcon, globalNewFolderDialog ) )
    {
        if(browserName == "IE")
        {
```



```

        form = newfolderform.XMLDocument.documentElement;
    }
    else if(browserName == "Netscape")
    {
        form=getFormByName(getString("web_newfolder_title"));
    }

    globalNewFolderDialog = createActionDialog( form,
                                                "common/actions/newfolder.xml",
                                                selcon,    // take selection?
                                                true );    // have apply button
    }
    globalNewFolderDialog.popup();
}

```

If the dialog box already exists, the code displays it instead of creating a new one. The dialog box is implemented using Teamcenter **actionDialog** code. The **actionDialog** code generates a generically abstracted dialog box that incorporates behaviors used by many Teamcenter dialog boxes, so the code needed to create an instance of the dialog box is terse.

The description of the form itself is in the form renderer XML format required for the **actionDialog** code. The XML data is held within a data island in the HTML page structure. Many dialogs boxes take a server trip to assemble the appropriate form (dynamic dialog boxes as opposed to static). This occurs if performance is an issue (for example, the **User Settings** dialog box), or if Teamcenter suggests values based on the selected object (for example, the **New Process** dialog box).

The following sample shows the generated XML form code:

```

<form name="New Folder">
<field name="name" displayname="Name" required="true" />
<field name="description" displayname="Description" />
</form>

```

The **createActionDialog** method takes this XML code as input and renders it as HTML. The **popupNewFolderDialog()** function calls the **popup** method on the dialog box instance. This displays the dialog box to the user.

New Folder dialog box

Submitting the data and submitting the Web request

After the user fills in the form and clicks **OK**, the **actionCall** function in the generic **actionDialog** code is executed using the DHTML **onclick** event attached to the **OK** button. The **actionCall** function builds a request URL based on the associated action (the **newfolder.xml** file), the form input values, and the currently selected object from the tree. The associated action was an argument to the **actionDialog** code earlier; the form values are collected by the **buildURL()** method in the *staging_location* **\webapp_root\teamcenter\dhtml\common\formrenderer.js** file, and the currently selected object from the tree is taken using an interface to the **treeRenderer** code. The request URL arguments look like the following:

```
tc_file=actions/newfolder.xml&_name=example&_description=&_type=Folder&parent_uid=
wcysT3taAAAMeC
```

The associated action argument specifies a given script to execute on the server. To compare it to a function call, the **newfolder.xml** argument is similar to the API function name; the rest of the URL is similar to the argument list. This creates a request object suitable for submission to the XML loader.

Up to this point, all operations are on the client side. The XML loader is the module that submits requests to the server. Teamcenter allows only a single logical request to be active at a time with a blocking and queuing mechanism because Web users prefer to avoid multiple requests, even though JavaScript is inherently threaded and the Teamcenter pool manager can queue multiple requests and serialize them on the server side.

Processing on the Teamcenter server

The HTTP server transmits a request to the Teamcenter pool manager. The pool manager then queues the query until an available TcScript process is ready to accept it. For more detailed information, see [Getting Started with Customization](#).

The **tcserver** process executes the **newfolder.xml** script, calling ITK as appropriate, to create the new folder and to attach it to its specified parent. There are several ITK functions needed to create and save a folder, including the **FL_create** function. You can see the actual script in the **TC_ROOT\web\htdocs\tc\common\actions\newfolder.xml** file. If an error occurs during the script processing, an exception is thrown and a POM rollback occurs to maintain the all-or-nothing transactionality.

Depending on the result, the script outputs either an XML refresh message or an XML error stack to the client. When creating a folder, the refresh message format is *refresh this parent object and insert this new folder*. The first message ensures that the last modified dates and users are synchronized. The following code sample shows a typical success response:

```
<?xml version="1.0" encoding="iso-8859-1"?>
<update>
<refresh>
<line tag="wcysT3taAAAMeC " name="Home" link="/tc/webclient/wcysT3taAAAMeC" type="Folder"
  class="Folder">
<prop name="actions"/>
<prop name="object_type">Folder</prop>
<prop name="owning_user" link="/tc/webclient/Q_wsTuYwAAAMeC" class="User">phin</prop>
<prop name="owning_group" link="/tc/webclient/Qc9YS80iAAAMeC" class="Group">
  MyLocation.MyDepartment.TC.P&O.MyCompany</prop>
<prop name="last_mod_date">02-Nov-2008 11:13</prop>
<prop name="checked_out"></prop>
<prop name="release_status_list"></prop>
<children type="Folder" link="/tc/webclient/ wcysT3taAAAMeC?TC_file=data/wsochildren.xml"/>
<children type="WhereReferenced" link="/tc/webclient/ wcysT3taAAAMeC?
  TC_file=data/wherereferenced.xml"/>
</line>
</refresh>
<insert parent_uid="wcysT3taAAAMeC">
<line tag="A6$segTGAAAMeC" name="example" link="/tc/webclient/A6$segTGAAAMeC" type="Folder"
  class="Folder">
<prop name="actions"/>
<prop name="object_type">Folder</prop>
<prop name="relation"/><prop name="object_desc"></prop>
<prop name="owning_user" link="/tc/webclient/Q_wsTuYwAAAMeC" class="User">phin</prop>
<prop name="owning_group" link="/tc/webclient/Qc9YS80iAAAMeC" class="Group">
  MyLocation.MyDepartment.TC.P&O.MyCompany</prop>
<prop name="last_mod_date">02-Nov-2008 11:13</prop>
<prop name="checked_out"></prop>
<prop name="release_status_list"></prop>
<children type="GRM" link="/tc/webclient/A6$segTGAAAMeC?TC_file=data/wsochildren.xml"/>
<children type="WhereUsed" link="/tc/webclient/A6$segTGAAAMeC?TC_file=data/whereused.xml"/>
<children type="WhereReferenced" link="/tc/webclient/A6$segTGAAAMeC?TC_file=
  data/wherereferenced.xml"/>
</line>
</insert>
</update>
```

If the new folder cannot be created, an **errorstack** message is returned with the XML code of the error stack.

After the message is sent, the server script completes and the TcScript process signals its availability to perform another query.

Receiving the response and displaying feedback

When the message is received, a generic update callback specified by the **actionDialog** code processes the XML, updates the parent folder, and dynamically inserts the new folder into the DOM. For details, see the *staging_location\webapp_root\teamcenter\dhtml\common\update.js* file. For each type of update message, the appropriate action is performed. In the new folder example, an insert message containing details of the new folder is received. The new folder is inserted into the tree below the appropriate parent. In addition, a refresh message is received that updates the parent line with any new data. If the creation of the new folder fails, an **errorstack** message is received. This is interpreted by the refresh callback, and an HTML alert dialog box is displayed to the user with the errors.

If the user clicks **OK**, the dialog box closes on success. If there is an error, the dialog box remains open so the user can correct the error. If the user clicks **Apply**, the dialog box remains open.

Basic thin client customization

Thin client customization recommendations

Basic thin client customization tasks include customizing the menus, style sheets, and icons.

When customizing Teamcenter, put as much of your customization into the core code and as little into the user interface as possible. If your installation uses multiple interfaces, this minimizes duplication. Also, use simpler configuration and tailoring techniques before using custom code.

Whenever possible, avoid making changes to existing source files. Instead, create new custom JavaScript files for implementing new functionality under the *staging_location\webapp_root\teamcenter\dhtml\custom* directory. When including custom JavaScript files (for example, **custom_form1.js**), add them to the *TC_ROOT\web\htdocs\tc\custom\custom_head.ish* file (shown in following code sample) instead of the **head.ish** file:

```
print '<script type="text/javascript"
src="teamcenter/dhtml/custom/custom_form1.js"></script>' + NL
```

If you do this, you do not need to merge the old and new **head.ish** files when you apply a maintenance patch or upgrade to a new version.

Tip:

A free Firefox add-on called Firebug is especially useful. The three most useful features are:

- **Net**

View the HTTP requests and responses.

- **Console**

View the in-page **XMLHttpRequest** (XHR) calls and the XML responses.

- **Script**

Debug your JavaScript.

Top-level pages

Top-level pages are Teamcenter Web requests that are returned within an HTML wrapper. Every Teamcenter Web application, such as My Teamcenter and Structure Manager, has a top-level page that initializes the client. Most actions do not depend on how the browser achieves this initial state. After the top-level page loads, subsequent actions within the page, such as tree expansions and dialog box instantiations, result in requests for XML that are handled within the page. The initial HTML returned consists of the following:

- JavaScript files to cache on the client (client-side components)
- Small XML data islands
- Menu string data (user/group/role/version)
- Menu entry suppression data
- Special search menu data
- In-page JavaScript for rendering the current page from the Teamcenter data XML
- XML menus
- A large XML data island containing the Teamcenter data to be rendered
- A small amount of HTML code to create the basic page structure
- Another data island containing the XML description of the static forms worth caching on the client

To see the HTML code, view the browser source for one of the pages. Most HTML code is generated by the `TC_ROOT\web\htdocs\tc\common\head.ish` file. If you need to include common HTML elements

across the entire HTML layout, add them to the `TC_ROOT\web\htdocs\tc\custom\custom_head.ish` file. The in-page JavaScript contains an **onLoad()** function initiated by the browser **onLoad** event. This function renders the Teamcenter XML data island as displayable HTML code.

Directory structure

There are two kinds of files: static and dynamic.

Static files are consumed directly and cached by the browser. These include **.js** files, image files (**.gif**, **.jpg**, and **.png**), some static **.xml** files for menus (all in the **intl** directory), and **.css** files. You choose the thin client staging directory (*staging_location*) during the J2EE Web tier installation.

For more information, see the [Windows Server Installation](#). You can find JavaScript files in the subdirectories of the *staging_location\webapp_root* directory.

For example, the **formrender.js** file is located in the *staging_location\webapp_root\teamcenter\dhtml\common* directory. The following table shows the directories that contain the various types of static files.

File type	Directory
Style sheets	<code>webapp_root\teamcenter\dhtml\common\css</code>
Common components (for example, the renderers)	<code>webapp_root\teamcenter\dhtml\common</code>
Common dialog code	<code>webapp_root\teamcenter\dhtml\apps\dialogs</code>
Internationalized files and menus	<code>webapp_root\teamcenter\dhtml\common\intl\language</code>
Teamcenter type icons	<code>webapp_root\typeicons</code>
Teamcenter icons	<code>webapp_root\teamcenter\dhtml\icons</code>
Images	<code>webapp_root\teamcenter\dhtml\images</code>
Custom JavaScript files	<code>webapp_root\teamcenter\dhtml\custom</code>

Dynamic files are written in the TcScript language and interpreted at run time by the **tcserver** process. They have **.xml**, **.isx**, **.ish**, **.html**, and **.is** extensions and reside in the `TC_ROOT\web\htdocs\tc` directory. The directory for dynamic files is created when Teamcenter is installed. The following table shows the directories that contain the various types of dynamic files.

File type	Directory
Application files	<code>TC_ROOT\web\htdocs\tc</code> subdirectories such as workspace , inbox , and pse
Common XML data retrieval scripts	<code>TC_ROOT\web\htdocs\tc\common\data</code>
Common XML action scripts	<code>TC_ROOT\web\htdocs\tc\common\actions</code>
Application-specific XML data retrieval scripts	<code>TC_ROOT\web\htdocs\tc\application\data</code>
Application-specific XML action scripts	<code>TC_ROOT\web\htdocs\tc\application\actions</code>
Redirection scripts	<code>TC_ROOT\web\htdocs\tc\redirs</code>

Deploying thin client customization changes

In the thin client, files are either static or dynamic:

- **Static files**
 Static files are deployed in the Web tier and are consumed directly and cached by the browser. These files include JavaScript (**.js**), images (**.png**, **.gif**, **.jpg**), a few static XMLs for menus under the **intl** directory, and cascading style sheets (**.css**).
 To deploy static file changes to the thin client, regenerate the **tc.ear** file using the Web Application Manager, and deploy the updated file to your Web application server.
 1. Change the **.js** files in the `staging_location\webapp_root` directory.
 2. Regenerate the **tc.ear** file and deploy in the application server domain.
 3. Clear your browser cache so the new version can be loaded.
- **Dynamic files**
 Dynamic files are written in TcScript, used only on the server, and interpreted at run time by the **tcserver** process. These files have **.xml**, **.isx**, **.ish**, **.html**, and **.is** extensions.
 To deploy dynamic file changes, restart your **tcserver** process to pick up changes:
 1. Change the **.xml**, **.isx**, or **.ish** files in the `TC_ROOT\web\htdocs\tc` directory.
 2. Restart your **tcserver** process to pick up changes. Either restart the pool manager or log off Teamcenter and log on again. You can avoid this step by setting the **TC_WEB_NO_CACHE** environment variable to **TRUE**.
 For more information, see *Write TcScript*.

Thin client preferences

Many preferences affect the behavior of the Teamcenter thin client. The details are documented in the Teamcenter help collection. The most useful of these options can be set directly in the thin client with the **Edit→Options** menu command. Set the site options before end users start modifying their personal options. Teamcenter sets default options out-of-the-box, but you can change them to more appropriate settings for your site.

Note:

Most Web-specific options begin with **WEB_**.

The following table contains important preferences that affect the thin client.

Preference	Definition
CR_allow_alterate_procedures	Determines if only assigned processes are permitted.
EPM_adhoc_signoffs	Enables or disables ad hoc signoff.
WEB_auto_assign_ds_id	Determines if dataset IDs are auto-assigned.
WEB_core_help_server	Sets the URL location of the general Teamcenter help collection; it is used to create the Help→General Collection menu command.
WEB_dataset_shown_relations	Controls the contents displayed on the Dataset page (for example, show reference objects and show multiple file versions).
WEB_dataset_upload_mode	Sets the default dataset upload mode for the site. Set to 0 if the dataset does not need to be checked out to upload a file to it (auto-checkout). Set to 1 if the dataset must be checked out explicitly to upload a file to it.
WEB_default_site_server	Specifies the address of the Web server; set this to generate visualization bookmarks.
WEB_displayed_new_menu_objects	Defines the list of items in the New menu.
WEB_help_server	Defines the URL location of the Teamcenter Web help collection; it is used to create the Help→Web Collection menu command.
WEB_max_search_results	Sets the maximum number of results to show in the search page.
WEB_processes	Defines the EPM processes to be listed from the Web (if not set, processes are not filtered).

Preference	Definition
WEB_protocol	Sets the protocol used by the Web server (default is http:\).
WEB_title_contents	Defines the title bar formula.
WEB_type-name_default_relation	Customizes the default paste relation by type.

The following table contains important preference that affect visualization in the thin client.

Preference	Definition
SecondaryVMUDatasets	Lists the non-VMU dataset types that could contain VMU datasets.
VMU_Datasets	Visualization dataset types that should be launched using VMU.
VMU_FileSearchOrder	Defines the valid named reference types for VMU datasets.

Cascading style sheets (CSS)

CSS is a W3C standard used to separate stylistic elements from data and other display logic. The thin client uses CSS to package the display and style information in one place. Changes to the style sheet are reflected across the entire application. This makes applying a custom style to the thin client much easier and more portable. You should become familiar with CSS basics.

Use cascading style sheets to design the thin client user interface (HTML pages, dialog boxes, and forms). The files related to CSS are located in the *staging_location\webapp_root\teamcenter\dhtml\common\css* directory.

If you create other style sheets, the **WEB_style_sheet** preference determines which one is used, and the **WEB_style_sheet_list** and **WEB_style_sheet_list_names** preferences define the available style sheets. To change the style sheet being used in the thin client, choose **Edit→Options→Advanced→Style Sheet**, click the arrow in the **Style Sheet** box, and select the style sheet, such as **Teamcenter Default**. The **Teamcenter Default** style sheet corresponds to the **tc.css** style sheet file specified in the **WEB_style_sheet** preference. To create a custom style sheet, modify the **tc.css** file and save it with a different name.

There are three additional style sheets you can use to change the look of the thin client user interface:

- **siemens/skin.css**
Defines the Siemens PLM Software look for the user interface.

- **ui-core.css**
Defines core widget styles, including styles for page header, layout, and multicolumn support.
- **yui-core.css**
Defines core widget styles for fonts, grids, containers, resize components, data tables, menus, layout manager, tab views, and tree views.

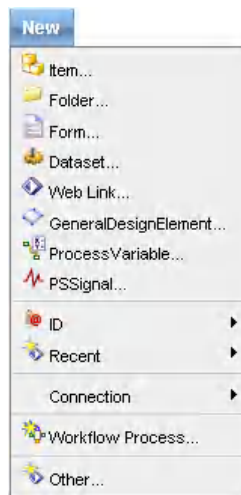
Common changes such as font sizes, colors, and background colors are achieved by modifying the **skin.css** file. For example, the default style for the top menu bar menus defined in the following code examples from the **skin.css** file:

```
.siemens-skin .yuimenuitemlabel {
    color: #000000;
    cursor: default;
    font-size: 85%;
    padding: 0 15px 0 7px;
    text-decoration: none;
```

And:

```
.siemens-skin .yuimenu .bd
    background-color: #EFEFEF;
    border: 1px solid #808080;
```

The following figure shows the menu using the CSS in the previous code examples.



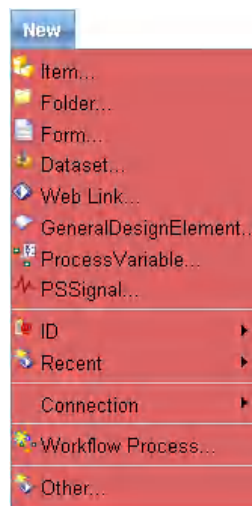
Default menu

To change the background color of the menu to red and to change the font, set the style in CSS as shown in the following code examples:

```
.siemens-skin .yuimenuitemlabel {
  color: #000000;
  cursor: default;
  font-size: 100%; // font size increased
  font-weight: bold; //added style for bolder text
  padding: 0 15px 0 7px;
  text-decoration: none;
}
```

And:

```
.siemens-skin .yuimenu .bd
  background-color: #DD6464; // changed to reddish
  border: 1px solid #000000; //changed to black
```



Customized menu

Thin client menu system

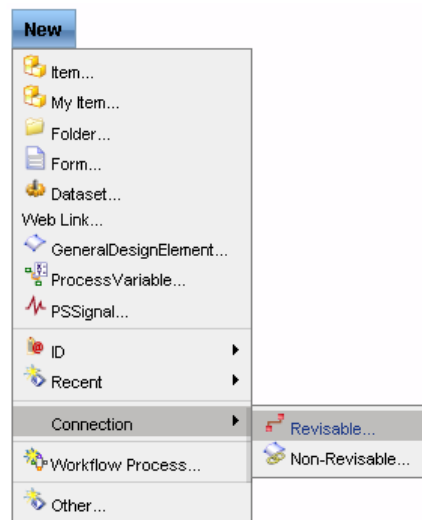
Orientation to the thin client menu system

The menus and left-hand navigation (LHN) toolbar are configured using static XML from the Web tier. The top menus retain the contextual actions (application or selection context specific). There is one menu for each application or top-level page. The LHN provides a consistent location for a simple set of context-free actions. The LHN is consistent throughout the applications. There are three basic types of actions:

- Navigate to a new URL, either replacing the current page or opening a new browser (for example, go to the **Home** folder).
- Open a dialog box (for example, create a new item).
- Execute an action within the current page (for example, clipboard actions).

The behavior of all three types may depend on the current selection within the tree, though the default LHN actions generally do not. The menu XML files are in the *staging_location\webapp_root\teamcenter\dhtml\common\intl\language* directory. There is one LHN definition file (**toolbar.xml**) and one top-level menu definitions file for each application (**applicationmenu.xml**). For example, the following code from the **wsomenu.xml** file renders the menu in the following figure:

```
<menu type="Connection" title="Connection">
  <menuentry ID="Revisable" title="Revisable..." key="newPSConnectionAction"
url="javascript:popupNewBusinessObject('PSConnection', true, false);"
image="teamcenter/dhtml/icons/PSConnection.png"/>
  <menuentry ID="Non-Revisable" title="Non-Revisable..."
key="newGDELinkAction" url="javascript:popupNewBusinessObject('GeneralDesignElementLink',
true, true);" image="teamcenter/dhtml/icons/GeneralDesignElementLink.png"/>
</menu>
```



New Connection menu

The **menuentry** attributes are:

- **title**
Specifies the localized display name for the entry.
- **key**
Specifies the name used for menu entry suppression configuration.
- **url**
Specifies the URL or arbitrary section of JavaScript.

Note:

The menu style is controlled by a CSS style sheet. The LHN uses different tags with similar attributes in the **toolbar.xml** file.

For more information, see [Cascading style sheets \(CSS\)](#).

To deploy menu changes, regenerate the **.ear** or **.war** files.

For more information, see [Deploying thin client customization changes](#).

Modifying menu commands

To make changes across an entire site, you can manually add, remove, or modify menu commands by editing the ***menu.xml** files in the *staging_location\webapp_root\teamcenter\dhtml\common\intl* *Vlanguage* directory.

The **WEB_menu_entry_new_window** preference controls which menu commands are targeted to open in new browsers. You can further tune this setting using the **target** attribute to target specific browsers rather than always opening a new one.

For example, the help pages are configured by default to open in a new browser using the **WEB_menu_entry_new_window** preference. If you choose the **Help→Web Collection** menu command to open the documentation from your **Home** folder, it opens in a new browser. If the **target** attribute is not set, and you return to the browser displaying your **Home** folder and choose **Help→Web Collection** again, another browser opens even though one is already open. However, the **target** attribute is set by default to use a specific browser as seen in the **commonmenus.xml** file:

```
<menuentry title="Web Collection" key="applicationHelpAction"
url="webclient?
TC_file=redirs/tcwebhelp" target="tchelp" ></menuentry>
```

With the **target** attribute set, if you return to the browser displaying your **Home** folder and choose **Help→Web Collection** again, the already opened help browser is displayed. The **target** attribute is used only if the menu command is listed in the **WEB_menu_entry_new_window** preference; otherwise it is ignored.

Customizing menus

To add common menus commands to all applications, add the menu details in the **custom_menu_additions.xml** file found in the *staging_location\webapp_root\teamcenter\dhtml\intl* *Vlanguage* directory.

Custom menu files are defined for each application. For example, the standard menu file for the My Teamcenter application is **wsomenu.xml**, and the custom menu file is **wsomenu_custom.xml**. If you add menu commands to the **wsomenu_custom.xml** file, those menu commands are rendered.

The **action** attribute in the **menu** node of the XML code defines the behavior. If the **action** attribute value is **replace**, the entire menu is replaced. If the value is **add** or the attribute is not defined, the menu commands are added to the existing menu.

To add custom menu commands in all applications, define the commands in the **custom_menu_additions.xml** file. Custom static menu files take precedence over standard static menu files.

Note:

To deploy the changes, regenerate the **.ear** or **.war** files.

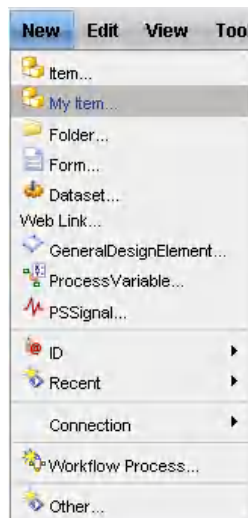
For more information, see *Deploying thin client customization changes*.

Change the business objects displayed in the New menu

You can change the business objects displayed in the **New** menu by changing the **WEB_displayed_new_menu_objects** preference. Use this preference to add, remove, or change ordering of the business objects in the menu.

For example, you can use this preference to add a custom business object to the **New** menu:

1. Create a custom business object using the Business Modeler IDE and install it to the server.
For more information, see the *Business Modeler IDE*.
2. Add the custom business object to the **WEB_displayed_new_menu_objects** preference.
3. Add an icon for the business object.
For more information, see the *Business Modeler IDE*.
4. Regenerate the **tc.ear** file using the Web Application Manager, and deploy the updated file to your Web application server.
When you open the **New** menu, the custom business object is displayed.



Custom business object on the New menu

Note:

When you create a business object using the **New** menu, the business object type is added to the **New→Recent** menu. The number of business objects added to this menu is defined by the `Create_WorkspaceObject_mru_max` preference.

Adding and modifying business object icons in the thin client

Business object icons are named and displayed by business object. You can modify them in the `staging_location\webapp_root\typeicons` directory.

If you add a custom business object to Teamcenter, add a business object icon for it in both the thin client and the rich client. You must name the business object icon using the exact, case-sensitive business object name. If the business object has spaces in its name, you must use an underscore in the icon name for each space. You can reuse copies of the same icon for different business objects. If Teamcenter cannot find the business object icon, it silently loads the class icon. However, this prevents caching, which slows performance and should be avoided.

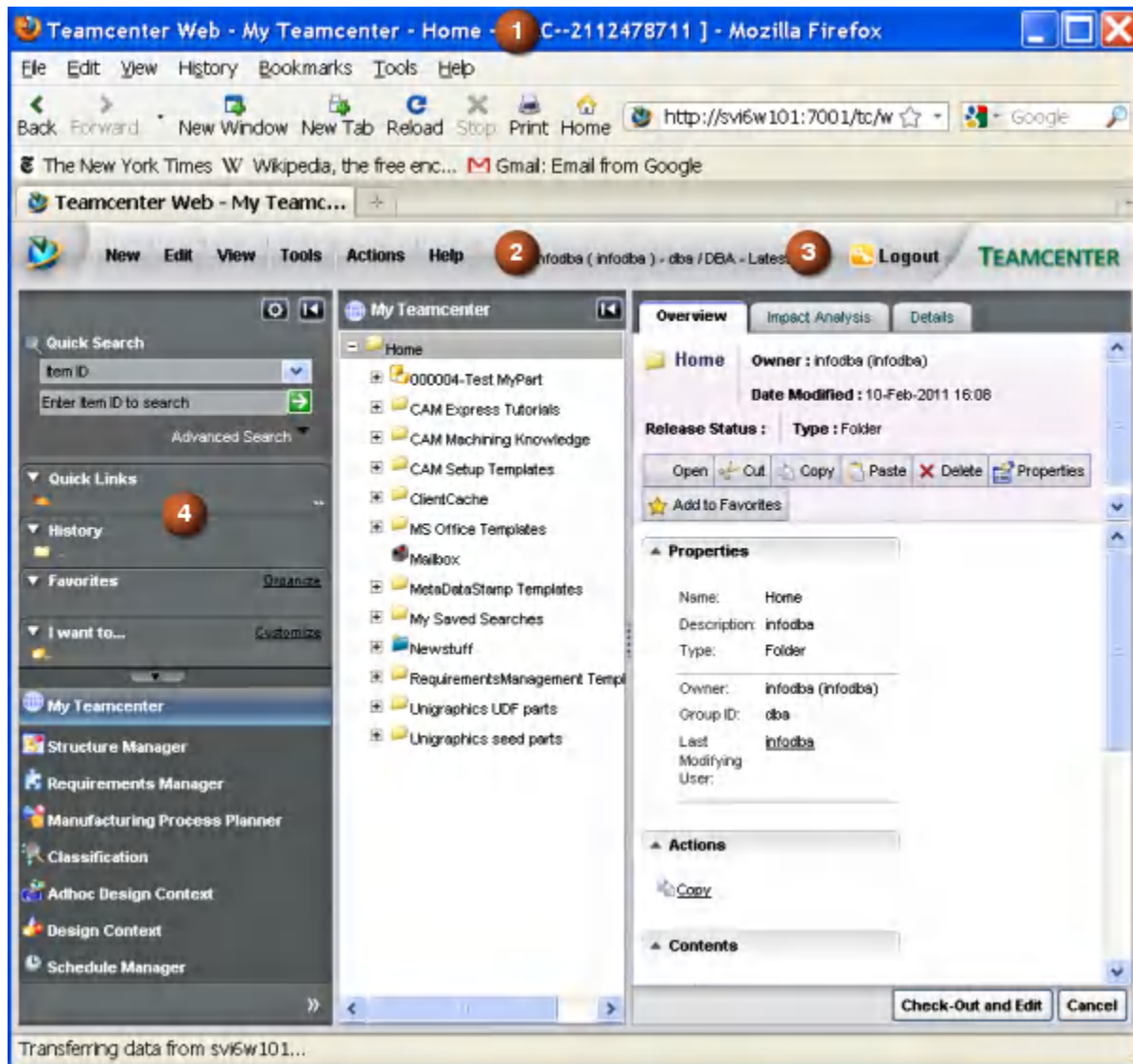
Note:

When you add a new business object icon, add it in the PNG format.

For instructions about changing business object icons in the rich client, see the [Business Modeler IDE](#).

Configuration settings in the user interface

The following figure shows some of the important configuration settings that control the user interface.



Configuration settings in the user interface

- | | | |
|---|------------------------------|---|
| 1 | WEB_title_contents | Preference that specifies what appears in the browser title bar. |
| 2 | wsomenu.xml | File that specifies what appears in the thin client menu bar. Find this file in the <i>staging_location\webapp_root\teamcenter\html\common\intl\language</i> directory. |
| 3 | appbar_title_contents | Preference that specifies the title string that appears in the application bar. |
| 4 | toolbar.xml | File that specifies what appears in the navigation pane. Find this file in the <i>staging_location\webapp_root\teamcenter\</i> |

`dhtml\common\intl\language` directory.

Customizing forms for the thin client

Methods of customizing forms for the thin client

Forms can be customized in the following ways:

- XML rendering template (XRT)
For more information, see [Introduction to style sheets](#).
- Altering of form contents
For more information, see [Altering form content](#).
- Custom form override mechanism
For more information, see [Thin client custom form override mechanism](#).

You can customize Teamcenter forms in many ways. A general strategy is to put as much of your customization into the core and as little into the user interface as possible. If your site uses multiple interfaces, as most do, you minimize duplicated effort. Some custom forms can be done completely in the core and are inherently supported by all Teamcenter interfaces. Other customizations require user interface effort, particularly forms with specialized display or interaction requirements.

Forms are used to store information. You can create a class with the required attributes and create a form based on the class. When you click a form object in the thin client, a request is sent to the Teamcenter server to retrieve the form object attributes, fields, and their values. Response XML code is sent to the client with the details. The XML is parsed to get the field details and render them as various components.

The following are the data types and components supported by forms:

Data types and components

Example

String – text field

The screenshot shows the 'New Item' dialog box with the 'Item Information' tab selected. The 'Name' field is highlighted, containing the text 'This is a string text field.' The other fields (ID, Revision, Description, Unit of Measure, Create Alternate ID, Check-Out Item Revision on Create) are empty. The 'Finish' button is highlighted.

Large strings (strings of defined database length greater than 60 characters) – text area

The screenshot shows the 'New Item' dialog box with the 'Item Information' tab selected. The 'Description' field is highlighted, containing the text 'This is a large string text area.' The other fields (ID, Revision, Name, Unit of Measure, Create Alternate ID, Check-Out Item Revision on Create) are empty. The 'Finish' button is highlighted.

Data types and components

Example

Date – calendar

The screenshot shows a form titled 'Checked-Out Dataset...' with several input fields: 'Checked-Out by User:', 'Checked-Out After:', 'Checked-Out Before:', 'Owning User:', 'Owning Group:', and 'Type:'. A 'Calendar' dialog box is open over the form, displaying a calendar for February 2011. The date 14 is selected. Below the calendar grid, the time is set to 14 Hrs and 22 Mins. The dialog has 'OK', 'Cancel', and 'Clear' buttons.

Boolean – check box

The screenshot shows a form with two checkboxes, both of which are checked (indicated by a green checkmark in a box):
 Create Alternate ID: ☒
 Check-Out Item Revision on Create: ☒

Multiple choices – buttons

The screenshot shows a dialog box titled 'Create/Edit Process Assignment Lists'. It has a 'Select List Type' section with four radio buttons: 'New' (selected), 'My Lists', 'My Group Lists', and 'Other Lists'. Below this is a 'Name' field with a red asterisk indicating it is required. A 'Close' button is at the bottom left.

List of values (LOVs can be exhaustive or suggestive. In case of suggestive LOVs, the user can enter a value in the LOV field) – list box

The screenshot shows a dialog box titled 'New Dataset'. It has two tabs: 'Type' (selected) and 'Upload'. Under the 'Type' tab, there is a 'Type' field with a red asterisk. A list box is open below the field, showing a list of values: 3D PDF Generation Inputs, Adobellustrator, Bitmap, Bitmap_Thumbnail, Briefcase, CAEAnalysisDS, CAEBC, CAEConn (highlighted), CAEGeom, CAELoad, CAEMesh, CAEResults, CAESolution, and CAESolver. At the bottom, there are 'Back', 'Next', 'Finish', and 'Cancel' buttons.

Altering form content

The form customization techniques change the client-side presentation of the existing form data, but they do not affect the XML form data coming from the server. To add new or alter existing fields, you can write custom XML nodes under the **<custom>** tag. This technique can be used along with any other customization technique.

- The standard `TC_ROOT\web\htdocs\tcl\common\data\form.isx` file is typically used for retrieving the XML code on the server side.
- The **form.isx** file includes the `TC_ROOT\web\htdocs\tcl\custom\data\custom_forms_data.xml` file. You can add extra attributes and their values to the form XML as needed. This XML code can also include attributes that need to be overridden.
- On the client, the XML code is transformed by removing the custom nodes and integrating with the standard form XML.
- While rendering, the attributes in **<custom>** field nodes are given preference when the same set of nodes exist in the standard form XML.

The resulting XML code looks like the following on the server side:

```
<form>
  <field name=attr1 ...></field>
  <field name=attr2 ...></field>
  <field name=attr3 ...></field>
  <custom>
    <field name=attr3 ...><lov>values...</lov></field>
    <field name=attr4 ...></field>
  </custom>
</form>
```

After transformation on the client, the XML looks like the following:

```
<form>
  <field name=attr1 ...></field>
  <field name=attr2 ...></field>
  <field name=attr3 ...><lov>values...</lov></field>
  <field name=attr4 ...></field>
</form>
```

Thin client custom form override mechanism

If you cannot customize the Web user interface form with the preferred techniques, you can use the form override mechanism to customize most of them. This mechanism provides a unified and abstract interface to the most useful form characteristics. These characteristics can then be special cased by form type. It is especially useful because you can have a complete form customization in one file rather than

placing hooks in several places, so you can port customizations more easily from one version of Teamcenter to another.

Several characteristics summarize Teamcenter dialog boxes. These characteristics are factored and parameterized as arguments to the **createActionDialog()** function in the *staging_location* `\webapp_root\teamcenter\dhtml\common\actiondialog.js` file. The arguments are:

- **actionCall**
- **actionScript**
- **applyButton**
- **formNode**
- **numCols**
- **renderer**
- **resizable**
- **takeSelection**

You can override these inputs based on your dialog box type using the customization mechanism in the *staging_location* `\webapp_root\teamcenter\dhtml\custom\customform.js` file. You can override one, none, or all of these parameters based on the form type.

In this context, a form is a subtype of dialog box. This information applies to all dialog boxes, but is most commonly used with forms. By default, forms have the required XML **type** box, but other dialog boxes do not. All dialog boxes use the XML **<form>** node and are created using the **createActionDialog** function.

Parameter	Definition and default value
actionCall	Specifies the JavaScript function called to parse your arguments and calls your action script. See the genericActionCall() function in the actiondialog.js file. This parameter should be the actual function handle (the name without quotation marks), not a string. The default is genericActionCall .
actionScript	Specifies the TcScript action or file to execute when you click the OK or Apply button. The form default is found in the <code>TC_ROOT\web\htdocs\tc\common\actions\formmodify.xml</code> file; other dialog boxes have different defaults.

Parameter	Definition and default value
applyButton	Specifies whether or not to add an Apply button to the form in addition to the OK and Cancel buttons. Values are either true or false . The form default is true ; other dialog boxes have different defaults.
formNode	Specifies the DOM reference to the first XML node describing your dialog box. Do not change this parameter—it must not be overridden.
numCols	Specifies the number of columns in the dialog box. You must use 2 or 3 unless you are substituting your own renderer and argument parser in the actionCall parameter. The form default is 2 . Some dialog boxes (such as Route task perform , Time task , and Out of office) explicitly pass in 3 .
renderer	Specifies the JavaScript function called to render your XML into your dialog table. This must be a string literal (quoted), not a function handle like in the actionCall parameter. The default is "processFORM" for both forms and dialogs.
resizable	Specifies whether the dialog box is dynamically resized or static. The default is false (in other words, static). Set to any non-null value to turn resizing on.
takeSelection	Specifies whether or not to pass the current tree selection to your action script. It is either true or false . The form default is false ; other dialog boxes have different defaults.

These parameters are not listed in the order they are used. To better understand their usage, think about them in the order they are used. The **formNode**, **applyButton**, **resizable**, **numCols**, and **renderer** parameters are used for creating the form. Then, on submission, the **actionCall** function is called, which uses the **takeSelection** parameter and makes a request to the server file, **actionScript**.

You can override these parameters by initializing new values in a special multidimensional associative array called **globalCustomDialogType**. Additionally in this file, you must define any new functions you mention in the **renderer** or **actionCall** parameters.

You do not need to initialize parameters to their default values; you only need to set the ones you want to override.

In section 1 of the **customform.js** file, add a new parameter array to **globalCustomDialogType**, keyed by the type name. For each parameter you want to override, add the value to the parameter array, keyed by the parameter name. An example is shown in the following figure.

In section 2, if you are overriding **renderer** or **actionCall**, define your new functions. Your new **renderer** is passed two parameters automatically: the DOM reference to the XML dialog node and the DOM reference to the **div** table the dialog box is created in. Therefore, your **renderer** should accept two inputs. Your new **actionCall** does not take any arguments. An example is shown in the following code example:

```

/ SECTION 1 //
// create and initialize associative arrays
// associative area of custom dialog types and renderers
var globalCustomDialogType = new Array();    //instantiate global array

// instantiate inner arrays for any dialog types you want to override here...
// remark form example:
globalCustomDialogType[ "remark form" ] = new Array();
    // parameter array for "remark form" type

// custom form example:
globalCustomDialogType[ "custom form" ] = new Array();
    // parameter array for "custom form" type

// override 2 remark form parameters:
globalCustomDialogType[ "remark form" ][ "renderer" ] = "processREMARKFORM";
    //note quotes

globalCustomDialogType[ "remark form" ][ "actionCall" ] = mightyMightyActionCall;
    //note no quotes

// override everything on custom form type
globalCustomDialogType[ "custom form" ][ "renderer" ] = "processREMARKFORM";
    //note quotes

globalCustomDialogType[ "custom form" ][ "actionCall" ] = mightyMightyActionCall;
    //note NO quotes

globalCustomDialogType[ "custom form" ][ "actionScript" ] = "myspecialcustomscript.xml";
    // note that this tcscript file must be placed in the actions directory

globalCustomDialogType[ "custom form" ][ "takeSelection" ] = true;
    //pass selected line to actionScript

globalCustomDialogType[ "custom form" ][ "applyButton" ] = false;
    //OK/cancel only

globalCustomDialogType[ "custom form" ][ "numCols" ] = 7;
    // 7 column dialog

// SECTION 2 //
// define your renderer here
// example:
//   Of course calling processFORM is exactly what you do not want to
//   actually do because this renders your form XML using our default renderer.
//   But you could use this skeleton to test that you have hooked everything
//   up properly.

function processREMARKFORM( customFormNode, table )
{
    alert( "Success!" );
    processFORM( customFormNode, table );
}

function mightyMightyActionCall()
{
    alert( "Call successful" );
}

```

Thin client custom form override example

This example creates a custom item revision master form. Copy the code and paste it in the **customform.js** file in the *staging_location\webapp_root\teamcenter\dhtml\custom* directory.

The customized form has a second column. The original form has only one column.

```
var globalCustomDialogType = new Array();
globalCustomDialogType[ "ItemRevision Master" ] = new Array();

// override everything on itemrevision master form type
globalCustomDialogType[ "ItemRevision Master" ][ "renderer" ] = "processIRMFORM";
//note quotes
globalCustomDialogType[ "ItemRevision Master" ][ "actionCall" ] = irmActionCall;
//note NO quotes
globalCustomDialogType[ "ItemRevision Master" ][ "actionScript" ] = "common/actions
/formmodify.xml";
// note that this tcscript file must be placed in the actions directory
globalCustomDialogType[ "ItemRevision Master" ][ "takeSelection" ] = true;
//pass selected line to actionScript
globalCustomDialogType[ "ItemRevision Master" ][ "applyButton" ] = false; //OK/cancel only
globalCustomDialogType[ "ItemRevision Master" ][ "numCols" ] = 4; //4 column dialog
globalCustomDialogType[ "ItemRevision Master" ][ "resizable" ] = true; //resizable dialog

function processIRMFORM( irmFormNode, table )
{
    initializeForm( irmFormNode, table );
    var dialog = getDialog( table );
    var tbody = table.firstChild;
    var fieldNode = irmFormNode.firstChild;
    var row;

    // create the dialog box title and also the header rows within the tbody
    //      table      thead      tr
    row = dialog.headtable.firstChild.firstChild;
    row.firstChild.innerHTML = irmFormNode.getAttribute( "name" );
    row = document.createElement( "tr" );
    tbody.appendChild( row );
    var column1 = document.createElement( "td" );
    var column2 = document.createElement( "td" );
    var column3 = document.createElement( "td" );
    var column4 = document.createElement( "td" );

    row.appendChild( column1 );
    row.appendChild( column2 );
    row.appendChild( column3 );
    row.appendChild( column4 );

    row.modifiable = true;
    column1.innerHTML = irmFormNode.firstChild.getAttribute( "displayname" );
    column1.name = irmFormNode.firstChild.getAttribute( "name" );
    var input = document.createElement( "input" );
    input.type = "text";
    input.value = irmFormNode.firstChild.getAttribute( "value" );
    column2.appendChild( input );

    column3.innerHTML = irmFormNode.firstChild.nextSibling.getAttribute( "displayname" );
    column3.name = irmFormNode.firstChild.nextSibling.getAttribute( "name" );
}
```



```

input = document.createElement( "input" );
input.type = "text";
input.value = irmFormNode.firstChild.nextSibling.getAttribute( "value" );
column4.appendChild( input );
dialog.popup();
}
function irmActionCall(table, popDownOnSuccess)
{
    doRequest( new irmAction( table, popDownOnSuccess ) );
}

function irmAction( table, popDownOnSuccess )
{
    var actionScript = table.getAttribute("actionScript");
    if(actionScript == null)
    {
        actionScript = table.actionScript;
    }

    var request = "webclient?TC_file=" + actionScript;
    request += buildIrmURL( table );

    if ( table.takeSelection )
    {
        if( table.takeSelection == 'top' )
        {
            request += "&parent_uid=" + getTopLine().iman_parent_uid;
        }
        else
        {
            request += "&parent_uid=" + firstSelectedRowOrTopLine().iman_parent_uid;
            if( table.takeSelection == 'lhn' )
            {
                request += "&lhn=true";
            }
        }
    }

    this.url = request;

    this.dialog = getDialog(table);
    this.callback = updateCallback;
    this.popDownOnSuccess = popDownOnSuccess;
}

function buildIrmURL( table )
{
    // go through the table body
    // for each row
    //     if it's a row with two columns
    //         add the QUOTED string _row.fieldName=rightCell.input thing.value.somewhere

    var URL = "";
    // add object tag
    if ( table.tc_object )
    {
        URL += "&object=" + table.tc_object;
    }
}

```

```

var dialog = getDialog(table);
var tbody = dialog.getTBodyFromTable( table );

if( tbody == null )
{
    alert("Cannot handle null. NULL tbody pointer.\n");
}
var row = tbody.firstChild;
while ( row )
{
    // certain fields use this to indicate an unset value should not be submitted
    if( row.modifiable )
    {
        var nameCell = row.firstChild; // 1st td
        if ( nameCell )
        {
            var valueCell = nameCell.nextSibling; // 2nd td
            if ( valueCell )
            {
                URL = URL + "&_" + fromUnicode( nameCell.name )
                    + "=" + fromUnicode( queryInputValue( valueCell ) );
            }
        }
        // second set of inputs
        nameCell = row.firstChild.nextSibling.nextSibling; // 3rd td
        if ( nameCell )
        {
            var valueCell = nameCell.nextSibling; // 2nd td
            if ( valueCell )
            {
                URL += URL + "&_" + fromUnicode( nameCell.name )
                    + "=" + fromUnicode( queryInputValue( valueCell ) );
            }
        }
        row = row.nextSibling;
    }
}
return URL;
}

```

Customize property names in the user interface

You may want to change how a property name appears in the user interface. For example, you have a property named **my_prop** but want it displayed as **My Property**. Use the Business Modeler IDE to define the display names of properties.

For more information, see the [Business Modeler IDE](#).

Customizing the thin client with TcScript

Write TcScript

TcScript is a proprietary server-side scripting language used to access Teamcenter. Typically the data retrieved is used to create Web pages. The data generated can be potentially in any format. In the out-of-the-box Teamcenter thin client, HTTP requests come in and XML is returned.

You may need to write server-side TcScript to satisfy your customization requirements. By making modifications to existing TcScript, you can alter the data returned to the client. By writing your own TcScript, you can implement new functionality.

The thin client optimizes performance by caching TcScript parse trees. You do not see changes made to TcScript code until your **tcserver** process is restarted by either restarting the pool manager, killing your **tcserver** process, or logging off and logging on again. You can avoid restarting the **tcserver** process by setting the **TC_WEB_NO_CACHE** environment variable to **TRUE** so you can see your TcScript changes immediately. Once you complete your modifications, set the variable back to **FALSE** for improved performance.

TcScript has its own structure and command syntax, but its most important characteristic is that it has a binding layer to Teamcenter ITK. You must be familiar with ITK to develop TcScript. For further information and examples using TcScript, see the existing code. One broad end-to-end example is described in *Process for generating a thin client page*.

TcScript sections of pages are interpreted by the thin client. To identify these sections, you must enclose TcScript in **<script>** tags. In this particular example, the entire file is written in TcScript.

1. Begin with the following line:

```
<script type="text/IMAN">
```

2. Include useful TcScript helper functions using paths relative to the **htdocs** root directory:

```
include 'common/basic.isx'
include 'actions/relations.isx'
include 'actions/update.isx'
```

3. Print an XML document header:

```
print XMLDefaultHeader()
```

The **XMLDefaultHeader** function is defined in the **TC_ROOT\web\htdocs\tc\common\basic.isx** file. The **print** statement is used to output data. Anything printed is sent to the requesting browser.

4. Make the first ITK call. Most ITK functions are available in TcScript.

```
POM_place_markpoint( 'mymark' )
```

This function places a mark point in the database that you can roll back to if anything goes wrong.

5. Add a **try** call:

```
try
```

TcScript supports try/catch blocks. This makes error handling much simpler. Instead of checking the return value of each ITK function, wrap anything that could potentially fail in a try block and handle the potential error in the catch block.

6. Add the first comment. Use the hash (#) character to denote TcScript comments.

```
# Must have a name....
```

7. You cannot create an item without a name, so use a conditional statement to evaluate the **&_name** input parameter. All arguments passed in the requesting URL are available in TcScript. If you look at the URL requesting the page in the access log, you should see **&_name=xxx** in the argument section of the URL. This URL can also be found in the TcScript **syslog** and the client **Temporary Internet Files** address field.

```
if _name == ''
then
# You must supply a name for an Item(Revision)
EMH_store_error( 1, 63065 )
throw 'error'
endif
```

Note that the quotation marks are single quotation marks. Always use single quotation marks in TcScript to denote string literals. If no name is supplied, an error is stored on the stack and thrown directly to the catch, skipping the lines in between.

8. Split the **parent_uid** argument in two. This **parent_uid** is the Teamcenter UID (or tag) of the line selected as the insertion point for the new item concatenated with the relation type UID (tag). Thin client UIDs are Web-safe versions of **tag_t**. The UID and **tag_t** are essentially the same, because the conversions are handled in the bindings to the ITK. Therefore, they are referred to as tags. The ITK output parameters are passed as literals. This is synonymous with passing a pointer.

```
WEB_decompose_double_tag( parent_uid, 'parent', 'relation' )
```

9. Initialize the rest of the input arguments:

```
item_id = _id
rev_id = _revid
```

10. Call ITK to validate or create them, and then handle the response.

```

USER_validate_item_rev_id (item_id, rev_id, _type,
'modified_item_id',
'modified_rev_id', 'status')
# USER_invalid_id -- reject
# USER_valid_id -- create as given
# USER_override_id -- create as modified (silent change, for
example,
#                               for case conversion)
# USER_modified_id -- may create as given, has returned a better
version
if status == 'USER_invalid_id'
then
# Failed to create item -- invalid id supplied
EMH_store_error( 1, 63080 )
throw 'error'
elif status == 'USER_override_id'
then
if modified_item_id != ''
then
item_id = modified_item_id
endif
if modified_rev_id != ''
then
rev_id = modified_rev_id
endif

```

11. Call the ITK to create the item. The **newItem** and **newRev** tags are set as output.

```

# create the item with default rev id
ITEM_create_item( item_id, _name, _type, rev_id, 'newItem',
'newRev' )

```

12. Set the description for the item and revision and save them.

```

WSOM_set_description( newItem, _description )
WSOM_set_description( newRev, _description )
AOM_save( newRev )
AOM_save( newItem )
AOM_refresh( newRev, 0 )
AOM_refresh( newItem, 0 )

```

13. Link it to the appropriate place:

```

parent_uid = MakeRelation( parent_uid, newItem )

```

14. If there are no ITK errors or a thrown error, you can assume success and can send an XML message to the browser asking for the parent line to be refreshed (in case any displayed properties have changed) and for the new item to be inserted into the tree.

This message is discussed in more detail in *Processing on the Teamcenter server*.

```
print '<update>'
+ RefreshUpdate( WSOLine( parent ) )
+ InsertUpdate( parent_uid, WSOLine( newItem ) )
+ '</update>'
```

15. Delete the mark point because you do not need to roll back.

```
POM_forget_markpoint( mymark )
```

If a thrown error was caught, you roll back to preserve the all-or-nothing nature of the transaction.

16. Add a contextual error message to the stack and print the XML error stack. This function is defined in the **basic.isx** file.

```
catch error
POM_roll_to_markpoint( mymark, 'stateChanged' )
EMH_store_error( 1, 63009 )
print ErrorStack( '' ) + NL
```

17. Close the **try** block with an **endtry** statement and close the **script** tag. This concludes the script. The XML message sent to the client is interpreted.

```
endtry
</script>
```

TcScript values

Types

TcScript values can have one of the following types:

Type	Definition
Array	Array of TcScript values of any type
Date	Teamcenter date
Integer	Integer number
Logical	Logical or Boolean value (TRUE or FALSE)
Object	Teamcenter object (in other words, a Teamcenter tag)
String	Literal text string

In most cases, the value type is not relevant to the author of a Teamcenter Web page because TcScript automatically converts values to a suitable type in the ITK bindings. There is one exception: the string to integer conversion. All HTTP arguments are strings, so if you have **&index=22** in your URL, the variable `index` is set to the string **22** in your TcScript environment. If you then need this value as an integer to pass to ITK, you must explicitly convert it using the **WEB_string_to_int** ITK function.

Strings

String literals are enclosed by single quotation marks. All whitespace (including new lines) between the quotes is included verbatim in the string. For example:

String	Description
<code>'hello world'</code>	Literal text string
<code>"</code>	Empty string
<code>'</code> <code>'</code>	New line character
<code>'<table></code> <code><tr></code> <code><td>NW</td></code> <code><td>NE</td></code> <code></tr></code> <code><tr></code> <code><td>SW</td></code> <code><td>SE</td></code> <code></tr></code> <code></table>'</code>	Literal segment of HTML

System constants

The following constants are defined in TcScript.

Constants	Definition
EMPTYARRAY	Empty array
FALSE	Logical value false
NL	Literal new line
NULLTAG	Object value representing a Teamcenter NULLTAG
TRUE	Logical value true

Variables

Values can be assigned to a variable. For example:

```
id = IMAN_user.user_id
```

The variable **id** is created automatically if it does not exist before the assignment. The variable can then be used or reassigned throughout the rest of the page.

Array variables can be assigned in requests as follows:

```
selection[0]=a&selection[1]=b
```

This creates an array selection with two elements in it. This array notation is only valid in requests. TcScript does not have a mechanism for dereferencing particular elements of an array. Instead, arrays are generally looped through incrementally. In TcScript, arrays can be formed using the `::` concatenation operator. For example:

```
selection = a :: b
```

Reserved variables

The following metavariables are defined by TcScript. Their values are set appropriately immediately before each page is processed:

Metavariable	Definition
IMAN_user	Current user's tag
IMAN_role	Current user's role tag
IMAN_group	Current user's group
IMAN_config_rule	Current configuration rule tag
IMAN_site_name	Name of the site
MANGLED_QUERY	Query with standard mangling (to allow POST/GET)
QUERY	Page query tag
TC_VERSION	Teamcenter version string

TcScript operators

String operators

The only string operator in TcScript is the concatenation operator (+). This operator converts its left and right hand operands to strings and concatenates them.

Example code	Result
<code>'hello' + 'world'</code>	<code>'helloworld'</code>
<code>'hello' + ' ' + 'world'</code>	<code>'hello world'</code>
<code>name = 'hello'</code> <code>id = 'world'</code> <code>name + ' ' + id</code>	<code>'hello world'</code>

Integer operators

TcScript does not implement any mathematical functions directly. To perform addition, subtraction, multiplication, or division of TcScript integers, call the **WEB_arithmetic** ITK function:

```
WEB_arithmetic( num_left, op, num_right, output )
```

num_left and **num_right** are integers, **op** is an operator character, and **output** is the mathematical result. The operator character can be:

- +
- -
- *
- \

For example:

```
index = 0
WEB_arithmetic( index, '+', 1, 'index' )
```

This code adds 1 to the value of **index**.

TcScript array operators

LENGTH

The **LENGTH** function returns the number of elements in an array. For example:

Example	Result
LENGTH(animals)	'3'
LENGTH(EMPTYARRAY)	'0'

REVERSE

The **REVERSE** function reverses a list. For example:

Example	Result
REVERSE(dates)	{ '17-Jun-1998', '23-Nov-1997', '05-Apr-1998' }
REVERSE(EMPTYARRAY)	EMPTYARRAY

SORT

The **SORT** function sorts a list by a specified property. The syntax is:

```
SORT( array, criterion )
```

The **criterion** argument is a string value giving the name of the property by which to sort the array: For example:

Example	Result
SORT(birthdates, 'individual')	{ { 'henry hamster', '23-Nov-1997' }, { 'laura lion', '05-Apr-1998' }, { 'vicky vicuna' , '17-Jun-1998' } }

SPLICE

The **SPLICE** function splices two lists together, returning a single array of objects. The syntax is:

```
SPLICE( list1, name1, list2, name2 )
```

The **list1** and **list2** arguments have equal lengths. The *n*th object in the returned array is a pair of the *n*th element of the first list, and the *n*th element of the second list. The first and second elements of

each pair can then be accessed by the **name1** and **name2** field names, respectively. This provides a mechanism for constructing a multi-dimensional array from HTTP name/values pairs. The **SPLICE** function is rarely used but allows simultaneous iteration over two arrays. Note that several of the data functions such as **FORM** and **QUERYFIELDS** effectively return **SPLICE** lists. The following example shows iterating over a **SPLICE** array. Let **birthdates** be **SPLICE(animals, 'individual', dates, 'day')**. The value of **birthdates** can be represented as:

```
{
  { 'laura lion',      '05-Apr-1998' },
  { 'henry hamster',   '23-Nov-1997' },
  { 'vicky vicuna' ,   '17-Jun-1998' } }
```

The following code lists out the animals and their birthdays:

```
for birthday in birthdates

    print birthday.individual + ' was born on ' + birthday.day

    print <br>

next
```

The **SPLICE3** function is similar to the **SPLICE** function. It splices three lists together, returning a single array of objects. The syntax is:

```
SPLICE3( list1, name1, list2, name2, list3, name3 )
```

Accessing Teamcenter data with TcScript

Calling ITK with TcScript

Data can be retrieved using functions or properties. There are three kinds of functions available in TcScript:

- Bound ITK
- Specialized TcScript
- Helper functions

Helper functions are written in TcScript and access Teamcenter data using one of the standard methods, so they are not described. They are used only to conveniently wrap these methods behind a layer of abstraction. Additionally, Teamcenter data can be accessed using properties.

ITK is the preferred method for accessing Teamcenter data in TcScript. Most ITK functions can be called from TcScript. The function returns a logical value **TRUE** or **FALSE** depending on whether it succeeded or not; errors must be handled by examining the error stack.

For simplicity in this create folder example, the return value is checked explicitly. The **try/catch** block is preferred.

```
name = 'my folder'
if( FL_create( name, 'my generic description', 'newFolder' ) )
then
AOM_save( newFolder )
endif
```

You can pass the string, object, array of objects, integer, date, and logical input arguments directly into the ITK function. Input arguments, if defined earlier, should be referred to in the ITK call without quotation marks. You can also pass an input argument as a value; in the case of a string, use single quotation marks as shown in this example. Output arguments are returned by passing the name of a variable as a string; when the function completes, the output value is set to this variable. In this example, the **newFolder** variable is not defined yet and therefore mentioned as a string within single quotation marks. When the function completes, the **newFolder** variable is set to the value of the folder tag.

All memory management issues involving output arguments are handled automatically by TcScript. Most **enum** values can be passed in as literal strings. ITK structure types are not supported.

Properties

Given a Teamcenter object in TcScript, you can access any property of the object (including any user-defined properties) with the **.** operator. The syntax is:

```
objectName[.attr]+
```

This allows property accesses to be chained together. For example:

```
item.object_name
rev.structure_revisions
namedRef.ref.object_type
```

Writing specialized ITKs

Functions beginning with the **WEB_** prefix are a special class of ITK implemented to perform specific functions for the thin client. They are treated exactly like standard ITK. They should only be used in cases where standard ITK functions do not provide the needed capability. The string functions can be used with only a small amount of data.

Function	Description
WEB_arithmetic	Performs mathematical operations like +, -, *, and \.
WEB_compare	Used to compare numbers.
WEB_string_to_int	Converts a string to an integer.

LOG function

The **LOG** function is a useful debugging tool to keep data and debugging separate. For example:

```
my_val = 'blue'
LOG( 'the sky is ' + my_val + NL )
```

This writes **the sky is blue**\n to the **tcserverpid.syslog** file.

Work with the user exits sample file

User exits are places in the server where you can add additional behavior by attaching an extension. Examples of user exits for thin client customization using TcScript can be found in the **TC_ROOT\sample\examples\user_web.c** sample file.

For more information about user exits and extensions, see the [Business Modeler IDE](#).

1. Install sample files:
 - a. Start Teamcenter Environment Manager (TEM).
 - b. In the **Maintenance** panel, select **Configuration Manager** and click **Next**.
 - c. In the **Configuration Maintenance** panel, select **Perform maintenance on an existing configuration** and click **Next**.
 - d. In the **Configuration Selection** panel, select the configuration from which the corporate server was installed. Click **Next**.
 - e. In the **Feature Maintenance** panel, under the **Teamcenter** section, select **Add/Remove Features**. Click **Next**.
 - f. In the **Features** panel, under **Server Enhancements**, select **Sample Files**.
 - g. Click **Next**.
 - h. In the **Confirm Selections** panel, click **Next**.
The sample files are placed at the following location:

`TC_ROOT\sample`

2. Locate the **user_web.c** file in the following directory:

`TC_ROOT\sample\examples`

3. Open the **user_web.c** file and follow the directions in the commented text.

TcScript Statements

for

The **for** statement is used to iterate over an array. The syntax is:

```
for variable in array
# body
# do something with each variable
next
```

Each value in an array is assigned to *variable* in turn, and *body* is processed for each value.

if

The syntax of the **if** statement is:

```
if cond
then
# body
[ elif cond then body ] *
[ else body ]
endif
```

The *body* segment can be any arbitrary set of TcScript statements, and *cond* is a logical expression. Logical expressions can contain logical variables, logical operators, and parentheses.

The binary logical operators are of the form: *lvalue op rvalue*, where *op* is one of the following:

- **and**
True if *lvalue* and *rvalue* are both true.
- **or**
True if *lvalue* or *rvalue* is true.
- **==**
True if *lvalue* is equal to *rvalue*.

- **!=**
True if *lvalue* is not equal to *rvalue*.
- **isa**
True if *lvalue* is an instance of the class defined by *rvalue*.
- **hasa**
True if *rvalue* is the name of a valid property of the object defined by *lvalue*.

The **and** and **or** operators use short circuit evaluation; that is, *rvalue* is not evaluated if the final value of the expression can be established by only evaluating *lvalue*.

The only unary logical operator is **not**, which negates the value of its argument. The precedence of the operators in decreasing order is:

1. **==, !=, isa, hasa**
2. **not**
3. **and**
4. **or**

Note:

The optional **elif** keyword often simplifies complex **if** statements.

include

The syntax of the include statement is:

```
include 'filename'
```

The *filename* argument is the full file name of another thin client page relative to the document root. The included page is interpreted in line, as if it were textually inserted into the including page. This means that the included page must not have **<script>** tags.

def/enddef

This statement is used to define your own functions in TcScript. The syntax is:

```
def saveWorld( world )
# do stuff here
saveWorld = TRUE
enddef
```

Note that **enddef** has two **ds**. Also note that a value can be returned by setting it equal to the function name, as illustrated in the syntax.

TcScript error handling

TcScript try/catch error handling

The easiest way to manage error handling in TcScript is to use **try/catch** statements. The syntax for a typical case looks like the following:

```
# print my header (document type, encoding)
print XMLDefaultHeader()
# potentially place a DB mark point
POM_place_markpoint( 'mymark' )
try
output = ...
# perform some ITK functions
...
output += ...
...
print output
catch error
# potentially rollback to markpoint
POM_roll_to_markpoint( mymark, 'stateChanged' )
# potentially add contextual error to stack
EMH_store_error( 1, 63xxx )
# print error stack
print ErrorStack( '' ) + NL
endtry
```

This format is convenient because you do not have to check every return code at every potential point of failure. Instead, a block that includes *n* points of failure can be wrapped between the **try** and **catch** statements. Any error within that block halts execution of the **try** block statements and jumps to the **catch** block. If ITK fails, it puts its own error on the stack; it is still useful to add your own error for context. Another important point is that no output is printed to the client until the last point of failure is passed. This is done by building up the response as a variable (**output** in this case). This allows two possible responses: success (in the form of some XML to be rendered), or failure (in the form of an XML error stack to be rendered). You generally do not want to mix the two responses. If you are building an HTML response, this is not as important because most browsers are tolerant of broken HTML. However, if you are building an XML response, it is important because XML parsers are not tolerant of broken XML. Therefore, in the thin client interface, most requests return either success or failure and nothing in between.

TcScript syntax errors

Any errors encountered while parsing a TcScript page are reported within the page and list the line number and context of the error. The TcScript parser uses recursive descent parsing. If there are multiple

error messages, the later messages may be meaningless. When debugging, focus on the first syntax error. Some of the most common errors committed by TcScript programmers are:

- Misspelling **enddef**.
- Using an **if** statement without a **then** statement.
- Attempting simple arithmetic, which is not supported, or trying to pass a string to an ITK function expecting an integer.

TcScript error function calls

Teamcenter errors can be caused by property access, saved query execution, or any TcScript or ITK function call.

Use the following TcScript function calls to identify Teamcenter errors:

- **ERRORS**

The **ERRORS** function returns an array representing the current state of the Teamcenter error stack. The information is returned as an array of errors, where each error has three pseudo-properties: the **ifail** integer, the error text message, and the severity integer. The syntax is:

```
ERRORS ( )
```

More conveniently, you can call the TcScript helper functions that wrap this call, defined in the `TC_ROOT\web\htdocs\tcl\common\basic.isx` file. The most common is:

```
ErrorStack( text )
```

It takes a contextual text message as input and returns the error stack as XML. The contextual message is displayed to the user, but it is not added to the Teamcenter error stack. If you opt to add a contextual error to the stack before displaying the error, you would typically pass an empty string as input.

Two similar functions of limited use are **alertErrorStack(text)** and **HTMLErrorStack(text)**. The **alertErrorStack** function wraps the error stack with some alert display logic, while the **HTMLErrorStack** function returns the stack in displayable HTML. This is used with top-level page errors where there is no guarantee that the error stack rendered is available on the client side.

- **CLEARERRORS**

The **CLEARERRORS** procedure clears the Teamcenter error stack. The syntax is:

```
CLEARERRORS ( )
```

This procedure is useful if there are specific errors you want to handle. If you know a particular ITK function may return an error you can recover from or handle non-fatally, wrap that function in its own **try** block. This usually means that you have nested **try** statements. In the **catch** section of the inner **try** statement, you attempt to handle the error. If successful, call **CLEARERRORS** and continue. If

unsuccessful, use the **throw** statement to jump to your outer **catch** section. Another useful helper function in this case is **throwIfErrorNotArray** (see the **basic.isx** file).

Property error table

If you run the thin client with the **IMAN_SDL_MAGIC** environment variable set, any error generated while accessing an object property dumps a property table into the output page. This table shows the name type and value of each valid property of the relevant object. Do not use this facility in production. The Type Browser is another useful property interrogation tool and can be found in the system administrator Web pages.

TcScript helper functions

Location of TcScript helper functions

Helper functions are defined in the `TC_ROOT\web\htdocs\tc\common\basic.isx` file. Any TcScript file that includes the **basic.isx** file can use these functions.

car

DESCRIPTION

Returns the first element of an array.

SYNTAX

car(*list*)

ARGUMENTS

- *list*
Array.

RETURN VALUE

Returns the first element of the array.

Class

DESCRIPTION

Returns the name of the Teamcenter class of an object.

SYNTAX

Class(*object*)

ARGUMENTS

- *object*
Teamcenter object tag.

RETURN VALUE

Returns the Teamcenter class of your input object.

contains

DESCRIPTION

Checks whether *list* contains *value*.

SYNTAX

contains(*list*, *value*)

ARGUMENTS

- *list*
Array that may have the specified value.
- *value*
Value to look for.

RETURN VALUE

Returns either **true** or **false**.

ErrorStack

DESCRIPTION

Returns the error stack details in XML format.

SYNTAX

ErrorStack(*text*)

ARGUMENTS

- *text*
Contextual text message.

RETURN VALUE

Returns the error stack as XML.

HTMLDefaultHeader

DESCRIPTION

Returns a header appropriate to an HTML document including a localized encoding string. Encoding string comes from the UID string table.

SYNTAX

HTMLDefaultHeader()

ARGUMENTS

None.

RETURN VALUE

Returns an HTML header string.

HTMLErrorStack

Description

Returns XML with the error messages; used for potential top-level page errors.

Syntax

HTMLErrorStack(*text*)

Arguments

- *text*
Contextual text message.

Return value

Returns the error stack as XML.

imanText

DESCRIPTION

Returns a localized Teamcenter string from the UID table.

SYNTAX

imanText(*key*)

ARGUMENTS

- *key*
Teamcenter key string.

RETURN VALUE

Returns the localized Teamcenter string from the UID table.

Message

DESCRIPTION

Returns the message in XML format. The Teamcenter Web renderer knows how to render this XML format into a user message.

SYNTAX

Message(*text*)

ARGUMENTS

- *text*
Message string to be displayed.

RETURN VALUE

Returns an XML-formatted message.

quote

DESCRIPTION

Escapes special XML characters within a string.

SYNTAX

quote(*unquoted*)

ARGUMENTS

- *unquoted*
String to be quoted.

RETURN VALUE

Returns the XML-safe representation of the string.

quoteImanText

DESCRIPTION

Returns a localized Teamcenter string from the UID table encased in quotation marks.

SYNTAX

quoteImanText(*key*)

ARGUMENTS

- *key*
Teamcenter key string.

RETURN VALUE

Returns a localized and quoted Teamcenter string from the UID table.

removeArrayElement

DESCRIPTION

Removes the given element from the array.

SYNTAX

removeArrayElement(*element*, *list*)

ARGUMENTS

- *element*
Element to remove.
- *list*
Array that contains the element.

RETURN VALUE

Returns the modified array.

replaceArrayElement

DESCRIPTION

Replaces the specified element of an array with the new element.

SYNTAX

replaceArrayElement(*oldElement*, *newElement*, *list*)

ARGUMENTS

- *oldElement*
Existing element of the input array.
- *newElement*
New element to replace the existing element.
- *list*
Array with element to be replaced.

RETURN VALUE

Returns the modified array.

replaceChar

DESCRIPTION

Replaces a character in the string with another character.

SYNTAX

replaceChar(*string*, *oldchar*, *newchar*, *front*)

ARGUMENTS

- *string*
Input string.
- *oldchar*
Existing character in the input string.
- *newchar*
New character to replace existing character.
- *front*
Empty string as "".

RETURN VALUE

Returns the modified string.

replaceString

DESCRIPTION

Replaces a substring in the string with another substring.

SYNTAX

replaceString(*string*, *oldstring*, *newstring*)

ARGUMENTS

- *string*
Input string.
- *oldstring*
Existing string in the input string.

- *newstring*
New string to replace existing string.

RETURN VALUE

Returns the modified string.

startsWith

DESCRIPTION

Checks if a string starts with a substring.

SYNTAX

startsWith(*str*, *subStr*)

ARGUMENTS

- *str*
Input string.
- *subStr*
Substring to check.

RETURN VALUE

Returns either **TRUE** or **FALSE**.

throwIfErrorNot

DESCRIPTION

Either rethrows your error or clears it depending on whether or not it is in your *expectedValues* array.

SYNTAX

throwIfErrorNot(*caughtError*, *expectedValue*)

ARGUMENTS

- *caughtError*
Input to the **catch** block.
- *expectedValue*

Value to compare with *caughtError*.

RETURN VALUE

Returns an error if *caughtError* is not the *expectedValue*.

throwIfErrorNotArray

DESCRIPTION

Either rethrows your error or clears it depending on whether or not it is in your *expectedValues* array.

SYNTAX

throwIfErrorNotArray(*caughtError*, *expectedValues*)

ARGUMENTS

- *caughtError*
Input to the **catch** block.
- *expectedValues*
Array of error codes to suppress.

RETURN VALUE

Returns an error if *caughtError* is not in the *expectedValues* array.

tokenize

DESCRIPTION

Divides the input string into tokens using separator.

SYNTAX

tokenize(*input*, *separator*)

ARGUMENTS

- *input*
Input string.
- *separator*
Character to use as a separator.

RETURN VALUE

Returns an array of tokens.

XMLDefaultHeader

DESCRIPTION

Adds a header appropriate to an XML document. The encoding string comes from the UID string table.

SYNTAX

XMLDefaultHeader()

ARGUMENTS

None.

RETURN VALUE

Returns a header appropriate to an XML document including a localized encoding string.

A. Glossary

A

AIF

See **Application Integration Framework (AIF)**.

Application Integration Framework (AIF)

Integration framework that enables developers to build custom interfaces to applications for Teamcenter and NX. This framework provides the foundation through which applications can be launched and executed in a standard manner. It also provides the basic design for applications, the base classes and methods, and a methodology for creating and handling events generated by the user interface. In addition, the AIF provides tools to handle the registration of components, represented by Java beans, and a mechanism for locating and passing messages to those components.

C

class

Set of objects that share the same list of attributes but distinguishable by the value the attributes acquire for specific objects. For example, the Automobile class can be defined by the brand, color, and price, but each car associated to the Automobile class has a different brand, color, and price combination.

class hierarchy

Structure defining subclasses that inherit the attributes of their superclasses, also called their parents or ancestors.

Client

Role played by a software component of a system when it requests particular services be performed on its behalf by another entity, a server. See also *server*.

client tier

Teamcenter architectural tier that comprises the Teamcenter clients, Teamcenter integrations with third-party applications, and the third-party applications associated with the integrations.

corporate server

Host computer at the center of a Teamcenter network. This host contains the Teamcenter application root directory, Teamcenter data directory, licensing, File Management System (FMS), and volumes. For installations that include the web tier (four-tier architecture), the corporate server also contains the Teamcenter server manager. Multiple application clients can map to or mount the corporate server.

F

form

Teamcenter workspace object used to display product information (properties) in a predefined template. Forms are often used to create an electronic facsimile of a hardcopy form in Teamcenter. See also **master form**.

four-tier architecture

Teamcenter architecture that includes four tiers: resource tier, client tier, web tier, and enterprise tier. Contrast with **two-tier architecture**.

M

master form

Teamcenter workspace object used to display product information (properties) in a predefined template. Master forms are used to display product information in a standardized format.

My Teamcenter

In the Teamcenter rich client and thin client, application that is the main access point for managing product information. My Teamcenter provides the functionality for creating objects in the Teamcenter database, querying the database for objects, checking in and checking out objects, and managing tasks. Users can also open objects, automatically launching the related application. Each user has a personal My Teamcenter window that displays product information as graphical objects. Although users share product information across the enterprise, they organize this information individually in personal workspaces.

N

navigation pane

Rich client framework component that displays buttons of the applications available for use in the rich client. Clicking the application button launches the application.

P

preference

Configuration variable stored in a Teamcenter database and read when a Teamcenter session is initiated. Preferences allow administrators and users to configure many aspects of a session, such as user logon names and the columns displayed by default in a properties table.

R

Registry Editor

Teamcenter application that enables editing Teamcenter rich client registry files. This application is used only for editing registry files that are used for internationalization, dynamic class invocation, and configuration in the rich client framework.

registry file

Properties (**.properties**) file that contains the user-defined configuration settings (keys and values) that are relative to how the application displays and performs in the Teamcenter rich client. Each application registered in the rich client has a **.properties** file known as a registry file.

rich client

Java-based user interface to Teamcenter installed on user workstations. The rich client accesses Teamcenter databases using a remote or local server. Compare to **thin client**.

rich client framework

Component of the rich client that integrates and runs various applications from a common platform. These applications can be off-the-shelf applications such as NX CAD/CAM/CAE, Microsoft Office, custom applications, and Java plug-ins.

S

server

System software component that performs a specifically defined set of software services on behalf of one or more clients. In a typical Teamcenter installation, servers are centralized on dedicated hosts that support a large number of clients. Clients are distributed on hosts connected to the servers via various networking techniques. See also **Client**.

T

Teamcenter Application Registry

Independent web-based service that allows a Teamcenter product to look up other available Teamcenter products for launching a linked object. Administrators can register and unregister installed instances of a Teamcenter product in the registry.

thin client

Teamcenter user interface that provides a streamlined browser-based view of product information stored in a Teamcenter database. The thin client is configured in the web tier, which creates and serves its web pages to the client. Compare to **rich client**.

two-tier architecture

Teamcenter architecture that includes a resource tier and a client tier. The resource tier comprises the database server and database. The client tier comprises the Teamcenter rich client, third-party

applications that integrate with the rich client, and a local server. This architecture supports only the Teamcenter rich client. Contrast with **four-tier architecture**.

W

Web Browser

Teamcenter application that provides access to Internet Web pages from within the rich client framework. The web browser is a rich client window that acts as a Web browser, enabling you to navigate and view Web pages within the rich client rather than switching to a separate Web browser. The web browser also provides the ability to access MIME (Multipurpose Internet Mail Extension) file types and to view files created in other applications, such as Microsoft Word and Excel, through the Web browser.

Siemens Industry Software

Headquarters

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 972 987 3000

Americas

Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 314 264 8499

Europe

Stephenson House
Sir William Siemens Square
Frimley, Camberley
Surrey, GU16 8QD
+44 (0) 1276 413200

Asia-Pacific

Suites 4301-4302, 43/F
AIA Kowloon Tower, Landmark East
100 How Ming Street
Kwun Tong, Kowloon
Hong Kong
+852 2230 3308

About Siemens PLM Software

Siemens PLM Software, a business unit of the Siemens Industry Automation Division, is a leading global provider of product lifecycle management (PLM) software and services with 7 million licensed seats and 71,000 customers worldwide.

Headquartered in Plano, Texas, Siemens PLM Software works collaboratively with companies to deliver open solutions that help them turn more ideas into successful products. For more information on Siemens PLM Software products and services, visit www.siemens.com/plm.

© 2019 Siemens Product Lifecycle Management Software Inc. Siemens, the Siemens logo and SIMATIC IT are registered trademarks of Siemens AG. Camstar, D-Cubed, Femap, Fibersim, Geolus, I-deas, JT, NX, Omneo, Parasolid, Solid Edge, Syncrofit, Teamcenter and Tecnomatix are trademarks or registered trademarks of Siemens Product Lifecycle Management Software Inc. or its subsidiaries in the United States and in other countries. All other trademarks, registered trademarks or service marks belong to their respective holders.